A ONE-SEMESTER COURSE

Mathematics in Collaboration

Learning to ask, understand, prove, and investigate with AI

From first mathematical sentences to complete proofs, with a look at current research

Eighteen weeks • Worked proofs • Exercises with complete solutions
Small computational experiments • An introduction to Lean

To the reader

This book starts from ordinary arithmetic. If you can add, subtract, multiply, and divide fractions and whole numbers, including negative numbers and zero, you have enough background to begin. Mathematicians call the whole numbers $\ldots, -2, -1, 0, 1, 2, \ldots$ the *integers*. No university mathematics is assumed: you do not need calculus, linear algebra, or any experience with proofs, and you do not need to know how to program. Chapter 1 reviews the algebra and notation we need and explains what definitions, theorems, and proofs are.

We use the real numbers—all the points of the number line, including numbers such as $\sqrt{2}$ that are not fractions—as a familiar tool. Along with the ordinary rules of arithmetic, such as $a + b = b + a$, we accept four properties of numbers without proof. These are our *starting facts*, and each is stated again where it is first used:

- every nonnegative real number has exactly one nonnegative square root (Chapter 1);
- for every real number, however large, there is a larger integer (the *Archimedean property*, Chapter 2);
- every nonempty set of positive integers has a smallest member (the *well-ordering property*, Chapter 4);
- the real numbers are *complete*: roughly, the number line has no holes. Suppose that for every distance, however small, all the terms of a sequence from some point on lie within that distance of one another. Then the terms approach some point of the line. For example, cutting off $\sqrt{2} = 1.41421\ldots$ after more and more digits gives the fractions 1, 1.4, 1.41, 1.414, ..., and these approach the point $\sqrt{2}$. If the line contained only fractions, there would be a hole at $\sqrt{2}$ (Chapter 11).

Many students approach a mathematics course with the question "How much do I have to memorize before I can start?" This book is organized around a different question: *How can I enter a mathematical conversation, notice what is missing from my understanding, and learn enough to take part responsibly?* An AI assistant can be a useful partner in that conversation. It can suggest examples, explanations, arguments, and code. It can also make mistakes that sound convincing, and it cannot understand the mathematics for you. Throughout the book you will practice three skills that make such a partner useful: asking for exactly the step you are missing, noticing when an answer has quietly changed an assumption, and explaining what an argument really establishes.

Three problems run through the semester. Can a group of people be given different tasks so that each person receives a task they find acceptable? Can a process that repeatedly improves an approximate answer settle down to an exact solution? Can a sequence of plus and minus signs be arranged so that it has almost no resemblance to shifted copies of itself? We begin each one with small examples that you can work out by hand. Followed far enough, these problems lead to Hall's matching theorem (Chapters 8 and 9), the contraction theorem (Chapters 11 and 12), and a question about special matrices that has been open since the 1960s and for which a proof was claimed in

2026 (Chapter 14). A fourth strand, finite probability (Chapter 13), shows how computing an average can prove that an object with a desired property exists, even when the argument does not say how to build it. That chapter then turns one such proof into a step-by-step construction.

Most definitions are introduced as answers to problems, so that you can see why they are worded as they are. Major proofs appear first as plans, then as complete arguments, and finally as arguments you can modify and test, for instance by removing one hypothesis (one of the assumptions a theorem makes) and finding the step that breaks. Along the way the book collects eight *habits*, short pieces of advice about how to make progress, each in a shaded box headed by its number and name. Habit 1, for example, is *choose a representation that exposes the goal*: write the objects you are working with in a form in which the thing you want to prove can be seen, as when you write a number as $2k$, with k an integer, to show that it is even. Habit 7, *make failure informative*, applies when an attempt to build something fails: look for a concrete object that shows why it cannot be built. For instance, three people who between them find only two of the tasks acceptable cannot all receive different tasks. Later chapters, and the guide in Appendix E, return to several of these habits when they are needed again.

How to use a chapter

Each chapter opens with a shaded box headed "This week" that says what the chapter is for. Whenever the text raises a question or points out a pattern, try a small example of your own before you look for the answer, either further on in the text or from an assistant. Then read the explanation slowly. Many sections stop at a question marked **Pause and decide**, followed by a *Check*. Cover the check, commit to an answer, and only then read on. When you read a proof, ask two questions about every line: *Why is this line true?* and *Why is this line here?* The second question asks which later step needs the line. You understand the proof when you can answer both.

Near the end of each chapter there are six exercises, numbered by chapter: Exercise 7.3 is the third exercise of Chapter 7. The first four are *core* exercises, which practice the main ideas directly. The last two are *transfer* exercises, which ask you to use the same ideas in a new setting or with a changed hypothesis. Chapter 9 is the one exception: there only the fifth exercise is a transfer exercise, and the sixth is the midterm portfolio, a larger piece of work described in *Planning the semester*. Complete solutions to all the exercises are in Appendices A–C. They are grouped by week, since the course covers one chapter per week: the section headed "Week 7" contains the solutions for Chapter 7. Look at a solution only after a genuine attempt; the introduction to Appendix A suggests how to use the solutions.

Two further appendices are for reference. Appendix D assumes no programming experience. It explains just enough of the programming language Python to run the book's computer experiments. It also tells you where to find the official installation instructions for Lean, the *proof assistant* used in Chapters 15 and 16, and how to have Lean check the book's Lean examples on your own computer. A proof assistant is a computer program that checks every step of a proof. Appendix E begins with a table of the book's main symbols, each with the place where it is explained, so that you can find your way back when a symbol looks unfamiliar.

Keeping track of evidence

For each important statement you meet, keep two separate records in your notebook. The first says how well you understand it: *understood* (you can explain it in your own words and use it in a new situation), *can follow* (you can check each step but could not yet explain it yourself), or *open* (you cannot yet follow it). Chapter 17, which studies a long proof, adds one more label, *borrowed*, for a result that you use without having studied its proof. The second record says what supports the statement: *proved* (in this book or by you), *cited from another source*, *checked by computer for finitely many cases*, or *not established*. For example, if a computer confirms that $1 + 3 + 5 + \cdots + (2n - 1) = n^2$ for every n from 1 to 1000, the second record says "checked by computer for finitely many cases." Only after the induction proof of Theorem 5.1 in Chapter 5 does it say "proved." The four starting facts need a label of their own. We use them without proof, so record each of them as an *assumption*. More advanced books prove them, starting from a careful construction of the numbers.

A proof that a computer program has checked step by step can still be poorly understood, and a pattern you understand well from experiments can still be unproved. Keeping both records stops these two kinds of confidence from blurring into one.

Where the course ends

The course ends with an independent investigation of your own, called the *capstone*. It is open-ended, so it has no answer key. Instead, Chapter 18 works through a complete model investigation, and Section E.6 gives a *rubric*: a table of the qualities on which your own investigation can be assessed.

By the end of the semester you should be able to rebuild several complete proofs from their main ideas, without having memorized their wording. You should also be able to run and check small mathematical experiments on a computer, carry out a small, carefully limited investigation like the capstone with the help of an AI assistant, and read short proofs written in Lean. Most research papers will still be out of reach, because a specialized proof can require a great deal of further study. What you will have is a method for an unfamiliar proof: find out exactly which background it needs, learn that background, and connect it back to the proof you wanted to understand.

Whether an argument is correct and whether it is new are separate questions. The theorems proved in this book are well-established results, presented so that you can follow and rebuild them. A result you discover for yourself may be correct and still be well known. It becomes a contribution to research only after it has been compared carefully with what is already known and its evidence has been checked.

Planning the semester

The schedule below covers eighteen teaching weeks, one chapter per week. Each week is planned around three hours of class and about four to six hours of reading and exercises on your own. Chapters 1–6 deliberately move slowly, because everything later depends on them. Chapters 8 and 9 form one two-week unit, and so do Chapters 11 and 12: in each pair, the first chapter develops the ideas that the long proof in the second chapter uses, so plan the two weeks together. The last two chapters, 17 and 18, bring together what came before; they introduce new ideas only when one particular proof or investigation needs them.

The table lists the topic of each week and the main piece of work you produce in it. Terms you may not know yet, such as residue classes or the kernel of a map, are explained in the chapters where they appear.

Week	Topic	What you produce
1	First proofs: variables, definitions, even and odd	A precise claim and a fully explained proof about even and odd numbers.
2	Statements, quantifiers, and negation	A statement written in symbols, negated, and tested on examples.
3	Sets and maps	A proof about composed functions and a counterexample to a converse.
4	How a proof finds its next step	A proof recorded step by step: the known facts, the current goal, and the reason for each step.
5	Induction and invariants	An induction proof about a recursively defined sequence and a proof that an algorithm stops.
6	Equivalence and symmetry	An operation on residue classes, with a proof that it does not depend on the representatives chosen.
7	Graphs, counting, and extremal choice	A proof by counting in two ways and a shortest-path argument.
8	Assignments and augmenting paths	An assignment algorithm explained by what it preserves and why it stops.
9	Hall's theorem and proof reconstruction	A complete proof of Hall's theorem and the midterm portfolio.
10	Vectors, linearity, and representation	A kernel test for injectivity and a proof of the Cauchy–Schwarz inequality.
11	Distance, limits, and completeness	A convergence argument with explicit tolerances.
12	The contraction theorem	A full proof with a numerical error certificate.
13	Averaging and finite probability	An existence proof by averaging and a step-by-step construction.
14	Sign sequences and research experiments	Exact correlation computations and a proof that a circulant Hadamard matrix can have only certain sizes.
15	Lean: statements and proof states	Small proofs rebuilt in a formal language.
16	Lean: definitions, trust, and repair	A formal statement checked against its intended meaning, and a list of the axioms its proof uses.

Week	Topic	What you produce
17	Reading an unfamiliar proof	A map of the background a proof needs, and a reading record for a research proof.
18	A small investigation from start to finish	A checkable write-up of the model investigation and a plan for your own.

Read each chapter in several sittings, breaking at its section headings. For a long proof, a single sitting might cover only the statement and one of its smaller arguments. When you come back, start by saying what has already been established and what remains to be done. The worked examples are part of the main path through each chapter, so do not skip them. If an example is already familiar, work through it anyway, then change one of its assumptions and see what happens. In a taught course, the four core exercises of each chapter form the required weekly work, and the transfer exercises provide extra practice once the core exercises feel secure. Week 9 is the exception: its sixth exercise, Exercise 9.6, is the midterm portfolio, and it is required.

A weekly rhythm

If the class meets three times a week, each week can follow a simple pattern. In the first meeting, set out the question the chapter is about and work out one small example together. In the second, reconstruct the main proof and compare two possible next moves at a difficult step. In the third, examine a proposed argument—one written by a classmate, taken from the text, or produced by an AI assistant—and decide exactly what it proves and where it might fail. If you are working on your own, follow a similar sequence: read the "This week" box and try a small example yourself, read the chapter while answering its "Pause and decide" questions, rebuild the main proof with the book closed, work the core exercises, and finish with a transfer exercise.

Assessment that rewards understanding

One reasonable weighting gives 40% to the weekly core exercises, 25% to the midterm portfolio in Week 9, and 35% to the final investigation. If AI assistance is allowed, each submission should say openly how it was used. Besides its mathematics, every submission should contain three things: a short explanation of its central step, written without help; one boundary case that you have tested, such as zero, the empty set, or the smallest allowed input; and a statement of anything borrowed from elsewhere or not yet checked. The grade should reflect the mathematics and the understanding you show, not the length of a conversation log or the polish of generated prose.

The midterm portfolio, Exercise 9.6, is about the assignment problem of Chapters 8 and 9, in which a *full assignment* gives every student a different acceptable task. It has four parts:

- a full assignment for the graph of Exercise 9.2, presented so that a reader can check it;
- a graph with no full assignment, together with a group of students who between them accept fewer tasks than there are students in the group, which shows that no full assignment exists;

- a complete proof, in your own words, of Hall's theorem, which says that a full assignment exists whenever every group of students, taken together, accepts at least as many tasks as the group has members;
- a paragraph explaining why handing out tasks one at a time, without ever taking one back, can get stuck even when a full assignment exists.

Exercise 9.6 states each part precisely, and its solution in Appendix B is a model portfolio.

The final investigation is the independent capstone of Section 18.7. That section lists what the write-up should contain, from a precise statement of the question to an explanation of what is new. It also describes the short oral discussion that ends the investigation: you explain one delicate step, change an assumption, and predict what fails. Section E.6 suggests a grading rubric.

Contents

To the reader **i**

Planning the semester **iv**

1 A first mathematical conversation **1**

1.1 What does a pattern promise? . . . 1
1.2 Letters, brackets, and the equals sign . . . 1
1.3 The small algebra moves we will use . . . 3
1.4 A definition that makes a proof possible . . . 4
1.5 Definition, example, claim, theorem, proof . . . 5
1.6 Reading and writing a first proof . . . 6
1.7 Keeping track of what depends on what . . . 8
1.8 Sums, number systems, inequalities, and square roots . . . 9

2 Statements, choices, and assumptions **12**

2.1 An implication is a conditional promise . . . 12
2.2 And, or, and not . . . 14
2.3 The order of choices changes the meaning . . . 16
2.4 Negation: describing how a statement fails . . . 19
2.5 Contraposition and vacuous truth . . . 21
2.6 Absolute value, squares, and intervals . . . 22

3 Objects, maps, and information **27**

3.1 Sets express the boundary of a question . . . 27
3.2 A function includes its input and output sets . . . 30
3.3 Injective, surjective, and reversible . . . 32
3.4 Images and preimages answer different questions . . . 35

4 How a proof finds its next step **38**

4.1 Working backward and working forward . . . 38
4.2 The contrapositive and proof by contradiction . . . 40
4.3 The square root of two is irrational . . . 42
4.4 Existence is not uniqueness . . . 44

4.5 Reading a long proof as a sequence of smaller promises 45

5 Induction, invariants, and progress **48**

5.1 From a few sums to every sum 48
5.2 Why induction is not circular, and when strong induction helps 50
5.3 A sequence defined by a recurrence 52
5.4 An invariant shows what cannot change 54
5.5 Euclid's algorithm: why it is right and why it stops 55

6 Sameness, symmetry, and structure **59**

6.1 Which differences matter? 59
6.2 Congruence and residue classes 61
6.3 Well-definedness: does the answer depend on the representative? 64
6.4 Structure-preserving maps 65
6.5 What symmetry permits us to assume 68
6.6 Order is a different structure 70

7 Graphs: counting, paths, and extremal choices **73**

7.1 A graph is a pattern of connections 73
7.2 Double counting and the handshake theorem 75
7.3 The pigeonhole principle and the average 77
7.4 Walks, paths, and shortest objects 78
7.5 Directed edges and a guaranteed sink 80
7.6 Tournaments and a vertex of maximum outdegree 82
7.7 Second out-neighborhoods and a research problem 83

8 Assignments, repairs, and obstructions **87**

8.1 Why one acceptable task per person is not enough 87
8.2 Maximal does not mean maximum 89
8.3 An alternating repair 90
8.4 Searching for an augmenting path 93
8.5 A failed search exposes a shortage 95
8.6 The whole procedure on a small example 98

9 Hall's theorem: a complete proof apprenticeship **101**

9.1 A condition that checks every group of students 101
9.2 First prove the necessary direction 102

9.3 A constructive proof from the search procedure . . . 103
9.4 A second proof: equality tells us where to split . . . 104
9.5 Why every part of the case split is needed . . . 108
9.6 Mathematical taste from equality cases . . . 110

10 Linearity: changing the language of a problem 112
10.1 Vectors are ordered measurements . . . 112
10.2 A linear map is determined by its building blocks . . . 115
10.3 Kernel: the information lost at zero . . . 117
10.4 The geometry of an inner product . . . 119
10.5 Deriving a bound by removing a component . . . 121
10.6 Matrix multiplication records composition . . . 124

11 Distance, error, and the meaning of a limit 128
11.1 Distance begins with absolute value . . . 128
11.2 A metric keeps only the properties we use . . . 129
11.3 Convergence promises eventual control . . . 132
11.4 Limits are unique . . . 134
11.5 Cauchy sequences do not need a known destination . . . 135
11.6 Completeness excludes missing limits . . . 136
11.7 A geometric bound controls an entire tail . . . 139

12 The contraction theorem: an unfamiliar proof made readable 143
12.1 From solving an equation to repeating a map . . . 143
12.2 The theorem and its dependency map . . . 145
12.3 The proof, one step at a time . . . 148
12.4 What the error estimates are for . . . 150
12.5 A quadratic example, worked by hand . . . 151
12.6 Remove the hypotheses one at a time . . . 153

13 Averaging, randomness, and existence 157
13.1 A finite probability model . . . 157
13.2 Expectation is a weighted average . . . 160
13.3 Indicators convert events into quantities . . . 163
13.4 Every graph has a large cut . . . 164
13.5 Turning the average argument into a construction . . . 166
13.6 A bound on rare large values . . . 168

14 Sign sequences: from experiments to a research boundary **171**
14.1 A sequence with small echoes . . . 171
14.2 Periodic correlation is a different quantity . . . 173
14.3 Circulant matrices turn correlations into dot products . . . 175
14.4 The sum of all correlations reveals a square . . . 178
14.5 Orthogonality also forces divisibility by four . . . 179
14.6 An exact finite experiment . . . 182
14.7 What current research adds . . . 183

15 Lean I: reading the state of a proof **186**
15.1 A checker is not the same as an explanation . . . 186
15.2 Reading Lean's basic notation . . . 187
15.3 An implication as a change of goal . . . 190
15.4 Conjunction means two obligations . . . 191
15.5 Composition of injective functions in Lean . . . 192
15.6 Existence statements, and the type of a number . . . 195
15.7 Three ways to use an equation . . . 196

16 Lean II: definitions, dependencies, and trust **198**
16.1 The statement must be right before the proof can help . . . 198
16.2 A complete existence proof in Lean . . . 199
16.3 Sets as membership predicates . . . 203
16.4 Encoding the exclusions in a second out-neighborhood . . . 205
16.5 A successful proof can still be irrelevant . . . 207
16.6 Placeholders and axiom dependencies . . . 208
16.7 A debugging sequence that does not lose the mathematics . . . 210

17 Reading an unfamiliar proof one piece at a time **214**
17.1 A new theorem on a familiar foundation . . . 214
17.2 Capsule one: small regions that cover the space . . . 215
17.3 Capsule two: keeping infinitely many indices . . . 217
17.4 Capsule three: subsequences, and a plan for the proof . . . 218
17.5 The complete proof . . . 219
17.6 Why neither hypothesis can be dropped . . . 222
17.7 The prerequisite map after the work is done . . . 223
17.8 Reading a formalized research proof with the same method . . . 226

18 A complete AI-assisted investigation **230**
18.1 The question that starts the project . . . 230
18.2 Testing the first guess . . . 231
18.3 The one-step estimate . . . 232
18.4 Can the constant be improved? . . . 237
18.5 A numerical experiment with exact arithmetic . . . 238
18.6 The finished investigation as a short paper . . . 241
18.7 Your independent capstone . . . 242

A Solutions for Chapters 1–6 **245**
A.1 Week 1 . . . 245
A.2 Week 2 . . . 247
A.3 Week 3 . . . 250
A.4 Week 4 . . . 252
A.5 Week 5 . . . 254
A.6 Week 6 . . . 257

B Solutions for Chapters 7–12 **260**
B.1 Week 7 . . . 260
B.2 Week 8 . . . 263
B.3 Week 9 . . . 266
B.4 Week 10 . . . 270
B.5 Week 11 . . . 272
B.6 Week 12 . . . 275

C Solutions for Chapters 13–18 **279**
C.1 Week 13 . . . 279
C.2 Week 14 . . . 281
C.3 Week 15 . . . 284
C.4 Week 16 . . . 286
C.5 Week 17 . . . 289
C.6 Week 18 . . . 292

D Running the examples on a computer **297**
D.1 The smallest amount of Python needed . . . 297
D.2 An exact fraction is not a rounded decimal . . . 300
D.3 Three experiments . . . 302

D.4 Checking the Lean examples 307

E A compact guide to notation and mathematical judgment 310

E.1 A notation map, with places to return to 310

E.2 Structures, what proofs use, and changes of description 314

E.3 Choosing the next move 316

E.4 Turning a reading difficulty into a precise question 318

E.5 What a finished reading record contains 319

E.6 A suggested capstone rubric 320

E.7 What you can do at the end of the course 321

References and further reading 323

CHAPTER 1

A first mathematical conversation

This week

Turn a familiar numerical pattern into a precise statement, and prove it. Review the algebra and notation needed to state such a pattern exactly, and learn how definitions, examples, claims, theorems, and proofs differ. Begin reading an AI assistant's arguments critically: check which numbers an argument really covers, and look for any step it has skipped.

1.1 What does a pattern promise?

Suppose two people each have an odd number of tokens. If they put their tokens together, must the total be even? Try a few cases: $3 + 5 = 8$, $7 + 9 = 16$, and $1 + 1 = 2$. Every total is even. But there are infinitely many pairs of odd numbers, and no list of examples can include them all. The examples tell us what happened for the pairs we tried. To know what must happen for every pair, including the ones nobody has tried, we need a *proof*: an argument that explains why the total is always even.

To turn the pattern into mathematics, we first say exactly which numbers it is about. An *integer* is a whole number that may be positive, negative, or zero: $\ldots, -2, -1, 0, 1, 2, \ldots$, where the dots mean that the list continues forever in both directions. The collection of all integers is written $\mathbb{Z}$. Now the pattern becomes a precise statement:

For every pair of odd integers, their sum is even.

This statement says more than the token story. Nobody holds -3 tokens, but -3 is an odd integer, so the statement also covers sums such as $-3 + 5 = 2$ and $-7 + (-1) = -8$. Once negative numbers are allowed, the words "odd" and "even" need exact meanings too; Section 1.4 supplies them.

So when you bring a claim like this to a classmate or to an AI assistant, the first question to settle is not "Can you prove this?" It is "Which numbers does the claim talk about, and what exactly does it say about them?" Suppose an assistant quietly replaces "integers" by "positive integers" and then gives a correct argument. It has answered a different question. The new statement happens to be true, but its proof says nothing about a sum such as $-3 + 5$.

1.2 Letters, brackets, and the equals sign

Exact meanings and proofs are written in the language of algebra, so we begin by reviewing that language. A letter such as n can stand for an integer whose value we have not fixed. A letter used in this way is called a *variable*. Writing a number directly in front of a letter means multiplication:

$2n$ means $2 \times n$. Two letters written side by side are multiplied too, so ab means $a \times b$. A raised dot also means multiplication, so $2 \cdot 4$ is the same as 2×4. Between two numerals the dot cannot be left out, because 24 already means twenty-four.

The quantities being multiplied in a product are called its *factors*, and the quantities being added in a sum are called its *terms*. For example, the product $2n$ has the factors 2 and n, and the sum $2n + 1$ has the terms $2n$ and 1. The expression $n + 1$ is one more than n, and n^2 means $n \times n$. Parentheses, also called brackets, group an expression: $2(n + 1)$ is twice the entire quantity $n + 1$. It equals $2n + 2$, not $2n + 1$, because the 1 inside the parentheses is doubled too. At $n = 5$, for instance, $2(5 + 1) = 12$, while $2 \cdot 5 + 1 = 11$.

An equals sign says that the two expressions on either side of it have the same value. It does not mean "and now calculate the next thing." For example,

$$2(3 + 1) = 2 \cdot 4 = 8$$

is a correct chain, because all three expressions have the value 8. By contrast, the chain $2 + 3 = 5 \cdot 2 = 10$ is wrong. Its first expression, $2 + 3$, has the value 5, while its middle expression, $5 \cdot 2$, has the value 10, and 5 is not equal to 10. The writer meant two separate steps: first $2 + 3 = 5$, then $5 \cdot 2 = 10$. When the values differ, write two separate equations or two sentences.

We will use the familiar rules of arithmetic freely. Addition and multiplication can each be done in either order and grouped in either way, and multiplication distributes over addition:

$$a + b = b + a, \qquad (a + b) + c = a + (b + c),$$
$$ab = ba, \qquad (ab)c = a(bc), \qquad a(b + c) = ab + ac.$$

The last rule is called the *distributive rule*. It also works with the sum written first, because multiplication can be done in either order: $(a + b)c = c(a + b) = ca + cb = ac + bc$. These rules hold for all integers, and more generally for all the real numbers described in Section 1.8.

The same integer can be written in many ways. For example, $12 = 3 \cdot 4 = 2 \cdot 6$. The choice can matter: the form $2 \cdot 6$ shows at a glance that 12 is twice an integer, while the form $3 \cdot 4$ does not display a factor 2. The proof in Section 1.6 turns this observation into a method.

A fraction a/b means a divided by b, and it is defined only when $b \neq 0$. (The symbol $\neq$ means "is not equal to.") If two expressions are equal, then adding the same number to both, subtracting the same number from both, or multiplying both by the same number gives two expressions that are still equal. Dividing both by the same number also gives equal expressions, as long as that number is not zero, because division by zero is not defined. So before dividing both sides of an equation by an expression such as x, whose value is unknown, we must deal separately with the case in which that expression is zero. Forgetting this case is one of the most common ways to lose solutions of an equation, as the next question shows.

Pause and decide. An assistant starts from the equation $x^2 = x$, divides both sides by x, and concludes that $x = 1$. Has it found every solution?

Check. No. Dividing by x is allowed only when $x \neq 0$, so the argument says nothing about $x = 0$. In fact $x = 0$ satisfies the original equation, since $0^2 = 0$. A complete argument treats two cases. If

$x = 0$, the equation holds. If $x \neq 0$, dividing both sides by x gives $x = 1$, and indeed $1^2 = 1$. So the solutions are exactly 0 and 1.

1.3 The small algebra moves we will use

An expression such as $2k + 1$ is built from operations you already know. To evaluate it at $k = 3$, replace every occurrence of k by 3: the expression $2k + 1$ becomes $2 \cdot 3 + 1 = 7$. This replacement is called *substitution*. Substituting $k = 3$ does not claim that k always equals three. It only computes the value of the expression for that one input.

Sometimes we give a rule its own name. Writing $f(n) = n + 1$ means that the rule named f adds one to whatever input n it is given; for example, $f(3) = 3 + 1 = 4$. Read $f(n)$ as "the output of f at the input n," not as f multiplied by n. Such a rule is called a *function*, and Chapter 3 studies functions in detail. For now the notation simply lets us refer to a formula and its values without writing the formula out each time.

To rewrite an expression without changing its value, we may use any arithmetic rule from Section 1.2, reading it in either direction. Reading the distributive rule from left to right gives $2(k + 1) = 2k + 2$; this is called *expanding* or *multiplying out*. Reading the same equality from right to left gives $2k + 2 = 2(k + 1)$; this is called *factoring*. Factoring is often useful in a proof because it brings a common factor, here 2, out into the open.

An equality that holds for every allowed value of its letters is called an *identity*. An identity is different from an equation to be solved. For example, $2(k + 1) = 2k + 2$ is an identity, because the distributive rule makes the two sides equal for every integer k. The equation $2k + 1 = 7$, on the other hand, asks which values of k make it true. Subtracting one from both sides gives $2k = 6$, and dividing both sides by two gives $k = 3$. Substituting $k = 3$ into the original equation confirms the answer: $2 \cdot 3 + 1 = 7$.

For a longer example of multiplying out, take the product $(2k + 1)(2\ell + 1)$ of two brackets. The letter ℓ is a curly lowercase L, read "ell"; the curl keeps it from being mistaken for the digit 1. We use two different letters, k and ℓ, so that they are free to stand for different numbers; Section 1.7 explains why that matters. We multiply out one bracket at a time:

$$\begin{aligned}(2k + 1)(2\ell + 1) &= 2k(2\ell + 1) + 1 \cdot (2\ell + 1)\\ &= (4k\ell + 2k) + (2\ell + 1)\\ &= 4k\ell + 2k + 2\ell + 1.\end{aligned}$$

The first line uses the rule $(a + b)c = ac + bc$ with $a = 2k$, $b = 1$, and $c = 2\ell + 1$: each term of the first bracket is multiplied by a copy of the second bracket. The second line multiplies out each copy with the rule $a(b + c) = ab + ac$. The first copy becomes $2k \cdot 2\ell + 2k \cdot 1$. Here $2k \cdot 2\ell = (2 \cdot 2)(k\ell) = 4k\ell$ because factors may be reordered and regrouped, and $2k \cdot 1 = 2k$. In the second copy, $1 \cdot (2\ell + 1) = 2\ell + 1$. The third line removes the brackets, which is allowed because addition may be regrouped.

A *power* records repeated multiplication: $2^3 = 2 \cdot 2 \cdot 2 = 8$. The *exponent* 3 counts how many equal factors are multiplied. It does not count additions: $2 + 2 + 2 = 6$, which is $2 \cdot 3$, not 2^3. In

general, if n is a positive integer, then a^n is the product of n factors, each equal to a; the number a is called the *base*. In particular, $a^1 = a$.

Two rules for powers follow directly from counting factors. If we write a row of two factors of 2 next to a row of three factors of 2, we get a row of five:

$$2^2 \cdot 2^3 = (2 \cdot 2)(2 \cdot 2 \cdot 2) = 2^5.$$

In the same way, n factors of a followed by m more factors of a make $n + m$ factors of a. So $a^n a^m = a^{n+m}$ for every number a and all positive integers n and m. Division cancels factors instead:

$$\frac{2^5}{2^2} = \frac{2 \cdot 2 \cdot 2 \cdot 2 \cdot 2}{2 \cdot 2} = 2 \cdot 2 \cdot 2 = 2^3.$$

In general, if $n > m$, the m factors of a below the line cancel m of the n factors above it, and $n - m$ factors remain. Cancelling a factor a means dividing by a, so it requires $a \neq 0$. Thus $a^n/a^m = a^{n-m}$ when n and m are positive integers with $n > m$ and $a \neq 0$.

We now extend the notation to the exponent zero and to negative exponents, in the only way that keeps these two rules working. Dividing a^n by itself gives 1, while the subtraction rule would give $a^{n-n} = a^0$. So we agree that $a^0 = 1$ whenever $a \neq 0$. Next, $1/a^n$ is the same as a^0/a^n, which the subtraction rule would turn into $a^{0-n} = a^{-n}$. So we agree that $a^{-n} = 1/a^n$ whenever n is a positive integer and $a \neq 0$. For example, $2^{-3} = 1/2^3 = 1/8$. With these agreements, the rules $a^n a^m = a^{n+m}$ and $a^n/a^m = a^{n-m}$ hold for all integers n and m, positive, negative, or zero, as long as $a \neq 0$. Checking this takes some bookkeeping: one sorts the possible signs of n and m into cases and counts factors in each case. We skip that check and use both rules freely. For instance, $2^3 \cdot 2^{-1} = 8 \cdot \frac{1}{2} = 4 = 2^2$, exactly as the rule $a^n a^m = a^{n+m}$ predicts, since $3 + (-1) = 2$. Powers such as 2^{-n} appear regularly from Chapter 5 onward.

Pause and decide. Why is $(a + b)^2$ not in general equal to $a^2 + b^2$?

Check. The square means $(a+b)(a+b)$. Multiplying out one bracket at a time gives $a^2+ab+ba+b^2$, and since $ba = ab$ this is $a^2 + 2ab + b^2$. The extra term $2ab$ is not zero in general. For $a = 1$ and $b = 2$, we get $(1 + 2)^2 = 9$ but $1^2 + 2^2 = 5$; the difference 4 is exactly $2ab = 2 \cdot 1 \cdot 2$.

1.4 A definition that makes a proof possible

To prove anything about odd and even numbers, we need to know exactly what those words mean. A *definition* gives an exact meaning to a word or symbol. It is an agreement about how the word will be used from now on, not a discovery about numbers. What we need is a meaning precise enough to decide, for any integer at all, whether it qualifies.

Mathematics books set important definitions apart under a numbered heading, so that they are easy to find and to refer to later. In this book, definitions, examples, and proved results (theorems, propositions, and lemmas, whose roles Section 1.5 explains) share a single numbering within each chapter. The first part of each number is the chapter, and the second part counts the numbered items within it. Definition 1.1 below is the first numbered item of this chapter, and Proposition 1.2

in Section 1.6 is the second. Exercises are numbered separately, so Exercise 1.1 at the end of the chapter is a different item from Definition 1.1. Here is our agreement about even and odd integers.

Definition 1.1 (Even and odd integers)**.** An integer a is *even* if $a = 2k$ for some integer k. It is *odd* if $a = 2k + 1$ for some integer k.

For example, 8 is even because $8 = 2 \cdot 4$, and 7 is odd because $7 = 2 \cdot 3 + 1$. Negative integers are included: -3 is odd because $-3 = 2 \cdot (-2) + 1$. Zero is even because $0 = 2 \cdot 0$. Whether an integer is even or odd is called its *parity*.

You may wonder whether every integer is even or odd, and whether some integer could be both. In fact every integer is exactly one of the two, but that is a separate fact that needs its own argument. Lemma 4.1 in Chapter 4 proves it. This chapter does not need it.

The words "for some integer k" mean that at least one integer k makes the equation true. The requirement that k be an *integer* is essential. If k were allowed to be any number, then every integer a would have the form $2k$, simply by choosing $k = a/2$; for instance, $7 = 2 \cdot 3.5$. The definition does not ask whether a number can be divided by two. It asks whether the result of dividing by two is an integer, and 3.5 is not. The same requirement does the work in the definition of odd: the equation $8 = 2 \cdot 3.5 + 1$ has the right shape, but it does not make 8 odd, because 3.5 is not an integer.

When a definition asks for *some* integer k with a certain property, a particular integer that has the property is called a *witness*. For the odd integer 7, the witness is $k = 3$, because $7 = 2 \cdot 3 + 1$. The witness is not 7 itself: it is the extra integer that makes the equation $7 = 2k + 1$ true. For -3, the witness is $k = -2$. Each odd integer has only one witness, and different odd integers have different witnesses: Section 1.7 shows that the witness for an odd integer a is always $(a - 1)/2$.

A definition can be used in two directions. Suppose we are told that a is odd. Then we may introduce an integer k with $a = 2k + 1$ and use that equation in our reasoning; this direction *unpacks* the information. Conversely, suppose we want to show that a given integer a is odd. Then it is enough to name an integer k and check that $a = 2k + 1$; this direction *recognizes* that the definition is satisfied. Our first proof uses both directions: it unpacks the information that two numbers are odd, and it recognizes that their sum meets the definition of even.

1.5 Definition, example, claim, theorem, proof

Mathematical sentences do different jobs, and a reader who mixes up the jobs can easily misjudge what has been shown. Before we prove our first result, it helps to tell five of these jobs apart.

A definition sets a meaning. Definition 1.1 is a rule for using the words *even* and *odd*, so there is nothing to prove before we adopt it. Checking that a particular number meets the rule may still take some work.

An example shows the meaning in one instance. The equation $7 = 2 \cdot 3 + 1$ shows the definition of odd at work for the number 7. A *nonexample* is an object that does not meet the definition. For instance, 4 is not odd: if $4 = 2k + 1$, then subtracting one gives $2k = 3$ and dividing by two gives $k = 3/2$, which is not an integer. So no integer witness exists. Examples and nonexamples together show where the boundary of a definition lies.

A claim asserts something that needs justification. The sentence "Adding two odd integers always gives an even integer" is not part of any definition. To establish it, we must show that whenever two integers are each one more than twice an integer, their sum is twice an integer.

A theorem is a claim that has been proved. Books use several headings for proved results. The heading *theorem* is usually kept for the results the author most wants to emphasize. A *proposition* is a proved result of more moderate importance, a *lemma* is proved mainly to help prove something else, and a *corollary* follows quickly from an earlier result. These words describe the role a result plays in the book; all of them are held to the same standard of proof, and none allows steps to be skipped. A *conjecture*, by contrast, is a statement that has been proposed but not yet proved.

The word *proposition* has a second meaning, used in logic and in the proof assistant Lean (Chapter 15): any statement that is either true or false, whether or not anyone has proved it. In that sense "$2 + 2 = 5$" is a proposition, a false one. Context makes clear which meaning is intended.

A proof supplies the justification. It starts from what we are given and from facts already accepted, makes deductions that can each be justified, and arrives at exactly the promised conclusion. A proof does not ask the reader to trust the author; it lets the reader see for themselves why no allowed case has been left out. The small square □ at the end of a proof marks where the argument ends.

A *hypothesis*, also called an assumption or premise, is a condition under which a result is asserted. In Proposition 1.2, which we prove in the next section, the hypotheses are that the two numbers are integers and that both are odd. The *conclusion* is that their sum is even. A proof may use the hypotheses at any point. It may not use the conclusion, because the conclusion is exactly what it is supposed to establish.

Some facts are accepted without proof as the starting rules of a theory; these are called *axioms*. In this course we accept two kinds of facts in this way: the familiar rules of arithmetic, such as $a + b = b + a$, and four further *starting facts* about numbers, which are listed in *To the reader*. The first starting fact, about square roots, appears in Section 1.8. A claim does not become a starting fact just because it is hard to prove. If an argument relies on a claim that has not been proved and is not one of the facts we have agreed to accept, the argument has a gap.

Pause and decide. Classify these three sentences. (1) "Even means twice an integer." (2) "Six is even because $6 = 2 \cdot 3$." (3) "The sum of two odd integers is even."

Check. Sentence (1) is a definition. Sentence (2) is an example: it checks the definition for one number and names the witness 3. Sentence (3) is a general claim about every pair of odd integers. It becomes an established result once it has been proved, and that happens in the next section as Proposition 1.2.

1.6 Reading and writing a first proof

Proofs use a few small words in a precise way. A proof often begins with "let" or "take." The sentence "Let a and b be odd integers" means: give the names a and b to two odd integers that could be *any* odd integers at all. It does not mean "choose your two favorite examples." Since we do not know which odd integers a and b are, the proof may use only the fact that they are odd integers. That is exactly why the finished argument works whichever odd integers are supplied.

The word "some" does a different job. Once an odd integer a has been given, the definition tells us that $a = 2k + 1$ for *some* integer k. We do not need to know the numerical value of k; it is enough to know that such an integer exists and to give it a name. The phrase "by the definition" tells the reader where that information came from. Words such as "hence" and "therefore" announce a deduction, and the reader should always be able to see the reason that justifies it.

Read the next result in three passes. First, identify what is assumed and what is promised. Second, read the plan to see why a particular way of writing the numbers is chosen. Third, follow the proof line by line, checking the arithmetic and making sure that the last step really matches the definition of even. The plan describes how one might discover the proof. The proof is the argument that actually establishes the claim.

Proposition 1.2. *The sum of any two odd integers is even.*

Plan. Write each odd integer in the form given by the definition of odd, so that its oddness is visible. Then rearrange the sum until it has the form required by the definition of even: twice an integer.

Proof. Let a and b be odd integers. By the definition of odd (Definition 1.1), there are integers k and ℓ with $a = 2k + 1$ and $b = 2\ell + 1$. The integers k and ℓ may be equal or different; the argument does not depend on which. Substituting these expressions gives

$$\begin{aligned} a + b &= (2k + 1) + (2\ell + 1) && \text{substitute for } a \text{ and } b \\ &= 2k + 2\ell + 2 && \text{reorder and regroup the terms, add } 1 + 1 \\ &= 2(k + \ell + 1) && \text{factor out 2.} \end{aligned}$$

The number $k + \ell + 1$ is an integer, because a sum of integers is an integer. Hence $a + b$ is twice an integer. By the definition of even, $a + b$ is even. □

The final sentence matters. It connects the expression we computed to the property we promised. A proof should not end simply because the algebra looks tidy. It ends when the required conclusion has been reached.

To see how one general argument covers many particular cases, take the two odd integers from Section 1.4: $a = 7$ and $b = -3$. Their witnesses are $k = 3$ and $\ell = -2$. The proof builds the witness $k + \ell + 1 = 3 + (-2) + 1 = 2$ for the evenness of the sum, and indeed $7 + (-3) = 4 = 2 \cdot 2$. This numerical check is not a second proof. It lets us see what each symbol in the general proof was doing.

Notice what the proposition does *not* say. It does not say that every sum is even. Its conclusion is promised only when *both* numbers are odd. If one of them is even, for example in $2 + 3 = 5$, the proposition makes no promise at all. Keeping the hypotheses attached to a result is as important as remembering the result itself.

Habit 1: choose a representation that exposes the goal

Before calculating, ask what form the answer must take. Then choose a way of writing the objects, a *representation*, in which that form can appear. If the goal is to show that a number is even, try to display a factor of 2. The same habit will return in later chapters with other goals. To show that every person in a group can be given a different acceptable task, write down who receives which task. To show that an approximation is close to the true value, write down the difference between them. When an assistant proposes a representation, ask why it helps before letting it continue the calculation.

1.7 Keeping track of what depends on what

Look again at the proof of Proposition 1.2. The two odd integers a and b can be any odd integers. The witnesses k and ℓ are not chosen freely: k is determined by a, and ℓ is determined by b. Indeed, solving $a = 2k + 1$ for k gives $k = (a - 1)/2$, and in the same way $\ell = (b - 1)/2$. For $a = 7$ this gives $k = 3$. If a changes to 9 while b stays the same, then k changes to 4, but ℓ does not change. The witness for the evenness of the sum, $k + \ell + 1$, is in turn built from k and ℓ. Keeping track of which quantity depends on which is a genuine part of the argument.

Suppose an assistant writes both odd integers as $2k + 1$, using the same k for each. Then it computes $(2k + 1) + (2k + 1) = 2(2k + 1)$, which is certainly even. The algebra is correct, but the argument covers only the case in which the two odd integers are equal, as in $5 + 5$ or $(-3) + (-3)$. It says nothing about $3 + 5$, because no single integer k makes both $3 = 2k + 1$ and $5 = 2k + 1$ true. A useful question to ask of any argument is: "Which quantities can be anything, and which are determined by the others?" Chapter 2 makes this question precise with the phrases "for every" and "there exists."

Separate witnesses matter just as much with three numbers. Let us show that the sum of three odd integers is odd. Let a, b, and c be odd integers. By the definition of odd, there are integers k, ℓ, and m with $a = 2k + 1$, $b = 2\ell + 1$, and $c = 2m + 1$. We use three different letters because the three odd integers may be different. Adding gives

$$a + b + c = (2k + 1) + (2\ell + 1) + (2m + 1) = 2k + 2\ell + 2m + 3.$$

At this point it is tempting to say that the expression "looks odd," but that is not yet what the definition asks for. The definition asks for the form *twice an integer, plus one*. So split 3 as $2 + 1$ and factor 2 out of all the terms that contain it:

$$2k + 2\ell + 2m + 3 = 2k + 2\ell + 2m + 2 + 1 = 2(k + \ell + m + 1) + 1.$$

The number $k + \ell + m + 1$ is an integer, because a sum of integers is an integer. So $a + b + c$ has exactly the form required by Definition 1.1, and $a + b + c$ is odd. The arithmetic was easy. As Habit 1 recommends, the real work was recognizing which form the definition demands and steering the expression into it.

1.8 Sums, number systems, inequalities, and square roots

The rest of this chapter introduces notation that later chapters use constantly: long sums, the standard collections of numbers, inequalities, and square roots. To name the numbers in a list, we often use one letter with a small label written below and to the right of it, called a *subscript*: a_1, a_2, a_3, a_4 are the first, second, third, and fourth numbers of a list. A subscript is a label, not a multiplication: a_2 is the second number in the list, whereas $2a$ is twice a. The notation

$$\sum_{j=1}^{4} a_j = a_1 + a_2 + a_3 + a_4$$

means: add the numbers named a_1, a_2, a_3, and a_4. The large symbol $\sum$ is the Greek capital letter sigma, an S for "sum." The letter j is a temporary *index*, a counter that takes the values $1, 2, 3, 4$ in turn: it starts at the value written below the sigma and stops at the value written above it.

Suppose the numbers are $a_1 = 1$, $a_2 = 3$, $a_3 = 5$, and $a_4 = 7$. Then the sum is $1 + 3 + 5 + 7 = 16$. Renaming the index does not change the sum: $\sum_{r=1}^{4} a_r$ is also 16, because the index is only a counter. Changing the number above the sigma from 4 to 3 does change it: $\sum_{j=1}^{3} a_j = 1 + 3 + 5 = 9$.

Notice also that the totals $1 + 3 + 5 = 9 = 3^2$ and $1 + 3 + 5 + 7 = 16 = 4^2$ are *perfect squares*, that is, squares of integers. The shorter totals fit the same pattern: $1 = 1^2$ and $1 + 3 = 4 = 2^2$. Is the sum of the first n odd positive integers always n^2? This is exactly the situation of Section 1.1: the examples suggest a pattern, but only a proof can settle it. Theorem 5.1 in Chapter 5 supplies that proof.

The quantity being added can be any expression involving the index. To evaluate such a sum, substitute each value of the index in turn and add the results. With the same numbers as before,

$$\sum_{j=1}^{3} (a_j + 5) = (a_1 + 5) + (a_2 + 5) + (a_3 + 5) = 6 + 8 + 10 = 24.$$

The index itself may also appear in the expression:

$$\sum_{j=1}^{3} j^2 = 1^2 + 2^2 + 3^2 = 1 + 4 + 9 = 14.$$

Several collections of numbers have standard names. A *set* is a collection of objects, called its *members*; Chapter 3 studies sets carefully. Curly braces are used to list the members of a set.

- $\mathbb{N} = \{0, 1, 2, 3, \ldots\}$ is the set of *natural numbers*. The dots mean that the displayed pattern continues forever. In this book zero counts as a natural number; when we want only the positive integers $1, 2, 3, \ldots$, we will say so explicitly.
- $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ is the set of integers from Section 1.1.
- $\mathbb{Q}$ is the set of *rational numbers*: numbers that can be written as a fraction a/b in which a and b are integers and $b \neq 0$. Examples are $3/4$, $-5/2$, and $7 = 7/1$.

- $\mathbb{R}$ is the set of *real numbers*: all the numbers on the number line. Every rational number is a real number, but the number line also contains numbers that are not fractions of integers. For example, by Pythagoras' theorem the diagonal of a square with sides of length 1 has a length whose square is $1^2 + 1^2 = 2$. That length is $\sqrt{2}$, the positive number whose square is 2. (Square roots are discussed at the end of this section.) Theorem 4.3 in Chapter 4 proves that $\sqrt{2}$ cannot be written as a fraction of integers.

Each of these sets contains the one before it: every natural number is an integer, every integer is a rational number, and every rational number is a real number. When a letter is said to be real, every point of the number line is an allowed value, including negative numbers, zero, and numbers that are not fractions.

To compare real numbers we use four inequality signs. The statement $x < y$ means that x is strictly smaller than y, and $x \leq y$ means that x is smaller than or equal to y, that is, at most y. Likewise, $x > y$ means that x is strictly larger than y, and $x \geq y$ means that x is at least y. For example, $3 \leq 3$ is true, but $3 < 3$ is false. The signs $<$ and $>$ give *strict* inequalities, while $\leq$ and $\geq$ give *non-strict* ones. The two kinds disagree only when the two sides are equal, and that boundary case is where many mistakes in proofs occur. A chain such as $0 \leq x \leq 1$ states two conditions at once: $0 \leq x$ and $x \leq 1$.

A number x is *positive* if $x > 0$ and *negative* if $x < 0$. It is *nonnegative* if $x \geq 0$. So zero is nonnegative, but it is neither positive nor negative.

Finally, we need square roots. A number whose square is t is called a *square root* of t. We accept without proof the familiar fact that every nonnegative real number t has exactly one nonnegative square root; this is the first of the starting facts listed in *To the reader*. That root is written $\sqrt{t}$. For example, $\sqrt{9} = 3$: both 3 and -3 have square 9, and the square-root sign always picks the nonnegative one. A negative number has no real square root at all, because the square of a real number is never negative.

Because the square-root sign always gives a nonnegative answer, $\sqrt{x^2}$ is not always equal to x. At $x = -3$, for instance, $\sqrt{(-3)^2} = \sqrt{9} = 3$. In general, $\sqrt{x^2} = x$ when $x \geq 0$, and $\sqrt{x^2} = -x$ when $x < 0$; in the second case $-x$ is the right answer because it is nonnegative and $(-x)^2 = x^2$. In both cases $\sqrt{x^2}$ is the distance from x to 0 on the number line. Section 2.6 calls this distance the *absolute value* of x and writes it $|x|$.

Exercises

Exercises 1.1–1.4 are core exercises, and Exercises 1.5 and 1.6 are transfer exercises. Complete solutions are in Appendix A, under "Week 1."

Exercise 1.1. Evaluate $2(x + 3)$ and $2x + 3$ at $x = 4$. Explain in words why the two answers differ.

Exercise 1.2. Prove that the sum of an even integer and an odd integer is odd. Name the integer that serves as the witness for the oddness of the sum.

Exercise 1.3. An assistant argues that the product of two odd integers is always odd, because $(2k + 1)^2$ is odd for every integer k. Explain which products of odd integers this argument fails to

cover, and then give a complete proof that the product of any two odd integers is odd.

Exercise 1.4. Let $a_1 = 2$, $a_2 = -1$, and $a_3 = 4$. Write out the sum $\sum_{j=1}^{3}(2a_j + 1)$ term by term, and then evaluate it.

Exercise 1.5. Find every real solution of the equation $x(x - 2) = 0$. Then explain why dividing both sides by x does not, on its own, give a complete answer. You may use the fact that a product of two real numbers is zero only when at least one of the two factors is zero.

Exercise 1.6. Someone wants to know whether a formula $f(n)$ is zero for every positive integer n. They check $n = 1, 2, \ldots, 10$, find $f(n) = 0$ each time, and conclude that $f(n) = 0$ always. Show that this reasoning is unsafe: construct a formula $f(n)$ with $f(n) = 0$ for $n = 1, 2, \ldots, 10$ but $f(11) \neq 0$, and compute $f(11)$. Hint: use a product with one factor for each checked value of n.

Before moving on

Close the book and explain, in your own words, why the sum of two odd integers is even. You do not need the exact wording of the proof, but you should be able to say what the witnesses k and ℓ are, why they are given different letters, and why the last step is a return to the definition of even. You should also be able to say how a definition, a checked example, and a claim about every allowed input differ from one another. These distinctions stay just as important later in the book, when the objects become much less familiar than odd and even numbers.

CHAPTER 2

Statements, choices, and assumptions

This week

Read exactly what a mathematical statement promises and what it leaves open. Keep track of which choices may depend on which inputs. Negate statements correctly, use counterexamples to test claims, and work with absolute values and squares without losing cases.

2.1 An implication is a conditional promise

In mathematics, a *statement* is a sentence that is either true or false. "Seven is odd" is a true statement, and "$2 + 2 = 5$" is a false one. A sentence such as "n is even" is not yet true or false, because it depends on the value of n; it becomes a statement as soon as a value of n is fixed. A question such as "Is 15 odd?" or an instruction such as "Add 3 to both sides" is not a statement at all.

Many mathematical statements have the form "if . . . , then . . . ." Our first example of such a statement needs one new word. An integer n is *divisible by* an integer d when $n = dk$ for some integer k. For example, 20 is divisible by four, because $20 = 4 \cdot 5$ and 5 is an integer. On the other hand, 6 is not divisible by four: the equation $6 = 4k$ forces $k = 6/4 = 3/2$, which is not an integer. As with even and odd numbers in Section 1.4, the number k is called a witness, and the requirement that it be an integer is essential.

Now consider the statement

If an integer is divisible by four, then it is even.

It has two parts. The part after "if" is the *hypothesis*, the condition that is assumed. The part after "then" is the *conclusion*, what is promised once the hypothesis holds. For an integer n, let P stand for the statement "n is divisible by four" and let Q stand for the statement "n is even." A statement of the form "if P, then Q" is called an *implication*. We write it $P \Rightarrow Q$ and read it "P implies Q." Like any statement, an implication can be true or false. Our sentence makes its promise for every integer n at once; Section 2.3 introduces a symbol for "for every."

The promise is easy to prove. Let n be an integer that is divisible by four. Then $n = 4k$ for some integer k, and so $n = 2(2k)$. The number $2k$ is an integer, so n is twice an integer. By the definition of even (Definition 1.1), n is even.

Notice what the implication does *not* say. It makes no promise about an integer that is not divisible by four. Such an integer may be even, like 6, or odd, like 7, and both possibilities are consistent with the statement. A common reading error is to run the implication backward, as if it also said that every even integer is divisible by four.

That backward statement has its own name. The *converse* of $P \Rightarrow Q$ is $Q \Rightarrow P$. Here the converse says "if an integer is even, then it is divisible by four," and it is false: 6 is even but, as we saw, not divisible by four. A statement and its converse are different promises, and one of them can be true while the other is false.

A particular input that shows a statement about every input to be false is called a *counterexample*. For an implication, a counterexample is an input for which the hypothesis is true and the conclusion is false. The integer 6 is a counterexample to the converse above: it satisfies the converse's hypothesis, because it is even, and it violates the converse's conclusion, because it is not divisible by four. One counterexample is enough. The converse claims to hold for *every* integer, and a single failure shows that this claim is wrong.

So when you test an implication, only inputs that satisfy the hypothesis can reveal a failure. An input such as 7, about which "divisible by four implies even" promises nothing, can never be a counterexample to it. Keep this in mind when you ask an assistant to test a claim. Inputs chosen at random often miss the hypothesis and so test nothing. Ask instead for inputs that satisfy the hypothesis, and then check whether each one satisfies the conclusion.

Sometimes an implication and its converse are both true. The statement $P \Leftrightarrow Q$, read "P if and only if Q," promises both $P \Rightarrow Q$ and $Q \Rightarrow P$. The phrase splits into two halves. "P if Q" means $Q \Rightarrow P$. "P only if Q" says that P can hold only when Q holds as well; in other words, whenever P holds, so does Q, and that is $P \Rightarrow Q$. For example, "an integer is divisible by four only if it is even" is another way of saying "divisible by four implies even." When $P \Leftrightarrow Q$ holds, we say that P and Q are *equivalent*. To prove an "if and only if" statement, we prove the two implications separately.

For example, for every integer n,

$$n \text{ is even if and only if } n^2 \text{ is even.}$$

One direction is a short calculation. If $n = 2k$ for an integer k, then $n^2 = 4k^2 = 2(2k^2)$, which is twice an integer. The other direction is harder, because knowing that $n^2 = 2w$ for some integer w gives no direct way to write n itself as twice an integer. Section 2.5 describes the idea that gets around this, and Lemma 4.2 in Chapter 4 carries out the proof.

A result applies only after its hypothesis is checked

To use "divisible by four implies even" for the number 20, we first check the hypothesis: $20 = 4 \cdot 5$, and 5 is an integer. Only then does the result tell us that 20 is even. Two ingredients are needed: the implication itself, proved once for all integers, and evidence that this particular number satisfies its hypothesis. Quoting the implication does not supply the evidence.

Here is what goes wrong when the check is skipped. Suppose an assistant writes: "30 is even, because every integer divisible by four is even." The conclusion happens to be true, but the reason is not valid. The number 30 is not divisible by four, since $30 = 4k$ would force $k = 15/2$, so the result says nothing about 30. A correct reason is simply $30 = 2 \cdot 15$.

Necessary and sufficient, stronger and weaker

Two words describe the roles of P and Q in a true implication $P \Rightarrow Q$. We say that P is a *sufficient* condition for Q: knowing P is enough to guarantee Q. We also say that Q is a *necessary* condition for P: P cannot hold unless Q holds too. So divisibility by four is sufficient for evenness, and evenness is necessary for divisibility by four. The converse, "even implies divisible by four," fails, as 6 shows, so evenness is not sufficient for divisibility by four. When $P \Leftrightarrow Q$ holds, P is both necessary and sufficient for Q.

When every object satisfying P also satisfies Q, but not the other way around, we call P the *stronger* condition and Q the *weaker* one. The stronger condition demands more, so it is harder to satisfy. Divisibility by four is stronger than evenness, because divisibility by four implies evenness but, as we just saw, not conversely. The same words are used for whole statements. A stronger statement promises more than a weaker one; for instance, a statement about every integer is stronger than the same statement about positive integers only.

Pause and decide. Consider the claim "If an integer is divisible by six, then it is divisible by three." For each of 12, 9, and 7, decide whether it could be a counterexample. Is the claim true? Is its converse true?

Check. A counterexample would have to be divisible by six and not divisible by three. The number 12 is divisible by six, and it is also divisible by three, since $12 = 3 \cdot 4$, so it agrees with the claim. Neither 9 nor 7 is divisible by six, so the claim promises nothing about them, and neither can be a counterexample. The claim is true: if $n = 6k$ for an integer k, then $n = 3(2k)$, and $2k$ is an integer. Its converse, "if an integer is divisible by three, then it is divisible by six," is false, and 9 is a counterexample: $9 = 3 \cdot 3$ is divisible by three, but $9 = 6k$ would force $k = 3/2$, which is not an integer.

2.2 And, or, and not

Statements can be combined into longer statements. Throughout this section, n is an integer, P is the statement "n is even," and Q is the statement "n is divisible by three." The letters P and Q stand for whole statements, not for numbers. Each of them becomes true or false once the value of n is known.

The statement $P \wedge Q$, read "P and Q," is true when both P and Q are true. It is called the *conjunction* of P and Q. The statement $P \vee Q$, read "P or Q," is true when at least one of P and Q is true. It is called the *disjunction* of P and Q. "At least one" includes the case in which both are true: 6 is even and also divisible by three, so for $n = 6$ the statement $P \vee Q$ is true. In everyday speech, "tea or coffee" often means one but not both. In mathematics, "or" allows both unless the author explicitly says otherwise. Finally, $\neg P$, read "not P," is the *negation* of P. It is true exactly when P is false.

The table below works out these combinations for four integers. Each integer represents one of the four ways in which P and Q can be true (T) or false (F).

n	P	Q	$P \wedge Q$	$P \vee Q$	$\neg(P \vee Q)$	$(\neg P) \wedge (\neg Q)$
6	T	T	T	T	F	F
4	T	F	F	T	F	F
3	F	T	F	T	F	F
5	F	F	F	F	T	T

Read the table one row at a time. For $n = 4$, the statement P is true and Q is false, so $P \vee Q$ is true but $P \wedge Q$ is false. For $n = 5$, both P and Q are false, so $P \wedge Q$ and $P \vee Q$ are both false. The last two columns use the same rules on the results: $\neg(P \vee Q)$ is the opposite of the $P \vee Q$ column, and $(\neg P) \wedge (\neg Q)$ is true only in a row where P and Q are both false.

These two columns agree in every row:

$$\neg(P \vee Q) \quad \Longleftrightarrow \quad (\neg P) \wedge (\neg Q).$$

In words: an integer fails to be "even or divisible by three" exactly when it is not even *and* not divisible by three. To show that 5 lacks this property, it is not enough to show that 5 is not even; we must also show that 5 is not divisible by three.

By contrast, an integer fails to be "even and divisible by three" as soon as one of the two requirements fails. The number 4 fails because it is not divisible by three, and the number 3 fails because it is not even. In symbols,

$$\neg(P \wedge Q) \quad \Longleftrightarrow \quad (\neg P) \vee (\neg Q).$$

These two rules are called *De Morgan's laws*. The long arrow $\Longleftrightarrow$ is the same "if and only if" as $\Leftrightarrow$ in Section 2.1: the statements on its two sides are true in exactly the same situations.

The table proves the first law for *any* two statements P and Q, not only for these four integers. Its last two columns were filled in using only the truth values of P and Q in the same row. Each of P and Q is either true or false, so there are only four combinations, and the table has a row for each of them. The second law can be checked in the same way. Its left side, $\neg(P \wedge Q)$, is false only in the row where P and Q are both true. Its right side, $(\neg P) \vee (\neg Q)$, is false only in that row too, because there neither $\neg P$ nor $\neg Q$ is true. In the other three rows both sides are true.

The same kind of bookkeeping describes how an implication fails. Here P and Q may be any two statements, not only the two fixed at the start of this section. The implication $P \Rightarrow Q$ fails exactly when its hypothesis holds and its conclusion fails. Therefore

$$\neg(P \Rightarrow Q) \quad \Longleftrightarrow \quad P \wedge (\neg Q).$$

This is precisely the description of a counterexample from Section 2.1: an input that satisfies the hypothesis and violates the conclusion. In the other three situations the implication counts as true. The table lists all four:

P	Q	$P \Rightarrow Q$
T	T	T
T	F	F
F	T	T
F	F	T

The last two rows surprise many people. Think of an everyday promise: "if it rains, I will bring an umbrella." On a dry day the promise is not broken, whether I bring an umbrella or not. In the same way, the implication "if n is divisible by four, then n is even" is not broken by $n = 6$, which is even, or by $n = 7$, which is odd. Neither number is divisible by four, so the promise does not apply to them, and at both values the implication is true. Section 2.5 looks at what this means when the hypothesis can never be true.

Parentheses and grouping

Parentheses show which statements belong together, just as they group numbers in arithmetic, and moving them can change the meaning. Compare $\neg(P \wedge Q)$ with $(\neg P) \wedge Q$, where P and Q are again the statements "n is even" and "n is divisible by three." For $n = 5$, the first is true, because 5 is not both even and divisible by three. The second is false, because it requires 5 to be divisible by three. By convention, the symbol $\neg$ applies only to the statement immediately after it, so $\neg P \wedge Q$ means $(\neg P) \wedge Q$. In this book we usually write such parentheses anyway, to make the grouping easy to see.

2.3 The order of choices changes the meaning

Many statements talk about all the members of a collection, or about at least one of them. A *universal* statement, such as "every integer divisible by four is even," claims that every allowed input has a property. There are infinitely many integers, so a proof cannot check them one at a time. Instead, as in the proof of Proposition 1.2, it works with an input about which we know nothing except that it is allowed. Such an input is called *arbitrary*. Since the argument uses nothing special about it, the same argument applies to each input. An *existence* statement, such as "some integer is both even and divisible by three," claims that at least one allowed input has the property. A single example proves it, and 6 will do. The statement does not claim that its example is the only one, and indeed 12 works too. Mathematics has symbols for "every" and "some," and when several of them appear in one statement, the order in which they appear matters.

The symbol $\forall$ means "for every," and the symbol $\exists$ means "there exists" or "there is at least one." These two symbols are called *quantifiers*, because they say for how many values of a letter a condition must hold: for all of them, or for at least one. We write $x \in \mathbb{R}$, read "x in $\mathbb{R}$," to say that x belongs to the set $\mathbb{R}$ of real numbers, that is, that x is a real number. The set that a quantified letter ranges over is called its *domain*. In "$\forall x \in \mathbb{R}$" the domain of x is $\mathbb{R}$.

Compare the two statements

$$\forall x \in \mathbb{R}\ \exists y \in \mathbb{R}: \quad y > x, \tag{A}$$

$$\exists y \in \mathbb{R}\ \forall x \in \mathbb{R}: \quad y > x. \tag{B}$$

Read aloud, (A) says "for every real number x, there is a real number y such that $y > x$." Statement (B) says "there is a real number y such that, for every real number x, $y > x$." The colon separates the quantifiers from the condition that the chosen values must satisfy. The two lines contain the same symbols in a different order, and they make very different promises.

Statement (A) is true. Someone supplies a real number x, and we may then respond with a number y that suits it. Taking $y = x + 1$ works, since $x + 1 > x$. Our response depends on the input: for $x = 3$ we answer 4, and for $x = -10$ we answer -9.

Statement (B) is false. It asks us to choose a single number y *first*, and that one number must be larger than every real number. Whatever y we choose, the input $x = y + 1$ defeats it, because $y > y + 1$ is false. So no choice of y works.

It helps to read a quantified statement as a game between two players: a skeptic, who doubts the statement, and a prover, who defends it. The quantifiers are read from left to right, and each one is a move. At a $\forall$, the skeptic picks a value, which may be any member of the domain. At an $\exists$, the prover picks a value. At the end, the prover wins if the condition after the colon holds for the values that were picked. The statement is true exactly when the prover has a way to win however the skeptic plays. In (A) the prover answers after seeing x; in (B) the prover must commit to y first.

The general rule is that a value chosen at an $\exists$ may depend on every value chosen before it, but not on anything chosen after it. This is the dependence we met in Section 1.7, where the witnesses k and ℓ were determined by the odd integers a and b. Swapping two neighboring quantifiers of the same kind, such as the two in $\forall x \in \mathbb{R}\ \forall y \in \mathbb{R}$, does not change the meaning, as long as the domain of neither letter refers to the other. (In "for every integer N and every integer $n \geq N$," the domain of n depends on N, so N must come first.) Swapping a $\forall$ with an $\exists$ can change it completely, as (A) and (B) show. When a proof that chooses several quantities becomes confusing, one of the most useful questions to ask is: *at what stage is this quantity chosen, and which other quantities may it depend on?*

Tolerances and the inequality rules they need

In later chapters we often need one number to be close to another, within some allowed error. A *tolerance* is a positive amount of error that we are prepared to allow. It is traditionally written ε, the Greek letter epsilon, but the letter is just a name. A tolerance might be $1/10$, or $1/1000$, or any other positive real number, and a statement that begins "for every positive ε" must hold for all of these values. In symbols we write this as $\forall \varepsilon > 0$, a short form of "for every real number ε with $\varepsilon > 0$."

Working with tolerances means working with inequalities between fractions, and there the sign of a number matters. Three rules cover almost everything we need.

- Adding the same number to both sides keeps the direction of an inequality: from $2 < 5$ we get $2 + 7 < 5 + 7$.
- Multiplying or dividing both sides by the same *positive* number keeps the direction: multiplying $2 < 5$ by 3 gives $6 < 15$.
- Multiplying or dividing both sides by the same *negative* number reverses the direction: multiplying $2 < 5$ by -1 gives $-2 > -5$.

The same three rules hold with $\leq$ in place of $<$.

The *reciprocal* of a nonzero number a is $1/a$; for instance, the reciprocal of 4 is $1/4$, and the

reciprocal of $1/4$ is 4. The second rule tells us what taking reciprocals does to an inequality between positive numbers. Suppose $0 < a < b$. The product ab is positive, so we may divide both sides of $a < b$ by ab without reversing the inequality:

$$\frac{a}{ab} < \frac{b}{ab}, \qquad \text{that is,} \qquad \frac{1}{b} < \frac{1}{a}.$$

So for positive numbers, taking reciprocals reverses the order: from $2 < 5$ we get $1/5 < 1/2$. The same argument with $\leq$ shows that $0 < a \leq b$ implies $1/b \leq 1/a$. The rule can fail when the two numbers are not both positive. For instance, $-1 < 2$, but $1/(-1) = -1$ is smaller than $1/2$, not larger.

Here is a typical question about a tolerance: for which positive integers N is $1/N < 1/100$? The rule we just proved, that taking reciprocals of positive numbers reverses the order, answers it in both directions. If $N > 100$, then applying the rule to $0 < 100 < N$ gives $1/N < 1/100$. Conversely, if $1/N < 1/100$, then applying the rule to the positive numbers $1/N < 1/100$ gives $100 < N$, because the reciprocal of $1/N$ is N. So $1/N < 1/100$ holds exactly when $N > 100$. Thus $N = 101$ works, while $N = 100$ gives $1/N = 1/100$, and then the strict inequality fails. For a general tolerance ε, we will want a positive integer N larger than $1/\varepsilon$. Such an integer always exists, because of the following property of the real numbers. We accept it without proof; it is the second of the starting facts listed in *To the reader*.

The Archimedean property: for every real number r, there is an integer N with $N > r$.

In other words, the integers go on forever along the number line: beyond any real number, however large, there is an integer. When r is positive, any integer $N > r$ is positive as well, since $N > r > 0$.

Example 2.1. Consider the statement "for every positive real number ε, there is a positive integer N with $1/N < \varepsilon$." Here is a proof. Let ε be a positive real number. By the Archimedean property, there is an integer N with $N > 1/\varepsilon$. This N is positive, because $1/\varepsilon > 0$ (divide both sides of $1 > 0$ by the positive number ε). Both $1/\varepsilon$ and N are positive, so taking reciprocals reverses the inequality $1/\varepsilon < N$:

$$\frac{1}{N} < \frac{1}{1/\varepsilon} = \varepsilon.$$

The order of the quantifiers allows N to depend on ε, and it has to: a smaller tolerance may need a larger N. For instance, if $\varepsilon = 0.003$, then $1/\varepsilon = 333.33\ldots$, so $N = 334$ is a suitable choice, and indeed $1/334 = 0.002994\ldots$ is less than 0.003. The smaller choice $N = 333$ would not do, because $1/333 = 0.003003\ldots$ is larger than 0.003.

Pause and decide. Can a single positive integer N satisfy $1/N < \varepsilon$ for every positive ε?

Check. No. Once N is fixed, the skeptic may choose $\varepsilon = 1/(2N)$, which is positive. Then $1/N = 2\varepsilon$, and $2\varepsilon > \varepsilon$, so $1/N < \varepsilon$ fails. The claim in this question has the shape of (B): it asks for one N chosen before ε. The true statement in Example 2.1 has the shape of (A): N is chosen after ε is known.

2.4 Negation: describing how a statement fails

To use quantifiers well, we must be able to negate statements that contain them. We write $P(x)$ for a statement about x that becomes true or false once a value of x is chosen. For example, if $P(x)$ is the statement "$x^2 > 4$," then $P(3)$ is the true statement $9 > 4$, and $P(1)$ is the false statement $1 > 4$. The rules below work in the same way whatever the domain of x is, so in stating them we leave out the domain and the colon and write simply $\forall x\ P(x)$ and $\exists x\ P(x)$.

A universal statement $\forall x\ P(x)$ is false exactly when at least one x fails to satisfy $P(x)$, and such an x is a counterexample. An existence statement $\exists x\ P(x)$ is false exactly when no x satisfies $P(x)$, that is, when every x fails. In symbols,

$$\neg(\forall x\ P(x)) \iff \exists x\ \neg P(x), \qquad \neg(\exists x\ P(x)) \iff \forall x\ \neg P(x).$$

For example, let x range over the real numbers and let $P(x)$ be "$x^2 > 4$" again. The universal statement $\forall x\ P(x)$ is false, because $x = 1$ fails: $1^2 > 4$ is false. So $x = 1$ is a counterexample to $\forall x\ P(x)$, and at the same time a witness for its negation $\exists x\ \neg P(x)$. The existence statement $\exists x\ P(x)$ is true, with witness $x = 3$. So its negation $\forall x\ \neg P(x)$, which claims that no real number has a square larger than 4, is false. In both negations the domain stays the same: the negation of a statement about every real number is a statement about some real number, not about things that are not real numbers.

To negate a statement with several quantifiers, apply these rules one quantifier at a time, from left to right. Here is the negation of statement (A) from Section 2.3, worked out step by step:

$$\begin{aligned}\neg\big(\forall x \in \mathbb{R}\ \exists y \in \mathbb{R} :\ y > x\big) &\iff \exists x \in \mathbb{R}\ \neg\big(\exists y \in \mathbb{R} :\ y > x\big)\\ &\iff \exists x \in \mathbb{R}\ \forall y \in \mathbb{R} :\ \neg(y > x)\\ &\iff \exists x \in \mathbb{R}\ \forall y \in \mathbb{R} :\ y \le x.\end{aligned}$$

The first step uses the rule $\neg(\forall x\ P(x)) \Leftrightarrow \exists x\ \neg P(x)$, where $P(x)$ is the whole statement "$\exists y \in \mathbb{R} :\ y > x$." This is a statement about x, since it becomes true or false once x is chosen. So the negation moves past $\forall x$ and turns it into $\exists x$. The second step uses the other rule in the same way, with x already fixed and "$y > x$" as the statement about y, so the negation moves past $\exists y$ and turns it into $\forall y$. The third step negates the final condition. Overall, each $\forall$ became an $\exists$ and each $\exists$ became a $\forall$, the quantifiers kept their order and their domains, and the condition after the colon was negated. The result claims that some real number x is at least as large as every real number y. Such an x would be an *upper bound* for all the real numbers. This negation is false, as it must be, since (A) is true: for any proposed x, the number $y = x + 1$ is larger.

Notice that $y > x$ became $y \le x$, not $y < x$. For two real numbers, exactly one of $y < x$, $y = x$, and $y > x$ is true. So when $y > x$ is false, two possibilities remain, $y < x$ and $y = x$, and $y \le x$ says exactly that. The failure of a strict inequality includes the case of equality.

Pause and decide. Negate the statement "Every real number x satisfies $x^2 \ge x$." Which is true, the statement or its negation?

Check. The negation is "There is a real number x with $x^2 < x$." The word "every" became "there is," the domain is still the real numbers, and $\ge$ became $<$. The negation is true, and $x = 1/2$ is

a witness: $(1/2)^2 = 1/4$, which is less than $1/2$. So $x = 1/2$ is a counterexample to the original statement, which is therefore false.

One acceptable task each, or different tasks for all?

The same reading applies outside arithmetic. Suppose a class has some students and a list of tasks, and for each student and each task, the student either finds the task acceptable or does not. Consider the statement

For every student, there is an acceptable task.

Its negation is "There is a student for whom no task is acceptable." With the quantifiers kept visible, this reads: "there is a student such that, for every task, that task is not acceptable to that student."

Now compare the original statement with a stronger one: "All the students can be given different tasks, each acceptable to the student who receives it." In the first statement, each student's task is chosen after looking at that student alone, so two students may be offered the same task. The second statement asks for a single assignment, chosen once, that works for all the students at the same time. This is the contrast between (A) and (B) again: the first statement has the form "for every student there is a task," while the second begins "there is an assignment such that, for every student, ..." The first statement can be true while the second is false: what can be done for each student separately cannot always be done for all of them together. Exercise 2.5 asks you to build an example. Chapters 8 and 9 work out exactly when such an assignment exists. The answer, Hall's theorem, is one of the main results of this book.

A negation worked out slowly

Here is a longer example. We negate the statement

For every real number x, there is an integer m that is divisible by three and satisfies $m > x$.

Start with a single input. Fix a real number x. A proposed response m succeeds only if it meets *both* requirements: m is divisible by three, and $m > x$. By De Morgan's law $\neg(P \wedge Q) \Leftrightarrow (\neg P) \vee (\neg Q)$, a response fails when at least one requirement fails: m is not divisible by three, or $m \leq x$. For the whole statement to fail, there must be at least one input x for which *every* integer m fails. So the negation is

There is a real number x such that every integer m fails at least one requirement: m is not divisible by three, or $m \leq x$.

In symbols, the original statement is

$$\forall x \in \mathbb{R}\ \exists m \in \mathbb{Z}: \quad (m \text{ is divisible by } 3) \wedge (m > x),$$

and its negation is

$$\exists x \in \mathbb{R}\ \forall m \in \mathbb{Z}: \quad \neg(m \text{ is divisible by } 3) \vee (m \leq x).$$

Negating a statement describes what its failure would look like. It does not tell us whether failure actually happens. Here the original statement is true, so its negation is false. To see this, let x be a real number. The Archimedean property gives an integer k with $k > x/3$. Multiplying both sides by the positive number 3 gives $3k > x$, so $m = 3k$ is divisible by three and larger than x.

It usually helps to describe the failure in plain words first and to write the symbols afterwards. The same habit lets you check someone else's negation. If an assistant produces a negation by mechanically flipping symbols, say in your own words what would make the original statement fail, and compare that description with what the assistant wrote. A sentence can look correct and still describe the wrong kind of failure.

2.5 Contraposition and vacuous truth

The *contrapositive* of $P \Rightarrow Q$ is $\neg Q \Rightarrow \neg P$. In words: if the conclusion fails, then the hypothesis must have failed too. An implication and its contrapositive are *logically equivalent*: each one is true exactly when the other is, whatever the statements P and Q are. The word "logically" signals that this follows from the form of the two statements alone, so the following argument works for any P and Q.

- Suppose $P \Rightarrow Q$ is true, and suppose Q fails. If P held, the implication would force Q to hold, which it does not. So P fails. This proves $\neg Q \Rightarrow \neg P$.
- Suppose $\neg Q \Rightarrow \neg P$ is true, and suppose P holds. If Q failed, the contrapositive would force P to fail, which it does not. So Q holds. This proves $P \Rightarrow Q$.

The second argument concludes that Q holds because the possibility that Q fails has been ruled out. That step uses the principle that every statement is either true or false, so ruling out one possibility leaves only the other. Ordinary mathematics accepts this principle, and we use it freely.

Pause and decide. Use the rule $\neg(P \Rightarrow Q) \Leftrightarrow P \wedge (\neg Q)$ from Section 2.2 to describe exactly when the contrapositive $\neg Q \Rightarrow \neg P$ is false. Compare this with the situation in which $P \Rightarrow Q$ is false.

Check. The contrapositive has hypothesis $\neg Q$ and conclusion $\neg P$. So it is false exactly when $\neg Q$ is true and $\neg P$ is false, that is, when Q is false and P is true. This is the same situation in which $P \Rightarrow Q$ is false. Two statements that fail in exactly the same situations are true in exactly the same situations, so they are equivalent.

Do not confuse the contrapositive with the converse. The statement "divisible by four implies even" is equivalent to its contrapositive, "not even implies not divisible by four." For example, 7 is not even, and the contrapositive correctly tells us that 7 is not divisible by four. The statement is *not* equivalent to its converse, "even implies divisible by four," which fails at $n = 6$.

The contrapositive is often the easier version to prove. Section 4.2 proves "if n^2 is even, then n is even" by proving its contrapositive, "if n is not even, then n^2 is not even." The hypothesis of the contrapositive, "n is not even," is about n itself rather than about n^2. That section also proves that an integer which is not even must be odd (Lemma 4.1), so the new hypothesis lets us write $n = 2k + 1$, a form we can calculate with.

Vacuous truth

What happens when the hypothesis of an implication can never hold? Consider the statement

Every integer that is both even and odd is larger than 100.

Written as an implication, it says: for every integer n, if n is both even and odd, then $n > 100$. A counterexample would have to be an integer that is both even and odd and is at most 100. But no integer is both even and odd. Indeed, if $n = 2k$ and $n = 2\ell + 1$ for integers k and ℓ, then $2k = 2\ell + 1$, so $2(k - \ell) = 1$ and $k - \ell = 1/2$, which is impossible because $k - \ell$ is an integer and $1/2$ is not. (Section 4.2 returns to this argument as a first example of proof by contradiction.) Since no integer is both even and odd, there is no counterexample, and the statement counts as true. This agrees with the rule of Section 2.2: an implication fails only when its hypothesis holds and its conclusion fails, and here the hypothesis never holds.

A statement of this kind is called *vacuously true*: it is true because there is nothing for it to be about. It does not tell us that an integer which is both even and odd exists. Nor does it tell us anything about integers larger than 100. The statement "every integer that is both even and odd is smaller than 0" is vacuously true as well, for the same reason.

A theorem whose hypotheses can never all hold at once is vacuously true in the same way. Its proof may be perfectly correct and still tell us nothing about the situation we cared about. Suppose an assistant proves a result about "every real number x with $x^2 < 0$." The square of a real number is never negative, so no such x exists, and the result is vacuously true however impressive its conclusion looks. Before relying on a theorem, check that at least one object satisfies all of its hypotheses. Vacuous truth appears again in Chapter 3, where it explains why the empty set, the set with no members, is contained in every set.

2.6 Absolute value, squares, and intervals

This section puts the logic of the chapter to work on real numbers. Inequalities that involve squares are a common place to lose half of an answer, and the words "and" and "or" are what keep track of both halves.

The *absolute value* $|x|$ of a real number x is its distance from zero on the number line. Thus $|3| = 3$, $|-3| = 3$, and $|0| = 0$. As a formula,

$$|x| = \begin{cases} x & \text{if } x \geq 0, \\ -x & \text{if } x < 0. \end{cases}$$

The large brace lists two cases: use the first line when $x \geq 0$ and the second line when $x < 0$. In the second case the result is positive, because the negative of a negative number is positive; for instance, $|-3| = -(-3) = 3$. So $|x| \geq 0$ for every real number x. The same two cases appeared at the end of Section 1.8, where we found that $\sqrt{x^2}$ is x when $x \geq 0$ and $-x$ when $x < 0$. So $\sqrt{x^2} = |x|$ for every real number x.

Absolute values behave well under multiplication: for all real numbers c and x,

$$|cx| = |c|\,|x|.$$

To check this rule, we go through the possible signs of c and x.

- If $c = 0$ or $x = 0$, then $cx = 0$, and both sides are zero.
- If $c > 0$ and $x > 0$, then $cx > 0$, and both sides equal cx.
- If $c < 0$ and $x < 0$, then $cx > 0$, so $|cx| = cx$, while $|c|\,|x| = (-c)(-x) = cx$.
- If one of c and x is positive and the other is negative, then $cx < 0$, so $|cx| = -cx$. When $c > 0$ and $x < 0$, we get $|c|\,|x| = c(-x) = -cx$; when $c < 0$ and $x > 0$, we get $|c|\,|x| = (-c)x = -cx$.

These cases cover every possibility, so the rule always holds. Taking $c = -1$ gives $|-x| = |-1|\,|x| = |x|$: changing the sign of a number does not change its distance from zero. Later chapters use the product rule often, for instance to see how an error changes when it is multiplied by a constant.

For a number $b \geq 0$, the statement $|x| \leq b$ says that x is at distance at most b from zero, that is,

$$|x| \leq b \quad \Longleftrightarrow \quad -b \leq x \leq b.$$

To check this, consider the two cases in the formula for $|x|$. If $x \geq 0$, then $|x| = x$, so $|x| \leq b$ says $x \leq b$; and $-b \leq x$ holds automatically, since $-b \leq 0 \leq x$. If $x < 0$, then $|x| = -x$, so $|x| \leq b$ says $-x \leq b$, which becomes $x \geq -b$ after multiplying both sides by -1; and $x \leq b$ holds automatically, since $x < 0 \leq b$.

The same argument with strict inequalities shows that, for a number $b > 0$,

$$|x| < b \quad \Longleftrightarrow \quad -b < x < b.$$

Both rules can be applied to a difference $x - c$ in place of x. The number $|x - c|$ is the distance between x and c on the number line: it equals $x - c$ when $x \geq c$ and $c - x$ when $x < c$, so either way it is the larger of the two numbers minus the smaller. For instance, $|7 - 5| = |5 - 7| = 2$. Applied to $x - 5$ with $b = 1$, the rule for $|x| < b$ turns $|x - 5| < 1$ into $-1 < x - 5 < 1$, and adding 5 to each part gives $4 < x < 6$. In general, for $b > 0$, the statement $|x - c| < b$ holds exactly when $c - b < x < c + b$: the number x lies less than b away from c on the number line.

Now take $b = 2$ in the rule for $|x| \leq b$: $|x| \leq 2$ means $-2 \leq x \leq 2$, which is the conjunction "$-2 \leq x$ and $x \leq 2$." By De Morgan's law, its negation is a disjunction. The negation of $-2 \leq x$ is $x < -2$, and the negation of $x \leq 2$ is $x > 2$, so the negation of $|x| \leq 2$ is "$x < -2$ or $x > 2$." In other words, $|x| > 2$ means that $x < -2$ or $x > 2$. This condition has two separate pieces, one on each side of zero, and the same two-sided form appears when an inequality involves a square, as we now show.

We need two facts about squares. First, squaring preserves order for nonnegative numbers. Suppose $0 \leq a < b$. Multiplying out one bracket at a time gives

$$(b - a)(b + a) = b^2 + ba - ab - a^2 = b^2 - a^2.$$

Both factors on the left are positive: $b - a > 0$ because $a < b$, and $b + a > 0$ because b is positive (it is larger than $a \geq 0$) and a is nonnegative. A product of two positive numbers is positive, so $b^2 - a^2 > 0$, that is, $a^2 < b^2$. In the same way, if $0 \leq a \leq b$, then both factors are nonnegative, so $b^2 - a^2 \geq 0$ and $a^2 \leq b^2$. The condition that the numbers are nonnegative cannot be dropped: $-3 < 2$, but $(-3)^2 = 9$ is larger than $2^2 = 4$.

Second, $x^2 = |x|^2$ for every real number x. If $x \geq 0$, this says $x^2 = x^2$. If $x < 0$, it says $x^2 = (-x)^2$, which is true because $(-x)(-x) = x \cdot x$.

Now we can solve the inequality $x^2 > 4$ completely. Since $4 = 2^2$, we compare $|x|$ with 2.

- If $|x| > 2$, apply the squaring rule for $<$ (if $0 \leq a < b$, then $a^2 < b^2$) with $a = 2$ and $b = |x|$: we get $4 < |x|^2 = x^2$.
- If $|x| \leq 2$, apply the squaring rule for $\leq$ (if $0 \leq a \leq b$, then $a^2 \leq b^2$) with $a = |x|$ and $b = 2$, which is allowed because $|x| \geq 0$: we get $x^2 = |x|^2 \leq 4$.

The first case shows that $|x| > 2$ implies $x^2 > 4$. The second case shows that $|x| \leq 2$ implies $x^2 \leq 4$, which is the contrapositive of "$x^2 > 4$ implies $|x| > 2$." Together, the two cases prove

$$x^2 > 4 \quad \Longleftrightarrow \quad |x| > 2 \quad \Longleftrightarrow \quad x < -2 \text{ or } x > 2.$$

For instance, $x = -3$ satisfies $x^2 = 9 > 4$ even though x is not greater than 2, while $x = 1.5$ gives $x^2 = 2.25$, which is not greater than 4. A tempting shortcut, "take square roots to get $x > 2$," loses the whole negative half of the answer.

Ranges of real numbers like these have a standard notation. For real numbers $a < b$, each of the following sets is called an *interval*:

- $[a, b]$ is the set of all real x with $a \leq x \leq b$;
- (a, b) is the set of all real x with $a < x < b$;
- $[a, b)$ is the set of all real x with $a \leq x < b$, and $(a, b]$ is the set of all real x with $a < x \leq b$.

A square bracket includes its endpoint, and a round bracket excludes it. So 1 and 3 belong to $[1, 3]$ but not to $(1, 3)$, while 2 belongs to both. An interval may also have no endpoint on one side. For example, the notation $[0, \infty)$ means all real x with $x \geq 0$, and $(-\infty, -2)$ means all real x with $x < -2$. The symbol ∞, read "infinity," only says that there is no endpoint on that side. Infinity is not a real number, so it is never placed next to a square bracket. In this notation, $|x| \leq 2$ says that x belongs to $[-2, 2]$, and $x^2 > 4$ says that x belongs to $(-\infty, -2)$ or to $(2, \infty)$.

Round brackets are also used for an *ordered pair*, such as $(1, 3)$, which lists two entries in a definite order; Chapter 3 uses ordered pairs. The surrounding sentence tells you whether $(1, 3)$ means an interval or a pair.

The ideas of this chapter, from hypotheses and conclusions to the order of choices, combine into a routine worth following whenever a statement is proposed or changed.

Habit 2: keep a statement contract

Before working on a statement, write down its *contract*: the domain of each variable, the hypotheses, the conclusion, and the order in which the quantities are chosen. Whenever the statement changes, for example when you or an assistant repair it or generalize it, compare the new version with the contract. Which hypothesis became weaker or stronger? Did the conclusion become weaker or stronger? A changed statement can be true and still fail to answer the original question. In Section 1.1, an assistant that replaced "integers" by "positive integers" gave a correct proof of a weaker statement, one that says nothing about a sum such as $-3 + 5$.

Exercises

Exercises 2.1–2.4 are core exercises, and Exercises 2.5 and 2.6 are transfer exercises. Complete solutions are in Appendix A, under "Week 2."

Exercise 2.1. For a real number x, consider the implication "If $x = 3$, then $x^2 = 9$." State its converse and its contrapositive. Decide which of the three statements are true, and justify each answer.

Exercise 2.2. Negate the statement "For every integer n, there is an integer m with $m > n$ and m even." Keep the domain of each variable unchanged, and take care when you negate the "and." Give the negation both in words and in symbols.

Exercise 2.3. Decide whether each of the statements

$$\forall x \in \mathbb{R}\ \exists y \in \mathbb{R} :\ x + y = 0 \qquad \text{and} \qquad \exists y \in \mathbb{R}\ \forall x \in \mathbb{R} :\ x + y = 0$$

is true. For a true statement, explain how to choose y. For a false statement, show that no choice of y works.

Exercise 2.4. Find a counterexample to the claim "If a real number x satisfies $x^2 > 4$, then $x > 2$." Then repair the claim by changing only its conclusion, so that the repaired claim is true for every real number x. Choose a conclusion that describes exactly which real numbers satisfy $x^2 > 4$.

Exercise 2.5. Give an example of two students and a list of tasks in which each student accepts at least one task, but the two students cannot be given different acceptable tasks. Then find a condition on *groups* of students that is stronger than "each student accepts at least one task" and can be checked by counting. It should fail in your example, and it should hold whenever all the students can be given different acceptable tasks. Explain why it must hold in that case. (Hint: for a group of students, count the tasks that at least one member of the group accepts, and compare with the number of students in the group.)

Exercise 2.6. For each positive real number ε, show how to choose a positive integer N such that every integer $n \geq N$ satisfies $1/n < \varepsilon$. Explain why N may depend on ε, and why it must not depend on the integer n that is tested afterwards.

Before moving on

Close the book and check that you can do the following in your own words. Take the statement "if an integer is divisible by four, then it is even," state its converse and its contrapositive, and say which of the three are true and why. Explain why statement (A) of Section 2.3 is true and statement (B) is false, using the idea of choices made in order. Write the negation of (A) and say in words what it claims. Explain why the negation of an "and" statement is an "or" statement, and why "every integer that is both even and odd is larger than 100" is true but tells us nothing about any integer. Show that, for every positive ε, some positive integer N satisfies $1/N < \varepsilon$, and say why N must be allowed to depend on ε. Finally, find all real numbers x with $x^2 > 4$ without losing the negative ones.

These skills come back throughout the book. In Chapter 11, the definition of a limit is a statement with three quantifiers, and its meaning depends on their order. In Chapters 8 and 9, the gap between "each student has an acceptable task" and "all the students can be given different acceptable tasks" leads to Hall's theorem. In Chapters 15 and 16, the proof assistant Lean writes implications and quantifiers explicitly, and the usual way to prove an existence statement there is to supply an actual witness together with a proof that it has the required property.

CHAPTER 3

Objects, maps, and information

This week

Use sets to say exactly which objects a question is about, and functions to describe how inputs are turned into outputs. Prove facts about sets and functions by unpacking their definitions, one membership condition at a time. Decide whether a function loses information by sending different inputs to the same output, and notice when a proof needs a property only at the points it actually uses.

3.1 Sets express the boundary of a question

Section 1.1 suggested that the first question to ask about any claim is: which objects does it talk about? A *set* is the mathematical tool for recording the answer. It is a collection of objects, called its *members* or *elements*, and it is completely determined by which objects belong to it. The set $A = \{1, 3, 5\}$ has three elements: 1, 3, and 5. Listing a member twice does not create a new element, so $\{1, 1, 3\} = \{1, 3\}$. The order of the list does not matter either: $\{3, 1\} = \{1, 3\}$.

We write $x \in A$, read "x is in A" or "x belongs to A," when x is a member of A, and $x \notin A$ when it is not. For example, $3 \in \{1, 3, 5\}$ and $2 \notin \{1, 3, 5\}$. The *empty set* $\varnothing$ is the set with no members at all. The notation $A \subseteq B$, read "A is a *subset* of B" or "A is contained in B," means that every member of A is also a member of B. For example, $\{1, 3\} \subseteq \{1, 2, 3\}$. On the other hand, $\{1, 4\}$ is not a subset of $\{1, 2, 3\}$, because 4 belongs to the first set but not to the second. Every set A is a subset of itself, since each member of A is certainly a member of A.

To prove $A \subseteq B$, begin with "Let $x \in A$." Here x is *arbitrary*: as in the proofs of Chapter 1, it stands for any member of A whatsoever, so the argument may use only the fact that $x \in A$. Then show that $x \in B$. Because x was arbitrary, this one argument covers every member of A at once.

Three operations build new sets from old ones. The *union* $A \cup B$ is the set of objects that belong to at least one of A and B. This is the inclusive "or" of Section 2.2: an object that lies in both sets also lies in the union. The *intersection* $A \cap B$ is the set of objects that belong to both A and B. The *difference* $A \setminus B$ is the set of objects that belong to A but not to B. For $A = \{1, 2, 3\}$ and $B = \{3, 4\}$,

$$A \cup B = \{1, 2, 3, 4\}, \qquad A \cap B = \{3\}, \qquad A \setminus B = \{1, 2\}.$$

Notice that $B \setminus A = \{4\}$ differs from $A \setminus B$. The order of the two sets matters for a difference, though not for a union or an intersection.

To describe a set by a property instead of a list, we write $\{x \in A : P(x)\}$ for the set of those members of A that satisfy the property P. Read the colon as "such that." For example, $\{n \in \mathbb{Z} : n \text{ is even}\}$ is the set of even integers. Likewise $\{x \in \mathbb{R} : x^2 \leq 4\}$ is the interval $[-2, 2]$.

Indeed, Section 2.6 showed that $x^2 > 4$ holds exactly when $x < -2$ or $x > 2$, so $x^2 \leq 4$ holds exactly when neither happens, that is, when $-2 \leq x \leq 2$. Two of the operations above can be written in this notation: $A \cap B = \{x \in A : x \in B\}$ and $A \setminus B = \{x \in A : x \notin B\}$.

Membership, containment, and equality

The symbols $\in$ and $\subseteq$ are easy to confuse, but they connect different kinds of objects: $\in$ has an object on its left and a set on its right, while $\subseteq$ has a set on both sides. The statement $1 \in \{1, 2\}$ is about the number 1: it says that 1 is one of the members. The statement $\{1\} \subseteq \{1, 2\}$ is about the set $\{1\}$: it says that every member of $\{1\}$ is also a member of $\{1, 2\}$. Both statements are true, and in general $x \in B$ holds exactly when $\{x\} \subseteq B$, because the only member of $\{x\}$ is x itself. But the two symbols cannot be exchanged. The statement $\{1\} \in \{1, 2\}$ is false, because the two members of $\{1, 2\}$ are the numbers 1 and 2, and neither of them is the set $\{1\}$. The expression $1 \subseteq \{1, 2\}$ has no meaning, because 1 is a number, not a set.

Two sets are *equal* when they have exactly the same members. So to prove $D = E$, we prove two containments: $D \subseteq E$ (everything in D is in E) and $E \subseteq D$ (everything in E is in D). For the first, begin with "Let $x \in D$," and do not assume at the outset that x also belongs to E: deriving that is the whole point of the argument.

The empty set is a subset of every set: $\varnothing \subseteq E$ for every set E. To see why, ask what would make this containment false. We would need a member of $\varnothing$ that is not in E. Since $\varnothing$ has no members at all, no such counterexample can exist, so the containment is true. This is the set version of vacuous truth from Section 2.5. It also means that a proof beginning "Let $x \in D$" remains valid when D happens to be empty. The proof explains what happens whenever a member is supplied; it never claims that a member exists.

Here is a first set equality to prove. Before reading the proof, try the claim on small sets: take $A = \{1, 2, 3\}$, $B = \{2, 4\}$, and $C = \{3, 4\}$. On one side, $B \cup C = \{2, 3, 4\}$, and keeping only the members that also lie in A leaves $A \cap (B \cup C) = \{2, 3\}$. On the other side, $A \cap B = \{2\}$ and $A \cap C = \{3\}$, and their union is again $\{2, 3\}$. The proof below explains why the two sides agree for all sets, not only for these three.

Proposition 3.1. *For any sets A, B, C, we have $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$.*

Proof. We prove the two containments separately.

First let $x \in A \cap (B \cup C)$. Then $x \in A$ and $x \in B \cup C$, so $x \in B$ or $x \in C$. If $x \in B$, then x lies in both A and B, so $x \in A \cap B$. If $x \in C$, then in the same way $x \in A \cap C$. In either case x belongs to at least one of the sets $A \cap B$ and $A \cap C$, so $x \in (A \cap B) \cup (A \cap C)$. Since x was an arbitrary member of $A \cap (B \cup C)$, this proves $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$.

Conversely, let $x \in (A \cap B) \cup (A \cap C)$. Then $x \in A \cap B$ or $x \in A \cap C$. In the first case $x \in A$ and $x \in B$; in the second, $x \in A$ and $x \in C$. In both cases $x \in A$, and x lies in B or in C, that is, $x \in B \cup C$. Thus $x \in A \cap (B \cup C)$, which proves the reverse containment. The two containments together give the equality. □

The proof never asked what the elements were. They could be numbers, people, or tasks; the

argument used only the meanings of "and" and "or" applied to membership. That is why the identity holds for sets of any kind of object.

Pause and decide. For $A = \{1, 2, 3\}$, $B = \{2, 4\}$, and $C = \{3, 4\}$, compare $A \setminus (B \cup C)$ with $(A \setminus B) \cap (A \setminus C)$. Do you expect the two sides to agree for all sets?

Check. Here $B \cup C = \{2, 3, 4\}$, so $A \setminus (B \cup C) = \{1\}$. Also $A \setminus B = \{1, 3\}$ and $A \setminus C = \{1, 2\}$, whose intersection is $\{1\}$. The two sides agree for all sets. An object x belongs to $A \setminus (B \cup C)$ when $x \in A$ and the statement "$x \in B$ or $x \in C$" is false. By De Morgan's law $\neg(P \vee Q) \Leftrightarrow (\neg P) \wedge (\neg Q)$ (Section 2.2), this says that $x \in A$, $x \notin B$, and $x \notin C$. Membership in $(A \setminus B) \cap (A \setminus C)$ says that $x \in A$ and $x \notin B$, and also $x \in A$ and $x \notin C$, which is the same condition.

Counting members and forming pairs

A set is *finite* if its members can be written down in a list that ends. A set that is not finite, such as $\mathbb{N}$ or $\mathbb{Z}$, is called *infinite*. For a finite set S, the notation $|S|$ means the number of its members, each counted once. For example, if $S = \{2, 4, 6\}$, then $|S| = 3$: we count the members, we do not add them up. The empty set has $|\varnothing| = 0$. The number $|S|$ is called the *size* or *cardinality* of S. Bars around a real number mean its absolute value (Section 2.6); bars around a finite set mean its size. The kind of object inside the bars tells you which meaning is intended.

Two sets A and B are *disjoint* if they have no member in common, that is, if $A \cap B = \varnothing$. For disjoint finite sets, $|A \cup B| = |A| + |B|$: each member of the union lies in exactly one of the two sets, so it is counted exactly once on the right. Overlapping sets behave differently. For example, $|\{1, 2\}| + |\{2, 3\}| = 4$, but $|\{1, 2\} \cup \{2, 3\}| = |\{1, 2, 3\}| = 3$, because the common member 2 is counted twice in the sum. In general, the sum $|A| + |B|$ counts every member of $A \cap B$ twice and every other member of $A \cup B$ once. Subtracting $|A \cap B|$ removes the extra count of each common member:

$$|A \cup B| = |A| + |B| - |A \cap B| \qquad \text{for all finite sets } A, B.$$

In the example, $2 + 2 - 1 = 3$.

In a set, the order of the members does not matter, but sometimes order is exactly what we need to record. An *ordered pair* (a, b) has a first entry a and a second entry b. Two ordered pairs are equal exactly when their first entries are equal and their second entries are equal. So $(1, 3) \neq (3, 1)$, although $\{1, 3\} = \{3, 1\}$. When we describe a situation mathematically, we have to decide which kind of object fits it. The three students in a study group form a set, because only membership matters. The winner and the runner-up of a race form an ordered pair, because swapping them changes the result.

The *Cartesian product* $A \times B$ is the set of all ordered pairs (a, b) with $a \in A$ and $b \in B$. If $A = \{1, 2\}$ and $B = \{p, q\}$, where p and q are two different objects, then

$$A \times B = \{(1, p), (1, q), (2, p), (2, q)\}.$$

In general, each of the $|A|$ choices of first entry can be combined with each of the $|B|$ choices of second entry, so $|A \times B| = |A| \cdot |B|$ for finite sets. The multiplication sign in $A \times B$ does not

multiply the elements of A and B. The product is a set of pairs, and only its *size* is a product of numbers. Products are useful whenever a single input consists of two pieces. For instance, adding two real numbers takes a pair (x, y) from $\mathbb{R} \times \mathbb{R}$ and returns the number $x + y$. In Chapter 11, the distance between two points of a set X will likewise take its input from $X \times X$.

3.2 A function includes its input and output sets

A *function* $f : A \to B$, read "f from A to B," assigns to each input $x \in A$ exactly one output $f(x) \in B$. The set A is the *domain* of f: the set of allowed inputs. The set B is the *codomain*: the set in which the outputs are required to lie. We also call the elements of B *targets*. The words *map* and *mapping* mean the same as function.

Not every target needs to occur as an output. The outputs that actually occur form the *image* of f:

$$f(A) = \{f(x) : x \in A\}.$$

This is a second way of using set-builder notation: it describes the set of all values $f(x)$ obtained as x runs through A. For a small example, let $A = \{p, q, r\}$, where p, q, r are three different objects, let $B = \{1, 2, 3, 4\}$, and define $f(p) = 1$, $f(q) = 3$, and $f(r) = 3$. This is a function, because each of the three inputs has exactly one output in B. Two inputs share the output 3, which is allowed. The image is $f(A) = \{1, 3\}$, and the targets 2 and 4 are never reached.

For $f : \mathbb{R} \to \mathbb{R}$ given by $f(x) = x^2$, the image is the set $[0, \infty)$ of nonnegative real numbers. Two facts are needed to see this. Every output x^2 is nonnegative, so the image is contained in $[0, \infty)$. Conversely, every $t \geq 0$ is an output, because $f(\sqrt{t}) = (\sqrt{t})^2 = t$, where $\sqrt{t}$ is the square root from Section 1.8. The codomain of this f, however, is all of $\mathbb{R}$. The codomain is part of how we specify a function; the image is something we work out from the rule.

So a function is described by three pieces of information: its domain, its codomain, and its rule. To name a rule without giving it a letter, we write an arrow such as $x \mapsto x^2$, read "x maps to x^2." Two functions $f, g : A \to B$ with the same domain and the same codomain are *equal* when $f(x) = g(x)$ for every $x \in A$. Functions with different domains or different codomains count as different functions, even when they use the same rule. For example, the rule $x \mapsto x^2$ with domain $\mathbb{R}$ and codomain $\mathbb{R}$, and the same rule with domain $\mathbb{R}$ and codomain $[0, \infty)$, are two different functions: for the second, every target is an output, while for the first the negative targets are never reached. Section 3.3 gives this difference a name.

Pause and decide. Does the rule "send x to a number y with $y^2 = x$" define a function from $\mathbb{R}$ to $\mathbb{R}$?

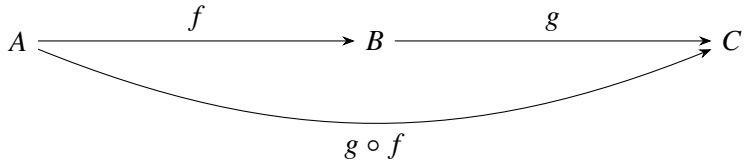
Check. No, for two separate reasons. The input -1 has no output at all, since no real number has a negative square. The input 4 has two candidate outputs, 2 and -2, and the rule does not say which one to take. A function must give *exactly one* output for *every* input. The rule $t \mapsto \sqrt{t}$ from $[0, \infty)$ to $\mathbb{R}$ repairs both problems: it shrinks the domain to the numbers that have a square root, and it always picks the nonnegative root.

The *identity map* $\mathrm{id}_A : A \to A$ sends each element to itself: $\mathrm{id}_A(x) = x$ for every $x \in A$. If

$f : A \to B$ and $g : B \to C$, their *composition* $g \circ f : A \to C$ applies f first and g second:

$$(g \circ f)(x) = g(f(x)).$$

This makes sense because every output of f lies in B, which is exactly the set of allowed inputs of g. Since we read from left to right, it is easy to get the order wrong: in $g \circ f$, the function written on the right acts first. Reading $g \circ f$ aloud as "g after f" helps.



Composing two concrete maps

Let $f, g : \mathbb{R} \to \mathbb{R}$ be given by $f(x) = x + 1$ and $g(y) = 2y$. Starting at 3, the map f gives 4, and then g gives 8. Thus $(g \circ f)(3) = 8$. Starting instead with g gives 6, and then f gives 7, so $(f \circ g)(3) = 7$. The two orders give different answers at 3, so $g \circ f$ and $f \circ g$ are different functions.

The general formulas are

$$(g \circ f)(x) = g(x + 1) = 2(x + 1) = 2x + 2 \qquad \text{and} \qquad (f \circ g)(x) = f(2x) = 2x + 1.$$

These are exactly the two expressions compared in Section 1.2, where $2(n + 1)$ turned out to equal $2n + 2$, not $2n + 1$.

The definition of $g \circ f$ requires the codomain of f to be the domain of g, so that every output of f is an allowed input of g. To see what goes wrong without this requirement, let $f : \mathbb{R} \to \mathbb{R}$ be $f(x) = x - 1$, and let $g : [0, \infty) \to \mathbb{R}$ be $g(t) = \sqrt{t}$. The expression $g(f(x))$ makes sense at $x = 5$, where it equals $g(4) = 2$. It makes no sense at $x = 0$, because $f(0) = -1$ is not an allowed input of g. So $g \circ f$ is not defined. If we instead use the same rule $x \mapsto x - 1$ as a function from $[1, \infty)$ to $[0, \infty)$, its codomain is now the domain of g, and the composition $x \mapsto \sqrt{x - 1}$ is defined on $[1, \infty)$.

Undoing a function

A function $f : A \to B$ is *reversible*, or *invertible*, if some function $h : B \to A$ undoes it from both sides:

$$h(f(a)) = a \ \text{ for every } a \in A \qquad \text{and} \qquad f(h(b)) = b \ \text{ for every } b \in B.$$

Such an h is called an *inverse* of f. Because two functions are equal when they agree at every input, the two conditions can be written as $h \circ f = \mathrm{id}_A$ and $f \circ h = \mathrm{id}_B$. For $f : \mathbb{R} \to \mathbb{R}$ with $f(x) = x + 1$, the map $h(y) = y - 1$ is an inverse: $h(f(x)) = (x + 1) - 1 = x$ and $f(h(y)) = (y - 1) + 1 = y$.

Both conditions must hold at *every* input. The square root seems to undo squaring, since $\sqrt{3^2} = 3$. But for the square map $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$, we get $\sqrt{f(-3)} = \sqrt{9} = 3$, not -3. A rule that

undoes f at some inputs but not at others is not an inverse. In fact this f has no inverse at all. An inverse h would need $h(1) = h(f(1)) = 1$ and also $h(1) = h(f(-1)) = -1$, but a function has only one output at 1.

A function has at most one inverse. Suppose that h and k are both inverses of f, and let $b \in B$. Since k is an inverse of f, we have $f(k(b)) = b$, and applying h to both sides gives $h(f(k(b))) = h(b)$. Since h is an inverse of f, we have $h(f(a)) = a$ for every $a \in A$; with $a = k(b)$ this says $h(f(k(b))) = k(b)$. Comparing the two results gives $h(b) = k(b)$. This holds for every $b \in B$, so $h = k$. We may therefore speak of *the* inverse of f. It is usually written f^{-1}. This symbol does not mean the reciprocal $1/f$. For $f(x) = x + 1$, the inverse is $f^{-1}(y) = y - 1$, whereas the reciprocal gives $1/f(y) = 1/(y + 1)$. At $y = 1$, for instance, $f^{-1}(1) = 0$, while $1/f(1) = 1/2$.

3.3 Injective, surjective, and reversible

Which functions can be undone? The answer combines two properties, each of which is useful on its own.

A function $f : A \to B$ is *injective* if different inputs always give different outputs: whenever $x \neq y$, also $f(x) \neq f(y)$. An injective function loses no information, because each output tells us which input produced it. In proofs we usually work with the contrapositive form of this definition (Section 2.5), which says the same thing with equalities in place of inequalities:

$$\forall x, y \in A : \qquad f(x) = f(y) \Rightarrow x = y.$$

Here "$\forall x, y \in A$" is short for "$\forall x \in A\ \forall y \in A$." In words: if two inputs have the same output, they were the same input. This form is easier to use because an equation gives us something to calculate with: from $x + 1 = y + 1$ we can subtract 1 and get $x = y$. An assumption such as $x \neq y$ gives us very little to go on. So to prove that f is injective, take arbitrary $x, y \in A$, assume $f(x) = f(y)$, and deduce $x = y$.

A function $f : A \to B$ is *surjective* if every target is an output:

$$\forall b \in B\ \exists a \in A : \qquad f(a) = b.$$

Equivalently, the image $f(A)$ is the whole codomain B. In the game of Section 2.3, the skeptic hands us a target b, and we, as the prover, respond with an input a, which may depend on b. An input a with $f(a) = b$ is a *witness* that b is reached, just as 3 is the witness for the oddness of $7 = 2 \cdot 3 + 1$ in Section 1.4. To prove that f is surjective, take an arbitrary $b \in B$ and produce a witness for it.

A function that is both injective and surjective is *bijective*, and is called a *bijection*. Other books often say *one-to-one* for injective and *onto* for surjective. Injectivity keeps different inputs apart; surjectivity makes sure that no target is missed. A function can have either property without the other, as the next two examples show.

The map $f : \{p, q\} \to \{1, 2, 3\}$ with $f(p) = 1$ and $f(q) = 2$ is injective, because its two inputs have different outputs. It is not surjective, because no input reaches the target 3. The map

$g : \{p, q, r\} \to \{1, 2\}$ with $g(p) = 1$, $g(q) = 1$, and $g(r) = 2$ is surjective, because both targets are reached. It is not injective, because $g(p) = g(q)$ although $p \neq q$. If we are told only that the output of g is 1, we cannot tell whether the input was p or q: that information has been lost.

Here is how a bijection is undone in a small case. Let $f : \{p, q, r\} \to \{1, 2, 3\}$ be given by $f(p) = 2$, $f(q) = 3$, and $f(r) = 1$. Every target is reached exactly once, so f is bijective. Reading each assignment backward, so that $f(p) = 2$ becomes $h(2) = p$, gives the inverse: $h(1) = r$, $h(2) = p$, and $h(3) = q$. You can check both conditions directly; for example, $h(f(q)) = h(3) = q$ and $f(h(1)) = f(r) = 1$.

Why does this recipe work for every bijection? Call an input a a *preimage* of a target b when $f(a) = b$; in other words, a preimage of b is a witness that b is reached. To build an inverse h, we must decide, for every target b, which input $h(b)$ should be. The two properties of a bijection are exactly what make this decision possible and unambiguous.

- Surjectivity says that every target b has *at least one* preimage, so there is something to choose.
- Injectivity says that every target has *at most one* preimage: if $f(a_1) = b$ and $f(a_2) = b$, then $f(a_1) = f(a_2)$, and so $a_1 = a_2$. So there is no choice to make.

We may therefore define $h(b)$ to be *the* unique preimage of b. Now check the two conditions for an inverse. By the definition of h, the input $h(b)$ is sent to b, so $f(h(b)) = b$. For the other condition, start with an input a and look at its output $f(a)$. The input a is certainly a preimage of $f(a)$, and that preimage is unique, so $h(f(a)) = a$.

The converse also holds: a function with an inverse must be both injective and surjective. Exercise 3.5 asks you to prove this, in a sharper form that uses each of the two conditions on an inverse separately. Together, the two facts say that a function is reversible exactly when it is bijective.

The square map $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$, is neither injective nor surjective: $f(1) = f(-1)$, and the target -1 is never reached. This agrees with Section 3.2, where we saw that it has no inverse.

Now keep the rule but change both sets: let $s : [0, \infty) \to [0, \infty)$ be given by $s(x) = x^2$. This new function is a bijection. For injectivity, suppose $s(x) = s(y)$, and call this common value t. Then x and y are both nonnegative numbers whose square is t. By the fact accepted in Section 1.8, a nonnegative real number has exactly one nonnegative square root, so $x = y$. For surjectivity, a target $t \geq 0$ is reached by the input $\sqrt{t}$, which is nonnegative. The inverse of s is the square-root function $t \mapsto \sqrt{t}$ from $[0, \infty)$ to $[0, \infty)$. Both conditions for an inverse can be checked directly: $\sqrt{s(x)} = \sqrt{x^2} = x$ for every $x \geq 0$ (Section 1.8), and $s(\sqrt{t}) = (\sqrt{t})^2 = t$ for every $t \geq 0$. Shrinking the domain removed the collision between 1 and -1, and shrinking the codomain removed the negative targets that could never be reached.

For finite sets, these properties also compare sizes. Suppose A and B are finite and $f : A \to B$ is injective. The $|A|$ different inputs then have $|A|$ different outputs, and all of these outputs lie in B. So B contains at least $|A|$ different members, that is, $|A| \leq |B|$. If f is also surjective, those $|A|$ different outputs make up all of B, so $|A| = |B|$. Thus a bijection between two finite sets shows that they have the same size, without counting either one. The same counting also works in the other direction when the sizes agree. If $|A| = |B|$ and $f : A \to B$ is injective, then the $|A|$ different

outputs are $|B|$ different members of B, so they make up all of B, and f is surjective as well. Read as a contrapositive, the first fact says that if $|A| > |B|$, then no map from A to B is injective: every such map sends two different inputs to the same output. This is the simplest case of the pigeonhole principle. Chapter 7 states the principle in general as Proposition 7.4, in the language of objects placed in boxes.

Both properties survive composition. Here is the statement for injectivity; Exercise 3.2 asks you to prove the one for surjectivity.

Theorem 3.2. *If $f : A \to B$ and $g : B \to C$ are injective, then $g \circ f$ is injective.*

Plan. An equality of final outputs should let us recover an equality at the intermediate stage, and then an equality of inputs.

Proof. Take any $x, y \in A$ and assume $g(f(x)) = g(f(y))$. The elements $f(x)$ and $f(y)$ belong to B, so injectivity of g applies to them and gives $f(x) = f(y)$. Injectivity of f then gives $x = y$. This is exactly the implication required for $g \circ f$ to be injective. □

An assistant may summarize this proof as "cancel g, then cancel f." That shorthand is fine once it is clear that each cancellation is an application of injectivity. Functions cannot be cancelled in general. For $g : \mathbb{R} \to \mathbb{R}$ with $g(y) = y^2$ we have $g(1) = g(-1)$, yet $1 \neq -1$, so cancelling g here would lead to a false conclusion.

Proposition 3.3. *Let $f : A \to B$ and $g : B \to C$. If $g \circ f$ is injective, then f is injective. However, g need not be injective.*

Proof. Let $x, y \in A$ with $f(x) = f(y)$. Applying the same function g to equal objects gives equal results, so $g(f(x)) = g(f(y))$. Injectivity of $g \circ f$ then gives $x = y$. Hence f is injective.

For the second assertion, one example is enough. Take $A = \{0, 1\}$, $B = \{0, 1, 2\}$, $C = \{0, 1\}$, and define

$$f(0) = 0, \quad f(1) = 1, \qquad g(0) = 0, \quad g(1) = g(2) = 1.$$

The composition sends 0 to 0 and 1 to 1, so it is injective. Yet g is not injective, because the different inputs 1 and 2 have the same output. □

Look at why the example works. The composition $g \circ f$ only ever applies g to outputs of f, that is, to points of the image $f(A) = \{0, 1\}$. The collision $g(1) = g(2)$ involves the point 2, which f never reaches, so the composition cannot detect it. Studying why a counterexample works often shows what the true statement should be. Here it suggests that g must be injective on the image of f, though not necessarily anywhere else.

To say this precisely, we need one more term. If S is a subset of the domain of g, the *restriction* of g to S is the function from S to C that uses the same rule as g but accepts only inputs from S. Whenever $g \circ f$ is injective, the restriction of g to $f(A)$ is injective. To check this, let $b_1, b_2 \in f(A)$ with $g(b_1) = g(b_2)$. Because both points lie in the image, there are inputs $x, y \in A$ with $b_1 = f(x)$ and $b_2 = f(y)$. Then $g(f(x)) = g(f(y))$, so injectivity of $g \circ f$ gives $x = y$, and therefore $b_1 = f(x) = f(y) = b_2$. In the example, the restriction of g to $\{0, 1\}$ sends 0 to 0 and 1 to 1, which is injective.

This diagnosis also suggests an extra hypothesis under which g itself must be injective. If f is surjective, then $f(A)$ is all of B, so no input of g goes untested.

Proposition 3.4. *Let $f : A \to B$ and $g : B \to C$. If $g \circ f$ is injective and f is surjective, then g is injective.*

Proof. Since f is surjective, $f(A) = B$, so the restriction of g to $f(A)$ is g itself. We showed above that this restriction is injective whenever $g \circ f$ is injective. Hence g is injective. □

Each hypothesis has a visible job: injectivity of $g \circ f$ makes g injective on the outputs of f, and surjectivity of f makes those outputs all of B.

3.4 Images and preimages answer different questions

Let $f : A \to B$. We can apply f to a whole set of inputs at once. For $S \subseteq A$, the *image* of S is

$$f(S) = \{f(x) : x \in S\},$$

the set of outputs produced by inputs from S. It answers the question "where do these inputs go?" When S is the whole domain A, this is the image $f(A)$ from Section 3.2. Going the other way, for $T \subseteq B$, the *preimage* of T is

$$f^{-1}(T) = \{x \in A : f(x) \in T\},$$

the set of inputs whose outputs land in T. It answers the question "which inputs end up here?" Despite the notation, $f^{-1}(T)$ makes sense for every function f, whether or not f has an inverse; we return to this point after some examples. When $T = \{b\}$ consists of a single target, $f^{-1}(\{b\})$ is the set of all preimages of b in the sense of Section 3.3. So f is surjective exactly when every such set has at least one member, and injective exactly when every such set has at most one member.

For the square map $f : \mathbb{R} \to \mathbb{R}$, $f(x) = x^2$, here are some images and preimages:

$$f(\{-1, 1, 2\}) = \{1, 4\}, \qquad f^{-1}(\{1\}) = \{-1, 1\},$$
$$f^{-1}(\{4, 9\}) = \{-3, -2, 2, 3\}, \qquad f^{-1}(\{-1\}) = \varnothing.$$

Notice the braces. The value $f(3) = 9$ is a number, while the image $f(\{3\}) = \{9\}$ is a set; images and preimages are always sets. The preimage $f^{-1}(\{-1\})$ is empty because no real number has square -1. The set T may also be an interval: $f^{-1}([0, 4]) = [-2, 2]$. Indeed, a real number x lies in this preimage when $0 \le x^2 \le 4$. The first inequality holds for every real x, and the second holds exactly when $-2 \le x \le 2$, as we saw in Section 3.1 using the work of Section 2.6.

The square map has no inverse, as Section 3.2 showed, yet all of its preimages make sense. When f does have an inverse $h : B \to A$, the two possible readings of the symbol $f^{-1}(T)$ agree: the preimage of T under f is the same set as the image $h(T)$ of T under the inverse function. To see this, take $x \in A$. If x lies in the preimage, then $t = f(x)$ belongs to T, and $x = h(f(x)) = h(t)$, so $x \in h(T)$. Conversely, if $x = h(t)$ for some $t \in T$, then $f(x) = f(h(t)) = t$, which lies in T, so x lies in the preimage.

Pause and decide. For the square map $f : \mathbb{R} \to \mathbb{R}$, compute $f^{-1}(f(\{2\}))$. Do you get back $\{2\}$?

Check. First $f(\{2\}) = \{4\}$. Then $f^{-1}(\{4\}) = \{-2, 2\}$, which is larger than $\{2\}$. Going forward and then backward returns every input that shares an output with 2, and -2 is such an input. In general $S \subseteq f^{-1}(f(S))$, because each $x \in S$ has its output $f(x)$ in $f(S)$. If f is injective, nothing extra comes back. Indeed, if $x \in A$ and $f(x) \in f(S)$, then $f(x) = f(s)$ for some $s \in S$, and injectivity gives $x = s \in S$. So $f^{-1}(f(S)) = S$ for every $S \subseteq A$ when f is injective.

Preimages fit well with the set operations of Section 3.1. For any $S, T \subseteq B$,

$$f^{-1}(S \cap T) = f^{-1}(S) \cap f^{-1}(T).$$

Read the left side as "the inputs whose outputs land in the intersection." We can justify the identity one membership condition at a time. For any $x \in A$,

$$\begin{aligned}
x \in f^{-1}(S \cap T) &\iff f(x) \in S \cap T && \text{definition of preimage} \\
&\iff f(x) \in S \text{ and } f(x) \in T && \text{definition of intersection} \\
&\iff x \in f^{-1}(S) \text{ and } x \in f^{-1}(T) && \text{definition of preimage} \\
&\iff x \in f^{-1}(S) \cap f^{-1}(T) && \text{definition of intersection.}
\end{aligned}$$

Each double arrow $\iff$ means "if and only if," so each step is valid in both directions. Reading the chain downward shows that every member of the left side lies in the right side, and reading it upward shows the reverse containment. Both sides consist only of members of A, and x was an arbitrary element of A, so the two sets have exactly the same members. No inverse function was used. Section 16.3 in Chapter 16 expresses these same membership conditions in the proof assistant Lean. Exercise 3.4 asks for the corresponding identity for unions, and Exercise 3.6 asks whether images behave as well as preimages do.

> **Habit 3: identify the information actually used**
>
> When you read or write a proof, ask which facts it uses and where it uses them. A statement about every target can be much stronger than a statement about the outputs that are reached. In Proposition 3.3, the composition $g \circ f$ tests g only at points of the image $f(A)$, so a conclusion about g on all of B asks for more than the argument can give, and in this case more than is true. The same question can also show that a result reaches further than it first seems to: the proof of Proposition 3.1 used only the pattern of membership, never what the elements were, so it holds for sets of any objects. Before you propose a generalization, or accept one from an assistant, find the part of each object that the proof actually visits.

Exercises

Exercises 3.1–3.4 are core exercises, and Exercises 3.5 and 3.6 are transfer exercises. Complete solutions are in Appendix A, under "Week 3."

Exercise 3.1. For $A = \{1, 2, 4\}$ and $B = \{2, 3, 4\}$, find $A \cup B$, $A \cap B$, and $A \setminus B$.

Exercise 3.2. Let $f : A \to B$ and $g : B \to C$ be surjective. Prove that $g \circ f : A \to C$ is surjective. Think about which witness you must find first.

Exercise 3.3. Let $f : \mathbb{Z} \to \mathbb{Z}$ be given by $f(n) = 2n$. Decide whether f is injective and whether it is surjective, and prove your answers. Then keep the domain and the rule, change only the codomain, and obtain a bijection.

Exercise 3.4. Let $f : A \to B$ and $S, T \subseteq B$. Prove that $f^{-1}(S \cup T) = f^{-1}(S) \cup f^{-1}(T)$ by following the membership conditions on both sides.

Exercise 3.5. Let $f : A \to B$ and $h : B \to A$. Suppose that $h \circ f = \mathrm{id}_A$; such an h is called a *left inverse* of f. Prove that f is injective. Suppose instead that $f \circ h = \mathrm{id}_B$; such an h is called a *right inverse* of f. Prove that f is surjective.

Exercise 3.6. Let $f : A \to B$ and $S, T \subseteq A$. Is $f(S \cap T) = f(S) \cap f(T)$ always true? Prove the containment that always holds, and give a counterexample to equality.

CHAPTER 4

How a proof finds its next step

This week

Find the next step of a proof by working backward from what you must show and forward from what you know. Use direct proof, proof of the contrapositive, proof by contradiction, and separate proofs of existence and uniqueness, and learn when each one helps. Prove that every integer is even or odd, and that $\sqrt{2}$ is irrational. Read a long proof by checking, at each step that uses an earlier result, that the result really applies.

4.1 Working backward and working forward

Every proof has a *goal*, the statement it must establish, and a supply of facts it is allowed to use: the hypotheses, the definitions, and results proved or accepted earlier. When you do not know what to write next, there are two places to look. *Working backward* starts from the goal and asks: what would be enough to establish this? *Working forward* starts from the facts and asks: what follows from what I already have? A proof is found when the two directions meet, that is, when something you can derive from the facts is exactly what working backward told you would be enough.

Once you unpack the definitions in it, the goal often tells you exactly what you must produce or deduce. To show that an integer is even, you must find an integer w such that the number equals $2w$ (Definition 1.1). To show that a function f is injective, you take two inputs x and y with $f(x) = f(y)$, and you must deduce that $x = y$ (Section 3.3). To show that an object with some property exists, the usual way is to name a particular object and check that it has the property. As in Section 1.4, an object that passes this check is called a *witness*. Occasionally existence is proved without naming any witness: one shows that if no suitable object existed, something impossible would follow. Such an argument is a proof by contradiction, a method explained in Section 4.2, and Chapter 13 contains proofs of this kind.

A proof state on paper

Here is a small example. We prove:

$$\textit{If } n \textit{ is an even integer, then } n^2 \textit{ is even.}$$

Section 2.1 proved this in one line. This time we slow down to watch how such a proof is found. At each moment of the search, we keep two lists: the facts we already have, and what we still have to show. Together, these two lists are called the *proof state*. Proof assistants such as Lean display exactly this pair after every step (Chapter 15), and keeping it on paper is just as useful.

At the start, the only fact is the hypothesis, "n is an even integer," and the goal is "n^2 is even." Working backward, we unpack the goal with the definition of even. It becomes: *find an integer w with $n^2 = 2w$*. The letter w is new: it names an integer we still have to find. Working forward, we unpack the hypothesis in the same way, but with a different letter: $n = 2k$ for some integer k. Unlike w, the integer k is handed to us by the hypothesis. Squaring gives $n^2 = 4k^2$, and factoring out a two gives $n^2 = 2(2k^2)$. This has exactly the shape the goal asks for, so we choose $w = 2k^2$. It is an integer, because a product of integers is an integer. For instance, if $n = 6$, then $k = 3$ and $w = 2 \cdot 3^2 = 18$, and indeed $6^2 = 36 = 2 \cdot 18$. Only now does the goal move from the "still to show" list to the "known" list. The table records the search.

Step	Known	Still to show
Start	n is an even integer	n^2 is even
Unpack the goal	n is an even integer	an integer w with $n^2 = 2w$
Unpack the hypothesis	$n = 2k$, where k is an integer	an integer w with $n^2 = 2w$
Square and factor	$n = 2k$ and $n^2 = 2(2k^2)$	an integer w with $n^2 = 2w$
Choose $w = 2k^2$	$n^2 = 2w$, and w is an integer	nothing: the proof is finished

The search moved in both directions, but the written proof runs forward only:

> Let n be an even integer. Then $n = 2k$ for some integer k, so $n^2 = 4k^2 = 2(2k^2)$. Since $2k^2$ is an integer, n^2 is even.

A proof like this one, which starts from the hypotheses and reaches the conclusion through a chain of deductions, is called a *direct proof*.

Keeping the goal on the "still to show" list until it has been earned protects against a common mistake: writing "since n^2 is even, $n^2 = 2w$ for some integer w" in the middle of the proof. That sentence uses the conclusion as though it were already known, so the argument goes in a circle. Working backward is still the right way to search: it told us to look for an integer w with $n^2 = 2w$. In the finished proof, however, every fact must come from the hypotheses, from definitions, or from earlier results, and the goal appears only at the end, as the conclusion that has been reached. The next two sections show other ways to organize a proof when such a direct chain is hard to find.

Pause and decide. Let $f : \mathbb{Z} \to \mathbb{Z}$ be given by $f(n) = n + 3$. To prove that f is surjective, what does the goal ask you to produce? Find it by working backward, and then check it by working forward.

Check. By the definition of surjective (Section 3.3), the goal is: for each integer b, find an integer a with $f(a) = b$, that is, $a + 3 = b$. Working backward from this equation suggests $a = b - 3$. Working forward, $b - 3$ is an integer, because a difference of integers is an integer, and $f(b - 3) = (b - 3) + 3 = b$. So for every integer b, the integer $b - 3$ is a witness that b is reached, and f is surjective.

4.2 The contrapositive and proof by contradiction

Now turn the claim of the previous section around. Its converse (Section 2.1) asks: if n^2 is even, must the integer n be even? Working forward from the hypothesis gives $n^2 = 2w$ for some integer w. But this equation describes n^2, not n. Taking square roots tells us only that $n = \sqrt{2w}$ or $n = -\sqrt{2w}$, and neither expression is visibly twice an integer. The forward direction stalls.

So look at the contrapositive instead (Section 2.5):

If n is not even, then n^2 is not even.

An implication and its contrapositive are equivalent, so proving this statement also proves "if n^2 is even, then n is even." At first the new hypothesis seems no better. "The integer n is not even" tells us only what n is *not*, and there is nothing to calculate with. It would help a great deal to know that an integer which is not even must be odd, because then we could write $n = 2k + 1$ and square it. Section 1.4 stated, without proof, that every integer is either even or odd, but not both. We now prove this. Both halves of the proof use a method called *proof by contradiction*, so we begin with that method.

Proof by contradiction

To prove a claim by contradiction, temporarily suppose that the claim is false, and keep all the original hypotheses. Then reason from this supposition until you reach a *contradiction*: some statement and its negation both follow. If the supposition were true, everything correctly deduced from it and from the hypotheses would be true as well. But a statement and its negation cannot both be true. So the supposition is false, and the claim is true. The last step uses the principle from Section 2.5 that every statement is either true or false: once "the claim is false" has been ruled out, the only possibility left is that the claim is true.

Section 2.5 already contained a short argument of this kind, in its discussion of vacuous truth. Suppose some integer n were both even and odd. Then $n = 2k$ and $n = 2\ell + 1$ for some integers k and ℓ, so $2k = 2\ell + 1$. Subtracting 2ℓ from both sides gives $2(k - \ell) = 1$, and dividing by two gives $k - \ell = 1/2$. Now $k - \ell$ is an integer, because a difference of integers is an integer, while $1/2$ is not an integer, since it lies strictly between the consecutive integers 0 and 1. So the same number would be an integer and not an integer. That is a statement together with its negation, so the supposition is impossible: *no integer is both even and odd.*

Notice how definite the end point of such an argument must be. If the supposition led only to something that "looks unlikely" or "seems strange," nothing would be proved. A proof by contradiction is finished only when it reaches a genuine impossibility. The supposition is also temporary, so mark it clearly, for example with the words "suppose, for contradiction." Inside the argument we may use it freely, but once the contradiction has been reached it is discarded, and nothing deduced from it counts as an established fact afterward.

Every integer is even or odd

The other half, that every integer is even or odd, needs one more fact about the positive integers. We accept it without proof, just as Section 1.8 accepted that every nonnegative real number has exactly one nonnegative square root; it is the third of the starting facts listed in *To the reader*.

Well-ordering property: every nonempty set of positive integers has a smallest member.

For example, the positive integers whose square is larger than 50 form a nonempty set, since $10^2 = 100 > 50$. Its smallest member is 8: the squares of $1, 2, \ldots, 7$ are $1, 4, 9, 16, 25, 36, 49$, none of which is larger than 50, while $8^2 = 64$ is. The property is easy to believe. If a set of positive integers contains some number b, test the numbers $1, 2, \ldots, b$ one at a time and stop at the first one that belongs to the set. The search must stop, at b if not before, and the number where it stops is the smallest member, since every smaller positive integer was tested and rejected. This search is not a proof, however: it takes for granted that counting up from 1 reaches b after finitely many steps, and that is itself a basic fact about the positive integers, of the same kind as the property. Each part of the property matters. The empty set has no smallest member, because it has no members at all. The set of negative integers has no smallest member either, because for every negative integer n, the integer $n - 1$ is negative and smaller. Nor does the set of positive rational numbers, because for every positive fraction r, the fraction $r/2$ is positive and smaller.

Lemma 4.1. *Every integer is either even or odd, and no integer is both.*

Proof. We showed above that no integer is both even and odd. It remains to show that every integer n is even or odd. If $n = 0$, then n is even, since $0 = 2 \cdot 0$.

Next we show that every positive integer is even or odd. Suppose, for contradiction, that some positive integer is neither even nor odd. Then the set of such positive integers is nonempty, so by the well-ordering property it has a smallest member; call it m. The integer m is not 1, because $1 = 2 \cdot 0 + 1$ is odd. So $m \geq 2$, and $m - 1$ is a positive integer smaller than m. Since m is the *smallest* positive integer that is neither even nor odd, the smaller positive integer $m - 1$ does not belong to the set; that is, $m - 1$ is even or odd. If $m - 1 = 2k$ for an integer k, then $m = 2k + 1$, so m is odd. If $m - 1 = 2k + 1$ for an integer k, then $m = 2k + 2 = 2(k + 1)$, so m is even. Either way m is even or odd, which contradicts the way m was chosen. Hence every positive integer is even or odd.

Finally, let n be negative. Then $-n$ is a positive integer, so by the previous paragraph $-n = 2k$ or $-n = 2k + 1$ for some integer k. In the first case $n = -2k = 2(-k)$, so n is even. In the second case $n = -2k - 1 = 2(-k - 1) + 1$, so n is odd. For instance, $7 = 2 \cdot 3 + 1$, and correspondingly $-7 = 2 \cdot (-4) + 1$. □

The contradiction in this proof is of a different kind from the one before it. There, two integers had to satisfy $k - \ell = 1/2$, which no pair of integers can do. Here, the number m turns out to have a property that it was chosen *not* to have. A proof that begins "choose the smallest . . . " often ends this way. The next section uses the same pattern with a smallest denominator, and Chapter 5 uses it to explain why proof by induction works.

Back to the contrapositive

With Lemma 4.1 available, we can finish the argument we started. The next section needs the result twice, so we record it as a lemma.

Lemma 4.2. *For every integer n, the integer n is even if and only if n^2 is even.*

Proof. Let n be an integer. If n is even, then n^2 is even, by the direct proof in Section 4.1.

For the other direction we prove the contrapositive: if n is not even, then n^2 is not even. So suppose n is not even. By Lemma 4.1, n is odd, so $n = 2k + 1$ for some integer k. Then

$$\begin{aligned} n^2 &= (2k+1)(2k+1) \\ &= 4k^2 + 4k + 1 \\ &= 2(2k^2 + 2k) + 1. \end{aligned}$$

Since $2k^2 + 2k$ is an integer, n^2 is odd. By Lemma 4.1 again, an odd integer is not even, so n^2 is not even. This proves the contrapositive. Since an implication and its contrapositive are equivalent, it follows that if n^2 is even, then n is even. □

The contrapositive worked because its hypothesis gave us something to calculate with: once Lemma 4.1 had turned "not even" into "odd," we had the representation $n = 2k + 1$, which we could square. In general, try the contrapositive when the negation of the conclusion is easier to unpack than the hypothesis. When the hypothesis is the easier one, as in Section 4.1, prove the statement directly.

Pause and decide. The proof of Lemma 4.2 used Lemma 4.1 twice. Which half of Lemma 4.1 was used each time?

Check. The first use needs "every integer is even or odd": it turns the hypothesis "n is not even" into "n is odd," which gives the representation $n = 2k + 1$. The second use needs "no integer is both": the calculation shows that n^2 is odd, and we must still conclude that n^2 is not even, which is what the contrapositive promises.

4.3 The square root of two is irrational

Recall from Section 1.8 that a *rational number* is a number that can be written as a fraction a/b, where a and b are integers and $b \neq 0$. A real number that is not rational is called *irrational*. The symbol $\sqrt{2}$ denotes the nonnegative real number whose square is 2. We now prove that $\sqrt{2}$ is irrational.

Theorem 4.3. *The number $\sqrt{2}$ is irrational.*

Why argue by contradiction? The word "irrational" means "not rational," so the theorem says what $\sqrt{2}$ is *not*. A direct proof would have to deal with every possible fraction at once and show that none of them equals $\sqrt{2}$, and it is hard to see where such a proof would begin. Supposing the opposite, that $\sqrt{2}$ is rational, hands us a fraction of integers that equals $\sqrt{2}$, and with its numerator

and denominator we can calculate. This is the same difficulty that the hypothesis "n is not even" caused in Section 4.2. Statements saying that something is impossible, or that no object of some kind exists, are often proved by contradiction for this reason.

The proof chooses a fraction for $\sqrt{2}$ with the smallest possible positive denominator, so it helps to see what such a choice looks like for an ordinary rational number. The number 0.75 can be written as $3/4$, $6/8$, $9/12$, and in infinitely many other ways. The positive integers j for which $0.75 = a/j$ with an integer a are $4, 8, 12, 16, \ldots$, and the smallest of them is 4. Choosing the smallest denominator picks out the fraction $3/4$, which cannot be reduced any further.

Plan. Suppose $\sqrt{2} = a/b$ with b as small as possible. Squaring will force both a and b to be even. Then cancelling a factor of two gives a fraction for $\sqrt{2}$ with a smaller positive denominator, which contradicts the choice of b.

Proof. Suppose, for contradiction, that $\sqrt{2}$ is rational. Then $\sqrt{2} = p/q$ for some integers p and q with $q \neq 0$. If q is positive, this fraction already has a positive denominator. If q is negative, then $p/q = (-p)/(-q)$, and $-q$ is positive. So in either case $\sqrt{2}$ can be written as a fraction of integers with a positive denominator. In other words, the set of positive integers j for which $\sqrt{2} = a/j$ with some integer a is nonempty. By the well-ordering property (Section 4.2), this set has a smallest member; call it b, and let a be an integer with $\sqrt{2} = a/b$.

Squaring both sides of $\sqrt{2} = a/b$ gives $2 = a^2/b^2$, because $(\sqrt{2})^2 = 2$ and, by the rule for multiplying fractions, $(a/b)(a/b) = (a \cdot a)/(b \cdot b)$. Multiplying by b^2 gives $a^2 = 2b^2$. Since b^2 is an integer, a^2 is even. By Lemma 4.2, a is even, so $a = 2c$ for some integer c. Substituting this into $a^2 = 2b^2$ gives

$$(2c)^2 = 2b^2, \qquad \text{that is,} \qquad 4c^2 = 2b^2, \qquad \text{so} \qquad b^2 = 2c^2.$$

Thus b^2 is even, and Lemma 4.2 shows that b is even. Write $b = 2d$ with d an integer. Since b is positive, $d = b/2$ is positive, and $d < b$ because $b = d + d$ and $d > 0$. Cancelling the common factor 2 gives

$$\sqrt{2} = \frac{a}{b} = \frac{2c}{2d} = \frac{c}{d}.$$

So d is a positive integer for which $\sqrt{2} = c/d$ with c an integer, and d is smaller than b. This contradicts the choice of b as the smallest such denominator. Therefore $\sqrt{2}$ is not rational; it is irrational. □

Parity alone gives no contradiction: fractions whose numerator and denominator are both even certainly exist, such as $6/8$. The contradiction comes from parity together with the choice of b. A fraction chosen with the smallest possible positive denominator turned out to be reducible to one with a smaller positive denominator.

Could a computer settle the question instead? A calculator shows $\sqrt{2} \approx 1.41421356$, and computers have produced billions of further digits. But any finite list of digits is the beginning of some fraction: 1.41421356 is exactly the fraction $141421356/100000000$. So no number of computed digits can prove that $\sqrt{2}$ is irrational. The proof above succeeds because it shows that *no* fraction of integers, however large its numerator and denominator, is equal to $\sqrt{2}$.

Pause and decide. Where did the proof use the fact that the denominator b is positive?

Check. In two places. First, the well-ordering property applies only to sets of *positive* integers. If negative denominators were allowed, there would be no smallest one: from $\sqrt{2} = a/b$ with $b > 0$ we also get $\sqrt{2} = (-a)/(-b) = (-2a)/(-2b) = (-3a)/(-3b) = \cdots$, and the denominators $-b, -2b, -3b, \ldots$ become more and more negative without end. Second, at the end we needed $d = b/2$ to be a positive denominator smaller than b. For a negative b this fails; for example, if $b = -4$, then $b/2 = -2$, which is larger than b, not smaller.

4.4 Existence is not uniqueness

The equation $x^2 = 1$ has real solutions, but it does not have a *unique* real solution, that is, exactly one. It has exactly two, 1 and -1. Both of them square to 1, and there are no others: a solution makes $x^2 - 1 = (x - 1)(x + 1)$ equal to zero, and a product of real numbers is zero only when at least one factor is zero, so $x = 1$ or $x = -1$. In general, the statement "there is a unique solution" makes two separate claims:

- *existence*: at least one number satisfies the condition;
- *uniqueness*: any two numbers that satisfy the condition are equal, so there is at most one.

This is the same split we met in Section 3.3, where surjectivity gave every element of the codomain at least one preimage and injectivity gave it at most one. Each claim needs its own argument.

Consider the equation

$$x = \frac{x + 1}{2},$$

where x is a real number. *Existence* is settled by a witness: the number 1 satisfies the equation, because $(1 + 1)/2 = 2/2 = 1$.

For *uniqueness*, suppose that two real numbers x and y both satisfy the equation, so that

$$x = \frac{x + 1}{2} \qquad \text{and} \qquad y = \frac{y + 1}{2}.$$

Subtract the second equation from the first. On the right, the numerators give $(x+1)-(y+1) = x-y$, because the two constant terms 1 cancel. So

$$x - y = \frac{x - y}{2}.$$

Multiplying both sides by two gives $2(x - y) = x - y$, and subtracting $x - y$ from both sides gives $x - y = 0$. Hence $x = y$. Together, the two parts show that the equation has exactly one real solution, namely 1.

Notice that the uniqueness argument never solves the equation. It takes two hypothetical solutions and shows that they cannot differ. Comparing two solutions can be simpler than solving, because whatever the two equations share, such as the constant terms here, cancels when they are subtracted. Chapter 12 turns this comparison into the uniqueness part of the contraction theorem, a general

result about repeating a map until the output settles down; that chapter begins with the same equation $x = (x + 1)/2$.

A uniqueness argument on its own proves only that there is *at most one* solution, and that is also true of an equation with no solutions at all. For example, consider the equation $2x = 1$, where x is required to be an integer. If x and y are integers with $2x = 1$ and $2y = 1$, then subtracting gives $2(x - y) = 0$, so $x = y$. This argument is correct, yet the equation has no integer solution: the only number with $2x = 1$ is $x = 1/2$, which is not an integer. That is why the existence check for $x = (x + 1)/2$, substituting $x = 1$, cannot be skipped.

Be careful, too, with the phrase "the solution." The word "the" quietly claims that there is exactly one. A proof that begins "let x be the solution of $x^2 = 1$" has already gone wrong, because there are two solutions; any conclusion it reaches, such as "so x is positive," may be true of one solution and false of the other. Before writing "let x be the solution," make sure that existence and uniqueness have both been established. If they have not, say what you are really assuming, for example "suppose x is a solution" or "if a solution exists, then . . . ." Otherwise the main difficulty of the problem can slip unnoticed into the first sentence of the proof.

Pause and decide. Try the comparison argument on the equation $x = x$, where x is a real number. What happens, and why is that the right outcome?

Check. If $x = x$ and $y = y$, subtracting gives $x - y = x - y$. This is true for all real numbers x and y, so it gives no information about whether $x = y$. That is as it should be: every real number solves $x = x$, so uniqueness is false, and no correct argument could prove it.

4.5 Reading a long proof as a sequence of smaller promises

Long proofs are usually assembled from shorter results: lemmas, propositions, and theorems, whose roles were described in Section 1.5. Each such result works like a small promise: if you supply its hypotheses, it delivers its conclusion. So whenever a proof, whether in a book, in a paper, or from an assistant, uses an earlier result, stop and identify three things. What does the result need? Why does the proof have it at this point? What does the result give back?

Here is that check carried out for the proof of Theorem 4.3, which uses one starting fact once and one lemma twice.

Result used	What it needs	Why it holds here	What it gives
Well-ordering property	a nonempty set of positive integers	the assumed fraction for $\sqrt{2}$, with both signs changed if necessary, supplies a positive denominator	a smallest positive denominator b
Lemma 4.2	an integer whose square is even	a is an integer, and $a^2 = 2b^2$ is twice the integer b^2	a is even
Lemma 4.2 again	an integer whose square is even	b is an integer, and $b^2 = 2c^2$ is twice the integer c^2	b is even

In each row the third column must be checked, not assumed. A sentence such as "by a standard result, a is even" makes this check impossible: the reader cannot tell which result is meant, so cannot see what it needs or confirm that the proof has it. A careful proof names the result and shows that its hypotheses hold.

When you check why a proof has some fact at a given point, notice also whether that fact rests on a temporary assumption (Section 4.2). Inside the proof of Theorem 4.3 we deduced that a and b are both even, but only under the supposition that $\sqrt{2}$ is rational, which is what supplied the fraction a/b. Those numbers turned out not to exist, so the deductions about them must not be used outside the proof.

> **Habit 4: ask for a local contract, not just more words**
>
> Habit 2 (Section 2.6) asked for the contract of a whole statement. When a single step of a proof is hard to follow, whether the proof is your own, a book's, or an assistant's, ask for the contract of that step. What is known at this point? What is the next goal? Which exact result is being used, and why do its hypotheses hold here? Which quantity is being constructed, and from what? These questions locate the missing mathematics more reliably than a request to make the explanation "simpler."

Here is the contract of one step, from the proof of Lemma 4.1: "If $m - 1 = 2k + 1$ for an integer k, then $m = 2k + 2 = 2(k + 1)$, so m is even." Known at this point: $m - 1 = 2k + 1$, where k is an integer. Next goal: m is even or odd. Result used: none; the step needs only Definition 1.1, which says that m is even as soon as m is twice some integer. Quantity constructed: the witness $k + 1$, built from k; it is an integer because a sum of integers is an integer. With these four answers written down, each part of the step can be checked on its own.

Exercises

Exercises 4.1–4.4 are core exercises, and Exercises 4.5 and 4.6 are transfer exercises. Complete solutions are in Appendix A, under "Week 4."

Exercise 4.1. Give a direct proof that if a and b are even integers, then ab is divisible by four.

Exercise 4.2. Let n be an integer. Prove, by proving the contrapositive, that if $3n + 1$ is even, then n is odd.

Exercise 4.3. Let r be a rational number and u an irrational number. Prove that $r + u$ is irrational. Your proof will need the fact that the difference of two rational numbers is rational; explain why this fact holds.

Exercise 4.4. Prove that the equation $x = 3x - 4$ has exactly one real solution. Give the existence part and the uniqueness part separately.

Exercise 4.5. An assistant is asked to solve the equation $x^2 = 2$ for a real number x, and answers "so $x = \sqrt{2}$." What has the assistant missed? Find all real solutions of $x^2 = 2$, and prove that your list is complete.

Exercise 4.6. Prove that $\sqrt{8}$ is irrational in two ways. First, deduce it from Theorem 4.3. Second, without using Theorem 4.3, adapt its proof so that it works for $\sqrt{8}$, and explain how the parity steps have to change.

CHAPTER 5

Induction, invariants, and progress

This week

Prove a statement about every natural number by proving a first case and one general step, and learn strong induction, whose step may use all the earlier cases. Use an invariant to show what a process can never change, and a nonnegative integer that keeps decreasing to show that a process must stop; Euclid's algorithm for the greatest common divisor uses both. See how proving a more general statement can make an induction proof simpler.

5.1 From a few sums to every sum

Add up the odd positive integers, one at a time:

$$1 = 1, \qquad 1 + 3 = 4, \qquad 1 + 3 + 5 = 9, \qquad 1 + 3 + 5 + 7 = 16.$$

The totals $1, 4, 9, 16$ are the squares $1^2, 2^2, 3^2, 4^2$. The nth odd positive integer is $2n - 1$ (for $n = 1, 2, 3$ this gives $1, 3, 5$), so the pattern suggests that

$$1 + 3 + \cdots + (2n - 1) = n^2 \qquad \text{for every } n \geq 1.$$

Section 1.8 noticed this pattern and asked whether it always holds. As Section 1.1 explained, a few examples cannot settle that. Even a computer check of the first thousand values of n would tell us only about those thousand values. What we need is a reason why the pattern can never break.

Here is the reason. Compare the statement for n with the statement for $n + 1$. The left side gains one new term, the next odd number, which is $2(n + 1) - 1 = 2n + 1$. The right side changes from n^2 to $(n + 1)^2$, an increase of

$$(n + 1)^2 - n^2 = n^2 + 2n + 1 - n^2 = 2n + 1.$$

So both sides increase by exactly the same amount, $2n + 1$. If the two sides are equal for some value of n, they are still equal for the next value. They are equal for $n = 1$, so they are equal for $n = 2$, hence for $n = 3$, and so on forever. This way of reasoning is called *induction*.

The principle of induction

Let $P(n)$ be a statement about a natural number n that becomes true or false once a value of n is chosen, as in Section 2.4. For instance, $P(n)$ might be the statement "$1 + 3 + \cdots + (2n - 1) = n^2$." (What the left side means when $n = 0$ is explained below.) To prove that $P(n)$ is true for every natural number n, it is enough to prove two things:

1. the *base case*: $P(0)$ is true;
2. the *induction step*: for every natural number k, if $P(k)$ is true, then $P(k+1)$ is true.

In the induction step we let k stand for a natural number but do not say which one, and we assume that $P(k)$ is true. This temporary assumption is called the *induction hypothesis*. From it we must deduce $P(k+1)$. Because the argument never uses a particular value of k, it works for every value of k at once.

Why do these two facts give $P(n)$ for every n? The base case gives $P(0)$. The induction step with $k = 0$ turns $P(0)$ into $P(1)$; with $k = 1$ it turns $P(1)$ into $P(2)$; and so on. To reach $P(5)$, for example, we use the induction step five times, with $k = 0, 1, 2, 3, 4$. In the same way, every natural number n is reached after n uses of the step. Section 5.2 gives a more careful version of this argument and explains why assuming $P(k)$ is not the same as assuming what we want to prove.

Induction does not have to start at 0. If a statement is claimed only for $n \geq 1$, prove $P(1)$ as the base case and prove the induction step for every $k \geq 1$. The same chain then reaches $1, 2, 3, \ldots$. Any other starting point works the same way: to prove $P(n)$ for every integer $n \geq 2$, say, prove $P(2)$ and prove the induction step for every $k \geq 2$.

Starting at zero

We could prove the formula by an induction that starts at $n = 1$, where the base case is $1 = 1^2$. But zero is a natural number in this book, and the formula also makes sense at $n = 0$ once we agree what a sum with no terms means; this agreement is used throughout the book. In sigma notation the left side is $\sum_{j=1}^{n}(2j-1)$. For $n = 0$ this is $\sum_{j=1}^{0}(2j-1)$, where the index j is supposed to run from 1 up to 0. There are no such values of j, so this sum has no terms at all. A sum with no terms is called an *empty sum*, and its value is defined to be 0. This is the only sensible choice: adding no numbers to a running total should leave the total unchanged, and the only number whose addition changes nothing is 0.

The agreement also keeps the rule "the sum of the first $k+1$ terms is the sum of the first k terms plus the last term" true when $k = 0$: the one-term sum $\sum_{j=1}^{1}(2j-1)$ equals $0 + 1 = 1$. The proof of Theorem 5.1 uses this rule in its induction step, including the step from 0 to 1.

Watching one induction step

Before writing the general step, watch it happen at $k = 3$. The formula at 3 says $1 + 3 + 5 = 3^2 = 9$. The next sum is the old sum plus the next odd number, 7:

$$1 + 3 + 5 + 7 = (1 + 3 + 5) + 7 = 9 + 7 = 16 = 4^2.$$

The induction step in the proof below does exactly the same thing with k in place of 3.

Theorem 5.1. *For every $n \in \mathbb{N}$,*

$$\sum_{j=1}^{n}(2j-1) = n^2.$$

For $n = 0$ the left side is the empty sum, which is zero.

Proof. We use induction on n.

Base case. For $n = 0$ the left side is the empty sum, which is 0, and the right side is $0^2 = 0$.

Induction step. Let k be a natural number, and assume the induction hypothesis

$$\sum_{j=1}^{k}(2j-1) = k^2.$$

We must show that $\sum_{j=1}^{k+1}(2j-1) = (k+1)^2$. Split the sum for $k+1$ into its first k terms and its last term, the term with $j = k+1$:

$$\begin{aligned}\sum_{j=1}^{k+1}(2j-1) &= \sum_{j=1}^{k}(2j-1) + \big(2(k+1)-1\big) \\ &= k^2 + \big(2(k+1)-1\big) && \text{by the induction hypothesis} \\ &= k^2 + 2k + 1 \\ &= (k+1)^2.\end{aligned}$$

This is the formula with $k+1$ in place of n. By induction, the formula holds for every natural number n. □

If an induction argument puzzles you, do what we did above for $k = 3$: rewrite its induction step with a specific number, then return to the general k and check that each line would work just as well for any other value.

5.2 Why induction is not circular, and when strong induction helps

A first reaction to induction is often suspicion: the induction step *assumes* the formula at k, so are we assuming what we want to prove? We are not. The induction step proves a conditional statement, "*if* the formula holds at k, *then* it holds at $k+1$," and it proves this for every k at once. It never claims that the formula holds at any particular k. The base case supplies the first actual truth, and the conditional statement then carries that truth forward one step at a time. Think of an endless row of dominoes. The induction step says that each domino, if it falls, knocks over the next one. The base case pushes the first domino.

Here is a more careful way to see that no natural number can slip through. It is a proof by contradiction, like the proofs in Chapter 4. Suppose we have proved the base case $P(0)$ and the induction step, and imagine that P nevertheless fails for some natural number. Then the set of natural numbers at which P fails is nonempty. Every nonempty set of natural numbers has a smallest member. This is the well-ordering property of Section 4.2, which was stated there for positive integers; allowing 0 changes nothing, because if 0 belongs to the set it is the smallest member, and otherwise the set consists of positive integers. Call the smallest member m. Now look at what we know about m:

- $m \neq 0$, because $P(0)$ was proved directly. So $m \geq 1$, and $m - 1$ is a natural number.

- $P(m-1)$ is true, because $m-1$ is smaller than m, and m is the *smallest* number where P fails.
- The induction step, applied with $k = m-1$, turns $P(m-1)$ into $P(m)$. So $P(m)$ is true.

So $P(m)$ is both false, by the choice of m, and true, by the last bullet. This contradiction shows that P fails nowhere: $P(n)$ is true for every natural number n. The pattern of this argument, "choose a smallest counterexample and derive a contradiction," is called the *minimal-counterexample* method. The proof of Lemma 4.1 used it to show that every positive integer is even or odd, and many other proofs begin the same way.

Strong induction

Sometimes the step to $k+1$ needs more than the case immediately before it. *Strong induction* lets us assume all of $P(0), P(1), \ldots, P(k)$ when proving $P(k+1)$. To prove $P(n)$ for every natural number n by strong induction, prove

1. the base case: $P(0)$ is true; and
2. the strong induction step: for every natural number k, if $P(0), P(1), \ldots, P(k)$ are all true, then $P(k+1)$ is true.

The minimal-counterexample argument shows that this works too. If m were the smallest natural number where P fails, then $m \neq 0$ by the base case. Each of $P(0), P(1), \ldots, P(m-1)$ would be true, because each of these numbers is smaller than m. The strong induction step with $k = m-1$ would then give $P(m)$, a contradiction.

Like ordinary induction, strong induction can start at a number other than 0. To prove $P(n)$ for every integer $n \geq 2$, say, prove $P(2)$, and prove for every $k \geq 2$ that if $P(2), P(3), \ldots, P(k)$ are all true, then $P(k+1)$ is true.

A classic example shows why the extra strength helps. Recall from Section 2.1 that an integer n is divisible by an integer d when $n = dk$ for some integer k. A positive integer d with this property is called a *divisor* of n. An integer $p \geq 2$ is *prime* if its only positive divisors are 1 and p. For example, 2, 3, 5 and 7 are prime, but 12 is not, since 3 is a divisor of 12. By a *product of primes* we mean a product of two or more primes, which need not be different, such as $12 = 2 \cdot 2 \cdot 3$.

Proposition 5.2. *Every integer $n \geq 2$ is a prime or a product of primes.*

Proof. We use strong induction on n, starting at 2.

Base case. The number 2 is prime, because its only positive divisors are 1 and 2.

Induction step. Let $n \geq 3$, and assume that every integer m with $2 \leq m < n$ is a prime or a product of primes. This assumption about all the smaller cases is the induction hypothesis; here n plays the role of $k+1$ in the strong induction step.

If n is prime, there is nothing to prove. Otherwise n has a positive divisor a other than 1 and n, so $n = ab$ for some integer b. We check that both a and b lie in the range covered by the assumption, that is, between 2 and $n-1$:

- Since a and n are positive, $b = n/a$ is positive; as an integer, $b \geq 1$. Hence $n = ab \geq a$.

Together with $a \neq 1$ and $a \neq n$, this gives $2 \leq a \leq n-1$.

- Since $a \geq 2$, we have $b = n/a \leq n/2 < n$. Since $a < n$, we have $b = n/a > 1$, so $b \geq 2$. Hence $2 \leq b \leq n-1$.

By the assumption, a is a prime or a product of primes, and so is b. Multiplying these together writes $n = ab$ as a product of primes. □

For example, $12 = 3 \cdot 4$. The number 3 is prime and $4 = 2 \cdot 2$, so $12 = 3 \cdot 2 \cdot 2$. Ordinary induction would let us assume the result only for $n-1$, and that is useless here: for $n = 12$ we need the cases 3 and 4, not 11. We cannot know in advance which smaller numbers a and b will turn up, so we need the result for all of them. The second proof of Hall's theorem, in Section 9.4, uses strong induction for the same reason: it splits a problem into two smaller problems whose sizes are not known in advance.

Notice also that the proof checked that a and b really are smaller than n and at least 2 before using the assumption about them. Every use of an induction hypothesis needs a check of this kind.

5.3 A sequence defined by a recurrence

A *sequence* is an endless list of numbers $x_0, x_1, x_2, \ldots$, one for each natural number. Each x_n is called a *term* of the sequence. The subscript n gives its position in the list, counting from 0: x_0 is the first term, x_1 the second, and in general x_n is the term in position n. A *recurrence* defines a sequence step by step: it gives the first term and a rule that computes each later term from the earlier ones. Let

$$x_0 = 0 \qquad \text{and} \qquad x_{n+1} = \frac{x_n + 1}{2} \quad \text{for every } n \in \mathbb{N}.$$

Read the notation carefully: x_{n+1} is the name of the term after x_n, whereas $x_n + 1$ means one added to the current term. In words, the rule says: to get the next term, add one to the current term and halve the result. Starting from $x_0 = 0$,

$$x_1 = \frac{0+1}{2} = \frac{1}{2}, \qquad x_2 = \frac{\frac{1}{2}+1}{2} = \frac{3}{4}, \qquad x_3 = \frac{\frac{3}{4}+1}{2} = \frac{7}{8}.$$

The terms seem to approach 1. They fall short of 1 by 1, $\frac{1}{2}$, $\frac{1}{4}$ and $\frac{1}{8}$, each gap half the one before. This suggests the formula

$$x_n = 1 - 2^{-n},$$

where $2^{-n} = 1/2^n$ as in Section 1.3. We prove it by induction on n.

For the base case, take $n = 0$. The formula gives $1 - 2^{-0} = 1 - 2^0 = 1 - 1 = 0$, which is x_0. For the induction step, let n be a natural number and assume that $x_n = 1 - 2^{-n}$. (Here the letter n plays the role that k played in Section 5.1; the name of the letter does not matter.) Substitute the

induction hypothesis into the rule for the next term:

$$\begin{aligned} x_{n+1} &= \frac{x_n + 1}{2} && \text{the rule for the next term} \\ &= \frac{(1 - 2^{-n}) + 1}{2} && \text{induction hypothesis} \\ &= \frac{2 - 2^{-n}}{2} \\ &= 1 - \frac{2^{-n}}{2} \\ &= 1 - 2^{-(n+1)}. \end{aligned}$$

The last equality uses $2^{-n}/2 = 1/(2^n \cdot 2) = 1/2^{n+1} = 2^{-(n+1)}$. This is the formula at $n + 1$, so by induction it holds for every natural number n.

The formula tells us more than the first few terms did. The gap between x_n and 1 is exactly $1 - x_n = 2^{-n}$. This gap halves at every step, so the terms come as close to 1 as we like. For instance, $x_{10} = 1 - 1/1024$, which is within 0.001 of 1, and $x_{20} = 1 - 1/2^{20}$ is within one millionth of 1, since $2^{20} = 1048576$. Yet the gap is never zero, so no term ever equals 1.

Tracking the error instead

Why is 1 the right number to compare with? If some term were exactly 1, the next term would be $(1 + 1)/2 = 1$ again: the rule leaves 1 unchanged. A number that an update rule leaves unchanged is called a *fixed point* of the rule. Section 4.4 showed that 1 is the only real number x with $x = (x + 1)/2$, so 1 is the only fixed point of this rule.

This suggests a more revealing way to organize the calculation. Instead of tracking x_n itself, track its *error* $x_n - 1$, the difference between the current term and the fixed point. The error is the gap $1 - x_n$ studied above, with its sign reversed: it is negative when $x_n < 1$, and its absolute value $|x_n - 1|$ is the distance from x_n to 1. Subtract 1 from both sides of the rule:

$$x_{n+1} - 1 = \frac{x_n + 1}{2} - 1 = \frac{x_n + 1 - 2}{2} = \frac{x_n - 1}{2}.$$

In words: each update exactly halves the error. Nothing in this calculation used the starting value $x_0 = 0$. So if we keep the same rule but allow *any* real starting value x_0, induction gives

$$x_n - 1 = 2^{-n}(x_0 - 1) \qquad \text{for every } n \in \mathbb{N}.$$

At $n = 0$ both sides equal $x_0 - 1$, because $2^0 = 1$. If the formula holds at n, halving both sides and using the rule $x_{n+1} - 1 = (x_n - 1)/2$ gives it at $n + 1$. Exercise 5.5 asks you to write out this induction in full. With $x_0 = 0$ the formula becomes $x_n - 1 = -2^{-n}$, which is the formula $x_n = 1 - 2^{-n}$ proved above.

This more general statement covers every starting value at once, yet its induction step is shorter than the one for $x_n = 1 - 2^{-n}$. It records only what each step does, namely halve the error, while the first formula mixed that fact with the particular starting value 0. So when an induction proof

gets tangled, try proving a more general statement instead; its induction step can be simpler. We will meet the same idea in the contraction theorem of Chapter 12, where the distance to a fixed point shrinks at every step by at least a fixed factor.

5.4 An invariant shows what cannot change

Many problems describe a process: there is a collection of possible *states*, and there are *legal moves*, each of which turns one state into another. An *invariant* of such a process is a quantity or property that every legal move leaves unchanged.

Here is an example. A row contains n signs, each a plus or a minus, where $n \geq 2$. The states of this process are the rows of n signs. A legal move picks two different positions in the row and flips the sign in each: a plus becomes a minus, and a minus becomes a plus. Can we start from a row of all plus signs and reach a row with exactly one minus sign?

To find an invariant, replace each plus sign by the number 1 and each minus sign by -1, and multiply all the numbers in the row together. Call the result the *product of the row*. Flipping one sign multiplies that entry by -1, and therefore multiplies the whole product by -1. A legal move flips two signs, so it multiplies the product by $(-1)(-1) = 1$: the product never changes. For instance, the row $+\ -\ +\ +$ has product $1 \cdot (-1) \cdot 1 \cdot 1 = -1$. Flipping the first two signs gives $-\ +\ +\ +$, whose product is still -1.

A row of all plus signs has product 1. Call a row *reachable* if some sequence of legal moves turns the all-plus row into it. Induction on the number of moves shows that every reachable row also has product 1. To set this up, let $P(k)$ be the statement "every row obtained from the all-plus row by k legal moves has product 1." The base case $P(0)$ holds, because after 0 moves the row is still all plus signs. For the induction step, assume $P(k)$ and take any row obtained by $k+1$ moves. It comes from a row obtained by k moves, whose product is 1 by the induction hypothesis, followed by one more move, which does not change the product. So its product is 1, and $P(k+1)$ holds. A row with exactly one minus sign has product -1, so it can never be reached, however many moves we make.

Notice what this proof did not do: it never examined sequences of moves one by one. That would be hopeless, because moves can be repeated, so there are infinitely many sequences. Instead it found a quantity that no sequence of moves can change.

The product also has a simple meaning. If the row has m minus signs, the plus signs contribute factors of 1, so the product is $(-1)^m$. This is 1 when m is even and -1 when m is odd. Seen this way, a legal move changes the number of minus signs by $+2$ (two plus signs flipped), by -2 (two minus signs flipped) or by 0 (one of each). The number of minus signs starts at 0, so it stays even.

Here the invariant is the only obstacle: every row with an even number of minus signs *can* be reached from the all-plus row. Choose the positions that should hold minus signs, pair them up, and flip one pair at a time. In general, though, an invariant can only show that a state is unreachable; to show that a state can be reached, we must exhibit moves that actually get there.

An invariant on its own also does not show that a process comes to an end. Nothing in the sign-flipping rules ever forces us to stop: we can flip the first two signs, flip them back, and repeat

forever, and every row along the way has product 1. For a process that is meant to finish, we need a separate argument that it makes *progress*. The next section studies such a process, Euclid's algorithm, and shows both kinds of argument working together.

5.5 Euclid's algorithm: why it is right and why it stops

Recall from Section 5.2 that a positive integer d is a divisor of an integer a when $a = dk$ for some integer k. We also say that d *divides* a, and we write $d \mid a$. Do not confuse $d \mid a$ with the fraction d/a: the first is a statement, which is true or false, and the second is a number. For example, $6 \mid 84$ is true, because $84 = 6 \cdot 14$, while the fraction $6/84$ is the number $1/14$. On the other hand, 5 does not divide 84, because $84 = 5k$ would force $k = 84/5 = 16.8$, which is not an integer. Every positive integer d divides 0, because $0 = d \cdot 0$.

Let a and b be nonnegative integers that are not both zero. A *common divisor* of a and b is a positive integer that divides both of them. Their *greatest common divisor*, written $\gcd(a, b)$, is the largest common divisor. For example, the positive divisors of 12 are $1, 2, 3, 4, 6, 12$ and those of 18 are $1, 2, 3, 6, 9, 18$, so the common divisors of 12 and 18 are $1, 2, 3, 6$, and $\gcd(12, 18) = 6$. The definition has to exclude $a = b = 0$: every positive integer divides 0, so 0 and 0 have no largest common divisor.

Why is there always a largest common divisor? The set of common divisors is not empty, because 1 divides every integer. It is also finite. At least one of a and b is positive; say $a > 0$ (otherwise exchange the two letters in what follows). If d divides a, then $a = dk$ where $k = a/d$ is positive, because a and d are. Since k is a positive integer, $k \geq 1$, and so $a = dk \geq d$. Thus every common divisor lies among $1, 2, \ldots, a$, and a nonempty finite set of integers has a largest member.

Division with remainder

Let a be any integer and b a positive integer. Then there are unique integers q and r with

$$a = qb + r, \qquad 0 \leq r < b.$$

The integer q is the *quotient*; when $a \geq 0$, it counts how many whole copies of b fit into a. The integer r is the *remainder*, what is left over. The remainder is also written $a \bmod b$.

For example, the largest multiple of 5 that is at most 17 is $15 = 3 \cdot 5$, so $17 = 3 \cdot 5 + 2$: the quotient is 3 and the remainder is $17 \bmod 5 = 2$. The equation $17 = 2 \cdot 5 + 7$ is also true, but it is not division with remainder, because 7 is not less than 5. For a negative number, the largest multiple of 5 that is at most -7 is $-10 = (-2) \cdot 5$, so $-7 = (-2) \cdot 5 + 3$ and $(-7) \bmod 5 = 3$. The remainder is never negative.

Division with remainder is a familiar fact of integer arithmetic, and the well-ordering property shows where q and r come from. Let S be the set of all numbers of the form $a - qb$, with q an integer, that are at least 0. For $a = 17$ and $b = 5$, for example, $S = \{2, 7, 12, 17, 22, \ldots\}$. The set S is never empty. If $a \geq 0$, then $q = 0$ puts a itself in S. If $a < 0$, then $q = a$ puts $a - ab = (-a)(b - 1)$ in S, because $-a$ is positive and $b - 1 \geq 0$. So S is a nonempty set of natural numbers, and by the well-ordering property (in the form of Section 5.2, which allows 0) it has a

smallest member. Call it r, and let q be an integer with $r = a - qb$. Then $a = qb + r$ and $r \geq 0$. Also $r < b$: if $r \geq b$, then $r - b = a - (q + 1)b$ would also belong to S and be smaller than r, which is impossible. Making $a - qb$ as small as possible is the same as making the multiple qb as large as possible without exceeding a, and that is how the examples above found q: for $a = 17$ and $b = 5$ the smallest member of S is $2 = 17 - 3 \cdot 5$.

No other pair of integers works. Suppose also that $a = q'b + r'$ with $0 \leq r' < b$. Subtracting the equation $a = qb + r$ from this one gives $0 = (q' - q)b + (r' - r)$, so $r' - r = (q - q')b$. Since r and r' both lie between 0 and $b - 1$, their difference lies between $0 - (b - 1)$ and $(b - 1) - 0$; in particular $-b < (q - q')b < b$. Dividing by the positive number b shows that the integer $q - q'$ lies strictly between -1 and 1, so $q - q' = 0$. Hence $q' = q$, and then $r' = a - q'b = a - qb = r$.

For $b = 2$ the remainder is 0 or 1, so the fact says that every integer can be written as $2q$ or $2q + 1$, that is, every integer is even or odd, as Lemma 4.1 showed.

One step of Euclid's algorithm

An *algorithm* is a list of precise instructions that can be followed mechanically, one step at a time, like the school method for long division. Euclid's algorithm, which appears in Euclid's *Elements*, written around 300 BC, computes greatest common divisors by repeating a single step: replace the pair (a, b) by the pair (b, r), where $r = a \bmod b$. For example, $(84, 30)$ is replaced by $(30, 24)$, because $84 = 2 \cdot 30 + 24$. The common divisors of 84 and 30 are $1, 2, 3, 6$, and so are the common divisors of 30 and 24. This is no coincidence.

Lemma 5.3. *Let $a \geq 0$ and $b > 0$ be integers, and write $a = qb + r$ with integers q, r and $0 \leq r < b$. Then the common divisors of a and b are exactly the common divisors of b and r. Consequently* $\gcd(a, b) = \gcd(b, r)$.

Proof. Let d be a positive integer. First suppose that d divides both a and b, say $a = ds$ and $b = dt$ with integers s and t. Then

$$r = a - qb = ds - qdt = d(s - qt).$$

Since $s - qt$ is an integer, d divides r. So every common divisor of a and b is a common divisor of b and r.

Conversely, suppose that d divides both b and r, say $b = dt$ and $r = du$ with integers t and u. Then

$$a = qb + r = qdt + du = d(qt + u),$$

so d divides a as well. So every common divisor of b and r is a common divisor of a and b.

The two directions together show that the pairs (a, b) and (b, r) have exactly the same set of common divisors. Both greatest common divisors are defined, because $b > 0$. Each is the largest member of the same set, so they are equal. □

Both directions are needed. With only the first, the pair (b, r) might have an extra common divisor, larger than every common divisor of a and b, and then $\gcd(b, r)$ would be bigger than $\gcd(a, b)$.

The algorithm, and why its answer is right

Euclid's algorithm starts with a pair (a, b) of nonnegative integers, not both zero. To keep the starting pair in view, we call the current pair (u, v).

1. Start with $(u, v) = (a, b)$.
2. If $v = 0$, stop and output u.
3. Otherwise replace (u, v) by $(v,\ u \bmod v)$ and go back to step 2.

For $(a, b) = (84, 30)$ the divisions are

$$84 = 2 \cdot 30 + 24, \qquad 30 = 1 \cdot 24 + 6, \qquad 24 = 4 \cdot 6 + 0.$$

The successive pairs are $(84, 30)$, $(30, 24)$, $(24, 6)$, $(6, 0)$, and the algorithm outputs 6. If the first number is smaller than the second, the first step simply swaps them. For $(30, 84)$, for example, $30 = 0 \cdot 84 + 30$, so the next pair is $(84, 30)$.

Why is the output the greatest common divisor? Every number that appears in the pairs is nonnegative, because a and b are and a remainder is never negative. A step is taken only when v is positive, so Lemma 5.3, applied with u and v in place of a and b, covers every step. By the lemma, each step leaves the greatest common divisor of the current pair unchanged. By induction on the number of steps, $\gcd(u, v) = \gcd(a, b)$ holds for every pair that the algorithm produces. This is the invariant of the algorithm.

When the algorithm stops, the current pair has the form $(g, 0)$, and $g > 0$. Indeed, if no step was taken, then $b = 0$ and $g = a$, which is positive because a and b are not both zero. If at least one step was taken, then g was the second number of the previous pair, which was positive. Since every positive integer divides 0, the common divisors of g and 0 are exactly the positive divisors of g, and the largest of these is g itself. So $\gcd(g, 0) = g$, and by the invariant, $g = \gcd(a, b)$.

Why the algorithm stops

The invariant shows that *if* the algorithm stops, its answer is correct. It does not show that the algorithm ever stops. For that we need a measure of progress. Watch the second number v of the current pair. Each step replaces it by $u \bmod v$, a remainder on division by v, and such a remainder is less than v. So the second number is a nonnegative integer that strictly decreases at every step.

An integer that decreases goes down by at least 1: if w and v are integers with $w < v$, then $w \le v - 1$. The second number starts at b, so by induction on s, after s steps it is at most $b - s$. It is also never negative, so $0 \le b - s$, that is, $s \le b$. Hence at most b steps can occur. The algorithm takes another step whenever the second number is positive, so within b steps the second number must reach 0, and then the algorithm stops and outputs the first number. For $(84, 30)$ the second numbers were $30, 24, 6, 0$: the bound allows at most 30 steps, and in fact 3 were enough. The same reasoning shows that a strictly decreasing list of nonnegative integers cannot go on forever: if it starts at c, it has at most $c + 1$ entries.

Pause and decide. Does a strictly decreasing nonnegative real quantity prove that a process stops?

Check. No. The gaps $1 - x_n = 2^{-n}$ in Section 5.3, namely $1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$, decrease forever and remain positive. The termination proof for Euclid's algorithm depends on the decreasing quantity being a nonnegative *integer*. An integer that decreases must drop by at least 1 each time, so it cannot keep decreasing forever without becoming negative. A real number can decrease by smaller and smaller amounts and never run out of room.

> **Habit 5: separate preservation from progress**
>
> For a process that repeats one step, ask two separate questions. What stays true after every step? And what quantity measures progress: a nonnegative integer that must decrease, or an error that must shrink? A preserved property can prove that an answer is correct without proving that the process ever stops. A quantity that keeps shrinking, like an error that halves at every step, can come as close to zero as we like without ever reaching it; it shows that the process approaches the answer, not that the process stops.

Exercises

Exercises 5.1–5.4 are core exercises, and Exercises 5.5 and 5.6 are transfer exercises. Complete solutions are in Appendix A, under "Week 5."

Exercise 5.1. Prove by induction that $\sum_{j=1}^{n} j = n(n+1)/2$ for every $n \in \mathbb{N}$. Write out every line of algebra in the induction step.

Exercise 5.2. Let $y_0 = 3$ and $y_{n+1} = 2y_n$ for every $n \in \mathbb{N}$. Compute y_1, y_2 and y_3, guess a formula for y_n, and prove your formula by induction.

Exercise 5.3. Run Euclid's algorithm on the pair $(1071, 462)$. Record each quotient and remainder, the successive pairs, and the greatest common divisor that the algorithm outputs.

Exercise 5.4. A row of five signs starts with all five signs plus. As in Section 5.4, a legal move flips the signs in two different positions. Can some sequence of legal moves produce a row with exactly three minus signs? Justify your answer without listing sequences of moves.

Exercise 5.5. For the recurrence $x_{n+1} = (x_n + 1)/2$ with an arbitrary real starting value x_0, prove the error formula $x_n - 1 = 2^{-n}(x_0 - 1)$ for every $n \in \mathbb{N}$ by induction. Write out both the base case and the induction step in full. Then explain why the error $x_n - 1$ is never exactly zero when $x_0 \neq 1$.

Exercise 5.6. A proof establishes $P(0)$, and it shows that for every natural number k, if $P(k)$ is true, then $P(k+2)$ is true. For which natural numbers n does it follow that $P(n)$ is true? Which additional base case would give $P(n)$ for every natural number n? Explain why.

CHAPTER 6

Sameness, symmetry, and structure

This week

Recognize when two different descriptions name the same object, and say exactly which differences are being ignored. Check that a rule defined on equivalence classes does not depend on which representatives are chosen. Identify maps that preserve a structure, and see why the symmetries of an object form a group. Use symmetry to simplify a problem without silently discarding cases. Compare objects with a partial order, and tell a maximal element from a maximum.

6.1 Which differences matter?

The fractions $1/2$ and $2/4$ look different, but they name the same rational number. Here is a second example. Three people, Alice, Ben, and Chen, are linked by acquaintance: Alice knows Ben and Ben knows Chen, but Alice and Chen have never met. If we replace Alice, Ben, and Chen by the numbers 1, 2, 3, in that order, the description becomes "1 is linked to 2, and 2 is linked to 3." The labels have changed, but the pattern of connections has not. (Chapter 7 studies such patterns under the name *graphs*.) Finally, think of a necklace with three beads, two red and one blue. Reading the colors around the necklace gives red, red, blue, or red, blue, red, or blue, red, red, depending on the bead where we start. The written lists differ, yet it is the same necklace.

In each example we have chosen to ignore certain differences: how a fraction is written, what the people are called, where we start reading. Such a choice should be stated precisely, because whether two things count as "the same" depends on the question being asked. If the question is "what is the numerator?", then $1/2$ and $2/4$ give different answers, and one cannot be replaced by the other. Saying "these are the same" without saying in what sense can quietly change the problem.

Relations and equivalence relations

To describe sameness precisely, we first need the more general idea of a relation. A *relation* on a set A is a rule that tells us, for every ordered pair (x, y) of elements of A, whether x is related to y. We write $x \sim y$, read "x is related to y," when it is. Two familiar examples are the order $x < y$ on the integers and the relation "x has the same birth month as y" on a set of people. The pair is ordered because the answer can depend on the order: $1 < 2$ is true, but $2 < 1$ is false. One way to record a relation completely is to collect all the ordered pairs (x, y) for which $x \sim y$. The result is a subset of the Cartesian product $A \times A$ of Section 3.1.

Definition 6.1 (Equivalence relation)**.** A relation $\sim$ on a set A is an *equivalence relation* if it has the following three properties.

- *Reflexivity:* $x \sim x$ for every $x \in A$.
- *Symmetry:* for all $x, y \in A$, if $x \sim y$, then $y \sim x$.
- *Transitivity:* for all $x, y, z \in A$, if $x \sim y$ and $y \sim z$, then $x \sim z$.

To show that a relation is an equivalence relation, we must check all three properties, and each of them for arbitrary elements of A, not only for a few convenient examples.

Consider the students taking a course, and let $x \sim y$ mean that x and y were born in the same month. To check the three properties, let x, y, and z be any students in the course. Certainly x was born in the same month as x, so the relation is reflexive. If x has the same birth month as y, then y has the same birth month as x, so the relation is symmetric. If x and y share a birth month, and y and z share a birth month, then all three were born in y's month, so x and z share it as well. The relation is transitive, and therefore it is an equivalence relation.

An equivalence relation sorts the elements of its set into separate classes. The *equivalence class* of an element $x \in A$ is the set of all elements of A that are related to x:

$$[x] = \{y \in A : y \sim x\}.$$

For birth months, the class of a student born in March is the set of all students in the course who were born in March. There is one class for each month in which at least one student was born, and every student belongs to exactly one of these classes. Proposition 6.2 in Section 6.2 proves that every equivalence relation sorts its set in this clean way. Knowing that two students lie in the same class tells us that they share a birth month, and nothing more.

Many natural relations are not equivalence relations. The relation "is a friend of" usually fails transitivity: Alice may be friends with Ben, and Ben with Chen, while Alice and Chen are not friends at all. Relations that express rough similarity fail in the same way. On the real numbers, let $x \approx y$ mean $|x - y| \leq 1$, so that x and y are within distance 1 of each other. This relation is reflexive, because $|x - x| = 0$, and symmetric, because $|x - y| = |y - x|$. It is not transitive: $0 \approx 0.8$ and $0.8 \approx 1.6$, but $|0 - 1.6| = 1.6 > 1$.

So "numbers close to each other" cannot be sorted into separate classes. Suppose we tried, putting each number into the same class as every number close to it. Small steps lead from any number to any other: $0 \approx 0.8$, $0.8 \approx 1.6$, $1.6 \approx 2.4$, and so on. These steps would force 0, 0.8, 1.6, 2.4, and eventually every real number into one single class, although 0 and 100 are far from close. Looking at the numbers close to a single number shows the trouble in another way. The numbers close to 0 form the interval $[-1, 1]$, and the numbers close to 0.8 form the interval $[-0.2, 1.8]$. These two sets overlap, since both contain 0.5, but they are not equal, since only the first contains -1. Proposition 6.2 will show that the classes of an equivalence relation never overlap in this partial way. Transitivity, the one property of Definition 6.1 that $\approx$ lacks, is what rules it out.

Pause and decide. Is the relation $x \leq y$ on the integers an equivalence relation? Check each of the three properties.

Check. No. Reflexivity holds, since $x \leq x$ for every integer x. Transitivity holds too: if $x \leq y$ and $y \leq z$, then $x \leq z$. Symmetry fails, because $1 \leq 2$ is true but $2 \leq 1$ is false. One failure is enough.

The relation $\leq$ describes comparison rather than sameness, and Section 6.6 studies relations of that kind.

6.2 Congruence and residue classes

Fix a positive integer m. For integers a and b, we write

$$a \equiv b \pmod{m},$$

and say that a is *congruent to b modulo m*, when m divides $a - b$, that is, when $a - b = mk$ for some integer k. (The word "divides" was defined in Section 5.5.) In words, a and b differ by a whole number of steps of size m. A clock gives an everyday example with $m = 12$: the times 15:00 and 3:00 put the hour hand in the same place, because $15 - 3 = 12$. For $m = 4$, we have $1 \equiv 5 \pmod 4$, since $1 - 5 = -4 = 4 \cdot (-1)$, and $1 \equiv -3 \pmod 4$, since $1 - (-3) = 4 = 4 \cdot 1$. On the other hand, $1 \not\equiv 2 \pmod 4$, because $1 - 2 = -1$ is not a multiple of four.

Congruence modulo m is an equivalence relation on $\mathbb{Z}$. We check the three properties of Definition 6.1, for arbitrary integers a, b, c.

- *Reflexivity.* $a - a = 0 = m \cdot 0$, so $a \equiv a \pmod m$.
- *Symmetry.* If $a \equiv b \pmod m$, then $a - b = mk$ for an integer k. So $b - a = m(-k)$, and $-k$ is an integer, which gives $b \equiv a \pmod m$.
- *Transitivity.* If $a \equiv b$ and $b \equiv c \pmod m$, then $a - b = mk$ and $b - c = m\ell$ for integers k and ℓ. So
$$a - c = (a - b) + (b - c) = mk + m\ell = m(k + \ell),$$
and $k + \ell$ is an integer, which gives $a \equiv c \pmod m$.

Because of symmetry, it does not matter whether we test $a - b$ or $b - a$: one is a multiple of m exactly when the other is.

Residue classes

The equivalence class of an integer a under this relation is called its *residue class modulo m* and is written $[a]_m$. (*Residue* is an old word for remainder.) It consists of all integers congruent to a. An integer x satisfies $x \equiv a \pmod m$ exactly when $x - a = mk$ for some integer k, that is, when $x = a + mk$. So the class consists of all integers of the form $a + mk$ with k an integer:

$$[a]_m = \{x \in \mathbb{Z} : x \equiv a \pmod m\} = \{a + mk : k \in \mathbb{Z}\}.$$

For modulus four, the class of 1 is

$$[1]_4 = \{\ldots, -7, -3, 1, 5, 9, \ldots\} :$$

start at 1 and move up or down in steps of four. Any member of a class can be used to name the class, and a member used in this way is called a *representative* of the class. The integer 5 is another

representative of the same class. Starting at 5 and moving in steps of four reaches exactly the same integers, so $[5]_4 = [1]_4$, even though $5 \neq 1$. Keep three levels apart here: the integer 1; the set $[1]_4$ of all integers congruent to it; and the collection of all residue classes modulo four, which we describe at the end of this section.

By contrast, the class $[2]_4 = \{\ldots, -6, -2, 2, 6, 10, \ldots\}$ has no member in common with $[1]_4$. Two residue classes are always either equal, like $[1]_4$ and $[5]_4$, or without a common member, like $[1]_4$ and $[2]_4$. They never overlap only partly, and the same is true for the classes of every equivalence relation, as we now prove.

Proposition 6.2. *Let $\sim$ be an equivalence relation on a set A, and let $x, y \in A$.*

(a) $x \in [x]$.

(b) $[x] = [y]$ *if and only if* $x \sim y$.

(c) The classes $[x]$ *and* $[y]$ *are either equal or disjoint.*

Proof. (a) By reflexivity, $x \sim x$, and this says that $x \in [x]$.

(b) First suppose $[x] = [y]$. By part (a), $x \in [x]$, so $x \in [y]$, which means $x \sim y$.

Conversely, suppose $x \sim y$. We prove two containments. Let $z \in [x]$, so that $z \sim x$. Together with $x \sim y$, transitivity gives $z \sim y$, so $z \in [y]$. This proves $[x] \subseteq [y]$. For the reverse containment, symmetry turns $x \sim y$ into $y \sim x$. Now let $z \in [y]$, so that $z \sim y$. Together with $y \sim x$, transitivity gives $z \sim x$, so $z \in [x]$. This proves $[y] \subseteq [x]$, and the two containments give $[x] = [y]$.

(c) Suppose $[x]$ and $[y]$ are not disjoint. Then some element z belongs to both, so $z \sim x$ and $z \sim y$. Symmetry turns $z \sim x$ into $x \sim z$, and transitivity applied to $x \sim z$ and $z \sim y$ gives $x \sim y$. By part (b), $[x] = [y]$. So two classes that share even a single element are equal. □

Each of the three properties of an equivalence relation does a job in this proof. Reflexivity puts every element into its own class, so no element is left out. Symmetry and transitivity make sure that classes cannot overlap partly.

A *partition* of a set A is a collection of nonempty subsets of A, called the *pieces*, such that every element of A lies in exactly one piece. In other words, the pieces cover the whole set and no two different pieces overlap. Proposition 6.2 says that the equivalence classes of any equivalence relation form a partition. Indeed, each class $[x]$ is nonempty, because it contains x by part (a). Every element x lies in at least one class, namely $[x]$, again by part (a). And no element lies in two different classes, because by part (c) two classes that share an element are equal. The birth-month classes of Section 6.1 are an example: the pieces are the sets of students born in the same month.

For congruence, part (b) gives a test that we will use again and again:

$$[a]_m = [b]_m \quad \text{exactly when} \quad a \equiv b \pmod{m}, \text{ that is, when } m \text{ divides } a - b.$$

For example, $[1]_4 = [5]_4$ because 4 divides $5 - 1$, while $[1]_4 \neq [2]_4$ because $2 - 1 = 1$ is not a multiple of four.

A standard name for each class

Every residue class modulo m contains *exactly one* of the integers $0, 1, \ldots, m - 1$.

At least one. Let a be any integer. Division with remainder (Section 5.5) gives integers q and r with $a = qm + r$ and $0 \le r < m$. Then $r - a = m \cdot (-q)$, so $r \equiv a \pmod{m}$, and r belongs to $[a]_m$. For example, $-7 = (-2) \cdot 4 + 1$, so 1 is a member of $[-7]_4$ that lies in $\{0, 1, 2, 3\}$.

At most one. Suppose, for a contradiction, that two different integers from the list $0, 1, \ldots, m-1$ both belong to $[a]_m$. Call the smaller one r and the larger one s, so that $0 \le r < s \le m - 1$. Then $s \equiv a$ and $r \equiv a$. Symmetry turns the second into $a \equiv r$, and transitivity gives $s \equiv r \pmod{m}$, so $s - r = mk$ for some integer k. Since $r < s$, the number $s - r = mk$ is positive. As m is positive, k must be positive too: if $k \le 0$, then $mk \le 0$. A positive integer is at least 1, so $k \ge 1$ and $s - r = mk \ge m$. This contradicts $s - r \le m - 1$, which holds because $s \le m - 1$ and $r \ge 0$. So a class cannot contain two different integers from the list.

It follows that there are exactly m residue classes modulo m, namely

$$[0]_m,\ [1]_m,\ \ldots,\ [m-1]_m.$$

Every class is one of these: if r is the member of $[a]_m$ in the list, then $r \equiv a$, so $[a]_m = [r]_m$ by the test above. And no two of them are equal: if $[r]_m = [s]_m$ for different r and s in the list, then this one class would contain both r and s, by part (a) of Proposition 6.2, and we have just seen that this is impossible.

The "at least one" argument also tells us which integer of the list lies in $[a]_m$: it is the remainder of a on division by m, which Section 5.5 writes as $a \bmod m$. This gives a second form of the test: *two integers are congruent modulo m exactly when they have the same remainder on division by m.* Indeed, if $a \equiv b \pmod{m}$, then $[a]_m = [b]_m$, and the one member of this class in the list is both $a \bmod m$ and $b \bmod m$. Conversely, if a and b have the same remainder r, then $a \equiv r$ and $b \equiv r$, so $a \equiv b$ by symmetry and transitivity. For example, 17 and -7 are congruent modulo 4, since $17 - (-7) = 24 = 4 \cdot 6$, and indeed $17 \bmod 4 = 1$ and $(-7) \bmod 4 = 1$.

Notice that the word "mod" is used in two ways. The statement $17 \equiv -7 \pmod{4}$ relates two integers, and like any statement it is either true or false. The expression $17 \bmod 4$ is a single integer, the remainder 1. So $17 \equiv 1 \pmod{4}$ is true, while $17 = 1$ is false.

Pause and decide. Which of the integers $0, 1, \ldots, 6$ belongs to the class $[-20]_7$? Name two other representatives of the same class.

Check. Division with remainder gives $-20 = (-3) \cdot 7 + 1$ with $0 \le 1 < 7$, so the answer is 1, and $[-20]_7 = [1]_7$. Any integer of the form $1 + 7k$ is another representative, for instance 8 and -6. A common slip is to write $-20 = (-2) \cdot 7 - 6$ and report -6. That equation is true, but -6 is not in the list $0, 1, \ldots, 6$: division with remainder always produces a remainder between 0 and $m - 1$, even for a negative number.

The set of all residue classes modulo m is written $\mathbb{Z}/m\mathbb{Z}$, where $m\mathbb{Z}$ stands for the set of all multiples of m. Read the notation as "the integers, with differences that are multiples of m ignored." For example,

$$\mathbb{Z}/4\mathbb{Z} = \{[0]_4, [1]_4, [2]_4, [3]_4\}$$

is a set with four elements, and each of those elements is itself an infinite set of integers. This is the third of the three levels mentioned earlier. More generally, the set of all equivalence classes

of an equivalence relation on a set A is called the *quotient* of A by the relation, and $\mathbb{Z}/m\mathbb{Z}$ is the quotient of $\mathbb{Z}$ by congruence modulo m.

6.3 Well-definedness: does the answer depend on the representative?

We would like to add residue classes. The natural rule is this: to add two classes, choose a representative of each, add the two integers, and take the class of the sum.

$$[a]_m + [b]_m = [a + b]_m.$$

But a class has many representatives, and the rule does not say which ones to choose. If two people choose different representatives of the same two classes, do they get the same answer? Unless they always do, the rule does not define an addition of classes at all.

Try this with $[1]_4 + [2]_4$. Using the representatives 1 and 2 gives $[3]_4$. Using 5 instead of 1 and -2 instead of 2 gives $[5 + (-2)]_4 = [3]_4$ again. Choosing 5 and 6 gives $[11]_4$, which is the same class as $[3]_4$, because $11 - 3 = 8$ is a multiple of four. The integer sum can change from one choice to another, but its class does not. Here is why this always happens.

Suppose the first person uses a and b, and the second uses a' and b', where $[a']_m = [a]_m$ and $[b']_m = [b]_m$. By Proposition 6.2(b), $a' \equiv a$ and $b' \equiv b \pmod{m}$, so $a' - a = mk$ and $b' - b = m\ell$ for some integers k and ℓ. In other words, $a' = a + mk$ and $b' = b + m\ell$. Then

$$a' + b' = a + b + m(k + \ell).$$

Since $k + \ell$ is an integer, m divides $(a' + b') - (a + b)$, and so $[a' + b']_m = [a + b]_m$. Both people get the same class.

A rule whose inputs are classes, but which is computed from chosen representatives, is called *well-defined* when its output depends only on the classes and not on which representatives were chosen. We have just shown that addition of residue classes is well-defined. It is a rule about the classes themselves, not about the names we happened to choose for them.

Notice what the general calculation proved, and what it did not. It did not prove $a' + b' = a + b$, which is usually false: $5 + 6 = 11$, not 3. It proved that the two sums differ by a multiple of m, and by Proposition 6.2(b) that is exactly what is needed for the two output classes to be equal. Before asking whether two outputs agree, ask what kind of object the output is.

Rules that fail, and how to spot them

Here is a rule that is not well-defined. Try to define an integer-valued function F on $\mathbb{Z}/4\mathbb{Z}$ by $F([a]_4) = a$. The class $[1]_4$ can also be written $[5]_4$, so the rule would give both $F([1]_4) = 1$ and $F([5]_4) = 5$ for the same input class. A function must give exactly one output for each input (Section 3.2), so this rule does not define a function. We can repair it by saying which representative to use: let $F([a]_4)$ be the unique member of $[a]_4$ in $\{0, 1, 2, 3\}$, which exists by Section 6.2. This rule is well-defined, because the output is now determined by the class itself, and the choice of name plays no part. For instance, $F([5]_4) = F([1]_4) = F([-7]_4) = 1$. In the notation of Section 5.5, $F([a]_4) = a \bmod 4$.

The same check catches proposals that sound reasonable. Suppose someone suggests: "the parity of a class modulo three is the parity of any of its representatives." But 0 and 3 represent the same class modulo three, since $3 - 0 = 3$, and 0 is even while 3 is odd. The rule gives two different answers for one input, so it does not define anything. A single pair of representatives that gives different answers is enough to show that a rule is not well-defined. To show that a rule *is* well-defined, on the other hand, we need an argument that covers all representatives, like the calculation for addition above. Does the parity rule fare better for classes modulo four?

Pause and decide. Does the rule $G([a]_4) = [a]_2$ depend on the representative? What does the answer tell you about the parity of the members of a class modulo four?

Check. It does not depend on the representative. Suppose $[a']_4 = [a]_4$, so that $a' = a + 4k$ for an integer k. Then $a' - a = 2(2k)$, so $a' \equiv a \pmod 2$ and $[a']_2 = [a]_2$. The rule is therefore a well-defined function from $\mathbb{Z}/4\mathbb{Z}$ to $\mathbb{Z}/2\mathbb{Z}$. Since $[0]_2$ is the set of even integers and $[1]_2$ is the set of odd integers, the output $G([a]_4)$ records the parity of a. So all members of a class modulo four have the same parity. The step that made this work is that 2 divides 4; Exercise 6.5 explores this further.

The map G loses information: it sends both $[0]_4$ and $[2]_4$ to $[0]_2$, so it is not injective. Yet it respects addition. Addition on $\mathbb{Z}/2\mathbb{Z}$ is well-defined by the same calculation as before, with $m = 2$, and for all integers a and b,

$$G([a]_4 + [b]_4) = G([a + b]_4) = [a + b]_2 = [a]_2 + [b]_2 = G([a]_4) + G([b]_4).$$

The first and third equalities are the rules for adding classes modulo four and modulo two, and the second and fourth are the definition of G. For instance, with $a = 1$ and $b = 3$, the left side is $G([4]_4) = [4]_2 = [0]_2$, and the right side is $[1]_2 + [3]_2 = [4]_2 = [0]_2$. In words, adding first and then applying G gives the same result as applying G first and then adding. A map can respect addition without keeping every distinction between its inputs. Section 6.4 gives this property a name and looks at other things a map can respect.

6.4 Structure-preserving maps

A *structure* on a set is extra information that the set carries, such as an operation or a relation. An *operation* on a set S takes two elements of S and returns an element of S. In the language of Section 3.1, it is a function from $S \times S$ to S. Addition of integers and addition of residue classes are operations. The integers carry several structures at once: addition, multiplication, and the order $\leq$. A *graph*, studied in Chapter 7, is a set of points, called *vertices*, together with the information of which pairs of vertices are joined; joined vertices are called *adjacent*. A set with a distance, studied in Chapter 11, carries a number measuring how far apart any two of its points are. When you meet a new subject, ask what its objects carry beyond being members of a set.

For sets that have an addition, a map T *preserves addition* when

$$T(x + y) = T(x) + T(y) \qquad \text{for all } x \text{ and } y \text{ in its domain.}$$

The sum on the left is computed in the domain and the sum on the right in the codomain, and these may be different kinds of addition. The map G from $\mathbb{Z}/4\mathbb{Z}$ to $\mathbb{Z}/2\mathbb{Z}$ at the end of Section 6.3 is an example. For a simpler one, the map $V : \mathbb{Z} \to \mathbb{Z}$ with $V(x) = 3x$ preserves addition, because $3(x + y) = 3x + 3y$. The map $U : \mathbb{Z} \to \mathbb{Z}$ with $U(x) = x + 1$ does not: $U(x + y) = x + y + 1$, while $U(x) + U(y) = x + y + 2$. Preserving addition is part of the definition of a linear map in Chapter 10.

For graphs, the structure is adjacency. An *isomorphism* from one graph to another is a bijection f from the vertices of the first graph to the vertices of the second that preserves adjacency in both directions: for all vertices u and v of the first graph, if u and v are adjacent, then $f(u)$ and $f(v)$ are adjacent, and if $f(u)$ and $f(v)$ are adjacent, then u and v are adjacent. In the acquaintance example of Section 6.1, the map with $f(\text{Alice}) = 1$, $f(\text{Ben}) = 2$, $f(\text{Chen}) = 3$ is an isomorphism onto the pattern in which 1 is linked to 2 and 2 to 3. The bijection with $g(\text{Alice}) = 2$, $g(\text{Ben}) = 1$, $g(\text{Chen}) = 3$ is not an isomorphism: Ben and Chen know each other, but $g(\text{Ben}) = 1$ and $g(\text{Chen}) = 3$ are not linked. The second direction matters as well. Suppose instead that 1, 2, and 3 were all linked to one another. The map f would still send linked people to linked numbers, but it would send Alice and Chen, who have never met, to the linked pair 1 and 3. A chain of three people and a triangle of three mutually linked numbers are different patterns, and the second direction is what keeps f from counting as an isomorphism between them.

The first direction says that f sends adjacent vertices to adjacent vertices. The second says the same about the inverse map f^{-1}, which sends $f(u)$ and $f(v)$ back to u and v. In general, an *isomorphism* is a bijection that preserves the structure and whose inverse also preserves it. What must be preserved depends on the subject: adjacency for graphs, addition for sets with an addition, distance for sets with a distance. Two objects joined by an isomorphism are called *isomorphic*. They differ only in the names of their elements.

Symmetries

A *symmetry* of an object is an isomorphism from the object to itself. This is a second meaning of the word, unrelated to the symmetry property of a relation in Definition 6.1. Picture four seats around a round table, labeled 0, 1, 2, 3 in order. As the structure, take the information of which seats are next to each other: 0 and 1, 1 and 2, 2 and 3, and 3 and 0. In other words, the seats are the vertices of a graph, and two seats are adjacent when they are next to each other. Moving everyone one seat along is a symmetry. As a map on seat labels, it is

$$r(i) = (i + 1) \bmod 4,$$

so $r(0) = 1$, $r(1) = 2$, $r(2) = 3$, and $r(3) = 0$. Let us check that r is a symmetry. It is a bijection, because each of the four seats is reached exactly once. The four pairs of neighbors, 0, 1 and 1, 2 and 2, 3 and 3, 0, go to the pairs 1, 2 and 2, 3 and 3, 0 and 0, 1, which are again neighbors. The two pairs of seats that are not neighbors, namely the opposite pairs 0, 2 and 1, 3, go to the pairs 1, 3 and 2, 0, which are again opposite. So two seats are neighbors exactly when their images are neighbors. The inverse of r is rotation by three seats, because moving one seat and then three more is a full turn: $1 + 3 = 4 \equiv 0 \pmod 4$.

The symmetries of an object, taken together, have four useful features. Let f, g, and h be symmetries.

- *Closure.* Doing g and then f is again a symmetry. The composition $f \circ g$ is a bijection, because compositions of injective maps are injective (Theorem 3.2) and compositions of surjective maps are surjective (Exercise 3.2). It preserves the structure. For a graph, for instance, if u and v are adjacent, then so are $g(u)$ and $g(v)$, because g is a symmetry, and then so are $f(g(u))$ and $f(g(v))$, because f is one. Its inverse is $g^{-1} \circ f^{-1}$, which first undoes f and then undoes g. (Check: $(f \circ g)(g^{-1}(f^{-1}(y))) = f(f^{-1}(y)) = y$ for every y, and $g^{-1}(f^{-1}(f(g(x)))) = g^{-1}(g(x)) = x$ for every x.) This inverse preserves the structure for the same reason, since f^{-1} and g^{-1} do.
- *Associativity.* $(f \circ g) \circ h = f \circ (g \circ h)$. For every input x, both sides give $f(g(h(x)))$: the left side is $(f \circ g)(h(x)) = f(g(h(x)))$, and the right side is $f((g \circ h)(x)) = f(g(h(x)))$. Both mean "apply h, then g, then f," so the brackets do not matter.
- *Identity.* Doing nothing, that is, applying the identity map of Section 3.2, is a symmetry: it is a bijection, it is its own inverse, and it changes nothing, so it certainly preserves the structure. Section 3.2 wrote the identity map on a set A as id_A. Here the identity map always acts on the object whose symmetries we study, so we leave out the subscript and write simply id. Moreover $f \circ \mathrm{id} = \mathrm{id} \circ f = f$.
- *Inverses.* A symmetry f is a bijection, so it has an inverse f^{-1} (Section 3.3), and $f \circ f^{-1} = f^{-1} \circ f = \mathrm{id}$. These two equations also say that f is an inverse of f^{-1}, so f^{-1} is a bijection, because a map with an inverse is bijective (Exercise 3.5). It preserves the structure, because an isomorphism preserves the structure in both directions. So f^{-1} is again a symmetry.

These four features occur so often that they have a name.

Definition 6.3 (Group)**.** A *group* is a set S together with an operation $*$ on S, which combines any two elements $x, y \in S$ into an element $x * y \in S$, such that the following hold.

- *Associativity:* $(x * y) * z = x * (y * z)$ for all $x, y, z \in S$.
- *Identity:* there is an element $e \in S$ with $e * x = x * e = x$ for every $x \in S$.
- *Inverses:* for every $x \in S$ there is an element $y \in S$ with $x * y = y * x = e$, where e is the identity element from the previous condition.

Closure is built into the definition, because an operation on S must return an element of S. The four features above say that the symmetries of an object form a group under composition. The integers form a group under addition: the sum of two integers is an integer, addition is associative, 0 is an identity, and $-a$ is an inverse of a. So does $\mathbb{Z}/m\mathbb{Z}$ under the addition of Section 6.3. Its identity is $[0]_m$, and $[-a]_m$ is an inverse of $[a]_m$, because $[a]_m + [-a]_m = [0]_m$. Associativity is inherited from the integers: both $([a]_m + [b]_m) + [c]_m$ and $[a]_m + ([b]_m + [c]_m)$ equal $[a+b+c]_m$. Not every operation gives a group. The integers under multiplication have the identity 1, but 2 has no inverse, because no integer y satisfies $2y = 1$.

The definition does not require that $x * y = y * x$. A group in which this holds for all x and y is called *commutative*. The integers under addition form a commutative group, but many groups of symmetries are not commutative. Here is an example. Place three letters in a row as ABC. Let s swap the letters in the first two positions, and let t swap the letters in the last two positions. Doing s and then t gives

$$ABC \;\to\; BAC \;\to\; BCA,$$

while doing t and then s gives

$$ABC \;\to\; ACB \;\to\; CAB.$$

The results differ, so $t \circ s \neq s \circ t$. The rearrangements of three positions, that is, the bijections from a three-element set to itself, are the symmetries of a set that carries no extra structure. They form a group under composition, and since s and t are two of them, this group is not commutative.

Now let us see the four features in the rotations of the table. Write r_0, r_1, r_2, r_3, where r_j moves everyone j seats along, so that $r_j(i) = (i + j) \bmod 4$ and r_1 is the map r from before. Applying r_3 and then r_2 moves everyone five seats, which is the same as moving one seat, so $r_2 \circ r_3 = r_1$. In general, composing two rotations adds their amounts modulo four, so the result is always one of r_0, r_1, r_2, r_3; this is closure. The identity is r_0. The rotations r_1 and r_3 are inverses of each other, as we saw for r, and r_2 is its own inverse (Exercise 6.3 asks you to check this). Associativity needs no separate check, because composition of maps is always associative. These particular symmetries also commute, because addition of their amounts does. The rotations are not the only symmetries of the table. For instance, the reflection that keeps seats 0 and 2 in place and swaps seats 1 and 3 also keeps neighbors next to each other and opposite seats opposite, so it is a symmetry as well. But the four rotations already form a group on their own.

Naming the pattern helps us recognize it elsewhere. The rotation r_j behaves exactly like the residue class $[j]_4$: composing rotations adds their amounts modulo four, just as adding classes adds representatives modulo four. To say this precisely, let φ send the class $[j]_4$ to the rotation r_j, for $j = 0, 1, 2, 3$. Every class modulo four is exactly one of $[0]_4$, $[1]_4$, $[2]_4$, $[3]_4$ (Section 6.2), so the rule assigns each class exactly one rotation, whichever representative we start from. In other words, φ is a well-defined function from $\mathbb{Z}/4\mathbb{Z}$ to the set of rotations. It is a bijection, because the four classes go to the four rotations, one each. The structure that matters here is the group operation: addition of classes on one side, composition of rotations on the other. So φ preserves the structure when

$$\varphi(x + y) = \varphi(x) \circ \varphi(y) \qquad \text{for all classes } x \text{ and } y \text{ modulo four.}$$

To check this, take $x = [j]_4$ and $y = [k]_4$ with j and k in $\{0, 1, 2, 3\}$, and let $c = (j+k) \bmod 4$. Then $x+y = [j+k]_4 = [c]_4$, so $\varphi(x+y) = r_c$. On the other side, $\varphi(x) \circ \varphi(y) = r_j \circ r_k$ moves everyone k seats and then j more, which is r_c as well. The inverse of φ preserves the structure too, by the same calculation read backward: $\varphi^{-1}(r_j \circ r_k) = \varphi^{-1}(r_c) = [c]_4 = [j]_4 + [k]_4 = \varphi^{-1}(r_j) + \varphi^{-1}(r_k)$. So φ is an isomorphism from the group $\mathbb{Z}/4\mathbb{Z}$ to the group of rotations. The two groups differ only in the names of their elements.

6.5 What symmetry permits us to assume

Consider a sequence of signs $a = (a_0, a_1, \ldots, a_{n-1})$ with $n \geq 1$, where each entry a_i is 1 or -1. We number the positions from 0, so the last entry is a_{n-1}. Rows of signs like this appeared in Section 5.4, and Chapter 14 studies them in depth. By a *pair product* we mean a product $a_i a_j$ of two entries of the sequence. Negating every sign turns a into the sequence $-a = (-a_0, -a_1, \ldots, -a_{n-1})$, and it does not change any pair product, because

$$(-a_i)(-a_j) = a_i a_j.$$

For example, $(-1, 1, 1, -1)$ and its negation $(1, -1, -1, 1)$ have the same products of neighboring entries: -1, 1, -1 in both cases. So any quantity built only from pair products, such as the sum of neighboring products $a_0a_1 + a_1a_2 + \cdots + a_{n-2}a_{n-1}$, takes the same value on a and on $-a$.

The same goes for properties of sequences. Say that two sign sequences a and b of the same length *have the same pair products* if $a_i a_j = b_i b_j$ for all positions i and j. A property of sign sequences *depends only on the pair products* if any two sequences with the same pair products either both have the property or both lack it. The property "the sum of neighboring products is -1" is an example. Since a and $-a$ have the same pair products, such a property holds for a exactly when it holds for $-a$. When we study a property that depends only on pair products, we may therefore assume that the first sign is 1: if a sequence starts with -1, negate it.

Replacing every object by a convenient standard form in this way is called a *normalization*. Two facts make this one legitimate.

- The change preserves the property under study: a sequence has the property exactly when its negation has it.
- Every case can be brought into the standard form: a sequence either begins with 1 already, or it begins with -1 and its negation begins with 1.

If either fact fails, the reduction fails too. Here is a failure of the first fact. Consider the claim that the two signs of a sequence of length two add up to at least 0. The two sequences that begin with 1 are $(1, 1)$ and $(1, -1)$, with sums 2 and 0, so the claim holds for both of them. But it is false for $(-1, -1)$, whose sum is -2. Negation turns the sum of the signs into its negative, so it does not preserve the property "the sum is at least 0," and checking the sequences that begin with 1 told us nothing about the others.

The second fact can fail as well. Negation can always make the first sign equal to 1, but it cannot always make the first and the last sign both equal to 1. The sequence $(1, -1)$ and its negation $(-1, 1)$ each have one sign of each kind. A normalization "the first and last signs are both 1" would therefore leave such sequences out, even when the property under study is preserved by negation. For example, the property $a_0a_{n-1} = 1$ depends only on a pair product. It holds for every sequence whose first and last signs are both 1, yet it fails for $(1, -1)$, where $a_0a_1 = -1$. Checking only the normalized sequences would wrongly suggest that it always holds.

What "without loss of generality" must justify

Mathematicians often announce a reduction like this with the phrase *without loss of generality*: "Without loss of generality, $a_0 = 1$." Read the phrase as a promise that no case has been lost. It does not give permission to add a convenient extra condition. Whether the phrase comes from a textbook, from an assistant, or from your own draft, ask which symmetry is being used, why it preserves the property at stake, and why every case can be brought into the assumed form.

Here is the argument behind the promise for sign sequences. Let P be a property of sign sequences that depends only on the pair products, and suppose we have proved that P holds for every sign sequence of length n whose first entry is 1. Let a be any sign sequence of length n. If $a_0 = 1$, then a is one of the cases already proved. If $a_0 = -1$, put $b = -a$. Then $b_0 = 1$, so P holds

for b. The sequences a and b have the same pair products, and P depends only on these, so P holds for a as well. Either way P holds for a, and since a was arbitrary, P holds for every sign sequence of length n. Both facts were used: the second to produce b with $b_0 = 1$, and the first to carry P from b back to a.

Pause and decide. You want to prove that $a_0a_1 + a_0a_2 + a_1a_2 \geq -1$ for every sign sequence (a_0, a_1, a_2). Is it justified to assume $a_0 = 1$? If so, how many sequences are left to check? Check them.

Check. It is justified: the expression is built only from pair products, so negating all three signs does not change its value. With $a_0 = 1$, four sequences remain: $(1, 1, 1)$, $(1, 1, -1)$, $(1, -1, 1)$, and $(1, -1, -1)$. The expression equals 3, -1, -1, and -1 on them, and each value is at least -1. The other four sign sequences of length three are the negations of these four and give the same values, so the inequality holds for all eight.

6.6 Order is a different structure

Equivalence relations describe sameness. A different kind of relation describes *comparison*: one object is smaller than another, or lies below it, or is contained in it. Write $x \preceq y$ for such a relation on a set A, and read it as "x is below y."

Definition 6.4 (Partial order)**.** A relation $\preceq$ on a set A is a *partial order* if it has the following three properties.

- *Reflexivity:* $x \preceq x$ for every $x \in A$.
- *Antisymmetry:* for all $x, y \in A$, if $x \preceq y$ and $y \preceq x$, then $x = y$.
- *Transitivity:* for all $x, y, z \in A$, if $x \preceq y$ and $y \preceq z$, then $x \preceq z$.

Reflexivity and transitivity are the same requirements as for an equivalence relation, but symmetry has been replaced by antisymmetry. Symmetry says that whenever the relation holds in one direction, it also holds in the other. Antisymmetry says that it holds in both directions only between an object and itself. The usual order $\leq$ on the real numbers is a partial order: $x \leq x$; if $x \leq y$ and $y \leq x$, then $x = y$; and if $x \leq y$ and $y \leq z$, then $x \leq z$. The strict order $<$ is not a partial order in this sense, because it is not reflexive: $1 < 1$ is false.

Set inclusion $\subseteq$ is also a partial order on any collection of sets. Every set is a subset of itself. If $X \subseteq Y$ and $Y \subseteq X$, then $X = Y$, because two sets are equal exactly when each is contained in the other (Section 3.1). And if $X \subseteq Y$ and $Y \subseteq Z$, then every member of X lies in Y and hence in Z, so $X \subseteq Z$.

Two elements x and y are *comparable* if $x \preceq y$ or $y \preceq x$. Under $\leq$, any two real numbers are comparable. Under inclusion, this can fail: neither of $\{1\}$ and $\{2\}$ contains the other. The word *partial* in the name signals this: a partial order need not compare every pair, although some, such as $\leq$, do.

Maximal and maximum

Because some pairs may be incomparable, there are two different notions of a "largest" element of a collection, and they must not be confused. Let C be a subset of A. We call it a *collection*, because in the main examples its members are themselves sets. Let x belong to C.

- The element x is *maximal* in C if nothing in C lies strictly above it: there is no y in C with $x \preceq y$ and $y \neq x$.
- The element x is a *maximum* of C if everything in C lies below it: $y \preceq x$ for every y in C.

A maximum is always maximal. To see this, let x be a maximum, and suppose y is in C with $x \preceq y$. Since x is a maximum, also $y \preceq x$, and antisymmetry gives $y = x$. So nothing in C lies strictly above x. The same use of antisymmetry shows that a collection has at most one maximum: if x and x' are both maxima, then $x' \preceq x$ and $x \preceq x'$, so $x = x'$.

The converse fails. Take the collection $\{\{1\}, \{2\}\}$, ordered by inclusion. Both members are maximal, because neither lies inside the other. Neither member is a maximum, for the same reason. A collection can therefore have several maximal elements and no maximum. It can also have no maximal element at all. The integers under $\leq$ are an example: for every integer n, the integer $n + 1$ lies strictly above it.

With numbers ordered by $\leq$ we never see the difference between maximal and maximum. When every two elements are comparable, a maximal element x is automatically a maximum. Indeed, take any y in the collection. Either $y \preceq x$, which is what we want, or $x \preceq y$; in the second case maximality forces $y = x$, and then $y \preceq x$ by reflexivity. So the distinction between maximal and maximum is new precisely because inclusion, and many other natural orders, leave some pairs incomparable.

Chapter 8 meets the same contrast in a slightly different form. There, a matching assigns tasks to people, with at most one task for each person and at most one person for each task. A matching is *maximal* when no new person–task pair can be added to it, and *maximum* when it has the largest possible number of pairs. A matching can be maximal without being maximum. In both settings, being impossible to improve by one obvious step is weaker than being best overall.

> **Habit 6: ask what a change of description preserves**
>
> Before you treat different objects as the same, or replace an object by a convenient standard form, say exactly which relation you are using and which property the change preserves. Before you apply a formula to equivalence classes, check that the result does not depend on the representatives. When an argument, yours or an assistant's, says "without loss of generality," ask which symmetry justifies it and whether every case can be brought into the assumed form. A short check of this kind can keep a whole argument from being about the wrong object.

Exercises

Exercises 6.1–6.4 are core exercises, and Exercises 6.5 and 6.6 are transfer exercises. Complete solutions are in Appendix A, under “Week 6.”

Exercise 6.1. List the members of $[-2]_5$ that lie between -12 and 12 inclusive. Which integer in $\{0, 1, 2, 3, 4\}$ represents the same class?

Exercise 6.2. Prove that multiplication of residue classes, defined by $[a]_m \cdot [b]_m = [ab]_m$, is well-defined. That is, show that if $[a']_m = [a]_m$ and $[b']_m = [b]_m$, then $[a'b']_m = [ab]_m$.

Exercise 6.3. For the four seats around the table in Section 6.4, show that the rotation r_2 by two seats is its own inverse. Explain why this does not make r_2 the identity.

Exercise 6.4. Is “has the same number of elements as” an equivalence relation on a collection of finite sets? Check each of the three properties.

Exercise 6.5. Let m and d be positive integers. Prove that the rule $[a]_m \mapsto [a]_d$ is a well-defined function from $\mathbb{Z}/m\mathbb{Z}$ to $\mathbb{Z}/d\mathbb{Z}$ exactly when d divides m.

Exercise 6.6. Give a family of three sets, ordered by inclusion, that has exactly two maximal members and no maximum. Then add one set to the family so that the new family has a maximum.

CHAPTER 7

Graphs: counting, paths, and extremal choices

This week

Describe a pattern of connections as a graph. Prove an identity by counting one collection of objects in two different ways. Use the pigeonhole principle and averages to show that something must exist. Choose an object that is as short or as long as possible, and show that this choice has the property we want. Learn the language of graphs that the chapters on matchings build on, and the language of directed graphs needed to read the exact statement of a research problem.

7.1 A graph is a pattern of connections

Many situations consist of objects, some pairs of which are connected: people and the handshakes between them, towns and the roads that join them, students and the classmates they know. To study such a situation mathematically, we keep only two pieces of information: which objects there are, and which pairs of them are connected. Everything else, such as how long a road is or where a town lies on a map, is set aside. The structure that remains is called a graph.

Definition 7.1 (Graph)**.** A *graph* consists of a finite set V, whose members are called *vertices*, and a set E, whose members are called *edges*. Each edge is a set $\{u, v\}$ of two different vertices u and v. These two vertices are the *endpoints* of the edge, and we say that the edge *joins* them.

The full name of this kind of graph is a *finite simple undirected graph*, and each of the three words records a choice.

- *Finite*: the set V of vertices is finite, in the sense of Section 3.1. The set E of edges is then finite too, because a finite set has only finitely many pairs of members.
- *Undirected*: an edge has no direction. Since the order in which members of a set are listed does not matter, $\{u, v\} = \{v, u\}$, so the edge does not run from one endpoint to the other; it simply joins them. Section 7.5 introduces edges that do have a direction.
- *Simple*: there are no *loops*, that is, edges joining a vertex to itself, because the two endpoints of an edge must be different. Also, two vertices are joined by at most one edge: E is a set, and listing $\{u, v\}$ twice does not give it a second member, just as $\{1, 1, 3\} = \{1, 3\}$. Some applications need loops or several edges between the same two vertices, but we will not.

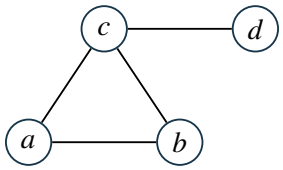
Throughout this book, the word *graph* on its own means a graph in the sense of Definition 7.1. Graphs whose edges have a direction are always called *directed* graphs.

We usually write the edge $\{u, v\}$ more briefly as uv. Since $\{u, v\} = \{v, u\}$, the names uv and vu denote the same edge. In Section 1.2, two letters written side by side meant a product. In this chapter, two vertex names written side by side mean an edge, and the context always makes clear which is intended.

Here is an example that we will use throughout the chapter. Its vertex set is $V = \{a, b, c, d\}$, and its edge set is

$$E = \{ab,\ bc,\ ca,\ cd\}.$$

We call it the *example graph* G. In a drawing, each vertex is a small circle, and each edge is a line between its two endpoints:



A drawing is only a picture of a graph. The positions of the circles, the lengths and shapes of the lines, and any points where two lines happen to cross carry no information; a crossing point is not a vertex. If we moved the circle for d to the far left of the page, we would get a different picture of the same graph, because the same pairs of vertices would still be joined. Section 6.4 made a similar point about names: two graphs joined by an isomorphism differ only in the names of their vertices, and they have the same pattern of connections.

Two vertices u and v are *neighbors*, or *adjacent*, when uv is an edge. The *degree* of a vertex v, written $\deg(v)$, is the number of its neighbors. In the example graph, the neighbors of c are a, b, and d, so $\deg(c) = 3$. The four degrees are

$$\deg(a) = 2, \qquad \deg(b) = 2, \qquad \deg(c) = 3, \qquad \deg(d) = 1.$$

A vertex with no neighbors at all has degree 0. It still belongs to the graph; it is simply not an endpoint of any edge.

The degree of v can also be read as the number of edges that have v as an endpoint. Each neighbor w of v gives one such edge, namely vw. Different neighbors give different edges, and every edge with endpoint v arises in this way from its other endpoint. For example, the edges at c are ca, cb, and cd, one for each of its neighbors a, b, and d. Here the word "simple" matters: if two edges could join the same pair of vertices, a single neighbor could account for two edges. We will use this second reading of the degree in the next section.

Pause and decide. In a graph with n vertices, can a vertex have degree larger than $n - 1$?

Check. No. The neighbors of a vertex v are vertices different from v, because the two endpoints of an edge are different. So the neighbors of v lie among the other $n - 1$ vertices, and $\deg(v) \le n - 1$. This bound is reached exactly when v is joined to every other vertex. In the example graph, which has 4 vertices, the vertex c has the largest possible degree, 3.

7.2 Double counting and the handshake theorem

Add up the degrees in the example graph G: $2 + 2 + 3 + 1 = 8$. The graph has four edges, and 8 is exactly twice 4. This is no accident, and the reason can already be seen in this small graph.

Consider every pair (v, e) in which v is a vertex, e is an edge, and v is an endpoint of e. Because an edge is the set of its two endpoints, "v is an endpoint of e" simply means $v \in e$. The table below has one row for each vertex of G and one column for each edge. An entry 1 marks a pair (v, e) with $v \in e$, and an entry 0 means that v is not an endpoint of e.

	ab	bc	ca	cd	row total
a	1	0	1	0	2
b	1	1	0	0	2
c	0	1	1	1	3
d	0	0	0	1	1
column total	2	2	2	2	8

Now count the entries equal to 1 in two ways. Row by row, the row of a vertex has a 1 for each edge at that vertex, so its total is the degree of the vertex, and the row totals add up to $2 + 2 + 3 + 1 = 8$. Column by column, every edge has exactly two endpoints, so every column total is 2, and the four columns give $2 + 2 + 2 + 2 = 8$. Both totals count the same eight entries, so they must agree.

This method, counting one collection of objects in two ways and setting the two answers equal, is called *double counting*. Its essential step is to describe the collection precisely. Then we can check that each grouping counts every member exactly once, with nothing left out and nothing counted twice.

To state the general result we need one piece of notation. Section 1.8 introduced sums such as $\sum_{j=1}^{4} a_j$, whose index runs through consecutive integers. The notation $\sum_{v \in V} \deg(v)$ means: add the number $\deg(v)$ once for each vertex v in V. For the example graph,

$$\sum_{v \in V} \deg(v) = \deg(a) + \deg(b) + \deg(c) + \deg(d) = 2 + 2 + 3 + 1 = 8.$$

The order in which the vertices are taken does not matter. The rules $a + b = b + a$ and $(a + b) + c = a + (b + c)$ of Section 1.2, applied as many times as needed, let us reorder and regroup the terms of any finite sum without changing its value. In the same way, $\sum_{e \in E} 2$ adds the number 2 once for each edge. Since $|E|$ is the number of edges (Section 3.1), this sum equals $2|E|$. For the example graph it is $2 + 2 + 2 + 2 = 8$, the sum of the column totals in the table.

Theorem 7.2 (Handshake theorem)**.** *In every graph with vertex set V and edge set E,*

$$\sum_{v \in V} \deg(v) = 2|E|.$$

The name comes from a party. Make each guest a vertex, and join two guests by an edge when they shook hands. Then $\deg(v)$ is the number of hands that guest v shook, and the theorem says that the total of these numbers is twice the number of handshakes: every handshake is counted once by each of the two people involved.

Proof. Let P be the set of all pairs (v, e) with $v \in V$, $e \in E$, and $v \in e$. We count P in two ways.

First group the pairs by their vertex. For a fixed vertex v, the pairs in P with first entry v are the pairs (v, e) in which e is an edge with endpoint v. As Section 7.1 showed, there are exactly $\deg(v)$ such edges. Every pair in P has exactly one first entry, so it lies in exactly one of these groups. Adding the sizes of the groups therefore counts each pair exactly once, and $|P| = \sum_{v \in V} \deg(v)$.

Next group the pairs by their edge. For a fixed edge $e = \{u, w\}$, the pairs in P with second entry e are (u, e) and (w, e). These are two different pairs, because $u \neq w$. Again every pair lies in exactly one group, so $|P| = \sum_{e \in E} 2 = 2|E|$.

Both expressions are equal to $|P|$, so they are equal to each other. □

The proof used both parts of the word "simple": the group of an edge has exactly two members because the two endpoints of an edge are different, and the group of a vertex v has exactly $\deg(v)$ members because two vertices are joined by at most one edge.

The handshake theorem has a consequence that is not at all obvious from a drawing. In the example graph, the vertices of odd degree are c, of degree 3, and d, of degree 1: there are two of them. That this number is even is no coincidence either.

Proposition 7.3. *In every graph, the number of vertices of odd degree is even.*

Proof. Let k be the number of vertices of odd degree. By Lemma 4.1, every degree is either even or odd, and not both. So we can split the degree sum into two parts: the sum S_{even} of the degrees of the vertices of even degree, and the sum S_{odd} of the degrees of the vertices of odd degree. Each vertex contributes its own degree, even when another vertex has the same degree. In the example graph, $S_{\text{even}} = \deg(a) + \deg(b) = 4$ and $S_{\text{odd}} = \deg(c) + \deg(d) = 4$. Then $S_{\text{even}} + S_{\text{odd}} = 2|E|$ by the handshake theorem.

- S_{even} is even, because a sum of even numbers is even: a factor 2 can be taken out of every term. (If no vertex has even degree, S_{even} is the empty sum 0, which is even too.)
- Therefore $S_{\text{odd}} = 2|E| - S_{\text{even}}$ is the difference of two even numbers, and so it is even: $2x - 2y = 2(x - y)$.

Now write the degrees of the k vertices of odd degree, using Definition 1.1, as $2\ell_1 + 1, \ldots, 2\ell_k + 1$ with integers $\ell_1, \ldots, \ell_k$. Adding them gives

$$S_{\text{odd}} = (2\ell_1 + 1) + \cdots + (2\ell_k + 1) = 2(\ell_1 + \cdots + \ell_k) + k,$$

because each of the k terms contributes one 1. Hence $k = S_{\text{odd}} - 2(\ell_1 + \cdots + \ell_k)$ is again the difference of two even numbers, so k is even. (If $k = 0$, the conclusion holds at once, since 0 is even.) □

A drawing shows one graph at a time; the proposition speaks about every graph at once. It says, for instance, that no graph has exactly one vertex of odd degree. It also settles a puzzle: can seven people at a party each shake hands with exactly three of the others? Make each person a vertex and each handshake an edge. All seven vertices would have the odd degree 3, so the number of vertices of odd degree would be 7, which is odd. The proposition forbids this, so no arrangement of

handshakes works. The handshake theorem gives the same answer even more directly: the degree sum would be $7 \cdot 3 = 21$, an odd number, but it must equal the even number $2|E|$.

7.3 The pigeonhole principle and the average

If many objects are placed in a few boxes, some box must be crowded. Here is the precise statement, one of the most useful facts in counting.

Proposition 7.4 (Pigeonhole principle)**.** *Let n, r, and k be natural numbers. If n objects are placed in r boxes and $n > rk$, then some box contains more than k objects.*

Proof. We prove the contrapositive (Section 2.5). By Section 2.4, the negation of the conclusion is "every box contains at most k objects," and the negation of $n > rk$ is $n \leq rk$. So suppose that every box contains at most k objects. Each object lies in exactly one box, so n is the sum of the numbers of objects in the r boxes. Each of these r numbers is at most k, so their sum is at most rk, that is, $n \leq rk$. □

The name pictures pigeons settling into the holes of a pigeon house: if there are more pigeons than holes, some hole holds at least two pigeons. This is the case $k = 1$, the one used most often. For example, among any 13 people, two have their birthdays in the same month: the people are the objects, the 12 months are the boxes, and $13 > 12 \cdot 1$. Larger values of k work in the same way. Among any 25 people, some month contains the birthdays of at least three of them, because $25 > 12 \cdot 2$. In each case the principle tells us that such a month *exists*, but not which month it is.

The pigeonhole principle can also be stated in the language of functions from Chapter 3. Placing each object in a box defines a function from the set of objects to the set of boxes. In that language, the case $k = 1$ is the fact from Section 3.3 that a function from a finite set A to a finite set B with $|A| > |B|$ cannot be injective: two different inputs must share an output.

Pause and decide. A class has 30 students. Must some month contain the birthdays of at least three of them? Of at least four?

Check. Take the 12 months as boxes. Since $30 > 12 \cdot 2 = 24$, the pigeonhole principle with $k = 2$ shows that some month contains more than two birthdays, that is, at least three. For four birthdays we would need $k = 3$, but 30 is not larger than $12 \cdot 3 = 36$, so the principle gives no guarantee. In fact no guarantee is possible: six months with three birthdays each and six months with two each account for $6 \cdot 3 + 6 \cdot 2 = 30$ students, and no month holds four.

A closely related fact concerns averages. The *average* of real numbers $a_1, \ldots, a_n$, where $n \geq 1$, is

$$\bar{a} = \frac{a_1 + \cdots + a_n}{n}.$$

(The bar over the a is part of the name; read $\bar{a}$ as "a bar.")

Proposition 7.5 (Averaging principle)**.** *Let $a_1, \ldots, a_n$ be real numbers, where $n \geq 1$, and let $\bar{a}$ be their average. Then at least one of the numbers is at least $\bar{a}$, and at least one of them is at most $\bar{a}$.*

Proof. Suppose, for contradiction, that every a_i is strictly less than $\bar{a}$. Adding these n strict inequalities gives

$$a_1 + \cdots + a_n < n\bar{a}.$$

(Two strict inequalities $x < y$ and $x' < y'$ can be added: $x + x' < y + x' < y + y'$, by the rule for adding the same number to both sides from Section 2.3. Repeating this adds all n of them. Since $n \geq 1$, there is at least one inequality to add, so the result is strict.) But the definition of the average says exactly that $n\bar{a} = a_1 + \cdots + a_n$, so this number would be strictly less than itself, which is impossible. Hence some a_i satisfies $a_i \geq \bar{a}$.

The same argument with every inequality reversed proves the second claim: if every a_i were strictly greater than $\bar{a}$, adding would give $a_1 + \cdots + a_n > n\bar{a}$, again impossible. □

For instance, the numbers $1, 3, 8$ have average $12/3 = 4$. The number 8 is at least 4, and the numbers 1 and 3 are at most 4. The condition $n \geq 1$ is needed for the average to make sense at all: with $n = 0$ the formula would divide by zero, and there would be no number to choose.

The pigeonhole principle is a special case of the averaging principle. If n objects lie in $r \geq 1$ boxes, the numbers of objects in the boxes add up to n, so their average is n/r. When $n > rk$, this average is larger than k, and the averaging principle gives a box containing at least n/r objects, which is more than k.

The words "at least one" are essential: an average does not control every number. Apply the averaging principle to the degrees of a graph with $n \geq 1$ vertices and m edges. By the handshake theorem the degrees add up to $2m$, so their average is $2m/n$, and therefore some vertex has degree at least $2m/n$. In the example graph G, with $n = m = 4$, the average degree is 2, and the vertex c has degree $3 \geq 2$, as promised. Nothing is promised about the other degrees; Exercise 7.5 asks how far from the average they can be. Chapter 13 develops this observation into a method for proving that an object exists: compute an average over random choices, and conclude that some choice is at least as good as the average.

7.4 Walks, paths, and shortest objects

How can we travel from one vertex to another by moving along edges? To answer this precisely, we need names for routes through a graph.

A *walk* from a vertex u to a vertex w is a finite sequence of vertices $v_0, v_1, \ldots, v_k$ that starts at $v_0 = u$, ends at $v_k = w$, and in which every two consecutive vertices are joined by an edge: $v_{i-1}v_i$ is an edge for every $i = 1, \ldots, k$. The *length* of the walk is k, the number of steps it takes. This is one less than the number of entries in the list. A walk may visit the same vertex more than once and may use the same edge more than once; each use of an edge counts as a separate step. A *path* is a walk in which no vertex appears twice. A single vertex v_0, with no steps at all, counts as a walk and a path of length 0 from v_0 to itself.

In the example graph G, the sequence a, c, b, c, d is a walk of length 4 from a to d. Each consecutive pair is an edge: ac, cb, bc, and cd. The walk uses the edge bc twice, once from c to b and once back, and it visits c twice, so it is not a path. The sequence a, c, d is a path of length 2

from a to d. The sequence a, d is not a walk at all, because ad is not an edge. Since edges have no direction, reading a walk backward gives a walk in the opposite direction: d, c, a is a walk from d to a. We call two vertices *connected* when there is a walk from one to the other.

A walk that repeats a vertex contains a detour that can be cut out. In the walk a, c, b, c, d, the vertex c appears in positions 1 and 3, counting positions from 0. Between these two visits the walk makes the round trip c, b, c, which ends where it began. Delete this round trip, keeping a single copy of c, and what remains is a, c, d. The step from c to d used to follow the second visit to c; now it follows the first visit. It is still a legitimate step, because both visits are at the same vertex c. The shortened walk still runs from a to d, and it is two steps shorter. The next lemma uses this operation to show that a walk of smallest possible length can never repeat a vertex.

Lemma 7.6. *Let u and w be vertices of a graph, and suppose that there is a walk from u to w. Then there is a walk from u to w of smallest possible length, and every such shortest walk is a path. In particular, there is a path from u to w.*

Proof. The length of a walk is a natural number. The set of lengths of walks from u to w is nonempty, because at least one such walk exists. By the well-ordering property (Section 4.2, extended to natural numbers in Section 5.2), this set has a smallest member L. Now let $v_0, v_1, \ldots, v_L$ be any walk from u to w of this smallest length L.

Suppose, for contradiction, that some vertex appears twice in this walk: $v_i = v_j$ for some positions $i < j$. Delete the entries $v_{i+1}, \ldots, v_j$, leaving the sequence

$$v_0,\ \ldots,\ v_i,\ v_{j+1},\ \ldots,\ v_L.$$

If $j = L$, nothing comes after v_i, and the sequence simply ends at v_i. We check that this new sequence is a walk from u to w.

- Each pair of consecutive entries is joined by an edge. Every such pair was already consecutive in the original walk, except for the new pair v_i, v_{j+1}, which occurs only when $j < L$. Since $v_i = v_j$, the edge $v_i v_{j+1}$ is the same as $v_j v_{j+1}$, which is an edge of the original walk.
- The sequence starts at $v_0 = u$.
- It ends at w. If $j < L$, its last entry is $v_L = w$. If $j = L$, its last entry is $v_i = v_j = v_L = w$.

Its length is $L - (j - i)$, because $j - i$ entries were deleted. Since $j > i$, this is less than L, which contradicts the choice of L as the smallest length. Hence no vertex appears twice. Since the shortest walk was arbitrary, every shortest walk from u to w is a path. In particular, there is a path from u to w, because a shortest walk of length L exists. □

When u and w are connected, the *distance* from u to w is the length of a shortest walk from u to w. By Lemma 7.6, this is also the length of a shortest path from u to w: a shortest walk is itself a path, and no path can be shorter than the shortest walk, because every path is a walk. In the example graph, the distance from a to d is 2. The path a, c, d has length 2, and no shorter walk exists: a walk of length 0 would need $a = d$, and a walk of length 1 would need ad to be an edge. Notice that establishing this distance took two facts, a walk of length 2 and a reason why no shorter walk exists.

The proof of Lemma 7.6 never built a path step by step. It chose a walk that was as short as possible and showed that this choice already has the property we wanted, because any repeated vertex would allow a cut that produces an even shorter walk. An object chosen to be as small or as large as possible, such as a shortest walk from u to w, is called *extremal*, and an argument that starts from such a choice is called an *extremal argument*. We have met this idea before: Section 4.3 chose a fraction for $\sqrt{2}$ with the smallest possible positive denominator, and Section 5.2 chose a smallest counterexample. The next two sections give two more examples, a longest path and a vertex with as many outgoing edges as possible.

7.5 Directed edges and a guaranteed sink

Some connections have a direction: a one-way street, a link from one web page to another, a game that one player won against another. We record such a connection as an arrow.

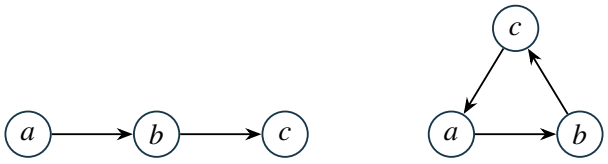
Definition 7.7 (Directed and oriented graphs)**.** A *directed graph* consists of a set V of vertices and a set of *directed edges*. Each directed edge is an ordered pair (u, v) of two different vertices; we write it $u \to v$ and draw it as an arrow from u to v. An *oriented graph* is a directed graph in which no two vertices are joined in both directions: $u \to v$ and $v \to u$ never both occur.

Because the entries of an ordered pair come in a definite order (Section 3.1), $u \to v$ and $v \to u$ are different directed edges. As with graphs, there are no loops: a directed edge always joins two different vertices. Some pairs of vertices may have no edge in either direction. Unlike the graphs of Definition 7.1, a directed graph is allowed to have infinitely many vertices, so we say "finite" explicitly whenever a result needs it.

We also use $u \to w$ as a statement: it says that (u, w) is a directed edge, and we say that $u \to w$ *fails* when (u, w) is not a directed edge. This arrow between two vertices is a different use of the symbol from the arrow in a function $f : A \to B$ (Chapter 3), which points from a set of inputs to a set of outputs. If $u \to w$, we call w an *out-neighbor* of u. The *outdegree* of u is the number of its out-neighbors, that is, the number of arrows leaving u. A *sink* is a vertex with outdegree 0: no arrow leaves it.

Walks and paths now have to follow the arrows. A *directed walk* from v_0 to v_k is a sequence of vertices $v_0, v_1, \ldots, v_k$ with $v_{i-1} \to v_i$ for every $i = 1, \ldots, k$; its length is k. A *directed path* is a directed walk with no repeated vertex. A *directed cycle* is a directed walk $v_0, v_1, \ldots, v_k$ with at least one step, $k \geq 1$, that returns to its start, $v_k = v_0$, and has no other repetition: the vertices $v_0, v_1, \ldots, v_{k-1}$ are all different. A directed graph is *acyclic* when it has no directed cycle. An acyclic directed graph can still look like a triangle when drawn. The directed graph with $a \to b$, $b \to c$, and $a \to c$ has no directed cycle: every arrow goes from a letter to a later letter of the alphabet, so no route along the arrows returns to its start.

In an oriented graph, every directed cycle passes through at least three vertices. A cycle with one step would be a loop $v_0 \to v_0$, and a cycle with two steps would need both $v_0 \to v_1$ and $v_1 \to v_0$. Both are forbidden. Here are two oriented graphs, each with vertices a, b, c:



On the left is the *chain* $a \to b \to c$. It is acyclic: every arrow leads further along the chain, so no directed walk can come back. Its vertex c is a sink, while a and b are not. On the right is the directed cycle $a \to b \to c \to a$.

Does every directed graph have a sink? Not without some hypotheses. A directed graph with no vertices has no sink, simply because it has no vertices at all. An infinite directed graph can fail too. Take the natural numbers as vertices, with an edge $n \to n+1$ for every n. Every vertex points to the next one, so there is no sink. Yet this graph is acyclic, because the numbers strictly increase along any directed walk, so a walk can never return to its starting number. Exercise 7.3 asks what can happen when the graph is finite but not acyclic. The next theorem shows that a sink must exist once the graph is nonempty, finite, and acyclic.

Theorem 7.8. *Every nonempty finite acyclic oriented graph has a sink.*

Here *nonempty* means that there is at least one vertex. The theorem promises *one* vertex with no outgoing edge; it does not say that every vertex has this property. In the chain, only c is a sink. In a directed graph with a single vertex and no edges, that one vertex is a sink. The word "oriented" in the theorem does not exclude any acyclic directed graph: if a directed graph had both $u \to v$ and $v \to u$, then u, v, u would be a directed cycle with two steps. So every acyclic directed graph is automatically oriented. We keep the word because the conjecture in Section 7.7 is about oriented graphs.

Plan. Take a directed path that is as long as possible, and look at its last vertex. An arrow leaving that vertex would either reach a new vertex, which would give a longer path, or return to the path, which would close a directed cycle.

Proof. A directed path never repeats a vertex, so it has at most $|V|$ vertices and therefore length at most $|V| - 1$. (This is the pigeonhole principle: a list of more than $|V|$ vertices must contain some vertex twice.) There is at least one directed path: since the graph is nonempty, we can pick a vertex, and a single vertex is a directed path of length 0. So the lengths of the directed paths form a nonempty set of natural numbers, all at most $|V| - 1$. This set is finite, because it is contained in $\{0, 1, \ldots, |V| - 1\}$, so it has a largest member. Choose a directed path of this greatest length, and write it $v_0 \to v_1 \to \cdots \to v_k$.

Suppose, for contradiction, that its last vertex v_k has an out-neighbor w.

- If w is not on the path, then $v_0 \to v_1 \to \cdots \to v_k \to w$ is a directed walk with no repeated vertex, that is, a directed path. Its length is $k + 1$, which contradicts the choice of a path of greatest length.
- If w is on the path, say $w = v_j$, then $j \neq k$, because $v_k \to v_k$ would be a loop. So $j < k$. Then $v_j \to v_{j+1} \to \cdots \to v_k \to v_j$ is a directed walk with $k - j + 1$ steps that returns to its start, and its vertices $v_j, \ldots, v_k$ before the return are all different. It is a directed cycle, which contradicts acyclicity.

Both cases are impossible. Hence v_k has no out-neighbor, so it is a sink. □

Each hypothesis has a visible job in this proof. Nonemptiness supplies at least one path to choose from. Finiteness guarantees a directed path of greatest length; in the infinite directed graph on the natural numbers, every directed path can be extended by one more step. Acyclicity rules out an arrow from the last vertex back into the path.

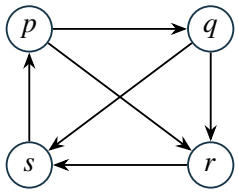
The proof also suggests a way to find a sink. Start at any vertex and keep following arrows, choosing any outgoing arrow at each step, for as long as possible. In an acyclic directed graph the walk never returns to a vertex it has already visited: the first time it did so, the stretch of the walk between the two visits would be a directed cycle. So the walk reaches a new vertex at every step. If the graph is also finite, the walk can visit at most $|V|$ different vertices, so it must stop after at most $|V| - 1$ steps. It can stop only at a vertex with no outgoing arrow, that is, at a sink.

7.6 Tournaments and a vertex of maximum outdegree

In a round-robin competition, every pair of players meets exactly once, and every game has a winner. We can record the results as an oriented graph: the players are the vertices, and $u \to v$ means that u beat v.

Definition 7.9 (Tournament)**.** A *tournament* is an oriented graph in which every pair of different vertices is joined by exactly one directed edge: for all vertices $u \neq v$, exactly one of $u \to v$ and $v \to u$ holds.

In a tournament, the outdegree of a vertex is the number of players it beat. Here is a tournament with four players p, q, r, s and the six results $p \to q$, $p \to r$, $q \to r$, $q \to s$, $r \to s$, and $s \to p$:



Each of the six pairs of players appears exactly once, as a tournament requires. The outdegrees are 2 for p, 2 for q, 1 for r, and 1 for s. The two diagonal arrows cross in the middle of the picture, but, as in Section 7.1, the crossing point is not a vertex.

Call a vertex v a *king* if it can reach every other vertex by a directed path of length at most two. In the language of the competition, for every other player u, either v beat u, or v beat someone who beat u. In the example, p is a king: it beat q and r directly, and it reaches s through $p \to q \to s$. So is q: it beat r and s, and it reaches p through $q \to s \to p$. The player r is not a king. Its only win was against s, and s beat only p, so within two steps r reaches only s and p, not q. Must every nonempty finite tournament have a king? An extremal choice answers this question.

Proposition 7.10. *In a nonempty finite tournament, every vertex of maximum outdegree is a king: it can reach every other vertex by a directed path of length at most two.*

A vertex of maximum outdegree always exists. The tournament is finite and nonempty, so its outdegrees form a nonempty finite set of natural numbers, which has a largest member. So the proposition shows in particular that every nonempty finite tournament has a king.

Proof. Let v be any vertex of maximum outdegree, and let A be the set of its out-neighbors, so that the outdegree of v is $|A|$. Let u be any vertex other than v. There are two ways in which u might be reached quickly.

- If $v \to u$, then u is reached in one step.
- If some $a \in A$ satisfies $a \to u$, then u is reached in two steps by $v \to a \to u$. This is a directed path, because its three vertices are different: $a \neq v$ because $v \to a$ and there are no loops, $u \neq v$ by the choice of u, and $a \neq u$ because $a \to u$.

Suppose, for contradiction, that neither happens. Since $v \to u$ fails and $u \neq v$, the tournament condition gives $u \to v$. Also $u \notin A$, because $v \to u$ fails, so u is different from every $a \in A$. For each $a \in A$, the tournament condition gives $a \to u$ or $u \to a$, and $a \to u$ has been excluded, so $u \to a$. Thus u beat every member of A, and it also beat v, which is not a member of A because a vertex is never its own out-neighbor. So u has at least $|A| + 1$ out-neighbors, while v has exactly $|A|$. This contradicts the assumption that v has maximum outdegree. Therefore one of the two ways always occurs, and v is a king. □

The proof looked only at the games of u against v and against the members of A. If neither way of reaching u works, those $|A| + 1$ games alone give u more wins than v, and the extremal property of v, having as many wins as anyone, rules that out.

Pause and decide. In the example tournament, s won only one game, so it does not have maximum outdegree. Is s a king? What does your answer say about the converse of Proposition 7.10?

Check. Yes, s is a king. It beat p, and p beat q and r, so $s \to p$, $s \to p \to q$, and $s \to p \to r$ reach all three other players within two steps. So a king need not have maximum outdegree. The proposition gives a sufficient condition for being a king, not a necessary one. Its converse, "every king has maximum outdegree," is false, and s is a counterexample. So the kings of the example tournament are p, q, and s.

The next section studies a different and much harder question about two steps in a directed graph: how does the number of vertices reached for the first time in the second step compare with the number reached in the first step? Being a king does not settle that comparison (Exercise 7.6).

7.7 Second out-neighborhoods and a research problem

This section states a problem about directed graphs that resisted every attempt at a proof for more than thirty years. Stating it exactly needs one more idea: distance measured along the arrows.

The deletion argument of Lemma 7.6 works just as well for directed walks: cutting out the part between two visits to the same vertex leaves a shorter directed walk with the same start and end, because the arrow that used to leave the second visit now leaves the first, which is the same vertex. So, as in the lemma, whenever there is a directed walk from v to w, there is a shortest one, and it is a directed path. Its length is called the *directed distance* from v to w. Unlike distance in a graph, directed distance depends on the order of the two vertices. In the chain $a \to b \to c$, the directed distance from a to c is 2, but there is no directed walk from c back to a at all.

Now let v be a vertex of a directed graph with vertex set V. Write $N_1^+(v)$ for the set of out-neighbors of v, and call it the *first out-neighborhood* of v. Its members are exactly the vertices at directed distance 1 from v. The *second out-neighborhood* of v is the set

$$N_2^+(v) = \{w \in V : \ w \neq v, \ w \notin N_1^+(v), \text{ and } v \to u \to w \text{ for some vertex } u\}.$$

The superscript + is a reminder that we follow the arrows outward from v, and the subscript counts the steps. In words, w belongs to $N_2^+(v)$ when some two-step walk $v \to u \to w$ reaches it but no shorter walk does: w is not v itself, at distance 0, and not a first out-neighbor, at distance 1. So $N_2^+(v)$ is the set of vertices at directed distance exactly two from v. It is a set of vertices, not of routes: if several intermediate vertices u lead to the same w, then w is still a single member of $N_2^+(v)$ and is counted once in $|N_2^+(v)|$.

In an oriented graph, the condition $w \neq v$ holds automatically, because a route $v \to u \to v$ would need edges in both directions between v and u. In a directed graph that is not oriented, such a route can occur, and the condition $w \neq v$ is what prevents v from counting as its own second out-neighbor.

Two small examples show the definition at work. In the oriented graph with edges $a \to b$, $b \to c$, $a \to c$, and $c \to d$, the two-step routes from a are $a \to b \to c$ and $a \to c \to d$. The vertex c is at the end of the first route, but it is not in $N_2^+(a)$, because the edge $a \to c$ already makes it a first out-neighbor. So

$$N_1^+(a) = \{b, c\}, \qquad N_2^+(a) = \{d\}, \qquad |N_1^+(a)| = 2, \qquad |N_2^+(a)| = 1.$$

In the directed cycle $a \to b \to c \to a$, each vertex has exactly one first and one second out-neighbor; for instance, $N_1^+(a) = \{b\}$ and $N_2^+(a) = \{c\}$. Here nothing is excluded: the only two-step route from a is $a \to b \to c$, and c is neither a nor an out-neighbor of a.

In 1990 Paul Seymour proposed the following statement, now known as *Seymour's second-neighborhood conjecture*:

> Every nonempty finite oriented graph has a vertex v with $|N_2^+(v)| \geq |N_1^+(v)|$.

In words: some vertex reaches at least as many new vertices in its second step as in its first. In the four-vertex example above, the vertex a fails the inequality, since $1 < 2$. The vertex b satisfies it, since $N_1^+(b) = \{c\}$ and $N_2^+(b) = \{d\}$, so $1 \geq 1$. So does d, for a simpler reason. It is a sink, and a sink always satisfies the inequality: it has no out-neighbors, so it has no two-step routes either, and both of its out-neighborhoods are empty. So the interesting graphs are those in which every vertex has at least one out-neighbor.

The word "oriented" cannot be dropped from the conjecture. Consider the directed graph with two vertices u and v and both edges $u \to v$ and $v \to u$. The only two-step route from u is $u \to v \to u$, which returns to u itself, so $N_1^+(u) = \{v\}$ and $N_2^+(u) = \varnothing$. The same holds for v, so neither vertex satisfies the inequality.

For decades the conjecture remained open, although special cases were settled; the case of tournaments, for example, was proved by David Fisher in 1996. In September 2026, a *preprint*, that is, a research paper made public before a journal has reviewed it, claimed a proof of the full

conjecture. It was released in OpenAI's public mathematics repository together with a formal proof in the Lean proof assistant [3]. The repository's description of the formalized statement uses the same conventions as this section: the second out-neighborhood consists of the vertices at directed distance exactly two, so v and its first out-neighbors are excluded, and graphs with sinks are allowed. The proof is long and draws on ideas well beyond this course. Section 16.4 shows how the definition of $N_2^+(v)$ is written in Lean, and Section 17.8 describes how to begin reading a formalized proof of this kind.

If a news report or an AI assistant tells you that "Seymour's conjecture has been proved," the first thing to check is *which statement* was proved. A proof that counted two-step routes instead of vertices, or that allowed first out-neighbors to count as second out-neighbors, would establish a weaker statement. Each change can only make the second count larger, and a larger second count makes the inequality easier to satisfy. The definitions in this section are what let you ask that question precisely.

We can already prove one special case completely.

Proposition 7.11. *Every nonempty finite acyclic oriented graph has a vertex v with $|N_2^+(v)| \geq |N_1^+(v)|$.*

Proof. By Theorem 7.8, the graph has a sink v. As we saw above, both out-neighborhoods of a sink are empty. Hence $|N_2^+(v)| = 0 \geq 0 = |N_1^+(v)|$. □

This is a complete proof for acyclic graphs, and no proof at all for graphs with directed cycles, which is where all the difficulty lies. In fact, a counterexample to the conjecture would have to be a nonempty finite oriented graph without a sink, and by Theorem 7.8 such a graph must contain a directed cycle. Keeping track of which graphs a proof covers is part of keeping the statement contract of Habit 2 (Section 2.6): the hypothesis "acyclic" belongs to the contract of Proposition 7.11, and removing it changes the statement into the conjecture itself.

Exercises

Exercises 7.1–7.4 are core exercises, and Exercises 7.5 and 7.6 are transfer exercises. Complete solutions are in Appendix B, under "Week 7."

Exercise 7.1. Let G be the example graph of Section 7.1. (a) Add the edge bd to G. List the degrees of the vertices of the new graph, and find their sum and the number of vertices of odd degree. Check that both answers agree with Theorem 7.2 and Proposition 7.3. (b) Do the same for the graph obtained from G by deleting the edge cd instead.

Exercise 7.2. Let x_1, x_2, x_3, x_4, x_5 be five integers, not necessarily different. Prove that there are two indices $i \neq j$ for which x_i and x_j leave the same remainder on division by four. Then give four integers that leave four different remainders on division by four, to show that five cannot be replaced by four.

Exercise 7.3. Show that Theorem 7.8 becomes false if the word "acyclic" is removed: find a nonempty finite oriented graph with no sink. Use as few vertices as possible, and prove that every

nonempty oriented graph with fewer vertices has a sink. Which step of the proof of Theorem 7.8 breaks down for your graph?

Exercise 7.4. Consider the oriented graph with vertices a, b, c, d and edges $a \to b$, $a \to c$, $b \to d$, and $c \to d$. Find $N_1^+(v)$ and $N_2^+(v)$ for every vertex v, and decide which vertices satisfy $|N_2^+(v)| \geq |N_1^+(v)|$. Be careful with d, which can be reached from a along two different routes.

Exercise 7.5. Consider a graph with $n \geq 1$ vertices and m edges. Prove that some vertex has degree at most $2m/n$. Then explain, using an example, why this gives no guarantee that all the degrees are close to the average $2m/n$.

Exercise 7.6. Let v be a king in a tournament, so that every other vertex can be reached from v by a directed path of length at most two. Does it follow that $|N_2^+(v)| \geq |N_1^+(v)|$? A tournament on three vertices is enough to settle the question.

CHAPTER 8

Assignments, repairs, and obstructions

> **This week**
>
> Describe an assignment problem precisely as a graph. See why choices that look reasonable one at a time can lead to a dead end, and learn how to repair an earlier choice without creating a new conflict. Build a search procedure that either enlarges the current assignment or, when it fails, finds a group of students who together accept too few tasks, which proves that no full assignment exists.

8.1 Why one acceptable task per person is not enough

Section 2.4 looked at the statement that all the students of a class can be given different tasks, each acceptable to the student who receives it. The *assignment problem* asks when this can be done, and how to find such an assignment when it can. Here is the simplest way it can fail. Two students, Ada and Ben, each accept task 1, and neither accepts any other task. Looked at one at a time, each student has an acceptable task. Together, however, they cannot be given different tasks, because there is only one task that either of them will take. This is the kind of example that Exercise 2.5 asked you to build.

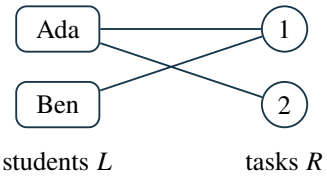
Now change the example a little. Suppose Ada accepts tasks 1 and 2, while Ben still accepts only task 1. This time a solution exists: Ada receives task 2 and Ben receives task 1. But suppose we hand out tasks *greedily*: we go through the students one at a time, give each of them some acceptable task that is still free, and never change a task once it has been given. If we start with Ada and happen to give her task 1, then nothing is left for Ben, and the procedure is stuck. Getting stuck in this way does not prove that the problem has no solution; we have just seen a solution. It shows only that the tasks handed out so far cannot be extended to an assignment for everyone unless some earlier choice is changed. This chapter shows how to change earlier choices in a controlled way, and what we learn when even that fails.

The graph of acceptable pairs

To study such problems precisely, we describe them by a graph in the sense of Definition 7.1. The vertices are the students and the tasks, and an edge joins a student to a task exactly when the student accepts that task. In this graph no edge joins two students, and no edge joins two tasks. Graphs of this shape have a name.

Definition 8.1 (Bipartite graph)**.** A graph is *bipartite*, with *parts* L and R, if its vertex set is the union of two disjoint sets L and R and every edge has one endpoint in L and the other endpoint in R.

The letters stand for "left" and "right," because we usually draw the vertices of L in a column on the left and those of R in a column on the right. Throughout this chapter, L is the set of students and R is the set of tasks. An edge with endpoints $s \in L$ and $t \in R$ means that student s accepts task t. In Chapter 7 this edge would be written st, but that is hard to read when the vertices are called Ada or 1, so we write it s–t instead, with the student first. Here is the graph of the second example:



Its three edges are Ada–1, Ada–2, and Ben–1. We call it the *Ada–Ben graph*. As in Chapter 7, the point where two lines cross in the drawing is not a vertex.

The neighbors of a student s are exactly the tasks that s accepts. We write $N(s)$ for this set of tasks. In the example, $N(\text{Ada}) = \{1, 2\}$ and $N(\text{Ben}) = \{1\}$.

Matchings

An assignment selects some of the edges: the student–task pairs that are actually chosen. The requirement that no task goes to two students, and that no student receives two tasks, says that the selected edges never share an endpoint.

Definition 8.2 (Matching)**.** A *matching* in a graph is a set M of edges of the graph, no two of which share an endpoint. A vertex is *matched* by M if it is an endpoint of some edge of M, and *unmatched* otherwise. Two vertices joined by an edge of M are *partners*. In a bipartite graph with parts L and R, a matching *saturates* L if every vertex of L is matched.

A matched vertex lies on exactly one edge of M: on at least one because it is matched, and on at most one because no two edges of M share an endpoint. So a matched vertex has exactly one partner. In our setting, a matched student holds exactly one task, the student's partner, and a matched task is held by exactly one student, whom we call the task's *owner*. For tasks we also say *used* instead of matched and *unused* instead of unmatched. The empty set of edges is a matching as well: it has no edges at all, so certainly no two of its edges share an endpoint. It leaves every vertex unmatched.

In the Ada–Ben graph, {Ada–2, Ben–1} is a matching that saturates L. The set {Ada–1, Ben–1} is not a matching, because both of its edges have the endpoint 1. A matching that saturates L is exactly a solution of the assignment problem: every student receives a task that the student accepts, and no two students receive the same task. We call such a matching a *full assignment*. A full assignment need not use every task. When there are more tasks than students, some tasks are always left unused.

8.2 Maximal does not mean maximum

In the Ada–Ben graph, a greedy procedure that gives Ada task 1 produces the matching {Ada–1} and then gets stuck, although the matching {Ada–2, Ben–1} solves the problem. Two words describe the difference between these matchings.

Definition 8.3 (Maximal and maximum matchings)**.** A matching M is *maximal* if no edge can be added to it: for every edge e of the graph that is not in M, the set $M \cup \{e\}$ is not a matching. A matching M is *maximum* if no matching in the graph has more edges than M.

The matching {Ada–1} is maximal. The graph has two other edges, and each of them shares an endpoint with Ada–1: the edge Ada–2 shares the endpoint Ada, and the edge Ben–1 shares the endpoint 1. Adding Ada–2 would give Ada two tasks, and adding Ben–1 would give task 1 two owners. But {Ada–1} is not maximum, because {Ada–2, Ben–1} is a matching with two edges. To get from the first matching to the second, we must remove the edge Ada–1 as well as add new edges. A greedy procedure never removes anything, so it can stop at a maximal matching that is not maximum.

Every maximum matching is maximal. Indeed, if some edge e could be added to a matching M, then $M \cup \{e\}$ would be a matching with one more edge than M, so M would not be maximum. The example shows that the converse is false.

Pause and decide. Is the empty matching maximal?

Check. Only when the graph has no edges. A set consisting of a single edge is always a matching, because there is no second edge with which it could share an endpoint. So if the graph has an edge e, then e can be added to the empty matching, which is therefore not maximal. If the graph has no edges, there is nothing to add, and the empty matching is both maximal and maximum.

Comparison with Chapter 6

Section 6.6 defined maximal and maximum elements for any partial order. The matchings of a graph are sets of edges, so we can order them by inclusion, $\subseteq$. Say that a matching M' *strictly contains* M if $M \subseteq M'$ and $M' \neq M$; this is what "lies strictly above" means for the inclusion order. In that order, M is maximal when no matching strictly contains it, and this agrees with Definition 8.3. Indeed, if some edge e can be added to M, then $M \cup \{e\}$ is a matching that strictly contains M. Conversely, suppose that a matching M' strictly contains M. Since $M \subseteq M'$ and $M' \neq M$, some edge e of M' is not in M. The set $M \cup \{e\}$ lies inside M'. Removing edges from a matching cannot create a shared endpoint, so every set of edges contained in a matching is again a matching. Hence $M \cup \{e\}$ is a matching, and the edge e can be added to M.

The word *maximum* is a different matter. In Section 6.6, a maximum lies above every element of the collection, so a maximum matching in that sense would contain every other matching. The Ada–Ben graph has no such matching: a matching containing both {Ada–1} and {Ben–1} would contain two edges with the endpoint 1. For matchings, *maximum* always means *largest number of edges*. Two maximum matchings can be quite different. If students a and b both accept tasks 1 and 2, then $\{a\text{–}1,\ b\text{–}2\}$ and $\{a\text{–}2,\ b\text{–}1\}$ are both maximum matchings, and neither contains the other.

So when an extremal argument (Section 7.4) begins "take a largest matching," it must say which kind it means: a matching that no other matching strictly contains, or a matching with as many edges as possible.

A matching as a function

A matching also tells each matched student which task to do. For the matching {Ada–1}, Ada has task 1 and Ben has none, so this is only a partial assignment. When a matching saturates L, it gives every student a task, and then it describes a function from L to R in the sense of Chapter 3. More precisely, matchings that saturate L correspond exactly to injective functions $f : L \to R$ such that s–$f(s)$ is an edge for every student s.

- Let M be a matching that saturates L. Every student s is matched, so s has exactly one partner; let $f(s)$ be that partner. Then s–$f(s)$ is an edge, because it belongs to M. The function f is injective: if two different students s and s' had $f(s) = f(s') = t$, then the two different edges s–t and s'–t of M would share the endpoint t.
- Conversely, let $f : L \to R$ be injective with s–$f(s)$ an edge for every s. The edges s–$f(s)$ for $s \in L$ form a matching that saturates L. Indeed, take two of these edges, s–$f(s)$ and s'–$f(s')$ with $s \neq s'$. A student in L cannot equal a task in R, because L and R are disjoint, so a shared endpoint would have to be a shared student, $s = s'$, or a shared task, $f(s) = f(s')$. The first is excluded by the choice of s and s', and the second by injectivity. Finally, every student s lies on its edge s–$f(s)$, so the matching saturates L.

The two conditions on f do different jobs. Injectivity (Section 3.3) keeps two students from receiving the same task. The edge condition makes sure that each student receives a task they accept. Neither condition can be left out. In the Ada–Ben graph, the function with $f(\text{Ada}) = f(\text{Ben}) = 1$ satisfies the edge condition but is not injective. The function with $f(\text{Ada}) = 1$ and $f(\text{Ben}) = 2$ is injective, but Ben does not accept task 2.

8.3 An alternating repair

Return to the Ada–Ben graph and the stuck matching $M = \{\text{Ada–1}\}$. Ben is unmatched, and his only acceptable task is held by Ada. The repair is to move Ada to her other task, task 2, and then give task 1 to Ben. Read along the edges of the graph, this repair follows the route

$$\text{Ben} - 1 - \text{Ada} - 2,$$

which is the path Ben, 1, Ada, 2 in the notation of Section 7.4. Of its three edges, Ben–1, Ada–1, and Ada–2, the first and the third are not in M, while the second is. The repair removes from M the edge of the path that was in M and adds the two edges of the path that were not. The result is $\{\text{Ben–1},\ \text{Ada–2}\}$.

Here is what happens at each vertex of the path.

Vertex	Before the repair	After the repair
Ben	no task	task 1
task 1	held by Ada	held by Ben
Ada	task 1	task 2
task 2	unused	held by Ada

Task 1 passes from Ada to Ben, and Ada exchanges task 1 for task 2, so each of these two vertices still has exactly one partner. The two vertices at the ends of the path, Ben and task 2, were unmatched and are now matched. One more student has a task, and nobody who had a task is left without one.

Alternating and augmenting paths

To describe this repair in general, fix a matching M. Two sets of edges must be kept apart. The edge set of the graph lists the *allowed* pairs and never changes, while M lists the pairs *currently chosen* and changes as we repair it. We call the edges of M *matched edges* and the other edges of the graph *unmatched edges*. Recall from Section 7.4 that a walk $v_0, v_1, \dots, v_k$ takes k steps, the ith step running along the edge $v_{i-1}v_i$, and that a path is a walk with no repeated vertex.

Definition 8.4 (Alternating walk, augmenting path)**.** Let M be a matching. A walk is *alternating* if its steps use matched and unmatched edges alternately: of any two consecutive steps, one uses an edge of M and the other uses an edge not in M. An *augmenting path* for M is an alternating path that has at least one step and whose first and last vertices are both unmatched.

Let $v_0, v_1, \dots, v_k$ be an augmenting path. Its first edge v_0v_1 is unmatched, because v_0 is unmatched and so lies on no edge of M. For the same reason its last edge $v_{k-1}v_k$ is unmatched. Since the first edge is unmatched and the edges alternate, the list of edges begins

$$\text{unmatched},\ \text{matched},\ \text{unmatched},\ \text{matched},\ \dots$$

So the ith edge is unmatched when i is odd and matched when i is even. The last edge, the kth, is unmatched, so k is odd. Writing $k = 2r + 1$, the path has $r + 1$ unmatched edges, in the odd positions $1, 3, \dots, 2r + 1$, and r matched edges, in the even positions $2, 4, \dots, 2r$.

In a bipartite graph every edge joins L to R, so the vertices of a walk alternate between students and tasks. After an even number of steps a walk is back in the part where it started, and after an odd number of steps it is in the other part. So a walk with an odd number of steps starts and ends in different parts. An augmenting path has an odd number of steps, so one of its ends is an unmatched student and the other is an unused task. We always list the path starting from the student. The simplest augmenting path is a single edge joining an unmatched student to an unused task; then $k = 1$ and $r = 0$. In the Ada–Ben repair, $k = 3$ and $r = 1$.

Pause and decide. With $M = \{\text{Ada–1}\}$, is the single edge Ben–1 an augmenting path?

Check. No. It is an alternating path with one step, and Ben is unmatched, but its other endpoint, task 1, is held by Ada. Adding Ben–1 to M would give task 1 two owners. An augmenting path must end at an unused task, and here the only way to reach one is to continue through Ada to task 2.

Flipping along an augmenting path

For a longer augmenting path the same exchange happens all along the path. Each student in the middle of the path gives up the task just before them on the path and takes the task just after them. Each task in the middle of the path changes owner. The unmatched student at the start receives a task, and the unused task at the end receives an owner. To *flip* M along a walk P means to remove from M the matched edges that P uses and to add the unmatched edges that P uses; edges that P does not use stay as they were. The next lemma checks that flipping along an augmenting path always gives a larger matching.

Lemma 8.5. *Let M be a matching and let P be an augmenting path for M. Flipping M along P gives a matching M' with exactly one more edge than M.*

Proof. Write P as $v_0, v_1, \ldots, v_k$. Every edge of M' is an edge of the graph, because it is an edge of M or an edge of P. To show that M' is a matching, we check that every vertex lies on at most one edge of M'.

The vertices of a path are all different. Therefore a vertex v_i of P lies on the edge $v_{i-1}v_i$ (if $i \geq 1$) and on the edge v_iv_{i+1} (if $i \leq k-1$), and on no other edge of P: an edge $v_{j-1}v_j$ of P contains v_i only if $v_i = v_{j-1}$ or $v_i = v_j$, that is, only if $j = i+1$ or $j = i$. With this in hand, there are three kinds of vertices to check.

- *A vertex v that is not on P.* No edge of P contains v, so the flip does not change any edge at v. The edges of M' at v are exactly the edges of M at v, and there is at most one of those.
- *An end of P.* Take v_0; the argument for v_k is the same. The only edge of P at v_0 is v_0v_1, which is unmatched, so the flip adds it. Since v_0 is unmatched, no edge of M contains v_0, and the edges at v_0 that are not on P remain outside M'. So v_0 lies on exactly one edge of M'.
- *A vertex v_i inside P, with $0 < i < k$.* Its two path edges $v_{i-1}v_i$ and v_iv_{i+1} are consecutive edges of P, so one of them is in M and the other is not. The flip removes the one in M and adds the other. Because M is a matching and v_i already lies on a matched edge of P, no edge of M outside P contains v_i. Edges at v_i that are neither in M nor on P are not added by the flip. So the only edge of M' at v_i is the path edge that was added.

Hence no vertex lies on two edges of M', and M' is a matching.

Now count. The edges of P are all different, because they join different pairs of vertices. As shown before the lemma, P has $r+1$ unmatched edges and r matched edges for some natural number r. The flip removes the r matched edges from M and adds the $r+1$ unmatched edges, none of which was in M. Therefore $|M'| = |M| - r + (r+1) = |M| + 1$. □

Here is the lemma at work on a longer path, which we will meet again in Section 8.6. Let students a, b, c accept the tasks $N(a) = \{1, 2\}$, $N(b) = \{1\}$, $N(c) = \{2, 3\}$, and let $M = \{a\text{–}1,\ c\text{–}2\}$. Then

$$b - 1 - a - 2 - c - 3$$

is an augmenting path: b is unmatched, task 3 is unused, and the edges b–1, a–1, a–2, c–2, c–3 are unmatched, matched, unmatched, matched, unmatched in turn. Before the flip, a holds task 1

and c holds task 2. Afterwards b holds 1, a holds 2, and c holds 3. The path has $k = 5$ edges, three unmatched and two matched, so $r = 2$ and the matching grows from two edges to three.

The proof relied on the fact that a path never repeats a vertex. If a vertex appeared twice on an alternating walk, it could lie on two unmatched edges of the walk; flipping would add both of them and give that vertex two partners. Exercise 8.4 asks you to find exactly where the proof uses this fact, to build such an example, and to show how the problem can be avoided.

What is preserved and what makes progress

Habit 5 (Section 5.5) asks two questions about a repeated step: what stays true, and what measures progress? Consider a procedure that repeatedly flips along augmenting paths. By Lemma 8.5, the property "the selected edges form a matching" stays true at every step, and the number $|M|$ of selected edges grows by exactly one at every flip. This number measures progress, and it cannot grow forever. Each edge of a matching contains exactly one student, and no two edges of a matching contain the same student, so sending each edge of M to its student is an injective map from M to L, and therefore $|M| \leq |L|$ (Section 3.3). Put differently, the nonnegative integer $|L| - |M|$ decreases by one at every flip, so a procedure that starts from the empty matching makes at most $|L|$ flips. Two questions remain: how to search for an augmenting path, and what we learn when the search finds none.

8.4 Searching for an augmenting path

Let M be a matching and let x be an unmatched student. How can we find an augmenting path that starts at x, or find out that there is none? Think about what such a path must look like. It leaves x along an unmatched edge to some task. If that task is unused, the path can stop there. If the task is used, the path cannot stop there, because an augmenting path ends at an unmatched vertex. Its next step must use a matched edge, because the steps alternate, and a used task lies on only one matched edge, the one joining it to its owner. So the next vertex has to be the owner. From the owner the path continues along an unmatched edge to another task, and so on. This suggests exploring the graph with two kinds of moves:

- from a student s, move to any task t for which s–t is an unmatched edge;
- from a used task t, move to its owner.

The search

The search keeps track of which vertices it has *reached*. When it reaches a new vertex, it also records the vertex it came from, called the *predecessor* of the new vertex. To *inspect* a reached student s, go through the unmatched edges s–t at s one at a time, and do the following for each of them.

- If the task t has already been reached, do nothing and go on to the next edge.

- Otherwise, mark t as reached, with predecessor s. If t is unused, stop: the search has *succeeded*. If t is used, also mark its owner as reached, with predecessor t, and go on to the next edge.

The search itself runs as follows.

1. Mark x as reached. We call x the *root* of the search; it has no predecessor.
2. Choose a reached student that has not been inspected yet, and inspect it.
3. Repeat step 2 until the search succeeds, or until every reached student has been inspected. In the second case the search has *failed*.

A vertex that has already been reached is never marked a second time. For a task this is built into the rule. For a student, notice first that the root x is never the owner of a task, because x is unmatched. Every other student is marked only as the owner of a task that is being reached for the first time, and a student owns only one task. So when the owner of a newly reached task t is marked, it has not been marked before: an earlier marking would have been as the owner of t, and then t would already have been reached. This keeps the search from going round in circles. Each vertex is reached at most once, and each student is inspected at most once. An inspection looks at finitely many edges, and there are finitely many students, so the search always ends.

Suppose the search succeeds at an unused task t. Follow the predecessors backward from t: the predecessor of t is a student; the predecessor of that student, unless it is x, is the task that student holds; and so on. Every vertex was reached later than its predecessor, so going backward we meet the vertices in order of decreasing time of arrival. Hence no vertex is met twice. Since there are only finitely many vertices, the backward walk cannot go on forever. It can stop only at a reached vertex without a predecessor, and the only such vertex is x. Read forward, these vertices form a path from x to t. Every step into a task used an unmatched edge, and every step into a student used a matched edge, so the path is alternating. Both of its ends, x and t, are unmatched, and the path has at least one step, because x is a student and t is a task. So it is an augmenting path, and Lemma 8.5 lets us flip along it. Section 8.6 carries out this search on a small example.

The rule above does not say which uninspected student to choose next. A common choice is to inspect the students in the order in which they were reached; the search is then called *breadth-first search*. Nothing in this chapter depends on the order.

The whole procedure

Repeating the search gives a procedure for the assignment problem. In outline:

```
Start with the empty matching M.
While some student is unmatched:
    choose an unmatched student x and search from x;
    if the search succeeds:
        flip M along the augmenting path it found;
    if the search fails:
        stop, and report the set A of students it reached.
If the loop ends because every student is matched, report M.
```

Every pass through the loop that does not stop the procedure adds one edge to M, and M never has more than $|L|$ edges. So, as at the end of Section 8.3, the procedure stops after at most $|L|$ flips. Notice that when a search fails, the procedure does not merely report failure. It reports a set of students. Section 8.5 shows that this set proves that no matching saturates L.

What a failed search guarantees

When the search fails, the reached vertices have the following two properties. Section 8.5 relies on both of them.

(C1) If s is a reached student and s–t is an unmatched edge, then t has been reached.

(C2) Every reached task is used, and its owner has been reached.

Property (C1) holds because a failed search has inspected every reached student, and inspecting s follows every unmatched edge at s. Property (C2) holds because reaching an unused task would have ended the search with success, and because the owner of a used task is marked as reached at the moment the task is.

A search that follows only the routes that look promising, or that gives up when its first route is blocked, can stop while (C1) is still false. When such a search fails, we learn nothing about the problem, because an augmenting path might use an edge that the search never tried. We call a search *complete* if it stops only when it has found an unused task or has inspected every reached student. The search described in this section is complete.

When a complete search fails, it answers the question we started with: no augmenting path starts at x. To see this, suppose there were one, say $x = v_0, v_1, \ldots, v_k$. Its vertices alternate between students and tasks, starting with the student x, so v_i is a student when i is even and a task when i is odd. Its edges alternate between unmatched and matched, starting with an unmatched one (Section 8.3), so the edge v_iv_{i+1}, which is the $(i+1)$st edge, is unmatched when i is even and matched when i is odd. In short, the edge leaving a student on the path is unmatched, and the edge leaving a task is matched. We show by induction on i that every vertex v_i of the path has been reached. The first vertex, $v_0 = x$, has been reached. Suppose that v_i has been reached, for some $i < k$.

- If v_i is a student, the next edge v_iv_{i+1} is unmatched, so v_{i+1} has been reached by (C1).
- If v_i is a task, then by (C2) it is used and its owner has been reached. The next edge v_iv_{i+1} is matched, and the only matched edge at a used task joins it to its owner, so v_{i+1} is that owner.

In both cases v_{i+1} has been reached. Hence the last vertex v_k, which is a task, has been reached, and by (C2) it is used. But the last vertex of an augmenting path is unused. This contradiction shows that no augmenting path starts at x.

8.5 A failed search exposes a shortage

To say that a group of students is short of tasks, we need the set of all the tasks that the group accepts. For a set S of students, let $N(S)$ be the set of tasks that at least one student of S accepts. In other

words, $N(S)$ is the union of the sets $N(s)$ for s in S; for example, $N(\{a, b, c\}) = N(a) \cup N(b) \cup N(c)$. A task accepted by several students of S is still only one member of $N(S)$. In the Ada–Ben graph, $N(\{\text{Ada}, \text{Ben}\}) = \{1, 2\} \cup \{1\} = \{1, 2\}$. For a single student, $N(\{s\}) = N(s)$, and the empty set of students accepts no tasks at all, so $N(\varnothing) = \varnothing$.

Now suppose that the complete search of Section 8.4, started at an unmatched student x, fails. Let A be the set of students it reached, including x, and let B be the set of tasks it reached.

An example

Let three students a, b, c each accept exactly the tasks 1 and 2, let $M = \{a\text{–}1,\ b\text{–}2\}$, and search from the root $x = c$.

- Inspect c. Both of its edges are unmatched. The edge c–1 reaches task 1, which is held by a, so a is reached as well. The edge c–2 reaches task 2, which is held by b, so b is reached.
- Inspect a. Its only unmatched edge is a–2, and task 2 has already been reached.
- Inspect b. Its only unmatched edge is b–1, and task 1 has already been reached.

Every reached student has been inspected and no unused task was found, so the search fails, with $A = \{c, a, b\}$ and $B = \{1, 2\}$.

The matching pairs task 1 with a and task 2 with b. Every reached student except the root appears in exactly one of these pairs, and the root c appears in none. So A has exactly one more member than B. Moreover, the three students of A accept no tasks other than 1 and 2, so $N(A) = B$. Three students who together accept only two tasks cannot all be given different tasks. The next proposition shows that the same thing happens whenever a complete search fails.

Proposition 8.6. *Let M be a matching in a bipartite graph and let x be an unmatched student. Suppose that the complete search from x fails, and let A and B be the sets of students and tasks that it reached. Then*

$$N(A) = B \qquad \text{and} \qquad |N(A)| = |A| - 1.$$

Proof. The proof has two steps. The first counts the reached vertices, and the second shows that the search has found every task that the students of A accept. Step 1 uses property (C2) of a failed search from Section 8.4, and Step 2 uses property (C1).

Step 1: $|B| = |A| - 1$. For each task t in B, let $f(t)$ be the owner of t. This makes sense, because by (C2) every reached task is used, so it has exactly one owner. Again by (C2), the owner is a reached student. It is not x, because x is unmatched. So f is a function from B to $A \setminus \{x\}$. It is injective: if two different tasks t and t' had the same owner s, then s would lie on the two different edges s–t and s–t' of M, which is impossible in a matching. It is surjective: a student s in A other than x can only have been reached by a move from the task it holds, so that task lies in B, and f sends it to s. Hence f is a bijection, and by Section 3.3 the two sets have the same size: $|B| = |A \setminus \{x\}| = |A| - 1$.

Step 2: $N(A) = B$. We show that each set is contained in the other. Every task t in B was reached by a move from some reached student s along an edge s–t, so $t \in N(s)$, which is contained in $N(A)$. Thus $B \subseteq N(A)$. For the reverse containment, let t be any task in $N(A)$, and choose a

student $s \in A$ that accepts t.

- If the edge s–t is unmatched, then t has been reached, by (C1).
- If the edge s–t is matched, then s is matched, so $s \neq x$, and t is the task that s holds. As in Step 1, the student s was reached by a move from the task it holds, so t was reached before s.

In both cases $t \in B$. Thus $N(A) \subseteq B$, and so $N(A) = B$.

Combining the two steps gives $|N(A)| = |B| = |A| - 1$. □

The smallest case is a useful check. If the root x accepts no task at all, the search inspects x, reaches nothing, and fails at once, with $A = \{x\}$ and $B = \varnothing$. Then $N(A) = N(x) = \varnothing = B$, and $|N(A)| = 0 = |A| - 1$.

Why is Step 2 needed? Step 1 compares the students in A only with the tasks that the search happened to reach. To show that these students are really short of tasks, we must count *all* the tasks they accept, and that is what Step 2 does. Here the completeness of the search is essential. In the example above, suppose the search had stopped right after reaching task 1 and its owner a, before following the edge c–2. Then $A = \{c, a\}$ and $B = \{1\}$, and still $|B| = |A| - 1$. But $N(A) = \{1, 2\}$ has two members, as many as A, so these two students are not short of tasks at all. The comparison of sizes would prove nothing. The complete search cannot stop in this way: property (C1) forces it to follow the edge c–2 as well.

Shortage sets are certificates

A set S of students with $|N(S)| < |S|$ is called a *shortage set*. If a shortage set exists, then no matching saturates L. To see this, suppose that some matching saturated L. Give each student s of S its partner in that matching. This partner is a task that s accepts, so it lies in $N(S)$. Different students of S receive different partners, because two edges of a matching never share an endpoint. So we would have $|S|$ different tasks in $N(S)$, which is impossible when $N(S)$ has fewer than $|S|$ members.

By Proposition 8.6, the set A reported by a failed search satisfies $|N(A)| = |A| - 1 < |A|$, so it is a shortage set, and no full assignment exists. A shortage set is an *obstruction*: it blocks every possible way of assigning tasks, not just the choices that one search happened to make. It is also easy to check. To confirm that S is a shortage set, list the tasks accepted by the students of S and count them. There is no need to trust the search that found S, or to run it again. Evidence that can be checked directly in this way is called a *certificate*, and a shortage set is also called a *shortage certificate*. A full assignment is a certificate in the other direction: to confirm that it solves the assignment problem, check that every student appears in exactly one of its pairs, that each pair is an edge, and that no task appears twice.

So the procedure of Section 8.4 always ends with a certificate: either a full assignment, which can be checked pair by pair, or a shortage set, which can be checked by counting. Chapter 9 turns this observation into a theorem.

Habit 7: make failure informative

A failed attempt to build something is not, by itself, a proof that it cannot be built. Ask whether the failure can be turned into a concrete object that violates a necessary condition. In this chapter, the necessary condition is that every group of students accepts at least as many tasks as it has members, and the object is a group that accepts too few. In another problem, a failed attempt to invert a function may turn up two different inputs with the same output, which shows that the function has no inverse. When a search, yours or an assistant's, reports that nothing was found, ask whether it also produced such an object, and whether the search could have missed anything.

8.6 The whole procedure on a small example

Three students a, b, c and three tasks 1, 2, 3 are given, with acceptable tasks

$$N(a) = \{1, 2\}, \qquad N(b) = \{1\}, \qquad N(c) = \{2, 3\}.$$

We run the procedure of Section 8.4, taking the unmatched students as roots in the order a, c, b, and inspecting the edges at each student in increasing order of task number.

1. The matching starts empty. The search from a follows the edge a–1 and finds that task 1 is unused. The augmenting path is the single edge a–1, and flipping along it gives $M = \{a\text{–}1\}$.
2. The search from c follows the edge c–2 and finds that task 2 is unused. Flipping gives $M = \{a\text{–}1,\ c\text{–}2\}$.
3. Now b is the only unmatched student, and the only task b accepts is held by a. These first two rounds made exactly the choices of a greedy procedure, and a greedy procedure would now be stuck. The search from b reaches the following vertices, in this order.

Vertex reached	Predecessor	Reason
b	—	the root of the search
task 1	b	inspecting b: the edge b–1 is unmatched
a	task 1	task 1 is held by a
task 2	a	inspecting a: the edge a–2 is unmatched (a–1 is matched)
c	task 2	task 2 is held by c
task 3	c	inspecting c: the edge c–3 is unmatched (c–2 is matched)

Task 3 is unused, so the search succeeds. Following the predecessors backward from task 3 gives 3, c, 2, a, 1, b, and read forward this is the augmenting path

$$b - 1 - a - 2 - c - 3$$

that we met after Lemma 8.5. The picture shows the flip.



Solid: matched edges, removed by the flip.
Dashed: unmatched edges, added by the flip.

After the flip, the matching is $\{b\text{–}1,\ a\text{–}2,\ c\text{–}3\}$. Every student is matched, so the procedure stops and reports this assignment. Anyone can check it: $1 \in N(b)$, $2 \in N(a)$, $3 \in N(c)$, and the three tasks are different. The augmenting path showed exactly how to revise the greedy choices a–1 and c–2.

Now remove task 3 from c's options, so that $N(c) = \{2\}$, and run the procedure again in the same order. The first two rounds are unchanged and give $M = \{a\text{–}1,\ c\text{–}2\}$. The search from b again reaches task 1, then a, then task 2, then c. But now the only edge at c is the matched edge c–2, so inspecting c reaches nothing new. Every reached student has been inspected, and the search fails with

$$A = \{b, a, c\}, \qquad B = \{1, 2\}.$$

As Proposition 8.6 predicts, $|B| = |A| - 1$ and

$$N(A) = N(a) \cup N(b) \cup N(c) = \{1, 2\} \cup \{1\} \cup \{2\} = \{1, 2\} = B.$$

So $|N(A)| = 2 < 3 = |A|$, and A is a shortage set. The failure is not the result of an unlucky order of choices. The three students together accept only two tasks, so no way of handing out tasks, in any order, could give all three of them different acceptable tasks.

Exercises

Exercises 8.1–8.4 are core exercises, and Exercises 8.5 and 8.6 are transfer exercises. Complete solutions are in Appendix B, under "Week 8."

Exercise 8.1. In the Ada–Ben graph of Section 8.1, Ada accepts tasks 1 and 2, and Ben accepts only task 1. List all the matchings of this graph, decide which of them are maximal and which are maximum, and conclude that {Ada–1} is the only matching that is maximal but not maximum.

Exercise 8.2. Students a, b, c accept the tasks $N(a) = \{1, 2\}$, $N(b) = \{1, 3\}$, $N(c) = \{2\}$. Start from the matching $\{a\text{–}2,\ b\text{–}1\}$. Use the search of Section 8.4 to find an augmenting path, and write down the matching obtained by flipping along it.

Exercise 8.3. Three students each accept exactly the tasks 1 and 2, as in the example of Section 8.5. Give a shortage certificate, and explain why the fact that every student has two acceptable tasks is not enough for a full assignment. Then show that no smaller group of these students is a shortage set.

Exercise 8.4. The proof of Lemma 8.5 uses the fact that an augmenting path has no repeated vertex. (a) Explain where the proof uses this fact. Give an example of a matching and an alternating walk from an unmatched student to an unused task that visits some vertex twice, such that flipping

along the walk does not give a matching. (b) Suppose that some alternating walk leads from an unmatched student to an unused task. Prove that a shortest such walk has no repeated vertex, so that it is an augmenting path.

Exercise 8.5. A procedure works with matchings in a bipartite graph whose set of students is L, and each of its steps enlarges the current matching by exactly one edge, for example by flipping along an augmenting path. Explain why the fact that the matching grows at every step does not, on its own, prove that the procedure stops, and why the finiteness of L supplies the missing ingredient. State an upper bound on the number of steps when the procedure starts from a matching with k edges.

Exercise 8.6. Suppose that the complete search of Section 8.4, started at an unmatched student x, fails, and let A and B be the sets of students and tasks it reached. (a) Without looking back at the proof of Proposition 8.6, prove carefully that $N(A) = B$, treating matched and unmatched edges separately, and point out exactly where the completeness of the search is used. (b) Now suppose instead that the search is stopped early: it has reached no unused task, but some reached student has not been completely inspected. Let A and B again be the sets of students and tasks reached so far. Which of the two containments $B \subseteq N(A)$ and $N(A) \subseteq B$ must still hold? Give an example, different from the one in Section 8.5, in which the other one fails. Then show that whenever it fails, A is not a shortage set.

CHAPTER 9

Hall's theorem: a complete proof apprenticeship

This week

Prove Hall's theorem, which says exactly when every student can be given a different acceptable task. Read a proof with several stages and rebuild it yourself. Prove the easy direction directly, and prove the harder direction twice, once with the search procedure of Chapter 8 and once by strong induction. See why a group of students who between them accept too few tasks is the only possible obstruction to a full assignment. Learn how the case of equality in an inequality can show where to split a problem into smaller ones.

9.1 A condition that checks every group of students

Throughout this chapter we use the model of Chapter 8. A finite bipartite graph (Definition 8.1) has finitely many vertices, split into two disjoint sets: the set L of students and the set R of tasks. An edge s–t joins a student s to a task t that s accepts. A matching (Definition 8.2) is a set of edges no two of which share an endpoint, and it saturates L when every student lies on one of its edges. So a matching that saturates L is exactly a full assignment: every student receives a task they accept, and no task is given to two students.

For a set $S \subseteq L$ of students, $N(S)$ is the set of tasks that are acceptable to at least one student in S, as in Section 8.5. For a single student x we write $N(x)$ instead of $N(\{x\})$; it is the set of tasks that x accepts. The letter N stands for *neighbors*: the tasks in $N(S)$ are the vertices joined by an edge to some member of S.

Section 8.5 showed that a *shortage set*, a group S of students with $|N(S)| < |S|$, makes a full assignment impossible: those students need $|S|$ different tasks, but fewer than $|S|$ tasks are acceptable to anyone in the group. Hall's theorem says that this is the only thing that can go wrong. It concerns the condition

$$|N(S)| \geq |S| \qquad \text{for every subset } S \subseteq L. \tag{H}$$

In words: every group of students accepts, between them, at least as many tasks as there are students in the group. We call (H) *Hall's condition*. You may have met it already: it is the group condition in the solution to Exercise 2.5.

Why must we check every group? Checking each student on their own misses students who compete for the same few tasks. Checking only the whole class misses a smaller group that is short

of tasks. Here is an example. Let

$$N(a) = N(b) = \{1\}, \qquad N(c) = \{2, 3\}.$$

Every student accepts at least one task, and the whole class $L = \{a, b, c\}$ accepts the three tasks $1, 2, 3$, as many tasks as there are students. Yet the group $\{a, b\}$ accepts only task 1. Two students cannot both receive task 1, so no full assignment exists. The trouble lies in a group of two students, and neither the single-student checks nor the whole-class check looks at that group.

So read (H) as a list of tests, one for each group S. To run the test for S, collect into $N(S)$ every task that some member of S accepts, and compare $|N(S)|$ with $|S|$. A task accepted by three members of S still counts only once in $N(S)$, because it can be given to only one of them.

How many tests are there? Suppose the class has n students. To form a group, decide for each student separately whether they belong to it. That is two choices for each of the n students, so there are $2 \cdot 2 \cdots 2 = 2^n$ groups, counting the empty group and the whole class. For three students there are eight tests. The empty group $S = \varnothing$ has no members, so $N(\varnothing) = \varnothing$ and its test reads $0 \geq 0$, which always holds. A group with a single student x tests $|N(x)| \geq 1$, that is, whether x accepts at least one task. The whole class tests whether there are at least as many acceptable tasks as students overall. The remaining groups, which have at least two students but are not the whole class, test competition within part of the class, and the example above shows that this competition can be exactly where an assignment fails.

Theorem 9.1 (Hall's matching theorem)**.** *Let G be a finite bipartite graph with parts L and R. Then G has a matching that saturates L if and only if $|N(S)| \geq |S|$ for every subset $S \subseteq L$.*

The theorem is an "if and only if" statement, so it makes two separate claims. The first says that if a full assignment exists, then (H) holds. In the language of Section 2.1, Hall's condition is *necessary* for a full assignment. The second claim says that if (H) holds, then a full assignment exists, so the condition is also *sufficient*.

Philip Hall published this theorem in 1935. It has since become one of the basic tools of combinatorics, the part of mathematics that studies finite arrangements and how to count them. Section 9.2 proves the necessary direction. We then prove the sufficient direction twice. The first proof, in Section 9.3, uses the search procedure of Chapter 8 and actually constructs an assignment. The second, in Section 9.4, uses strong induction from Chapter 5. For further reading on graphs and matchings, see [1].

9.2 First prove the necessary direction

We begin with the easier half of Theorem 9.1: if a full assignment exists, then every group passes its test. The argument is the one that Section 8.5 used for shortage sets, now written as a direct proof.

Proof that Hall's condition is necessary. Suppose that a matching M saturates L, and let S be any subset of L. Each student $s \in S$ lies on exactly one edge of M: on at least one because M saturates L, and on at most one because no two edges of M share an endpoint. Call the task at the other end

of that edge $f(s)$. This defines a function $f : S \to N(S)$. Its values really do lie in $N(S)$: the pair s–$f(s)$ is an edge of the graph, so the task $f(s)$ is acceptable to s, which is a member of S.

The function f is injective. If two different students $s \neq s'$ had $f(s) = f(s')$, that task would be an endpoint of the two different edges s–$f(s)$ and s'–$f(s')$ of M, which a matching does not allow. As we saw in Section 3.3, an injective function from a finite set S to a finite set $N(S)$ produces $|S|$ different members of $N(S)$, so $|S| \leq |N(S)|$. Since S was an arbitrary subset of L, condition (H) holds. □

The proof also covers the empty group. When $S = \varnothing$, the function f has no inputs at all, and the conclusion is simply $0 \leq 0$.

Here is the proof at work on the graph of Section 8.6, where

$$N(a) = \{1, 2\}, \qquad N(b) = \{1\}, \qquad N(c) = \{2, 3\},$$

and the matching b–1, a–2, c–3 saturates L. For the group $S = \{a, b\}$, the function f sends a to 2 and b to 1. These are two different tasks, and both lie in $N(S) = \{1, 2\}$. So $N(S)$ has at least two members, as Hall's condition requires.

Notice how little the proof needed. It looked only at the edges of the matching that touch S, and it never asked how the matching was found. Read as a contrapositive, it says that if some group S has $|N(S)| < |S|$, then no matching saturates L. In other words, a shortage set is a certificate of impossibility.

The other direction is harder. This time we are given only the inequalities in (H), and they say nothing directly about which student should receive which task. From them alone we must produce an actual assignment, in which the choices for all the students fit together at once.

9.3 A constructive proof from the search procedure

Chapter 8 already contains almost everything we need for the sufficient direction. Recall the complete search of Section 8.4. It works with a fixed matching M and starts at an unmatched student x. From a reached student, the search follows every edge that is not in M to the task at its other end. When it reaches a task that is held by some student, it also reaches that task's owner. Each vertex is reached at most once, and for each reached vertex other than x the search remembers its predecessor, the vertex from which it was reached. The search succeeds as soon as it reaches an unused task, and it fails if it has inspected every reached student without finding one. There are finitely many vertices, so one of these two things happens. Chapter 8 proved a fact about each outcome.

- If the search succeeds, then following the predecessors back to x gives an augmenting path (Section 8.4). Flipping M along this path gives a matching with one more edge (Lemma 8.5).
- If the search fails, let A be the set of students it reached, x included, and let B be the set of tasks it reached. Then $N(A) = B$ and $|B| = |A| - 1$ (Proposition 8.6). Hence $|N(A)| = |A| - 1 < |A|$.

In the second case, A is a shortage set. Hall's condition says that no group of students is short of tasks, so under (H) the second case can never happen. That is the whole idea of the proof.

Constructive proof that Hall's condition is sufficient. Assume that the graph satisfies (H). Start with the empty matching. As long as some student is unmatched, carry out the following round. Choose an unmatched student x and run the complete search from x. If the search failed, then by Proposition 8.6 the set A of students it reached would satisfy $|N(A)| = |A| - 1 < |A|$, contradicting (H). So the search reaches an unused task, and flipping the matching along the resulting augmenting path increases its number of edges by one.

Each edge of a matching contains exactly one student, and different edges contain different students, so a matching has at most $|L|$ edges. The matching starts with 0 edges and gains one edge in every round, so after k rounds it has k edges, and therefore $k \leq |L|$. Thus there are at most $|L|$ rounds. The rounds stop only when no student is unmatched, so the final matching saturates L. □

If $L = \varnothing$, no round takes place at all. The empty matching already saturates L, because there is no student to cover.

Section 8.6 shows both outcomes on a small example. With $N(a) = \{1, 2\}$, $N(b) = \{1\}$ and $N(c) = \{2, 3\}$, the graph satisfies Hall's condition (you can check all eight inequalities). The first two rounds there give a task 1 and c task 2. This is an unlucky start, because task 1 is the only task that b accepts. The third round repairs it along the augmenting path b–1–a–2–c–3, exactly as the proof promises. When task 3 is removed from c's options, Hall's condition fails for the whole class, which then accepts only tasks 1 and 2. The search from b fails, and it reports the group $\{a, b, c\}$, which is the only group whose test fails.

Section 8.5 showed that the procedure always ends with a certificate: a matching that saturates L, or a shortage set. Hall's theorem says that, for each graph, exactly one of the two kinds of certificate exists. If some matching saturates L, the necessary direction shows that there is no shortage set. If no matching saturates L, the sufficient direction, read as a contrapositive, shows that (H) fails, and any group where it fails is a shortage set. This is Habit 7 (Section 8.5) carried to its end: every failure of the procedure produces a concrete obstruction, and Hall's theorem says that a shortage set is the only kind of obstruction there is.

9.4 A second proof: equality tells us where to split

The constructive proof relies on the machinery of alternating paths. A second proof, by strong induction, needs no search at all. It brings out an idea that is useful far beyond matchings: pay special attention to the cases in which an inequality holds with equality. Throughout this section, G is a finite bipartite graph with parts L and R.

Definition 9.2 (Tight group)**.** A set $S \subseteq L$ of students is *tight* if $|N(S)| = |S|$.

When Hall's condition holds, every group has $|N(S)| \geq |S|$, so a tight group is one with no spare tasks: it accepts exactly as many tasks as it has students. The empty group is always tight, because $|N(\varnothing)| = 0 = |\varnothing|$. In the graph of Section 8.6, where $N(a) = \{1, 2\}$, $N(b) = \{1\}$ and $N(c) = \{2, 3\}$, the group $\{b\}$ is tight because $N(b) = \{1\}$, and so is $\{a, b\}$, because $N(\{a, b\}) = \{1, 2\}$. The whole class $\{a, b, c\}$ is tight too, since it accepts exactly the three tasks $1, 2, 3$. The group $\{c\}$ is not tight, because c accepts two tasks.

Why do tight groups matter? Suppose that S is tight and that a full assignment exists. The

students of S receive $|S|$ different tasks, all of them in $N(S)$, and $N(S)$ has only $|S|$ members. So the students of S use up every task in $N(S)$, and all the other students must be served by tasks outside $N(S)$. A tight group is therefore a self-contained block. This suggests splitting the problem in two: solve it for the students of S using only the tasks in $N(S)$, and solve it for everyone else without those tasks.

To use this idea in an induction, both pieces must be smaller than the original problem. A subset S of L is called *proper* if $S \neq L$, and we then write $S \subsetneq L$. Suppose that S is nonempty and proper. Then $|S| > 0$. Since $S \subseteq L$ but $S \neq L$, at least one student of L lies outside S, so $|S| < |L|$. The set $L \setminus S$ of remaining students has $|L| - |S|$ members, and this is also fewer than $|L|$, because $|S| > 0$. So both S and $L \setminus S$ have fewer students than L.

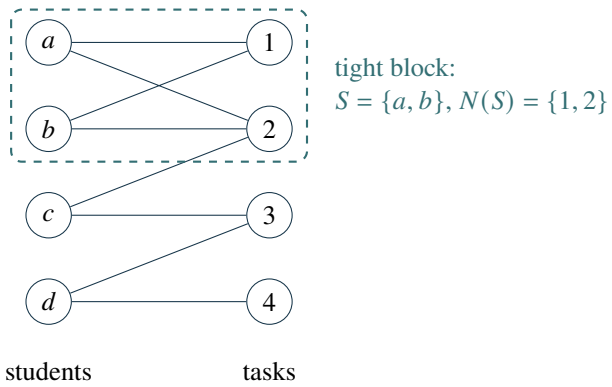
Proof plan. Use strong induction on the number of students. If some nonempty proper group S is tight, solve the problem for the students of S using only the tasks in $N(S)$, and separately solve it for the remaining students using only the remaining tasks. If no nonempty proper group is tight, every such group has at least one spare task. This spare capacity makes it safe to give any one student any task that student accepts, and we show that the smaller problem that remains still satisfies Hall's condition.

The two cases in small examples

Here is the first case of the plan at work. Four students have the options

$$N(a) = N(b) = \{1, 2\}, \qquad N(c) = \{2, 3\}, \qquad N(d) = \{3, 4\}.$$

In the drawing below, students are on the left and tasks on the right, and an edge means that the student accepts the task. The dashed box surrounds a tight block.



This graph satisfies Hall's condition, as you can confirm by checking all $2^4 = 16$ inequalities of (H). The group $S = \{a, b\}$ accepts exactly two tasks, so it is tight. Any full assignment must give a and b the tasks 1 and 2, in some order. So we set a and b aside, together with tasks 1 and 2. Each remaining student keeps only the tasks outside $\{1, 2\}$: student c now accepts only task 3, and student d accepts tasks 3 and 4. These two students can take 3 and 4 respectively. Together with a–1 and b–2, this gives a full assignment.

For a group T of remaining students, write $N'(T)$ for the set of tasks that someone in T accepts once the tasks of $N(S)$ have been removed:

$$N'(T) = N(T) \setminus N(S).$$

The mark $'$, read "prime", shows that $N'(T)$ belongs to the smaller problem. This set can be smaller than $N(T)$. For $T = \{c\}$, the original set $N(T) = \{2, 3\}$ shares task 2 with $N(S) = \{1, 2\}$, and after the removal $N'(T) = \{3\}$. Why must at least one of c's tasks survive the removal? Look at the combined group $S \cup T = \{a, b, c\}$. By Hall's condition it accepts at least three tasks; here $N(S \cup T) = \{1, 2, 3\}$. The tight block S accounts for only two of these tasks, so at least one of them lies outside $N(S)$. No student of S accepts that task, so c, the only student of T, must accept it.

The general proof must show that $|N'(T)| \geq |T|$ for every group T of remaining students. As in the case $T = \{c\}$, it gets this from Hall's condition for the combined group $S \cup T$. Hall's condition for T alone would not be enough, because it gives no control over how many of T's tasks lie in $N(S)$ and disappear. For instance, Hall's condition for $T = \{c\}$ says only that $|N(c)| \geq 1$. That would still be true if c accepted only task 2, and then c would have nothing left after the removal. But in that case the group $\{a, b, c\}$ would accept only the two tasks 1 and 2, and Hall's condition for $S \cup T$ would fail.

Here is the second case of the plan in a small example. Let $N(a) = \{1, 2\}$ and $N(b) = \{2, 3\}$. The nonempty proper groups are $\{a\}$ and $\{b\}$. Each accepts two tasks, one more than it needs, so neither is tight. The plan lets us give a any task that a accepts, even task 2, which b also wants. Then b still has task 3, and $\{a\text{–}2,\ b\text{–}3\}$ is a full assignment. Here the spare task of the group $\{b\}$ absorbs the loss of task 2. If we give a task 1 instead, b loses nothing and keeps both of its tasks.

The inductive proof

We prove the sufficient direction of Theorem 9.1 by strong induction on the number of students (Section 5.2). For each natural number n, let $P(n)$ be the statement:

> *Every finite bipartite graph whose left part has exactly n vertices, and which satisfies Hall's condition, has a matching that saturates its left part.*

The statement places no restriction on the right part, which may have any finite number of vertices. This matters, because the smaller graphs to which we apply the induction hypothesis lose some tasks as well as some students. For any finite bipartite graph, Hall's condition means (H) computed in that graph: every set U of vertices in its left part is joined, by edges of that graph, to at least $|U|$ vertices of its right part. In a smaller graph, the tasks joined to U can be fewer than the tasks in $N(U)$, because some edges are gone.

Inductive proof that Hall's condition is sufficient. Base case, $n = 0$. If L is empty, the empty matching saturates L, because there is no student to cover. So $P(0)$ holds.

Induction step. Let $n \geq 1$, and assume that $P(m)$ holds for every natural number $m < n$, that is, that $P(0), P(1), \ldots, P(n-1)$ all hold. This assumption is the induction hypothesis, and n plays the role of $k + 1$ in the strong induction step of Section 5.2. Let G be a finite bipartite graph with

parts L and R, where $|L| = n$, and suppose that G satisfies (H). We must find a matching of G that saturates L. There are two cases.

Case 1: some nonempty proper subset of L is tight. Choose a set S with $\varnothing \neq S \subsetneq L$ and $|N(S)| = |S|$.

The students in S. Let G_1 be the graph with left part S, right part $N(S)$, and as edges all the edges of G that join a student of S to a task of $N(S)$. Every task that a student of S accepts lies in $N(S)$, so G_1 keeps every edge of G at the students of S. Hence, for every $U \subseteq S$, the set of tasks joined to U in G_1 is exactly $N(U)$. Since U is also a subset of L, condition (H) for G gives $|N(U)| \geq |U|$. So G_1 satisfies Hall's condition. Since $0 < |S| < n$, the induction hypothesis $P(|S|)$ gives a matching M_1 of G_1 that saturates S.

The other students. Let G_2 be the graph with left part $L \setminus S$, right part $R \setminus N(S)$, and as edges all the edges of G that join a student of $L \setminus S$ to a task of $R \setminus N(S)$. Let T be any subset of $L \setminus S$. A task is joined to T in G_2 exactly when someone in T accepts it and it does not lie in $N(S)$. So the set of tasks joined to T in G_2 is $N'(T) = N(T) \setminus N(S)$.

We show that $|N'(T)| \geq |T|$. The sets S and T are disjoint, because T contains no student of S, so $|S \cup T| = |S| + |T|$. A task is acceptable to someone in $S \cup T$ exactly when it is acceptable to someone in S or to someone in T, so $N(S \cup T) = N(S) \cup N(T)$. Applying (H) to the set $S \cup T$ therefore gives

$$|N(S) \cup N(T)| \geq |S| + |T|.$$

Every member of $N(S) \cup N(T)$ lies either in $N(S)$ or in $N(T) \setminus N(S)$, and no member lies in both. So the union is made up of the two disjoint sets $N(S)$ and $N'(T)$, and

$$|N(S) \cup N(T)| = |N(S)| + |N'(T)|$$

(Exercise 9.3 asks you to check this carefully). Combining the last two displays,

$$|N(S)| + |N'(T)| \geq |S| + |T|.$$

Now we use tightness. Substitute $|N(S)| = |S|$ and subtract $|S|$ from both sides to obtain $|N'(T)| \geq |T|$. This holds for every $T \subseteq L \setminus S$, including $T = \varnothing$. So G_2 satisfies Hall's condition. Since S is nonempty, G_2 has $n - |S| < n$ students, and the induction hypothesis $P(n - |S|)$ gives a matching M_2 of G_2 that saturates $L \setminus S$.

Combining the two matchings. The edges of G_1 and of G_2 are edges of G, so $M_1 \cup M_2$ is a set of edges of G. Every edge of M_1 joins a student of S to a task of $N(S)$, and every edge of M_2 joins a student of $L \setminus S$ to a task of $R \setminus N(S)$. So no student and no task lies both on an edge of M_1 and on an edge of M_2. Within each of the two matchings, no two edges share an endpoint either. Hence $M_1 \cup M_2$ is a matching of G. Every student of L lies in S or in $L \setminus S$, so that student is matched by M_1 or by M_2. Thus $M_1 \cup M_2$ saturates L.

Case 2: no nonempty proper subset of L is tight. Let S be a nonempty proper subset of L. Condition (H) gives $|N(S)| \geq |S|$, and $|N(S)| \neq |S|$ because S is not tight, so $|N(S)| > |S|$. Both sides are integers, and an integer larger than the integer $|S|$ is at least $|S| + 1$ (Exercise 9.4). Therefore

$$|N(S)| \geq |S| + 1 \qquad \text{for every set } S \text{ with } \varnothing \neq S \subsetneq L.$$

Since $n \geq 1$, the set L is not empty, so we can choose a student $x \in L$; any student will do. Condition (H) for the group $\{x\}$ gives $|N(x)| \geq 1$, so x accepts at least one task y. Let G' be the graph obtained from G by deleting the student x, the task y, and every edge at x or at y. Its left part $L \setminus \{x\}$ has $n - 1$ students, and its right part is $R \setminus \{y\}$.

We check Hall's condition for G'. Let $T \subseteq L \setminus \{x\}$. If $T = \varnothing$, the condition $0 \geq 0$ holds. Otherwise, consider the tasks joined to T in G'. The only edges of G that G' has lost are those at x and those at y. The edges at x do not touch T, because $x \notin T$. So a task is joined to T in G' exactly when it lies in $N(T)$ and is not y, and the set of these tasks is $N(T) \setminus \{y\}$. The set T is nonempty, and it is a proper subset of L because it does not contain x. So $|N(T)| \geq |T| + 1$. Removing the single task y removes at most one member of $N(T)$, so

$$|N(T) \setminus \{y\}| \geq |N(T)| - 1 \geq |T|.$$

Thus G' satisfies Hall's condition, and the induction hypothesis $P(n - 1)$ gives a matching M' of G' that saturates $L \setminus \{x\}$.

The edges of M' are edges of G, and so is x–y. This edge shares no endpoint with any edge of M', because x and y are not vertices of G'. So $M' \cup \{x\text{–}y\}$ is a matching of G. It matches x and every student of $L \setminus \{x\}$, so it saturates L.

When $n = 1$, the set $L = \{x\}$ has no nonempty proper subset at all, so Case 1 cannot occur and this case is the one that applies. Then $L \setminus \{x\}$ is empty, M' is the empty matching given by $P(0)$, and the single edge x–y saturates L.

Every graph falls into Case 1 or Case 2, and in both cases G has a matching that saturates L. So $P(n)$ holds. By strong induction, $P(n)$ holds for every natural number n, and this proves the sufficient direction of Hall's theorem. □

9.5 Why every part of the case split is needed

Each condition in the case split of the inductive proof has a definite job. A good way to see the job is to drop the condition and watch the proof break.

Why must S be nonempty? Recall that the empty group is always tight. If Case 1 allowed $S = \varnothing$, then the first smaller problem would have no students, and the second would be the original problem all over again, because $L \setminus \varnothing = L$ and $R \setminus N(\varnothing) = R$. The induction hypothesis says nothing about a graph with n students, so the proof would make no progress.

Why must S be proper? The whole class can be tight as well. In the four-student example of Section 9.4, the whole class accepts exactly the four tasks $1, 2, 3, 4$. If Case 1 allowed $S = L$, the first of the two smaller problems would again contain all n students. It would be the original problem with only the tasks that nobody accepts thrown away, and the induction hypothesis could not be applied to it.

Why must S be tight? Case 1 removes every task in $N(S)$ from the second problem. If S had spare tasks, this could throw away tasks that other students need. Take $N(a) = \{1, 2\}$ and $N(b) = \{2\}$, and let $S = \{a\}$. This group is nonempty and proper, but it is not tight, since $|N(S)| = 2 > 1$. Removing $N(S) = \{1, 2\}$ leaves b with no acceptable task, although a–1, b–2 is a full assignment.

In the proof, tightness is used at exactly one point: we substitute $|N(S)| = |S|$ and subtract $|S|$ from both sides.

Pause and decide. In Case 1, why do we remove every task in $N(S)$, rather than only the tasks that the matching M_1 actually gives to the students in S?

Check. Because S is tight, these are the same tasks: M_1 gives the $|S|$ students of S different tasks inside $N(S)$, which has only $|S|$ members, so it uses them all. Describing the removed tasks as $N(S)$ has one advantage: we can name them before M_1 has been found, so the two smaller problems can be set up and solved independently.

Why does Case 2 need the extra unit? Giving x the task y can cost each remaining group one of its acceptable tasks. The inequality $|N(T)| \geq |T| + 1$ is exactly enough to pay for that loss. Without it, an arbitrary first choice can be fatal. Recall the example from Section 8.1 in which Ada accepts tasks 1 and 2 and Ben accepts only task 1. If we give Ada task 1, Ben is left with nothing, although Ada–2, Ben–1 is a full assignment. The proof never makes this mistake. The group {Ben} is nonempty, proper and tight, so this graph belongs to Case 1, not Case 2. Case 1 first serves the tight block {Ben} with task 1, and then Ada takes task 2.

Applying a theorem means checking all of its hypotheses

Suppose an assistant proposes the proof "split the students into a group and the rest, and apply induction to each part." That is not yet a proof of the theorem. The induction hypothesis is not the statement "smaller graphs have matchings that saturate their left parts." That statement is false: the graph with $N(a) = N(b) = \{1\}$ has only two students and no such matching. What it actually says is that every finite bipartite graph with fewer than n students *that satisfies Hall's condition* has such a matching. So a proof must show that both smaller problems satisfy Hall's condition. This check is the heart of the proof, and the example under "Why must S be tight?" shows that it fails for a badly chosen group.

Each time the proof applies the induction hypothesis, it must supply three things:

1. a finite bipartite graph, with its left part and right part named;
2. a check that its left part has fewer than n students;
3. a check that it satisfies Hall's condition, that is, an inequality for every subset of its left part.

In Case 1 the proof applies the hypothesis to the two graphs G_1 and G_2, and in Case 2 to the graph G'. The first two items are easy. Deleting some vertices of a finite bipartite graph, together with the edges at them, leaves a finite bipartite graph, and each smaller graph has lost at least one student. The third item is where the work lies, because deleting tasks can destroy Hall's condition. The proof supplies it in two steps. For each group of students in a smaller graph, it first works out exactly which tasks are joined to that group there: $N(U)$ in G_1, $N'(T) = N(T) \setminus N(S)$ in G_2, and $N(T) \setminus \{y\}$ in G'. Then it shows that this set has at least as many members as the group.

Section 2.1 made the same point about a single implication: a result applies only after its hypothesis has been checked. When a proof says "by the induction hypothesis" or "by the theorem," write out the hypotheses and check each one for the object at hand.

9.6 Mathematical taste from equality cases

The inductive proof of Hall's theorem uses the inequality $|N(S)| \geq |S|$ in two different ways. When it holds with equality, the group S is rigid: any full assignment must use up all of $N(S)$ on the students of S. That rigidity told us how to split the problem into two independent pieces. When the inequality is strict, the fact that sizes of sets are integers gives a margin of at least one, and that margin was enough to absorb the loss of one task. The proof comes from asking separately what each situation allows.

The rigidity in the equality case is a general fact from Section 3.3: an injective function $f : A \to B$ between finite sets of the same size is also surjective, because its $|A|$ different outputs lie in B, which has only $|A|$ members, so they make up all of B. The tight-block argument is exactly this, with the students of S as A, the tasks of $N(S)$ as B, and as f the function that sends each student of S to the task the matching gives that student.

Whenever you meet an inequality, ask more than whether it can be improved: ask what happens when it holds with equality. If equality forces a recognizable structure, that structure may let you break the problem into smaller pieces. And when the inequality is strict, ask how large the margin is and what it can pay for. You will meet a margin of this second kind in Chapter 12. The contraction theorem (Theorem 12.2) requires a number q with $0 \leq q < 1$. Its error bounds divide by the gap $1 - q$, so they make sense only because the gap is positive, and they are strong only when the gap is not too small.

Exercises and midterm portfolio

Exercises 9.1–9.4 are core exercises, and Exercise 9.5 is a transfer exercise. Exercise 9.6 is the midterm portfolio; it draws on both this chapter and Chapter 8. Complete solutions are in Appendix B, under "Week 9," where the solution to Exercise 9.6 is a model portfolio.

Exercise 9.1. Give a graph with four students and four tasks in which every student accepts at least two tasks and $|N(L)| = |L|$, but no matching saturates L. Find a group S of students with $|N(S)| < |S|$, and explain why the tests for single students and for the whole class do not detect the problem.

Exercise 9.2. Let $N(a) = \{1\}$, $N(b) = \{1, 2\}$ and $N(c) = \{2, 3\}$. Check Hall's condition for every subset of $L = \{a, b, c\}$, find every nonempty proper tight subset, and find a matching that saturates L.

Exercise 9.3. In Case 1 of the inductive proof in Section 9.4, prove that $|N(S) \cup N(T)| = |N(S)| + |N(T) \setminus N(S)|$. Do not assume that $N(S)$ and $N(T)$ are disjoint; in the example of that section they are not.

Exercise 9.4. In Case 2 of the inductive proof, a nonempty proper subset S of L satisfies $|N(S)| > |S|$. Explain why it follows that $|N(S)| \geq |S| + 1$. Would the same step be valid if $|N(S)|$ and $|S|$ were replaced by arbitrary real numbers?

Exercise 9.5. Let d be a positive integer. Suppose that every student accepts exactly d tasks and that every task is acceptable to at most d students. Use double counting (Section 7.2) and Hall’s theorem to prove that a full assignment exists. Where does your proof use $d > 0$?

Exercise 9.6. Midterm portfolio. Assemble a short portfolio with four parts. (a) For the graph of Exercise 9.2, give a matching that saturates L, and explain how a reader can check it. (b) For a graph with no such matching, such as the one you found in Exercise 9.1, give a group S of students with $|N(S)| < |S|$, explain how a reader can check it, and explain why it proves that no full assignment exists. (c) Write out, in your own words, a complete proof that Hall’s condition implies the existence of a matching that saturates L. You may follow either proof in this chapter. You may quote results proved in Chapter 8, such as Lemma 8.5 and Proposition 8.6, if you state what they say. (d) Write a paragraph that explains the difference between a maximal and a maximum matching, and why a greedy assignment can get stuck even when a full assignment exists.

CHAPTER 10

Linearity: changing the language of a problem

This week

Work with vectors as ordered lists of numbers, and recognize a linear map as a map that preserves addition and scaling. See why a matrix, a finite table of numbers, can describe such a map completely. Use the kernel to decide when a linear map is injective, prove the Cauchy–Schwarz and triangle inequalities, and see why matrices are multiplied the way they are. Throughout, notice how a change of language, such as a new basis or a matrix in place of a formula, can explain why a fact is true instead of only confirming it by calculation.

10.1 Vectors are ordered measurements

Let n be a positive integer. A *vector* in $\mathbb{R}^n$ is an ordered list $x = (x_1, x_2, \ldots, x_n)$ of n real numbers, and the numbers $x_1, \ldots, x_n$ are its *coordinates*. So $\mathbb{R}^n$ is the set of all such lists. For $n = 2$ the vectors are ordered pairs (x_1, x_2) of real numbers. We picture (x_1, x_2) as the point of the plane that lies x_1 units to the right of a fixed point, the *origin*, and x_2 units above it; a negative coordinate means a step to the left or downward. The coordinates need not describe a position, though. A shop could record 3 kilograms of flour and 5 kilograms of sugar as the vector $(3, 5)$. Order matters: as with the ordered pairs of Section 3.1, $(1, 3)$ and $(3, 1)$ are different vectors, and for the shop they describe different stock. Two vectors are equal exactly when they agree in every coordinate.

Vectors are added coordinate by coordinate. To multiply a vector by a real number c, multiply every coordinate by c:

$$(x_1, \ldots, x_n) + (y_1, \ldots, y_n) = (x_1 + y_1, \ldots, x_n + y_n),$$
$$c(x_1, \ldots, x_n) = (cx_1, \ldots, cx_n).$$

For example, $(1, 3) + (2, -1) = (3, 2)$ and $2(1, 3) = (2, 6)$. In this setting a real number such as c is called a *scalar*, because multiplying by it rescales the vector. The *zero vector* has every coordinate equal to zero. We write it simply as 0 and let the context tell us whether 0 means a number or a vector. The *negative* of x is $-x = (-1)x = (-x_1, \ldots, -x_n)$, and subtraction means adding the negative: $x - y = x + (-y) = (x_1 - y_1, \ldots, x_n - y_n)$. The familiar rules of arithmetic carry over from numbers to vectors, because each rule can be checked one coordinate at a time. For instance, $x + y = y + x$, $x + 0 = x$, $x - x = 0$, $c(x + y) = cx + cy$, and $(a + b)x = ax + bx$ for all vectors x, y and all scalars a, b, c. The last rule lets us collect terms, as in $2x + 3x = 5x$.

A *linear combination* of vectors $v_1, \ldots, v_k$ is a vector of the form

$$a_1 v_1 + a_2 v_2 + \cdots + a_k v_k,$$

where $a_1, \ldots, a_k$ are scalars, called the *coefficients* of the combination. Here the subscripts on $v_1, \ldots, v_k$ number the vectors in a list; each v_j is a whole vector, not a coordinate. For example,

$$2(1,3) - (2,-1) = (2,6) + (-2,1) = (0,7)$$

is a linear combination of $(1,3)$ and $(2,-1)$ with coefficients 2 and -1.

In the plane, the vectors $e_1 = (1,0)$ and $e_2 = (0,1)$ serve as building blocks. Every vector is a linear combination of them, because

$$x_1 e_1 + x_2 e_2 = (x_1, 0) + (0, x_2) = (x_1, x_2).$$

Moreover, the coefficients are forced. If $(x_1, x_2) = a_1 e_1 + a_2 e_2$, the same calculation shows that the right side is (a_1, a_2). Two ordered pairs are equal only when their entries agree, so $a_1 = x_1$ and $a_2 = x_2$.

A list of vectors in $\mathbb{R}^n$ is called a *basis* of $\mathbb{R}^n$ when every vector of $\mathbb{R}^n$ can be written as a linear combination of the list in exactly one way. We have just shown that e_1, e_2 is a basis of $\mathbb{R}^2$. The same reasoning works in $\mathbb{R}^n$ for the vectors $e_1, \ldots, e_n$, where e_j has a 1 in position j and a 0 in every other position. Multiplying e_j by x_j gives the vector with x_j in position j and zeros elsewhere, so adding these n vectors puts x_j in position j for every j:

$$x_1 e_1 + x_2 e_2 + \cdots + x_n e_n = (x_1, x_2, \ldots, x_n).$$

So every vector is a combination of $e_1, \ldots, e_n$, and comparing coordinates shows that the coefficients must be $x_1, \ldots, x_n$. This list is called the *standard basis* of $\mathbb{R}^n$. In $\mathbb{R}^3$, for instance, $(4,5,0) = 4e_1 + 5e_2 + 0e_3$. The number of coordinates of e_j depends on n, so e_1 means $(1,0)$ in the plane but $(1,0,0)$ in $\mathbb{R}^3$; the context always tells us which is meant.

Two requirements: reaching every vector, and reaching it only once

The definition of a basis makes two separate demands. Call a way of writing a vector as a linear combination of the list a *representation* of the vector. First, every vector must have a representation, as $(3,-2) = 3e_1 - 2e_2$ does in the standard basis. A list with this property, so that every vector of $\mathbb{R}^n$ is a linear combination of it, is said to *span* $\mathbb{R}^n$. Second, the representation must be *unique*: the same vector must never arise from two different lists of coefficients. A basis is a list that does both.

Neither demand implies the other. The list consisting of the single vector $(1,1)$ passes the uniqueness test: if $a(1,1) = b(1,1)$, then $(a,a) = (b,b)$, so $a = b$, and no vector is made from $(1,1)$ in two different ways. But this list does not span the plane, because every multiple $a(1,1) = (a,a)$ has equal coordinates, so no multiple equals $(1,0)$. In the other direction, the list $(1,0), (0,1), (1,1)$ does span the plane, since already

$$(x_1, x_2) = x_1(1,0) + x_2(0,1) + 0(1,1).$$

But it does not give unique coefficients. The vector $(1, 1)$ can be made as $0(1, 0) + 0(0, 1) + 1(1, 1)$, and also as $1(1, 0) + 1(0, 1) + 0(1, 1)$.

Uniqueness has a convenient test. A list $v_1, \ldots, v_k$ is called *linearly independent* if the only linear combination of it that equals the zero vector is the one whose coefficients are all zero:

$$a_1 v_1 + \cdots + a_k v_k = 0 \quad \text{implies} \quad a_1 = a_2 = \cdots = a_k = 0.$$

A list is linearly independent exactly when no vector can be written as a combination of the list in two different ways. To see why, suppose first that the list is linearly independent and that some vector x has two representations,

$$x = a_1 v_1 + \cdots + a_k v_k \qquad \text{and} \qquad x = b_1 v_1 + \cdots + b_k v_k.$$

Subtract the second from the first and collect the terms that contain each v_j:

$$(a_1 - b_1)v_1 + \cdots + (a_k - b_k)v_k = x - x = 0.$$

By independence, every difference $a_j - b_j$ is zero, so the two representations were the same after all. Conversely, if the list is not linearly independent, then some combination with at least one nonzero coefficient equals 0. The combination with all coefficients zero also equals 0, so the zero vector has two different representations.

For the list $(1, 0)$, $(0, 1)$, $(1, 1)$, subtracting the second representation of $(1, 1)$ found above from the first gives

$$(-1)(1, 0) + (-1)(0, 1) + 1(1, 1) = (0, 0),$$

a combination equal to zero whose coefficients are not all zero. So this list is not linearly independent. In short, a basis is a list that spans and is linearly independent.

Pause and decide. Is the list $(1, 2)$, $(2, 4)$ a basis of $\mathbb{R}^2$?

Check. No, and it fails both requirements. It is not linearly independent, because $2(1, 2) + (-1)(2, 4) = (0, 0)$. It does not span the plane either: every combination $a(1, 2) + b(2, 4) = (a + 2b,\ 2a + 4b)$ has second coordinate equal to twice its first, so $(1, 0)$ is never reached.

A basis can be chosen to fit the problem

The standard basis is not the only basis of the plane. Take $w_1 = (1, 1)$ and $w_2 = (1, -1)$. Which coefficients a and b give $aw_1 + bw_2 = (x_1, x_2)$? Since $aw_1 + bw_2 = (a + b,\ a - b)$, comparing coordinates shows that we need

$$a + b = x_1 \qquad \text{and} \qquad a - b = x_2.$$

Adding these two equations gives $2a = x_1 + x_2$, and subtracting the second from the first gives $2b = x_1 - x_2$. So the only possible coefficients are

$$a = \frac{x_1 + x_2}{2} \qquad \text{and} \qquad b = \frac{x_1 - x_2}{2},$$

and these coefficients do work, since with them $a+b=x_1$ and $a-b=x_2$. Every vector of the plane therefore has exactly one representation, and w_1, w_2 is a basis. For instance, $(3,1) = 2w_1 + 1w_2$. In this basis, the coefficients of a vector are half the sum and half the difference of its usual coordinates. The vectors themselves have not changed; only the way we describe them has.

This basis suits questions about totals. Suppose the shop from the start of this section cares only about its total stock $x_1 + x_2$. The stock with coefficients a and b in the new basis is $aw_1 + bw_2 = (a+b,\ a-b)$, so its total is $(a+b)+(a-b) = 2a$. For instance, the stock $(3,5)$ has $a=4$ and $b=-1$, and its total is $2\cdot 4 = 8$ kilograms. The total depends only on the first coefficient, and changing b moves stock from one item to the other without changing the total. Choosing a description that suits the question is Habit 1 (Section 1.6) once more, now applied to vectors instead of even numbers. It is a theme of this chapter, and this particular basis will return several times.

10.2 A linear map is determined by its building blocks

Section 6.4 said that a map preserves addition when it sends the sum of two inputs to the sum of their outputs. A map $T : \mathbb{R}^n \to \mathbb{R}^m$ is *linear* when it preserves both addition and multiplication by scalars:

$$T(x+y) = T(x) + T(y) \qquad \text{and} \qquad T(cx) = cT(x)$$

for all vectors $x, y \in \mathbb{R}^n$ and all scalars c. The two conditions can be combined into one: T is linear exactly when

$$T(ax+by) = aT(x) + bT(y) \qquad \text{for all } x, y \in \mathbb{R}^n \text{ and all } a, b \in \mathbb{R}.$$

Indeed, if T satisfies the two conditions, then $T(ax+by) = T(ax) + T(by) = aT(x) + bT(y)$. Conversely, the combined condition with $a=b=1$ is preservation of addition, and with $b=0$ it says $T(ax) = aT(x)$, because $ax + 0y = ax$ and $aT(x) + 0T(y) = aT(x)$.

Taking $c=0$ in the second condition shows that a linear map sends the zero vector to the zero vector: for any vector x, we have $0x = 0$, so $T(0) = T(0x) = 0\,T(x) = 0$.

A linear map sends every linear combination to the same combination of the outputs. For three vectors, first split off the last term using preservation of addition. Then apply the combined condition to the first piece and preservation of scaling to the second:

$$T(a_1v_1 + a_2v_2 + a_3v_3) = T(a_1v_1 + a_2v_2) + T(a_3v_3) = a_1T(v_1) + a_2T(v_2) + a_3T(v_3).$$

Splitting off one term at a time in the same way works for any number k of terms (formally, this is an induction on k, as in Chapter 5):

$$T(a_1v_1 + \cdots + a_kv_k) = a_1T(v_1) + \cdots + a_kT(v_k).$$

Now apply this to the standard basis. For a linear map $T : \mathbb{R}^2 \to \mathbb{R}^m$, writing $(x_1, x_2) = x_1e_1 + x_2e_2$ gives

$$T(x_1, x_2) = T(x_1e_1 + x_2e_2) = x_1T(e_1) + x_2T(e_2).$$

Here $T(x_1, x_2)$ is short for $T\big((x_1, x_2)\big)$, the output at the vector (x_1, x_2); we leave out one pair of brackets. In the same way, writing $x = x_1 e_1 + \cdots + x_n e_n$ shows that every linear map $T : \mathbb{R}^n \to \mathbb{R}^m$ satisfies

$$T(x) = x_1 T(e_1) + x_2 T(e_2) + \cdots + x_n T(e_n).$$

So the n vectors $T(e_1), \ldots, T(e_n)$ determine the output $T(x)$ at every one of the infinitely many inputs x. This is why a linear map can be recorded in a finite table of numbers.

Suppose, for example, that $T : \mathbb{R}^2 \to \mathbb{R}^2$ is linear, with $T(e_1) = (1, 1)$ and $T(e_2) = (1, -1)$. Then

$$T(x_1, x_2) = x_1(1, 1) + x_2(1, -1) = (x_1 + x_2,\ x_1 - x_2).$$

The two outputs $T(e_1)$ and $T(e_2)$ are the basis vectors w_1 and w_2 of Section 10.1. A *matrix* is a rectangular table of numbers. We record T by the matrix whose first column is $T(e_1)$ and whose second column is $T(e_2)$:

$$A = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad Ax = \begin{pmatrix} x_1 + x_2 \\ x_1 - x_2 \end{pmatrix}.$$

Here Ax stands for $T(x)$, the output of the map that A records. We sometimes write a vector horizontally, as (x_1, x_2), and sometimes vertically, as a column, as in this display. Both name the same ordered list. When we apply a matrix to a vector, as in Ax, we write the vector as a column to the right of the matrix, because the row rule below pairs each row of the matrix with that column.

An *m-by-n matrix* has m rows and n columns. We write A_{ij} for the entry in row i and column j; the first subscript always names the row. For the matrix A above, $A_{12} = 1$ and $A_{22} = -1$. In general, an m-by-n matrix A acts on a vector $x \in \mathbb{R}^n$ and produces the vector $Ax \in \mathbb{R}^m$ whose ith coordinate is

$$(Ax)_i = \sum_{j=1}^{n} A_{ij} x_j = A_{i1}x_1 + A_{i2}x_2 + \cdots + A_{in}x_n \qquad (1 \le i \le m).$$

In words: to find the ith coordinate of Ax, multiply the entries of row i by the matching coordinates of x, and add the products. We call this the *row rule*. Section 10.4 gives this kind of sum of products its usual name, the dot product.

For example, take

$$M = \begin{pmatrix} 2 & -1 & 3 \\ 1 & 0 & 1 \end{pmatrix} \qquad \text{and} \qquad x = (4, 5, 0).$$

The first row gives $2 \cdot 4 + (-1) \cdot 5 + 3 \cdot 0 = 3$, and the second row gives $1 \cdot 4 + 0 \cdot 5 + 1 \cdot 0 = 4$, so $Mx = (3, 4)$. The input must have three coordinates, because each row has three entries; the output has two coordinates, one for each row. Here $m = 2$ and $n = 3$. In general the numbers m and n may differ, and a matrix with $m = n$ is called *square*.

The row rule agrees with the way A was built from its columns. For any m-by-n matrix A, the ith coordinate of the combination

$$x_1(\text{column 1 of } A) + x_2(\text{column 2 of } A) + \cdots + x_n(\text{column } n \text{ of } A)$$

is $x_1A_{i1} + x_2A_{i2} + \cdots + x_nA_{in}$, which is exactly $(Ax)_i$. So Ax is this combination of the columns, with the coordinates of x as coefficients. In the example,

$$Mx = 4(2, 1) + 5(-1, 0) + 0(3, 1) = (8, 4) + (-5, 0) + (0, 0) = (3, 4),$$

as before. Taking $x = e_j$ shows that Ae_j is column j of A. This is why we put $T(e_j)$ in column j. For any linear map $T : \mathbb{R}^n \to \mathbb{R}^m$, build the m-by-n matrix A whose column j is $T(e_j)$, for $j = 1, \ldots, n$. Since Ax is the combination of the columns with coefficients $x_1, \ldots, x_n$, we get $Ax = x_1T(e_1) + \cdots + x_nT(e_n)$, and this equals $T(x)$. So the matrix A records T exactly; it is called the *matrix of* T.

Conversely, every m-by-n matrix A gives a linear map $x \mapsto Ax$ from $\mathbb{R}^n$ to $\mathbb{R}^m$. Take vectors $x, y \in \mathbb{R}^n$ and scalars a, b. The jth coordinate of $ax + by$ is $ax_j + by_j$, so by the row rule the ith coordinate of $A(ax + by)$ is

$$\sum_{j=1}^{n} A_{ij}(ax_j + by_j) = a\sum_{j=1}^{n} A_{ij}x_j + b\sum_{j=1}^{n} A_{ij}y_j,$$

where we multiplied out each bracket and collected the terms containing a and those containing b. The right side is the ith coordinate of $aAx+bAy$. This holds for every i, so $A(ax+by) = aAx+bAy$.

The two constructions undo each other. Starting from a matrix A, the matrix of the map $x \mapsto Ax$ has column j equal to Ae_j, which is column j of A; so we get A back. Starting from a linear map T, its matrix A satisfies $Ax = T(x)$ for every x, so we get T back, and no other matrix does this: if $Bx = T(x)$ for every x, then column j of B is $Be_j = T(e_j)$, so $B = A$. Thus each linear map from $\mathbb{R}^n$ to $\mathbb{R}^m$ has exactly one matrix, each m-by-n matrix comes from exactly one linear map, and the two are just different descriptions of the same object. For this reason we often say that a matrix maps $\mathbb{R}^n$ to $\mathbb{R}^m$.

Pause and decide. Is $F(x_1, x_2) = (x_1 + 1, x_2)$ linear?

Check. No. A linear map sends the zero vector to the zero vector, but $F(0, 0) = (1, 0)$. The map F is a *translation*: it adds the fixed vector $(1, 0)$ to every input. It does preserve differences, since $F(x) - F(y) = x - y$, but it moves the zero vector, and that alone rules out linearity.

10.3 Kernel: the information lost at zero

The *kernel* of a linear map $T : \mathbb{R}^n \to \mathbb{R}^m$ is the set of inputs that T sends to the zero vector:

$$\ker T = \{x \in \mathbb{R}^n : T(x) = 0\}.$$

The kernel always contains 0, because a linear map sends 0 to 0 (Section 10.2). For a matrix A, the kernel $\ker A$ means the kernel of the map $x \mapsto Ax$. As an example, let $P : \mathbb{R}^2 \to \mathbb{R}^2$ be the map $P(x_1, x_2) = (x_1, 0)$, which keeps the first coordinate and replaces the second by zero. It is linear, because

$$P(ax + by) = (ax_1 + by_1,\ 0) = a(x_1, 0) + b(y_1, 0) = aP(x) + bP(y).$$

Its output $(x_1, 0)$ is the zero vector exactly when $x_1 = 0$, so $\ker P = \{(0, t) : t \in \mathbb{R}\}$, the vertical axis.

Why look at the kernel? If two inputs x and y have the same output, then their difference lies in the kernel, because linearity (with $a = 1$ and $b = -1$) gives

$$T(x - y) = T(x) - T(y) = 0.$$

Call a pair of different inputs with the same output a *collision*. Every collision $x \neq y$ therefore produces a nonzero vector $x - y$ in the kernel. For the map P, the inputs $(2, 5)$ and $(2, 7)$ collide, and their difference $(0, -2)$ lies in $\ker P$. The kernel collects in one place all the information that T cannot see.

Theorem 10.1. *A linear map $T : \mathbb{R}^n \to \mathbb{R}^m$ is injective if and only if its kernel consists only of the zero vector, that is,* $\ker T = \{0\}$.

Proof. Suppose first that T is injective, and let $x \in \ker T$. Then $T(x) = 0 = T(0)$, and injectivity gives $x = 0$. So the kernel contains no vector other than 0.

Conversely, suppose that $\ker T = \{0\}$, and let $x, y \in \mathbb{R}^n$ satisfy $T(x) = T(y)$. Linearity gives $T(x - y) = T(x) - T(y) = 0$, so $x - y$ lies in the kernel. Hence $x - y = 0$, that is, $x = y$. Therefore T is injective. □

Both directions of the proof used linearity. The first direction needed $T(0) = 0$, so that an input with output zero could be compared with the input 0. The second needed linearity to turn the equation $T(x) = T(y)$ into $T(x - y) = 0$. Without linearity the theorem fails. The map $F : \mathbb{R} \to \mathbb{R}$ with $F(x) = x^2$ sends only 0 to 0, yet it is not injective, because $F(1) = F(-1)$.

Uniqueness is not existence

The kernel answers a question about uniqueness. Fix a target $b \in \mathbb{R}^m$ and consider the equation $T(x) = b$. If x_0 is one solution, then the solutions are exactly the vectors $x_0 + k$ with $k \in \ker T$. Indeed, each such vector is a solution, because $T(x_0 + k) = T(x_0) + T(k) = b + 0 = b$. And every solution x has this form: the vector $k = x - x_0$ satisfies $T(k) = T(x) - T(x_0) = b - b = 0$, so k lies in the kernel, and $x = x_0 + k$. In particular, when $\ker T = \{0\}$, each equation $T(x) = b$ has at most one solution.

At most one is not the same as exactly one: a kernel equal to $\{0\}$ does not promise that every target is reached. Here we regard $\mathbb{R}$ as $\mathbb{R}^1$, the lists with one coordinate. The map $t \mapsto (t, 0)$ from $\mathbb{R}$ to $\mathbb{R}^2$ is linear, by the same check as for P, and its kernel is $\{0\}$, because $(t, 0) = (0, 0)$ only when $t = 0$. So this map is injective. But no input reaches $(0, 1)$, so it is not surjective.

It is tempting to think that two equations in two unknowns always have exactly one solution. Consider the map $T : \mathbb{R}^2 \to \mathbb{R}^2$ with $T(x_1, x_2) = (x_1 + x_2,\ 2x_1 + 2x_2)$. It is linear, because by the row rule it is the map recorded by the matrix whose rows are $(1, 1)$ and $(2, 2)$. Solving $T(x_1, x_2) = (b_1, b_2)$ means solving the two equations $x_1 + x_2 = b_1$ and $2x_1 + 2x_2 = b_2$. The left side of the second equation is always twice the left side of the first. So when $b_2 = 2b_1$ the second equation only repeats the first, and when $b_2 \neq 2b_1$ the two equations contradict each other.

Both coordinates of $T(x_1, x_2)$ are zero exactly when $x_1 + x_2 = 0$, that is, when $x_2 = -x_1$. So the kernel of T consists of the vectors $(t, -t)$ with $t \in \mathbb{R}$; for instance, it contains the nonzero vector $(1, -1)$. For the target $(2, 4)$ there are infinitely many solutions. One of them is $(2, 0)$, so by the description above the solutions are exactly the vectors $(2, 0) + (t, -t) = (2 + t,\ -t)$ with $t \in \mathbb{R}$. Taking $t = -1$, for instance, gives the solution $(1, 1)$. For the target $(1, 0)$ there is no solution at all, because 0 is not twice 1. Counting equations is not enough. What matters here is that the map loses information: it sends a whole line of vectors $(t, -t)$ to zero.

Inverting the matrix of sums and differences

For the matrix A of Section 10.2, apply its sum-and-difference rule twice:

$$\begin{aligned} A(Ax) &= A(x_1 + x_2,\ x_1 - x_2) \\ &= \big((x_1 + x_2) + (x_1 - x_2),\ (x_1 + x_2) - (x_1 - x_2)\big) \\ &= (2x_1,\ 2x_2) = 2x. \end{aligned}$$

So applying A twice simply doubles every vector. This single identity is enough to show that A is a bijection and to find its inverse. If $Ax = 0$, then $2x = A(Ax) = A0 = 0$, so $x = 0$. Thus $\ker A = \{0\}$, and A is injective by Theorem 10.1. For surjectivity, take any target $b \in \mathbb{R}^2$ and try $x = \frac{1}{2}Ab$. By linearity,

$$Ax = A\big(\tfrac{1}{2}Ab\big) = \tfrac{1}{2}A(Ab) = \tfrac{1}{2} \cdot 2b = b.$$

So every target is reached, and A is a bijection. The map $b \mapsto \frac{1}{2}Ab$ is its inverse: we have just checked that $A\big(\frac{1}{2}Ab\big) = b$, and in the other order $\frac{1}{2}A(Ax) = \frac{1}{2} \cdot 2x = x$.

Written out in coordinates, the inverse sends $b = (b_1, b_2)$ to

$$\tfrac{1}{2}Ab = \left(\frac{b_1 + b_2}{2},\ \frac{b_1 - b_2}{2}\right).$$

These are exactly the coefficients of b in the basis $w_1 = (1, 1)$, $w_2 = (1, -1)$ found in Section 10.1. That is no coincidence. The columns of A are w_1 and w_2, so $Ax = x_1w_1 + x_2w_2$, and solving $Ax = b$ means finding the coefficients of b in this basis. Inverting A and changing to the basis of its columns are the same problem, stated in two languages.

10.4 The geometry of an inner product

Adding and scaling vectors tells us nothing about lengths or right angles. For those we need a way to multiply two vectors. For vectors $u, v \in \mathbb{R}^n$, define their *dot product* by

$$u \cdot v = \sum_{j=1}^{n} u_j v_j = u_1v_1 + u_2v_2 + \cdots + u_nv_n.$$

The result is a number, not a vector. For example, $(1, 2, 3) \cdot (4, -5, 6) = 4 - 10 + 18 = 12$. The dot product is also called the standard *inner product* on $\mathbb{R}^n$. In this language, the row rule of Section 10.2 says that $(Ax)_i$ is the dot product of row i of A with x.

The dot product has three basic properties, and these are the only facts about it that the proofs in the next section use.

- *Symmetry:* $u \cdot v = v \cdot u$, because each product $u_j v_j$ equals $v_j u_j$.
- *Linearity in each argument:* we call u the *first argument* of $u \cdot v$ and v the *second argument*. Here an argument means an input, not a piece of reasoning. If one argument is held fixed, the dot product is linear in the other. For instance, for a fixed v the map $u \mapsto u \cdot v$ from $\mathbb{R}^n$ to $\mathbb{R}$ is linear. For the first argument, the definition of the dot product gives

$$\begin{aligned}(au + bw) \cdot v &= \sum_{j=1}^{n} (au_j + bw_j)v_j \\ &= a \sum_{j=1}^{n} u_j v_j + b \sum_{j=1}^{n} w_j v_j = a(u \cdot v) + b(w \cdot v).\end{aligned}$$

  The same expansion gives $u \cdot (av + bw) = a(u \cdot v) + b(u \cdot w)$ for the second argument. Also $0 \cdot v = 0$ for every v, where the 0 on the left is the zero vector, because each product $0\, v_j$ is zero.
- *Positivity:* $u \cdot u = u_1^2 + \cdots + u_n^2 \geq 0$, and $u \cdot u = 0$ only when $u = 0$. Each square is nonnegative, so the sum is nonnegative. A sum of nonnegative numbers is zero only if every term is zero, and $u_j^2 = 0$ only when $u_j = 0$.

The *Euclidean length*, or *norm*, of a vector u is

$$\|u\|_2 = \sqrt{u \cdot u} = \sqrt{u_1^2 + u_2^2 + \cdots + u_n^2}.$$

The subscript 2 in $\|u\|_2$ is a label for this way of measuring size, which is built from squares. It is not the second coordinate of u, and it does not mean that u has two coordinates. The label is useful because Chapter 11 meets other ways of measuring the size of a vector.

By positivity, the number under the root is nonnegative, so the square root exists. Since $\sqrt{t} = 0$ only when $t = 0$, positivity also shows that $\|u\|_2 = 0$ exactly when $u = 0$. In the plane, $\|(3,4)\|_2 = \sqrt{9+16} = 5$. By Pythagoras' theorem this is the distance from the origin to the point $(3,4)$, because that distance is the longest side of a right triangle whose two shorter sides have lengths 3 and 4.

Multiplying a vector by a scalar c multiplies its length by $|c|$: $\|cu\|_2 = |c|\ \|u\|_2$. To see this, use linearity in each argument to get $\|cu\|_2^2 = (cu) \cdot (cu) = c^2(u \cdot u)$. Since $|c|^2 = c^2$, the two numbers $\|cu\|_2$ and $|c|\ \|u\|_2$ are nonnegative and have the same square, namely $c^2\, \|u\|_2^2$. A nonnegative number is determined by its square (Section 1.8), so they are equal.

Two vectors u and v are *orthogonal* when $u \cdot v = 0$. In the plane, two nonzero vectors are orthogonal exactly when they are perpendicular in the usual sense. To see why, look at the triangle with corners at the origin, at the point u, and at the point v. By Pythagoras' theorem, as in the example of $(3,4)$, the sides from the origin have squared lengths $u_1^2 + u_2^2$ and $v_1^2 + v_2^2$, and the side from u to v has squared length $(u_1 - v_1)^2 + (u_2 - v_2)^2$. Expanding each square by $(p-q)^2 = p^2 - 2pq + q^2$ gives

$$(u_1 - v_1)^2 + (u_2 - v_2)^2 = (u_1^2 + u_2^2) + (v_1^2 + v_2^2) - 2(u_1 v_1 + u_2 v_2).$$

Pythagoras' theorem and its converse say that the angle at the origin is a right angle exactly when the squared length of the opposite side equals the sum of the squared lengths of the other two sides. By the display, this happens exactly when $2(u_1v_1 + u_2v_2) = 0$, that is, when $u \cdot v = 0$.

This reasoning needs a genuine triangle, so one case remains: u and v might lie on one line through the origin. Then $v = cu$ for some number $c \neq 0$, and $u \cdot v = c(u \cdot u)$ is not zero, because $u \cdot u > 0$ by positivity. Two arrows along the same line are not perpendicular either, so the claim holds in this case too.

A quarter turn counterclockwise takes a vector (a, b) to $(-b, a)$; for instance, it takes $(1, 0)$ to $(0, 1)$ and $(0, 1)$ to $(-1, 0)$. So a nonzero vector (a, b) is perpendicular to $(-b, a)$, and indeed

$$(a, b) \cdot (-b, a) = -ab + ba = 0.$$

The algebraic definition makes sense in $\mathbb{R}^n$ for every n, even when we can no longer draw a picture. The zero vector is orthogonal to every vector, since $0 \cdot v = 0$.

For $u = (1, 1)$ and $v = (1, -1)$, the two columns of the matrix A of Section 10.2, we get $u \cdot v = 1 \cdot 1 + 1 \cdot (-1) = 0$. The positive product from the first coordinate cancels the negative product from the second. Chapter 14 studies sums of products of entries 1 and -1, called correlations, and asks when they are zero. Each such product is 1 or -1, so a correlation is zero exactly when it contains as many products equal to 1 as products equal to -1, as in this example.

10.5 Deriving a bound by removing a component

How large can a dot product be, compared with the lengths of the two vectors? When $v = u$, the dot product is $u \cdot u = \|u\|_2^2 = \|u\|_2 \, \|u\|_2$, exactly the product of the two lengths. A natural guess is that the dot product can never be larger than that in size:

$$|u \cdot v| \leq \|u\|_2 \, \|v\|_2 \, .$$

Let us test it on $u = (3, 1)$ and $v = (1, 1)$. Here $u \cdot v = 4$, $\|u\|_2 = \sqrt{10}$ and $\|v\|_2 = \sqrt{2}$, so the guess claims that $4 \leq \sqrt{10}\sqrt{2}$. To check this without decimals, compare squares: $4^2 = 16$ and $(\sqrt{10}\sqrt{2})^2 = 10 \cdot 2 = 20$, and $16 \leq 20$, so the guess holds in this example. Comparing squares is legitimate because, for nonnegative numbers p and q, we have $p \leq q$ exactly when $p^2 \leq q^2$. Section 2.6 showed that $0 \leq p \leq q$ implies $p^2 \leq q^2$. Conversely, if $p > q \geq 0$, the strict version of the same rule gives $p^2 > q^2$, so $p^2 \leq q^2$ rules out $p > q$.

How could we find a reason that works for all vectors? Start from a fact that holds for every real number t: the vector $u - tv$ has a nonnegative squared length,

$$(u - tv) \cdot (u - tv) \geq 0.$$

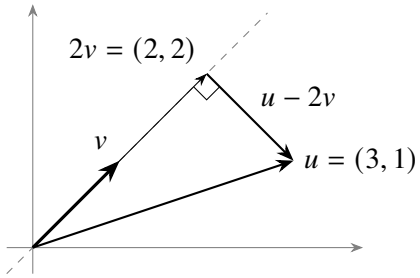
Each choice of t gives an inequality of this kind. Saying that a squared length is at least 0 tells us the most when that squared length is as small as possible, so the most informative choice is the t that makes $u - tv$ as short as possible, that is, the one that brings the multiple tv as close to u as possible. Geometrically, the closest multiple is the one for which the leftover piece $u - tv$ meets the line through v at a right angle. So we look for the t that makes $u - tv$ orthogonal to v, by solving

$$(u - tv) \cdot v = 0.$$

By linearity in the first argument, this equation says $u \cdot v - t(v \cdot v) = 0$. When $v \neq 0$, positivity gives $v \cdot v > 0$, so we may divide by $v \cdot v$ and obtain

$$t = \frac{u \cdot v}{v \cdot v}.$$

This is the value of t used in the proof below. With this t, the vector tv is called the *component* of u along v, and $u - tv$ is the *residual*: what remains of u after its component along v is removed. In our example $t = 4/2 = 2$, so the component is $2v = (2, 2)$ and the residual is $u - 2v = (3, 1) - (2, 2) = (1, -1)$. The residual has dot product $1 - 1 = 0$ with $v = (1, 1)$, as planned, and its squared length is 2. The picture below shows this example: the residual meets the dashed line through v at a right angle.



The theorem below bounds the absolute value $|u \cdot v|$. That is more informative than a bound on $u \cdot v$ alone, because a dot product can be negative. By Section 2.6, the inequality $|u \cdot v| \leq \|u\|_2 \, \|v\|_2$ says that $u \cdot v$ lies between $-\|u\|_2 \, \|v\|_2$ and $\|u\|_2 \, \|v\|_2$.

Theorem 10.2 (Cauchy–Schwarz inequality)**.** *For all $u, v \in \mathbb{R}^n$, $|u \cdot v| \leq \|u\|_2 \, \|v\|_2$.*

Plan. The squared length of $u - tv$ is nonnegative for every real t. Expand it, and then substitute the value of t found above.

Proof. If $v = 0$, then $u \cdot v = 0$ and $\|v\|_2 = 0$, so both sides of the inequality are zero and it holds. Assume from now on that $v \neq 0$, so that $v \cdot v > 0$. For any real t, expand the squared length of $u - tv$, using linearity first in the first argument and then in the second:

$$\begin{aligned}(u - tv) \cdot (u - tv) &= u \cdot (u - tv) - t\big(v \cdot (u - tv)\big) \\ &= u \cdot u - t(u \cdot v) - t(v \cdot u) + t^2(v \cdot v) \\ &= u \cdot u - 2t(u \cdot v) + t^2(v \cdot v).\end{aligned}$$

The last step uses symmetry, $v \cdot u = u \cdot v$. The left side is a squared length, so by positivity the right side is nonnegative for every real t.

Now choose $t = (u \cdot v)/(v \cdot v)$, which is defined because $v \cdot v > 0$. Then $t(u \cdot v) = (u \cdot v)^2/(v \cdot v)$, and

$$t^2(v \cdot v) = \frac{(u \cdot v)^2}{(v \cdot v)^2}\,(v \cdot v) = \frac{(u \cdot v)^2}{v \cdot v}.$$

Substituting these two expressions gives

$$\begin{aligned} 0 &\le u \cdot u - 2\,\frac{(u \cdot v)^2}{v \cdot v} + \frac{(u \cdot v)^2}{v \cdot v} \\ &= u \cdot u - \frac{(u \cdot v)^2}{v \cdot v}. \end{aligned}$$

Add the fraction to both sides and multiply by the positive number $v \cdot v$:

$$(u \cdot v)^2 \le (u \cdot u)(v \cdot v) = \|u\|_2^2 \, \|v\|_2^2 .$$

Finally, $|u \cdot v|$ and $\|u\|_2 \, \|v\|_2$ are nonnegative numbers. Their squares are $(u \cdot v)^2$, because $|a|^2 = a^2$ for every real number a, and $\|u\|_2^2 \, \|v\|_2^2$. Nonnegative numbers compare in the same way as their squares, so $|u \cdot v| \le \|u\|_2 \, \|v\|_2$. □

When does equality hold?

When is $|u \cdot v|$ exactly equal to $\|u\|_2 \, \|v\|_2$? For $v \neq 0$, the answer is: exactly when u is a scalar multiple of v.

The proof already contains the reason. With the chosen t, it showed that the squared length of the residual is

$$\|u - tv\|_2^2 = u \cdot u - \frac{(u \cdot v)^2}{v \cdot v}.$$

Multiplying by $v \cdot v$ gives

$$(u \cdot u)(v \cdot v) - (u \cdot v)^2 = (v \cdot v) \, \|u - tv\|_2^2 .$$

The left side is the gap between the two sides of the squared inequality. Two nonnegative numbers are equal exactly when their squares are equal, so equality holds in the theorem exactly when this gap is zero. Since $v \cdot v > 0$, the gap is zero exactly when $\|u - tv\|_2 = 0$, that is, when the residual $u - tv$ is the zero vector, which means $u = tv$. So if equality holds, then u is a multiple of v. Conversely, suppose that $u = sv$ for some number s. Then

$$t = \frac{(sv) \cdot v}{v \cdot v} = \frac{s(v \cdot v)}{v \cdot v} = s,$$

so the residual is $u - sv = 0$, the gap is zero, and equality holds.

For example, $u = (2, 2)$ and $v = (1, 1)$ give $u \cdot v = 4$ and $\|u\|_2 \, \|v\|_2 = \sqrt{8}\sqrt{2} = 2\sqrt{2} \cdot \sqrt{2} = 4$. This is equality, as expected, since $u = 2v$. For $u = (3, 1)$ and $v = (1, 1)$ the gap is $20 - 16 = 4$, and indeed $(v \cdot v) \, \|u - 2v\|_2^2 = 2 \cdot 2 = 4$. When $v = 0$, both sides of the inequality are zero, so equality holds for every u. Altogether, equality holds exactly when $v = 0$ or u is a scalar multiple of v.

Section 9.6 recommended asking, of every bound, what happens when it is attained. Here the answer shows that the bound is as good as possible: no constant factor smaller than 1 can be placed in front of its right-hand side. When $u = v \neq 0$, both sides equal $\|v\|_2^2$, so a smaller right-hand side, such as $0.9 \, \|u\|_2 \, \|v\|_2$, would fail for these vectors.

From the dot-product bound to a distance bound

In the plane, walking from one point to another by way of a third point is never shorter than walking there directly. Chapter 11 needs this fact in $\mathbb{R}^n$, and Cauchy–Schwarz gives it quickly. For any $u, v \in \mathbb{R}^n$, expand using linearity in each argument and symmetry:

$$\begin{aligned}\|u+v\|_2^2 &= (u+v)\cdot(u+v)\\ &= u\cdot u + u\cdot v + v\cdot u + v\cdot v\\ &= \|u\|_2^2 + 2(u\cdot v) + \|v\|_2^2\\ &\le \|u\|_2^2 + 2|u\cdot v| + \|v\|_2^2\\ &\le \|u\|_2^2 + 2\,\|u\|_2\,\|v\|_2 + \|v\|_2^2\\ &= (\|u\|_2 + \|v\|_2)^2.\end{aligned}$$

The first inequality holds because $2(u\cdot v) \le 2|u\cdot v|$. Indeed, every real number a satisfies $a \le |a|$: if $a \ge 0$ the two sides are equal, and if $a < 0$ then $a < 0 \le |a|$. The second inequality uses Cauchy–Schwarz. The last line is the expansion $(p+q)^2 = p^2 + 2pq + q^2$. Both $\|u+v\|_2$ and $\|u\|_2 + \|v\|_2$ are nonnegative, and nonnegative numbers compare in the same way as their squares. We obtain the *triangle inequality*

$$\|u+v\|_2 \le \|u\|_2 + \|v\|_2 .$$

To see where the name comes from, take three points $x, y, z \in \mathbb{R}^n$ and put $u = x - y$ and $v = y - z$. Then $u + v = x - z$, and the inequality becomes

$$\|x-z\|_2 \le \|x-y\|_2 + \|y-z\|_2 .$$

In the plane, $\|x-z\|_2$ is the ordinary distance between the points x and z, by Pythagoras' theorem. So the inequality says that in the triangle with corners x, y, z, the side from x to z is at most as long as the other two sides together. Exercise 10.5 asks when the two sides of the triangle inequality are equal.

10.6 Matrix multiplication records composition

Section 10.3 applied the matrix A twice and found $A(Ax) = 2x$. We often apply one linear map after another, and then we want a single matrix that records the combined map. So let A now be any m-by-n matrix and B any k-by-m matrix. Then A maps $\mathbb{R}^n$ to $\mathbb{R}^m$ and B maps $\mathbb{R}^m$ to $\mathbb{R}^k$, so the composition "B after A," which sends x to $B(Ax)$, maps $\mathbb{R}^n$ to $\mathbb{R}^k$. This composition is linear, because linearity of A and then of B gives

$$B\big(A(ax+by)\big) = B\big(aAx + bAy\big) = a\,B(Ax) + b\,B(Ay).$$

By Section 10.2, the composition is therefore recorded by a k-by-n matrix. This matrix is called the *product* BA. By definition, $(BA)x = B(Ax)$ for every $x \in \mathbb{R}^n$. As with $g \circ f$ in Section 3.2, the factor written on the right acts first.

To find the entries of BA, recall from Section 10.2 that column j of any matrix M is Me_j. So column j of BA is

$$(BA)e_j = B(Ae_j) = B\,(\text{column } j \text{ of } A).$$

Column j of A is the vector $(A_{1j}, A_{2j}, \ldots, A_{mj})$. The entry $(BA)_{ij}$ is the ith coordinate of column j of BA, so the row rule for B, applied to this vector, gives

$$(BA)_{ij} = \sum_{r=1}^{m} B_{ir}A_{rj} = B_{i1}A_{1j} + B_{i2}A_{2j} + \cdots + B_{im}A_{mj}$$

for $1 \le i \le k$ and $1 \le j \le n$. In words, the entry of BA in row i and column j is the dot product of row i of B with column j of A. This makes sense only when each row of B has as many entries as each column of A, that is, when B has as many columns as A has rows. The row-by-column rule may look strange at first sight. The calculation shows where it comes from: it is exactly what composing the two maps requires.

For a numerical example, multiply the matrix A of Section 10.2 by itself:

$$\begin{aligned} AA &= \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \\ &= \begin{pmatrix} 1\cdot 1 + 1\cdot 1 & 1\cdot 1 + 1\cdot(-1) \\ 1\cdot 1 + (-1)\cdot 1 & 1\cdot 1 + (-1)\cdot(-1) \end{pmatrix} = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}. \end{aligned}$$

This matrix sends (x_1, x_2) to $(2x_1, 2x_2)$, so it doubles every vector, in agreement with the calculation $A(Ax) = 2x$ in Section 10.3.

The order of the factors matters, just as $g \circ f$ and $f \circ g$ were different functions in Section 3.2. For

$$C = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \qquad \text{and} \qquad D = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix},$$

the row-by-column rule gives

$$CD = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix} \qquad \text{but} \qquad DC = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}.$$

The maps explain the difference. The matrix C sends (x_1, x_2) to $(x_1 + x_2,\ x_2)$, adding the second coordinate to the first, while D sends (x_1, x_2) to $(x_1,\ x_1 + x_2)$, adding the first coordinate to the second. Starting from $(1, 0)$, the product CD applies D first and reaches $(1, 1)$, and then C gives $(2, 1)$. The product DC applies C first, which leaves $(1, 0)$ unchanged, and then D gives $(1, 1)$. These two results are the first columns of CD and DC.

The transpose and orthogonal rows

The *transpose* A^{T} of an m-by-n matrix A is the n-by-m matrix obtained by exchanging rows and columns: $(A^{\mathsf{T}})_{ij} = A_{ji}$. So column j of A^{T} is row j of A. For example,

$$\begin{pmatrix} 2 & -1 & 3 \\ 1 & 0 & 1 \end{pmatrix}^{\mathsf{T}} = \begin{pmatrix} 2 & 1 \\ -1 & 0 \\ 3 & 1 \end{pmatrix}.$$

By the row-by-column rule, the entry $(AA^{\mathsf{T}})_{ij}$ is the dot product of row i of A with column j of A^{T}, which is row j of A. So the entries of AA^{T} are the dot products of the rows of A with one another:

$$(AA^{\mathsf{T}})_{ij} = \sum_{r=1}^{n} A_{ir}A_{jr} = (\text{row } i \text{ of } A) \cdot (\text{row } j \text{ of } A).$$

The n-by-n *identity matrix* I has ones on its *diagonal*, that is, in the positions (i, i), and zeros everywhere else. Its columns are the standard basis vectors $e_1, \dots, e_n$. By the row rule, $(Ix)_i = x_i$, because the only nonzero entry in row i is the 1 in column i. So $Ix = x$ for every $x \in \mathbb{R}^n$, and I is the matrix of the identity map. For $n = 2$,

$$I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Multiplying a matrix by a number means multiplying every entry by that number, so nI has n in each diagonal position and zeros elsewhere. For an n-by-n matrix A, the equation $AA^{\mathsf{T}} = nI$ therefore says two things at once. The diagonal entries say that every row of A has squared length n. The entries off the diagonal say that any two different rows of A are orthogonal.

The matrix A of Section 10.2 is an example with $n = 2$. Exchanging its rows and columns does not change it, so $A^{\mathsf{T}} = A$ and $AA^{\mathsf{T}} = AA = 2I$. Its rows $(1, 1)$ and $(1, -1)$ indeed have squared length 2 and are orthogonal. Square matrices with entries ± 1 that satisfy $AA^{\mathsf{T}} = nI$, where n is the number of rows, are called *Hadamard matrices*. They are central to Chapter 14, which follows a question about them all the way to current research.

Exercises

Exercises 10.1–10.4 are core exercises, and Exercises 10.5 and 10.6 are transfer exercises. Complete solutions are in Appendix B, under "Week 10."

Exercise 10.1. For $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $x = (1, -2)$, compute Ax. Then explain the answer using the columns of A.

Exercise 10.2. Find the kernel of the linear map $T : \mathbb{R}^2 \to \mathbb{R}$ given by $T(x, y) = x + y$. Then describe all solutions of $T(x, y) = 5$.

Exercise 10.3. The map $T : \mathbb{R}^2 \to \mathbb{R}^2$ given by $T(x, y) = (x + y, x - y)$ is the map recorded by the matrix A of Section 10.2, and Section 10.3 found its inverse from the equation $A(Ax) = 2x$. Check this answer directly: show that $(s, t) \mapsto \big((s+t)/2, (s-t)/2\big)$ is the inverse of T by computing both compositions.

Exercise 10.4. Compute the dot product and the lengths of $u = (1, 2)$ and $v = (2, -1)$. Check the Cauchy–Schwarz inequality for this pair.

Exercise 10.5. The proof of the triangle inequality $\|u + v\|_2 \le \|u\|_2 + \|v\|_2$ in Section 10.5 passes through two inequalities. By asking when each of them is an equality, prove that for vectors

$u, v \in \mathbb{R}^n$ the equation $\|u + v\|_2 = \|u\|_2 + \|v\|_2$ holds exactly when $v = 0$ or $u = sv$ for some number $s \geq 0$. Then give nonzero vectors u and v, with u a multiple of v, for which the inequality is strict.

Exercise 10.6. Let $u_1, \ldots, u_k \in \mathbb{R}^n$ be nonzero vectors that are pairwise orthogonal, which means that $u_i \cdot u_j = 0$ whenever $i \neq j$. Show that $a_1 u_1 + \cdots + a_k u_k = 0$ forces every coefficient a_j to be zero; in other words, the list $u_1, \ldots, u_k$ is linearly independent. Then give an example showing that the word "nonzero" cannot be left out.

CHAPTER 11

Distance, error, and the meaning of a limit

This week

Describe distance by three simple rules, so that one argument can work for numbers, vectors, and other objects. Replace the impression that a sequence "gets close" to a point by a precise promise that covers every positive tolerance. See why steps that keep getting smaller need not make a sequence settle down, while steps that shrink by a fixed factor do. Recognize completeness as the property that guarantees a limit inside the space for every sequence whose terms eventually all lie as close together as we like.

11.1 Distance begins with absolute value

Recall from Section 2.6 that the absolute value $|x|$ of a real number x is x when $x \geq 0$ and $-x$ when $x < 0$. For instance, $|4| = 4$ and $|-3| = -(-3) = 3$. The *distance* between two real numbers x and y is $|x - y|$, the length of the stretch of the number line between them. For $x = -2$ and $y = 3$ it is $|-2-3| = |-5| = 5$: walking from -2 to 3 takes two units to reach 0 and three more to reach 3. The order of the two numbers does not matter, since $|y - x| = |3 - (-2)| = |5| = 5$ as well. In general $y - x = -(x - y)$, and changing the sign of a number does not change its absolute value, so $|y - x| = |x - y|$.

This chapter starts from the familiar distance $|x - y|$ and asks which of its properties our arguments actually use. Once those few properties are written down, any argument that relies only on them works just as well for other objects, such as the vectors of Chapter 10, provided they come with a distance that has the same properties.

The most important of these properties is the *triangle inequality*: for all real numbers x and y,

$$|x + y| \leq |x| + |y|.$$

For example, $x = 5$ and $y = -2$ give $|x + y| = 3$, which is less than $|x| + |y| = 7$. When x and y are both nonnegative, or both nonpositive, the two sides are equal; for instance, $x = -5$ and $y = -2$ give $|x + y| = |-7| = 7$ and $|x| + |y| = 5 + 2 = 7$.

To prove the inequality, recall another rule from Section 2.6: for a number $b \geq 0$, the statement $|x| \leq b$ holds exactly when $-b \leq x \leq b$. Taking $b = |x|$, the true statement $|x| \leq |x|$ becomes $-|x| \leq x \leq |x|$. In the same way, $-|y| \leq y \leq |y|$. Two inequalities in the same direction may be added: if $p \leq r$ and $s \leq t$, then $p + s \leq r + s \leq r + t$. Adding $-|x| \leq x$ and $-|y| \leq y$ gives

$-|x| - |y| \le x + y$, and adding $x \le |x|$ and $y \le |y|$ gives $x + y \le |x| + |y|$. Together,

$$-(|x| + |y|) \le x + y \le |x| + |y|.$$

Now use the same rule in the other direction, with $x + y$ in place of x and the nonnegative number $b = |x| + |y|$. It turns this pair of inequalities into $|x + y| \le |x| + |y|$.

The triangle inequality is most often used in a second form. Given three real numbers a, b and c, take $x = a - b$ and $y = b - c$. Then $x + y = a - c$, because the two copies of b cancel, and the triangle inequality becomes

$$|a - c| \le |a - b| + |b - c|.$$

Read as a statement about distances, it says that the distance from a to c is at most the distance from a to b plus the distance from b to c. Going from a to c by way of a stopping point b is never shorter than going directly. For $a = 0$, $b = 5$ and $c = 2$, the direct distance is 2, while the route through b has length $5 + 3 = 8$. When b lies between a and c, as in $a = 0$, $b = 1$, $c = 2$, the two sides are equal.

This second form explains why the triangle inequality appears in almost every error estimate in this chapter. To show that two quantities a and c are close, it is often easier to find a third quantity b that is close to each of them, and then to add the two small distances.

Here is a first error estimate. Suppose x approximates a number a with error at most ε, and y approximates a number b with error at most δ, where δ (the Greek letter delta) is a second tolerance: $|x - a| \le \varepsilon$ and $|y - b| \le \delta$. How well does $x + y$ approximate $a + b$? Regroup the difference as $(x + y) - (a + b) = (x - a) + (y - b)$ and apply the triangle inequality:

$$|(x + y) - (a + b)| \le |x - a| + |y - b| \le \varepsilon + \delta.$$

So when approximations are added, their error bounds add. For example, take $a = 2$, $b = 5$, $\varepsilon = 0.01$ and $\delta = 0.02$. Then $x + y$ is within 0.03 of 7. This bound is reached when both errors point the same way: $x = 2.01$ and $y = 5.02$ give $x + y = 7.03$. When the errors point in opposite directions, they partly cancel: $x = 2.01$ and $y = 4.98$ give $x + y = 6.99$, only 0.01 away from 7. Since we know only the sizes of the two errors, not their directions, $\varepsilon + \delta$ is the best guarantee available: it promises that the error is never larger, not that it is always this large.

Pause and decide. Suppose $|x-a| \le \varepsilon$ and $|y-b| \le \delta$. Does it follow that $|(x-y)-(a-b)| \le \varepsilon-\delta$? If not, what bound is correct?

Check. No. Take $a = b = 0$, $x = \varepsilon$ and $y = -\delta$. Both hypotheses hold, and $(x-y)-(a-b) = \varepsilon+\delta$, which is larger than $\varepsilon - \delta$ because $\delta > 0$. The correct bound is $\varepsilon + \delta$. To prove it, write $(x-y)-(a-b) = (x-a)+\big(-(y-b)\big)$ and apply the triangle inequality. Since $|-(y-b)| = |y-b|$, the result is $|(x - y) - (a - b)| \le |x - a| + |y - b| \le \varepsilon + \delta$. Subtracting two approximations does not subtract their errors; in the worst case the errors still add.

11.2 A metric keeps only the properties we use

Let X be a set. We call its members *points*, even when they are numbers or vectors. A distance on X should assign a real number $d(x, y)$ to each pair of points $x, y \in X$. In the language of

Sections 3.1 and 3.2, d is a function from the Cartesian product $X \times X$ to $\mathbb{R}$: it takes an ordered pair (x, y) of points as its input and returns a real number. The following definition lists the three properties that such a function must have before we call it a metric.

Definition 11.1 (Metric)**.** A *metric* on a set X assigns a real number $d(x, y)$ to every pair of points $x, y \in X$ and satisfies these requirements for all $x, y, z \in X$:

1. $d(x, y) \geq 0$, and $d(x, y) = 0$ holds exactly when $x = y$.
2. $d(x, y) = d(y, x)$.
3. $d(x, z) \leq d(x, y) + d(y, z)$.

A set together with a chosen metric is called a *metric space* and is written (X, d). The number $d(x, y)$ is called the *distance* between x and y.

The first requirement says that distances are never negative, that every point is at distance zero from itself, and that two different points are always a positive distance apart. The second, called *symmetry*, says that the distance from x to y equals the distance from y to x. The third is the triangle inequality of Section 11.1, now demanded of d: going from x to z by way of a third point y is never shorter than going directly.

Like every definition, this one can be used in the two directions described in Section 1.4. If we are told that d is a metric, we may *unpack* the definition and use any of the three requirements in an argument. If we want to show that a new formula is a metric, we must *recognize* it as one, which means checking all three requirements. Calling a formula a distance does not make it one, as the next question shows.

Pause and decide. On the real line, let $d(x, y) = (x - y)^2$. Is d a metric?

Check. No. The first two requirements hold: a square is never negative, $(x - y)^2 = 0$ exactly when $x = y$, and $(y - x)^2 = (x - y)^2$. The triangle inequality fails. Take $x = 0$, $y = 1$ and $z = 2$. Then $d(x, z) = 4$, while $d(x, y) + d(y, z) = 1 + 1 = 2$, so the route through y comes out *shorter* than the direct route. One counterexample is enough to show that d is not a metric.

Examples

The real line with $d(x, y) = |x - y|$ is a metric space. The absolute value is never negative, and $|x - y| = 0$ exactly when $x - y = 0$, that is, when $x = y$. Symmetry, $|x - y| = |y - x|$, and the triangle inequality $|x - z| \leq |x - y| + |y - z|$ were both proved in Section 11.1. When we treat the real line as a metric space without naming a metric, this is the metric we mean.

The set $\mathbb{R}^n$ of vectors $x = (x_1, \ldots, x_n)$ from Section 10.1 carries several useful metrics. The first is the *Euclidean distance* $d_2(x, y) = \|x - y\|_2$, the length of the difference vector. In the plane, where $n = 2$, it is the ordinary straight-line distance; from $(0, 0)$ to $(3, 4)$ it is $\sqrt{3^2 + 4^2} = 5$.

Let us check the three requirements for d_2. The first holds because a length is never negative and is zero only for the zero vector (Section 10.4), and $x - y$ is the zero vector exactly when $x = y$. Symmetry holds because $y - x$ has the coordinates of $x - y$ with every sign changed, and changing signs does not change the squares under the root. For the triangle inequality, Section 10.5 ends with the inequality $\|x - z\|_2 \leq \|x - y\|_2 + \|y - z\|_2$ for any three vectors x, y and z. In our new

notation it reads $d_2(x, z) \le d_2(x, y) + d_2(y, z)$.

Another useful choice is

$$d_\infty(x, y) = \max_{1 \le j \le n} |x_j - y_j|,$$

the largest of the n numbers $|x_1 - y_1|, \ldots, |x_n - y_n|$. The symbol ∞ in the subscript is just part of the traditional name. This metric records the worst error in any single coordinate. For $x = (1, 5)$ and $y = (4, 3)$, the coordinate differences have sizes 3 and 2, so $d_\infty(x, y) = 3$, while $d_2(x, y) = \sqrt{9 + 4} = \sqrt{13} \approx 3.61$.

Here is the proof of the triangle inequality for d_∞. Let $x, y, z \in \mathbb{R}^n$ and fix a coordinate j. The triangle inequality for real numbers gives the first step below. For the second step, note that $|x_j - y_j|$ is one of the numbers whose largest is $d_\infty(x, y)$, so $|x_j - y_j| \le d_\infty(x, y)$; in the same way, $|y_j - z_j| \le d_\infty(y, z)$. Thus

$$|x_j - z_j| \le |x_j - y_j| + |y_j - z_j| \le d_\infty(x, y) + d_\infty(y, z).$$

The right side is the same number for every j. So it is at least as large as each of the numbers $|x_1 - z_1|, \ldots, |x_n - z_n|$, and in particular at least as large as the largest of them, which is $d_\infty(x, z)$. This proves $d_\infty(x, z) \le d_\infty(x, y) + d_\infty(y, z)$. Exercise 11.3 checks the other two requirements.

Which metric to use depends on the question. If a measuring instrument must get *every* coordinate right to within 0.01, the condition $d_\infty(x, y) \le 0.01$ says exactly that. A bound on $d_2(x, y)$ would instead control all the coordinate errors together, through the sum of their squares. The two conditions are different. In the plane, a reading $x = (0.01, 0.01)$ of the true point $y = (0, 0)$ meets the instrument's requirement, since $d_\infty(x, y) = 0.01$, but $d_2(x, y) = \sqrt{0.0001 + 0.0001} \approx 0.0141$, so it fails the condition $d_2(x, y) \le 0.01$.

Norms, subsets, and spaces without addition

Both metrics on $\mathbb{R}^n$ are built from a way of measuring the size of a single vector. For d_2 the size is the Euclidean length, since $d_2(x, y) = \|x - y\|_2$. For d_∞ it is the largest coordinate size $\|x\|_\infty = \max_j |x_j|$, since $d_\infty(x, y) = \|x - y\|_\infty$. Measures of size like these are called norms. Precisely, a *norm* on $\mathbb{R}^n$ assigns to each vector x a real number $\|x\|$ such that, for all vectors x, y and every real number c,

1. $\|x\| \ge 0$, with equality exactly when x is the zero vector;
2. $\|cx\| = |c| \, \|x\|$;
3. $\|x + y\| \le \|x\| + \|y\|$.

The Euclidean length $\|x\|_2$ is a norm. Its first two properties were shown in Section 10.4, and the third is the triangle inequality of Section 10.5. The largest coordinate size $\|x\|_\infty$ is a norm too. It is never negative, and it is 0 exactly when every coordinate is 0. Multiplying every coordinate by c multiplies every coordinate size, and hence the largest one, by $|c|$. The third property is proved one coordinate at a time, just like the triangle inequality for d_∞ above.

In the other direction, every norm produces a metric by the rule $d(x, y) = \|x - y\|$. The first requirement of a metric comes from the first property of the norm, because $x - y$ is the zero

vector exactly when $x = y$. Symmetry comes from the second property with $c = -1$: since $y - x = (-1)(x - y)$, we get $\|y - x\| = |-1|\,\|x - y\| = \|x - y\|$. The triangle inequality comes from the third property: since $x - z = (x - y) + (y - z)$, we get $\|x - z\| \le \|x - y\| + \|y - z\|$.

A metric, however, does not need points to be added or subtracted at all. It needs only one number $d(x, y)$ for each pair of points. For this reason the rest of the chapter works in a general metric space. The arguments then apply also to sets in which addition makes no sense, or in which a sum can fall outside the set.

The most common way to obtain such a set is to take part of a larger metric space. If (X, d) is a metric space and C is a subset of X, then d, used only for pairs of points of C, is a metric on C. (In the language of Section 3.3, this is the restriction of d to $C \times C$.) Indeed, the three requirements hold for all points of X, so in particular they hold for all points of C. For example, the interval $[0, 1]$ with distance $|x - y|$ is a metric space. Its points can be added as real numbers, but a sum can leave the interval, as $0.7 + 0.6 = 1.3$ does. Arguments that use only distances never meet this difficulty.

11.3 Convergence promises eventual control

A *sequence* in a metric space (X, d) is an endless list $x_0, x_1, x_2, \ldots$ of points of X, one for each natural number. The sequences of numbers in Section 5.3 are the case $X = \mathbb{R}$. In a sequence the subscript counts terms: x_n is the term in position n, a whole point of X, and not a coordinate of a vector as in Section 11.2. The terms from some index N onward, $x_N, x_{N+1}, x_{N+2}, \ldots$, form a *tail* of the sequence. The next definition says precisely what it means for the terms to approach a point.

Definition 11.2 (Convergence)**.** A sequence $x_0, x_1, x_2, \ldots$ in a metric space (X, d) *converges* to a point $x_* \in X$ if, for every tolerance $\varepsilon > 0$, there is a threshold N such that every term from x_N onward lies at distance less than ε from x_*:

$$\forall \varepsilon > 0\ \exists N \in \mathbb{N}\ \forall n \ge N : \quad d(x_n, x_*) < \varepsilon.$$

The point x_* (the star is part of the name) is then called a *limit* of the sequence, and we write $x_n \to x_*$, read "x_n tends to x_*." A sequence that converges to some point of X is called *convergent*.

Read the quantifiers as the game between a skeptic and a prover from Section 2.3. The proposed limit x_* is fixed before the game begins. First the skeptic names a tolerance, say $\varepsilon = 0.01$. Then the prover names a threshold N, and may use the value of ε to choose it. Finally the skeptic picks *any* index $n \ge N$, and the prover wins if $d(x_n, x_*) < 0.01$. The sequence converges to x_* when the prover can win however the skeptic plays. A smaller tolerance usually forces a larger threshold, and that is allowed, because N is chosen after ε. What is not allowed is a threshold that works for only some of the later indices: a single N must handle every $n \ge N$ at once.

The statement $x_n \to x_*$ does not say that the sequence ever reaches its limit. The terms of $x_n = 1/(n + 1)$, for instance, are all positive, yet we now prove that they tend to 0.

Two sequences that converge

Take $x_n = 1/(n+1)$ on the real line, so that the sequence is $1, \frac{1}{2}, \frac{1}{3}, \ldots$. Let $\varepsilon > 0$ be given. By the Archimedean property (Section 2.3) we may choose an integer $N > 1/\varepsilon$; it is positive because $1/\varepsilon > 0$. Now let $n \geq N$. Then $n + 1 > N > 0$, and taking reciprocals reverses the order, so $1/(n+1) < 1/N$. Taking reciprocals in $N > 1/\varepsilon$ gives $1/N < \varepsilon$. Together,

$$d(x_n, 0) = \left| \frac{1}{n+1} - 0 \right| = \frac{1}{n+1} < \frac{1}{N} < \varepsilon.$$

This proves $x_n \to 0$. For the tolerance $\varepsilon = 0.01$, the recipe asks for an integer $N > 100$, so $N = 101$ will do, and indeed every term from $x_{101} = 1/102$ onward is less than 0.01. The threshold $N = 100$ would also work, because $x_{100} = 1/101$ is already less than 0.01 and the later terms are smaller still; $N = 99$ would not, because $x_{99} = 1/100$ is not less than 0.01. The definition asks only for *some* threshold that works, not for the smallest one.

Next return to the sequence $x_n = 1 - 2^{-n}$ of Section 5.3, which we described there as coming as close to 1 as we like. The definition turns that phrase into a statement we can prove. The distance from x_n to 1 is $|x_n - 1| = 2^{-n}$. To compare it with $1/(n+1)$, we first show by induction that $2^n \geq n + 1$ for every $n \in \mathbb{N}$. At $n = 0$ both sides equal 1. If $2^n \geq n + 1$ for some n, then

$$2^{n+1} = 2 \cdot 2^n \geq 2(n+1) = 2n + 2 \geq n + 2,$$

which is the claim for $n + 1$. Taking reciprocals gives $2^{-n} \leq 1/(n+1)$. So if $\varepsilon > 0$ and N is an integer with $N > 1/\varepsilon$, as before, then every $n \geq N$ satisfies $|x_n - 1| \leq 1/(n+1) < \varepsilon$. This proves $x_n \to 1$.

Because the definition speaks only about tails, changing finitely many terms of a sequence does not affect whether it converges, or what its limit is. Suppose, for instance, that the first thousand terms of $1/(n+1)$ were replaced by arbitrary numbers. The new sequence still tends to 0: given $\varepsilon > 0$, take a threshold that is at least 1000 and larger than $1/\varepsilon$, and the argument above applies unchanged to every later term. In the same way, if $x_n \to x_*$, then the shifted sequence $x_1, x_2, x_3, \ldots$, whose terms are x_{n+1}, also tends to x_*. Indeed, if $d(x_n, x_*) < \varepsilon$ for every $n \geq N$, then also $d(x_{n+1}, x_*) < \varepsilon$ for every $n \geq N$, since $n + 1 \geq N$ too.

What failure to converge looks like

Negating the definition with the rules of Section 2.4 shows what it means for a sequence *not* to converge to x_*:

$$\exists \varepsilon > 0 \; \forall N \in \mathbb{N} \; \exists n \geq N : \quad d(x_n, x_*) \geq \varepsilon.$$

In words, there is one fixed tolerance such that, however late we start, some later term is at least that far from x_*. Notice that the strict inequality $< \varepsilon$ has become $\geq \varepsilon$.

Consider the sequence $0, 1, 0, 1, \ldots$, in which $x_n = 0$ for even n and $x_n = 1$ for odd n. Its terms return to 0 again and again, but a single late term near 0 is not enough: the definition needs a whole tail near the proposed limit. Take $\varepsilon = 1/2$. For every threshold N, the odd index $n = 2N + 1$ is at least N, and $d(x_n, 0) = 1 \geq 1/2$. So the sequence does not converge to 0.

Pause and decide. Does the sequence $0, 1, 0, 1, \ldots$ converge to $1/2$? Does it converge to any real number at all?

Check. It does not converge to $1/2$. Every term is at distance exactly $1/2$ from $1/2$, so for the tolerance $\varepsilon = 1/2$ no term at all is close enough. In fact it converges to no real number L. Suppose it did, and take $\varepsilon = 1/2$. Some threshold N would make every term from x_N onward lie less than $1/2$ away from L. Among those terms are $x_{2N} = 0$ and $x_{2N+1} = 1$. The triangle inequality through L would then give $1 = |1 - 0| \le |1 - L| + |L - 0| < \frac{1}{2} + \frac{1}{2} = 1$, which is impossible.

11.4 Limits are unique

Can a sequence converge to two different points at once? On the real line the answer seems obvious, but the proof below uses only the three requirements of a metric, so it settles the question in every metric space.

Proposition 11.3. *A sequence in a metric space has at most one limit: if $x_n \to a$ and $x_n \to b$, then $a = b$.*

Proof. Suppose $x_n \to a$ and $x_n \to b$, and assume, for contradiction, that $a \neq b$. By the first requirement of a metric, the number $D = d(a, b)$ is then positive, and so is the tolerance $D/3$. Convergence to a supplies a threshold N_1 such that $d(x_n, a) < D/3$ for every $n \ge N_1$. Convergence to b supplies a threshold N_2 such that $d(x_n, b) < D/3$ for every $n \ge N_2$. Now take a single index n that is at least as large as both thresholds, for instance the larger of N_1 and N_2. The term x_n is then close to both points. The triangle inequality with x_n as the stopping point, together with the symmetry $d(a, x_n) = d(x_n, a)$, gives

$$D = d(a, b) \le d(a, x_n) + d(x_n, b) < \frac{D}{3} + \frac{D}{3} = \frac{2D}{3}.$$

So $D < 2D/3$; subtracting $2D/3$ from both sides gives $D/3 < 0$, which contradicts $D > 0$. Therefore $a = b$. □

Why the tolerance $D/3$? Any positive tolerance ε with $2\varepsilon \le D$ would do. The triangle inequality adds two distances, each less than ε, so it gives $D < 2\varepsilon \le D$, which is impossible. The tolerance $D/3$ is simply one convenient choice.

The key step is the choice of one index beyond both thresholds. The triangle inequality needs a single point x_n that is close to a and close to b at the same time, because that point is the stop on the route from a to b. A term close to a and a *different* term close to b would give two short pieces that do not join up into a route from a to b. The phrase "for every $n \ge N$" in the definition of convergence is what allows us to choose a common index.

Because of the proposition, we may speak of *the* limit of a convergent sequence. For example, the sequence $1/(n + 1)$ of Section 11.3 converges to 0 and to no other real number.

11.5 Cauchy sequences do not need a known destination

To prove that a sequence converges, the definition asks us to name the limit first. Often we do not know it. In Chapter 12, for example, a sequence is produced by applying the same rule again and again, and the point it approaches is exactly the unknown we are trying to find. The following condition speaks only about the terms themselves. It is named after the French mathematician Augustin-Louis Cauchy (1789–1857).

Definition 11.4 (Cauchy sequence)**.** A sequence $x_0, x_1, x_2, \ldots$ in a metric space (X, d) is a *Cauchy sequence* if, for every tolerance $\varepsilon > 0$, there is a threshold N such that any two terms from x_N onward are less than ε apart:

$$\forall \varepsilon > 0 \; \exists N \in \mathbb{N} \; \forall m, n \geq N : \quad d(x_m, x_n) < \varepsilon.$$

Compare this with the definition of convergence. There, the late terms had to be close to one fixed point x_*. Here, they have to be close to one another, and no limit point appears at all. The game has the same order as before: the skeptic names ε, the prover names N, and then the skeptic may pick *any two* indices m and n from N onward.

Proposition 11.5. *Every convergent sequence in a metric space is a Cauchy sequence.*

Proof. Suppose $x_n \to x_*$, and let $\varepsilon > 0$ be given. Apply the definition of convergence with the smaller tolerance $\varepsilon/2$. It supplies a threshold N such that $d(x_n, x_*) < \varepsilon/2$ for every $n \geq N$. Now let m and n be any two indices with $m, n \geq N$. Both x_m and x_n are less than $\varepsilon/2$ away from x_*. The triangle inequality with x_* as the stopping point, together with symmetry, gives

$$d(x_m, x_n) \leq d(x_m, x_*) + d(x_*, x_n) < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

The same threshold N works for every pair of indices from N on, so the sequence is Cauchy. □

We asked for the tolerance $\varepsilon/2$ rather than ε because the triangle inequality adds two distances, and two errors of size less than $\varepsilon/2$ add up to less than ε. Splitting a tolerance into pieces in this way is a standard move, and it will return in later chapters.

The converse, "every Cauchy sequence converges," is not true in every metric space. The terms of a Cauchy sequence crowd together, but the point they crowd around may be missing from the space. Section 11.6 gives an example and names the spaces in which the converse does hold.

Small steps are not enough

It is tempting to think that a sequence whose consecutive terms get closer and closer must be Cauchy. It need not be. Consider the *harmonic sums*

$$s_0 = 0, \qquad s_n = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \sum_{j=1}^{n} \frac{1}{j} \quad (n \geq 1).$$

The first few are $s_1 = 1$, $s_2 = 3/2$, $s_3 = 11/6$ and $s_4 = 25/12$. Each step adds one more term, so $s_{n+1} - s_n = 1/(n+1)$, and Section 11.3 showed that these step sizes tend to 0. Yet the sums do not

settle down. Look at $n = 4$:

$$s_8 - s_4 = \frac{1}{5} + \frac{1}{6} + \frac{1}{7} + \frac{1}{8} \geq \frac{1}{8} + \frac{1}{8} + \frac{1}{8} + \frac{1}{8} = \frac{1}{2}.$$

Each of the four terms is at least $1/8$, the smallest of them. The same reasoning works for every $n \geq 1$. The difference $s_{2n} - s_n = \frac{1}{n+1} + \cdots + \frac{1}{2n}$ has n terms, and each of them is at least the last and smallest one, $1/(2n)$. Hence

$$s_{2n} - s_n = \sum_{j=n+1}^{2n} \frac{1}{j} \geq n \cdot \frac{1}{2n} = \frac{1}{2}.$$

However late we start, doubling the index adds at least $1/2$. Each single step is small, but many small steps add up.

To prove that the harmonic sums are not Cauchy, negate the definition, as we did for convergence in Section 11.3:

$$\exists \varepsilon > 0 \; \forall N \in \mathbb{N} \; \exists m, n \geq N : \quad d(x_m, x_n) \geq \varepsilon.$$

For the harmonic sums, take $\varepsilon = 1/2$, and let N be any proposed threshold. Choose $n = \max\{N, 1\}$, the larger of N and 1. We need $n \geq 1$ because the estimate $s_{2n} - s_n \geq \frac{1}{2}$ holds only from $n = 1$ on; for $n = 0$ the difference is $s_0 - s_0 = 0$. Then choose $m = 2n$. Both indices are at least N, and $|s_m - s_n| = s_{2n} - s_n \geq 1/2$. So no threshold works for the tolerance $1/2$, and the sequence is not Cauchy.

Proposition 11.5 says that being convergent implies being Cauchy. Its contrapositive, in the sense of Section 2.5, says that a sequence that is not Cauchy does not converge. So the harmonic sums do not converge to any real number.

What the harmonic sums lack is control over many steps taken together, not over single steps. Section 11.7 shows that steps which shrink by a fixed factor at every stage provide exactly this control.

11.6 Completeness excludes missing limits

Section 11.5 showed that every convergent sequence is Cauchy and noted that the converse can fail. The spaces in which it never fails have a name.

Definition 11.6 (Complete metric space)**.** A metric space (X, d) is *complete* if every Cauchy sequence in X converges to a point of X.

Informally, a complete space has no holes: whenever the terms of a sequence eventually all lie as close to one another as we like, there is a point of the space that they approach. Notice that this asks for more than consecutive terms getting closer together; the harmonic sums of Section 11.5 take smaller and smaller steps and still fail to be Cauchy.

Completeness is a property of a whole space, not of a single sequence; it is a promise about every Cauchy sequence in the space at once. To use it, we prove that one particular sequence is Cauchy, and the definition then hands us a limit that belongs to the space. To show that a space is *not* complete, it is enough to find one Cauchy sequence in it that does not converge to any point of the space.

An interval with a missing endpoint

The interval $(0, 1)$, with distance $|x - y|$, is not complete. Consider the sequence $x_n = 2^{-(n+1)}$, that is, $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$. Every term lies in $(0, 1)$, because $0 < 2^{-(n+1)} \leq \frac{1}{2}$.

First, the sequence is Cauchy. On the real line it converges to 0. Indeed, Section 11.3 showed that $2^k \geq k + 1$ for every $k \in \mathbb{N}$; with $k = n + 1$ this gives $2^{n+1} \geq n + 2$. Taking reciprocals, $|x_n - 0| = 2^{-(n+1)} \leq 1/(n+2) < 1/(n+1)$, and the argument given there for $1/(n+1)$ shows that these distances become smaller than any $\varepsilon > 0$ from some threshold on. By Proposition 11.5, the sequence is Cauchy on the real line. The Cauchy condition involves only distances between terms, and those distances are the same whether we regard the terms as points of $(0, 1)$ or of $\mathbb{R}$. So the sequence is Cauchy in $(0, 1)$ as well.

Second, the sequence has no limit in $(0, 1)$. Suppose it converged to some point L of $(0, 1)$. Distances in $(0, 1)$ are distances in $\mathbb{R}$, so it would also converge to L on the real line. It already converges to 0 there, so Proposition 11.3 gives $L = 0$. But 0 is not in $(0, 1)$, a contradiction. The only possible destination of the sequence has been removed from the space.

The real line

The real line, with distance $|x - y|$, is complete. We accept this as a starting fact about the real numbers, just as we accepted the existence of square roots in Section 1.8 and the Archimedean property in Section 2.3. It is the last of the four starting facts listed in *To the reader* at the front of the book. A proof would require constructing the real numbers from simpler objects, which we do not do. The closed intervals that we prove complete later in this section owe their completeness to this one fact, and so, in the end, does every later argument that needs a limit to exist without being able to name it in advance.

The rational numbers show what the assumption rules out. Cutting off the decimal expansion of $\sqrt{2} = 1.41421\ldots$ after more and more digits gives the rational numbers $t_0 = 1$, $t_1 = 1.4$, $t_2 = 1.41$, $t_3 = 1.414$, $t_4 = 1.4142$, and so on. These numbers crowd together, and the point they crowd around is $\sqrt{2}$, which is not a fraction of integers (Theorem 4.3). So the set $\mathbb{Q}$ of rational numbers, with distance $|x - y|$, has a hole at $\sqrt{2}$: it is not complete. The completeness of $\mathbb{R}$ says that the number line has no such holes.

To make this precise, we follow the two steps used above for $(0, 1)$. Both need the fact that $t_k \to \sqrt{2}$ on the real line, so we check that first. The difference $\sqrt{2} - t_k$ consists of the digits of $\sqrt{2}$ after the kth decimal place; for instance, $\sqrt{2} - t_2 = 1.41421\ldots - 1.41 = 0.00421\ldots$, which is less than 0.01. In general the first k digits of $\sqrt{2} - t_k$ after the decimal point are zeros, so $0 \leq \sqrt{2} - t_k \leq 10^{-k}$. Moreover $10^{-k} \leq 2^{-k} \leq 1/(k+1)$, because $10^k \geq 2^k$ and, by Section 11.3, $2^k \geq k + 1$. So the argument given there for $1/(k+1)$ shows that $t_k \to \sqrt{2}$ on the real line.

With this in hand, the two steps go through. First, by Proposition 11.5, the sequence $t_0, t_1, t_2, \ldots$ is Cauchy on the real line. The distances $|t_m - t_n|$ are the same whether we regard the t_k as rational or as real numbers, so it is also a Cauchy sequence in $\mathbb{Q}$. Second, suppose it converged in $\mathbb{Q}$ to some rational number L. Distances in $\mathbb{Q}$ are distances in $\mathbb{R}$, so it would converge to L on the real line too, and Proposition 11.3 would give $L = \sqrt{2}$. That is impossible, because $\sqrt{2}$ is not rational.

So this Cauchy sequence has no limit in $\mathbb{Q}$.

Closed subsets of a complete space

Completeness passes from a space to many of its subsets. Let (X, d) be a metric space, let $C \subseteq X$, and give C the same distance d, as in Section 11.2.

Definition 11.7 (Closed subset)**.** A subset C of a metric space (X, d) is *closed* in X if, whenever a sequence of points of C converges to a point of X, that limit lies in C.

In words, you cannot leave C by taking limits.

Proposition 11.8. *If (X, d) is complete and C is closed in X, then C, with the same distance d, is complete.*

Proof. Take a Cauchy sequence of points of C. The Cauchy condition involves only distances between terms, and those distances are the same whether we regard the terms as points of C or of X. So the sequence is also Cauchy in X. Since X is complete, the sequence converges to some point of X. Since C is closed, that point lies in C, and the distances from the terms to it are again the same in C as in X. So the sequence converges to a point of C. Hence every Cauchy sequence in C converges in C. □

The interval $(0, 1]$ shows what goes wrong without closedness. The points $x_n = 1/(n+1)$ all lie in $(0, 1]$, and on the real line they converge to 0, which is not in $(0, 1]$. So $(0, 1]$ is not closed in $\mathbb{R}$. It is not complete either: exactly as for $(0, 1)$, the sequence is Cauchy in $(0, 1]$, and its only possible limit, 0, is missing from $(0, 1]$.

The closed interval $[0, 1]$, on the other hand, is closed in $\mathbb{R}$, and therefore complete. To prove that it is closed, suppose a sequence of points $x_n \in [0, 1]$ converges on the real line to a number L. We show that $L < 0$ and $L > 1$ are both impossible. We use a rule from Section 2.6: for every positive number ε, the inequality $|x - c| < \varepsilon$ holds exactly when $c - \varepsilon < x < c + \varepsilon$. With x_n in place of x and L in place of c, it says that $|x_n - L| < \varepsilon$ means $L - \varepsilon < x_n < L + \varepsilon$.

If $L < 0$, then L lies a distance $-L$ to the left of 0, the nearest point of the interval. We take half of this distance as the tolerance: $\varepsilon = -L/2$, which is positive. The idea is that every number less than ε away from L is still negative, while the late terms of the sequence must lie that close to L. In detail, convergence supplies a threshold N such that $|x_n - L| < -L/2$ for every $n \geq N$. In particular the term x_N satisfies

$$x_N < L + \left(-\frac{L}{2}\right) = \frac{L}{2} < 0.$$

This contradicts $x_N \in [0, 1]$. If $L > 1$, then L lies a distance $L - 1$ to the right of 1, and half of this distance gives the positive tolerance $\varepsilon = (L - 1)/2$. Now convergence supplies a threshold N such that $|x_n - L| < (L - 1)/2$ for every $n \geq N$, and the term x_N satisfies

$$x_N > L - \frac{L-1}{2} = \frac{L+1}{2} > 1,$$

where the last step holds because $L > 1$ gives $L + 1 > 2$. This again contradicts $x_N \in [0, 1]$. Both cases are impossible, so $0 \leq L \leq 1$, that is, $L \in [0, 1]$. Hence $[0, 1]$ is closed in the complete real line, and by Proposition 11.8 it is complete.

The same argument, with a and b in place of 0 and 1 and the tolerances $(a-L)/2$ and $(L-b)/2$, shows that every closed interval $[a,b]$ with $a<b$ is complete. For instance, if $L<a$, a late term x_N satisfies $x_N < L+(a-L)/2 = (L+a)/2 < a$, where the last step holds because $L<a$ gives $L+a<2a$. This is impossible for a point of $[a,b]$.

Pause and decide. Is the set $[0,\infty)$ of nonnegative real numbers, with distance $|x-y|$, complete?

Check. Yes. Suppose a sequence of nonnegative numbers converges on the real line to L. If $L<0$, the first case of the argument for $[0,1]$ applies word for word and produces a negative term, which is impossible. So $L\geq 0$, the set $[0,\infty)$ is closed in $\mathbb{R}$, and Proposition 11.8 shows that it is complete. A complete space need not have a right-hand end; what matters is that no limit is missing.

11.7 A geometric bound controls an entire tail

The harmonic sums of Section 11.5 failed to be Cauchy because their steps, though small, added up to a lot. This section shows that steps which shrink by a fixed factor at every stage cannot add up to much. The tool is the finite geometric sum.

The finite geometric sum

Let q be a real number with $q\neq 1$, and let r be a positive integer. Then

$$1+q+q^2+\cdots+q^{r-1}=\frac{1-q^r}{1-q}.$$

To see why, first take $r=4$ and call the sum $S=1+q+q^2+q^3$. Multiplying by q raises every power by one:

$$\begin{aligned} S &= 1+q+q^2+q^3,\\ qS &= \quad q+q^2+q^3+q^4. \end{aligned}$$

When the second line is subtracted from the first, the terms q, q^2 and q^3 cancel, leaving $S-qS=1-q^4$. The left side is $(1-q)S$, and $1-q\neq 0$, so we may divide by it: $S=(1-q^4)/(1-q)$. For a general r, the same subtraction applies to $S=1+q+\cdots+q^{r-1}$ and $qS=q+q^2+\cdots+q^r$. The terms q through q^{r-1} appear in both, so only 1 and q^r remain, and $(1-q)S=1-q^r$. As a check, take $q=1/2$ and $r=4$. The sum is $1+\frac{1}{2}+\frac{1}{4}+\frac{1}{8}=\frac{15}{8}$, and the formula gives $\left(1-\frac{1}{16}\right)\big/\left(1-\frac{1}{2}\right)=\frac{15}{16}\cdot 2=\frac{15}{8}$.

From now on let $0<q<1$. We need sums that start at a later power. For integers $m>n\geq 0$, factor out q^n and apply the formula with $r=m-n$:

$$\sum_{j=n}^{m-1} q^j = q^n\left(1+q+\cdots+q^{m-n-1}\right) = q^n\,\frac{1-q^{m-n}}{1-q} \leq \frac{q^n}{1-q}.$$

The final inequality holds because $q^{m-n}>0$, so $1-q^{m-n}<1$, and the factor $q^n/(1-q)$ is positive. The bound on the right does not depend on m: however many terms we add, a sum of consecutive powers starting at q^n never exceeds $q^n/(1-q)$. We call this the *geometric tail bound*.

Powers of a number between zero and one tend to zero

The bound $q^n/(1-q)$ is useful only if it becomes small as n grows, so we need $q^n \to 0$. Section 11.3 handled $q = 1/2$ by proving $2^n \geq n+1$. For a general q with $0 < q < 1$, we imitate that proof with $1/q$ in place of 2. Put $c = 1/q - 1$. Since $0 < q < 1$, we have $1/q > 1$, so $c > 0$, and $1/q = 1 + c$. We prove by induction that

$$(1+c)^n \geq 1 + nc \qquad \text{for every } n \in \mathbb{N}.$$

This is known as *Bernoulli's inequality*. For $c = 1$ it is the inequality $2^n \geq n+1$ of Section 11.3. At $n = 0$ both sides equal 1. If $(1+c)^n \geq 1 + nc$ for some n, multiply both sides by the positive number $1 + c$:

$$(1+c)^{n+1} \geq (1+nc)(1+c) = 1 + c + nc + nc^2 = 1 + (n+1)c + nc^2 \geq 1 + (n+1)c,$$

because $nc^2 \geq 0$. This is the inequality for $n+1$, so it holds for every n. Since $q^n = 1/(1+c)^n$, and both $(1+c)^n$ and $1 + nc$ are positive, taking reciprocals reverses the order and gives

$$0 < q^n \leq \frac{1}{1+nc}.$$

Now let $\varepsilon > 0$ be given, and use the Archimedean property to choose a positive integer $N > 1/(c\varepsilon)$. If $n \geq N$, then $1 + nc > nc \geq Nc > 1/\varepsilon$, where the last step multiplies $N > 1/(c\varepsilon)$ by the positive number c. Taking reciprocals once more gives $q^n \leq 1/(1+nc) < \varepsilon$. This proves $q^n \to 0$.

The threshold depends on ε and, through c, on q: the closer q is to 1, the smaller c is, and the larger the number $1/(c\varepsilon)$ that N must exceed. For example, $q = 0.9$ gives $c = 1/9$, and for $\varepsilon = 0.01$ the recipe asks for $N > 900$. In fact $0.9^n < 0.01$ already holds from $n = 44$ on, since $0.9^{44} \approx 0.0097$ while $0.9^{43} \approx 0.0108$. As in Section 11.3, a threshold produced by a proof may be far from the smallest one, and that is allowed.

Geometrically shrinking steps give a Cauchy sequence

Proposition 11.9. *Let $x_0, x_1, x_2, \ldots$ be a sequence in a metric space (X, d), and suppose there are numbers $D \geq 0$ and $0 < q < 1$ such that*

$$d(x_{j+1}, x_j) \leq Dq^j \qquad \textit{for every } j \in \mathbb{N}.$$

Then $d(x_m, x_n) \leq Dq^n/(1-q)$ whenever $m > n$, and the sequence is a Cauchy sequence.

Before the proof, here is an example. For the sequence $x_n = 1 - 2^{-n}$ of Section 11.3, each step is

$$x_{j+1} - x_j = \left(1 - 2^{-(j+1)}\right) - \left(1 - 2^{-j}\right) = 2^{-j} - 2^{-(j+1)} = 2^{-(j+1)} = \tfrac{1}{2}\left(\tfrac{1}{2}\right)^j.$$

This number is positive, so it equals its absolute value $d(x_{j+1}, x_j)$, and the hypothesis holds with $D = 1/2$ and $q = 1/2$. The proposition then promises $|x_m - x_n| \leq \frac{1}{2}\left(\frac{1}{2}\right)^n \Big/ \frac{1}{2} = 2^{-n}$ for all $m > n$. Directly, $|x_m - x_n| = 2^{-n} - 2^{-m}$, which is indeed less than 2^{-n}.

Proof. Step 1: from single steps to many steps. Fix n. We show by induction on m that, for every $m > n$,

$$d(x_m, x_n) \le \sum_{j=n}^{m-1} d(x_{j+1}, x_j).$$

In words, the distance from x_n to x_m is at most the total length of the single steps between them. For instance, with $m = n + 3$ the claim reads

$$d(x_{n+3}, x_n) \le d(x_{n+3}, x_{n+2}) + d(x_{n+2}, x_{n+1}) + d(x_{n+1}, x_n).$$

When $m = n + 1$, the sum has just one term, $d(x_{n+1}, x_n)$, and the claim is an equality. Suppose the claim holds for some $m > n$. The triangle inequality with x_m as the stopping point gives

$$d(x_{m+1}, x_n) \le d(x_{m+1}, x_m) + d(x_m, x_n) \le d(x_{m+1}, x_m) + \sum_{j=n}^{m-1} d(x_{j+1}, x_j) = \sum_{j=n}^{m} d(x_{j+1}, x_j),$$

which is the claim for $m + 1$.

Step 2: a bound for the whole tail. Combine Step 1 with the assumed bounds on the steps and with the geometric tail bound. For $m > n$,

$$d(x_m, x_n) \le \sum_{j=n}^{m-1} d(x_{j+1}, x_j) \le \sum_{j=n}^{m-1} Dq^j = D \sum_{j=n}^{m-1} q^j \le \frac{Dq^n}{1-q}.$$

The last step multiplies the geometric tail bound $\sum_{j=n}^{m-1} q^j \le q^n/(1-q)$ by the number D, which keeps the inequality in the same direction because $D \ge 0$.

Step 3: the Cauchy condition. Let $\varepsilon > 0$. If $D = 0$, Step 2 gives $d(x_m, x_n) \le 0$, and hence $d(x_m, x_n) = 0$, for all $m > n$, because distances are never negative. So all the terms are equal, and the threshold $N = 0$ works. Now let $D > 0$. The number $\varepsilon(1-q)/D$ is positive, and $q^n \to 0$, so there is a threshold N such that $q^n < \varepsilon(1-q)/D$ for every $n \ge N$. Multiplying by the positive number $D/(1-q)$ gives $Dq^n/(1-q) < \varepsilon$ for every $n \ge N$. Take any two indices $m, n \ge N$. If $m = n$, their distance is 0, which is less than ε. Otherwise, by symmetry of d, we may name them so that $m > n$, and Step 2 gives $d(x_m, x_n) \le Dq^n/(1-q) < \varepsilon$. So the sequence is Cauchy. □

The bound $Dq^n/(1-q)$ in Step 2 involves n but not m, so a single choice of n controls the whole tail from x_n onward, however far beyond n the index m goes. The harmonic sums had no such control: there the steps from index n to index $2n$ add up to at least $1/2$, however large n is, while here the steps after index n add up to at most $Dq^n/(1-q)$, which is as small as we like once n is large.

If the space is complete, as the real line and every closed interval are, then the definition of completeness turns the conclusion of the proposition into convergence: the sequence has a limit in the space. This combination of a geometric bound on the steps with completeness is the heart of the contraction theorem in Chapter 12.

Exercises

Exercises 11.1–11.4 are core exercises, and Exercises 11.5 and 11.6 are transfer exercises. Complete solutions are in Appendix B, under "Week 11."

Exercise 11.1. Suppose $|x - a| \leq 0.02$ and $|y - b| \leq 0.03$. Find an upper bound for $|(2x - y) - (2a - b)|$ that holds for all such numbers, and give values of x, y, a and b for which your bound is attained.

Exercise 11.2. Prove that the sequence $a_n = 3/(n + 1)$, for $n \in \mathbb{N}$, converges to 0. For each tolerance $\varepsilon > 0$, state explicitly the threshold N that you use.

Exercise 11.3. Show that the function d_∞ on $\mathbb{R}^n$ from Section 11.2 satisfies the first two requirements of Definition 11.1: $d_\infty(x, y) \geq 0$, with equality exactly when $x = y$, and $d_\infty(x, y) = d_\infty(y, x)$. Together with the triangle inequality proved in that section, this shows that d_∞ is a metric.

Exercise 11.4. A sequence $x_0, x_1, x_2, \ldots$ in a metric space satisfies $d(x_{j+1}, x_j) \leq 2(1/3)^j$ for every $j \in \mathbb{N}$. Find an upper bound for $d(x_m, x_n)$ that holds for all $m > n$ and does not depend on m. Then show that the sequence is Cauchy, with an explicit threshold for each tolerance $\varepsilon > 0$.

Exercise 11.5. Give a Cauchy sequence in the interval $(2, 3)$, with distance $|x - y|$, that has no limit in $(2, 3)$. Prove both properties of your sequence.

Exercise 11.6. Let A be a set of real numbers. A real number b is an *upper bound* for A if $a \leq b$ for every $a \in A$. A *maximum* of A is an upper bound for A that is itself a member of A. A *supremum*, or *least upper bound*, of A is an upper bound s for A such that $s \leq b$ for every upper bound b for A.

1. Show that 1 is a supremum of the interval $(0, 1)$.
2. Show that $(0, 1)$ has no maximum.
3. The completeness of the real line is often stated in the following form: *every nonempty set of real numbers that has an upper bound has a supremum*. Explain why this statement would become false if the word "supremum" were replaced by "maximum."

CHAPTER 12

The contraction theorem: an unfamiliar proof made readable

This week

Prove the contraction theorem: on a nonempty complete metric space, a map from the space to itself that multiplies every distance by at most a fixed factor below one has exactly one fixed point, and repeating the map from any starting point approaches it. Treat existence, uniqueness, convergence and the error bounds as four separate claims, each with its own short argument built from the tools of Chapter 11. Use the error bounds to guarantee the accuracy of an approximation without knowing the exact answer, and see, by removing the hypotheses one at a time, why each of them is needed.

12.1 From solving an equation to repeating a map

Section 4.4 proved that the equation

$$x = \frac{x+1}{2}$$

has exactly one real solution, namely $x = 1$. Section 5.3 reached the same number in a different way. Starting from $x_0 = 0$, it applied the rule $x \mapsto (x+1)/2$ again and again and obtained

$$0,\ \tfrac{1}{2},\ \tfrac{3}{4},\ \tfrac{7}{8},\ \ldots, \qquad \text{in general } x_n = 1 - 2^{-n},$$

and Section 11.3 proved that these numbers converge to 1. So the number 1 can be seen in two ways: as the solution of an equation, and as the limit of a repeated update. This chapter turns the second point of view into a general method for solving equations.

The general setting is a map $T : X \to X$, where X is a metric space as in Chapter 11. (Recall from Section 3.2 that a map is the same thing as a function.) A *fixed point* of T is a point $x_* \in X$ with $T(x_*) = x_*$, that is, a point that the map leaves where it is. Section 5.3 already used this name for the number 1 and the rule $x \mapsto (x+1)/2$. Starting from any point $x_0 \in X$, we can apply T over and over:

$$x_1 = T(x_0), \qquad x_2 = T(x_1), \qquad x_3 = T(x_2), \qquad \ldots,$$

in general $x_{n+1} = T(x_n)$ for every $n \in \mathbb{N}$. The points $x_0, x_1, x_2, \ldots$ are called the *iterates* of T starting from x_0. If the iterates converge and their limit is a fixed point, then repeating the map is a way of solving the equation $T(x) = x$. In practice we stop after finitely many steps, so what we obtain is an approximate solution, and we will want to know how good it is.

Repeating a map is a very simple algorithm, but it does not always work. Here are four ways in which it can fail.

- The iterates can run away. For $T(x) = 2x$ on $\mathbb{R}$, starting from $x_0 = 1$, the iterates are $1, 2, 4, 8, \ldots$, although T has the fixed point 0.
- The iterates can jump back and forth forever. For $T(x) = 1 - x$ on $\mathbb{R}$, starting from $x_0 = 0$, the iterates are $0, 1, 0, 1, \ldots$. The fixed point is $1/2$, since $1 - \frac{1}{2} = \frac{1}{2}$, but every iterate stays at distance $1/2$ from it.
- There can be several fixed points, and then the phrase "the solution" has no clear meaning, as Section 4.4 warned. For $T(x) = x^2$ on the interval $[0, 1]$, both 0 and 1 are fixed points.
- The iterates can approach a point that is missing from the space. Take $T(x) = x/2$ on the interval $(0, 1)$. An interval like this one, which leaves out both of its endpoints, is called an *open interval*. Starting from $x_0 = 1/2$, the iterates are $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$. They approach 0, which does not belong to $(0, 1)$, and in fact T has no fixed point in $(0, 1)$ at all.

We want conditions on T and X that rule out all of these failures in advance, so that we do not have to compute a few iterates and hope that the pattern continues. The first condition says that T brings points closer together. In stating it, and from now on, we often write Tx for $T(x)$ to save brackets; both mean the same point.

Definition 12.1 (Contraction)**.** Let (X, d) be a metric space, and let q be a real number with $0 \le q < 1$. A map $T : X \to X$ is a *contraction with factor* q if

$$d(Tx, Ty) \le q\, d(x, y) \qquad \text{for all } x, y \in X.$$

This inequality is called the *contraction inequality*, and q is called a *contraction factor* of T.

In words: for any two points x and y, the new points Tx and Ty are at most q times as far apart as x and y were. With $q = 1/2$, for instance, Tx and Ty are at most half as far apart as x and y. If T is a contraction with factor q, it is also one with every larger factor q' below one: if $q \le q' < 1$, then $q\, d(x, y) \le q'\, d(x, y)$, because distances are never negative. That is why we say "a" contraction factor rather than "the" contraction factor.

Reading the contraction inequality

Two features of the definition matter in everything that follows.

First, x and y are *any* two points of X. They need not be consecutive iterates, and neither of them needs to be close to a fixed point. For $T(x) = (x + 1)/2$ on $\mathbb{R}$, with the usual distance $d(x, y) = |x - y|$, we compute

$$|T(x) - T(y)| = \left|\frac{x+1}{2} - \frac{y+1}{2}\right| = \left|\frac{(x+1) - (y+1)}{2}\right| = \frac{1}{2}|x - y|.$$

So this T is a contraction with factor $1/2$. The constant terms cancel when two outputs are compared, just as they did in the uniqueness proof of Section 4.4.

Second, one number q must work for all pairs at once. It is not enough that every pair of different points moves closer together. If the amount of shrinking may depend on the pair, then the ratio $d(Tx, Ty)/d(x, y)$, for different points x and y, can come closer and closer to 1 as the pair changes,

and then the conclusions of this chapter can fail. Section 12.6 gives a map that brings every pair of different points strictly closer together and still has no fixed point.

The outputs must be allowed inputs

The second condition is built into the arrow of $T : X \to X$: every output $T(x)$ must again be a point of X. A map from a set to itself is called a *self-map*. This condition is what allows us to apply T to x_1, then to x_2, and so on, exactly as Section 3.2 required the outputs of one function to be allowed inputs of the next before composing them.

The condition is easy to overlook when a formula makes sense for more numbers than we intend to use. Take $T(x) = (x + 1)/2$ again, but now only on the interval $[0, 1/2]$. The contraction inequality still holds for points of this interval, with factor $1/2$, because it holds for all real numbers. Yet $T(1/2) = 3/4$ lies outside $[0, 1/2]$, so T is not a map from $[0, 1/2]$ to itself. Starting from $x_0 = 0$, we get $x_1 = 1/2$ and then $x_2 = 3/4$, and the iterates have already left the interval. On $[0, 1]$, by contrast, T is a self-map: if $0 \le x \le 1$, then $1 \le x + 1 \le 2$, so $1/2 \le T(x) \le 1$.

So before using a contraction, we check two separate things: that T maps X into X, and that the contraction inequality holds for all pairs of points of X.

Pause and decide. Is $T(x) = x^2$ a contraction of the interval $[0, 1]$? Is it a contraction of the interval $[0, 1/3]$?

Check. On $[0, 1]$ it is not. It is a self-map there, because $0 \le x \le 1$ gives $0 \le x^2 \le 1$. But for $x = 1$ and $y = 1/2$ we get $|T(x) - T(y)| = 1 - \frac{1}{4} = \frac{3}{4}$, which is larger than $|x - y| = \frac{1}{2}$; no factor $q < 1$, and not even $q = 1$, can work for this pair. On $[0, 1/3]$ it is a contraction. It is a self-map, because $0 \le x \le 1/3$ gives $0 \le x^2 \le 1/9 \le 1/3$. For the contraction inequality, factor the difference of two squares: $|x^2 - y^2| = |x - y|\,|x + y|$. For $x, y \in [0, 1/3]$ we have $0 \le x + y \le 2/3$, so $|T(x) - T(y)| \le \frac{2}{3}|x - y|$, and $q = 2/3$ is a contraction factor.

12.2 The theorem and its dependency map

The theorem below says that these two conditions rule out all four failures of Section 12.1, as long as the space is nonempty and complete. The theorem is also known as Banach's fixed-point theorem, after the Polish mathematician Stefan Banach (1892–1945), who published it in 1922.

Theorem 12.2 (Contraction theorem)**.** *Let (X, d) be a nonempty complete metric space, and let $T : X \to X$ be a contraction with factor q, where $0 \le q < 1$. Then the following hold.*

1. *T has exactly one fixed point x_*.*
2. *For every starting point $x_0 \in X$, the iterates defined by $x_{n+1} = T(x_n)$ converge to x_*, and for every $n \in \mathbb{N}$*
$$d(x_n, x_*) \le \frac{q^n}{1-q}\, d(x_1, x_0). \tag{12.1}$$
3. *For every point $z \in X$, whether or not it is an iterate,*
$$d(z, x_*) \le \frac{d(z, Tz)}{1-q}. \tag{12.2}$$

Section 1.3 agreed that $a^0 = 1$ for every nonzero number a. In this chapter we extend this agreement and read q^0 as 1 even when $q = 0$. So for $n = 0$ the bound (12.1) reads $d(x_0, x_*) \le d(x_1, x_0)/(1-q)$.

Four separate promises

The theorem makes four promises, and each of them needs its own argument.

- *Existence*: some point of X is left unchanged by T.
- *Uniqueness*: no two different points are left unchanged by T.
- *Convergence*: from every starting point, the iterates approach the fixed point.
- *Error bounds*: after finitely many steps, we can say how far we still are from the fixed point, even though we do not know where it is.

For a general map, the first two promises do not follow from each other, and even together they do not give the third. The examples of Section 12.1 show this. The map $x \mapsto x^2$ on $[0, 1]$ has fixed points, but more than one. The map $x \mapsto x/2$ on $(0, 1)$ has at most one fixed point, but only because it has none at all; as Section 4.4 pointed out, "at most one" is also true when there are no solutions. The map $x \mapsto 1 - x$ on $\mathbb{R}$ has exactly one fixed point, since $1 - x = x$ forces $x = 1/2$, yet its iterates from $x_0 = 0$ jump between 0 and 1 forever. The fourth promise asks for still more than convergence: a bound on the remaining error that we can compute after finitely many steps.

Pause and decide. The map $T(x) = 1 - x$ on $\mathbb{R}$ has a fixed point, but its iterates from $x_0 = 0$ do not converge. Which hypothesis of Theorem 12.2 does it fail?

Check. The space $\mathbb{R}$ is nonempty and complete, and T maps $\mathbb{R}$ to itself. The contraction inequality is what fails. For all real x and y, $|T(x) - T(y)| = |(1 - x) - (1 - y)| = |y - x| = |x - y|$, so T never shrinks a distance at all. A factor $q < 1$ would require $|x - y| \le q|x - y|$, which is false whenever $x \ne y$.

Two ways to measure the error

The *error* of a point $z \in X$ is its distance $d(z, x_*)$ from the fixed point. The last two parts of the theorem answer the practical question "how large is the error?" in two different ways.

The bound (12.1) is called the *a priori estimate*, from the Latin for "from what comes before." It uses only the factor q and the length $d(x_1, x_0)$ of the first step. So after a single step, before the rest of the iteration has been computed, it tells us how many steps will be enough for a desired accuracy.

The bound (12.2) is called the *residual estimate*. The number $d(z, Tz)$ is the *residual* of z: it measures how far z is from satisfying the equation $Tz = z$, and it is zero exactly when z is a fixed point. The point z may come from anywhere: it may be an iterate, a guess, or the output of some other method. The residual estimate turns one evaluation of T into a guaranteed upper bound on the error of z. A guaranteed bound of this kind, computed from known quantities, is called an *error certificate*.

Both bounds divide by $1 - q$, which is positive because $q < 1$. When q is close to 1, the factor $1/(1 - q)$ is large and the bounds are weak; for $q = 0.99$, for instance, it equals 100. At $q = 1$ the bounds would require dividing by zero. Section 12.4 shows both bounds at work.

The plan of the proof

Here is the whole proof in outline, before the details. Each step names the earlier facts it uses.

1. *The steps shrink geometrically.* Because X is nonempty, there is a point to start from, and because T maps X into X, all the iterates can be formed. The contraction inequality and induction (Chapter 5) then show that the step from x_n to x_{n+1} has length at most q^n times the length of the first step.
2. *The iterates form a Cauchy sequence.* Proposition 11.9, which rests on the triangle inequality and the geometric sum, bounds the distance between any two late iterates.
3. *A limit exists in X.* Completeness (Section 11.6) turns the Cauchy sequence into a convergent one. Call its limit x_*.
4. *The limit is a fixed point.* The triangle inequality, the contraction inequality and the definition of convergence show that the distance between Tx_* and x_* is smaller than every positive tolerance, so the two points are equal.
5. *There is only one fixed point.* The contraction inequality, applied to two fixed points, shows that they coincide. This is the comparison argument of Section 4.4 in a new setting.
6. *The error bounds hold.* The triangle inequality, the contraction inequality and the equation $Tx_* = x_*$ from Step 4 give (12.2). Applied to the iterate x_n, together with Step 1, the residual estimate gives (12.1).

A list like this, which records what each step of a proof depends on, is a *dependency map* of the proof. It tells you what you need to know before you read the proof, and it shows where each hypothesis is used. Here the prerequisites are induction, the triangle inequality, geometric sums, convergence, Cauchy sequences and completeness, all from Chapters 5 and 11. No calculus is

needed. Chapter 17 uses dependency maps to approach a proof whose ingredients are not all familiar yet.

12.3 The proof, one step at a time

The proof follows the six steps of Section 12.2.

Proof. **Step 1: the steps shrink geometrically.** Since X is nonempty, we can choose a starting point $x_0 \in X$; it may be any point at all. Define $x_{n+1} = T(x_n)$ for every $n \in \mathbb{N}$. Because T maps X into X, each iterate is again a point of X, so T can be applied to it. This is where the self-map hypothesis is used. Write

$$D = d(x_1, x_0)$$

for the length of the first step. We prove by induction on n that

$$d(x_{n+1}, x_n) \le q^n D \qquad \text{for every } n \in \mathbb{N}.$$

For $n = 0$ both sides equal D, because $q^0 = 1$. Now suppose the inequality holds for some n. Since $x_{n+2} = T(x_{n+1})$ and $x_{n+1} = T(x_n)$, the contraction inequality applied to the two points x_{n+1} and x_n gives

$$d(x_{n+2}, x_{n+1}) = d(Tx_{n+1}, Tx_n) \le q\, d(x_{n+1}, x_n) \le q \cdot q^n D = q^{n+1} D.$$

The last inequality multiplies the induction hypothesis by q, which is not negative. This is the claim for $n + 1$, which completes the induction.

Step 2: the iterates form a Cauchy sequence. Step 1 is exactly the hypothesis $d(x_{j+1}, x_j) \le Dq^j$ of Proposition 11.9, with $D = d(x_1, x_0)$, which is not negative because it is a distance. The proposition assumes $0 < q < 1$. If $q = 0$, use the factor $1/2$ instead: $q^j \le (1/2)^j$ for every j (both sides equal 1 when $j = 0$, and the left side is 0 when $j \ge 1$), so multiplying by $D \ge 0$ gives $d(x_{j+1}, x_j) \le D(1/2)^j$. Either way, the proposition shows that the iterates form a Cauchy sequence. Recall the idea: by the triangle inequality, $d(x_m, x_n)$ is at most the total length of the single steps from x_n to x_m, and Step 1 together with the geometric sum keeps that total small once n is large.

Step 3: completeness supplies a limit. The space X is complete, so the Cauchy sequence $x_0, x_1, x_2, \ldots$ converges to some point of X. Call this point x_*. This is the only place in the proof where completeness is used. It cannot be skipped: Section 12.6 gives a contraction of the incomplete space $(0, 1)$ whose iterates form a Cauchy sequence with no limit in the space.

Step 4: the limit is a fixed point. We show that the number $d(Tx_*, x_*)$ is zero. For every natural number n, use the triangle inequality with the point Tx_n as a stop on the way, then the contraction inequality for the pair x_*, x_n, and finally $Tx_n = x_{n+1}$:

$$d(Tx_*, x_*) \le d(Tx_*, Tx_n) + d(Tx_n, x_*) \le q\, d(x_*, x_n) + d(x_{n+1}, x_*).$$

Both distances on the right become small when n is large. To make this precise, let $\varepsilon > 0$. Because $x_n \to x_*$, the definition of convergence with tolerance $\varepsilon/2$ supplies a threshold N such that $d(x_k, x_*) < \varepsilon/2$ for every $k \ge N$. Now take $n = N$ in the display. Then both n and $n + 1$ are at least

N. Since $q < 1$, we have $q\,d(x_*, x_n) \le d(x_*, x_n) = d(x_n, x_*) < \varepsilon/2$, and also $d(x_{n+1}, x_*) < \varepsilon/2$. Hence

$$d(Tx_*, x_*) < \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

The number $r = d(Tx_*, x_*)$ does not depend on n or on ε, and we have shown that it is smaller than every positive ε. Such a number must be zero. Indeed, $r \ge 0$, and if r were positive, we could take $\varepsilon = r$ and obtain $r < r$, which is impossible. So $d(Tx_*, x_*) = 0$, and by the first requirement of a metric, $Tx_* = x_*$.

The argument never needs an iterate to equal x_*, and it could not rely on one: the iterates of $(x + 1)/2$ from $x_0 = 0$ never reach 1.

Step 5: there is only one fixed point. Suppose y_* is also a fixed point of T. Since $x_* = Tx_*$ and $y_* = Ty_*$, the contraction inequality gives

$$d(x_*, y_*) = d(Tx_*, Ty_*) \le q\,d(x_*, y_*).$$

Subtracting $q\,d(x_*, y_*)$ from both sides gives $(1 - q)\,d(x_*, y_*) \le 0$. Dividing by the positive number $1 - q$ gives $d(x_*, y_*) \le 0$. Distances are never negative, so $d(x_*, y_*) = 0$, and therefore $y_* = x_*$. As in Section 4.4, this argument compares two hypothetical solutions and never needs to find one.

Steps 1–5 also prove the convergence claim for *every* starting point. The starting point x_0 in Step 1 was arbitrary, and Steps 1–4 showed that its iterates converge to a fixed point of T. By Step 5, T has only one fixed point. So, whatever the starting point, the iterates converge to the same point x_*.

Step 6: the error bounds. We prove the residual estimate first, because the a priori estimate follows from it. Let z be any point of X. Use the triangle inequality with Tz as a stop, then $x_* = Tx_*$ from Step 4, then the contraction inequality:

$$d(z, x_*) \le d(z, Tz) + d(Tz, x_*) = d(z, Tz) + d(Tz, Tx_*) \le d(z, Tz) + q\,d(z, x_*).$$

Subtracting $q\,d(z, x_*)$ from both sides gives $(1 - q)\,d(z, x_*) \le d(z, Tz)$, and dividing by the positive number $1 - q$ gives (12.2).

For the a priori estimate, fix n and take z to be the iterate x_n. Its residual is $d(x_n, Tx_n) = d(x_n, x_{n+1}) = d(x_{n+1}, x_n)$, which Step 1 bounds by $q^n D$. So the residual estimate gives

$$d(x_n, x_*) \le \frac{d(x_n, x_{n+1})}{1 - q} \le \frac{q^n D}{1 - q} = \frac{q^n}{1 - q}\,d(x_1, x_0),$$

which is (12.1). This completes the proof. □

Pause and decide. Use the residual estimate (12.2) to give a second, one-line proof that T has at most one fixed point.

Check. If z is any fixed point, then $Tz = z$, so its residual $d(z, Tz)$ is 0. The residual estimate gives $d(z, x_*) \le 0/(1 - q) = 0$, so $z = x_*$. Every fixed point equals x_*.

12.4 What the error estimates are for

Predicting how many steps are enough

Take $T(x) = (x+1)/2$ on $\mathbb{R}$, which is a contraction with factor $q = 1/2$, and start from $x_0 = 0$. The first step gives $x_1 = 1/2$, so $D = d(x_1, x_0) = 1/2$, and the a priori estimate (12.1) becomes

$$|x_n - 1| \le \frac{(1/2)^n}{1 - 1/2} \cdot \frac{1}{2} = 2^{-n}.$$

Suppose we want an error below $0.001 = 1/1000$. Since $2^9 = 512$ and $2^{10} = 1024$, the bound 2^{-n} is still larger than $1/1000$ at $n = 9$, but at $n = 10$ it equals $1/1024$, which is smaller. So ten steps are guaranteed to be enough, and so is any larger number of steps, because the bound 2^{-n} only decreases as n grows. In this example the estimate cannot be improved: the formula $x_n = 1 - 2^{-n}$ of Section 5.3 shows that the error is exactly 2^{-n}, so the bound is attained for every n, and nine steps really are too few.

Usually the bound is not attained. It is built from the first step and the worst-case factor q, so the true error is often much smaller, and the estimate may ask for more steps than are actually necessary. It errs on the safe side, which is what a guarantee should do.

Certifying an approximation found by any method

As Section 12.2 explained, the residual estimate (12.2) certifies a candidate point z found by any method. Notice, though, that the residual by itself is not a bound on the error. It must be multiplied by $1/(1-q)$, and this factor can be large. For example, $T(x) = 0.99x + 0.01$ is a contraction of $\mathbb{R}$ with factor $q = 0.99$, since $|T(x) - T(y)| = 0.99\,|x - y|$, and its fixed point is 1, since $0.99 + 0.01 = 1$. The point $z = 0$ has the small residual $|0 - T(0)| = 0.01$, yet its error is $|0 - 1| = 1$, a hundred times larger. The residual estimate gives the bound $0.01/(1 - 0.99) = 1$, which is exactly the true error. A small residual guarantees a small error only when q is not too close to 1.

Using the length of the latest step

For an iterate x_n with $n \ge 1$, there is a third bound, which uses the step just taken:

$$d(x_n, x_*) \le \frac{q}{1-q}\, d(x_n, x_{n-1}).$$

To prove it, note that the residual of x_n is $d(x_n, Tx_n) = d(x_n, x_{n+1})$. Since $x_n = Tx_{n-1}$ and $x_{n+1} = Tx_n$, the contraction inequality gives

$$d(x_n, x_{n+1}) = d(Tx_{n-1}, Tx_n) \le q\, d(x_{n-1}, x_n) = q\, d(x_n, x_{n-1}).$$

Now the residual estimate with $z = x_n$ gives $d(x_n, x_*) \le d(x_n, x_{n+1})/(1-q) \le q\, d(x_n, x_{n-1})/(1-q)$.

This is called an *a posteriori estimate*, from the Latin for "from what comes after," because it can be evaluated only after the step from x_{n-1} to x_n has been computed. It uses the step length actually observed rather than a prediction made at the start, and it is never worse than the a priori estimate. Indeed, Step 1 of the proof gives $d(x_n, x_{n-1}) \le q^{n-1}D$, so

$$\frac{q}{1-q}\, d(x_n, x_{n-1}) \le \frac{q \cdot q^{n-1}D}{1-q} = \frac{q^n}{1-q}\, D,$$

and the right side is the a priori bound. When the observed steps shrink faster than the worst case $q^{n-1}D$ allows, the a posteriori estimate is much better. Section 12.5 compares the estimates on an example.

Pause and decide. For $T(x) = (x+1)/2$ and $x_0 = 0$, compute the a posteriori bound for x_3 and compare it with the true error $|x_3 - 1|$.

Check. The iterates are $x_2 = 3/4$ and $x_3 = 7/8$, so the latest step has length $7/8 - 3/4 = 1/8$. With $q = 1/2$ we have $q/(1-q) = 1$, so the bound is $|x_3 - 1| \le 1/8$. The true error is $1 - 7/8 = 1/8$, so the bound is attained, just as the a priori bound $2^{-3} = 1/8$ is for this map.

12.5 A quadratic example, worked by hand

In the example $T(x) = (x+1)/2$ we knew the fixed point from the start. In the next example we do not, and the theorem tells us about the fixed point before we have found it. Let

$$T(x) = \frac{x^2 + 1}{3} \qquad \text{on } X = [0, 1],$$

with the usual distance $|x - y|$. Before using the theorem, we check each hypothesis.

- *The space.* The interval $[0, 1]$ is nonempty, and Section 11.6 proved that it is complete.
- *The self-map condition.* If $0 \le x \le 1$, then $0 \le x^2 \le 1$, so $1 \le x^2+1 \le 2$ and $1/3 \le T(x) \le 2/3$. In particular $T(x) \in [0, 1]$, so T maps $[0, 1]$ into itself.
- *The contraction inequality.* For $x, y \in [0, 1]$, factor the difference of two squares, $x^2 - y^2 = (x - y)(x + y)$, and use $|ab| = |a|\,|b|$:

$$|T(x) - T(y)| = \frac{|x^2 - y^2|}{3} = \frac{|x - y|\,|x + y|}{3} \le \frac{2}{3}|x - y|.$$

  The last step uses $0 \le x + y \le 2$, which holds because both x and y lie between 0 and 1. So $q = 2/3$ is a contraction factor.

By Theorem 12.2, T has exactly one fixed point x_* in $[0, 1]$, and the iterates from every starting point in $[0, 1]$ converge to it. We now find out how close a few iterates are, without solving any equation.

Iterates and a certificate

Start from $x_0 = 0$. We compute with exact fractions; in particular, when we square an iterate we square the fraction itself, not a rounded decimal.

$$x_1 = \frac{0^2+1}{3} = \frac{1}{3},$$
$$x_2 = \frac{(1/3)^2+1}{3} = \frac{1/9+9/9}{3} = \frac{10/9}{3} = \frac{10}{27},$$
$$x_3 = \frac{(10/27)^2+1}{3} = \frac{100/729+729/729}{3} = \frac{829/729}{3} = \frac{829}{2187}.$$

Take $z = x_2$. Then $Tz = x_3$, so the residual of z is $|x_2 - x_3|$. To subtract, write x_2 over the denominator $2187 = 27 \cdot 81$, namely $x_2 = 810/2187$. Then

$$d(z, Tz) = \left|\frac{810}{2187} - \frac{829}{2187}\right| = \frac{19}{2187}.$$

Since $1/(1-q) = 1/(1-2/3) = 3$, the residual estimate (12.2) gives the certificate

$$d(x_2, x_*) \le 3 \cdot \frac{19}{2187} = \frac{19}{729}.$$

As a decimal, $19/729 = 0.02606\ldots$. When a certificate is rounded to a decimal, it must be rounded up, never down; otherwise it would no longer be a guarantee. Rounded up, the certificate says that the point $x_2 = 10/27 \approx 0.370$ lies within 0.027 of the fixed point, although we do not yet know where the fixed point is.

The other two estimates of Section 12.4 give weaker bounds for the same point. The a priori estimate, with $D = d(x_1, x_0) = 1/3$ and $n = 2$, gives

$$d(x_2, x_*) \le \frac{(2/3)^2}{1-2/3} \cdot \frac{1}{3} = \frac{4}{9} = 0.444\ldots,$$

because it uses only the first step. The a posteriori estimate uses the step from $x_1 = 9/27$ to $x_2 = 10/27$ and gives $d(x_2, x_*) \le \frac{2/3}{1/3} \cdot \frac{1}{27} = \frac{2}{27} = 0.0740\ldots$. The residual estimate is the sharpest of the three here, because it uses the extra information contained in x_3.

Checking against the exact solution

In this example we can also find x_* exactly, which lets us see how good the certificate is. The fixed-point equation $(x^2+1)/3 = x$ becomes $x^2 + 1 = 3x$, that is, $x^2 - 3x + 1 = 0$. Multiply by 4 and add 5 to both sides to complete the square:

$$4x^2 - 12x + 9 = 5, \qquad \text{that is,} \qquad (2x-3)^2 = 5.$$

If $a^2 = 5$, then $(a-\sqrt{5})(a+\sqrt{5}) = a^2 - 5 = 0$, so $a = \sqrt{5}$ or $a = -\sqrt{5}$. Hence $2x - 3 = \pm\sqrt{5}$, and $x = (3+\sqrt{5})/2$ or $x = (3-\sqrt{5})/2$. Since $4 < 5 < 9$, we have $2 < \sqrt{5} < 3$. So $(3+\sqrt{5})/2$ is larger

than $5/2$ and lies outside $[0, 1]$, while $3 - \sqrt{5}$ lies between 0 and 1, so $(3 - \sqrt{5})/2$ lies between 0 and $1/2$. The fixed point in $[0, 1]$ is therefore

$$x_* = \frac{3 - \sqrt{5}}{2} \approx 0.382.$$

The true error of x_2 is $x_* - 10/27 \approx 0.0116$, comfortably below the certificate $19/729 = 0.02606\ldots$, just as the theorem promised.

On the whole real line, the same formula has two fixed points, $(3 \pm \sqrt{5})/2$. This does not contradict uniqueness, because the theorem was applied on $[0, 1]$ only. On $\mathbb{R}$ the map is not a contraction: the argument of Step 5 uses only the contraction inequality, and it shows that a contraction never has two different fixed points. So the choice of the space X matters: the uniqueness in Theorem 12.2 is a statement about fixed points that lie in X.

12.6 Remove the hypotheses one at a time

Theorem 12.2 has four hypotheses: the space X is nonempty, X is complete, T maps X into itself, and T satisfies the contraction inequality with a factor $q < 1$. The proof used each of them at a definite place. A good way to understand why a hypothesis is there is to remove it, keep all the others, and look for an example in which the conclusion fails. This section does that for each hypothesis in turn.

Without completeness

Let $T(x) = x/2$ on the open interval $X = (0, 1)$. This T is a self-map, because $0 < x < 1$ gives $0 < x/2 < 1/2$, and it is a contraction with factor $1/2$, because $|x/2 - y/2| = |x - y|/2$. But it has no fixed point in X: the equation $x/2 = x$ forces $x = 0$, and 0 does not belong to $(0, 1)$.

The proof breaks down at Step 3. Starting from $x_0 = 1/2$, the iterates are $x_n = 2^{-(n+1)}$, that is, $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \ldots$. Steps 1 and 2 go through, and this is exactly the Cauchy sequence that Section 11.6 used to show that $(0, 1)$ is not complete. Its only possible limit is 0, which is missing from the space, so there is no point x_* for Steps 4–6 to work with.

Without a factor below one

Suppose we allow $q = 1$, so that the inequality only says $d(Tx, Ty) \le d(x, y)$: the map never increases distances. On $\mathbb{R}$, the identity map $T(x) = x$ satisfies this with equality, and every real number is a fixed point, so uniqueness fails. The map $T(x) = x+1$ also satisfies $|T(x)-T(y)| = |x-y|$, but the equation $x + 1 = x$ has no solution, so existence fails. So the value $q = 1$ does not merely make convergence slower; it can destroy existence or uniqueness. In the proof, Steps 2, 5 and 6 all need $1 - q$ to be positive: Step 2 relies on the geometric tail bound $q^n/(1 - q)$, and Steps 5 and 6 divide by $1 - q$, which is now zero. In Step 5, for instance, the inequality $(1 - q)\, d(x_*, y_*) \le 0$ becomes $0 \le 0$ and says nothing.

Even a map that brings every pair of different points strictly closer together can fail, if the shrinking gets weaker and weaker. This is the example promised in Section 12.1. Let $X = [1, \infty)$, the set of real numbers $x \geq 1$, and let

$$T(x) = x + \frac{1}{x}.$$

For $x \geq 1$ we have $T(x) > x \geq 1$, so T maps X into itself. The set $[1, \infty)$ is complete; Exercise 12.4 asks you to check this by adapting the argument given for $[0, 1]$ in Section 11.6. To compare two outputs, use $\frac{1}{x} - \frac{1}{y} = \frac{y-x}{xy} = -\frac{x-y}{xy}$:

$$T(x) - T(y) = (x - y) + \left(\frac{1}{x} - \frac{1}{y}\right) = (x - y)\left(1 - \frac{1}{xy}\right).$$

For $x, y \geq 1$ with $x \neq y$, at least one of the two numbers is larger than 1, so $xy > 1$ and the factor $1 - 1/(xy)$ lies strictly between 0 and 1. Hence $|T(x) - T(y)| < |x - y|$: every pair of different points moves strictly closer together. But the factor depends on the pair, and for large points it is close to 1. For $x = 11$ and $y = 10$, for instance, it is $1 - 1/110 = 109/110$, so this pair moves less than one percent closer.

In fact no single factor $q < 1$ works for all pairs. Let q be any number with $0 \leq q < 1$, and use the Archimedean property (Section 2.3) to choose an integer $k \geq 1$ with $k > 1/(1-q)$. For $x = k+1$ and $y = k$ the ratio $|T(x) - T(y)|/|x - y|$ is $1 - 1/(k(k+1))$. Since $k(k+1) = k^2 + k > k > 1/(1-q)$, taking reciprocals gives $1/(k(k+1)) < 1 - q$, so the ratio is larger than q. In other words, $|T(k+1) - T(k)| > q\,|(k+1) - k|$, and q is not a contraction factor.

Finally, T has no fixed point: $x + 1/x = x$ would require $1/x = 0$. Starting from $x_0 = 1$, the iterates 1, 2, $\frac{5}{2}$, $\frac{29}{10}$, ... keep increasing. Their squares grow by more than 2 at each step, because $(x + 1/x)^2 = x^2 + 2 + 1/x^2$. Since $x_0^2 = 1$, induction gives $x_n^2 \geq 1 + 2n$ for every n, so the iterates eventually pass any number we name: they grow without bound. The single steps $x_{n+1} - x_n = 1/x_n$ do get smaller and smaller, but, as with the harmonic sums of Section 11.5, small steps are not enough.

Without the self-map condition

Let $T(x) = x/2 + 1$, used only for inputs in $[0, 1]$. The interval $[0, 1]$ is nonempty and complete, and the contraction inequality holds with factor $1/2$, since $|T(x) - T(y)| = |x - y|/2$. But $T(1) = 3/2$ lies outside $[0, 1]$, so T is not a map from $[0, 1]$ to itself. The only real solution of $x/2 + 1 = x$ is $x = 2$, which lies outside $[0, 1]$, so T has no fixed point in the interval. The proof breaks down already in Step 1. Starting from $x_0 = 0$, we get $x_1 = 1$ and then $x_2 = 3/2$, which has left the space. Since T is used only for inputs in $[0, 1]$, the next iterate $T(x_2)$ is not even defined. If we ignore the interval and apply the formula on all of $\mathbb{R}$, where it is a contraction with factor $1/2$, Theorem 12.2 says that the iterates approach 2, a point outside $[0, 1]$.

Without nonemptiness

The last hypothesis is needed for a simpler reason. Suppose X is the empty set, so that there are no points at all. There is still a map $T : X \to X$: it has no inputs, so there is nothing for it to assign.

The requirements of a metric, of completeness and of the contraction inequality all hold vacuously, in the sense of Section 2.5, because there are no points to check them for. But there is no fixed point either, since there is no point. The proof uses nonemptiness only once, to choose the starting point x_0 in Step 1.

Summary

The table records where the proof uses each hypothesis and an example showing that it cannot be dropped.

Hypothesis	Used in the proof	Examples without it
X is nonempty	Step 1, to choose x_0	the empty set
X is complete	Step 3	$x \mapsto x/2$ on $(0, 1)$
T maps X into X	Step 1, to form the iterates	$x \mapsto x/2 + 1$ on $[0, 1]$
factor $q < 1$	Steps 2, 5 and 6	$x \mapsto x$ and $x \mapsto x + 1$ on $\mathbb{R}$
		$x \mapsto x + 1/x$ on $[1, \infty)$

Each example keeps three of the four hypotheses and breaks the remaining one, and in each of them the conclusion of the theorem fails. So none of the hypotheses can be dropped.

Exercises

Exercises 12.1–12.4 are core exercises, and Exercises 12.5 and 12.6 are transfer exercises. Complete solutions are in Appendix B, under "Week 12."

Exercise 12.1. Let $T(x) = (3x + 1)/4$ on $\mathbb{R}$. Find the fixed point of T and the smallest contraction factor of T. Then find a formula for the iterates x_n starting from $x_0 = 0$, and prove it by induction.

Exercise 12.2. Let T be a contraction with factor $q = 3/4$ of a nonempty complete metric space, with fixed point x_*, and let z be a point with $d(z, Tz) \leq 0.002$. What upper bound on $d(z, x_*)$ does Theorem 12.2 guarantee? Is the residual bound 0.002 itself an upper bound on $d(z, x_*)$ in every such case? Give a reason or an example.

Exercise 12.3. Let $T(x) = (x^2 + 1)/3$. Show that T maps the interval $[1/3, 2/3]$ into itself and that $4/9$ is a contraction factor of T on this interval. Starting from $x_0 = 0$, explain why every iterate from x_1 on lies in $[1/3, 2/3]$. Use the smaller factor to improve the certificate $d(x_2, x_*) \leq 19/729$ obtained in Section 12.5, and explain why the improvement is justified.

Exercise 12.4. Section 12.6 gives counterexamples in three groups: without completeness, without a factor below one, and without the self-map condition. For each example in these three groups, prove that it satisfies every hypothesis of Theorem 12.2 except the one named in the heading of its group. For $T(x) = x + 1/x$, this includes proving that $[1, \infty)$ is complete.

Exercise 12.5. Let a and b be real numbers with $|a| < 1$, and let $T(x) = ax + b$ on $\mathbb{R}$. Find the fixed point x_* of T, and prove by induction that for every starting point x_0 the iterates satisfy

$x_n - x_* = a^n(x_0 - x_*)$ for every $n \in \mathbb{N}$. How do the iterates move when $a < 0$, compared with when $0 < a < 1$?

Exercise 12.6. Let T and S be two self-maps of the same nonempty complete metric space (X, d), both contractions with factor q, where $0 \le q < 1$. Write x_T and x_S for their fixed points. Suppose there is a number $\eta \ge 0$ (the Greek letter eta) with $d(Tx, Sx) \le \eta$ for every $x \in X$. Prove that $d(x_T, x_S) \le \eta/(1 - q)$.

CHAPTER 13

Averaging, randomness, and existence

> **This week**
>
> Treat finite probability as weighted counting: list the outcomes, give each one a weight, and add. Use averages and the union bound to prove that an object exists, without mistaking an average for a guarantee about every outcome. Turn one such existence proof into a step-by-step construction, and bound how often a nonnegative quantity can be much larger than its average.

13.1 A finite probability model

When we toss a coin or pick a student at random, we cannot predict the result. Probability does not try to predict it. Instead, it lists everything that could happen, gives each possibility a weight, and then calculates. In this chapter the list of possibilities is always finite, so every calculation is a finite sum.

Definition 13.1 (Finite probability space)**.** A *finite probability space* consists of a finite set Ω, whose members are called *outcomes*, together with a number $p(\omega)$ for each outcome $\omega \in \Omega$, called the *probability* of ω, such that

$$p(\omega) \geq 0 \quad \text{for every } \omega \in \Omega, \qquad \text{and} \qquad \sum_{\omega \in \Omega} p(\omega) = 1.$$

The set Ω is called the *sample space*. An *event* is a subset $A \subseteq \Omega$, and the *probability* of the event A is

$$\mathbb{P}(A) = \sum_{\omega \in A} p(\omega).$$

The symbol Ω is the Greek capital letter omega, and ω is the lowercase omega. They are simply the traditional names for the sample space and for one of its outcomes. As in Section 7.2, the notation $\sum_{\omega \in A}$ means that we add one term for each member ω of A. Since the probabilities of all the outcomes add up to 1, while a sum with no terms is 0 (Section 5.1), the sample space cannot be empty.

Think of the experiment as producing exactly one outcome. An event A *occurs* when that outcome belongs to A. Two events are extreme cases. The whole sample space is an event that always occurs, and $\mathbb{P}(\Omega) = 1$. The empty set is an event that never occurs, and $\mathbb{P}(\varnothing) = 0$, again because a sum with no terms is 0.

In the simplest models, called *uniform*, every outcome has the same probability. Since there are $|\Omega|$ outcomes and their probabilities add up to 1, each outcome then has probability $1/|\Omega|$, and for

every event A

$$\mathbb{P}(A) = \frac{|A|}{|\Omega|},$$

because the sum defining $\mathbb{P}(A)$ has $|A|$ terms, each equal to $1/|\Omega|$. In a uniform model, finding a probability is a counting problem: count the outcomes in the event and divide by the number of all outcomes. When we say that a member of a finite set is chosen *uniformly at random*, we mean that this set is the sample space and that the model is uniform. For example, if one student is chosen uniformly at random from a class of 30, each student is chosen with probability $1/30$.

Two coins

Toss two fair coins, a first one and a second one, and record each result as 0 or 1 (say 1 for heads and 0 for tails). Write an outcome as two digits: the result of the first coin followed by the result of the second. The sample space is

$$\Omega = \{00,\ 01,\ 10,\ 11\}.$$

Because the coins are fair, we use the uniform model, in which each outcome has probability $1/4$. The event "the two results differ" is $\{01, 10\}$, so its probability is $2/4 = 1/2$. The event "at least one coin shows 1" is $\{01, 10, 11\}$, with probability $3/4$. We will use this example throughout the chapter.

Which outcomes are equally likely?

The outcomes 01 and 10 are different. In 01 the first coin shows 0 and the second shows 1; in 10 it is the other way round. It is tempting to merge them into a single description, "one of each," and to argue that the number of coins showing 1 is 0, 1, or 2, so that each of these has probability $1/3$. This is a mistake. The description "one of each" covers two of the four equally likely outcomes, so its probability is $2/4 = 1/2$, while "two 0s" and "two 1s" have probability $1/4$ each.

We may still use the three descriptions as outcomes, provided we give them the right weights. The sample space $\{0, 1, 2\}$, which records only the number of coins showing 1, with

$$p(0) = \tfrac{1}{4}, \qquad p(1) = \tfrac{1}{2}, \qquad p(2) = \tfrac{1}{4},$$

describes the same two tosses. It is a finite probability space that is not uniform. Giving each of 0, 1, 2 the probability $1/3$ also produces a finite probability space, but it describes a different experiment, not two tosses of fair coins.

A probability model is a piece of mathematics: we choose the outcomes and their probabilities, and everything else follows by calculation. Whether a model fits a real coin, a real survey, or a real game is a separate question, and no calculation inside the model can answer it.

Disjoint events and complements

Recall from Section 3.1 that two sets are *disjoint* when they have no member in common. If two events A and B are disjoint, then every outcome of $A \cup B$ lies in exactly one of them. So the sum

defining $\mathbb{P}(A \cup B)$ splits into a sum over A and a sum over B, and

$$\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B).$$

With two coins, for example, the events $\{00\}$ and $\{01, 10\}$ are disjoint, and their union $\{00, 01, 10\}$, the event that at most one coin shows 1, has probability $1/4 + 1/2 = 3/4$.

The *complement* of an event A is the event $\Omega \setminus A$, consisting of the outcomes that are not in A. It occurs exactly when A does not. The events A and $\Omega \setminus A$ are disjoint and their union is Ω, so the rule just proved gives

$$\mathbb{P}(A) + \mathbb{P}(\Omega \setminus A) = \mathbb{P}(\Omega) = 1.$$

In the two-coin example, the complement of "at least one coin shows 1" is $\{00\}$, and indeed $3/4 + 1/4 = 1$.

Consequently, if $\mathbb{P}(A) < 1$, then $\mathbb{P}(\Omega \setminus A) = 1 - \mathbb{P}(A) > 0$. An event of positive probability contains at least one outcome ω with $p(\omega) > 0$: if it had no outcomes, or if all of its outcomes had probability 0, its probability would be a sum of zeros, which is 0. Putting these two facts together gives a small but important rule: *if* $\mathbb{P}(A) < 1$*, then some outcome of positive probability lies outside* A. Section 13.3 uses this rule to show that some outcome avoids several unwanted events at once.

Independent fair labels

Several experiments in this chapter give one of two labels, 0 or 1, to each of n objects, as if a fair coin were tossed separately for each object. An outcome is then a list of n labels, one for each object. Each object has two possible labels, so there are $2 \cdot 2 \cdots 2 = 2^n$ such lists; this is the same reasoning that counted the groups of students in Section 9.1. We give every list the same probability $1/2^n$. The two-coin example is the case $n = 2$.

Now pick some k of the objects, and decide in advance which label each of them should get. A list in which these k objects carry the chosen labels may have any labels at the other $n - k$ objects, so there are 2^{n-k} such lists. Hence the probability that the k objects carry the chosen labels is

$$\frac{2^{n-k}}{2^n} = \frac{1}{2^k} = \underbrace{\frac{1}{2} \cdot \frac{1}{2} \cdots \frac{1}{2}}_{k \text{ factors}}.$$

For $k = 1$, this says that each object has label 0 with probability $1/2$ and label 1 with probability $1/2$. For larger k, it says that the probabilities for different objects multiply. For example, take $n = 3$ objects, and ask for the first object to have label 0 and the third to have label 1. The lists that do this are 001 and 011, since the second label is free. There are $2^{3-2} = 2$ of them, so the probability is $2/8 = 1/4 = \frac{1}{2} \cdot \frac{1}{2}$. This is what we mean when we say that the labels are chosen *independently and fairly*.

In general, two events A and B are called *independent* when $\mathbb{P}(A \cap B) = \mathbb{P}(A)\,\mathbb{P}(B)$. With two coins, for example, let $A = \{10, 11\}$ be the event that the first coin shows 1, and let $B = \{00, 10\}$ be the event that the second coin shows 0. Each has probability $1/2$. Their intersection is $A \cap B = \{10\}$, so $\mathbb{P}(A \cap B) = 1/4 = (1/2)(1/2)$, and the two events are independent.

13.2 Expectation is a weighted average

Often we care less about the outcome itself than about a number computed from it: how many coins show 1, the score of the student who was picked, how many edges of a graph have some property.

Definition 13.2 (Random variable and expectation)**.** A *random variable* on a finite probability space is a function $X : \Omega \to \mathbb{R}$. Its *expectation*, or *expected value*, is the number

$$\mathbb{E}\, X = \sum_{\omega \in \Omega} p(\omega) X(\omega).$$

Despite its name, a random variable is an ordinary function, in the sense of Chapter 3. The word "random" refers to the way its input is chosen. Once the outcome ω is known, $X(\omega)$ is an ordinary number. The expectation is a *weighted average* of these numbers: each value is multiplied by the probability of its outcome, and the results are added. The probabilities act as *weights*: they are nonnegative and add up to 1, and a more likely outcome counts for more. A course grade made up of 60% of the exam score and 40% of the homework score is a weighted average of the same kind, with weights 0.6 and 0.4; an exam score of 80 and a homework score of 90 give the grade $0.6 \cdot 80 + 0.4 \cdot 90 = 84$. In a uniform model every weight is $1/|\Omega|$, so

$$\mathbb{E}\, X = \frac{1}{|\Omega|} \sum_{\omega \in \Omega} X(\omega),$$

which is the ordinary average of Section 7.3, taken over all the outcomes. From now on we often write $\sum_\omega$ as short for $\sum_{\omega \in \Omega}$, the sum over all outcomes. A random variable that takes the same value c at every outcome is called *constant*, and its expectation is $\sum_\omega p(\omega) c = c \sum_\omega p(\omega) = c$.

For the two-coin experiment, let X count how many coins show 1. Its values are $X(00) = 0$, $X(01) = 1$, $X(10) = 1$, and $X(11) = 2$, so

$$\mathbb{E}\, X = \tfrac{1}{4} \cdot 0 + \tfrac{1}{4} \cdot 1 + \tfrac{1}{4} \cdot 1 + \tfrac{1}{4} \cdot 2 = 1.$$

The two middle terms have the same value of X, and we could have combined them into the single term $\frac{1}{2} \cdot 1$. Combining terms in this way works in general. For a number x, write $\mathbb{P}(X = x)$ for the probability of the event $\{\omega \in \Omega : X(\omega) = x\}$. If the different values of X are $x_1, \ldots, x_k$, group the terms of the sum defining $\mathbb{E}\, X$ according to the value of X. The group for the value x_i is the sum of the terms $p(\omega) x_i$ over the outcomes with $X(\omega) = x_i$. Taking out the common factor x_i leaves the sum of the probabilities of these outcomes, so the group equals $x_i\, \mathbb{P}(X = x_i)$. Therefore

$$\mathbb{E}\, X = \sum_{i=1}^{k} x_i\ \mathbb{P}(X = x_i).$$

For the two coins this reads $\mathbb{E}\, X = 0 \cdot \frac{1}{4} + 1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{4} = 1$. This is also the calculation we would make in the non-uniform space $\{0, 1, 2\}$ of Section 13.1, which records only the number of coins showing 1 and describes the same tosses. Because of this formula, a random variable is often described just by listing its values and their probabilities, as in "a random variable that equals 0 or 2, each with probability $1/2$." Any finite probability space that produces these values with these probabilities gives the same expectation.

In the two-coin example, the expectation 1 happens to be one of the values of X. It need not be. Toss one fair coin, and let Z be 0 if the coin shows 0 and 2 if it shows 1. Then $\mathbb{E}\,Z = \frac{1}{2}\cdot 0 + \frac{1}{2}\cdot 2 = 1$, but Z never equals 1. The expectation summarizes all the values; it does not predict the value you will see.

Linearity

Random variables on the same space can be added, and multiplied by numbers, one outcome at a time. The sum $X + Y$ is the random variable whose value at ω is $X(\omega) + Y(\omega)$; both X and Y are evaluated at the same outcome. For a real number c, the random variable cX has the value $cX(\omega)$ at ω.

Proposition 13.3 (Linearity of expectation)**.** *Let X and Y be random variables on the same finite probability space, and let c be a real number. Then*

$$\mathbb{E}(X+Y) = \mathbb{E}\,X + \mathbb{E}\,Y \qquad \textit{and} \qquad \mathbb{E}(cX) = c\ \mathbb{E}\,X.$$

Proof. Multiply out each term, and then split the finite sum into two sums:

$$\begin{aligned}\mathbb{E}(X+Y) &= \sum_{\omega} p(\omega)\big(X(\omega)+Y(\omega)\big)\\ &= \sum_{\omega} \big(p(\omega)X(\omega) + p(\omega)Y(\omega)\big)\\ &= \sum_{\omega} p(\omega)X(\omega) + \sum_{\omega} p(\omega)Y(\omega) = \mathbb{E}\,X + \mathbb{E}\,Y.\end{aligned}$$

Splitting is allowed because a finite sum may be added up in any order. For the second identity, take the common factor c out of every term: $\mathbb{E}(cX) = \sum_\omega p(\omega)cX(\omega) = c\sum_\omega p(\omega)X(\omega) = c\ \mathbb{E}\,X$. □

The first identity extends to any finite number of random variables $X_1, \dots, X_m$ on the same space:

$$\mathbb{E}(X_1 + \cdots + X_m) = \mathbb{E}\,X_1 + \cdots + \mathbb{E}\,X_m.$$

To see this, write $X_1 + \cdots + X_m$ as $(X_1 + \cdots + X_{m-1}) + X_m$, a sum of two random variables. The proposition gives $\mathbb{E}(X_1 + \cdots + X_m) = \mathbb{E}(X_1 + \cdots + X_{m-1}) + \mathbb{E}\,X_m$. The same step applied to $X_1 + \cdots + X_{m-1}$ splits off $\mathbb{E}\,X_{m-1}$, and so on, until only $\mathbb{E}\,X_1$ is left. Written out carefully, this is a proof by induction on m.

For example, with two coins let X_1 be the result of the first coin and X_2 the result of the second. Each of them equals 1 at two of the four outcomes, so $\mathbb{E}\,X_1 = \mathbb{E}\,X_2 = 1/2$. Their sum $X_1 + X_2$ is the number of coins showing 1, and its expectation is $1/2 + 1/2 = 1$, as we found directly above.

The proof never asked how X and Y are related, and linearity holds even when they are as closely linked as possible. Toss one fair coin, let X be its result, and let $Y = 1 - X$, so that Y is 1 exactly when X is 0. Knowing X tells us Y completely. Still, $X + Y$ is the constant 1, so $\mathbb{E}(X+Y) = 1 = \frac{1}{2} + \frac{1}{2} = \mathbb{E}\,X + \mathbb{E}\,Y$. This is what makes linearity so useful: to find the expected value of a complicated count, we can split the count into simple pieces and add their expectations, without first working out how the pieces affect one another.

Comparing expectations

Proposition 13.4 (Monotonicity of expectation)**.** *Let X and Y be random variables on the same finite probability space. If $X(\omega) \le Y(\omega)$ for every outcome ω, then $\mathbb{E}\,X \le \mathbb{E}\,Y$.*

Proof. For each outcome ω, multiply the inequality $X(\omega) \le Y(\omega)$ by $p(\omega)$. Since $p(\omega) \ge 0$, the inequality keeps its direction: $p(\omega)X(\omega) \le p(\omega)Y(\omega)$. Adding these inequalities over all outcomes gives $\mathbb{E}\,X \le \mathbb{E}\,Y$. □

An inequality that holds at every outcome separately, like the hypothesis $X(\omega) \le Y(\omega)$ here, is called a *pointwise* inequality. Monotonicity says that a pointwise inequality survives averaging. Whenever we "take expectations of both sides" of an inequality, this proposition is the justification. The reverse step is not allowed: $\mathbb{E}\,X \le \mathbb{E}\,Y$ does not imply that $X \le Y$ pointwise. In the one-coin example with $Y = 1 - X$ above, $\mathbb{E}\,X = \mathbb{E}\,Y = 1/2$, but when the coin shows 1 we have $X = 1$ and $Y = 0$.

Some outcome is at least the average

The averaging principle (Proposition 7.5) says that among $n \ge 1$ real numbers, at least one is at least their average and at least one is at most their average. An expectation is a weighted average, and the same is true for it.

Proposition 13.5. *Let X be a random variable on a finite probability space. Then some outcome ω with $p(\omega) > 0$ satisfies $X(\omega) \ge \mathbb{E}\,X$, and some outcome ω with $p(\omega) > 0$ satisfies $X(\omega) \le \mathbb{E}\,X$.*

Proof. Suppose, for contradiction, that $X(\omega) < \mathbb{E}\,X$ for every outcome ω with $p(\omega) > 0$. There is at least one such outcome, because the probabilities add up to 1 and so cannot all be 0. Multiply each of these strict inequalities by its positive weight, to get $p(\omega)X(\omega) < p(\omega)\ \mathbb{E}\,X$. For an outcome with $p(\omega) = 0$ we have instead the equality $p(\omega)X(\omega) = 0 = p(\omega)\ \mathbb{E}\,X$. Now add over all outcomes. At least one of the inequalities being added is strict, so the total is strict (if $x < y$ and $x' \le y'$, then $x + x' < y + x' \le y + y'$). This gives

$$\mathbb{E}\,X = \sum_{\omega} p(\omega)X(\omega) < \sum_{\omega} p(\omega)\ \mathbb{E}\,X = \mathbb{E}\,X \cdot \sum_{\omega} p(\omega) = \mathbb{E}\,X,$$

so $\mathbb{E}\,X < \mathbb{E}\,X$, which is impossible. The second claim is proved in the same way, with every inequality reversed. □

The proposition asks for an outcome with $p(\omega) > 0$, so that the outcome it finds is one that can actually occur. An outcome of probability 0 contributes nothing to $\mathbb{E}\,X$, so the expectation carries no information about it.

In the two-coin example, $\mathbb{E}\,X = 1$, and the outcomes 01, 10, and 11 all have $X \ge 1$. In the one-coin example with values 0 and 2, $\mathbb{E}\,Z = 1$, and only the outcome in which the coin shows 1 has $Z \ge 1$. The proposition promises at least one such outcome and nothing more. Like the pigeonhole principle, its proof shows that a good outcome exists without telling us which one it is. Section 13.4 uses the proposition to prove that a certain object exists, and Section 13.5 shows how, in that example, a good outcome can actually be found.

13.3 Indicators convert events into quantities

Events are sets of outcomes, while random variables assign numbers to outcomes. Indicators connect the two. The *indicator* of an event A is the random variable I_A given by

$$I_A(\omega) = \begin{cases} 1 & \text{if } \omega \in A, \\ 0 & \text{if } \omega \notin A. \end{cases}$$

So I_A is 1 when A occurs and 0 when it does not. Its expectation is the probability of A:

$$\mathbb{E}\, I_A = \sum_{\omega \in A} p(\omega) \cdot 1 + \sum_{\omega \notin A} p(\omega) \cdot 0 = \mathbb{P}(A).$$

For two coins, let $A = \{10, 11\}$ be the event that the first coin shows 1. Then I_A is 1 at 10 and 11 and 0 at 00 and 01, so $\mathbb{E}\, I_A = \frac{1}{4} + \frac{1}{4} = \frac{1}{2} = \mathbb{P}(A)$.

Indicators are useful for counting. Let $A_1, \ldots, A_m$ be events. At a given outcome, the sum $I_{A_1} + \cdots + I_{A_m}$ adds 1 for each event that occurs and 0 for each event that does not, so it counts how many of the events occur. By linearity of expectation, and because $\mathbb{E}\, I_{A_j} = \mathbb{P}(A_j)$ for each j,

$$\mathbb{E}\big(I_{A_1} + \cdots + I_{A_m}\big) = \mathbb{P}(A_1) + \cdots + \mathbb{P}(A_m).$$

In words, the expected number of events that occur is the sum of their probabilities, even when the events overlap.

The union bound

The union $A_1 \cup \cdots \cup A_m$ is the event that at least one of the A_j occurs. Its indicator records only *whether* some event occurs, not *how many* do. To compare this indicator with the count $I_{A_1} + \cdots + I_{A_m}$, return to two coins, and let A be the event that the first coin shows 1 and B the event that the second coin shows 1:

outcome	I_A	I_B	$I_A + I_B$	$I_{A \cup B}$
00	0	0	0	0
01	0	1	1	1
10	1	0	1	1
11	1	1	2	1

At every outcome, $I_{A \cup B} \le I_A + I_B$. The inequality is strict at 11, where both events occur. Taking expectations gives $\mathbb{P}(A \cup B) = 3/4$ on the left and $\mathbb{P}(A) + \mathbb{P}(B) = 1/2 + 1/2 = 1$ on the right, and indeed $3/4 \le 1$.

The same comparison works for any number of events. At an outcome where none of the events occurs, both $I_{A_1 \cup \cdots \cup A_m}$ and $I_{A_1} + \cdots + I_{A_m}$ are 0. At an outcome where at least one occurs, the indicator of the union is 1, while the sum is at least 1. So the pointwise inequality

$$I_{A_1 \cup \cdots \cup A_m} \le I_{A_1} + \cdots + I_{A_m}$$

holds at every outcome. Now take expectations of both sides. By monotonicity (Proposition 13.4) the inequality is kept. The left side becomes $\mathbb{P}(A_1 \cup \cdots \cup A_m)$, because the expectation of an indicator is the probability of its event, and by linearity the right side becomes $\mathbb{P}(A_1) + \cdots + \mathbb{P}(A_m)$. This proves the following inequality.

Proposition 13.6 (Union bound)**.** *For any events $A_1, \ldots, A_m$ in a finite probability space,*

$$\mathbb{P}(A_1 \cup \cdots \cup A_m) \leq \sum_{j=1}^{m} \mathbb{P}(A_j).$$

The union bound becomes an existence theorem when we think of $A_1, \ldots, A_m$ as *bad events*, the different ways in which an outcome can fail to be what we want. Suppose that

$$\mathbb{P}(A_1) + \cdots + \mathbb{P}(A_m) < 1.$$

Then the union bound gives $\mathbb{P}(A_1 \cup \cdots \cup A_m) < 1$. In Section 13.1 we saw that if an event has probability less than 1, then some outcome of positive probability lies outside it. So some outcome of positive probability lies outside the union. That outcome belongs to none of the events A_j: it avoids every bad event. An upper bound on probabilities has turned into a proof that a good outcome exists, and once again the proof does not say which outcome it is.

Here is an example. The students of a class have formed 7 committees, each with 4 members, and a student may sit on several committees. Can the class be split into two teams so that every committee has members in both teams? Give each student the label 0 or 1, independently and fairly, and let A_j be the bad event that the four members of committee j all get the same label. By the rule for independent fair labels with $k = 4$, the four members all get the label 0 with probability $1/2^4 = 1/16$, and they all get the label 1 with probability $1/16$. These two events are disjoint, so $\mathbb{P}(A_j) = 1/16 + 1/16 = 1/8$. The probabilities of the seven bad events add up to $7/8$, which is less than 1, so some labeling avoids every bad event. Put the students labeled 0 in one team and the others in the other team. Then no committee lies entirely in one team, so every committee has members in both. We never needed to know who sits on which committee.

Pause and decide. Suppose the probabilities of the bad events add up to exactly 1. Does it follow that some outcome avoids them all?

Check. Not necessarily. Toss one fair coin, and let the bad events be "the coin shows 0" and "the coin shows 1." Each has probability $1/2$, so the sum is 1, but the union of the two events is the whole sample space, and every outcome is bad. On the other hand, with two coins the events "the first coin shows 1" and "the second coin shows 1" also have probabilities adding up to 1, and the outcome 00 avoids both. When the sum is 1, the union bound says only that the union has probability at most 1, which is true of every event, so it decides nothing either way. The argument needs the sum to be strictly less than 1.

13.4 Every graph has a large cut

Suppose the students of a class are to be split between two rooms, and some pairs of students are known to chat a lot. We would like as many chatty pairs as possible to end up in different rooms.

In the language of Chapter 7, the students are the vertices of a graph, the chatty pairs are its edges, and we want to divide the vertices into two groups so that many edges run between the groups.

Definition 13.7 (Cut)**.** Let V be the vertex set of a graph. A *cut* of the graph is a pair of sets L and R with $L \cup R = V$ and $L \cap R = \varnothing$, so that every vertex lies in exactly one of them. An edge *crosses* the cut when one of its endpoints lies in L and the other in R. The *size* of the cut is the number of crossing edges.

The letters L and R suggest "left" and "right." One of the two sets is allowed to be empty. Take the example graph G of Section 7.1, with edges ab, bc, ca, and cd. For the cut $L = \{a, d\}$, $R = \{b, c\}$, the edges ab, ca, and cd cross and bc does not, so the cut has size 3. The cut $L = \{a, b, c, d\}$, $R = \varnothing$ has size 0. No cut of G has size 4. For that, the three edges ab, bc, and ca would all have to cross. But if ab and bc cross, then a and c both lie on the side opposite to b, so they lie on the same side, and ca does not cross.

In a small graph like G, we can find a largest cut by trying every possibility. For large graphs this is hopeless. Each vertex can go to either side, so a graph with 100 vertices has 2^{100} cuts. Since $2^{10} = 1024$ is more than 10^3, the number $2^{100} = (2^{10})^{10}$ is more than $(10^3)^{10} = 10^{30}$. In fact, finding a cut of the largest possible size is a famously hard problem: no fast method is known that works for every graph. The following theorem asks for less, and an averaging argument proves it in a few lines.

Theorem 13.8. *Every graph with m edges has a cut of size at least $m/2$.*

Plan. Put each vertex on the left or on the right by tossing a fair coin for it, independently of the other vertices. Count the crossing edges with indicators, find the expected number of crossing edges by linearity, and then use the fact that some outcome is at least the average.

Proof. Let V be the vertex set, E the edge set, and $n = |V|$. We use the model of independent fair labels from Section 13.1. An outcome is a labeling that gives each vertex the label 0 or 1, and each of the 2^n labelings has probability $1/2^n$. Each labeling determines a cut: L is the set of vertices labeled 0, and R is the set of vertices labeled 1.

Fix an edge $e = uv$, and let A_e be the event that u and v get different labels, that is, that e crosses the cut. The endpoints u and v are two different vertices. So, by the rule for independent fair labels with $k = 2$, each of the four possible label pairs for u and v, namely $(0, 0)$, $(0, 1)$, $(1, 0)$, and $(1, 1)$, has probability $1/4$. Exactly two of these pairs, $(0, 1)$ and $(1, 0)$, consist of different labels. The labelings that give u and v these two pairs form two disjoint events, so $\mathbb{P}(A_e) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$.

Let X be the size of the cut, viewed as a random variable on the labelings. At each labeling, X counts the edges whose endpoints have different labels, so

$$X = \sum_{e \in E} I_{A_e}.$$

By linearity of expectation and $\mathbb{E}\, I_{A_e} = \mathbb{P}(A_e)$,

$$\mathbb{E}\, X = \sum_{e \in E} \mathbb{P}(A_e) = m \cdot \frac{1}{2} = \frac{m}{2}.$$

By Proposition 13.5, some labeling satisfies $X \geq \mathbb{E}\, X = m/2$. The cut determined by that labeling has at least $m/2$ crossing edges. □

For a graph with no edges, the theorem holds for a simple reason: then $m/2 = 0$, and every cut has size at least 0.

The expected size of the cut is $m/2$, but this does not mean that every labeling gives a cut of size about $m/2$. Labeling every vertex 0 gives a cut of size 0, and that labeling is one of the possible outcomes. The argument guarantees only that *some* labeling does at least as well as the average. Confusing these two statements is a common mistake in probabilistic arguments; watch for it whenever you, or an AI assistant, summarize such a proof.

The proof used the independence of the labels only once, to find $\mathbb{P}(A_e) = 1/2$ for a single edge. It never asked how the crossing events of different edges are related, and they can be closely linked. Consider the *triangle*, the graph with three vertices a, b, c and the three edges ab, bc, ca. As we saw for the example graph G, these three edges can never all cross. So the event that all three cross has probability 0, although each edge crosses with probability $1/2$. If each edge crossed or not according to a separate fair coin of its own, all three would cross with probability $(1/2)^3 = 1/8$. Linearity of expectation needs none of this information: the expected number of crossing edges is still $1/2 + 1/2 + 1/2 = 3/2$.

The size of a cut is an integer. So when m is odd, a cut of size at least $m/2$ actually has size at least $(m + 1)/2$, the smallest integer that is at least $m/2$. For the triangle, $m = 3$, and the theorem gives a cut of size at least 2. Putting one vertex on one side and the other two on the other side gives exactly 2, and we have just seen that no cut of the triangle has size 3.

13.5 Turning the average argument into a construction

The proof of Theorem 13.8 shows that a good labeling exists, but it does not tell us how to find one, and trying all 2^n labelings is out of the question for large graphs. Instead, we can build a good labeling one vertex at a time. At each step we choose a side so that a certain score never goes down. The score is the expected final size of the cut, computed as if the remaining vertices were going to be placed by tossing coins.

The score of a partial placement

Suppose some of the vertices have already been placed on the left or on the right, and the others have not been placed yet. Call an edge *settled* when both of its endpoints have been placed, and *unsettled* otherwise. Let C be the number of settled edges that cross (that is, whose endpoints were placed on different sides), and let U be the number of unsettled edges. Define

$$F = C + \frac{U}{2}.$$

Equivalently, F is a sum with one term for each edge: a settled edge that crosses contributes 1, a settled edge that does not cross contributes 0, and an unsettled edge contributes $\frac{1}{2}$.

Here is where this formula comes from. Imagine that each vertex not yet placed were now put on the left or on the right by tossing a fair coin, independently for different vertices. A settled edge already crosses or already does not, whatever the coins show. An unsettled edge would cross

with probability $1/2$. If one endpoint has been placed, the edge crosses exactly when the coin for the other endpoint sends it to the opposite side, which happens with probability $1/2$. If neither endpoint has been placed, the edge crosses exactly when the two coins disagree, which happens with probability $1/2$, as in the proof of Theorem 13.8. By linearity of expectation, the expected size of the final cut would be $C + U/2 = F$.

We will never actually toss any coins. The construction only needs the formula $F = C + U/2$, which we can compute at every stage. Before any vertex is placed, every edge is unsettled, so $C = 0$, $U = m$, and $F = m/2$. After every vertex has been placed, every edge is settled, so $U = 0$ and $F = C$ is the size of the cut we have built. So if we can place the vertices one at a time without ever decreasing F, the final cut will have size at least $m/2$.

Placing one vertex

Let v be the next vertex to be placed. Suppose v has ℓ neighbors that have already been placed on the left and r neighbors that have already been placed on the right. Sort the edges that have v as an endpoint into three kinds, and see how the contribution of each one to F changes if we place v on the left.

- The ℓ edges joining v to placed neighbors on the left were unsettled, so each contributed $1/2$. Afterwards they are settled, with both endpoints on the left, so they do not cross and contribute 0.
- The r edges joining v to placed neighbors on the right also contributed $1/2$ each. Afterwards they are settled and cross, so each contributes 1.
- The edges joining v to vertices that have not been placed remain unsettled, and still contribute $1/2$ each.

Edges that do not have v as an endpoint are not affected at all. So placing v on the left changes F by

$$r \cdot \left(1 - \frac{1}{2}\right) + \ell \cdot \left(0 - \frac{1}{2}\right) = \frac{r - \ell}{2}.$$

In the same way, placing v on the right changes F by $(\ell - r)/2$. These two changes are negatives of each other, so at least one of them is nonnegative. This gives a simple rule: *put v on the side opposite to the majority of its placed neighbors*, that is, on the right if $\ell > r$ and on the left if $r > \ell$; if $\ell = r$, either side will do. With this choice F changes by $|\ell - r|/2 \geq 0$, so it never decreases.

Proposition 13.9. *Place the vertices of a graph with m edges one at a time, in any order, putting each vertex on the side opposite to the majority of its already placed neighbors, and on either side in case of a tie. The resulting cut has size at least $m/2$.*

Proof. Before any vertex is placed, $F = m/2$. By the calculation above, each placement made according to the rule changes F by $|\ell - r|/2 \geq 0$, so F never decreases, and after the last placement $F \geq m/2$. At that point every edge is settled, so $U = 0$ and $F = C$ is the size of the cut. Hence the cut has size at least $m/2$. □

This is a deterministic construction: it makes no random choices, and it always produces a cut of size at least $m/2$. It is also fast. To place v, we only need to look at the $\deg(v)$ neighbors of v,

and by the handshake theorem (Theorem 7.2) these numbers add up to $2m$ over all the vertices. For a large graph this is far less work than checking 2^n labelings. The idea of keeping track of the expected value while the choices are made one at a time is called the *method of conditional expectations*. An expected value computed with the choices made so far held fixed, and only the remaining choices left to chance, is called a *conditional expectation*; here it is simply F.

A full trace on the triangle

Take the triangle with vertices a, b, c and edges ab, bc, ca, and place the vertices in the order a, b, c. Initially nothing is placed, so $C = 0$, $U = 3$, and $F = 3/2$.

Vertex a has no placed neighbors, so $\ell = r = 0$, and either side will do; put a on the left. No edge has both endpoints placed yet, so $C = 0$, $U = 3$, and F stays $3/2$.

Vertex b has one placed neighbor on the left, namely a, and none on the right: $\ell = 1$, $r = 0$. Put b on the right. Edge ab is now settled and crosses, while bc and ca remain unsettled. Thus $C = 1$, $U = 2$, and $F = 1 + 2/2 = 2$. The increase is $(\ell - r)/2 = 1/2$, as predicted.

Vertex c has one placed neighbor on each side, a on the left and b on the right, so $\ell = r = 1$, and either choice changes F by 0. Put c on the left. Now bc crosses and ca does not, so $C = 2$, $U = 0$, and $F = 2$. The final cut is $L = \{a, c\}$, $R = \{b\}$, of size 2, which by Section 13.4 is the largest possible size for a cut of the triangle.

The construction need not find a largest cut

On the triangle the construction found a largest cut, but it does not always do so. Take the graph with vertices a, b, c, d and edges ab, bc, cd, whose three edges form the path a, b, c, d. Place the vertices in the order a, d, b, c, and break ties by choosing the left side. Neither a nor d has a placed neighbor when its turn comes, so both go to the left. Then b has its placed neighbor a on the left, so it goes to the right. Finally c has one placed neighbor on each side, b on the right and d on the left, so it goes to the left. The result is $L = \{a, c, d\}$ and $R = \{b\}$, a cut of size 2, since ab and bc cross and cd does not. The cut $L = \{a, c\}$, $R = \{b, d\}$ is better: all three edges cross it.

13.6 A bound on rare large values

The expectation also limits how often a quantity can be much larger than its average, provided the quantity is never negative. A random variable X is *nonnegative* when $X(\omega) \geq 0$ for every outcome ω. For a real number a, we write $\mathbb{P}(X \geq a)$ for the probability of the event $\{\omega \in \Omega : X(\omega) \geq a\}$.

Proposition 13.10 (Markov's inequality)**.** *Let X be a nonnegative random variable on a finite probability space, and let $a > 0$ be a real number. Then*

$$\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}\, X}{a}.$$

Proof. Split the sum defining the expectation into the outcomes where $X \geq a$ and the rest:

$$\mathbb{E}\, X = \sum_{\omega:\, X(\omega) \geq a} p(\omega) X(\omega) + \sum_{\omega:\, X(\omega) < a} p(\omega) X(\omega).$$

Here $\sum_{\omega:\, X(\omega)\geq a}$ means the sum over the outcomes ω with $X(\omega) \geq a$, that is, over the event $\{\omega \in \Omega : X(\omega) \geq a\}$; the second sum runs over all the other outcomes. In the first sum every value $X(\omega)$ is at least a, and every weight $p(\omega)$ is nonnegative, so $p(\omega)X(\omega) \geq p(\omega)a$ for each term. Hence the first sum is at least

$$\sum_{\omega:\, X(\omega) \geq a} p(\omega)\, a = a\ \mathbb{P}(X \geq a).$$

In the second sum every term $p(\omega)X(\omega)$ is nonnegative, because $p(\omega) \geq 0$ and $X(\omega) \geq 0$, so the second sum is at least 0. Together, $\mathbb{E}\, X \geq a\ \mathbb{P}(X \geq a)$, and dividing by the positive number a gives the claim. □

For example, suppose that the scores on a test are all nonnegative and that the class average is 20. Choose a student uniformly at random, and let X be that student's score. In a uniform model the expectation is the ordinary average, so $\mathbb{E}\, X = 20$, and $\mathbb{P}(X \geq 80)$ is the fraction of the class that scored 80 or more. Markov's inequality says that this fraction is at most $20/80 = 1/4$. You can also see this directly: if more than a quarter of the class had scored at least 80, those students alone would contribute more than $\frac{1}{4} \cdot 80 = 20$ to the average, and since no score is negative, the other students could not bring it back down to 20.

The bound need not be exact. For the number X of coins showing 1 in the two-coin experiment, X is nonnegative and $\mathbb{E}\, X = 1$, so Markov's inequality with $a = 2$ gives $\mathbb{P}(X \geq 2) \leq 1/2$. The true value is smaller: $\mathbb{P}(X \geq 2) = \mathbb{P}(\{11\}) = 1/4$. Exercise 13.6 asks for a case in which the bound is exact.

A convenient way to remember Markov's inequality uses multiples of the average. Suppose $\mathbb{E}\, X > 0$, and take $a = t\ \mathbb{E}\, X$ for a number $t > 0$. Then

$$\mathbb{P}(X \geq t\ \mathbb{E}\, X) \leq \frac{\mathbb{E}\, X}{t\ \mathbb{E}\, X} = \frac{1}{t}.$$

So a nonnegative quantity with a positive average is at least twice its average with probability at most $1/2$, and at least ten times its average with probability at most $1/10$. In the test example, 80 is four times the average, and the bound is $1/4$. This is the precise sense in which large values of a nonnegative quantity are rare.

The inequality needs the hypothesis that X is nonnegative. Without it, large negative values can balance large positive ones in the expectation. A random variable equal to 100 or -100, each with probability $1/2$, has expectation 0, so the right side of Markov's inequality would be $0/100 = 0$; yet $\mathbb{P}(X \geq 100) = 1/2$. In the proof, the hypothesis was used to say that the second sum is at least 0, and for this variable that sum is $\frac{1}{2} \cdot (-100) = -50$.

Habit 8: choose a quantity whose average is easier to find than its best value

When an object with a desired property is hard to construct directly, set up a finite random experiment whose outcomes are candidate objects. Count the features you want, or the ones you want to avoid, with indicators, and look for an average that you can compute exactly; linearity often makes this easy even when the features interact. Then say precisely what you have proved. Three kinds of conclusion are easy to confuse: that at least one suitable outcome exists, that a random outcome is unlikely to be bad, and that a suitable object can be built efficiently. The proof of Theorem 13.8 reached only the first, and the third needed the step-by-step construction of Section 13.5. Conclusions of the second kind come from upper bounds on the probability of a bad outcome, such as the union bound and Markov's inequality, when the bound they give is small.

Exercises

Exercises 13.1–13.4 are core exercises, and Exercises 13.5 and 13.6 are transfer exercises. Complete solutions are in Appendix C, under "Week 13."

Exercise 13.1. A random variable takes the values 0, 2, and 5 with probabilities $1/2$, $1/3$, and $1/6$. Compute its expectation, and name a value of the variable that is at least as large as the expectation.

Exercise 13.2. Label each vertex of a triangle with 0 or 1, and take the cut whose left side consists of the vertices labeled 0. List the possible sizes of this cut, and compute the average cut size over all eight labelings.

Exercise 13.3. Run the construction of Section 13.5 on the graph with vertices $1, 2, 3, 4$ and edges 12, 23, 34, 41 (here 12 stands for the edge $\{1, 2\}$, not the number twelve, and so on). Place the vertices in the order $1, 2, 3, 4$, and break ties by choosing the left side. At each step, record ℓ, r, the side chosen, and the new value of F. How large is the final cut, and how does it compare with the guarantee of Theorem 13.8?

Exercise 13.4. In a finite probability space, four bad events each have probability at most 0.2. Prove that some outcome of positive probability avoids all four. Then suppose instead that each bad event has probability at most 0.25. Does the same conclusion follow? Explain, using examples.

Exercise 13.5. Give an example of a random variable whose expectation is at least 5 but which takes the value 0 at some outcome of positive probability. This shows that a large expectation does not force every value to be large. Then state what the expectation does guarantee about the values of a random variable.

Exercise 13.6. Let X be a nonnegative random variable with $\mathbb{E}\,X = 3$. Use Markov's inequality to bound $\mathbb{P}(X \geq 12)$. Then give an example of such a random variable, on a finite probability space, for which $\mathbb{P}(X \geq 12)$ equals your bound.

CHAPTER 14

Sign sequences: from experiments to a research boundary

This week

Measure how closely a sequence of plus and minus signs matches shifted copies of itself. Then look for sequences that agree in exactly half of their positions with each rotated copy of themselves, and translate this search into a question about matrices whose rows are orthogonal. Prove two conditions that the size of such a matrix must satisfy, compare them with a complete computer search over short sequences, and see why a search can never replace a proof that covers every length. Finally, read a recent research claim about these matrices and say exactly what it adds to what you have proved.

14.1 A sequence with small echoes

Sequences of signs appeared in Section 6.5; in this chapter they are the main characters. A *sign sequence* of length n, where $n \geq 1$, is a list $a = (a_0, a_1, \ldots, a_{n-1})$ in which every entry is 1 or -1. As in Python (Appendix D), we number the positions from 0, so the last entry is a_{n-1}.

Think of the sequence as a pattern of pulses, and of a copy of it delayed by s steps as an echo. Suppose the pattern matches its own echoes only weakly. Then a receiver that compares what it hears with the pattern gets a strong match at the moment the pattern arrives and only weak matches at every other moment, so it can tell exactly when the pattern arrived. For this reason sequences of this kind are used in radar and in digital communication. To measure how well the pattern matches its echo, write the sequence down, write a copy shifted s places to the right underneath it, and look at the positions where the two rows overlap. For $a = (1, 1, 1, -1)$ and $s = 1$:

$$\begin{array}{cccccc} 1 & 1 & 1 & -1 & & \\ & 1 & 1 & 1 & -1 & \end{array}$$

In each overlapping position, multiply the two entries. The entry a_j of the shifted copy stands below the entry a_{j+s} of the original, so the product is $a_j a_{j+s}$. It is 1 when the two entries agree and -1 when they differ. Adding these products gives a single number.

Definition 14.1 (Aperiodic autocorrelation)**.** Let a be a sign sequence of length n. For each shift s with $1 \leq s \leq n-1$, the *aperiodic autocorrelation* of a at shift s is

$$A_s(a) = \sum_{j=0}^{n-1-s} a_j a_{j+s}.$$

When it is clear which sequence we mean, we write A_s instead of $A_s(a)$. Since each product is $+1$ for an agreement and -1 for a disagreement, A_s is the number of overlapping positions where the sequence agrees with its shifted copy, minus the number where it disagrees.

The prefix "auto" means that the sequence is compared with itself. The word "aperiodic" means that nothing wraps around: entries pushed past the end of the list simply drop off, as the last -1 in the picture does. Shifts of n or more are left out because the copy would no longer overlap the original at all. Shift zero is left out for a different reason: there every entry is compared with itself, and the sum is $a_0^2 + a_1^2 + \cdots + a_{n-1}^2 = n$, because every sign has square one. That value is the same for every sequence of length n, so it tells us nothing.

Decode the summation limits before computing

The term $a_j a_{j+s}$ makes sense only when both positions j and $j+s$ belong to the list $0, 1, \ldots, n-1$. The sum starts at $j = 0$. It must stop when the second position reaches the end of the list, that is, when $j + s = n - 1$, or $j = n - 1 - s$. So j takes the values $0, 1, \ldots, n - 1 - s$, and there are $(n - 1 - s) - 0 + 1 = n - s$ terms. This agrees with the picture: shifting by s places leaves an overlap of $n - s$ positions.

For a sequence of length five and shift two, the sum is

$$A_2 = a_0a_2 + a_1a_3 + a_2a_4,$$

with $5 - 2 = 3$ terms. There is no term a_3a_5, because a_5 is not in the list. At the largest shift, $s = n - 1$, a single term a_0a_{n-1} remains. Checking the first term, the last term, and the number of terms in a small case like this is a quick way to catch a wrong summation limit.

Keep the two kinds of subscript apart: the subscript on A_s is a shift, while the subscript on a_j is a position in the list.

A first example, and the sequences we are looking for

For $a = (1, 1, 1, -1)$ there are three shifts to check:

$$\begin{aligned} A_1 &= a_0a_1 + a_1a_2 + a_2a_3 = (1)(1) + (1)(1) + (1)(-1) = 1, \\ A_2 &= a_0a_2 + a_1a_3 = (1)(1) + (1)(-1) = 0, \\ A_3 &= a_0a_3 = (1)(-1) = -1. \end{aligned}$$

At shift one, for instance, the picture above shows two agreements and one disagreement, and $2 - 1 = 1$.

Definition 14.2 (Barker sequence)**.** A *Barker sequence* is a sign sequence a of length n with $|A_s(a)| \le 1$ for every shift s with $1 \le s \le n - 1$.

So a Barker sequence matches each of its echoes very weakly; the parity argument below shows that this is as weakly as possible. The sequence $(1, 1, 1, -1)$ is a Barker sequence, since its correlations are 1, 0, and -1. These sequences are named after R. H. Barker, who studied them in 1953 as synchronization patterns for digital communication. The definition also makes

sense for $n = 1$. A sequence of length one has no shifts to check, so the condition holds vacuously (Section 2.5), and both (1) and (-1) are Barker sequences.

Why does the definition ask for "at most one" rather than "zero"? Because A_s often cannot be zero. It is a sum of $n - s$ terms, each equal to 1 or -1. If m of these terms equal -1, then the other $n - s - m$ terms equal 1, and

$$A_s = (n - s - m) - m = (n - s) - 2m.$$

Subtracting the even number $2m$ does not change whether a number is even or odd. So A_s is even when $n - s$ is even, and odd when $n - s$ is odd; in particular, A_s can be zero only when $n - s$ is even. For a Barker sequence this fixes every correlation up to its sign. The only even integer with absolute value at most one is 0, and the only odd ones are 1 and -1. Hence a Barker sequence has $A_s = 0$ whenever $n - s$ is even, and $A_s = 1$ or $A_s = -1$ whenever $n - s$ is odd. In the example of length four, $n - s$ equals 3, 2, 1 for $s = 1, 2, 3$, and the correlations 1, 0, -1 follow exactly this pattern.

Pause and decide. Is $(1, 1, -1, -1)$ a Barker sequence?

Check. No. At shift two, $n - s = 2$ is even, so a Barker sequence would need $A_2 = 0$. But $A_2 = a_0a_2 + a_1a_3 = (1)(-1) + (1)(-1) = -2$. The other two correlations, $A_1 = 1 - 1 + 1 = 1$ and $A_3 = (1)(-1) = -1$, are fine, but a single bad shift is enough to fail the definition.

14.2 Periodic correlation is a different quantity

There is a second natural way to compare a sequence with a shifted copy of itself. Instead of letting entries fall off the end, write the n entries around a circle, like the hours on a clock face, and rotate the copy by s places. Now every entry has a partner: a position that runs past the end of the list comes back in at the start. In symbols, the position $j + s$ is replaced by its remainder on division by n, which Section 5.5 writes as $(j + s) \bmod n$. For $a = (1, 1, 1, -1)$ and $s = 1$, the picture of Section 14.1 becomes

$$\begin{array}{cccc} 1 & 1 & 1 & -1 \\ -1 & 1 & 1 & 1 \end{array}$$

The -1 that dropped off the end now comes back in at the front, under the first entry, so all four columns overlap and there are four products to add.

Definition 14.3 (Periodic autocorrelation)**.** Let a be a sign sequence of length n. For each shift s with $0 \le s \le n - 1$, the *periodic autocorrelation* of a at shift s is

$$C_s(a) = \sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n}.$$

Again we write C_s when the sequence is clear. Unlike A_s, this sum always has n terms, one for each position j. At shift zero every entry is paired with itself, so $C_0 = n$. This value carries no information about the sequence, but we keep shift zero in the definition because the value $C_0 = n$ is needed in Section 14.4.

For $a = (1, 1, 1, -1)$ and $s = 1$, the positions $j + 1$ of the second factors are $1, 2, 3, 4$, and the last of these wraps around to $4 \bmod 4 = 0$:

$$C_1 = a_0a_1 + a_1a_2 + a_2a_3 + a_3a_0 = 1 + 1 - 1 - 1 = 0.$$

In the same way,

$$C_2 = a_0a_2 + a_1a_3 + a_2a_0 + a_3a_1 = 1 - 1 + 1 - 1 = 0,$$
$$C_3 = a_0a_3 + a_1a_0 + a_2a_1 + a_3a_2 = -1 + 1 + 1 - 1 = 0,$$

and $C_0 = 4$. So every periodic correlation of this sequence at a nonzero shift is zero. As with A_s, each product is 1 for an agreement and -1 for a disagreement, so $C_s = 0$ says that the sequence agrees with its rotated copy in exactly half of the n positions. Sequences with this property at every nonzero shift are what the rest of the chapter is about.

How the two kinds of correlation are related

Start with a concrete case, $n = 5$ and $s = 2$. The periodic sum has five terms, and the last two are the ones that wrap around:

$$C_2 = \underbrace{a_0a_2 + a_1a_3 + a_2a_4}_{A_2} + \underbrace{a_3a_0 + a_4a_1}_{A_3}.$$

Swapping the factors turns the last two terms into $a_0a_3 + a_1a_4$, which is A_3. The same split works for every shift.

Fix a shift s with $1 \le s \le n - 1$. The terms of C_s fall into two groups, according to whether the second position wraps around.

- If $0 \le j \le n - 1 - s$, then $j + s \le n - 1$. Nothing wraps, and the term is a_ja_{j+s}. These are exactly the terms of A_s.
- If $n - s \le j \le n - 1$, then $n \le j + s \le (n - 1) + s < 2n$. Subtracting n once brings the position back into the list, so $(j + s) \bmod n = j + s - n$, and the term is a_ja_{j+s-n}.

Thus

$$C_s = \sum_{j=0}^{n-1-s} a_ja_{j+s} + \sum_{j=n-s}^{n-1} a_ja_{j+s-n}.$$

The first sum is A_s. To recognize the second, give its terms a new index $k = j + s - n$. The first term, $j = n - s$, has $k = 0$, and the last term, $j = n - 1$, has $k = s - 1$. Solving for the old index gives $j = k + n - s$. Substituting,

$$\begin{aligned}\sum_{j=n-s}^{n-1} a_ja_{j+s-n} &= \sum_{k=0}^{s-1} a_{k+n-s}a_k \\ &= \sum_{k=0}^{s-1} a_ka_{k+(n-s)} \\ &= A_{n-s}.\end{aligned}$$

The middle step only swaps the two factors in each product. The last step is the definition of A_{n-s}, the aperiodic correlation at shift $n-s$. This shift lies between 1 and $n-1$, so A_{n-s} is defined, and its index runs from 0 to $n-1-(n-s) = s-1$, exactly as in the line above it. So, for every shift s with $1 \le s \le n-1$,

$$C_s = A_s + A_{n-s}. \tag{14.1}$$

For $(1, 1, 1, -1)$ this gives $C_1 = A_1 + A_3 = 1 + (-1) = 0$, in agreement with the direct calculation.

The two kinds of correlation are related, but they measure different things: small aperiodic correlations do not make the periodic ones zero. The Barker sequence $(1, 1, 1, -1, 1)$ of length five has aperiodic correlations $A_1 = 0$, $A_2 = 1$, $A_3 = 0$, and $A_4 = 1$ (check them), and by (14.1) its periodic correlations $C_1 = A_1 + A_4 = 1$ and $C_2 = A_2 + A_3 = 1$ are not zero. To pass from one kind of correlation to the other, use equation (14.1), and say so when you do.

Pause and decide. Explain why, when the length n is odd, no periodic correlation C_s can be zero.

Check. The sum C_s has n terms, each equal to 1 or -1. The parity argument of Section 14.1 applies to any such sum: if m terms equal -1, the sum is $n-2m$, which has the same parity as n. So when n is odd, C_s is odd, and therefore it is not zero, because zero is even. In particular, for odd $n \ge 3$, no sign sequence of length n has all of its periodic correlations at nonzero shifts equal to zero. The sequence $(1, 1, 1, -1, 1)$ above, with $C_1 = C_2 = 1$, is one instance of this.

14.3 Circulant matrices turn correlations into dot products

The periodic correlations of a sequence can be read off from a matrix. Write the sequence a as the first row. Obtain each further row from the one above it by moving every entry one place to the right, with the last entry wrapping around to the front. After n rows, one more step would bring back the first row, so we stop there and have an n-by-n matrix. To match the positions in the sequence, we number its rows and columns from 0 to $n-1$, rather than from 1 to n as in Chapter 10.

Definition 14.4 (Circulant matrix)**.** Let a be a sign sequence of length n. The *circulant matrix* of a is the n-by-n matrix H with entries

$$H_{ij} = a_{(j-i) \bmod n}, \qquad 0 \le i, j \le n-1.$$

A matrix of this form is called *circulant*.

Row i is the sequence rotated i places to the right. Such a rotation moves the entry at position k to position $(k+i) \bmod n$. So the entry that lands in column j is the one that started at position $j-i$, or at position $j-i+n$ if $j-i$ is negative; in both cases that position is $(j-i) \bmod n$. For instance, in row 1 the entry in column 0 is $a_{(0-1) \bmod n} = a_{n-1}$, because $-1 = (-1) \cdot n + (n-1)$, so the remainder of -1 on division by n is $n-1$ (remainders are never negative, Section 5.5). The last entry of the sequence has wrapped around to the front. Row 0 is the sequence itself, because $H_{0j} = a_{j \bmod n} = a_j$. So every circulant matrix is the circulant matrix of its own first row. For our

length-four example $a = (1, 1, 1, -1)$,

$$H = \begin{pmatrix} 1 & 1 & 1 & -1 \\ -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \end{pmatrix}.$$

As j runs through $0, 1, \ldots, n-1$, the index $(j-i) \bmod n$ also runs through $0, 1, \ldots, n-1$, each value exactly once, only starting at a different place. For instance, with $n = 4$ and $i = 1$, the columns $j = 0, 1, 2, 3$ give the indices $3, 0, 1, 2$. In the language of Section 3.3, the map $j \mapsto (j-i) \bmod n$ is a bijection from $\{0, 1, \ldots, n-1\}$ to itself, and its inverse is $k \mapsto (k+i) \bmod n$. To check that these two maps undo each other, start with a number j in the list. Every integer is congruent modulo n to its remainder on division by n (Section 6.2), so $(j-i) \bmod n \equiv j - i \pmod n$, and adding i gives a number congruent to j. Congruent integers have the same remainder, and the remainder of j is j itself, because j already lies in the list $0, 1, \ldots, n-1$. Hence $\big((j-i) \bmod n + i\big) \bmod n = j$. In the same way, $\big((k+i) \bmod n - i\big) \bmod n = k$ for every k in the list. So each row of H contains every entry of a exactly once, in a rotated order. In particular, every row has squared length $a_0^2 + a_1^2 + \cdots + a_{n-1}^2 = n$.

Dot products of rows are periodic correlations

Now compare two rows of H, row i and row ℓ. Their dot product (Section 10.4) is

$$\sum_{j=0}^{n-1} a_{(j-i) \bmod n}\, a_{(j-\ell) \bmod n}.$$

For example, when $n = 4$, row 0 is (a_0, a_1, a_2, a_3) and row 1 is (a_3, a_0, a_1, a_2), so their dot product is $a_0a_3 + a_1a_0 + a_2a_1 + a_3a_2$. This is exactly C_3 as written out in Section 14.2. The calculation below shows that the dot product of any two rows is a periodic correlation in the same way.

To simplify the general sum, rename the summation index: put $k = (j-i) \bmod n$. We have just seen that k runs through $0, 1, \ldots, n-1$, each value once, as j does. So we may sum over k instead of j; the terms are the same, only listed in a different order. The second factor must also be written in terms of k. As in the check above, $k \equiv j - i \pmod n$, and therefore

$$j - \ell = (j-i) + (i-\ell) \equiv k + (i-\ell) \pmod n.$$

Congruent integers have the same remainder on division by n (Section 6.2 again), so $(j-\ell) \bmod n = (k+i-\ell) \bmod n$. Now let $s = (i-\ell) \bmod n$. Then $s \equiv i - \ell \pmod n$, so $(k+i-\ell) \bmod n = (k+s) \bmod n$ for the same reason, and the dot product becomes

$$\sum_{k=0}^{n-1} a_k\, a_{(k+s) \bmod n} = C_s.$$

Proposition 14.5. *Let a be a sign sequence of length n, and let H be its circulant matrix.*

(a) For all rows i and ℓ, the dot product of row i with row ℓ equals $C_{(i-\ell) \bmod n}(a)$.

(b) Any two different rows of H are orthogonal exactly when $C_s(a) = 0$ for every shift s with $1 \le s \le n-1$.

Proof. Part (a) is the calculation above. For part (b) we need to know which shifts occur in part (a). If $i \neq \ell$, then $i - \ell$ lies between $-(n-1)$ and $n-1$ and is not zero, so it is not a multiple of n, and the shift $(i - \ell) \bmod n$ is not zero. Conversely, every shift s with $1 \leq s \leq n-1$ does occur: rows $i = s$ and $\ell = 0$ give $(s - 0) \bmod n = s$.

Now suppose that $C_s = 0$ for every s with $1 \leq s \leq n-1$. By the first observation, the dot product of two different rows is one of these correlations, so it is zero. Conversely, suppose that any two different rows are orthogonal. By the second observation, each C_s with $1 \leq s \leq n-1$ is the dot product of the two different rows s and 0, so it is zero. □

Read the matrix identity entry by entry

Section 10.6 called a square matrix with entries ±1 a *Hadamard matrix* when it satisfies $HH^{\mathsf{T}} = nI$, where n is the number of rows. This number is called the *order* of the matrix. As that section showed, the identity $HH^{\mathsf{T}} = nI$ is really n^2 equations between numbers, one for each position (i, ℓ): the entry of HH^{T} there is the dot product of row i with row ℓ, and the entry of nI is n if $i = \ell$ and 0 otherwise. So the diagonal equations say that every row has squared length n, and the others say that different rows are orthogonal. For a matrix with entries ±1, the diagonal equations hold automatically, because each row consists of n entries with square one. So such a matrix is a Hadamard matrix exactly when any two different rows are orthogonal.

For the matrix H above, rows 0 and 1 have dot product

$$(1)(-1) + (1)(1) + (1)(1) + (-1)(1) = -1 + 1 + 1 - 1 = 0.$$

By Proposition 14.5(a) this is $C_{(0-1) \bmod 4} = C_3$, which we computed to be zero in Section 14.2. Row 0 dotted with itself gives $1 + 1 + 1 + 1 = 4$. The proposition takes care of all the remaining pairs at once.

Circulant Hadamard matrices

A *circulant Hadamard matrix* is a Hadamard matrix that is also circulant. By Proposition 14.5(b), the circulant matrix of a is a Hadamard matrix exactly when $C_s(a) = 0$ for every shift s with $1 \leq s \leq n-1$. Since every circulant matrix is the circulant matrix of its own first row, we arrive at the following statement:

> A circulant Hadamard matrix of order n exists exactly when some sign sequence of length n has $C_s = 0$ for every shift s with $1 \leq s \leq n-1$.

Our example $(1, 1, 1, -1)$ has $C_1 = C_2 = C_3 = 0$, so the matrix H displayed above is a circulant Hadamard matrix of order four.

Habit 6 (Section 6.6) asks what a change of description preserves. Here nothing is lost in the translation: a question about the correlations of a sequence and a question about the rows of a matrix have turned out to be the same question in two languages. Each language makes different tools available. The sequence language suggests sums and shifts, while the matrix language suggests dot products and orthogonality. The next section works with sums of correlations, and the one after it with orthogonal rows.

14.4 The sum of all correlations reveals a square

Which lengths n allow a sign sequence whose periodic correlations at all nonzero shifts are zero? A first restriction comes from adding up all the periodic correlations at once, including C_0. The total turns out to depend only on the sum of the entries. Let

$$r = \sum_{j=0}^{n-1} a_j = a_0 + a_1 + \cdots + a_{n-1}$$

be the sum of the entries of a. Each row of the circulant matrix contains every entry of a once, so r is also the sum of the entries in any row, and we call it the *row sum*. For $(1, 1, 1, -1)$, the row sum is $r = 2$.

To find the total, look first at the case $n = 3$, with unknown entries a_0, a_1, a_2. The three periodic correlations consist of these products:

$$\begin{aligned} C_0 &= a_0a_0 + a_1a_1 + a_2a_2, \\ C_1 &= a_0a_1 + a_1a_2 + a_2a_0, \\ C_2 &= a_0a_2 + a_1a_0 + a_2a_1. \end{aligned}$$

Together they contain each of the nine products a_ja_k, with j and k between 0 and 2, exactly once. (Here a_0a_1 and a_1a_0 count as two different products, because they come from different pairs (j, k).) Adding the display row by row gives $C_0 + C_1 + C_2$. Adding it column by column instead, the first column is $a_0a_0 + a_0a_1 + a_0a_2 = a_0(a_0 + a_1 + a_2)$, and the other two columns are $a_1(a_0 + a_1 + a_2)$ and $a_2(a_0 + a_1 + a_2)$. Together the three columns give $(a_0 + a_1 + a_2)(a_0 + a_1 + a_2)$, so

$$C_0 + C_1 + C_2 = (a_0 + a_1 + a_2)^2 = r^2.$$

The same idea works for every n. Each C_s is itself a sum over j, so the total is a double sum of $n \cdot n = n^2$ products, which we can picture as a table with one row for each shift s and one column for each position j. A finite sum may be added in any order, so instead of adding row by row we may add column by column:

$$\begin{aligned} \sum_{s=0}^{n-1} C_s &= \sum_{s=0}^{n-1}\sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n} = \sum_{j=0}^{n-1}\sum_{s=0}^{n-1} a_j a_{(j+s) \bmod n} \\ &= \sum_{j=0}^{n-1} a_j \left(\sum_{s=0}^{n-1} a_{(j+s) \bmod n} \right) \\ &= \sum_{j=0}^{n-1} a_j \left(\sum_{k=0}^{n-1} a_k \right) = \sum_{j=0}^{n-1} a_j\, r = r \sum_{j=0}^{n-1} a_j = r^2. \end{aligned}$$

Here is what happens at each step.

- The first line writes out each C_s from its definition and then changes the order of summation, as just explained.

- In passing to the second line, notice that a_j does not depend on s. So it is a common factor of all the terms in the inner sum and can be pulled out in front of it.
- Passing to the third line is the key step. Fix j. As s runs through $0, 1, \ldots, n-1$, the position $(j+s) \bmod n$ runs through $j, j+1, \ldots, n-1$ and then wraps around to $0, 1, \ldots, j-1$. It visits every position exactly once. For example, with $n = 4$ and $j = 2$, the values $s = 0, 1, 2, 3$ give the positions $2, 3, 0, 1$. So the inner sum is the sum of all the entries of a, added in a different order, and that is r.
- Finally, r is the same number for every j, so it can be pulled out of the outer sum. What remains is r times the sum of all the entries, that is, $r \cdot r$.

For $(1, 1, 1, -1)$ the identity says $4+0+0+0 = 2^2$. It holds for every sequence, not only for special ones. The sequence $(1, 1, -1, -1)$ has row sum $r = 0$ and periodic correlations $C_0 = 4$, $C_1 = 0$, $C_2 = -4$, $C_3 = 0$, which add up to $0 = 0^2$.

The identity does not use the fact that the entries are signs. The same rearrangement works for any real numbers $a_0, \ldots, a_{n-1}$. The signs are used only afterward, to know that $C_0 = n$ and that the row sum is an integer. This is Habit 3 (Section 3.4) at work: keeping track of which hypothesis is used where tells us which parts of the argument would still work in a more general setting.

Now suppose that $C_s = 0$ for every shift s with $1 \le s \le n-1$. Then the left side of the identity is just $C_0 = n$, and therefore

$$n = r^2. \tag{14.2}$$

Since r is a sum of signs, it is an integer. So the length n must be a perfect square, that is, the square of an integer. By Proposition 14.5(b), the first row of a circulant Hadamard matrix of order n is a sequence of exactly this kind, so the order of every circulant Hadamard matrix is a perfect square. In the language of Section 2.1, being a perfect square is a *necessary* condition for a number n to be the order of a circulant Hadamard matrix. Because the argument never used a particular value of n, it covers every length at once, which no list of computed examples could do.

Pause and decide. The computer search of Section 14.6 covers the lengths $1 \le n \le 13$. Which of these lengths pass the square test? Has one of them already been ruled out by another argument?

Check. The perfect squares in this range are 1, 4, and 9. Length 9 is odd, so by the pause at the end of Section 14.2 no periodic correlation of a sequence of length 9 can be zero. So passing the square test does not mean that a sequence of that length exists; it only means that this particular test cannot rule the length out. The other two lengths do have such sequences. For the sequence (1) the condition holds vacuously, because there are no shifts s with $1 \le s \le 0$. For length four, $(1, 1, 1, -1)$ has $C_1 = C_2 = C_3 = 0$.

14.5 Orthogonality also forces divisibility by four

The square condition of Section 14.4 relied on the circulant pattern. The next condition holds for every Hadamard matrix, circulant or not, as long as its order is greater than 2. Small orders show why that restriction is needed. The 1-by-1 matrix (1) is a Hadamard matrix of order 1, and the

matrix

$$\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

of Section 10.6 is a Hadamard matrix of order 2, but neither 1 nor 2 is divisible by four.

Lemma 14.6. *If a Hadamard matrix has order $n > 2$, then 4 divides n.*

Proof. The idea is to replace the matrix by a simpler Hadamard matrix of the same order, and then to look closely at three of its rows. Two kinds of change keep a matrix Hadamard.

- *Multiplying a column by -1.* This changes the sign of one entry in every row. In the dot product of two rows, the term that comes from that column is a product of two entries that have both changed sign, so it is unchanged, because $(-x)(-y) = xy$. The other terms are not touched at all. So every dot product of two rows stays the same, and the entries are still ± 1.
- *Rearranging the columns.* This applies the same rearrangement to every row. The dot product of two rows then consists of the same terms as before, added in a different order, so it does not change.

Neither change alters the order n, so it is enough to prove that 4 divides n for the new matrix. The new matrix is usually not circulant, even when the original one was, but that does not matter: the proof uses only two facts, that the entries are ± 1 and that different rows are orthogonal.

Step 1: make the first row consist of ones. For each column whose entry in the first row is -1, multiply that column by -1. Afterward the first row is $(1, 1, \ldots, 1)$.

Step 2: the second row has as many plus signs as minus signs. Since $n > 2$, the matrix has at least three rows. This step uses the second row, and Step 4 uses the third. The second row is orthogonal to the first. Since the first row consists of ones, their dot product is simply the sum of the entries of the second row, so this sum is zero. If the second row has p entries equal to 1, then it has $n - p$ entries equal to -1, and its sum is $p - (n - p) = 2p - n$. So $2p = n$. Hence n is even, and the second row has $n/2$ plus signs and $n/2$ minus signs. Put $h = n/2$.

Step 3: sort the second row. Rearrange the columns so that the h columns in which the second row has a 1 come first, and the h columns in which it has a -1 come last. The first row still consists of ones, and the second row is now

$$(\underbrace{1, \ldots, 1}_{h}, \underbrace{-1, \ldots, -1}_{h}).$$

Step 4: look at the third row. Let x be the sum of the first h entries of the third row, and y the sum of its last h entries. Its dot product with the first row is $x + y$. Its dot product with the second row is $x - y$, because the second row multiplies the first h entries by 1 and the last h entries by -1. The third row is orthogonal to both, so

$$x + y = 0 \qquad \text{and} \qquad x - y = 0.$$

Adding the two equations gives $2x = 0$, so $x = 0$. (Subtracting them gives $y = 0$ as well, although we will not need it.)

Step 5: count again. So the first h entries of the third row are signs whose sum is zero. The count of Step 2 applies to them: if q of these h entries equal 1, their sum is $q - (h - q) = 2q - h$, and this is zero, so $h = 2q$. Therefore $n = 2h = 4q$, and 4 divides n. □

To see the proof in action, apply it to the circulant Hadamard matrix H of Section 14.3. Its first row is $(1, 1, 1, -1)$, so Step 1 multiplies the last column by -1:

$$\begin{pmatrix} 1 & 1 & 1 & -1 \\ -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 \\ 1 & 1 & -1 & 1 \end{pmatrix} \longrightarrow \begin{pmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \end{pmatrix}.$$

The second row is now $(-1, 1, 1, -1)$, with its plus signs in columns 1 and 2 (counting from 0). Step 3 moves these two columns to the front, so the new column order is $1, 2, 0, 3$:

$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & -1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & 1 & -1 \end{pmatrix}.$$

The first row consists of ones, and the second row is $(1, 1, -1, -1)$. The third row is $(-1, 1, 1, -1)$. Its first half $(-1, 1)$ and its second half $(1, -1)$ both add up to zero, as Step 4 predicts. Its first half contains $q = 1$ plus sign, so Step 5 gives $h = 2q = 2$ and $n = 4q = 4$. You can check that the rows of the final matrix are still orthogonal. Notice also that it is no longer circulant: its second row is not its first row moved one place to the right.

Now we can combine the two necessary conditions.

Proposition 14.7. *If a circulant Hadamard matrix has order n, then $n = 1$ or $n = 4u^2$ for some positive integer u.*

Proof. Let a be the first row of the matrix. The matrix is the circulant matrix of a, so by Proposition 14.5(b), $C_s(a) = 0$ for every shift s with $1 \le s \le n - 1$. Equation (14.2) therefore gives $n = r^2$, where r is the row sum, an integer.

If $n = 1$, there is nothing more to prove. Otherwise $n \ge 2$. Moreover $n \ne 2$, because $n = r^2$ and 2 is not the square of an integer: if $|r| \le 1$ then $r^2 = |r|^2 \le 1$, and if $|r| \ge 2$ then $r^2 = |r|^2 \ge 4$. Hence $n > 2$, and Lemma 14.6 shows that 4 divides $n = r^2$. In particular r^2 is even, and so r is even. Indeed, if r were odd, say $r = 2k + 1$ for an integer k, then $r^2 = 4k^2 + 4k + 1$ would be odd. Write $|r| = 2u$, where u is a nonnegative integer. Since $n = r^2$ is positive, $r \ne 0$, and so $u \ge 1$. Then $n = r^2 = |r|^2 = 4u^2$. □

The proposition allows the orders 1 and $4u^2$ for $u = 1, 2, 3, \ldots$, that is,

$$1,\ 4,\ 16,\ 36,\ 64,\ 100,\ \ldots.$$

For the first two there are examples: the 1-by-1 matrix (1), and the matrix H of order four. About the orders $16, 36, 64, \ldots$ the proposition says nothing either way. A necessary condition can only rule orders out; it never produces a matrix. So these orders remain undecided, and they mark the boundary between what our elementary arguments can settle and what needs new ideas.

14.6 An exact finite experiment

A sign sequence of length n is fixed by n independent choices, one sign for each position, so there are 2^n sign sequences of length n. For small n a computer can examine every one of them. The table below shows the result of such an exhaustive search for $1 \le n \le 13$. Only integers are involved, so the arithmetic is exact, with no rounding. Section D.3 of Appendix D contains a short program that produces the table directly from the definitions. For each length the search counts two kinds of sequences:

- the Barker sequences;
- the sequences with $C_s = 0$ for every shift s with $1 \le s \le n - 1$, which by Section 14.3 are exactly the first rows of circulant Hadamard matrices.

Each sequence is counted separately, even when it is closely related to another one. For instance, a sequence and its negative are counted as two sequences. So are a sequence and its reversal (the same entries in the opposite order), and a sequence and its rotations, whenever these lists are different.

Length n	Barker sequences	Sequences with $C_s = 0$ for $1 \le s \le n - 1$
1	2	2
2	4	0
3	4	0
4	8	8
5	4	0
6	0	0
7	4	0
8	0	0
9	0	0
10	0	0
11	4	0
12	0	0
13	4	0

For each of these thirteen lengths the search examines every case, so, provided the program is correct, the table settles completely how many sequences of that length have each property. About any other length the table says nothing. Nor can the search simply be pushed much further, because each additional position doubles the number of sequences. Length 64, one of the orders that Proposition 14.7 leaves open, has 2^{64} sign sequences, which is more than $1.8 \cdot 10^{19}$. A computer that examined a billion sequences per second would need more than 580 years for that single length. And even a far faster computer could only ever check finitely many lengths, while there are infinitely many. Only a proof can settle the question for every length.

Every count in the table agrees with what we have proved, and several of its patterns are explained by our results.

- The last column is zero at every length except 1 and 4. Proposition 14.7 predicts this: below 16, the only orders it allows are 1 and 4, and at both of these we already know examples.
- For the odd lengths greater than 1 there is also a simpler explanation, given in the pause at the end of Section 14.2: when n is odd, every periodic correlation is odd, so none of them can be zero.
- Every count in the table is even. Negating every sign changes none of the correlations, aperiodic or periodic: each correlation is a sum of pair products, and negation does not change pair products (Section 6.5). Exercise 14.2 asks you to write this out. So whenever a sequence a is counted in one column of the table, its negative $-a$ is counted in the same column. Moreover $-a$ is never equal to a, since that would require $a_0 = -a_0$, that is, $a_0 = 0$. So the sequences counted in each entry of the table split into pairs $\{a, -a\}$, and their number is even.

Other features of the table do not follow from anything proved so far. One example is the absence of Barker sequences of lengths 6, 8, 9, 10, and 12. For the even lengths among these, Section 14.7 quotes a known result that links Barker sequences to circulant Hadamard matrices and so accounts for the zeros.

Because both properties depend only on pair products, the normalization of Section 6.5 would allow a faster search: examine only the sequences that begin with 1, and double each count. The program in Appendix D examines all 2^n sequences instead, so the evenness of every count is a small independent check on its output.

14.7 What current research adds

The question of this chapter is a well-known research problem. The *circulant Hadamard conjecture*, usually attributed to H. J. Ryser, states that there is no circulant Hadamard matrix of order greater than 4. In the language of this chapter: apart from the lengths 1 and 4, no sign sequence has all of its periodic correlations at nonzero shifts equal to zero. The conjecture has been open since the 1960s. Proposition 14.7 shows that a counterexample would need an order $4u^2$ with $u \geq 2$. Over the decades, researchers have proved much stronger restrictions on the possible orders, but none of these results rules out all of them.

Barker sequences have a related history. Since the early 1960s it has been known that a Barker sequence of odd length has length at most 13. In our table, the odd lengths that have Barker sequences are exactly 1, 3, 5, 7, 11, and 13.

For even lengths, the Barker question turns into a question about circulant Hadamard matrices. It has long been known that when $n > 2$ is even, every Barker sequence of length n also has $C_s = 0$ for every shift s with $1 \leq s \leq n - 1$, so its circulant matrix is a Hadamard matrix. We do not prove this here, but you can check one instance of it at length four. The eight Barker sequences of length four are $(1, 1, 1, -1)$, its three other rotations, and the negatives of these four. They are also the eight sequences counted in the last column. The same result explains the zeros in the Barker column at the even lengths 6, 8, 10, and 12. A Barker sequence of one of these lengths would be the first row of a circulant Hadamard matrix of that order, and by Proposition 14.7 such an order must be 1 or of the form $4u^2$, that is, one of $1, 4, 16, 36, \ldots$. None of the four lengths is on this list.

In the same way, a Barker sequence of even length greater than 4 would give a circulant Hadamard matrix of order greater than 4. So the circulant Hadamard conjecture, if true, also rules out such sequences, and whether they exist has likewise been open for decades.

In September 2026, a preprint claimed a proof of the circulant Hadamard conjecture. (Recall from Section 7.7 that a preprint is a research paper made public before a journal has reviewed it.) It was released in OpenAI's public mathematics repository, together with a formalization in Lean [4]. Lean is a *proof assistant*: a computer program that checks every step of a proof written in its own precise language. A *formalization* of a theorem is a version of its statement and proof written in such a language, so that the program can check it. Chapters 15 and 16 introduce Lean.

According to the repository's description, the formalized statement says that a real circulant Hadamard matrix of positive order n exists exactly when $n = 1$ or $n = 4$. The word "real" separates the matrices of this chapter, whose entries are the real numbers 1 and -1, from a related family with complex entries that researchers also study. A second formalized statement, described there as a consequence, says that a sign sequence of positive even length whose aperiodic correlations at all nonzero shifts have absolute value at most one must have length 2 or 4. In our terms, every Barker sequence of even length has length 2 or 4. The description also notes that the preprint's classification of Barker sequences of odd length is not part of the formalized statements.

Nothing in this chapter proves these claims. Our experiment stops at length 13, and Proposition 14.7 still permits the orders $4u^2$ with $u \geq 2$, that is, $16, 36, 64, \ldots$. What the chapter does give you is the vocabulary to read the claims precisely, and the exact list of orders that a complete proof has to rule out. Ruling them out requires far more mathematics than this course covers. When you meet a summary of a result like this, whether in a news report or from an AI assistant, first check which statement was proved: which matrices, which kind of correlation, which lengths. Chapter 17 returns to this example and describes how to begin reading a proof whose background you do not yet have.

Exercises

Exercises 14.1–14.4 are core exercises, and Exercises 14.5 and 14.6 are transfer exercises. Complete solutions are in Appendix C, under "Week 14."

Exercise 14.1. For the sequence $(1, 1, -1)$, compute the aperiodic correlations A_1 and A_2 and the periodic correlations C_1 and C_2 directly from their definitions. Check that your answers satisfy equation (14.1). Is $(1, 1, -1)$ a Barker sequence?

Exercise 14.2. Let a be a sign sequence of length n, and let $-a = (-a_0, -a_1, \ldots, -a_{n-1})$ be its negative. Prove that $A_s(-a) = A_s(a)$ for every shift s with $1 \leq s \leq n-1$, and that $C_s(-a) = C_s(a)$ for every shift s with $0 \leq s \leq n-1$.

Exercise 14.3. Let H be the circulant matrix of $(1, 1, 1, -1)$ displayed in Section 14.3, with rows and columns numbered from 0 to 3. Compute the entries of HH^{T} in positions $(0, 2)$, $(1, 3)$, and $(2, 2)$ directly, as dot products of rows. Check that they agree with $HH^{\mathsf{T}} = 4I$, and for each of the two off-diagonal entries, name the periodic correlation C_s that it equals.

Exercise 14.4. Let a be a sign sequence of length n. For which shifts s are both sides of the equation $C_{n-s} = C_s$ defined? For those shifts, prove the equation by renaming the summation index in the sum that defines C_{n-s}.

Exercise 14.5. Use the two necessary conditions of this chapter (the order of a circulant Hadamard matrix is a perfect square, and a Hadamard matrix of order greater than 2 has an order divisible by four) to decide which of the numbers 8, 9, 16, 25, 36 could be the order of a circulant Hadamard matrix. Does your answer show that a circulant Hadamard matrix of any of these orders exists?

Exercise 14.6. A computer examines all 2^n sign sequences of one fixed length n, looking for sequences with $C_s = 0$ for every shift s with $1 \le s \le n - 1$. Explain why finding a single such sequence proves that a circulant Hadamard matrix of order n exists, while finding none proves only that no circulant Hadamard matrix of this one order n exists. What would it take to prove that there is no circulant Hadamard matrix of any order greater than 4?

CHAPTER 15

Lean I: reading the state of a proof

> **This week**
>
> Meet the proof assistant Lean, and see the proof states of Chapter 4 written in a formal language. Before reading the steps of a Lean proof, read what the theorem says: its parameters, including its hypotheses, and the claim after its colon. Write small Lean proofs in which the ordinary mathematical argument stays visible.

15.1 A checker is not the same as an explanation

Lean is a programming language and a *proof assistant*: a program that checks proofs. You write a statement and its proof in a precise formal language, and Lean examines every step. If a step does not follow from what came before, Lean reports an error at that step. If Lean accepts the whole proof without any error or warning, then every step has been checked.

Lean is built around one idea. Every expression in Lean has a *type*, which says what kind of object it is. The natural number 3 has the type of natural numbers, and a function from natural numbers to natural numbers has a function type. Lean treats propositions in the same way: each proposition is itself a type, and a proof of the proposition is an object of that type. For example, the equation $2 + 2 = 4$ is a type, and a proof of this equation is an object of that type; the next section shows how such a proof is written. Checking a proof therefore comes down to checking that an object really has the type claimed for it. In practice, proofs are rarely written out as objects directly. They are written with *tactics*, instructions that build the proof object step by step, much as the sentences of a written proof build an argument. The official introductions explain these ideas in more detail [10, 12].

Checking a proof is not the same as explaining it. When Lean accepts a proof, you know that every step is valid. You do not yet know why the argument works, which step carries the main idea, or whether the statement says what you meant it to say. That understanding is still your job. It matters even more when a formal proof comes from an AI assistant, because a proof that Lean accepts can still be a proof of the wrong statement; Section 16.5 gives examples. For this reason, every Lean proof in this chapter is placed beside the ordinary argument it encodes.

You will not need to memorize a catalogue of tactics. The aim of this chapter is to make the proof state of Section 4.1 explicit: at each moment of a proof, which objects and facts are available, and what exactly remains to be shown? The examples prove facts that you already know from Chapters 2 and 3. That way, a difficulty with Lean's notation can be told apart from a difficulty with the mathematics.

You can read this chapter without installing anything. To run the examples yourself, install Lean

by following the official instructions [9]. They set up the free editor VS Code together with its Lean 4 extension. The examples in Chapters 15 and 16 use only the libraries that come with Lean itself, not the large mathematical library Mathlib, and they are written for Lean version 4.19.0.

What a Lean file looks like

A Lean file is an ordinary text file whose name ends in `.lean`, for example `proofs.lean`. It contains *declarations*: some define new objects, and others state and prove results. All the examples in Chapters 15 and 16 can go into one file of the following shape; Appendix D gives more detail.

```
import Std

namespace MathCourse

-- the definitions and theorems go here

end MathCourse
```

The first line, `import Std`, makes Lean's standard library available. The line `namespace MathCourse` starts a group of names. Every definition and theorem between it and `end MathCourse` receives the prefix `MathCourse`, so that, for example, the full name of the definition `twice` in the next section is `MathCourse.twice`. This keeps our names from colliding with names that Lean's libraries already use. The word `end` here closes the group of names; it has nothing to do with the end of a proof. Everything from `--` to the end of a line is a *comment*, which Lean ignores.

There are two ways to check the file. In the editor, Lean checks the file continuously as you type. When the cursor is inside a proof, a panel beside the file, called the *infoview*, shows the proof state at that point: the facts that are available and what remains to be proved. Alternatively, from a terminal—a window for typing commands to installed programs—the command `lean proofs.lean` checks the whole file at once. If it prints nothing, every declaration was accepted, without any error or warning. The proofs themselves are always typed into the file, never into the terminal.

Once Lean is installed, move the cursor through each proof one line at a time and compare the proof state in the infoview with the explanation in the text.

15.2 Reading Lean's basic notation

One symbol appears in almost every line of Lean: the colon. An expression `a : T` says that `a` is an object of type `T`; read the colon as "is a" or "has type." Here are three examples.

- `n : Nat` says that n is a natural number. The type `Nat` is Lean's name for $\mathbb{N}$, and, as in this book, it includes zero.
- `P : Prop` says that P is a proposition. Here "proposition" has the second meaning described in Section 1.5: a statement that is either true or false, whether or not anyone has proved it.

- `hp : P` says that `hp` is a proof of the proposition P. Such an object is also called *evidence* for P. When it is assumed rather than constructed, it is a *hypothesis*, and by convention the names of hypotheses begin with the letter `h`.

One symbol serves all three purposes because, as Section 15.1 explained, propositions are types and their proofs are objects of those types.

Lean writes function application with a space instead of parentheses: `f x` means $f(x)$. Parentheses are used only for grouping, so `f (x + 1)` means $f(x+1)$, while `f x + 1` means $f(x)+1$. A function can also be written down without being given a name. The expression `fun x => x + 1` is the function $x \mapsto x+1$, which sends each input to that input plus one. Applying it to 2, as in `(fun x => x + 1) 2`, gives `2 + 1`, which is 3.

Two more symbols are easy to confuse. The symbol `:=` means "is defined as" or "is proved by": it attaches a definition to a new name, or a proof to a theorem. The ordinary equals sign = appears inside statements and means equality, exactly as on paper.

Reading a declaration by its parts

Before reading a whole proof, it helps to read a declaration piece by piece. Here is a definition:

```
def twice (n : Nat) : Nat := n + n
```

The word `def` announces a definition, and `twice` is the name we chose for it. The part `(n : Nat)` declares the input, a natural number called `n`. The `: Nat` after it declares the type of the output. Everything after `:=` is the rule: the output is `n + n`. So `twice` is the function $\mathbb{N} \to \mathbb{N}$ given by $n \mapsto n+n$.

A theorem is declared in the same way. First come the word `theorem` and the theorem's name. Then come its *parameters*, if it has any: the objects and hypotheses that the theorem talks about, each written in parentheses like the input `(n : Nat)` of `twice`. A colon follows, and after it the *claim*, the proposition that the theorem asserts. Last come `:=` and the proof. The word `by` says that the proof will be given by tactics, written on the lines that follow. Here is a theorem about `twice`. It has no parameters and claims a single equality:

```
theorem twice_two : twice 2 = 4 := by
  rfl
```

To check a claim like this one, Lean can *reduce* expressions: it replaces a defined name by its definition and carries out arithmetic on specific numbers. Here `twice 2` reduces to `2 + 2`, and `2 + 2` reduces to `4`. The tactic `rfl` (short for *reflexivity*, the fact that every object equals itself) proves an equality whose two sides reduce to the same expression. Both sides of `twice 2 = 4` reduce to `4`, so `rfl` finishes the proof.

The definition and the theorem do different jobs. The definition is a choice: it gives `twice` its meaning, and there is nothing to prove. The theorem is a claim about that meaning, and it needs a proof. Had we defined `twice n` to be `n + 1`, the statement `twice 2 = 4` would amount to $3 = 4$, which is false, and `rfl` would fail with an error.

The tactic `rfl` has a limit. The equation `twice n = 2 * n` holds for every natural number n, and each numerical instance, such as `twice 5 = 2 * 5`, can be proved by `rfl`, because both sides reduce to `10`. But when n is a variable, neither side can be computed to a number. The left side reduces only as far as `n + n`, the right side stays `2 * n`, and these are different expressions, so `rfl` fails. A proof for every n needs an argument that works for a variable, such as induction (Chapter 5), rather than a computation. Lean's library already contains this result, $2n = n + n$ for every natural number n, under the name `Nat.two_mul`.

Changing the definition of `twice` changes what is proved, but renaming `twice` everywhere in the file would not, because names carry no mathematical content. Lean would accept a theorem called `all_problems_solved` whose claim is only `0 = 0`. To know what a theorem says, read its parameters, which include its hypotheses, and the claim after its colon.

A proposition and a proof of it are different objects

The two declarations `P : Prop` and `hp : P` say very different things. The expression `2 + 2 = 5` is a perfectly good proposition, and Lean accepts it as an object of type `Prop`; but it is false, so it has no proof: nothing has the type `2 + 2 = 5`. In general, `P : Prop` only says that P is a proposition, true or false, while a proof `hp : P` can exist only when P is true. So when `hp : P` appears among a theorem's parameters, it is the assumption that P holds.

An implication $P \Rightarrow Q$ can be used as a rule: give it a proof of P, and it returns a proof of Q. Lean writes this implication as `P -> Q`, using the same arrow as in the function notation $f : A \to B$, and for a good reason: a proof of `P -> Q` is a function that turns proofs of P into proofs of Q. So if `h : P -> Q` and `hp : P`, then `h hp`, the function `h` applied to `hp`, is a proof of Q. In ordinary words, `h hp` is the familiar inference "P holds, and P implies Q; therefore Q holds."

This way of reading statements and proofs is called *propositions as types*, and it is all the theory the rest of this chapter needs.

Pause and decide. Suppose `hpq : P -> Q`, `hqr : Q -> R`, and `hp : P`. What is the type of `hpq hp`? Which expression built from these three is a proof of R? Why does Lean reject `hqr hp`?

Check. The application `hpq hp` has type `Q`: the rule $P \Rightarrow Q$, applied to a proof of P, gives a proof of Q. Feeding that proof to `hqr` gives `hqr (hpq hp)`, a proof of R; the parentheses make `hpq hp` a single input. The expression `hqr hp` is rejected because `hqr` accepts only a proof of Q, while `hp` is a proof of P. Lean reports an "application type mismatch" error.

The table collects the notation of this section.

Lean	Meaning
`n : Nat`	n is a natural number, $n \in \mathbb{N}$
`P : Prop`	P is a proposition
`hp : P`	`hp` is a proof of P
`f x`	$f(x)$
`fun x => x + 1`	the function $x \mapsto x + 1$
`P -> Q`	$P \Rightarrow Q$; a proof of it turns proofs of P into proofs of Q
`:=`	is defined as, or is proved by
`by`	the proof that follows is given by tactics
`rfl`	proves an equality whose two sides reduce to the same expression

15.3 An implication as a change of goal

Our first proof with more than one step shows how Lean proves an implication. Suppose P, Q and R are propositions such that $P \Rightarrow Q$ and $Q \Rightarrow R$. We want to prove $P \Rightarrow R$. On paper the proof is short: assume P; then Q holds, because $P \Rightarrow Q$; then R holds, because $Q \Rightarrow R$. Hence $P \Rightarrow R$. In Lean it reads:

```
theorem implication_chain (P Q R : Prop)
    (hpq : P -> Q) (hqr : Q -> R) : P -> R := by
  intro hp
  have hq : Q := hpq hp
  exact hqr hq
```

Read the statement first. Writing several names before one colon, as in `(P Q R : Prop)`, declares them all at once. So the parameters are the three propositions `P`, `Q` and `R`, together with two hypotheses: `hpq`, a proof of `P -> Q`, and `hqr`, a proof of `Q -> R`. After the colon that follows the parameters comes the claim, `P -> R`. As with the results discussed in Section 4.5, the theorem works as a promise: supply three propositions and proofs of the two implications, and it delivers a proof of `P -> R`.

Now read the proof one line at a time. In Lean, the statement that still remains to be proved is called the *goal*, the same word as in Section 4.1: it is the entry on the "still to show" list. At the start, the goal is the claim `P -> R`.

- The tactic `intro hp` does what we do on paper when we write "assume P." It adds a proof of P, named `hp`, to the available facts, and it changes the goal from `P -> R` to `R`.
- The line `have hq : Q := hpq hp` records an intermediate fact. It gives the new fact the name `hq`, states its type `Q`, and supplies its proof after `:=`, namely the rule `hpq` applied to `hp`. The goal is still `R`, but there is one more fact available.
- Finally, `exact hqr hq` says: the expression `hqr hq` is a proof of exactly the current goal. It applies the rule $Q \Rightarrow R$ to `hq`, so its type is `R`. Lean accepts it, and no goal remains. The proof is complete.

The table records the proof state after each step.

Step	Facts added	Goal afterwards
Start	`P Q R : Prop`, `hpq : P -> Q`, and `hqr : Q -> R`	`P -> R`
`intro hp`	`hp : P`	`R`
`have hq`	`hq : Q`	`R`
`exact hqr hq`	none	none: the proof is complete

In the editor, Lean shows each of these states in its own notation. After `intro hp`, for example, the infoview shows

```
P Q R : Prop
hpq : P → Q
hqr : Q → R
hp : P
⊢ R
```

Each line above the symbol ⊢ lists something that is available: one or more names, a colon, and a type. The first line records that P, Q and R are propositions, and the next three lines are the hypotheses. The line that begins with ⊢, a symbol called the *turnstile*, shows the goal, the proposition that remains to be proved. Lean prints the arrow `->` as →. In the editor you may type either form; to get →, type `\to` followed by a space, and the editor replaces it by the arrow. This display is the proof state of Section 4.1, with the "known" list written above the "still to show" list. If you have Lean installed, place the cursor at the end of each line of the proof in turn, and check that the infoview agrees with the matching row of the table.

15.4 Conjunction means two obligations

To prove a conjunction $P \wedge Q$ you must prove two things, P and Q (Section 2.2). To use a hypothesis $P \wedge Q$, you may take either half of it. The next theorem does both. From $P \wedge Q$ it proves $Q \wedge P$, by taking the two halves apart and putting them back together in the opposite order. On paper: assume that P and Q both hold; then Q holds and P holds, so $Q \wedge P$.

```
theorem and_swap (P Q : Prop) : And P Q -> And Q P := by
  intro h
  exact And.intro h.right h.left
```

Lean's name for the conjunction $P \wedge Q$ is `And P Q`, and the infoview displays it as $P \wedge Q$. In the editor you may also type the symbol ∧ directly, as `\and` followed by a space. After `intro h`, the fact `h : And P Q` is available and the goal is `And Q P`. A proof of a conjunction is a pair of proofs, one for each half, and each half can be taken out of the pair: `h.left` is a proof of P, the first component, and `h.right` is a proof of Q, the second.

In the other direction, `And.intro` builds a pair: if `a` is a proof of R and `b` is a proof of S, then `And.intro a b` is a proof of `And R S`. The order of the two inputs must match the conjunction being built. The goal `And Q P` needs a proof of Q first and a proof of P second, so the last line of the proof uses `And.intro h.right h.left`.

The proof above supplies both halves at once. A second version makes the two obligations visible as two separate goals:

```
theorem and_swap_two_goals (P Q : Prop) :
    And P Q -> And Q P := by
  intro h
  constructor
  exact h.right
  exact h.left
```

The tactic `constructor` applies `And.intro` without supplying its two inputs, and each missing input becomes a goal of its own. So the goal `And Q P` is replaced by two goals, Q and P, which the infoview labels `case left` and `case right`. Each `exact` closes the first goal that is still open, so the first one proves Q and the second proves P. If the last line is left out, Lean reports the error "unsolved goals" at the word `by`, where the proof begins, and it displays the goal P that is still open.

This is the formal counterpart of a paper proof that must establish two set containments (Section 3.1), or both directions of an "if and only if" (Section 2.1). After a long first half it is easy to feel that the proof is finished. Lean keeps track of every open goal, so it does not let that mistake pass, and on paper we must keep the same list ourselves.

15.5 Composition of injective functions in Lean

Our next example is Theorem 3.2: if $f : A \to B$ and $g : B \to C$ are injective, then so is $g \circ f$. Stating it in Lean needs a little more notation for functions.

Arrows and implicit parameters

If `A` and `B` are types, then `A -> B` is the type of functions from `A` to `B`, and `f : A -> B` is the Lean form of $f : A \to B$. Do not confuse this arrow with the `=>` in `fun x => x + 1`. The arrow `->` builds a type, the type of all functions from one type to another, while `=>` separates the input of one particular function from its rule.

A function of two inputs has two arrows in its type. The type `A -> B -> C` means `A -> (B -> C)`: a function of this type takes an input of type `A` and returns a function, which in turn takes an input of type `B` and returns an output of type `C`. For example, Lean's addition function `Nat.add` has type `Nat -> Nat -> Nat`. Applied to 2, it gives `Nat.add 2`, the function $n \mapsto 2 + n$, and applying that function to 3 gives 5. This is why several inputs are written one after another, separated by spaces, as in `Nat.add 2 3`. It is also why application groups to the left: `f x y` means `(f x) y`, so `f` is applied to `x` first, and the result is applied to `y`.

Implications behave in the same way. The proposition `P -> Q -> R` means `P -> (Q -> R)`:

if P, then (if Q, then R). A proof of it takes a proof of P and then a proof of Q, and returns a proof of R.

The sets A, B, C of a mathematical statement become types in Lean. Just as `P : Prop` says that P is a proposition, `A : Type` says that `A` is a type of ordinary mathematical objects, such as `Nat`. So `A B C : Type` says that `A`, `B` and `C` are such types. In the declarations below these parameters are written in curly braces, as `{A B : Type}`. Braces mark *implicit* parameters, which Lean works out from the other inputs. When we write `Inj f`, for instance, Lean already knows that `f : A -> B`, so we do not have to write `A` and `B`. Parameters in round brackets are *explicit* and must be written at every use. Implicit parameters are still part of the statement; they are simply not written out each time.

The definition and the theorem

We write the definition of injectivity from Section 3.3 ourselves, so that every part of it is visible. Here it is, followed by the theorem.

```
def Inj {A B : Type} (f : A -> B) : Prop :=
  forall x y : A, f x = f y -> x = y

theorem inj_comp {A B C : Type}
    (f : A -> B) (g : B -> C)
    (hf : Inj f) (hg : Inj g) :
    Inj (fun x => g (f x)) := by
  intro x y h
  have hxy : f x = f y := hg (f x) (f y) h
  exact hf x y hxy
```

Read the definition first. For each function `f : A -> B`, it defines `Inj f` to be a proposition; that is what the `: Prop` just before `:=` says. The word `forall` is Lean's keyboard spelling of $\forall$, so `forall x y : A` means "for all x and y in A." The comma ends the list of variables. What follows it, `f x = f y -> x = y`, says that if the outputs agree, then the inputs agree. Lean reads each equation as a single unit before it reads the arrow, so this means `(f x = f y) -> (x = y)`. Altogether, `Inj f` is the statement $\forall x, y \in A,\ f(x) = f(y) \Rightarrow x = y$, the form of the definition that we used in the proofs of Chapter 3. In the infoview, Lean prints `forall` as $\forall$.

Next read the theorem's statement. It has implicit parameters `A`, `B`, `C`, explicit functions `f` and `g`, and two hypotheses, `hf : Inj f` and `hg : Inj g`. Its claim is `Inj (fun x => g (f x))`. The function `fun x => g (f x)` sends x to $g(f(x))$; it is the composition $g \circ f$, written without giving it a name.

To follow the proof we need one more fact: a proof of a "for all" statement is used like a function. It is a rule that accepts any element and returns a proof of the statement for that element, just as a proof of `P -> Q` turns proofs of P into proofs of Q. A proof of `Inj g` is used in this way because Lean looks inside the definition of `Inj` and finds a "for all" statement there. Since `hg : Inj g` proves $\forall x, y \in B,\ g(x) = g(y) \Rightarrow x = y$, applying `hg` to two elements `b1` and `b2` of `B` gives a proof of `g b1 = g b2 -> b1 = b2`. Applying that, in turn, to a proof `e` of `g b1 = g b2` gives

`hg b1 b2 e`, a proof of `b1 = b2`. So `hg` takes three inputs, two elements of `B` and evidence that their values under g agree, and it returns evidence that the two elements are equal. The same holds for `hf`, with elements of `A`.

Now the proof, line by line.

- The goal starts as `Inj (fun x => g (f x))`; the infoview prints it without the parentheses, as `Inj fun x => g (f x)`. Because `Inj` is defined as a "for all" statement followed by an implication, the tactic `intro` can look inside the definition. On a "for all" statement, `intro x` does what "take any x" does on paper: it adds an arbitrary element `x` to the available objects and removes the quantifier from the goal. On an implication, `intro` works as in Section 15.3: it assumes the hypothesis and leaves the conclusion as the goal. The line `intro x y h` takes three such steps at once. It introduces two arbitrary elements `x` and `y` of `A`, and a hypothesis `h` saying that their outputs under the composition agree. (The infoview shows `h` in an unexpected form; the end of this section explains why.) The goal becomes `x = y`. On paper: "Take any $x, y \in A$ and assume $g(f(x)) = g(f(y))$."
- In `hg (f x) (f y) h`, the two elements of `B` are `f x` and `f y`, not `x` and `y`, and `h` is the evidence that their values under g agree. The result is a proof of `f x = f y`, which the line records as `hxy`. On paper: "Injectivity of g gives $f(x) = f(y)$."
- Finally, `hf x y hxy` applies injectivity of f to `x`, `y` and `hxy`, and returns exactly the goal `x = y`. On paper: "Injectivity of f then gives $x = y$."

The Lean proof and the proof of Theorem 3.2 take the same two steps in the same order.

When Lean rejects an application, compare the types of its pieces. A common mistake is to skip the middle step and write `exact hf x y h`. Lean rejects this line with an "application type mismatch" error, because `hf` needs evidence that `f x = f y`, while `h` is evidence about the outputs of g. The missing step is exactly the one that injectivity of g supplies.

If you run this proof in the editor, you will see that Lean displays the hypothesis `h` as

```
h : (fun x => g (f x)) x = (fun x => g (f x)) y
```

rather than as `g (f x) = g (f y)`. The expression `(fun x => g (f x)) x` means "apply the rule $x \mapsto g(f(x))$ to the input x," and evaluating that application gives `g (f x)`. The letter `x` plays two different parts here. Inside `fun x => g (f x)` it is only a placeholder for the input, just like the x in $x \mapsto g(f(x))$, and any other letter would do. The `x` after the closing parenthesis is the element of `A` that `intro` introduced.

Evaluating an application of a `fun` expression in this way, by putting the actual input in place of the placeholder, is called *beta-reduction*. It is one more kind of the reduction described in Section 15.2. When Lean checks that a proof has the right type, it treats a proposition and its beta-reduced form as the same proposition. That is why `hg` accepts `h` as evidence that `g (f x) = g (f y)`. Some tactics, however, look for the literal written form of an expression, and Section 16.2 shows a case where we must first restate the goal in its beta-reduced form.

15.6 Existence statements, and the type of a number

To prove an existence statement, we name a witness and check that it has the required property (Section 4.1). For example, to prove that some natural number n satisfies $n + 1 = 4$, we offer the witness 3 and check that $3 + 1 = 4$. In Lean:

```
theorem a_witness : Exists (fun n : Nat => n + 1 = 4) := by
  exact Exists.intro 3 rfl
```

In Lean, the statement $\exists n \in \mathbb{N} : n + 1 = 4$ can be written as `Exists (fun n : Nat => n + 1 = 4)`, and the infoview displays it more briefly as `∃ n, n + 1 = 4`. The part `n : Nat` after `fun` says that the input `n` is a natural number. So the input to `Exists` is a function that turns each natural number n into the proposition $n + 1 = 4$. Applied to 3, this function gives the true proposition `3 + 1 = 4`; applied to 2, it gives the false proposition `2 + 1 = 4`. The whole expression says that at least one natural number has this property.

To prove the statement, `Exists.intro` takes two inputs: a witness, here 3, and a proof that the witness has the property, here a proof of `3 + 1 = 4`. That proof is `rfl`. This time `rfl` is not a tactic on a line of its own but an input to `Exists.intro`. Inside an expression, `rfl` is the name of a proof object, the proof that an object equals itself. Lean accepts it as a proof of `3 + 1 = 4` because `3 + 1` reduces to `4`, so the two sides are the same object. If we offered the witness 2 instead, Lean would ask for a proof of `2 + 1 = 4`, and `rfl` would fail, because that equation is false.

Using a hypothesis that says something exists involves the same two pieces, the witness and its property. If all we know is that some number has a property, we cannot yet refer to that number or use it in a calculation. First we must give it a name and record its property, as a paper proof does when it says "let n be such a number, so that $n + 1 = 4$." The complete proof about surjective functions in Section 16.2 does this unpacking with the tactic `cases`, and it also shows a witness that has to be built from other information.

The type of a number decides what subtraction means

The witness 3 had to be a natural number, because the statement declared `n : Nat`. The type of a variable matters in a second way too: it decides what the arithmetic symbols mean. A natural number cannot be negative, so subtraction of natural numbers needs a rule for the case in which the usual answer would be negative. Lean's rule is that natural-number subtraction is *truncated* at zero: if $a \geq b$, then `a - b` is the usual difference, and if $a < b$, then `a - b` is 0. The notation `(e : T)` tells Lean to read the expression `e` as an object of type `T`, and here it decides which subtraction the minus sign means. Thus `(7 - 10 : Nat)` is 0, while `(7 - 10 : Int)` is -3, because `Int`, Lean's name for the integers $\mathbb{Z}$, contains negative numbers. To see this, put the line `#eval (7 - 10 : Nat)` in your file, outside any proof. The command `#eval` asks Lean to compute an expression. The result `0` appears in the infoview when the cursor is on that line, and `lean proofs.lean` prints it too; this output is a result, not an error message. The line `#eval (7 - 10 : Int)` gives `-3`.

Truncated subtraction breaks familiar laws. For integers, $(a-b)+b = a$ always holds. For natural numbers it fails whenever $a < b$: in `Nat`, the expression `(7 - 10) + 10` equals $0 + 10 = 10$, not 7. So an argument that subtracts a larger number from a smaller one, and then treats the result as negative, cannot be copied unchanged into natural-number subtraction. On paper, too, a statement can change when the domain of its variables changes: the equation $x + 3 = 1$ has the solution $x = -2$ among the integers, but no solution among the natural numbers. This is why the statement contract of Habit 2 (Section 2.6) records the domain of every variable. When an assistant offers to repair a formal proof, check whether it has quietly changed the kind of number being used.

15.7 Three ways to use an equation

The first way to use an equation is to apply the same function to both sides. Equal inputs give equal outputs: if $x = y$, then $f(x) = f(y)$. In Lean, if `h` is a proof of `x = y`, then `congrArg f h` is a proof of `f x = f y`; the name is short for "congruence in the argument," where the *argument* of a function means its input. This rule works for every function f, and it should not be confused with injectivity. Injectivity goes in the opposite direction, from $f(x) = f(y)$ to $x = y$, and it holds only for some functions: the squaring map on $\mathbb{R}$ sends 1 and -1 to the same output (Section 3.3).

Another way to use an equation is to substitute. If `h : a = b`, the tactic `rw [h]` rewrites the goal, replacing every occurrence of the left side `a` by the right side `b`. After rewriting, `rw` tries `rfl` by itself, so a goal that has become an equation with two identical sides is closed at once. For example:

```
theorem add_one_of_eq (a b : Nat) (h : a = b) :
    a + 1 = b + 1 := by
  rw [h]
```

The rewrite turns the goal `a + 1 = b + 1` into `b + 1 = b + 1`, which `rw` then closes. On paper we would write: "since $a = b$, substitute b for a." The square brackets may also hold several equations separated by commas; `rw` then rewrites with them one after another.

The third way is for a chain of equalities: Lean's `calc` block [11]. Each line of the block states one link of the chain, followed by its justification after `:=`. Lean then combines the links, using the fact that equality is transitive: if $a = b$ and $b = c$, then $a = c$. Here is the first half of Exercise 3.5: a function with a left inverse is injective.

```
theorem left_inverse_inj {A B : Type}
    (f : A -> B) (g : B -> A)
    (h : forall x, g (f x) = x) : Inj f := by
  intro x y hxy
  calc
    x = g (f x) := (h x).symm
    _ = g (f y) := congrArg g hxy
    _ = y := h y
```

The hypothesis `h : forall x, g (f x) = x` says that g undoes f: $g(f(x)) = x$ for every x. Lean infers that x ranges over `A`, because `f` is applied to it. Since two functions are equal when

they agree at every input (Section 3.2), this is the condition $g \circ f = \mathrm{id}_A$ of Exercise 3.5, stated one input at a time. (The exercise calls the left inverse h; in the Lean code the name `h` belongs to the hypothesis instead.) As in Section 15.5, the line `intro x y hxy` opens the definition of `Inj f`. It introduces `x` and `y` in `A` and the hypothesis `hxy : f x = f y`, and it leaves the goal `x = y`. The `calc` block proves this goal in three links, the Lean form of the chain

$$x = g(f(x)) = g(f(y)) = y.$$

In the `calc` block, the underscore at the start of the second and third lines stands for the right side of the line before, so that no expression has to be written twice.

- The first link, `x = g (f x)`, comes from `h x`, the hypothesis applied to this particular `x`. But `h x` proves `g (f x) = x`, with the two sides in the other order. The suffix `.symm` (short for *symmetry*) reverses an equation: if `e` is a proof of `a = b`, then `e.symm` is a proof of `b = a`. The parentheses in `(h x).symm` make `.symm` apply to the proof `h x`. Without them, Lean reads `h x.symm` as `h` applied to `x.symm` and reports an error.
- The second link, `g (f x) = g (f y)`, is `congrArg g hxy`: apply g to both sides of $f(x) = f(y)$.
- The third link, `g (f y) = y`, is `h y`.

Lean joins the three links into a proof of `x = y`, which is the goal.

Exercises

Exercises 15.1–15.4 are core exercises, and Exercises 15.5 and 15.6 are transfer exercises. Complete solutions are in Appendix C, under "Week 15."

Exercise 15.1. In `h : P -> Q`, explain the roles of `h`, `P`, and `Q`. If `hp : P`, what does `h hp` provide?

Exercise 15.2. Write a Lean proof of $P \wedge Q \Rightarrow P$, and explain the one mathematical inference it uses.

Exercise 15.3. Write a Lean theorem stating that some natural number n satisfies $n + 2 = 7$, and prove it by giving a witness.

Exercise 15.4. Explain why `congrArg g h` cannot, in general, recover $x = y$ from `h : g x = g y`. Which hypothesis is missing?

Exercise 15.5. Using `Inj`, state and prove in Lean the first part of Proposition 3.3: if $g \circ f$ is injective, then f is injective. Do not assume that g itself is injective.

Exercise 15.6. Explain why `(7 - 10 : Nat)` and `(7 - 10 : Int)` describe different computations. What lesson does this give for translating a theorem from one kind of number to another?

CHAPTER 16

Lean II: definitions, dependencies, and trust

This week

Use Lean to bring out the choices that are hidden in a mathematical statement. Before trusting an accepted proof, read the theorem's hypotheses and the definitions it uses. Find out whether a proof depends on unfinished steps, and learn a way of repairing a failing proof that keeps the statement you meant to prove.

16.1 The statement must be right before the proof can help

When Lean accepts a proof without any error or warning, it guarantees one thing: the conclusion, exactly as it is written in Lean, follows from the hypotheses, exactly as they are written in Lean. It does not guarantee that the Lean statement says what anyone meant. A Lean statement is a translation of a mathematical sentence, perhaps from a research paper, a textbook exercise, or a request to an AI assistant, and a translation can be wrong even when the proof of it is perfectly correct. Lean cannot notice such a mistake, because it never sees the original sentence. Comparing the formal statement with the intended one is a separate task, and it is the reader's job, not the proof assistant's [14].

Here is how easily the two can drift apart. Section 7.7 defined the second out-neighborhood $N_2^+(v)$ of a vertex v as the set of vertices at directed distance exactly two from v. A tempting shortcut is to take instead all the endpoints w of directed walks $v \to u \to w$ of length two. Both descriptions are precise, and Lean would accept correct proofs about either one. But they describe different sets. Take the oriented graph from that section, with edges $a \to b$, $b \to c$, $a \to c$, and $c \to d$. The two-step walks from a are $a \to b \to c$ and $a \to c \to d$, so the shortcut gives the set $\{c, d\}$. The only vertex at directed distance exactly two from a is d, because the edge $a \to c$ puts c at distance one. So $N_2^+(a) = \{d\}$.

The difference matters for Seymour's conjecture. The set given by the shortcut always contains $N_2^+(v)$, and it can be larger, as it is here. A larger second set makes the inequality $|N_2^+(v)| \geq |N_1^+(v)|$ easier to satisfy. In this very graph, the vertex a fails the true inequality, since $|N_2^+(a)| = 1 < 2 = |N_1^+(a)|$, but it would pass with the shortcut, which gives a set of two vertices. So a "proof of Seymour's conjecture" that used the shortcut would prove a weaker statement than the conjecture. A Lean definition records every such choice, but someone still has to read the definition and notice which choice was made.

So before reading any tactics, compare the formal statement with the intended one, piece by

piece. Make a table with two columns. In the first, list the objects of the mathematical statement, its hypotheses, and its conclusion; in the second, write what represents each of them in Lean. This is the statement contract of Habit 2 (Section 2.6), written out for a formal statement, and Section 16.2 gives a complete example. While you fill in the table, check the places where translations most often go wrong. Most of them can be seen in Seymour's conjecture: *every nonempty finite oriented graph has a vertex v with $|N_2^+(v)| \geq |N_1^+(v)|$.*

- *Domains.* Does each variable range over the intended set? In Section 15.6, the expression $7 - 10$ meant 0 for natural numbers and -3 for integers.
- *Strict and non-strict inequalities.* With $>$ in place of $\geq$, the conjecture would be false. A graph with one vertex and no edges is a nonempty finite oriented graph. Its only vertex v has $|N_2^+(v)| = 0 = |N_1^+(v)|$, so no vertex satisfies $|N_2^+(v)| > |N_1^+(v)|$.
- *Nonemptiness.* A graph with no vertices has no vertex with the required property, simply because it has no vertex at all. Without the word "nonempty," the conjecture would be false.
- *Excluded cases.* The conditions $w \neq v$ and $w \notin N_1^+(v)$ in the definition of $N_2^+(v)$ are exactly what the shortcut above leaves out.
- *The order of quantifiers.* The conjecture claims that *some* vertex satisfies the inequality, not that every vertex does. In the four-vertex graph above, a fails the inequality, while the vertex d, which has no out-neighbors, satisfies it with $0 \geq 0$. In the same way, "for every $y \in B$ there is an $x \in A$ with $f(x) = y$" says that f is surjective, whereas "there is an $x \in A$ with $f(x) = y$ for every $y \in B$" asks a single input to be sent to every element of B at once. That is impossible as soon as B has two different elements (Section 2.3).

Finding the statement and its definitions

To fill in the table for an unfamiliar Lean theorem, read it in this order. First find the whole declaration, from the word `theorem` to the `:=` that begins the proof. Its parameters name the objects and the hypotheses, and the proposition after the colon that follows the parameters is the conclusion (Section 15.2). Next, look up each definition that appears in the statement and read its *body*, the part after its own `:=`. A definition can contain conditions that its short name does not reveal. Only when you understand the statement should you turn to the proof and see how it builds its evidence.

Names alone are not enough. Lean would accept a theorem called `all_problems_solved` whose claim is only `0 = 0` (Section 15.2). Definitions are no different. A definition called `Second`, for example, could describe the second out-neighborhood with or without the conditions $w \neq v$ and $w \notin N_1^+(v)$, and only its body tells you which.

16.2 A complete existence proof in Lean

In Section 15.6, the witness for an existence statement was easy to spot. In many existence proofs the witness has to be built from other information. Our example is Exercise 3.2: if $f : A \to B$ and $g : B \to C$ are surjective, then so is $g \circ f$. Recall from Section 3.3 that f is surjective when every

target has a preimage:

$$\forall b \in B\ \exists a \in A: \qquad f(a) = b.$$

Here is this definition in Lean, followed by the theorem and its proof.

```
def Surj {A B : Type} (f : A -> B) : Prop :=
  forall y : B, Exists (fun x : A => f x = y)

theorem surj_comp {A B C : Type}
    (f : A -> B) (g : B -> C)
    (hf : Surj f) (hg : Surj g) :
    Surj (fun x => g (f x)) := by
  intro c
  cases hg c with
  | intro b hb =>
    cases hf b with
    | intro a ha =>
      refine Exists.intro a ?_
      show g (f a) = c
      rw [ha, hb]
```

The statement

As Section 16.1 recommends, we read the statement before the proof. The body of the definition says that `Surj f` is the proposition `forall y : B, Exists (fun x : A => f x = y)`: for every `y` in `B`, some `x` in `A` satisfies `f x = y`. The body shows the domain `A`, the codomain `B`, and the order of the quantifiers. The target `y` is chosen first, and the input `x` is found afterward, so it may depend on `y`. The table compares the theorem with the mathematical statement.

Mathematics	Lean
sets A, B, C	types `A B C : Type`, as implicit parameters
functions $f : A \to B$ and $g : B \to C$	`f : A -> B` and `g : B -> C`
hypothesis: f is surjective	`hf : Surj f`
hypothesis: g is surjective	`hg : Surj g`
the composition $g \circ f$	`fun x => g (f x)`
conclusion: $g \circ f$ is surjective	`Surj (fun x => g (f x))`

Every part of the mathematical statement has a partner in Lean, and nothing extra has been assumed. Section 16.5 shows what an extra assumption can do.

The proof, line by line

Because `Surj` is defined as a "for all" statement, the tactic `intro` can look inside the definition, just as it looked inside `Inj` in Section 15.5. The line `intro c` introduces an arbitrary target `c` in `C`, and the infoview shows the goal

```
∃ x, (fun x => g (f x)) x = c
```

As at the end of Section 15.5, Lean shows the composition applied to `x` without evaluating it. In words, the goal asks for some x in A with $g(f(x)) = c$. We do not yet know which element to use. Our only sources of elements are the two hypotheses, and we start with the one about g, because the outputs of g lie in C, where the target c lives.

Since `hg` proves a "for all" statement, it can be applied to `c`, just as the hypothesis `hg : Inj g` was applied to elements of `B` in Section 15.5. The result `hg c` proves that some element of `B` is sent to `c` by `g`. Like all existence evidence, it consists of a witness together with a proof that the witness has the property. The lines

```
cases hg c with
| intro b hb =>
```

use the tactic `cases`, which takes evidence apart. Here they mean: unpack this existence evidence, call the witness `b` and the proof of its property `hb`, and continue with the indented lines below. These indented lines form a *branch* of the proof. Inside this branch the facts `b : B` and `hb : g b = c` are available, and the goal has not changed. The word `intro` after the vertical bar is not the tactic `intro`. It refers to `Exists.intro`, the only way to build existence evidence, which Section 15.6 used to combine a witness with a proof. Writing `| intro b hb` says: this evidence was built as `Exists.intro b hb`, so take it apart into those two pieces. The infoview labels the goal inside this branch `case intro`, after the same word; inside the second unpacking, described next, the label becomes `case intro.intro`.

The second unpacking does the same with `hf b`. Since b lies in B, the surjectivity of f applies to it, and it supplies a witness `a : A` together with `ha : f a = b`. Inside this second branch the available information is

$$b \in B, \quad g(b) = c, \qquad a \in A, \quad f(a) = b.$$

Now we have a candidate input, a. The line `refine Exists.intro a ?_` proposes a as the witness. The tactic `refine` works like `exact`, except that its argument may contain the placeholder `?_` for a part of the proof that has not been supplied yet; that part becomes the new goal. Here the missing part is the proof that a has the required property, and Lean displays the new goal as

```
(fun x => g (f x)) a = c
```

Again the composition is shown without being evaluated: this is the rule $x \mapsto g(f(x))$, applied to the input a. The line `show g (f a) = c` replaces the goal by its beta-reduced form. The tactic `show` accepts a new form of the goal only when Lean can check, by reducing expressions, that it is the same proposition as the old one. Here the check succeeds, because beta-reducing `(fun x => g (f x)) a` gives `g (f a)`.

The `show` line is needed. The next tactic, `rw [ha, hb]`, rewrites by searching the goal for the written expression `f a`, the left side of `ha`. In `(fun x => g (f x)) a` that expression does not occur: inside the rule, the goal contains `f x`, with the variable `x`. Without the `show` line, Lean stops at the `rw` line with the error

```
tactic 'rewrite' failed, did not find instance of the pattern in the target
    expression
  f a
```

After `show`, the expression `f a` is visible. Rewriting with `ha : f a = b` turns the goal into `g b = c`. Rewriting with `hb : g b = c` then turns it into `c = c`, which `rw` closes by itself (Section 15.7).

The table records the proof state after each step.

Step	Facts added	Goal afterwards
Start	`hf : Surj f` and `hg : Surj g`	`Surj (fun x => g (f x))`
`intro c`	`c : C`	`∃ x, (fun x => g (f x)) x = c`
unpack `hg c`	`b : B` and `hb : g b = c`	unchanged
unpack `hf b`	`a : A` and `ha : f a = b`	unchanged
`refine Exists.intro a ?_`	none	`(fun x => g (f x)) a = c`
`show g (f a) = c`	none	`g (f a) = c`
`rw [ha, hb]`	none	none: the proof is complete

The witnesses are found in the opposite order to the one in which the functions are applied. The composition applies f first and g second. The proof first finds an input to g that reaches the final target c, and then an input to f that reaches the intermediate value b. The solution of Exercise 3.2 finds them in the same order.

Pause and decide. Could the proof begin by unpacking `hf` instead of `hg`?

Check. No. The hypothesis `hf` says that every element of B has a preimage under f, so it can only be applied to an element of `B`. At the start, the only element available is the target `c`, which lies in `C`. Lean rejects `hf c` with an "application type mismatch" error, because `c` has type `C` where an element of `B` is expected. An element of B becomes available only after `hg c` has been unpacked and has supplied `b`. The types of the hypotheses force the order of the steps.

The ordinary proof beside the Lean proof

Here is the same argument written on paper.

> Let $c \in C$. Since g is surjective, there is $b \in B$ with $g(b) = c$. Since f is surjective, there is $a \in A$ with $f(a) = b$. Then $(g \circ f)(a) = g(f(a)) = g(b) = c$. Since c was arbitrary, $g \circ f$ is surjective.

Each sentence has a partner in the Lean proof. "Let $c \in C$" is `intro c`. The two sentences that begin with "Since g" and "Since f" are the two unpackings; each uses a hypothesis to obtain

a named witness. The chain of equalities proposes a as the witness and checks it, which Lean does with `refine`, `show`, and `rw`. The last sentence has no separate line in Lean. Because `c` was introduced as an arbitrary element, Lean already knows that the argument covers every `c`.

The indentation shows where each name may be used. The names `b` and `hb` exist only inside the branch that follows `| intro b hb =>`, and `a` and `ha` exist only inside the branch nested within it. Before the first unpacking, the proof knows that some element of B is sent to c, but it has no name for that element and cannot use it. A written proof behaves the same way: the sentence "there is $b \in B$ with $g(b) = c$" introduces the name b and records the one fact that is known about it. Notice also that b need not be the only element of B with this property, since c may have several preimages under g. The proof uses nothing about b except $g(b) = c$, so it works whichever of them b stands for. This is the care that Section 4.4 asked for when it distinguished "suppose x is a solution" from "let x be the solution."

16.3 Sets as membership predicates

A *predicate* on a set A is a rule that turns each element of A into a proposition. For example, "n is even" is a predicate on the integers. For each particular integer it gives a statement, such as "6 is even" or "7 is even," and each of these statements is either true or false. You have already met a predicate in Lean: in Section 15.6, the input to `Exists` was the function `fun n : Nat => n + 1 = 4`, which turns each natural number n into the proposition $n + 1 = 4$. In Lean, a predicate on a type `A` is a function of type `A -> Prop`.

A predicate describes a set, namely the set of elements for which it holds; in the set-builder notation of Section 3.1, a predicate P on A describes the set $\{x \in A : P(x)\}$. Conversely, a set $S \subseteq A$ gives the predicate "$x \in S$." So a subset of a type `A` can be represented in Lean by a predicate `s : A -> Prop`, where the proposition `s x` means that x belongs to the set. For example, the set of even natural numbers is represented by the predicate `fun n : Nat => Exists (fun k : Nat => n = 2 * k)`, which turns each natural number n into the proposition "$n = 2k$ for some natural number k." In Chapter 3 a set was a collection of members; here it is represented by its membership condition. Nothing is lost in the translation, because for every x the question "does x belong?" has the same answer in both descriptions.

A predicate states what membership means. It does not have to come with a quick way of deciding membership. For example, "some sign sequence of length n has periodic correlation $C_s = 0$ for every shift s with $1 \le s \le n - 1$" is a predicate on the positive integers n. For any one value of n, membership can be decided by checking all 2^n sign sequences of length n, as Section 14.6 did for $n \le 13$. But this soon takes far too long, and no amount of checking covers every n. Deciding exactly which n satisfy the predicate is the circulant Hadamard problem of Section 14.7, which had been open for decades when a proof was claimed in 2026.

Here are the intersection and the preimage, defined through their membership conditions, together with a theorem that connects them:

```
def meet {A : Type} (s t : A -> Prop) : A -> Prop :=
  fun x => And (s x) (t x)

def preimage {A B : Type} (f : A -> B)
    (s : B -> Prop) : A -> Prop :=
  fun x => s (f x)

theorem preimage_meet {A B : Type} (f : A -> B)
    (s t : B -> Prop) (x : A) :
    Iff (preimage f (meet s t) x)
      (meet (preimage f s) (preimage f t) x) := by
  rfl
```

Suppose `s` and `t` describe the sets S and T. The predicate `meet s t` holds at `x` when both `s x` and `t x` hold, so it describes the intersection $S \cap T$. The predicate `preimage f s` holds at `x` when `s (f x)` holds, that is, when $f(x) \in S$, so it describes the preimage $f^{-1}(S) = \{x \in A : f(x) \in S\}$ of Section 3.4. The expression `Iff P Q` is Lean's way of writing $P \Leftrightarrow Q$, "P if and only if Q." So the theorem `preimage_meet` is the identity

$$f^{-1}(S \cap T) = f^{-1}(S) \cap f^{-1}(T)$$

from Section 3.4, stated one element at a time: for each `x`, membership in the left side is equivalent to membership in the right side. Since two sets are equal exactly when they have the same members, this is the same as the equality of the two sets.

Why does `rfl` prove it? Unfold the definitions on the left side: `preimage f (meet s t) x` becomes `meet s t (f x)`, which becomes `And (s (f x)) (t (f x))`. Unfold the right side: `meet (preimage f s) (preimage f t) x` becomes `And (preimage f s x) (preimage f t x)`, which becomes the very same `And (s (f x)) (t (f x))`. Section 15.2 used `rfl` to prove an equality whose two sides reduce to the same expression. The same tactic also proves an equivalence whose two sides reduce to the same proposition, and Lean unfolds `preimage` and `meet` by itself while it compares them. No rewriting is needed.

The proof is short because the definitions already contain the reasoning. In Section 3.4, every step of the chain of equivalences for $f^{-1}(S \cap T)$ was justified by "definition of preimage" or "definition of intersection." Those are exactly the unfoldings that Lean carries out, so the single line `rfl` covers the whole chain.

Pause and decide. Suppose the right side of `preimage_meet` were written `meet (preimage f t) (preimage f s) x`, with `s` and `t` exchanged. The statement is still true. Would `rfl` still prove it?

Check. No. The left side still unfolds to `And (s (f x)) (t (f x))`, but the right side now unfolds to `And (t (f x)) (s (f x))`. These two propositions are equivalent, but they are not the same expression, so `rfl` fails. Lean's error message says that the left-hand side is "not definitionally equal" to the right-hand side, which means that the two sides do not reduce to the same expression. A proof would have to swap the two halves of the conjunction in each direction

of the equivalence, for instance by using the theorem `and_swap` of Section 15.4 once for each direction.

This example also shows why definitions deserve close attention. Lean writes the disjunction $P \vee Q$ of Section 2.2, "P or Q," as `Or P Q`, just as it writes $P \wedge Q$ as `And P Q`. Suppose `meet` had been defined with `Or` in place of `And`. The theorem `preimage_meet` would keep its name, its proof `rfl` would still work, and Lean would still accept it. But it would now be the statement about unions from Exercise 16.2, and the name `meet` would be misleading: what a statement says is determined by the bodies of its definitions, not by their names.

16.4 Encoding the exclusions in a second out-neighborhood

A directed graph whose vertices form a type `V` can be described in Lean by its edge relation `r : V -> V -> Prop`. As in Section 15.5, a type with two arrows describes functions of two inputs: `r` takes two vertices `v` and `w` and returns the proposition `r v w`, which we read as $v \to w$. A relation of this kind can describe more than the directed graphs of Section 7.5, because nothing in it rules out a loop $v \to v$, which Definition 7.7 does not allow. Like a directed graph, it also allows a pair of opposite edges $v \to w$ and $w \to v$, which an oriented graph does not. When a statement needs the graph to have no loops, or to be oriented, it must say so in extra hypotheses.

Recall from Section 7.7 that w belongs to the second out-neighborhood $N_2^+(v)$ when three conditions hold: $w \neq v$; w is not an out-neighbor of v; and $v \to u \to w$ for some vertex u. In Lean, the three conditions become a definition that turns each pair of vertices v, w into a proposition:

```
def Second {V : Type} (r : V -> V -> Prop)
    (v w : V) : Prop :=
  And (Not (w = v)) (And (Not (r v w))
    (Exists (fun u : V => And (r v u) (r u w))))
```

Here `Not P` is the negation of `P`. So `Not (w = v)` expresses $w \neq v$, and `Not (r v w)` says that there is no edge $v \to w$. The last part says that some vertex `u` satisfies both `r v u` and `r u w`. It asks for at least one route $v \to u \to w$, not for a unique one; if several vertices u give a route, w is still a single member of $N_2^+(v)$ (Section 7.7).

Lean also has the usual symbols for these connectives, and the infoview uses them. When the definition is unfolded, for instance with the tactic `unfold Second`, the infoview displays the body of `Second r v w` as

$$\neg\, \mathtt{w} = \mathtt{v} \wedge \neg\, \mathtt{r\ v\ w} \wedge \exists \mathtt{u},\ \mathtt{r\ v\ u} \wedge \mathtt{r\ u\ w}$$

with no parentheses at all. Three conventions tell you how to read it. First, $\neg$ applies only to the proposition immediately after it, such as the whole equation `w = v`, and it stops at the next $\wedge$. So $\neg$`w = v` $\wedge \neg$`r v w` means ($\neg$`w = v`) $\wedge$ ($\neg$`r v w`), which says that $w \neq v$ and that there is no edge $v \to w$. Second, $\exists$`u,` reaches as far to the right as possible, so it covers both `r v u` and `r u w`, just as `Exists` does in the definition. Third, $\wedge$ groups to the right, so that $P \wedge Q \wedge R$ means $P \wedge (Q \wedge R)$. In the Lean code of this book we write the names `And`, `Not`, and `Exists` instead, because they are easy to type and, together with the parentheses, they make the grouping visible.

To read the nesting, give the three conditions temporary names:

$$P:\ w \neq v, \qquad Q:\ \text{there is no edge } v \to w, \qquad R:\ v \to u \to w \text{ for some vertex } u.$$

The definition has the shape `And P (And Q R)`. A proof `h` of it is a pair (Section 15.4). Its first component is a proof of P, and its second component is itself a pair, made of a proof of Q and a proof of R. So `h.left` is a proof of P, and `h.right` is a proof of `And Q R`. Taking components once more, `h.right.left` is a proof of Q, and `h.right.right` is a proof of R. Taking a component of a pair in this way is called a *projection*.

Pause and decide. Take the oriented graph of Section 7.7 with edges $a \to b$, $b \to c$, $a \to c$, and $c \to d$, and let $v = a$. For each vertex w, decide which of the conditions P, Q, R hold. For which w does `Second r a w` hold?

Check. For $w = a$, condition P fails. Condition Q holds, because there is no edge $a \to a$, but R fails: the out-neighbors of a are b and c, and neither has an edge to a. For $w = b$, condition P holds, but Q fails because of the edge $a \to b$. Condition R fails as well, since neither b nor c has an edge to b. For $w = c$, condition P holds and so does R, through the route $a \to b \to c$, but Q fails because of the edge $a \to c$. For $w = d$, all three hold: $d \neq a$, there is no edge $a \to d$, and $a \to c \to d$ is a route. So `Second r a w` holds only for $w = d$, in agreement with $N_2^+(a) = \{d\}$. The vertex c is excluded by condition Q alone, one of the two conditions that the shortcut of Section 16.1 leaves out.

A small but useful consequence of the definition is that a second out-neighbor is never a first out-neighbor:

```
theorem second_not_first {V : Type}
    (r : V -> V -> Prop) (v w : V)
    (h : Second r v w) : Not (r v w) := by
  exact h.right.left
```

The proof is the projection `h.right.left` described above: it extracts the proof of Q, which is exactly the claim `Not (r v w)`. If the definition grouped the conditions differently, the projections would change with it. Exercise 16.3 asks you to extract the other two conditions.

The formal statement of Seymour's conjecture in the repository of Section 7.7 needs more than this one predicate. Like us, it uses a type `V` of vertices and an edge relation `r`. It adds hypotheses saying that there are finitely many vertices, that there is at least one, and that `r` is oriented: no vertex has an edge to itself, and no two vertices are joined in both directions. It also needs a way to count the members of each out-neighborhood. Its condition for second out-neighbors, however, is exactly the one above: $w \neq v$, there is no edge $v \to w$, and $v \to u \to w$ for some u [3]. The repository's statement file [6], the Lean file named in its configuration [8], even joins the three conditions by $\wedge$ in the same order. So the piece of the statement that the shortcut gets wrong can be checked on its own, before anyone studies the long proof. Section 17.8 reads the full formal statement piece by piece.

16.5 A successful proof can still be irrelevant

Lean accepts the following theorem without any error or warning:

```
theorem impossible_context (h : False) : (0 : Nat) = 1 := by
  exact False.elim h
```

Its conclusion, $0 = 1$, is false. Yet the theorem is correct, because the conclusion is not all that it says. The parameter `h : False` is a hypothesis, and `False` is the proposition that is never true, so it has no proof. The theorem says: *if* we had a proof of `False`, then $0 = 1$ would follow. This implication is true for the reason that Section 2.5 gave for vacuous truth: its hypothesis can never be satisfied. The proof uses the rule `False.elim`, which turns a proof of `False` into a proof of any proposition whatsoever. Since nobody can ever supply the hypothesis `h`, the theorem can never be used to conclude that $0 = 1$. A reader who looked only at the conclusion would completely misunderstand what was proved.

The same thing can happen less visibly when an assistant "repairs" a proof that Lean rejects by changing the statement, most often by adding a hypothesis. The new statement may well be true and its proof correct, and still not be the statement that was asked for. Here are two examples.

- The claim "for all real numbers x and y, if $x^2 = y^2$, then $x = y$" is false: take $x = 1$ and $y = -1$. Adding the hypotheses $x \geq 0$ and $y \geq 0$ makes it true, because the square map is injective on $[0, \infty)$ (Section 3.3). The repaired claim is a correct theorem, but it says nothing about negative numbers. Whether the repair is acceptable depends on whether those hypotheses were part of the intended statement.
- An existence theorem can be made trivial by assuming the very thing to be proved. Here is `surj_comp` with one extra hypothesis:

  ```
  theorem surj_comp_assumed {A B C : Type}
      (f : A -> B) (g : B -> C)
      (hf : Surj f) (hg : Surj g)
      (h : Surj (fun x => g (f x))) :
      Surj (fun x => g (f x)) := by
    exact h
  ```

  The new hypothesis `h` is the conclusion itself, so the proof only has to repeat it. Lean accepts this theorem. It does print a warning that the variables `hf` and `hg` are unused, a useful hint that the proof never needed the real hypotheses. But a warning is not an error, and a changed statement does not always leave such a trace: `impossible_context` produces no warning at all.

In both examples the repaired statement is true and has a correct proof, but the mathematical task has changed. So when a proof has been repaired, first compare the new statement with the old one, hypothesis by hypothesis and then conclusion with conclusion, as in the table of Section 16.2. Only then look at how the proof itself changed.

16.6 Placeholders and axiom dependencies

While a proof is being developed, a gap can be filled temporarily with the word `sorry`. Lean then accepts the declaration, but it prints the warning `declaration uses 'sorry'`, and it treats the missing step as if it had been assumed. Such a declaration is not a completed proof. In fact `sorry` closes any goal at all, even a false one: Lean accepts `theorem bad : (0 : Nat) = 1 := by sorry`, with the same warning.

The warning appears only at the declaration that contains `sorry`. For example, if the last line `rw [ha, hb]` of the proof of `surj_comp` is replaced by `sorry`, Lean warns at `surj_comp`. A later theorem whose proof applies `surj_comp` gets no warning of its own, even though it now rests on the same gap.

To find out whether a theorem depends on a gap anywhere, directly or through other theorems, use the command

```
#print axioms theoremName
```

with the theorem's name in place of `theoremName`. It lists every *axiom*, that is, every statement accepted without proof, on which the theorem's proof ultimately relies. A `sorry` anywhere among the theorem's dependencies shows up in this list as the axiom `sorryAx`.

Not every axiom signals a problem. Lean's foundations include three standard axioms that ordinary mathematics uses all the time:

- `propext`, short for *propositional extensionality*: two propositions that are logically equivalent are equal, so that if $P \Leftrightarrow Q$, then P may be replaced by Q anywhere;
- `Quot.sound`: in a quotient, the set of equivalence classes of Section 6.2, two related elements have the same class; for example, $[1]_4 = [5]_4$ in $\mathbb{Z}/4\mathbb{Z}$, because $5 - 1$ is divisible by 4;
- `Classical.choice`: once it has been proved that an object of some kind exists, Lean may pick one, even when no particular one can be described. From it, together with the other two axioms, Lean derives the principle that every proposition is either true or false. Proof by contradiction uses this principle in its last step (Section 4.2), so a proof by contradiction about an arbitrary proposition usually depends on `Classical.choice`.

Seeing these in the list is normal. Many results in Lean's library use `propext`, and Section 16.7 meets two of them. The research repository of Section 7.7 explicitly permits exactly these three axioms in its formal proof of Seymour's conjecture [8]. What matters is to know what each listed axiom says [13], and to notice an axiom that does not belong, such as `sorryAx`.

For three of the examples in Chapters 15 and 16, the commands

```
#print axioms MathCourse.inj_comp
#print axioms MathCourse.surj_comp
#print axioms MathCourse.preimage_meet
```

print the three lines

```
'MathCourse.inj_comp' does not depend on any axioms
'MathCourse.surj_comp' does not depend on any axioms
'MathCourse.preimage_meet' does not depend on any axioms
```

So these proofs use only the rules of Lean's logic and our own definitions. Because the commands use the full names, with the prefix `MathCourse`, they can be placed after the line `end MathCourse`.

An axiom list is not a substitute for reading the theorem's statement. The theorems `impossible_context` and `surj_comp_assumed` of Section 16.5 depend on no axioms at all, yet the first assumes `False` and the second assumes its own conclusion. In the other direction, a theorem whose list includes `Classical.choice` may be perfectly sound. To trust a theorem, you need to know both the assumptions written in its statement and the axioms on which its proof depends.

When you check a file, read everything Lean reports: errors, warnings (a `sorry` produces a warning, not an error), and the output of any `#print axioms` commands. Wait until Lean has finished checking the whole file before concluding that there are no problems. In the editor, a line that Lean has not yet reached shows no error, whether or not it contains one. Larger projects, and proofs from sources you do not trust, call for the stronger checking procedures described in the official documentation [14]. Section 17.8 describes the procedure used for the formal proof of Seymour's conjecture.

How far a check reaches

Lean's reports tell you whether a proof is complete. A different question is how many cases a check covers, even when it finishes without any problem. Compare three kinds of check.

- A Python program, such as the one in Section D.3, can run through all $2^4 = 16$ sign sequences of length 4 and report that exactly 8 of them have $C_1 = C_2 = C_3 = 0$ (Section 14.6). This covers 16 cases. The answer is only as reliable as the program. No checker compares the code with the definition of C_s, so a slip in the code, for instance in the wrap-around, could go unnoticed unless someone read the code carefully against the definition. And the run says nothing about any other length.
- A Lean proof of a statement about specific objects, such as `twice_two` in Section 15.2, is checked by Lean step by step. But it covers only the case it states: it proves `twice 2 = 4`, and nothing about `twice 3`.
- A Lean proof of a universal statement, such as `surj_comp`, covers every case at once: all types A, B, C, and all surjective functions between them. There are infinitely many such cases, and no program could test them one by one.

In all three, the result concerns the statement as it was written. A Python function or a Lean definition that encodes the wrong quantity, for instance the aperiodic correlation where the periodic one was meant, gives a correct answer to the wrong question.

16.7 A debugging sequence that does not lose the mathematics

When Lean rejects a line, it is tempting to try one tactic after another, or to ask an assistant for anything that Lean will accept. Either habit can end with a file that Lean accepts and a statement that nobody wanted. Finding and removing the cause of an error is called *debugging*. The following sequence keeps the mathematics in view while you debug.

1. *Keep the statement fixed.* Do not edit the theorem's statement while you look for the problem.
2. *Read the proof state.* Look at the goal, the available facts, and the error message.
3. *Translate into ordinary mathematics.* Say in words what the goal claims and what each fact says.
4. *Diagnose.* Decide which of four situations you are in.
    - A *difference of form*: the goal follows from the available facts, but something is written in a different form from the one the tactic expects.
    - A *missing lemma*: the goal is true, but the proof needs a fact that is not yet available.
    - A *mismatch of domains*: a variable ranges over a different set from the intended one.
    - A *goal that does not follow*: in some situation where all the available facts hold, the goal fails. The goal may even be false outright, like the equation $2 + 1 = 4$.
5. *Respond to the diagnosis.*
    - For a difference of form, a one-line change, such as a `show` line, is usually enough.
    - For a missing lemma, find one, read its exact statement with the command `#check` (described below), and make sure that its hypotheses hold in your situation before you use it.
    - For a mismatch of domains, go back to the statement contract and decide which domain was intended. If the statement has to change, change it on purpose and record the change, for instance in a comment above the theorem.
    - If the goal does not follow, stop looking for tactics: no tactic can derive it. First ask whether an earlier step caused the problem. In Section 15.6, for example, offering the witness 2 would have required a proof of the false equation $2 + 1 = 4$, and the remedy was a better witness. If the goal already fails to follow at the start of the proof, or after steps such as `intro` that only name arbitrary objects and assume the hypotheses, then the statement itself is false. Look for a *counterexample*, an instance in which every hypothesis holds and the conclusion fails. It shows that the statement cannot be proved as it stands, and what to do next is a decision about the statement, as the second worked example below shows.

Used in this way, Lean's messages become feedback on your mathematics, not obstacles to be cleared by whatever change Lean will accept.

Two worked examples

A difference of form. Leave out the line `show g (f a) = c` in the proof of `surj_comp`, and keep the statement as it is (step 1). Reading the proof state (step 2), we see that Lean did not find the pattern `f a` at the `rw` line, and that the goal is `(fun x => g (f x)) a = c`. Translated (step 3), the goal says $(g \circ f)(a) = c$, and the facts $f(a) = b$ and $g(b) = c$ prove it. So the diagnosis (step 4) is a difference of form: the goal follows from the facts, but `f a` is not written in it, as Section 16.2 explained. The response (step 5) is the line `show g (f a) = c`. It changes the form of the goal but not its meaning, and the statement of the theorem is untouched.

A false statement. Suppose we set out to prove `Surj (fun n : Nat => n + 1)`, the claim that the map $n \mapsto n + 1$ on the natural numbers is surjective. After `intro y`, the goal is

```
∃ x, (fun n => n + 1) x = y
```

For this arbitrary natural number y, we must find a natural number x with $x + 1 = y$. Suppose that every tactic we try fails. Before trying more, translate the goal and test a small case. For $y = 0$ we would need $x + 1 = 0$, and no natural number x satisfies this, because $x + 1 \geq 1$. So the goal is false when $y = 0$. The problem cannot come from an earlier step, because `intro y` only named an arbitrary target. Hence the statement itself is false, and the target 0 is a counterexample. No tactic will ever close this goal. An assistant that was asked only to make Lean accept the file could succeed only by changing the statement, for example by adding a hypothesis as in Section 16.5, or by leaving a gap marked `sorry` (Section 16.6).

What should happen next depends on what was intended, and only a person can decide that. If the claim was meant to be about natural numbers, it is false, and Lean can prove that it is false:

```
theorem not_surj_succ :
    Not (Surj (fun n : Nat => n + 1)) := by
  intro h
  cases h 0 with
  | intro n hn =>
    exact Nat.succ_ne_zero n hn
```

In Lean, `Not P` is defined to mean `P -> False`. So `intro h` assumes that the map is surjective and leaves the goal `False`. Applying `h` to the target `0` and unpacking the result gives a natural number `n` and a proof `hn` that $n + 1 = 0$; the infoview shows `hn` in the unreduced form `(fun n => n + 1) n = 0`, which Lean treats as the same proposition. The last step uses a lemma from Lean's library, `Nat.succ_ne_zero`, and step 5 of the sequence tells us to read its exact statement before using it. To see the statement of any named theorem or lemma, put the command `#check` followed by its name on a line of its own, outside every proof, just like the command `#eval` of Section 15.6. The output appears in the infoview when the cursor is on that line. Here the line `#check Nat.succ_ne_zero` makes Lean print

```
Nat.succ_ne_zero (n : Nat) : n.succ ≠ 0
```

This reads like a declaration: the parameter `(n : Nat)` comes first, and the claim follows the colon. In the claim, `n.succ`, the *successor* of `n`, is another way of writing `n + 1`, and ≠ is Lean's

symbol for "is not equal to": `a ≠ b` means `Not (a = b)`. So the lemma says that $n + 1 \neq 0$ for every natural number n, and it has no further hypotheses. Applied to our particular `n`, the lemma gives `Nat.succ_ne_zero n`, a proof of `Not (n + 1 = 0)`, that is, of `n + 1 = 0 -> False`. Applying this in turn to `hn` produces a proof of `False`, which is the goal.

If instead the claim was meant to be about integers, then the domain was mistranslated. Over $\mathbb{Z}$, the map $n \mapsto n + 1$ is surjective: the target y is reached by the input $y - 1$. The repair is to change `Nat` to `Int`. This is a deliberate change of the statement, and the comment in the first line below records it as one:

```
-- Statement changed on purpose: Int in place of Nat.
theorem succ_surj_int : Surj (fun n : Int => n + 1) := by
  intro y
  refine Exists.intro (y - 1) ?_
  show y - 1 + 1 = y
  exact Int.sub_add_cancel y 1
```

The structure is that of `surj_comp`: introduce a target, propose a witness, restate the goal in reduced form, and then prove the reduced goal. The library lemma `Int.sub_add_cancel a b` proves $(a - b) + b = a$ for all integers a and b, and with $a = y$ and $b = 1$ it is exactly the goal. The same step would fail for natural numbers. With truncated subtraction (Section 15.6), the proposed witness for the target 0 would be $0 - 1 = 0$, and $(0 - 1) + 1 = 0 + 1 = 1$, not 0.

If you run `#print axioms` on these two theorems, Lean lists `propext`, which the library lemmas `Nat.succ_ne_zero` and `Int.sub_add_cancel` use. As Section 16.6 explained, this is one of the standard axioms, and its appearance is no cause for concern.

Both responses keep the mathematics in view: the first turns a counterexample into a theorem, and the second corrects a translation openly. Neither hides a change of statement inside a proof that Lean merely accepts.

Exercises

Exercises 16.1–16.4 are core exercises, and Exercises 16.5 and 16.6 are transfer exercises. Complete solutions are in Appendix C, under "Week 16."

Exercise 16.1. In the proof of `surj_comp` (Section 16.2), explain what `hb` and `ha` state. Explain also why the witness that the proof finally proposes must be an element of the domain A of f.

Exercise 16.2. Define a predicate `join s t` that describes the union of the sets described by `s` and `t`, in the way that `meet s t` describes their intersection. Then state and prove in Lean the corresponding fact about preimages: for each `x`, `x` belongs to the preimage of the union exactly when it belongs to the union of the preimages.

Exercise 16.3. Suppose `h : Second r v w`. Write a Lean proof that $w \neq v$, and a Lean proof that some vertex u gives a route $v \to u \to w$.

Exercise 16.4. An assistant cannot prove a theorem, so it adds the hypothesis `h : False` and then proves the result. Explain precisely what has changed, and why the fact that Lean accepts the new file does not answer the original question.

Exercise 16.5. Design a review in three steps that detects, in turn, a changed domain, a missing exclusion (such as the condition that a second out-neighbor is not a first out-neighbor), and an unfinished proof containing `sorry`. For each step, say what you examine: which part of the Lean file, or which output of Lean.

Exercise 16.6. Explain the difference between three kinds of check: running a finite test in Python, proving a statement about specific objects in Lean, and proving a universal theorem in Lean. Give an example of each, different from the examples in Section 16.6, and say which cases your example covers.

CHAPTER 17

Reading an unfamiliar proof one piece at a time

> **This week**
>
> Read a proof whose background you only partly have, without first studying a whole subject. The example is a proof that every sequence in a complete, totally bounded metric space has a convergent subsequence. Draw a map of what this one proof needs, learn only the pieces that are new to you, and rebuild the whole argument. Then use the same method to begin reading a formalized research proof, keeping in mind that its short final file rests on many other files, which hold most of the mathematics.

17.1 A new theorem on a familiar foundation

Imagine meeting the following sentence in a research paper:

> Every sequence in a complete, totally bounded metric space has a convergent subsequence.

Most of its words are familiar. Chapter 11 defined metric spaces, convergence, Cauchy sequences and completeness, and Chapter 12 used all of them in a long proof. Two expressions are new: "totally bounded" and "subsequence."

The property in the conclusion, that every sequence has a convergent subsequence, is a form of what mathematicians call *compactness*, and the branch of mathematics called topology has a large general theory of it. One possible response is to decide that a whole course on that theory must come first. A more useful response is to ask what the two new ideas do in this particular argument, and to learn exactly that much. Here is how a productive first exchange with an AI assistant might go.

Reader: I know metric spaces, Cauchy sequences and completeness. Describe how the proof of this statement works, without quoting any theorem I may not know.

Assistant: Keep infinitely many terms of the sequence inside a small region, then inside a smaller one, and so on. Pick later and later terms from these nested selections. The picked terms form a Cauchy sequence, and completeness supplies their limit.

Reader: Which facts do those steps rely on that I may not have met yet?

Assistant: Three. You need to know that the space can be covered by finitely many small balls; that if infinitely many indices are split into finitely many groups, some group still contains infinitely many of them; and how to choose indices that keep increasing.

The reader's side of this exchange is an example of a *minimal-background request*: it says what the reader already knows and asks only for what one particular proof needs. The second question pins down which proof that is, because it asks about *those steps*, the ones the assistant has just outlined. This matters, because the minimum background depends on the proof you choose to read: a different proof of the same theorem could need different concepts. A good request of this kind also asks the assistant to show, right after each new concept, the step of the proof where it is used. That keeps the conversation tied to one argument instead of drifting into a survey of a whole subject. Habit 4 (Section 4.5) asked for the contract of a single step rather than for more words; a minimal-background request asks the same kind of precise question about a whole proof.

The assistant's first answer is compressed, and you are not expected to follow every phrase of it yet. Together, its two answers propose a *dependency map* of the proof, in the sense of Section 12.2: a list of the steps of the proof and of the facts that each step relies on. Once we mark which entries of such a map we already know and which are new to us, we call it a *prerequisite map*. The new entries are the missing pieces, and they are all we need to learn.

A proposed map is not yet a proof, and it may be incomplete; Section 17.7 points out two steps that this one leaves out. Sections 17.2, 17.3 and 17.4 supply the three missing pieces, each in a short self-contained lesson that we call a *capsule*, and the last of them also sets out the plan of the proof. Section 17.5 then assembles the complete proof. None of the general theory of compactness is needed.

17.2 Capsule one: small regions that cover the space

The first missing piece is a way to describe a whole metric space, at any desired precision, by finitely many small regions. The regions we use are balls. The proof needs such a description each time it moves on to a smaller region.

Definition 17.1 (Ball)**.** Let (X, d) be a metric space, let $c \in X$, and let $r > 0$. The *ball* with center c and radius r is the set of points of X at distance less than r from c:

$$B(c, r) = \{x \in X : d(x, c) < r\}.$$

The center always belongs to its own ball, because $d(c, c) = 0 < r$. Despite the name, a ball need not look round. On the real line, $B(c, r)$ is the open interval $(c - r, c + r)$, because $|x - c| < r$ means $c - r < x < c + r$ (Section 2.6). In the space $[0, 1]$, the ball $B(0, 1/4)$ consists of the points $x \in [0, 1]$ with $x < 1/4$, so it is the interval $[0, 1/4)$; points outside the space do not count. For the metric d_∞ on the plane (Section 11.2), the ball $B\big((0, 0), 1\big)$ is the square of points (x_1, x_2) with $|x_1| < 1$ and $|x_2| < 1$, because the larger of the two numbers $|x_1|$ and $|x_2|$ is less than 1 exactly when both of them are. What makes a set a ball is the distance used to define it, not its shape.

The assistant's list did not mention it, but the proof also needs one property of balls: a small ball keeps all of its points close to one another. If x and y both lie in $B(c, r)$, then the triangle inequality, with c as the stopping point, and symmetry give

$$d(x, y) \le d(x, c) + d(c, y) < r + r = 2r.$$

So any two points of a ball of radius r are less than $2r$ apart. We call this the *diameter bound* for balls, after the diameter of a circle, which is twice its radius. On the real line it says that two points

of the interval $(c - r, c + r)$ are closer together than the length $2r$ of the interval. In the proof, this bound turns the fact that many terms lie in one small ball into the fact that these terms are close to one another.

A collection of sets *covers* X if every point of X lies in at least one set of the collection.

Definition 17.2 (Totally bounded)**.** A metric space (X, d) is *totally bounded* if, for every $r > 0$, there are finitely many points $c_1, \ldots, c_k$ of X such that the balls $B(c_1, r), \ldots, B(c_k, r)$ cover X. In symbols:

$$\forall r > 0\ \exists k \in \mathbb{N}\ \exists c_1, \ldots, c_k \in X\ \forall x \in X\ \exists i \in \{1, \ldots, k\} : \quad d(x, c_i) < r.$$

Read the quantifiers as a game, as in Section 2.3. A skeptic names a radius $r > 0$. You answer with a finite list of centers, all of them points of the space; your list may depend on r. The skeptic then picks any point x of the space, and you win if the distance from x to at least one of your centers is less than r. The space is totally bounded if you can always win. A smaller radius usually needs more centers, and that is allowed. What the definition does not ask for is one finite list that works for every radius at once; the second example below shows that for the interval $[0, 1]$ no such list exists.

Two examples on the interval $[0, 1]$

The interval $[0, 1]$ *is totally bounded.* Let $r > 0$ be given. By the Archimedean property (Section 2.3), there is an integer N with $N > 1/r$. It is positive, because $1/r > 0$, and multiplying both sides of $N > 1/r$ by the positive number r/N gives $r > 1/N$. Use the $N + 1$ centers $0, \frac{1}{N}, \frac{2}{N}, \ldots, 1$, which are spaced $1/N$ apart. Let $x \in [0, 1]$. If x is one of the centers, it lies in its own ball. Otherwise x lies strictly between two consecutive centers, say $j/N < x < (j + 1)/N$, and its distance from the left one is less than the spacing:

$$d\Big(x, \frac{j}{N}\Big) = x - \frac{j}{N} < \frac{j+1}{N} - \frac{j}{N} = \frac{1}{N} < r.$$

So every point of $[0, 1]$ lies in at least one of the $N + 1$ balls of radius r. For instance, the radius $r = 0.3$ allows $N = 4$, since $4 > 1/0.3 \approx 3.33$, and the five balls of radius 0.3 centered at 0, 0.25, 0.5, 0.75 and 1 cover $[0, 1]$.

No single finite list works for every radius. An empty list covers nothing, so fix a list of $k \geq 1$ points $c_1, \ldots, c_k$ of $[0, 1]$, and take the radius $r = 1/(2k)$. The $k + 1$ points $0, \frac{1}{k}, \frac{2}{k}, \ldots, 1$ are at least $1/k = 2r$ apart from one another. Suppose the k balls $B(c_i, r)$ covered all of these points. Assign each of the $k + 1$ points to a ball that contains it. By the pigeonhole principle (Proposition 7.4), two of the points would share a ball, and by the diameter bound they would then be less than $2r$ apart. That is impossible, so some point of $[0, 1]$ is not covered.

Totally bounded spaces are bounded

A metric space is *bounded* if there is a real number M such that $d(x, y) \leq M$ for all points x and y of the space. Every totally bounded space is bounded. If the space is empty, there is nothing to prove. Otherwise, cover it by finitely many balls of radius 1, with centers $c_1, \ldots, c_k$, where $k \geq 1$,

and let M be the largest of the finitely many distances $d(c_i, c_j)$. Now let x and y be any two points. Say x lies in the ball about c_i and y in the ball about c_j. Using the triangle inequality twice, first with c_i and then with c_j as a stopping point, and using symmetry to write $d(c_j, y) = d(y, c_j) < 1$, gives

$$d(x, y) \leq d(x, c_i) + d(c_i, c_j) + d(c_j, y) < 1 + M + 1.$$

So the number $M + 2$ bounds every distance in the space.

The converse is false: a bounded space need not be totally bounded. To see this we need a new metric. On any set X, let

$$d(x, y) = \begin{cases} 0 & \text{if } x = y, \\ 1 & \text{if } x \neq y. \end{cases}$$

This is a metric, called the *discrete metric* on X. The first requirement of Definition 11.1 holds because d takes only the values 0 and 1 and is 0 exactly when $x = y$. The second holds because $x = y$ exactly when $y = x$, so $d(x, y)$ and $d(y, x)$ are always given by the same case. For the triangle inequality $d(x, z) \leq d(x, y) + d(y, z)$, consider two cases. If $x = z$, the left side is 0 and there is nothing to prove. If $x \neq z$, the left side is 1; the point y cannot be equal to both x and z, so at least one of the two distances on the right is 1, and the right side is at least 1.

Now take $X = \mathbb{N}$ with the discrete metric. Every distance is at most 1, so the space is bounded. But a ball of radius $1/2$ contains only its center: $B(n, 1/2) = \{m \in \mathbb{N} : d(m, n) < 1/2\} = \{n\}$, because every other natural number is at distance 1 from n. So k balls of radius $1/2$ contain at most k natural numbers, and no finite collection of them covers the infinitely many natural numbers. This space is bounded but not totally bounded. Boundedness asks for a single bound on all distances; total boundedness asks for a finite description of the space at every scale, which is much more.

Pause and decide. Is the real line, with distance $|x - y|$, totally bounded?

Check. No. A totally bounded space is bounded, as we just proved, but the real line is not bounded: for any proposed bound M, the points 0 and $|M| + 1$ are at distance $|M| + 1$, which is larger than M. So the real line cannot be totally bounded.

17.3 Capsule two: keeping infinitely many indices

A sequence $x_0, x_1, x_2, \ldots$ has one term for each index $n \in \mathbb{N}$, so it has infinitely many indices, even when its values repeat. The sequence $x_n = (-1)^n$ takes only the two values 1 and -1, yet the value 1 occurs at the infinitely many indices $0, 2, 4, \ldots$. A constant sequence has only one value, which it takes at all of its infinitely many indices. If we counted points instead, the assistant's plan would fail for a constant sequence: all of its terms are the same point, so no ball would ever contain infinitely many different terms, even though a constant sequence converges at once to its value. So the proof keeps track of sets of *indices*, not sets of points. When we say that a ball contains infinitely many terms of a sequence, we mean that it contains x_n for infinitely many indices n, whether or not these terms are different points.

Recall from Section 3.1 that a set is *finite* if its members can be written down in a list that ends, and *infinite* otherwise. For example, $\mathbb{N}$ is infinite, and so is the set $\{0, 2, 4, \ldots\}$ of even natural

numbers.

Lemma 17.3 (Infinite pigeonhole principle)**.** *Let S be an infinite set, and let $A_1, \ldots, A_k$ be finitely many sets whose union contains S. Then at least one of the sets $S \cap A_1, \ldots, S \cap A_k$ is infinite.*

Proof. Every member of S lies in at least one of the sets A_i, so

$$S = (S \cap A_1) \cup (S \cap A_2) \cup \cdots \cup (S \cap A_k).$$

Suppose, for contradiction, that each of the sets $S \cap A_i$ is finite. Write down the members of $S \cap A_1$, then the members of $S \cap A_2$, and so on up to $S \cap A_k$. The result is a list that ends, and every member of S appears in it at least once. A member that lies in several of the sets A_i appears several times; crossing out the repetitions leaves a list of the members of S that still ends. So S is finite, which contradicts the assumption that S is infinite. □

Compare this with the pigeonhole principle of Proposition 7.4, which says that when many objects are placed in a few boxes, some box must hold many of them. Here the objects are the members of S and the boxes are the sets $A_1, \ldots, A_k$: infinitely many objects in finitely many boxes force some box to hold infinitely many. Unlike the boxes of Chapter 7, these boxes may overlap; as the proof shows, an object in two boxes simply appears twice in the list.

For example, take $x_n = (-1)^n$ and the two balls $B(1, 1/2)$ and $B(-1, 1/2)$ on the real line. Let A_1 be the set of indices n with $x_n \in B(1, 1/2)$, which is the set of even numbers, and let A_2 be the set of indices with $x_n \in B(-1, 1/2)$, which is the set of odd numbers. Their union is $\mathbb{N}$, and the lemma, applied with $S = \mathbb{N}$, promises that at least one of them is infinite, without saying which. Here both are.

In the proof of the theorem, S will be the current infinite set of indices, and A_i will be the set of indices n for which x_n lies in the ith ball of a finite cover of the space. Because the balls cover the space, these sets together contain every index, and the lemma supplies one ball that contains x_n for infinitely many indices n in S.

17.4 Capsule three: subsequences, and a plan for the proof

Definition 17.4 (Subsequence)**.** Let $x_0, x_1, x_2, \ldots$ be a sequence. A *subsequence* of it is a sequence of the form

$$x_{n_1}, x_{n_2}, x_{n_3}, \ldots, \qquad \text{where } n_1 < n_2 < n_3 < \cdots$$

are natural numbers.

In words: we may skip terms, but the terms we keep must stay in their original order, and no index may be used twice. For the sequence $x_n = (-1)^n$, which is $1, -1, 1, -1, \ldots$, the indices $n_k = 2k$ give the subsequence $x_2, x_4, x_6, \ldots$, which is the constant sequence $1, 1, 1, \ldots$. By contrast, $x_1, x_0, x_3, x_2, \ldots$ is not a subsequence, because the indices $1, 0, 3, 2, \ldots$ do not increase. Neither is $x_0, x_0, x_0, \ldots$, because it uses the index 0 again and again.

Two counters are involved, and it helps to keep them apart. The number k counts positions in the subsequence, while n_k is the position of the same term in the original sequence. In the example above, the third term of the subsequence, where $k = 3$, is x_6. We number the terms of a

subsequence from $k = 1$, while the sequences of Chapter 11 started at index 0. This makes no difference: writing $y_k = x_{n_k}$, the definitions of convergence and of Cauchy sequences apply to $y_1, y_2, y_3, \ldots$ exactly as before, with thresholds that are at least 1.

How can we make sure that the chosen indices increase? The following fact is all we need. *An infinite set S of natural numbers contains a number larger than any given natural number M.* Otherwise every member of S would lie in the set $\{0, 1, \ldots, M\}$. Crossing out of the list $0, 1, \ldots, M$ every number that is not in S would then leave a list of the members of S that ends, so S would be finite. So if, at each stage of a construction, an infinite set of allowed indices is available, we can always choose an allowed index larger than the one chosen at the previous stage. This is what makes the chosen terms a subsequence, and not just an unordered collection of terms.

A plan with three kinds of objects

The construction in the proof involves three different kinds of objects, and it is important not to mix them up: points of the space, such as the term x_7; balls, which are sets of points; and sets of indices, which are sets of natural numbers. A ball contains points. A set of indices contains numbers; in our construction it contains the number 7 when the term x_7 lies in the chosen ball. If $x_7 = x_9$, this is one point of the ball, but it contributes two indices, 7 and 9.

Here is the plan. Start with every index allowed: $S_0 = \mathbb{N}$. At stage 1, cover the space by finitely many balls of radius $1/4$, which total boundedness makes possible. By Lemma 17.3, one of these balls contains x_n for infinitely many indices $n \in S_0$. Keep exactly those indices, and call the set of them S_1. By the diameter bound, any two terms x_i and x_j with $i, j \in S_1$ are less than $2 \cdot \frac{1}{4} = \frac{1}{2}$ apart. At stage 2, cover the space by finitely many balls of radius $1/8$, but now consider only the indices in S_1. One of the new balls contains x_n for infinitely many $n \in S_1$; keep those indices, and call the set of them S_2. Any two terms with indices in S_2 are less than $2 \cdot \frac{1}{8} = \frac{1}{4}$ apart.

In general, stage k uses balls of radius $2^{-(k+1)}$, and any two terms with indices in S_k are less than $2 \cdot 2^{-(k+1)} = 2^{-k}$ apart. Why the radius $2^{-(k+1)}$ and not 2^{-k}? The diameter bound doubles the radius, so starting from half of the desired distance gives the simple bound 2^{-k}. Any radii that shrink towards zero would work; this choice keeps the arithmetic tidy.

Each set S_k is a subset of S_{k-1}, because stage k only throws indices away and never adds new ones. So

$$S_0 \supseteq S_1 \supseteq S_2 \supseteq \cdots,$$

where $A \supseteq B$, read "A contains B," means the same as $B \subseteq A$. We say that the index sets are *nested*. The chosen balls, by contrast, need not be nested: the ball chosen at stage 2 may stick out of the ball chosen at stage 1. That does no harm. A term whose index lies in S_2 also has its index in S_1, so it lies in both chosen balls. In this way the nesting of the index sets carries the distance bound of each stage forward to all later stages. Section 17.5 makes this precise.

17.5 The complete proof

Theorem 17.5. *Let (X, d) be a complete, totally bounded metric space. Then every sequence $x_0, x_1, x_2, \ldots$ in X has a subsequence that converges to a point of X.*

Unlike the contraction theorem (Theorem 12.2), this theorem does not assume that X is nonempty: if X has no points, there is no sequence in X at all, and the statement is vacuously true (Section 2.5).

Proof. Let $x_0, x_1, x_2, \ldots$ be a sequence in X. The proof has five steps.

Step 1: nested infinite sets of indices. We construct, one stage at a time, infinite sets of indices $S_0 \supseteq S_1 \supseteq S_2 \supseteq \cdots$ such that, for every $k \geq 1$,

$$d(x_i, x_j) < 2^{-k} \qquad \text{for all } i, j \in S_k. \tag{17.1}$$

Start with $S_0 = \mathbb{N}$, which is infinite. Now let $k \geq 1$, and suppose that S_{k-1} has already been constructed and is infinite. Since X is totally bounded, finitely many balls of radius $2^{-(k+1)}$ cover X. For each of these balls, consider the set of indices n for which x_n lies in that ball. Every term x_n lies in at least one of the balls, so every index lies in at least one of these finitely many sets. By Lemma 17.3, applied to the infinite set S_{k-1}, there is a ball B among them such that the set

$$S_k = \{n \in S_{k-1} : x_n \in B\}$$

is infinite. By its definition, $S_k \subseteq S_{k-1}$. If $i, j \in S_k$, then x_i and x_j both lie in the ball B of radius $2^{-(k+1)}$, so the diameter bound gives $d(x_i, x_j) < 2 \cdot 2^{-(k+1)} = 2^{-k}$. This proves (17.1) for this value of k. Since S_k is again infinite, the next stage can begin, and the construction continues for $k = 1, 2, 3, \ldots$.

Step 2: increasing indices. Choose any $n_1 \in S_1$. For $k \geq 2$, once n_{k-1} has been chosen, choose $n_k \in S_k$ with $n_k > n_{k-1}$. Such an index exists because S_k is infinite, and an infinite set of natural numbers contains a number larger than any given one (Section 17.4). Then $n_1 < n_2 < n_3 < \cdots$, so the terms $x_{n_1}, x_{n_2}, x_{n_3}, \ldots$ form a subsequence. By construction, $n_k \in S_k$ for every $k \geq 1$.

Step 3: later terms keep the earlier bounds. Let $1 \leq k \leq \ell$. The sets are nested, so $S_\ell \subseteq S_{\ell-1} \subseteq \cdots \subseteq S_k$, and therefore $n_\ell \in S_\ell \subseteq S_k$. So not only the term chosen at stage k, but every term chosen at a later stage, has its index in S_k. In particular, if $\ell \geq k$ and $m \geq k$, then n_ℓ and n_m both lie in S_k, and (17.1) gives

$$d(x_{n_\ell}, x_{n_m}) < 2^{-k}.$$

Step 4: the subsequence is a Cauchy sequence. Let $\varepsilon > 0$. By the Archimedean property (Section 2.3), choose an integer K with $K > 1/\varepsilon$. It is positive, because $1/\varepsilon > 0$, so $K \geq 1$. This matters, because the bound (17.1) holds only from stage 1 on; the set $S_0 = \mathbb{N}$ carries no information about distances. Section 11.3 showed that $2^K \geq K + 1$, so

$$2^{-K} \leq \frac{1}{K+1} < \frac{1}{K} < \varepsilon.$$

For any $\ell, m \geq K$, Step 3 with $k = K$ gives $d(x_{n_\ell}, x_{n_m}) < 2^{-K} < \varepsilon$. So K is a threshold that works for the tolerance ε. Since $\varepsilon > 0$ was arbitrary, the subsequence is a Cauchy sequence.

Step 5: completeness supplies the limit. The space X is complete, so the Cauchy sequence $x_{n_1}, x_{n_2}, x_{n_3}, \ldots$ converges to some point x_* of X. This is the required subsequence. □

Watching the construction on an example

The interval $[-1, 2]$, with distance $|x - y|$, is complete, because it is a closed interval (Section 11.6). It is totally bounded by the same argument as for $[0, 1]$ in Section 17.2; Exercise 17.4 asks you to write out the details for every interval $[a, b]$ with $a < b$. In this space consider the sequence

$$x_n = (-1)^n + \frac{1}{n+1},$$

whose first terms are $x_0 = 2$, $x_1 = -\frac{1}{2}$, $x_2 = \frac{4}{3}$, $x_3 = -\frac{3}{4}$ and $x_4 = \frac{6}{5}$. The even-indexed terms $1 + \frac{1}{n+1}$ crowd towards 1 from above, and the odd-indexed terms $-1 + \frac{1}{n+1}$ crowd towards -1, never rising above $-\frac{1}{2}$.

At stage 1, cover $[-1, 2]$ by the thirteen balls of radius $1/4$ centered at $-1, -\frac{3}{4}, -\frac{1}{2}, \ldots, \frac{7}{4}, 2$. These balls do cover the interval: a point of $[-1, 2]$ that is not a center lies strictly between two consecutive centers, which are $1/4$ apart, so its distance from the left one is less than $1/4$. The ball $B(1, 1/4)$ is one of the thirteen. It contains x_n exactly when n is even and $\frac{1}{n+1} < \frac{1}{4}$, that is, when n is even and $n \geq 4$. So the construction may take $S_1 = \{4, 6, 8, \ldots\}$. Other balls also contain infinitely many terms, for instance $B(-1, 1/4)$, which contains x_n for every odd $n \geq 5$, and the construction is free to pick any of them.

At stage 2, use the balls of radius $1/8$ centered at the points $-1, -\frac{7}{8}, \ldots, 2$, which include 1. Among the indices in S_1, the ball $B(1, 1/8)$ contains x_n exactly when $\frac{1}{n+1} < \frac{1}{8}$, that is, when $n \geq 8$. So we may take $S_2 = \{8, 10, 12, \ldots\}$, which lies inside S_1. In the same way, stage k can use the balls of radius $2^{-(k+1)}$ centered at the points $-1 + j \cdot 2^{-(k+1)}$ for $j = 0, 1, \ldots, 3 \cdot 2^{k+1}$, and 1 is always one of these centers: $j = 2^{k+2}$ gives $-1 + 2^{k+2} \cdot 2^{-(k+1)} = -1 + 2 = 1$. If we keep choosing the ball around 1, stage k gives the set S_k of even numbers that are at least 2^{k+1}, because $\frac{1}{n+1} < 2^{-(k+1)}$ means $n + 1 > 2^{k+1}$, that is, $n \geq 2^{k+1}$.

Now choose the indices of Step 2, taking each time the smallest allowed index. This gives $n_1 = 4$, then $n_2 = 8$ (the smallest member of S_2 larger than 4), then $n_3 = 16$, and in general $n_k = 2^{k+1}$. The subsequence is $x_4, x_8, x_{16}, \ldots$, with terms $1 + \frac{1}{2^{k+1}+1}$, and it converges to 1. Had stage 1 chosen the ball $B(-1, 1/4)$ instead, every later index would have been odd, and the construction would have produced a subsequence converging to -1. The theorem promises *some* convergent subsequence, not a particular one. In this example the indices follow the simple formula $n_k = 2^{k+1}$; in general there is no such simple formula. What matters is only that each n_k lies in S_k and is larger than n_{k-1}.

Why the proof picks a new index at every stage

It is tempting to look for a single index n that lies in every set S_k, so that the one term x_n lies in every chosen ball. In general there is no such index, because nested infinite sets can have no member in common. For example, the sets $\{k, k+1, k+2, \ldots\}$ for $k = 1, 2, 3, \ldots$ are infinite and nested, but no natural number n belongs to all of them, since n is not a member of $\{n+1, n+2, \ldots\}$. The same happens in the construction we just watched. There S_k contains only numbers that are at least 2^{k+1}, and a natural number n is smaller than 2^{k+1} as soon as $k \geq n$, because $2^{k+1} > 2^k \geq k + 1 > n$. So no index lies in every S_k. Even when such an index exists, as it does for a constant sequence, it gives only one term, not a subsequence. This is why the proof picks a new, later index from each

set in turn. Step 3 shows that any two sufficiently late picks lie together in one earlier set, and that is exactly what the Cauchy condition asks for.

Pause and decide. Where does the proof use each hypothesis of Theorem 17.5?

Check. There are two hypotheses. Total boundedness is used at every stage of Step 1, to cover X by finitely many balls of radius $2^{-(k+1)}$. The finiteness of that cover is what allows Lemma 17.3 to keep infinitely many indices. Completeness is used once, in Step 5, to give the Cauchy subsequence a limit in X. The other ingredients, namely the diameter bound, the nesting of the index sets and the increasing choice of indices, are not hypotheses. They are facts proved along the way.

17.6 Why neither hypothesis can be dropped

As in Section 12.6, we remove one hypothesis at a time, keep the other, and look for an example in which the conclusion fails.

Without total boundedness

Give $\mathbb{N}$ the discrete metric of Section 17.2, in which different numbers are at distance 1. This space is complete. To see why, take a Cauchy sequence in it and apply the Cauchy condition with the tolerance $1/2$. It supplies a threshold N such that any two terms from x_N onward are at distance less than $1/2$. Distances in this space are only ever 0 or 1, so all these distances are 0, and every term from x_N onward equals x_N. Such a sequence converges to x_N, a point of the space: for every $\varepsilon > 0$ and every $n \geq N$, we have $d(x_n, x_N) = 0 < \varepsilon$.

The space is not totally bounded, as Section 17.2 showed. And the conclusion of the theorem fails. Consider the sequence $x_n = n$, that is, $0, 1, 2, \ldots$. In any subsequence $x_{n_1}, x_{n_2}, \ldots$ the indices increase, so the terms $n_1, n_2, \ldots$ are different numbers, and any two of them are at distance 1. For the tolerance $1/2$, no threshold can work, so the subsequence is not a Cauchy sequence. Since every convergent sequence is a Cauchy sequence (Proposition 11.5), the subsequence does not converge. In the proof, it is Step 1 that breaks down: no finite collection of balls of radius $1/4$ covers the space, so the selection cannot even begin.

Without completeness

The interval $(0, 1)$, with distance $|x - y|$, is totally bounded. The centers 0 and 1 used for $[0, 1]$ are not allowed here, because the centers must be points of the space, so we use midpoints instead. Given $r > 0$, the Archimedean property supplies an integer $N > 1/(2r)$, which is positive and satisfies $1/(2N) < r$. Split $[0, 1]$ into the N pieces $\left[\frac{j-1}{N}, \frac{j}{N}\right]$ for $j = 1, \ldots, N$. Their midpoints $\frac{2j-1}{2N}$ lie strictly between 0 and 1, so they are points of $(0, 1)$. Every $x \in (0, 1)$ lies in one of the pieces. A point of a piece is at most half the length of the piece away from its midpoint, so x is at distance at most $1/(2N) < r$ from that midpoint.

But $(0, 1)$ is not complete (Section 11.6), and the conclusion of the theorem fails. Consider the sequence $x_n = 1/(n + 2)$, that is, $\frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \ldots$, whose terms all lie in $(0, 1)$. On the real line it

converges to 0: given $\varepsilon > 0$, choose an integer $N > 1/\varepsilon$; then every $n \geq N$ gives $x_n = \frac{1}{n+2} < \frac{1}{N} < \varepsilon$.

Every subsequence $x_{n_1}, x_{n_2}, \ldots$ also converges to 0 on the real line. The indices increase by at least one at every step, starting from $n_1 \geq 0$, so induction gives $n_k \geq k - 1$ for every $k \geq 1$. Hence, with ε and N as above, every $k \geq N + 1$ has $n_k \geq N$ and therefore $x_{n_k} < \varepsilon$. Now suppose some subsequence converged to a point L of $(0, 1)$. Distances in $(0, 1)$ are distances on the real line, so the subsequence would converge to L on the real line too. It also converges to 0 there, so uniqueness of limits (Proposition 11.3) gives $L = 0$, which is not a point of $(0, 1)$. This contradiction shows that no subsequence converges in $(0, 1)$. In the proof, Steps 1–4 go through, and it is Step 5 that fails: the Cauchy subsequence exists, but the point it crowds around is missing from the space.

Together, the two examples show that the hypotheses do different jobs: total boundedness produces a Cauchy subsequence, and completeness gives it a limit. When you meet a new hypothesis, ask not only "What does it mean?" but also "Which step fails without it?"

17.7 The prerequisite map after the work is done

The assistant's list of prerequisites in Section 17.1 turned out to be incomplete. It mentioned finite covers, the infinite pigeonhole principle and increasing indices. It did not mention the diameter bound for balls, and it did not explain why nested selections make the chosen terms a Cauchy sequence, yet the proof needs both: the diameter bound in Step 1, and the nesting argument in Step 3. Gaps like these turn up when you rebuild a proof line by line, and finding them is part of the method: add each missing step to the map, learn it, and return to the argument. A list of prerequisites deserves your trust for a particular proof only once you have used it to rebuild that proof.

With every piece in place, the map of the proof can be written out in full. Each row of the table names one part of the argument, the facts that this part uses, and the conclusion it reaches.

Where	Facts used	Conclusion
Capsule one	triangle inequality and symmetry	diameter bound for balls
Capsule two	definition of a finite set	infinite pigeonhole principle (Lemma 17.3)
Capsule three	definition of a finite set	an infinite set of natural numbers contains a number larger than any given one
Step 1	total boundedness, infinite pigeonhole principle, diameter bound	nested infinite index sets S_k satisfying (17.1)
Step 2	the sets S_k are infinite, and an infinite set of natural numbers contains a number larger than any given one	increasing indices $n_k \in S_k$, hence a subsequence
Step 3	Step 2 (each n_ℓ lies in S_ℓ), nesting of the sets S_k, the bound (17.1)	terms from position k on are less than 2^{-k} apart
Step 4	Step 3, Archimedean property, $2^K \geq K + 1$	a Cauchy subsequence
Step 5	Step 4, completeness	a subsequence that converges to a point of the space

You can also read this map as a directed graph in the sense of Section 7.5. Each fact is a vertex, and there is an arrow from one fact to another when the first is used to obtain the second. For instance, Step 1 contributes arrows from total boundedness, from the infinite pigeonhole principle and from the diameter bound to the nested index sets. The hypotheses of the theorem and the facts brought in from earlier chapters have no arrows coming in, because nothing in this proof derives them. The final conclusion, a convergent subsequence, has no arrows going out: it is a sink.

Some of the ingredients were familiar before this chapter began. The triangle inequality, Cauchy sequences, completeness and the inequality $2^K \geq K + 1$ come from Chapter 11, the Archimedean property comes from Chapter 2, and the definition of a finite set comes from Chapter 3. The ordinary pigeonhole principle, on which the infinite version is modeled, comes from Chapter 7. The new ingredients were balls and their diameter bound, total boundedness, the infinite pigeonhole principle, subsequences, and the nested selection that ties them together. Each capsule pointed ahead to the step of the proof that uses it, and Section 17.5 then put every piece to work. That return to the proof matters: a concept you have seen doing its job in a proof stays with you, while one learned and left unused becomes disconnected vocabulary.

Check what you have learned, not only what you can name

How can you tell whether you have learned a new concept well enough for the proof at hand? Here is a check in four parts, carried out for total boundedness.

1. *State the problem the concept solves.* The proof needs to split the infinitely many indices into finitely many groups whose terms are close together, at every scale. Total boundedness provides, for any requested radius, a finite list of balls of that radius that covers the space, and

the balls give the groups.

2. *State the definition with its quantifiers in the right order.* The radius comes first. The list, and its length, may depend on the radius, but for that one radius the same list must cover every point.
3. *Test it on examples near the boundary.* The discrete metric on $\mathbb{N}$ shows that a bound on all distances is not enough to give finite covers by small balls. The interval $(0, 1)$ shows that finite covers by small balls do not provide the missing limits.
4. *Return to the exact line of the proof where the concept is used,* here the beginning of each stage of Step 1, and explain it. Why must the cover be finite? Because Lemma 17.3 applies only to finitely many sets. With infinitely many balls, each ball might contain just one term, as in the discrete metric on $\mathbb{N}$, and no ball would keep infinitely many indices.

If you can recite the definition but cannot explain that line, you have not yet learned the concept well enough for this proof.

Labels for the entries of the map

For a genuinely unfamiliar paper, it helps to mark each entry of the map with its status, as in the section "Keeping track of evidence" of *To the reader*.

- Mark a prerequisite as *understood* only when it passes the four-part check above and you can also use it in a setting slightly different from the one in which you learned it. After studying the interval $[0, 1]$ in Section 17.2, proving that every interval $[a, b]$ is totally bounded (Exercise 17.4) is a small test of this kind.
- Mark an entry as *can follow* when you can check each of its steps but could not yet explain it, or use it in a new setting, on your own. For instance, you might check every line of the proof of Lemma 17.3 and still be unable to say, without looking, which line would fail if there were infinitely many sets A_i.
- Mark a result as *borrowed* when you use it without having studied its proof. A borrowed result must still be applied correctly: check that its hypotheses hold in your situation and that its conclusion is exactly what you need. Learning its proof can be a separate task for later. In this chapter, the completeness of the real line is borrowed. Chapter 11 accepted it as a starting fact, and the completeness of the interval $[-1, 2]$ in the example of Section 17.5 rests on it.
- Mark an entry as *open* while you cannot yet follow it. An open entry records exactly what remains to be learned. Section 17.8 keeps a labeled record of this kind for a research proof, and much of it is still open.

A map may contain borrowed results and open entries, as long as it labels them clearly.

17.8 Reading a formalized research proof with the same method

We now apply the same method to the two research claims met earlier in the book: Seymour's second-neighborhood conjecture (Section 7.7) and the circulant Hadamard conjecture (Section 14.7). Claimed proofs of both appear in OpenAI's public mathematics repository [2]. The repository collects preprints, in the sense of Section 7.7, together with formal proofs in Lean. For each problem it also describes exactly which statements were formalized. That a paper appears in the repository does not, by itself, vouch for every claim in the paper. The right unit of reading is one exact theorem, together with its definitions and the results it depends on, not a title or an impressive final line.

Decoding the statement

Start with Seymour's conjecture. The repository's formal statement is about a nonempty finite oriented graph, and it asks for a vertex whose second out-neighborhood has at least as many vertices as its first [3]. Section 7.7 supplies the mathematical vocabulary, and Section 16.4 showed how the exclusions in the definition of a second out-neighbor are written in Lean. The official statement lives in a file of its own [6], and the theorem is called `exists_goodVertex`. That file declares the assumptions on separate lines and uses the symbols → and ∃. Collecting the assumptions in one place, writing `->` and `Exists` as in Chapter 16, and putting `Type` where the file has `Type*` (a technical detail about the sizes of types that makes no difference here), we can display the statement as follows.

```
theorem exists_goodVertex {V : Type}
    [Fintype V] [DecidableEq V] [Nonempty V]
    (r : V -> V -> Prop) (hr : IsOriented r) :
    Exists (fun v => GoodVertex r v) := by
  sorry
```

The repository builds on Mathlib, the large mathematical library mentioned in Chapter 15, which provides finite sets and ways of counting their members. Reading the statement piece by piece, we find the following.

- `V` is the type of vertices. Three assumptions are written in square brackets. `[Fintype V]` says that there are only finitely many vertices, and `[Nonempty V]` says that there is at least one. `[DecidableEq V]` says that, for any two vertices, it can be decided whether they are equal. This last assumption is a technical requirement of Mathlib's finite sets, and it does not exclude any graph: with the standard axiom `Classical.choice` of Section 16.6, Lean can supply it for every type.
- The edge relation `r` has type `V -> V -> Prop`, so `r v w` is the proposition $v \to w$, exactly as in Section 16.4.
- The explicit hypothesis `(hr : IsOriented r)` says that `r` has no loops and no pair of opposite edges. The definition of `IsOriented` has exactly these two parts, named `loopless` and `asymmetric`.

- The conclusion says that some vertex `v` satisfies `GoodVertex r v`. This is defined to mean that the set `firstNeighbors r v` has at most as many members as the set `secondNeighbors r v`. The first of these sets consists of the vertices `w` with `r v w`, that is, the out-neighbors of v. The second is defined by exactly the three conditions of Section 7.7: $w \neq v$, $v \to w$ fails, and $v \to u \to w$ for some vertex u.

Each piece of the formal statement matches a piece of the informal one: "nonempty" and "finite" are the bracketed assumptions, "oriented" is `IsOriented`, and the inequality $|N_2^+(v)| \geq |N_1^+(v)|$ is `GoodVertex`. No condition of the informal statement is missing. Reading the statement in this way needed nothing beyond Chapters 7, 15 and 16. After Section 16.6, the word `sorry` at the end may look alarming. It belongs there: this file only records what must be proved, and the proof is in another file.

Locating the main steps

The proof is in a different file [5]. That file imports three further files, named `Reduction`, `Pruning` and `ExtremalContradiction`, and its own proof is only a few lines long. It argues by contradiction. It assumes that no vertex is good, so that the graph is a counterexample; it applies a theorem from the reduction file to turn the counterexample into a new oriented graph with additional properties; and it then applies results from the other two files to reach a contradiction.

The reduction theorem [7] says that from any counterexample one can build, for some positive integer n, an oriented graph whose vertices are the pairs (i, j) of natural numbers with $i < n$ and $j \leq n$, and which has two further properties, named `PositiveIndegree` and `StrictSubsetGrowth`. The reduction file in turn imports three more files, so the map keeps branching. The name `PositiveIndegree` suggests that every vertex has at least one edge coming in, but a name is not a definition: as Section 15.2 showed, Lean accepts any name for any statement. To understand these properties, you have to open their definitions and read them.

Keeping a reading record

A *reading record* is a written account of a proof you are studying: what you have read, what you have understood, and what remains open. Section E.5 describes what a finished one contains. At this point, a reading record of the Seymour proof has entries of three different kinds.

- *Understood:* the final statement has been decoded, and its definitions match the informal conjecture.
- *Borrowed:* the logical role of the reduction is clear, since it turns a counterexample into a graph with extra properties, but its proof has not been read.
- *Open:* the meaning of `PositiveIndegree` and `StrictSubsetGrowth`, and the mathematics of the pruning step and of the final contradiction, remain to be studied from their sources.

The proof as a whole is long and uses ideas well beyond this course. A dependency sketch like this one is a way into the proof, not a replacement for it.

What the formal check compares

How does the repository connect the official statement to the proof? A configuration file [8] lists what the formal check compares: the file containing the official statement, the file containing the proof, the name of the theorem, and the axioms the proof may use. The permitted axioms are `propext`, `Quot.sound` and `Classical.choice`, the three standard axioms of Section 16.6. The check is designed to confirm that the proof file proves exactly the official statement, using no axioms beyond the permitted ones. It is the two statements that are compared, not merely their names: a theorem in the proof file with the same name as the official one would, by itself, say nothing about whether the two statements agree.

Reading the configuration file tells you what the check is designed to compare; it does not tell you that the check passed. Unless you run the check yourself, the statement that it passed is a claim made by the repository, and your reading record should attribute it: "the repository reports that the check passes," rather than "the check passes." In the same way, the record can describe the role of a file whose proofs you have not read, but it cannot vouch for those proofs. What it can report on your own authority is what you decoded or reconstructed yourself.

The circulant Hadamard conjecture

For the circulant Hadamard conjecture of Chapter 14, Section 14.7 has already read the repository's description of exactly which statements were formalized [4]. You have also worked through the proof of a necessary condition yourself: by Proposition 14.7, a circulant Hadamard matrix of order greater than 4 would need an order $4u^2$ with $u \geq 2$, such as 16, 36 or 64. That gives a precise next question: which lemma of the proof rules out these orders? A question informed by mathematics you have done yourself will get you much further than asking an assistant to "explain the whole repository."

Exercises

Exercises 17.1–17.4 are core exercises, and Exercises 17.5 and 17.6 are transfer exercises. Complete solutions are in Appendix C, under "Week 17."

Exercise 17.1. Suppose a ball contains x_n for infinitely many indices n. Explain why it does not follow that the sequence takes infinitely many different values in that ball. Does the proof of Theorem 17.5 need the sequence to take infinitely many different values?

Exercise 17.2. An assistant writing out the proof of Theorem 17.5 chooses an arbitrary $n_k \in S_k$ at each stage, without requiring $n_k > n_{k-1}$. Identify the gap and repair the choice. Explain why the repair is always possible.

Exercise 17.3. Let X be any set with the discrete metric of Section 17.2. (a) Prove that X is complete. (b) Prove that X is totally bounded if and only if X is finite. (c) Prove that a sequence $x_0, x_1, x_2, \ldots$ in X has a subsequence that converges in X exactly when there is a point c of X with $x_n = c$ for infinitely many indices n. Use this to explain, for $X = \mathbb{N}$ and the sequence $x_n = n$, the difference between completeness and the conclusion of Theorem 17.5.

Exercise 17.4. Let $a < b$ be real numbers. Give a complete proof that the interval $[a, b]$, with distance $|x - y|$, is totally bounded. State exactly what the number of balls in your cover may depend on, and what it may not depend on.

Exercise 17.5. A directed graph has the three vertices v, u, w and the edges $v \to u$, $u \to w$ and $v \to w$. Let $R_2(v)$ be the set of vertices z for which there is a route $v \to y \to z$ through some vertex y. Find $N_1^+(v)$, $N_2^+(v)$ and $R_2(v)$. Then explain what would go wrong if a formal statement of Seymour's conjecture used $R_2(v)$ in place of $N_2^+(v)$.

Exercise 17.6. Write a minimal-background request, in the sense of Section 17.1, for the proof of the contraction theorem (Theorem 12.2), and another for the proof of Theorem 17.5. For each of the two proofs, name a concept that it needs but the other proof does not. Then state what a reading record of a formalized research proof may honestly claim if you have not run its formal check yourself.

CHAPTER 18

A complete AI-assisted investigation

This week

Take the contraction theorem of Chapter 12 and change one of its assumptions: suppose that every update is carried out with a small error. Test a first guess about what happens, find a statement that is actually true, prove it, ask whether its constant can be improved, and check it against an exact computation. Then write the investigation up so that another reader can check every claim in it, and plan an investigation of your own.

18.1 The question that starts the project

The contraction theorem of Chapter 12 is about exact updates: each new point is $x_{n+1} = T(x_n)$, computed without any error. Real computations are rarely that clean. A computer keeps only finitely many digits, so it has to round each value of T before storing it; a calculator that keeps two decimal places might store 0.87 where the exact value is 0.875. A measuring instrument has limited precision, and a value produced by a separate numerical procedure is itself only approximate. In each case the new point is close to T of the old point, but not equal to it. So we ask: does the iteration still approach the fixed point? And if it does not, can we at least say how far from the fixed point it can end up?

This is a good question for a first investigation: it changes one feature of a theorem we already understand and keeps everything else. The chapter carries out the whole investigation as a worked example, from a first guess to a proved result with clearly stated limits. The bounds we reach are standard consequences of the contraction theorem. What matters here is the way the investigation proceeds.

The setting

Throughout the chapter, (X, d) is a nonempty complete metric space, and $T : X \to X$ is a contraction with factor q, where $0 < q < 1$:

$$d(Tx, Ty) \le q\, d(x, y) \qquad \text{for all } x, y \in X.$$

Asking for $q > 0$ costs nothing. A contraction with factor 0 satisfies $d(Tx, Ty) \le 0 \le \frac{1}{2} d(x, y)$, so it is also a contraction with factor $1/2$. In return, every power q^k that appears below is a positive number, and we never have to think about what 0^0 should mean. By Theorem 12.2, T has exactly one fixed point, which we call x_*.

In place of the exact iterates we now have points $y_0, y_1, y_2, \ldots$ of X and nonnegative numbers

$\varepsilon_0, \varepsilon_1, \varepsilon_2, \ldots$ such that

$$d(y_{n+1}, T(y_n)) \le \varepsilon_n \qquad \text{for every } n \in \mathbb{N}.$$

The starting point y_0 may be any point of X. The point y_{n+1} is what the update actually produced when it should have produced $T(y_n)$, and the number ε_n bounds the error made in that update. For short, we call ε_n the *update error* at step n; strictly speaking it is an upper bound for that error, and an upper bound is all we will need. If every ε_n is zero, then $y_{n+1} = T(y_n)$ for every n, and we are back to the exact iteration of Chapter 12.

Apart from these bounds, we assume nothing about the errors. The bounds ε_n may change from step to step, the errors need not follow any pattern, and they are not assumed to be random.

One more assumption is easy to overlook: every y_n must be a point of X. An error bound alone cannot guarantee this. Suppose, for instance, that $X = [0, 1]$ and a computed value comes out as -0.001. This number is very close to the interval, but it is not a point of X. The distance $d(y_{n+1}, T(y_n))$ is then not even defined, because d only measures distances between points of X, and T cannot be applied at the next step. In an application, the requirement $y_n \in X$ has to be checked separately, just like the self-map condition of Section 12.1.

18.2 Testing the first guess

A natural first guess is that small update errors do no lasting harm: if every ε_n is small, the points y_n should still converge to x_*. Before trying to prove a guess, it pays to try to break it. Here is how a short exchange with an AI assistant might do that.

Reader: I expect that small update errors still give convergence to x_*. Before I try to prove this, try to disprove it.

Assistant: A constant bias can shift the limit. Take $T(x) = qx$ on $\mathbb{R}$, whose fixed point is 0, and add the same small number $\eta > 0$ at every update: $y_{n+1} = qy_n + \eta$. Then y_n approaches $\eta/(1-q)$, not 0, however small η is.

Reader: Then convergence is the wrong target. Is there an error bound that still holds? And can it tell apart errors that stay bounded from errors that tend to zero?

Assistant: Start by bounding the distance from the next point to x_* by q times the present distance plus the update error. Then apply that bound over and over.

The assistant's two replies make two different contributions, and neither has been checked yet. The first reply is a counterexample built on a *constant bias*, an error that is the same at every update, like the number η here. Its hypotheses are easy to verify. Since $|T(x) - T(y)| = |qx - qy| = q|x - y|$, the map $T(x) = qx$ is a contraction of $\mathbb{R}$ with factor q. Its only fixed point is 0, because $qx = x$ gives $(1-q)x = 0$, and $1 - q \neq 0$. The error made in each update is

$$|y_{n+1} - T(y_n)| = |qy_n + \eta - qy_n| = \eta,$$

so every update error is $\varepsilon_n = \eta$, and η can be chosen as small as we like. A small case makes the conclusion plausible. With $q = 1/2$, $\eta = 1$ and $y_0 = 0$, the points are

$$0,\ 1,\ \tfrac{3}{2},\ \tfrac{7}{4},\ \tfrac{15}{8},\ \tfrac{31}{16},\ \ldots,$$

which seem to approach $2 = \eta/(1-q)$ rather than the fixed point 0. A few terms do not prove a limit, so Section 18.4 proves an exact formula for y_n.

The second reply is a plan for a proof. Section 18.3 carries it out: we derive the bound on the next distance ourselves and prove by induction what we get by applying it over and over. Section 18.4 then asks whether the resulting constant can be improved.

The counterexample has changed the question. Instead of the single question whether $y_n \to x_*$, we now ask three. How far from x_* can the points be? Under what conditions does that distance still tend to zero? And when it does not, where do the points end up? The constant-bias example shows that the three questions really are separate: the distance stays bounded, it does not tend to zero, and, as Section 18.4 will prove, the points converge to a different point, $\eta/(1-q)$. Habit 2 (Section 2.6) asks us to record exactly this kind of change in the statement contract. The hypotheses are the same as before, but the conclusion has been replaced by a more modest one of a different kind, an error bound, together with a separate question about when convergence survives.

Pause and decide. A calculator keeps two decimal places and always rounds down. It runs the update $x \mapsto (x+1)/2$, whose fixed point is 1, starting from 0. Each rounding error is less than 0.01. Do the stored values converge to 1?

Check. No. The stored values are 0, 0.5, 0.75, 0.87, 0.93, 0.96, 0.98, 0.99, 0.99, ... For instance, $(0.75+1)/2 = 0.875$ is stored as 0.87. Once the value 0.99 is reached, the exact update gives $(0.99+1)/2 = 0.995$, which is stored as 0.99 again. From then on the values never change: they converge to 0.99 and stay at distance 0.01 from the fixed point. Rounding down acts like a small bias that always pushes in the same direction, just as in the assistant's example.

18.3 The one-step estimate

Write

$$e_n = d(y_n, x_*)$$

for the distance from the nth point to the fixed point. We call e_n the *error* at step n. Keep the two kinds of error apart: e_n measures how far the point y_n is from x_*, while the update error ε_n measures how badly a single update went wrong.

To compare e_{n+1} with e_n, insert the exact update $T(y_n)$ as a stop between y_{n+1} and x_*:

$$\begin{aligned}
e_{n+1} &= d(y_{n+1}, x_*) \\
&\le d(y_{n+1}, T(y_n)) + d(T(y_n), x_*) && \text{triangle inequality} \\
&= d(y_{n+1}, T(y_n)) + d(T(y_n), T(x_*)) && \text{since } T(x_*) = x_* \\
&\le \varepsilon_n + q\, d(y_n, x_*) && \text{update error and contraction inequality} \\
&= q e_n + \varepsilon_n.
\end{aligned}$$

So for every $n \in \mathbb{N}$ we have the *one-step estimate*

$$e_{n+1} \le q e_n + \varepsilon_n.$$

The computation uses four facts, each of them once: the triangle inequality of the metric, the fixed-point equation $T(x_*) = x_*$, the bound on the update error, and the contraction inequality. Completeness enters only indirectly, through the existence of x_*. Habit 3 (Section 3.4) asks which information an argument actually uses. Here the list is short, and that shows how far the estimate reaches: X need not be a set of numbers, and the update errors need not follow any pattern.

Unfolding the estimate three times

The one-step estimate compares each error only with the one just before it. To see how an update error made early on affects later steps, apply the estimate repeatedly, substituting each bound into the next. Every substitution multiplies an inequality by q, which keeps the direction of the inequality because $q > 0$:

$$\begin{aligned} e_1 &\le q e_0 + \varepsilon_0, \\ e_2 &\le q e_1 + \varepsilon_1 \le q(q e_0 + \varepsilon_0) + \varepsilon_1 = q^2 e_0 + q\varepsilon_0 + \varepsilon_1, \\ e_3 &\le q e_2 + \varepsilon_2 \le q(q^2 e_0 + q\varepsilon_0 + \varepsilon_1) + \varepsilon_2 = q^3 e_0 + q^2\varepsilon_0 + q\varepsilon_1 + \varepsilon_2. \end{aligned}$$

At step three, the starting error e_0 has been multiplied by q three times, once for each update. The update error ε_0, made in the first update, has been multiplied by q twice, once for each of the two updates that came after it. The most recent update error ε_2 has not yet been reduced by any later update, so its coefficient is one.

The same counting works at every step. The update error ε_j is made while producing y_{j+1}. After that, the updates that produce $y_{j+2}, \dots, y_n$ follow, and there are $n - (j+1) = n - 1 - j$ of them. Each multiplies the effect of ε_j by q, so in the bound for e_n the error ε_j carries the weight q^{n-1-j}. With $n = 3$ this gives the weights q^2, q and 1 found above. The earliest error, ε_0, has weight q^{n-1}, and the latest, ε_{n-1}, has weight $q^0 = 1$; the sum in the theorem below runs between these two ends, from $j = 0$ to $j = n - 1$.

Two assumptions about the update errors

The theorem makes two different assumptions about the update errors. The first is that they are *bounded*: there is a fixed number $\eta \ge 0$ with $\varepsilon_j \le \eta$ for every j. Such errors may stay the same size forever, like the constant bias of Section 18.2. The second assumption is that they *tend to zero*: $\varepsilon_j \to 0$ in the sense of Definition 11.2, which here means that for every tolerance $\delta > 0$, all the update errors from some index onward are smaller than δ. (We write δ for the tolerance because the letter ε already names the update errors.)

Errors that tend to zero are automatically bounded, so the second assumption is the stronger one. To see this, take the tolerance 1: from some index J on, every update error is below 1. Only the finitely many errors $\varepsilon_0, \dots, \varepsilon_{J-1}$ come before J, so the largest of the numbers $1, \varepsilon_0, \dots, \varepsilon_{J-1}$ is a bound for all the update errors. In particular, part 2 of the theorem below applies to such errors as well.

Even so, errors that tend to zero can add up to a lot. For example, $\varepsilon_j = 1/(j+1)$ tends to zero by Section 11.3, but the total $\varepsilon_0 + \varepsilon_1 + \cdots + \varepsilon_{n-1} = 1 + \frac{1}{2} + \cdots + \frac{1}{n}$ is a harmonic sum, and Section 11.5

showed that these sums do not converge. We will see that $y_n \to x_*$ all the same. Bounded errors give a bound on the distance to x_*; errors that tend to zero give convergence.

Theorem 18.1 (Iteration with update errors)**.** *Let (X, d) be a nonempty complete metric space, let $T : X \to X$ be a contraction with factor q, where $0 < q < 1$, and let x_* be its fixed point. Let $y_0, y_1, y_2, \ldots$ be points of X and let $\varepsilon_0, \varepsilon_1, \varepsilon_2, \ldots$ be nonnegative numbers such that $d(y_{n+1}, T(y_n)) \le \varepsilon_n$ for every $n \in \mathbb{N}$. Write $e_n = d(y_n, x_*)$.*

1. *For every integer $n \ge 1$,*
$$e_n \le q^n e_0 + \sum_{j=0}^{n-1} q^{n-1-j} \varepsilon_j.$$
2. *If there is a number $\eta \ge 0$ with $\varepsilon_j \le \eta$ for every $j \in \mathbb{N}$, then for every integer $n \ge 1$,*
$$e_n \le q^n e_0 + \eta \, \frac{1 - q^n}{1 - q}.$$
3. *If $\varepsilon_n \to 0$, then $y_n \to x_*$.*

Before the proof, here is how to read part 2. The first term, $q^n e_0$, is what remains of the starting error, and it tends to zero as n grows. The second term is the combined effect of all the update errors so far. Since $1 - q^n < 1$, it is never larger than $\eta/(1 - q)$, and it approaches $\eta/(1 - q)$ as n grows. So after many steps the error is at most a little more than $\eta/(1 - q)$. Section 18.4 shows that the factor $1/(1 - q)$ cannot be lowered.

Proof. **Part 1.** We use induction on n. For $n = 1$, the sum has the single term $j = 0$, whose weight is $q^0 = 1$, so the claim reads $e_1 \le q e_0 + \varepsilon_0$. This is the one-step estimate with $n = 0$. Now suppose the claim holds for some $n \ge 1$. Apply the one-step estimate, and then the induction hypothesis multiplied by the positive number q:

$$\begin{aligned} e_{n+1} &\le q e_n + \varepsilon_n \\ &\le q \left(q^n e_0 + \sum_{j=0}^{n-1} q^{n-1-j} \varepsilon_j \right) + \varepsilon_n \\ &= q^{n+1} e_0 + \sum_{j=0}^{n-1} q^{n-j} \varepsilon_j + q^0 \varepsilon_n \\ &= q^{n+1} e_0 + \sum_{j=0}^{n} q^{n-j} \varepsilon_j. \end{aligned}$$

In the third line, $q \cdot q^{n-1-j} = q^{n-j}$, and the last term ε_n is written as $q^0 \varepsilon_n$ so that it becomes the term $j = n$ of the new sum. Since $n - j = (n + 1) - 1 - j$, the last line is the claim with $n + 1$ in place of n. By induction, the claim holds for every $n \ge 1$.

Part 2. Suppose $\varepsilon_j \le \eta$ for every j. Every weight q^{n-1-j} is positive, so replacing each ε_j by the number η, which is at least as large, cannot make the sum smaller. Then put $k = n - 1 - j$. As j runs from 0 up to $n - 1$, the exponent k runs from $n - 1$ down to 0, so the same powers of q

appear in the opposite order:

$$\sum_{j=0}^{n-1} q^{n-1-j}\varepsilon_j \le \eta \sum_{j=0}^{n-1} q^{n-1-j} = \eta \sum_{k=0}^{n-1} q^k = \eta\,\frac{1-q^n}{1-q}.$$

The last equality is the finite geometric sum $1 + q + \cdots + q^{r-1} = (1-q^r)/(1-q)$ of Section 11.7, with $r = n$ terms. Together with part 1, this proves part 2.

Part 3: the plan. Now suppose $\varepsilon_n \to 0$. We must show that for every tolerance $\delta > 0$ there is a threshold N such that $e_n < \delta$ for every $n \ge N$.

The sum in part 1 mixes two kinds of update errors. The early ones may be large, but there are only finitely many of them, and every later update multiplies their effect by another factor q. The late ones may be very many, but each of them is small, and their weights q^{n-1-j} add up to less than $1/(1-q)$. This suggests splitting the sum at some index J. Before J we rely on the shrinking powers of q; from J onward we rely on the update errors being small.

The order of the choices matters. First the tolerance δ is given. Then we choose J, depending on δ, so that every update error from index J onward is small. Only after J is fixed do we choose the threshold N, large enough to shrink the early part. The index J must not change with n: the early part has to be one fixed collection of finitely many terms, so that a single threshold N can make it small for every $n \ge N$ at once.

Part 3: choosing J. Let $\delta > 0$. The number $(1-q)\delta/2$ is positive, and $\varepsilon_j \to 0$, so the definition of convergence, applied with this tolerance and with the distance $|\varepsilon_j - 0| = \varepsilon_j$, gives an index J such that

$$\varepsilon_j < \frac{(1-q)\delta}{2} \qquad \text{for every } j \ge J.$$

A larger index works as well, so we may take $J \ge 1$. Then the early part below contains at least one update error, ε_0, and the formulas written out for it make sense. For every $n > J$, split the sum of part 1 into the terms with $j < J$ and the terms with $j \ge J$:

$$e_n \le \left(q^n e_0 + \sum_{j=0}^{J-1} q^{n-1-j}\varepsilon_j \right) + \sum_{j=J}^{n-1} q^{n-1-j}\varepsilon_j.$$

Part 3: the early part. Written out, the expression in parentheses is

$$q^n e_0 + q^{n-1}\varepsilon_0 + q^{n-2}\varepsilon_1 + \cdots + q^{n-J}\varepsilon_{J-1}.$$

The smallest exponent among these terms is $n - J$. Factoring out q^{n-J} leaves

$$C_J = q^J e_0 + q^{J-1}\varepsilon_0 + q^{J-2}\varepsilon_1 + \cdots + \varepsilon_{J-1},$$

so the early part equals $q^{n-J}C_J$. The number C_J is a sum of finitely many fixed nonnegative numbers. It depends on J, but not on n.

Part 3: the late part. Every ε_j with $j \ge J$ is less than $(1-q)\delta/2$, the weights are positive, and the sum has at least one term, $j = n-1$, because $n > J$. So

$$\sum_{j=J}^{n-1} q^{n-1-j}\varepsilon_j < \frac{(1-q)\delta}{2} \sum_{j=J}^{n-1} q^{n-1-j} = \frac{(1-q)\delta}{2} \sum_{k=0}^{n-1-J} q^k.$$

Here we substituted $k = n - 1 - j$ again: as j runs from J up to $n - 1$, the exponent k runs from $n - 1 - J$ down to 0. By the finite geometric sum, $\sum_{k=0}^{n-1-J} q^k = (1 - q^{n-J})/(1 - q)$, which is less than $1/(1 - q)$. Hence the late part is less than

$$\frac{(1-q)\delta}{2} \cdot \frac{1}{1-q} = \frac{\delta}{2}.$$

This explains the tolerance we asked of the update errors. The factor $1 - q$ in it cancels the factor $1/(1 - q)$ that comes from adding up the weights, and what is left is half of δ. The other half is kept for the early part.

Part 3: choosing N. So far, for every $n > J$,

$$e_n < q^{n-J} C_J + \frac{\delta}{2}.$$

If $C_J = 0$, this says $e_n < \delta/2 < \delta$ for every $n > J$, and $N = J + 1$ will do. If $C_J > 0$, recall from Section 11.7 that $q^m \to 0$. With the tolerance $\delta/(2C_J)$, this gives a threshold M such that $q^m < \delta/(2C_J)$ for every $m \geq M$, and we may take $M \geq 1$. Put $N = J + M$. For every $n \geq N$ we have $n > J$ and $n - J \geq M$, so $q^{n-J} C_J < \delta/2$, and therefore

$$e_n < \frac{\delta}{2} + \frac{\delta}{2} = \delta.$$

In both cases $d(y_n, x_*) = e_n < \delta$ for every $n \geq N$. Since $\delta > 0$ was arbitrary, this proves $y_n \to x_*$. □

Splitting off finitely many early terms is a technique worth remembering: the powers of q eventually damp the early terms, and the late terms are all small. It is the same idea as splitting a tolerance into two halves in Section 11.5, now combined with a split of the sum.

As promised, the proof never needed the total $\varepsilon_0 + \cdots + \varepsilon_{n-1}$ to stay bounded, so it covers $\varepsilon_j = 1/(j + 1)$. What it controls is the weighted sum, in which older update errors carry smaller powers of q.

Pause and decide. Part 3 of Theorem 18.1 assumes only that $\varepsilon_n \to 0$. Does it tell us how many steps are needed to get within 0.001 of x_*?

Check. No. The proof produces a threshold N for each tolerance, but N depends on the index J, and J depends on how quickly the update errors shrink, which part 3 does not specify. If the update errors shrink very slowly, the points may approach x_* very slowly too. For example, take $T(x) = qx$, whose fixed point is 0, and let $y_0 = 1$ and $y_{n+1} = qy_n + \varepsilon_n$. Each update multiplies by the positive number q and adds a nonnegative number, so every y_n is nonnegative, and the error e_n is y_n itself. For $n \geq 1$ this gives $e_n = qy_{n-1} + \varepsilon_{n-1} \geq \varepsilon_{n-1}$: the error is never smaller than the latest update error. With $\varepsilon_j = 1/(j + 1)$ we get $e_n \geq 1/n$, so the error is still at least 0.001 after 1000 steps. When more is known about the update errors, more can be said: Exercise 18.4 gives an explicit bound when they shrink at least as fast as the powers of the contraction factor, that is, when $\varepsilon_j \leq aq^j$ for some fixed number $a \geq 0$.

18.4 Can the constant be improved?

Is the factor $1/(1-q)$ in part 2 of Theorem 18.1 really needed? Our proof used the triangle inequality, which may give away more than necessary, so perhaps a cleverer proof would give a smaller factor. To find out, we test the bound on the assistant's example from Section 18.2.

Take $X = \mathbb{R}$, $T(x) = qx$ and $y_0 = 0$, let $\eta \geq 0$, and add exactly η at every update:

$$y_{n+1} = qy_n + \eta \qquad \text{for every } n \in \mathbb{N}.$$

Section 18.2 checked that T is a contraction with factor q, that its fixed point is $x_* = 0$, and that every update error is η. The first few points are $y_1 = \eta$, $y_2 = q\eta + \eta$ and $y_3 = q^2\eta + q\eta + \eta$, which suggests

$$y_n = \eta \sum_{k=0}^{n-1} q^k = \eta \, \frac{1-q^n}{1-q} \qquad \text{for every } n \in \mathbb{N}.$$

A pattern in the first few terms is only a guess, so we prove the formula by induction. At $n = 0$ it gives $\eta(1-1)/(1-q) = 0$, which is y_0. Now assume the formula for some n. Then

$$\begin{aligned} y_{n+1} &= qy_n + \eta \\ &= q \, \frac{\eta(1-q^n)}{1-q} + \eta \\ &= \frac{\eta(q - q^{n+1}) + \eta(1-q)}{1-q} \\ &= \frac{\eta(1-q^{n+1})}{1-q}, \end{aligned}$$

which is the formula for $n+1$. So it holds for every $n \in \mathbb{N}$. The induction never used the sign of η, so the same formula holds for every real number η, negative ones included. Since $y_n - \eta/(1-q) = -\eta q^n/(1-q)$ and $q^n \to 0$, the points approach $\eta/(1-q)$, as the assistant claimed. There is also a second way to see where they go: the update $y \mapsto qy + \eta$ is itself a contraction with factor q, and its fixed point, the solution of $y = qy + \eta$, is $\eta/(1-q)$. So by Theorem 12.2 the points converge to $\eta/(1-q)$, the fixed point of the biased map, not to the fixed point 0 of T.

Now compare the formula with the theorem. The starting error is $e_0 = |0 - 0| = 0$. Since $\eta \geq 0$ and $0 < q^n \leq 1$, every y_n is nonnegative, so

$$e_n = |y_n - 0| = y_n = \eta \, \frac{1-q^n}{1-q}.$$

The bound in part 2 is $q^n \cdot 0 + \eta(1-q^n)/(1-q)$, which is exactly the same number. So in this example the bound holds with equality at every step: the error is exactly as large as part 2 allows. Any bound that is valid for all iterations satisfying our assumptions must therefore be at least this large.

In particular, the factor $1/(1-q)$ cannot be replaced by a smaller number. Suppose someone proposes a number $K < 1/(1-q)$ and claims that every iteration satisfying our assumptions, with all update errors at most η, obeys $e_n \leq q^n e_0 + K\eta$ for every $n \geq 1$. Take $\eta > 0$ in the example.

There the claim says $\eta(1-q^n)/(1-q) \le K\eta$. Dividing by η and multiplying by $1-q$, both positive, turns this into $1-q^n \le K(1-q)$, that is,

$$q^n \ge 1 - K(1-q).$$

The right side is positive, because $K(1-q) < 1$. Since $q^n \to 0$, the inequality fails for every sufficiently large n. So the proposed bound is false for this example, and not just at one step but at every step from some point on.

An upper bound that holds for every example in some class is called *sharp* if it cannot be lowered: every smaller proposed bound fails for some example in the class. We have just shown that the factor $1/(1-q)$ in part 2 is sharp for the class of all iterations that satisfy our assumptions.

A sharp bound on the distance to x_* still does not tell us whether the points converge. Let $T(x) = 0$ on $\mathbb{R}$, let $\eta > 0$, and take $y_n = (-1)^n\eta$, so that the points are $\eta, -\eta, \eta, -\eta, \ldots$. This T is a contraction with every factor q in $(0,1)$, its fixed point is 0, every update error is $|(-1)^{n+1}\eta - 0| = \eta$, and every error e_n equals η, which is within the bound of part 2. Yet the points jump back and forth between η and $-\eta$ and do not converge. Exercise 18.3 asks you to check these claims. Bounded update errors give a bound on the distance to x_*, but not convergence. Part 3 of Theorem 18.1 needs its stronger hypothesis $\varepsilon_n \to 0$.

18.5 A numerical experiment with exact arithmetic

Now we test the theorem on a concrete example and compute everything exactly. Take $T(x) = (3x+1)/4$ on $\mathbb{R}$, the map of Exercise 12.1. It is a contraction with factor $q = 3/4$, because $|T(x) - T(y)| = \frac{3}{4}|x-y|$, and its fixed point is 1, because $(3\cdot 1 + 1)/4 = 1$. Add a bias of $1/100$ at every update:

$$y_{n+1} = \frac{3y_n + 1}{4} + \frac{1}{100}, \qquad y_0 = 0.$$

Every update error is $\eta = 1/100$. Since $\frac{3y_n+1}{4} + \frac{1}{100} = \frac{3}{4}y_n + \frac{1}{4} + \frac{1}{100}$ and $\frac{1}{4} + \frac{1}{100} = \frac{26}{100} = \frac{13}{50}$, the recurrence can be written as

$$y_{n+1} = \frac{3}{4}y_n + \frac{13}{50}.$$

Like T itself, the update multiplies by $3/4$ and then adds a constant. As in Section 18.4, it is therefore a contraction in its own right, with factor $3/4$. Its fixed point p solves $p = \frac{3}{4}p + \frac{13}{50}$, so $\frac{1}{4}p = \frac{13}{50}$ and $p = \frac{26}{25}$. As in Section 5.3, where we tracked the difference between each term and the fixed point instead of the term itself, put $z_n = y_n - \frac{26}{25}$. Subtracting the equation $\frac{26}{25} = \frac{3}{4}\cdot\frac{26}{25} + \frac{13}{50}$ from the recurrence gives

$$z_{n+1} = \frac{3}{4}z_n, \qquad z_0 = 0 - \frac{26}{25} = -\frac{26}{25},$$

so $z_n = -\frac{26}{25}\left(\frac{3}{4}\right)^n$ by induction. (This is the formula of Exercise 12.5, with $a = 3/4$ and $b = 13/50$, whose fixed point is $26/25$.) Therefore

$$y_n = \frac{26}{25}\left(1 - \left(\frac{3}{4}\right)^n\right) \qquad \text{for every } n \in \mathbb{N}.$$

A formula like this, which gives y_n directly from n without computing the earlier points, is called a *closed form* for y_n. Since $(3/4)^n \to 0$, the points converge to $26/25$, not to the fixed point 1 of T.

To apply Theorem 18.1, note that the starting error is $e_0 = |0 - 1| = 1$, the contraction factor is $q = 3/4$, and every update error is at most $\eta = 1/100$, so that $\eta/(1-q) = (1/100)/(1/4) = 1/25$. Part 2 of the theorem gives a guaranteed bound computed from known quantities, an *error certificate* in the sense of Section 12.2:

$$|y_n - 1| \le \left(\frac{3}{4}\right)^n + \frac{1}{25}\left(1 - \left(\frac{3}{4}\right)^n\right) \qquad \text{for every } n \in \mathbb{N}.$$

(Part 2 gives this for $n \ge 1$; at $n = 0$ it reads $1 \le 1$.) The closed form gives the true error exactly: $y_n - 1 = \frac{1}{25} - \frac{26}{25}\left(\frac{3}{4}\right)^n$. The table compares the two. At $n = 0$ the true error equals the certificate, and in every later row it is strictly smaller. The entries are exact fractions, rounded to four decimal places for display, so some certificates are shown rounded down, for instance $0.13610\ldots$ as 0.1361. A certificate that is to be used as a guarantee must be rounded up instead, as Section 12.5 explained. (The certificate at $n = 4$ is exactly $11/32 = 0.34375$, halfway between 0.3437 and 0.3438; it is shown as 0.3438.)

n	y_n	true error $\lvert y_n - 1\rvert$	certificate
0	0.0000	1.0000	1.0000
1	0.2600	0.7400	0.7600
2	0.4550	0.5450	0.5800
4	0.7109	0.2891	0.3438
8	0.9359	0.0641	0.1361
11	0.9961	0.0039	0.0805
12	1.0071	0.0071	0.0704
16	1.0296	0.0296	0.0496
20	1.0367	0.0367	0.0430
30	1.0398	0.0398	0.0402

The true error first decreases, reaching its smallest value, about 0.0039, at $n = 11$. Between $n = 11$ and $n = 12$ the points pass the fixed point 1, and from then on the error grows again, towards $1/25 = 0.04$. Someone who stopped at $n = 11$ would be tempted to conclude that the iteration converges to 1. The closed form shows that the points overshoot 1 and settle at $26/25$. The theorem adds something that no table can: before any computing, it guarantees that the error never exceeds the certificate.

The last program in Section D.3 of Appendix D carries out this experiment with exact fractions. For each n from 0 to 20, it computes y_n, checks the closed form, checks that the true error is at most the certificate, and prints the true error and the certificate, rounded to four decimal places. Because the arithmetic is exact, the checks themselves involve no rounding. Changing `range(21)` to `range(31)` in the program extends the run to $n = 30$, the last row of the table.

Changing the pattern of the update errors gives further experiments, again starting from $y_0 = 0$. With errors of alternating sign, $y_{n+1} = \frac{3y_n+1}{4} + \frac{(-1)^n}{100}$, the computed points soon jump back and

forth around 1: those with even n appear to approach $1 - \frac{1}{175}$, and those with odd n appear to approach $1 + \frac{1}{175}$. To see where $\frac{1}{175}$ comes from, suppose the even-indexed points settle at $1 + u$ and the odd-indexed ones at $1 + v$. An update from an even index adds $\frac{1}{100}$, and one from an odd index subtracts it, so

$$1 + v = \frac{3(1+u)+1}{4} + \frac{1}{100} \qquad \text{and} \qquad 1 + u = \frac{3(1+v)+1}{4} - \frac{1}{100},$$

that is, $v = \frac{3}{4}u + \frac{1}{100}$ and $u = \frac{3}{4}v - \frac{1}{100}$. Substituting the first equation into the second gives $u = \frac{9}{16}u + \frac{3}{400} - \frac{1}{100}$, so $\frac{7}{16}u = -\frac{1}{400}$ and $u = -\frac{1}{175}$; then $v = -\frac{3}{700} + \frac{1}{100} = \frac{1}{175}$. This calculation assumes that the points settle; it does not prove that they do.

With update errors that tend to zero, such as $\frac{1}{100(n+1)}$ at step n, the points converge to 1. Now compare the three patterns, constant, alternating and shrinking, with what Theorem 18.1 promises. All three have update errors at most $1/100$, so the certificate above holds in each case. The theorem guarantees convergence to 1 only for the shrinking errors, through part 3. For the alternating errors it says nothing either way about convergence: a computation can suggest the answer, but only a separate argument can prove it.

Sharp, but not always attained

Section 18.4 showed that the factor $1/(1-q)$ is sharp. The present example forces this factor too. Here $e_0 = 1$, so a bound with a smaller factor $K < 1/(1-q)$ would read $e_n \le t + K\eta$, where $t = (3/4)^n$. Since $|s| \ge s$ for every real number s, the closed form gives $e_n = |y_n - 1| \ge \frac{1}{25} - \frac{26}{25}t$. This lower bound exceeds $t + K\eta$ exactly when $\frac{51}{25}t < \frac{1}{25} - K\eta$. The right side is positive, because $K\eta < \eta/(1-q) = \frac{1}{25}$, and $t \to 0$. So from some index on, the errors are larger than the proposed bound.

Sharpness does not mean that the certificate equals the true error in every example. Here it does not, at any step after the start. Written out, the certificate is

$$\left(\frac{3}{4}\right)^n + \frac{1}{25} - \frac{1}{25}\left(\frac{3}{4}\right)^n = \frac{1}{25} + \frac{24}{25}\left(\frac{3}{4}\right)^n,$$

while the true error is $\left|\frac{1}{25} - \frac{26}{25}\left(\frac{3}{4}\right)^n\right|$. For $n \ge 1$ the certificate is strictly larger. To check this, write $t = (3/4)^n$, so that $0 < t < 1$. If $y_n \le 1$, the true error is $\frac{26}{25}t - \frac{1}{25}$, and the certificate exceeds it by $\frac{2}{25}(1-t) > 0$. If $y_n > 1$, the true error is $\frac{1}{25} - \frac{26}{25}t$, and the certificate exceeds it by $2t > 0$. The reason for the gap becomes visible when the true error is split into its two sources:

$$y_n - 1 = -\left(\frac{3}{4}\right)^n + \frac{1}{25}\left(1 - \left(\frac{3}{4}\right)^n\right).$$

The first term is what remains of the starting error. It is negative, because $y_0 = 0$ lies below the fixed point 1. The second term is the accumulated bias. It is positive, because the bias pushes upward. The two terms partly cancel. The certificate was obtained with the triangle inequality, which ignores signs and simply adds the sizes of the two terms. A bound that is larger than the true error in a particular example is called *conservative* for that example.

So a bound can be sharp for a whole class of examples and still be conservative for one particular member of the class. For a smaller class with extra structure, a better bound may be possible. This suggests a precise next question: which extra assumption would rule out the examples that force the factor $1/(1-q)$, and what is the best factor under that assumption? The alternating errors above suggest one assumption worth studying: their points appear to settle only $\frac{1}{175}$ away from 1, far less than the $\frac{1}{25}$ that the certificate allows, because errors of opposite signs partly cancel.

Alternating signs alone are not enough, however; the assumption must also say something about the map. Take $T(x) = -qx$ on $\mathbb{R}$. It is a contraction with factor q, since $|(-qx) - (-qy)| = q|x-y|$, and its fixed point is 0, since $-qx = x$ gives $(1+q)x = 0$. It sends every nonzero point to the other side of 0. Start from $y_0 = 0$ and let the update errors alternate: $y_{n+1} = -qy_n + (-1)^n\eta$ with $\eta > 0$, so that $|y_{n+1} - T(y_n)| = \eta$ at every step. Put $w_n = (-1)^n y_n$, and multiply the recurrence by $(-1)^{n+1}$. Since $(-1)^{n+1} \cdot (-q)y_n = q(-1)^n y_n$ and $(-1)^{n+1}(-1)^n = (-1)^{2n+1} = -1$, this gives $(-1)^{n+1}y_{n+1} = q(-1)^n y_n - \eta$, that is,

$$w_{n+1} = qw_n - \eta, \qquad w_0 = 0.$$

This is the constant-bias iteration of Section 18.4, with $-\eta$ in place of η, so $w_n = -\eta(1-q^n)/(1-q)$. Hence $e_n = |y_n| = |w_n| = \eta(1-q^n)/(1-q)$, and part 2 of Theorem 18.1 holds with equality at every step. From the second update on, each update error has the same sign as $T(y_n)$, so it pushes the point further away from 0, and nothing cancels.

18.6 The finished investigation as a short paper

An investigation is finished only when it has been written up so that another reader can check it without seeing your notes or your conversations with an assistant. For a project like this one, a short paper with four parts works well.

- *Introduction.* State the question: what happens to the iteration of a contraction when every update is made with an error? Explain why the first guess, convergence to the fixed point, is false, using the constant bias of Section 18.2. Then say which questions replace it: how far from x_* the points can be, and when they still converge.
- *Theory.* State the setting precisely: the space, the contraction factor q, and the model of the update errors, including the requirement that every y_n lies in X. Then give Theorem 18.1 with its proof, and the example of Section 18.4 showing that the factor $1/(1-q)$ cannot be lowered.
- *Experiments.* Present the exact computation of Section 18.5, with the program and enough detail that anyone can run it again and get the same output. A computation that others can repeat in this way is called *reproducible*. The experiments illustrate the proved statements; they do not replace the proofs.
- *Conclusion.* Say clearly what the results do not cover. The approximate points must stay in X. The factor q must be less than 1. For $q = 1$ the map need not have a fixed point at all, as $x \mapsto x + 1$ on $\mathbb{R}$ shows; and even when it has one, the sum $1 + q + \cdots + q^{n-1}$ behind part 2 equals n, so the bound $e_0 + n\eta$ grows without limit when $\eta > 0$, and the starting error is not reduced at all. And when the update errors are bounded but do not tend to zero, their effect

need not die out: the points may converge to a point other than x_*, as with the constant bias, or fail to converge altogether, as with the map $T(x) = 0$ and alternating points; both examples are in Section 18.4.

A paper should also say what is new in it. For this course project, an honest statement is that the project reconstructs and analyzes a standard argument about how errors in the individual updates affect an iteration. It must not claim that the result is new merely because the student, or the assistant, had not met it before. As *To the reader* pointed out at the front of the book, a result you discover for yourself may be correct and still be well known.

Before claiming that a result is new in research, compare its exact statement and assumptions with the published literature, especially the *primary sources*: the papers and books in which results first appeared. When one source describes a result from another, look up the result itself rather than relying on the description. Only after that comparison can you decide what, if anything, is new. An unsuccessful search, whether in a library catalogue or by an assistant, shows only that this search did not find the result; it does not prove that the result is new.

18.7 Your independent capstone

Now it is your turn. Choose one of the three problems that run through the book: assigning tasks, iterating a map, or sign sequences. Start with a question narrow enough that you can check every claim you make. Here are three possible starting points.

- *Assignments with capacities.* Allow a task to be given to several students, up to a fixed capacity, and find the right version of Hall's condition. Replacing a task of capacity c by c separate slots turns the question into one about ordinary matchings; Exercise 18.5 asks you to carry out this first step. You might then ask how the search procedure of Chapter 8 changes in this setting.
- *Iteration with a specified pattern of errors.* Pick a particular pattern of update errors, for example $\varepsilon_n = 1/(n+1)$, or $\varepsilon_n = s^n$ for a number s with $0 < s < 1$ and $s \neq q$, and find out as precisely as you can how fast the error e_n goes to zero. Exercise 18.4 treats the case $\varepsilon_n \leq aq^n$.
- *Sign sequences at a new length.* Extend the table of Section 14.6 by one more length, with a program that anyone can run to reproduce your counts. Then find out which operations on sequences, such as negating every sign, reversing the order, or rotating, preserve each of the two properties that the table counts.

None of these is expected to lead to a new theorem. Each is a manageable place to start, with room to go further.

Your write-up should contain:

- a precise statement of your question and of your result, and, if the question changed while you worked on it, both the original and the final version, with the difference explained;
- a map of the background your argument needs, like the prerequisite map of Section 17.7;
- a proof, or a counterexample;
- computer checks where they are relevant, together with the program, the software it runs on,

and the range of cases it covers;

- a record of the assumptions you used and of the evidence for each claim: proved, cited from another source, checked by computer for finitely many cases, or not established, as in the section "Keeping track of evidence" of *To the reader*;
- credit to every source you used, an honest account of how you used an AI assistant, and a list of what remains unsettled;
- a short explanation of what is new compared with the published literature, and what is new only to you.

The investigation ends with a short oral discussion. Expect to explain one delicate step of your argument, to change one of your assumptions, and to predict what fails. Your ability to answer these questions matters more than how polished your saved conversation with an assistant looks. Section E.6 of Appendix E suggests a grading rubric.

Before you hand in your work, an assistant can help you review it. Ask it for three things, one at a time. First, have it restate your argument in its own words, and check whether the restatement is faithful; a step that it gets wrong is often a step that you explained badly. Second, ask it for counterexamples to stronger versions of your statement, and ask whether each hypothesis is really needed. Third, have it check that your prose, your formulas, your code and any Lean statements all say the same thing. All three answers come from the same assistant, so they may share the same blind spots, and they are no substitute for a careful human reader such as a classmate or an instructor. Their value lies in the specific points they raise, each of which you then check yourself.

Exercises

Exercises 18.1–18.4 are core exercises, and Exercises 18.5 and 18.6 are transfer exercises. Complete solutions are in Appendix C, under "Week 18."

Exercise 18.1. Starting from the one-step estimate $e_{n+1} \le q e_n + \varepsilon_n$, derive a bound for e_4 in terms of e_0 and the update errors, without using summation notation, and check that it agrees with part 1 of Theorem 18.1 for $n = 4$. Then, in the bound for e_{10}, find the power of q that multiplies ε_6, and explain it by counting the updates that come after ε_6 is made.

Exercise 18.2. Let $0 < q < 1$, $\eta \ge 0$ and $a \ge 0$, and define $y_0 = a$ and $y_{n+1} = q y_n + \eta$ for every $n \in \mathbb{N}$. Prove by induction that $y_n = q^n a + \eta(1 - q^n)/(1 - q)$ for every $n \in \mathbb{N}$. Deduce that, for $T(x) = qx$ on $\mathbb{R}$, these points satisfy the bound of part 2 of Theorem 18.1 with equality at every step, whatever the starting error $e_0 = a$. Then take $a = 0$ and $\eta > 0$, and explain why the factor $1/(1 - q)$ in part 2 cannot be replaced by any smaller number K.

Exercise 18.3. Let $\eta > 0$, let $T(x) = 0$ for every $x \in \mathbb{R}$, and let $y_n = (-1)^n \eta$ for every $n \in \mathbb{N}$. Check that all the hypotheses of Theorem 18.1 hold with update errors $\varepsilon_n = \eta$, and check directly that the bound of part 2 holds. Then prove that the sequence $y_0, y_1, y_2, \ldots$ does not converge, and explain why a valid error bound does not settle whether a sequence converges.

Exercise 18.4. Suppose that the update errors in Theorem 18.1 satisfy $\varepsilon_j \le a q^j$ for every $j \in \mathbb{N}$, where $a \ge 0$ and q is the contraction factor. Derive the bound $e_n \le q^n e_0 + a n q^{n-1}$ for every $n \ge 1$,

and prove that this upper bound tends to zero as n grows. (Hint: the factor n grows, so it is not enough that $q^{n-1} \to 0$. Put $r = (1+q)/2$ and $t = q/r$, and compare nt^{n-1} with $1 + t + \cdots + t^{n-1}$.)

Exercise 18.5. A finite set L of students and a finite set R of tasks are given, and each student accepts some of the tasks, as in Chapter 9. For a set $S \subseteq L$, $N(S)$ is the set of tasks accepted by at least one student in S. Each task $r \in R$ now has a *capacity* c_r, which is a nonnegative integer. An *assignment with capacities* gives every student one task that the student accepts, in such a way that each task r is given to at most c_r students. Prove that an assignment with capacities exists if and only if

$$|S| \le \sum_{r \in N(S)} c_r \qquad \text{for every set } S \subseteq L.$$

When every capacity equals 1, this is Hall's theorem. (Hint: replace each task r by c_r separate slots, each of which can be given to only one student, as suggested in Section 18.7, and apply Hall's theorem to the new graph.)

Exercise 18.6. Write a short record of the investigation of the biased iteration $y_{n+1} = (3y_n + 1)/4 + 1/100$, $y_0 = 0$, of Section 18.5. It should contain the exact question, the background the argument needs, an outline of the proof, the counterexample that refutes the first guess, one numerical check done in exact arithmetic, and an honest statement of what is new. Then state one further question whose answer does not follow from the proofs in this chapter.

APPENDIX A

Solutions for Chapters 1–6

This appendix and the next two contain complete solutions to every exercise in the book. The solutions are grouped by week, and the section headed "Week N" holds the six solutions for Chapter N. Here you will find the solutions for Chapters 1–6: first proofs about even and odd numbers, statements and logic, sets and maps, proof strategies, induction and invariants, and equivalence and symmetry.

Make a serious attempt at an exercise before you read its solution, and then read the solution to learn from it, not only to confirm a final answer. If your attempt succeeded, compare it with the solution anyway, and check that you proved exactly what the exercise asked, in every case it covers, including boundary cases such as zero or the empty set. If your attempt stalled, find the point in the solution where your own argument stopped, and identify the fact or idea that carries the solution past that point. Then close the book and write the whole argument again in your own words. In either case, finish by testing yourself on a small variation, such as different numbers or one hypothesis removed, and check whether the same idea still works.

Some exercises, here and in the next two appendices, ask you to invent an example or to explain something in your own words. For these, the solution gives one model answer. A different answer can be just as correct, provided it meets every requirement of the exercise.

A.1 Week 1

Solution to Exercise 1.1

At $x = 4$,

$$2(x+3) = 2(4+3) = 2 \cdot 7 = 14, \qquad \text{whereas} \qquad 2x+3 = 2 \cdot 4 + 3 = 8 + 3 = 11.$$

In $2(x+3)$ we first add three to the input and then double the result, so the added three is doubled as well. In $2x+3$ we double only the input and add three afterwards, so the three is not doubled. Multiplying out confirms this: $2(x+3) = 2x+6$, which is always exactly 3 more than $2x+3$, whatever the value of x. At $x = 4$ the difference is $14 - 11 = 3$.

Solution to Exercise 1.2

We must show that the sum has the form required by the definition of odd: twice an integer, plus one.

Let a be an even integer and b an odd integer. By the definitions of even and odd, there are integers k and ℓ with $a = 2k$ and $b = 2\ell + 1$. We use two different letters because nothing says the

two witnesses are equal. Substitute and regroup:

$$\begin{aligned} a + b &= 2k + (2\ell + 1) \\ &= 2k + 2\ell + 1 \\ &= 2(k + \ell) + 1. \end{aligned}$$

The number $k + \ell$ is an integer, because a sum of integers is an integer. So $a + b$ is twice an integer plus one, which means that $a + b$ is odd. The witness for the oddness of the sum is $k + \ell$.

For example, $a = 4$ and $b = -3$ have witnesses $k = 2$ and $\ell = -2$, since $4 = 2 \cdot 2$ and $-3 = 2 \cdot (-2) + 1$. Their sum is 1, and the proof's witness is $k + \ell = 0$; indeed $1 = 2 \cdot 0 + 1$.

Solution to Exercise 1.3

The assistant's calculation multiplies an odd number by *itself*: writing both factors as $2k + 1$, with the same k, forces the two factors to be equal. So the argument covers only products such as $3 \cdot 3$ or $(-5) \cdot (-5)$. It does not cover the product of two different odd integers, for example $3 \cdot 5$. The fact the assistant relies on, that $(2k + 1)^2$ is odd for every integer k, is true, as the general proof below confirms. But it is a narrower statement than the claim about all products of two odd integers.

For the general claim, let a and b be odd integers. By the definition of odd, there are integers k and ℓ with $a = 2k + 1$ and $b = 2\ell + 1$, where k and ℓ may be different. Multiply out one bracket at a time, as in Section 1.3:

$$\begin{aligned} ab &= (2k + 1)(2\ell + 1) \\ &= 2k(2\ell + 1) + (2\ell + 1) \\ &= 4k\ell + 2k + 2\ell + 1 \\ &= 2(2k\ell + k + \ell) + 1. \end{aligned}$$

The number $2k\ell + k + \ell$ is an integer, because products and sums of integers are integers. So ab is twice an integer plus one, and therefore ab is odd. For instance, $3 = 2 \cdot 1 + 1$ and $5 = 2 \cdot 2 + 1$ give $k = 1$ and $\ell = 2$, so the witness is $2 \cdot 1 \cdot 2 + 1 + 2 = 7$, and indeed $3 \cdot 5 = 15 = 2 \cdot 7 + 1$. Setting $\ell = k$ in the general proof recovers the assistant's special case $(2k + 1)^2$.

Solution to Exercise 1.4

The index j takes the three values 1, 2, and 3, so the sum has three terms:

$$\begin{aligned} \sum_{j=1}^{3} (2a_j + 1) &= (2a_1 + 1) + (2a_2 + 1) + (2a_3 + 1) \\ &= (2 \cdot 2 + 1) + (2 \cdot (-1) + 1) + (2 \cdot 4 + 1) \\ &= 5 + (-1) + 9 = 13. \end{aligned}$$

As a check, the same sum can be regrouped as $2(a_1 + a_2 + a_3) + 3 = 2 \cdot 5 + 3 = 13$.

Solution to Exercise 1.5

By the fact quoted in the exercise, $x(x-2) = 0$ holds only if $x = 0$ or $x - 2 = 0$, that is, only if $x = 0$ or $x = 2$. Substituting shows that both values work: $0 \cdot (0-2) = 0$ and $2 \cdot (2-2) = 2 \cdot 0 = 0$. So the real solutions are exactly 0 and 2.

Dividing both sides of $x(x-2) = 0$ by x gives $x - 2 = 0$, and hence $x = 2$. But division by x is allowed only when $x \neq 0$, so this step says nothing about the value $x = 0$, and that value is in fact a solution. Division by x therefore finds only the solution 2. It gives a complete answer only if the case $x = 0$ is checked separately, exactly as in the equation $x^2 = x$ of Section 1.2.

Solution to Exercise 1.6

Use

$$f(n) = (n-1)(n-2)\cdots(n-10),$$

the product of the ten factors $n-1, n-2, \ldots, n-10$. If n is one of the integers $1, 2, \ldots, 10$, then one of the factors is $n - n = 0$, so the whole product is 0. At $n = 11$ the factors are $10, 9, \ldots, 1$, all positive, so

$$f(11) = 10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 3\,628\,800 \neq 0.$$

This formula agrees with the claim "$f(n) = 0$" at all ten checked values but fails at $n = 11$. Checking finitely many values cannot, on its own, establish a statement about every positive integer.

A.2 Week 2

Solution to Exercise 2.1

The original implication is true: if $x = 3$, then $x^2 = 3 \cdot 3 = 9$.

The converse is "If $x^2 = 9$, then $x = 3$." It is false, and $x = -3$ is a counterexample. It satisfies the hypothesis, since $(-3)^2 = 9$, but not the conclusion, since $-3 \neq 3$.

The contrapositive is "If $x^2 \neq 9$, then $x \neq 3$." It is true, because it is equivalent to the original implication, which is true. We can also see this directly: if x were equal to 3, then x^2 would equal 9, which contradicts the hypothesis $x^2 \neq 9$.

So the original statement and its contrapositive are true, and the converse is false. This is the same pattern as for "divisible by four implies even" in Sections 2.1 and 2.5: a true implication always has a true contrapositive, but its converse may fail.

Solution to Exercise 2.2

Work from the outside in. The original statement says that *every* integer n has at least one successful response m. It fails if *some* integer n has no successful response at all. So the negation begins "There is an integer n such that, for every integer m, ..." The domains stay the same: both n and m are still integers.

A proposed response m succeeds when $m > n$ *and* m is even. By De Morgan's law, it fails when at least one of these requirements fails: $m \le n$, or m is not even. The comparison $m > n$ is strict, so its failure is $m \le n$, which includes the case $m = n$. In symbols, the original statement is

$$\forall n \in \mathbb{Z}\ \exists m \in \mathbb{Z}: \quad (m > n) \wedge (m \text{ is even}),$$

and its negation is

$$\exists n \in \mathbb{Z}\ \forall m \in \mathbb{Z}: \quad (m \le n) \vee \neg(m \text{ is even}).$$

In words: there is an integer n such that every integer m fails at least one requirement, that is, m is at most n or m is not even. The "or" is inclusive. An m that fails both requirements, such as an odd integer smaller than n, is still an unsuccessful response and must not be left out.

Writing down the negation does not claim that it is true. In fact the original statement is true, so its negation is false. For any integer n, take $m = 2|n| + 2$. It is an integer, because $|n|$ is either n or $-n$, and it is even, because $m = 2(|n| + 1)$. It is also larger than n:

$$m = 2|n| + 2 \ge |n| + 2 > |n| \ge n.$$

The first inequality holds because $|n| \ge 0$, and the last because $|n|$ equals n when $n \ge 0$ and is positive, hence larger than n, when $n < 0$. For instance, $n = -5$ gives $m = 12$, and $n = 7$ gives $m = 16$.

Solution to Exercise 2.3

The first statement is true. The skeptic chooses x first, and the prover then chooses y already knowing x. The choice $y = -x$ works, because $x + (-x) = 0$. For example, $x = 5$ is answered by $y = -5$.

The second statement is false. It asks for a single y, chosen before x, with $x + y = 0$ for every real number x. Suppose such a y existed. Taking $x = 0$ would give $y = 0$, and taking $x = 1$ would give $1 + y = 0$, that is, $y = -1$. One number cannot equal both 0 and -1, so no such y exists. Put differently, whatever y is proposed, the skeptic can choose $x = 1 - y$, and then $x + y = 1 \ne 0$.

The two statements differ only in the order of their quantifiers, exactly as statements (A) and (B) do in Section 2.3.

Solution to Exercise 2.4

The number $x = -3$ is a counterexample. It satisfies the hypothesis, since $(-3)^2 = 9 > 4$, but not the conclusion, since -3 is not greater than 2.

The repaired claim is "If a real number x satisfies $x^2 > 4$, then $x < -2$ or $x > 2$," or equivalently "then $|x| > 2$." Section 2.6 proves that $x^2 > 4$ holds exactly when $|x| > 2$. In brief: if $|x| > 2$, squaring the inequality $2 < |x|$ between nonnegative numbers gives $4 < |x|^2 = x^2$; and if $|x| \le 2$, squaring this inequality in the same way gives $x^2 = |x|^2 \le 4$. The second case is the contrapositive of "if $x^2 > 4$, then $|x| > 2$," so the repaired claim is true for every real x. The first case shows the reverse: every real number that satisfies the new conclusion also satisfies $x^2 > 4$. So the new conclusion describes exactly the real numbers with $x^2 > 4$.

Other repairs give true statements but lose information. For instance, "if $x^2 > 4$, then $x \neq 0$" is true but much weaker: $x = 1$ satisfies $x \neq 0$ although $1^2 > 4$ fails. Restricting the claim to positive x would also make it true, but that changes the hypothesis, which the exercise does not allow, and the restricted claim would say nothing about negative numbers.

Solution to Exercise 2.5

Let Ada and Ben be the two students, and let tasks 1 and 2 be available. Suppose both students accept task 1 and neither accepts task 2. Each student accepts at least one task, namely task 1. But giving Ada and Ben different acceptable tasks would require two different tasks, each acceptable to the student who receives it, while task 1 is the only task either of them accepts.

Following the hint, the condition looks at groups of students:

for every group of students, the number of tasks acceptable to at least one member of the group is at least the number of students in the group.

In the example, the group made up of Ada and Ben has two students but only one acceptable task between them, and $1 \geq 2$ is false. So the condition fails, and it detects the problem.

The condition must hold whenever all the students can be given different acceptable tasks. Suppose such an assignment exists, and take any group of k students. The assignment gives these k students k different tasks, and each of those tasks is acceptable to the student who receives it, so it is acceptable to at least one member of the group. Hence at least k tasks are acceptable to members of the group.

The group condition is stronger than "each student accepts at least one task." Applied to a group consisting of a single student, it says exactly that this student accepts at least one task, and it also checks every larger group, which is where the example fails. Chapter 9 proves the converse: when there are finitely many students and tasks, the group condition guarantees that all the students can be given different acceptable tasks. This is Hall's theorem, Theorem 9.1.

Solution to Exercise 2.6

Let ε be a positive real number. By the Archimedean property, choose an integer N with $N > 1/\varepsilon$. It is positive, because $1/\varepsilon > 0$. Now let n be any integer with $n \geq N$. Then $0 < N \leq n$, so the rule from Section 2.3 that taking reciprocals of positive numbers reverses $\leq$ gives $1/n \leq 1/N$. As in Example 2.1, the inequality $N > 1/\varepsilon$ gives $1/N < \varepsilon$. Together,

$$\frac{1}{n} \leq \frac{1}{N} < \varepsilon.$$

For instance, for $\varepsilon = 1/100$ the choice $N = 101$ works, and for $\varepsilon = 0.003$ the choice $N = 334$ works.

The statement involves three choices, made in this order: first the tolerance ε is given, then we choose N, and then any integer $n \geq N$ may be tested. Since ε comes first, N may depend on it. In fact it must, because a smaller ε may need a larger N: the choice $N = 101$ works for $\varepsilon = 1/100$ but

not for $\varepsilon = 1/1000$, and no single N works for every ε. Indeed, testing $n = N$ shows that such an N would satisfy $1/N < \varepsilon$ for every positive ε, which the Pause and decide question at the end of Section 2.3 rules out.

The integer n comes after N. The phrase "every integer $n \geq N$" only makes sense once N has been fixed, because N decides which integers are tested. To see why this order matters, imagine that N could be chosen after seeing n. Then for any n we could choose $N = n + 1$. The tested integer n would not satisfy $n \geq N$, so the requirement "if $n \geq N$, then $1/n < \varepsilon$" would hold vacuously, as in Section 2.5. Since this trick works whatever condition follows "if $n \geq N$," a statement with the choices in that order promises nothing. With N fixed before n, the statement makes a real promise: from N onward, every number $1/n$ is less than ε. Chapter 11 defines limits by statements of exactly this shape.

A.3 Week 3

Solution to Exercise 3.1

The union is $A \cup B = \{1, 2, 3, 4\}$, the intersection is $A \cap B = \{2, 4\}$, and the difference is $A \setminus B = \{1\}$. The union lists each member once, including 2 and 4, which belong to both sets. The difference keeps the members of A that are not in B: the members 2 and 4 of A are removed because they also lie in B, and only 1 remains.

Solution to Exercise 3.2

We must prove that $g \circ f$ is surjective, so we start from the *output* end. Let $f : A \to B$ and $g : B \to C$ be surjective, and take an arbitrary target $c \in C$. The goal is to find a witness $a \in A$ with $g(f(a)) = c$.

Surjectivity of g applies to c, because c belongs to its codomain C. It supplies some $b \in B$ with $g(b) = c$. Surjectivity of f now applies to this particular b, because b belongs to its codomain B. It supplies some $a \in A$ with $f(a) = b$. Substitution checks the candidate:

$$(g \circ f)(a) = g(f(a)) = g(b) = c.$$

Since c was an arbitrary target, every element of C is reached, so the composition is surjective.

Notice the order. The functions are applied with f first and g second, but the witnesses are found the other way round: first b for g, then a for f. We could not start by choosing a, because until we know b we do not know which target of f to aim for. Neither witness needs to be unique; surjectivity promises only that at least one exists.

Solution to Exercise 3.3

Injective. Let $m, n \in \mathbb{Z}$ with $f(m) = f(n)$, that is, $2m = 2n$. Dividing both sides by the nonzero number 2 gives $m = n$. So f is injective.

Not surjective. The target 1 is never reached. If $2n = 1$, then $n = 1/2$, which is not an integer. So no integer n satisfies $f(n) = 1$, and f is not surjective.

A bijection. Let $2\mathbb{Z} = \{2k : k \in \mathbb{Z}\}$ be the set of even integers, and consider the rule $n \mapsto 2n$ as a function from $\mathbb{Z}$ to $2\mathbb{Z}$. This is a legitimate function, because every output $2n$ is even and so lies in the new codomain. It is still injective, by the same argument as before: changing the codomain does not change any output. It is now surjective: each member of $2\mathbb{Z}$ has the form $2k$ for some integer k, and the input k has output $2k$. Hence the new function is a bijection. The new codomain is exactly the image of the original function.

Solution to Exercise 3.4

Let $x \in A$. Then

$$\begin{aligned}
x \in f^{-1}(S \cup T) &\iff f(x) \in S \cup T && \text{definition of preimage} \\
&\iff f(x) \in S \text{ or } f(x) \in T && \text{definition of union} \\
&\iff x \in f^{-1}(S) \text{ or } x \in f^{-1}(T) && \text{definition of preimage} \\
&\iff x \in f^{-1}(S) \cup f^{-1}(T) && \text{definition of union.}
\end{aligned}$$

Each step is valid in both directions. Reading the chain downward shows that $f^{-1}(S \cup T) \subseteq f^{-1}(S) \cup f^{-1}(T)$, and reading it upward shows the reverse containment. Both sides consist only of members of A, and x was an arbitrary element of A, so the two sets have exactly the same members; they are equal. No injectivity, surjectivity, or inverse function is needed.

Solution to Exercise 3.5

Left inverse. Here $h(f(a)) = a$ for every $a \in A$. To prove that f is injective, take $x, y \in A$ and assume $f(x) = f(y)$. Applying the same function h to equal objects gives equal results:

$$h(f(x)) = h(f(y)).$$

This is legitimate because $f(x)$ and $f(y)$ belong to B, the domain of h. Now use the left-inverse identity once at x and once at y: the left side equals x and the right side equals y. So $x = y$, as required.

Right inverse. Here $f(h(b)) = b$ for every $b \in B$. To prove that f is surjective, take any target $b \in B$. The element $h(b)$ belongs to A, so it is a candidate witness. Its output is $f(h(b)) = b$, so the candidate works.

The first identity lets us recover inputs from outputs; the second lets us reach every target. A function can have one kind of one-sided inverse without the other. For example, let $A = \{0\}$, $B = \{0, 1\}$, $f(0) = 0$, and $h(0) = h(1) = 0$. Then $h(f(0)) = 0$, so $h \circ f = \mathrm{id}_A$ and f is injective. But f is not surjective, since no input reaches 1; correspondingly, $f(h(1)) = 0 \neq 1$, so $f \circ h \neq \mathrm{id}_B$. In fact f has no right inverse at all, since by the second part a right inverse would make f surjective.

Exchanging the roles of the two functions gives the reverse situation. Read the identity $h \circ f = \mathrm{id}_A$ as a statement about $h : B \to A$: it says that f is a right inverse of h. So h is surjective by the

second part. But h is not injective, since $h(0) = h(1)$, so by the first part h has no left inverse. In short, f has a left inverse but no right inverse, while h has a right inverse but no left inverse. An inverse satisfies both identities, so by the two parts above, every function that has an inverse is bijective.

Solution to Exercise 3.6

No, equality can fail, but one containment always holds.

The containment $f(S \cap T) \subseteq f(S) \cap f(T)$. Let $z \in f(S \cap T)$. Then some $x \in S \cap T$ satisfies $f(x) = z$. That same x belongs to S, so $z \in f(S)$, and it belongs to T, so $z \in f(T)$. Hence $z \in f(S) \cap f(T)$.

A counterexample to equality. Let $f : \mathbb{R} \to \mathbb{R}$ be $f(x) = x^2$, with $S = \{1\}$ and $T = \{-1\}$. Then $S \cap T = \varnothing$, and the image of the empty set is empty, because there are no inputs to produce outputs. So $f(S \cap T) = \varnothing$. On the other hand, $f(S) = \{1\}$ and $f(T) = \{1\}$, so $f(S) \cap f(T) = \{1\}$. The two sides differ.

The proof of the containment cannot be reversed. A member z of $f(S) \cap f(T)$ has a preimage $s \in S$ and a preimage $t \in T$, but these may be different inputs, as 1 and -1 are here, and then neither of them needs to lie in $S \cap T$. If f is injective, however, $f(s) = z = f(t)$ forces $s = t$, which lies in $S \cap T$; so for injective functions the equality does hold.

A.4 Week 4

Solution to Exercise 4.1

Let a and b be even integers. By the definition of even, there are integers k and ℓ with $a = 2k$ and $b = 2\ell$. We use two different letters because a and b may be different numbers. Then

$$ab = (2k)(2\ell) = 4k\ell.$$

Since $k\ell$ is a product of integers, it is an integer. So ab is four times an integer, which is exactly what "divisible by four" means (Section 2.1). For instance, $a = 6$ and $b = 10$ have witnesses $k = 3$ and $\ell = 5$, and indeed $ab = 60 = 4 \cdot 15$, where $15 = k\ell$.

Solution to Exercise 4.2

The statement to be proved has hypothesis "$3n + 1$ is even" and conclusion "n is odd." Its contrapositive is: *if n is not odd, then $3n + 1$ is not even.* We prove the contrapositive.

Suppose the integer n is not odd. By Lemma 4.1, every integer is even or odd, so n is even, and $n = 2k$ for some integer k. Substitute this into the quantity $3n + 1$:

$$3n + 1 = 3(2k) + 1 = 6k + 1 = 2(3k) + 1.$$

Since $3k$ is an integer, $3n + 1$ is odd. By Lemma 4.1 again, no integer is both even and odd, so $3n + 1$ is not even. This proves the contrapositive. An implication and its contrapositive are equivalent (Section 2.5), so the original statement is proved as well.

Solution to Exercise 4.3

Suppose, for contradiction, that the number $s = r + u$ is rational. Then $u = s - r$ is the difference of two rational numbers.

Such a difference is always rational. Indeed, write $s = a/b$ and $r = c/d$ with integers a, b, c, d, where $b \neq 0$ and $d \neq 0$. Putting both fractions over the common denominator bd gives

$$s - r = \frac{a}{b} - \frac{c}{d} = \frac{ad}{bd} - \frac{bc}{bd} = \frac{ad - bc}{bd}.$$

The numerator $ad - bc$ is an integer, and the denominator bd is an integer that is not zero, because a product of two nonzero numbers is nonzero. So $s - r$ is rational.

Hence $u = s - r$ would be rational, which contradicts the hypothesis that u is irrational. The supposition is therefore false, and $r + u$ is irrational. For example, with $r = 1$ and $u = \sqrt{2}$, this shows that $1 + \sqrt{2}$ is irrational.

Solution to Exercise 4.4

Existence. The number 2 is a solution: the right side is $3 \cdot 2 - 4 = 2$, which equals the left side.

Uniqueness. Suppose that x and y are both real solutions, so that $x = 3x - 4$ and $y = 3y - 4$. Subtracting the second equation from the first cancels the constant terms and gives $x - y = 3x - 3y = 3(x - y)$. Subtracting $x - y$ from both sides gives $0 = 2(x - y)$, so $x - y = 0$ and $x = y$. Thus any two solutions are equal.

Together the two parts show that the equation has exactly one real solution, namely 2. Uniqueness can also be shown by rearranging: if $x = 3x - 4$, then subtracting x from both sides gives $0 = 2x - 4$, so $2x = 4$ and $x = 2$. This calculation proves only that a solution, if there is one, must equal 2. The substitution check in the existence part is what shows that 2 really is a solution.

Solution to Exercise 4.5

The assistant has missed the negative solution $-\sqrt{2}$. The symbol $\sqrt{2}$ denotes the *nonnegative* number whose square is 2, but a negative number can also have square 2.

Both numbers are solutions. By the definition of $\sqrt{2}$, we have $(\sqrt{2})^2 = 2$. Also $(-\sqrt{2})^2 = (-1)^2(\sqrt{2})^2 = 2$.

There are no others. Let x be any real number with $x^2 = 2$. Multiplying out one bracket at a time, and using $(\sqrt{2})^2 = 2$, gives

$$(x - \sqrt{2})(x + \sqrt{2}) = x^2 + \sqrt{2}\,x - \sqrt{2}\,x - (\sqrt{2})^2 = x^2 - 2 = 0.$$

A product of two real numbers is zero only when at least one of the factors is zero (the fact used for $x^2 = 1$ in Section 4.4). So $x - \sqrt{2} = 0$ or $x + \sqrt{2} = 0$, that is, $x = \sqrt{2}$ or $x = -\sqrt{2}$.

Therefore the real solutions of $x^2 = 2$ are exactly $\sqrt{2}$ and $-\sqrt{2}$. The assistant's answer would be correct only under the extra assumption $x \geq 0$, which selects the first of the two.

Solution to Exercise 4.6

First proof: using Theorem 4.3. We begin by checking that $\sqrt{8} = 2\sqrt{2}$. Indeed, $2\sqrt{2}$ is nonnegative and $(2\sqrt{2})^2 = 4 \cdot 2 = 8$, and $\sqrt{8}$ is the only nonnegative number whose square is 8 (Section 1.8). Now suppose, for contradiction, that $\sqrt{8}$ is rational, say $\sqrt{8} = a/b$ with integers a and b and $b \neq 0$. Then

$$\sqrt{2} = \frac{\sqrt{8}}{2} = \frac{a}{2b},$$

and $2b$ is a nonzero integer, so $\sqrt{2}$ would be rational. This contradicts Theorem 4.3, so $\sqrt{8}$ is irrational.

Second proof: adapting the argument. Suppose, for contradiction, that $\sqrt{8}$ is rational. Exactly as in the proof of Theorem 4.3, we may write $\sqrt{8} = a/b$ with integers a and b, where b is positive and is the smallest positive denominator for which such a fraction exists. Squaring and multiplying by b^2 gives $a^2 = 8b^2 = 2(4b^2)$, so a^2 is even, and Lemma 4.2 shows that a is even, say $a = 2c$. Then $4c^2 = 8b^2$, so $c^2 = 2b^2$. This is the equation that the proof for $\sqrt{2}$ started from, with c in place of a, so we continue in the same way. Since c^2 is even, Lemma 4.2 shows that c is even, say $c = 2e$. Then $4e^2 = 2b^2$, so $b^2 = 2e^2$. Thus b^2 is even, and the same lemma shows that b is even, say $b = 2d$. Since $b > 0$, the integer d satisfies $0 < d < b$. Now

$$\sqrt{8} = \frac{a}{b} = \frac{2c}{2d} = \frac{c}{d}$$

is a fraction of integers with a positive denominator smaller than b, which contradicts the choice of b. Hence $\sqrt{8}$ is irrational.

How the parity steps change. In the proof for $\sqrt{2}$, a single halving of the numerator, $a = 2c$, already produces the equation $b^2 = 2c^2$, which shows that b is even. For $\sqrt{8}$, the first halving only turns $a^2 = 8b^2$ into $c^2 = 2b^2$. This is because $8 = 4 \cdot 2$, and replacing a by $2c$ removes exactly one factor of 4 from the equation. One extra halving, $c = 2e$, is therefore needed before we can conclude that b is even.

A.5 Week 5

Solution to Exercise 5.1

We use induction on n.

Base case. For $n = 0$ the sum $\sum_{j=1}^{0} j$ has no terms, so it is the empty sum, which is 0. The right side is $0 \cdot (0 + 1)/2 = 0$ as well.

Induction step. Let k be a natural number, and assume the induction hypothesis $\sum_{j=1}^{k} j =$

$k(k+1)/2$. Split off the last term of the sum for $k+1$:

$$\begin{aligned}
\sum_{j=1}^{k+1} j &= \sum_{j=1}^{k} j + (k+1) \\
&= \frac{k(k+1)}{2} + (k+1) && \text{by the induction hypothesis} \\
&= \frac{k(k+1)}{2} + \frac{2(k+1)}{2} \\
&= \frac{k(k+1) + 2(k+1)}{2} \\
&= \frac{(k+1)(k+2)}{2}.
\end{aligned}$$

In the third line we wrote $k+1$ as $2(k+1)/2$, so that both terms have denominator 2. In the last line we factored out the common factor $k+1$. The result is the formula with $k+1$ in place of n, because $(k+1)\big((k+1)+1\big)/2 = (k+1)(k+2)/2$. By induction, the formula holds for every $n \in \mathbb{N}$.

Solution to Exercise 5.2

The first terms are $y_1 = 2 \cdot 3 = 6$, $y_2 = 2 \cdot 6 = 12$ and $y_3 = 2 \cdot 12 = 24$. Each term is twice the one before, so y_n should be the starting value 3 doubled n times. This suggests the formula $y_n = 3 \cdot 2^n$, in which the exponent n counts the doublings.

Base case. At $n = 0$ the formula gives $3 \cdot 2^0 = 3 \cdot 1 = 3$, which is y_0.

Induction step. Let n be a natural number, and assume that $y_n = 3 \cdot 2^n$. Then

$$y_{n+1} = 2y_n = 2 \cdot (3 \cdot 2^n) = 3 \cdot 2^{n+1}.$$

This is the formula at $n+1$. By induction, $y_n = 3 \cdot 2^n$ for every $n \in \mathbb{N}$.

Solution to Exercise 5.3

The divisions are

$$1071 = 2 \cdot 462 + 147, \qquad 462 = 3 \cdot 147 + 21, \qquad 147 = 7 \cdot 21 + 0.$$

The quotients are $2, 3, 7$, and the remainders are $147, 21, 0$. The successive pairs are

$$(1071, 462), \quad (462, 147), \quad (147, 21), \quad (21, 0).$$

The second number of the last pair is 0, so the algorithm stops and outputs 21. By Lemma 5.3, no step changes the greatest common divisor, so

$$\gcd(1071, 462) = \gcd(462, 147) = \gcd(147, 21) = \gcd(21, 0) = 21.$$

As a check, $1071 = 21 \cdot 51$ and $462 = 21 \cdot 22$. Notice that the answer is the last nonzero remainder, not the last quotient.

Solution to Exercise 5.4

No. Replace each plus sign by 1 and each minus sign by -1, and consider the product of the five numbers, as in Section 5.4. The starting row has product $1^5 = 1$. A legal move flips two signs, so it multiplies the product by $(-1) \cdot (-1) = 1$; the product does not change. By induction on the number of moves, every row that can be reached has product 1. A row with exactly three minus signs and two plus signs has product $(-1)^3 \cdot 1^2 = -1$. So no sequence of legal moves, however long, produces such a row.

The same argument can be phrased in terms of counting. A legal move changes the number of minus signs by +2, -2 or 0, so that number starts at 0 and always stays even. Three is odd, so a row with exactly three minus signs is never reached.

Solution to Exercise 5.5

First rewrite the rule in terms of the error. Subtracting 1 from both sides of $x_{n+1} = (x_n + 1)/2$ gives, for every $n \in \mathbb{N}$,

$$x_{n+1} - 1 = \frac{x_n + 1}{2} - 1 = \frac{x_n + 1 - 2}{2} = \frac{x_n - 1}{2}.$$

Now we prove $x_n - 1 = 2^{-n}(x_0 - 1)$ by induction on n.

Base case. At $n = 0$ the right side is $2^{-0}(x_0 - 1) = 2^0(x_0 - 1) = 1 \cdot (x_0 - 1) = x_0 - 1$, which is the left side.

Induction step. Let n be a natural number, and assume that $x_n - 1 = 2^{-n}(x_0 - 1)$. Then

$$x_{n+1} - 1 = \frac{x_n - 1}{2} = \frac{2^{-n}(x_0 - 1)}{2} = 2^{-(n+1)}(x_0 - 1),$$

using the halving rule above, then the induction hypothesis, then $2^{-n}/2 = 2^{-(n+1)}$. This is the formula at $n + 1$, which completes the induction.

The error is never zero. Suppose that $x_0 \neq 1$, and let n be any natural number. The number $2^{-n} = 1/2^n$ is positive, so it is not zero, and $x_0 - 1$ is not zero by assumption. A product of two nonzero real numbers is never zero, so $x_n - 1 = 2^{-n}(x_0 - 1) \neq 0$. The size of the error, $2^{-n}\,|x_0 - 1|$, halves at every step, so the terms come as close to 1 as we like; but unless the sequence starts at 1, no term is ever exactly equal to 1.

Solution to Exercise 5.6

The base case gives $P(0)$. The step with $k = 0$ then gives $P(2)$, the step with $k = 2$ gives $P(4)$, and so on. So the proof covers the even numbers $0, 2, 4, \ldots$. To make "and so on" precise, apply ordinary induction on m to the statement $P(2m)$: the base case $m = 0$ is $P(0)$, and the step with $k = 2m$ turns $P(2m)$ into $P(2m + 2)$, which is $P\big(2(m + 1)\big)$. So $P(2m)$ holds for every $m \in \mathbb{N}$. An even natural number n equals $2m$ with $m = n/2$, and this m is a natural number because $n \geq 0$. Hence $P(n)$ holds for every even natural number n.

The proof says nothing about the odd numbers, because each use of the step moves from an even number to the next even number. Indeed, take $P(n)$ to be the statement "n is even." Then $P(0)$ is true, and the step holds, because if k is even, so is $k+2$. Yet $P(n)$ is false for every odd n. So the two facts alone do not give $P(n)$ for any odd n. The answer to the first question is therefore the even natural numbers, and only those.

The additional base case $P(1)$ is enough. Apply ordinary induction on m to the statement $P(2m+1)$, exactly as for the even numbers: the base case $m = 0$ is $P(1)$, and the step with $k = 2m+1$ turns $P(2m+1)$ into $P(2m+3)$, which is $P\big(2(m+1)+1\big)$. So $P(2m+1)$ holds for every $m \in \mathbb{N}$; in other words, the chain $P(1), P(3), P(5), \ldots$ covers the odd natural numbers.

Finally, every natural number n is even or odd (Lemma 4.1). If n is even, $P(n)$ holds by the first paragraph. If n is odd, then $n = 2m+1$ for some integer m. Here $2m = n - 1 \geq -1$, so $m \geq -\frac{1}{2}$, and an integer that is at least $-\frac{1}{2}$ is at least 0. So $m \in \mathbb{N}$, and $P(n)$, which is $P(2m+1)$, holds by the previous paragraph. Therefore $P(n)$ is true for every natural number n.

A.6 Week 6

Solution to Exercise 6.1

The class $[-2]_5$ consists of the integers $-2+5k$, where k is an integer. Taking $k = -2, -1, 0, 1, 2$ gives

$$-12, \quad -7, \quad -2, \quad 3, \quad 8.$$

These are all the members between -12 and 12: the next members on either side, -17 (for $k = -3$) and 13 (for $k = 3$), lie outside the interval. The member of the class in $\{0, 1, 2, 3, 4\}$ is 3, since $3 - (-2) = 5$ is a multiple of five. So $[-2]_5 = [3]_5$. Division with remainder gives the same answer: $-2 = (-1) \cdot 5 + 3$ with $0 \leq 3 < 5$.

Solution to Exercise 6.2

An operation on classes is well-defined when different names for the same input classes always give the same output class. Suppose $[a']_m = [a]_m$ and $[b']_m = [b]_m$. By Proposition 6.2(b), $a' \equiv a$ and $b' \equiv b \pmod{m}$, so we can write

$$a' = a + mk, \qquad b' = b + m\ell$$

for integers k and ℓ. The output classes $[a'b']_m$ and $[ab]_m$ are equal exactly when $a'b' - ab$ is a multiple of m. Compute that difference:

$$\begin{aligned} a'b' - ab &= (a+mk)(b+m\ell) - ab \\ &= ab + am\ell + mkb + m^2k\ell - ab \\ &= m(a\ell + bk + mk\ell). \end{aligned}$$

The expression in parentheses is an integer, so the difference is divisible by m. Therefore $[a'b']_m = [ab]_m$. Notice what was proved: the two classes are equal, while the products $a'b'$

and ab themselves can be different integers. For example, with $m = 4$, the choices $a = 1$, $b = 2$ and $a' = 5$, $b' = 6$ give the products 2 and 30, which differ by $28 = 4 \cdot 7$, so $[30]_4 = [2]_4$. Both representatives were allowed to change at the same time, so the argument covers every possible choice of names for the two classes.

Solution to Exercise 6.3

Label the seats 0, 1, 2, 3. The rotation r_2 sends seat j to seat $(j + 2) \bmod 4$, so

$$r_2(0) = 2, \qquad r_2(1) = 3, \qquad r_2(2) = 0, \qquad r_2(3) = 1.$$

Applying r_2 twice sends $0 \to 2 \to 0$, $1 \to 3 \to 1$, $2 \to 0 \to 2$, and $3 \to 1 \to 3$. So $r_2 \circ r_2$ leaves every seat where it was, that is, $r_2 \circ r_2 = r_0 = \mathrm{id}$. By the definition in Section 3.2, a map h is an inverse of r_2 when $h \circ r_2 = \mathrm{id}$ and $r_2 \circ h = \mathrm{id}$. For $h = r_2$, both conditions say $r_2 \circ r_2 = \mathrm{id}$, which we have just checked. So r_2 undoes itself: it is its own inverse. It is not the identity, because the identity leaves every seat in place, while r_2 moves seat 0 to seat 2. A symmetry that undoes itself need not do nothing.

Solution to Exercise 6.4

Yes. Write $X \sim Y$ when $|X| = |Y|$, that is, when X and Y have the same number of elements. Let X, Y, Z be sets in the collection.

- *Reflexivity:* $|X| = |X|$, so $X \sim X$.
- *Symmetry:* if $|X| = |Y|$, then $|Y| = |X|$, so $X \sim Y$ implies $Y \sim X$.
- *Transitivity:* if $|X| = |Y|$ and $|Y| = |Z|$, then $|X| = |Z|$, so $X \sim Y$ and $Y \sim Z$ imply $X \sim Z$.

Each property of $\sim$ comes from the same property of equality of numbers. Therefore the relation is an equivalence relation. The equivalence class of a set X consists of all sets in the collection that have as many elements as X; for instance, if $\{1, 2\}$ and $\{p, q\}$, with $p \neq q$, both belong to the collection, they lie in the same class. The relation sorts sets by size and ignores what their members are.

Solution to Exercise 6.5

The rule takes a class modulo m, chooses a representative a, and returns $[a]_d$. It is well-defined when any two representatives of the same class modulo m give the same class modulo d, that is, when $[a]_m = [b]_m$ always implies $[a]_d = [b]_d$. We prove the two directions of "exactly when" separately.

First suppose d divides m, and write $m = dt$ with t an integer. Let $[a]_m = [b]_m$. By Proposition 6.2(b), $a - b = mk$ for an integer k. Substituting $m = dt$ gives $a - b = d(tk)$, and tk is an integer, so d divides $a - b$. By the same proposition, $[a]_d = [b]_d$. Thus any two representatives of one input class give the same output class, and the rule is well-defined.

Conversely, suppose the rule is well-defined. The integers 0 and m represent the same class modulo m, because $m - 0 = m \cdot 1$. Their output classes must therefore agree: $[0]_d = [m]_d$. By Proposition 6.2(b) once more, d divides $m - 0 = m$.

For example, passing from classes modulo six to classes modulo three works, since 3 divides 6. Passing from classes modulo six to classes modulo four does not: 0 and 6 represent the same class modulo six, but $[0]_4 \neq [6]_4$, because $6 - 0 = 6$ is not a multiple of four.

Solution to Exercise 6.6

Use the family $\{\{1\}, \{1,2\}, \{3\}\}$, ordered by inclusion. The set $\{1,2\}$ is maximal, because neither of the other two members contains it. The set $\{3\}$ is maximal for the same reason. The set $\{1\}$ is not maximal, because $\{1\} \subseteq \{1,2\}$ and $\{1\} \neq \{1,2\}$. So there are exactly two maximal members. There is no maximum. A maximum is always maximal (Section 6.6), so it would have to be $\{1,2\}$ or $\{3\}$. But $\{3\}$ is not a subset of $\{1,2\}$, so $\{1,2\}$ is not a maximum, and $\{1,2\}$ is not a subset of $\{3\}$, so $\{3\}$ is not a maximum either.

Now add the set $\{1,2,3\}$. Every member of the new family $\{\{1\}, \{1,2\}, \{3\}, \{1,2,3\}\}$ is a subset of $\{1,2,3\}$:

$$\{1\} \subseteq \{1,2,3\}, \qquad \{1,2\} \subseteq \{1,2,3\}, \qquad \{3\} \subseteq \{1,2,3\}, \qquad \{1,2,3\} \subseteq \{1,2,3\}.$$

So $\{1,2,3\}$ is a maximum of the new family. Notice that $\{1,2\}$ and $\{3\}$ are no longer maximal, because $\{1,2,3\}$ lies strictly above each of them. Whether an element is maximal, or a maximum, always depends on the family it is compared with.

APPENDIX B

Solutions for Chapters 7–12

This appendix contains the solutions for Chapters 7–12: graphs, matchings and Hall's theorem, vectors and linear maps, limits and completeness, and the contraction theorem. The solution to Exercise 9.6 is a complete model of the midterm portfolio. The advice at the start of Appendix A on how to use the solutions applies here as well.

B.1 Week 7

Solution to Exercise 7.1

(a) The new graph has the five edges ab, bc, ca, cd, and bd. Reading off the neighbors of each vertex gives

$$\deg(a) = 2, \qquad \deg(b) = 3, \qquad \deg(c) = 3, \qquad \deg(d) = 2,$$

since a is joined to b, c; b to a, c, d; c to a, b, d; and d to b, c. The degree sum is $2 + 3 + 3 + 2 = 10 = 2 \cdot 5$, twice the number of edges, as Theorem 7.2 requires. The vertices of odd degree are b and c, so there are two of them, an even number, as Proposition 7.3 requires.

(b) Deleting cd from G leaves the three edges ab, bc, and ca. Now

$$\deg(a) = 2, \qquad \deg(b) = 2, \qquad \deg(c) = 2, \qquad \deg(d) = 0.$$

The vertex d is still part of the graph, but it has no neighbors, so its degree is 0. The degree sum is $2 + 2 + 2 + 0 = 6 = 2 \cdot 3$, again twice the number of edges. No vertex has odd degree, so the number of vertices of odd degree is 0, which is even.

In both parts, changing a single edge changed the degrees of exactly its two endpoints, each by one. So the degree sum changed by exactly 2, and the parities of exactly two degrees changed. In part (a), b became odd and d became even, so the count of odd-degree vertices stayed at two; in part (b), c and d both became even, so the count dropped from two to zero.

Solution to Exercise 7.2

Recall from Section 5.5 that the remainder of an integer on division by four is one of the four numbers $0, 1, 2, 3$; this holds for negative integers too, for example $-7 = (-2) \cdot 4 + 1$ has remainder 1. Use these four remainders as boxes, and place each index $i \in \{1, 2, 3, 4, 5\}$ in the box of the remainder of x_i. Five indices are placed in four boxes, and $5 > 4 \cdot 1$, so by the pigeonhole principle (Proposition 7.4 with $k = 1$) some box contains at least two different indices $i \neq j$. Then x_i and x_j leave the same remainder.

We placed the indices, not the values, in the boxes. This matters when two of the integers are equal: if, say, $x_1 = x_2 = 7$, the indices 1 and 2 land in the same box, and the conclusion holds with $i = 1$ and $j = 2$.

Four integers are not enough: $0, 1, 2, 3$ leave the four different remainders $0, 1, 2, 3$, so no two of them share a remainder. Here the pigeonhole principle does not apply, because 4 is not larger than $4 \cdot 1$.

Solution to Exercise 7.3

Take the directed cycle $a \to b \to c \to a$. It is an oriented graph: there are no loops, and each pair of vertices is joined in only one direction. It is nonempty and finite. Every vertex has an out-neighbor, since $a \to b$, $b \to c$, and $c \to a$, so there is no sink. The only hypothesis of Theorem 7.8 that fails is acyclicity, because a, b, c, a is a directed cycle.

Every nonempty oriented graph with fewer vertices has a sink. (The word "nonempty" matters: the graph with no vertices has no sink either, which is why Theorem 7.8 assumes nonemptiness.) With one vertex there can be no edges, because a directed edge needs two different vertices; so the vertex is a sink. With two vertices u and v, an oriented graph has at most one edge between them. If there is no edge, both vertices are sinks. If the edge is $u \to v$, then the only possible out-neighbor of v would be u, and $v \to u$ is forbidden because $u \to v$ is present; so v is a sink. (The case $v \to u$ is the same with the names exchanged.) Hence three is the smallest possible number of vertices, and the directed cycle on three vertices is a smallest counterexample.

In the proof of Theorem 7.8, a longest directed path in this graph is, for example, $a \to b \to c$; no directed path can have more than three vertices. Its last vertex c has the out-neighbor a, which lies on the path. This is the second case of the proof: the arrow $c \to a$ closes the directed cycle $a \to b \to c \to a$. The proof reached a contradiction in that case only by using acyclicity. Without that hypothesis there is no contradiction, and the argument cannot conclude that c is a sink.

Solution to Exercise 7.4

At a, the first out-neighborhood is $N_1^+(a) = \{b, c\}$. The two-step routes from a are $a \to b \to d$ and $a \to c \to d$. Both end at d, which is different from a and is not a first out-neighbor of a, so $N_2^+(a) = \{d\}$. The vertex d is one member of this set, even though two routes reach it, so $|N_2^+(a)| = 1$.

At b, we have $N_1^+(b) = \{d\}$. The vertex d has no out-neighbors, so there is no two-step route from b, and $N_2^+(b) = \varnothing$. The vertex c behaves in exactly the same way: $N_1^+(c) = \{d\}$ and $N_2^+(c) = \varnothing$. Finally, d is a sink, so $N_1^+(d) = N_2^+(d) = \varnothing$.

Comparing sizes: at a we get $1 < 2$, at b and at c we get $0 < 1$, and at d we get $0 \geq 0$. So only d satisfies $|N_2^+(v)| \geq |N_1^+(v)|$.

This agrees with Proposition 7.11, because the graph is acyclic and d is its sink. To see that there is no directed cycle, note that every vertex on a directed cycle has an arrow of the cycle entering it and one leaving it. No arrow enters a and no arrow leaves d, so a directed cycle could use only b and c. A cycle has at least one arrow, and that arrow would join two different vertices of the cycle,

that is, b and c. But there is no arrow between b and c in either direction.

Solution to Exercise 7.5

By the handshake theorem, the n degrees add up to $2m$, so their average is $2m/n$. By the second half of the averaging principle (Proposition 7.5), some vertex has degree at most this average. Written out directly: if every degree were strictly greater than $2m/n$, adding these n inequalities would give a degree sum strictly greater than $n \cdot (2m/n) = 2m$, which contradicts the handshake theorem.

The conclusion is that *some* degree is at most the average; it says nothing about how far the other degrees may be from it. For an example, let $n \geq 3$ and take a *star*: one vertex, the center, is joined to each of the other $n - 1$ vertices, called leaves, and there are no other edges. The star has $m = n - 1$ edges, so its average degree is

$$\frac{2(n-1)}{n} = 2 - \frac{2}{n},$$

which is less than 2. Every leaf has degree 1, which is below the average, since $2 - 2/n \geq 2 - 2/3 = 4/3 > 1$ for $n \geq 3$. The center, however, has degree $n - 1$. For $n = 101$, the average degree is $200/101 \approx 1.98$, while the center has degree 100. As n grows, the center's degree grows without bound, while the average stays below 2. So knowing the average does not prevent an individual degree from being very far from it.

Solution to Exercise 7.6

No. Take the tournament on the vertices a, b, c with $a \to b$, $a \to c$, and $b \to c$. Each of the three pairs is joined by exactly one directed edge, so this is a tournament. The vertex a is a king, because it reaches both other vertices in one step.

Now compute the neighborhoods of a. Its first out-neighborhood is $N_1^+(a) = \{b, c\}$. The only two-step route from a is $a \to b \to c$, since c has no out-neighbors. That route ends at c, which is already a first out-neighbor of a, so c is excluded and $N_2^+(a) = \varnothing$. Thus $|N_2^+(a)| = 0 < 2 = |N_1^+(a)|$, and the king a fails the inequality.

The two properties measure different things. Being a king means that every other vertex is within directed distance two. The inequality compares how many vertices are at directed distance exactly one with how many are at directed distance exactly two. A king that beats almost everyone directly has a large first out-neighborhood and leaves few vertices for the second. Some vertex of this tournament does satisfy the inequality, as Proposition 7.11 guarantees. The tournament is acyclic: no arrow enters a and no arrow leaves c, so a directed cycle could use only the vertex b, which is impossible, because every arrow joins two different vertices. Its sink c satisfies $0 \geq 0$.

B.2 Week 8

Solution to Exercise 8.1

The graph has three edges: Ada–1, Ada–2, and Ben–1.

The matchings. The empty set is a matching, and so is each single edge: {Ada–1}, {Ada–2}, and {Ben–1}. Of the three sets with two edges, {Ada–1, Ada–2} is not a matching, because both edges have the endpoint Ada, and {Ada–1, Ben–1} is not a matching, because both edges have the endpoint 1. The remaining one, {Ada–2, Ben–1}, is a matching: its two edges have different students and different tasks. The set of all three edges contains Ada–1 and Ada–2, so it is not a matching. Hence the graph has exactly five matchings.

Which are maximal. The empty matching is not maximal, since any edge can be added to it. Ben–1 can be added to {Ada–2}, and Ada–2 can be added to {Ben–1}, so these are not maximal either. The matching {Ada–1} is maximal: the two edges outside it each share an endpoint with Ada–1, the edge Ada–2 sharing the endpoint Ada and the edge Ben–1 sharing the endpoint 1. The matching {Ada–2, Ben–1} is maximal too: the only edge outside it is Ada–1, and adding it would give Ada two tasks.

Which are maximum. The largest number of edges in the list is two, so {Ada–2, Ben–1} is the only maximum matching.

So exactly two matchings are maximal, {Ada–1} and {Ada–2, Ben–1}, and only the second is maximum. Hence {Ada–1} is maximal but not maximum, and no other matching is maximal without being maximum. The example shows the difference between the two words: a maximal matching cannot be enlarged while keeping all of its current edges, whereas a maximum matching is compared with every matching in the graph, including those that undo current choices.

Solution to Exercise 8.2

In the matching $\{a\text{–}2,\ b\text{–}1\}$, the student c is unmatched, tasks 1 and 2 are used, and task 3 is unused. Search from c.

- Inspect c. Its only edge, c–2, is unmatched, so task 2 is reached with predecessor c. Task 2 is held by a, so a is reached with predecessor 2.
- Inspect a. The edge a–2 is matched; the edge a–1 is unmatched, so task 1 is reached with predecessor a. Task 1 is held by b, so b is reached with predecessor 1.
- Inspect b. The edge b–1 is matched; the edge b–3 is unmatched, so task 3 is reached with predecessor b. Task 3 is unused, and the search succeeds.

Following the predecessors back from task 3 gives the augmenting path

$$c - 2 - a - 1 - b - 3.$$

Its edges c–2, a–1, and b–3 are unmatched, its edges a–2 and b–1 are matched, and its ends c and 3 are unmatched. Flipping removes a–2 and b–1 and adds the other three edges, which gives the

matching

$$\{a\text{–}1,\ b\text{–}3,\ c\text{–}2\}.$$

It has three edges and saturates the set of students: each student receives an acceptable task, and the tasks 1, 3, 2 are different. Both a and b changed tasks, a from 2 to 1 and b from 1 to 3, so that c could receive the only task it accepts.

Solution to Exercise 8.3

Let S be the set of all three students. Then $N(S) = \{1, 2\}$, so $|N(S)| = 2 < 3 = |S|$, and S is a shortage set. It certifies that no full assignment exists: a full assignment would give the three students three different tasks, all of them in $N(S)$, but $N(S)$ has only two members. Anyone can check this certificate by listing the tasks the three students accept and counting them.

Each student has two options, but adding up the options, $2 + 2 + 2 = 6$, counts the same two tasks again and again. What matters is the number of *different* tasks available to the group, and that number is 2. No smaller group is a shortage set: the empty group accepts 0 tasks, which is not fewer than its 0 members; one student accepts $2 \geq 1$ tasks; and any two students together accept $2 \geq 2$ tasks. Indeed, any two of the students can be given different tasks; the shortage appears only when all three compete.

Solution to Exercise 8.4

(a) Write the augmenting path as $v_0, v_1, \ldots, v_k$. The proof uses the fact that its vertices are all different in two places. First, it concludes that an inner vertex v_i lies on exactly two edges of the path, one matched and one unmatched, and that an end lies on exactly one; so after the flip each of these vertices lies on exactly one edge of the new matching. Second, it counts the edges of the path as $r + 1$ unmatched and r matched ones, all different. If a vertex appeared twice, it could lie on two unmatched edges of the walk, both of which the flip adds, and it would then have two partners.

Here is an example. Take students x, s, u and tasks 1, 2, 3, with edges x–1, s–1, s–2, s–3, u–1, u–2, and the matching $M = \{s\text{–}1,\ u\text{–}2\}$. Student x is unmatched and task 3 is unused. The walk

$$x,\ 1,\ s,\ 2,\ u,\ 1,\ s,\ 3$$

uses the edges x–1, s–1, s–2, u–2, u–1, s–1, s–3, which are unmatched, matched, unmatched, matched, unmatched, matched, unmatched in turn. So it is an alternating walk from an unmatched student to an unused task, but it visits task 1 and student s twice. Flipping along it removes the matched edges s–1 and u–2 and adds the unmatched edges x–1, s–2, u–1, s–3. The result

$$\{x\text{–}1,\ s\text{–}2,\ u\text{–}1,\ s\text{–}3\}$$

is not a matching: task 1 lies on two of its edges, and so does student s. Cutting out the part of the walk between the two visits to task 1 leaves $x, 1, s, 3$, which is an augmenting path; flipping along it gives the matching $\{x\text{–}1,\ s\text{–}3,\ u\text{–}2\}$.

(b) Consider all alternating walks that start at an unmatched student and end at an unused task. Their lengths form a set of natural numbers, which is nonempty by assumption, so by the

well-ordering property there is a shortest such walk. Call it $v_0, v_1, \ldots, v_k$, and write $x = v_0$. Since x is a student and v_k is a task, $k \geq 1$. The graph is bipartite, so the vertices of the walk alternate between students and tasks: v_i is a student when i is even and a task when i is odd. The first edge is unmatched, because x is unmatched, and the edges alternate, so the mth edge of the walk is unmatched when m is odd and matched when m is even.

Suppose, for contradiction, that $v_i = v_j$ for some $i < j$. Since v_i and v_j are the same vertex, they lie on the same side, so i and j have the same parity, and $j - i$ is even. Cut out the entries $v_{i+1}, \ldots, v_j$, as in the proof of Lemma 7.6. What remains, $v_0, \ldots, v_i, v_{j+1}, \ldots, v_k$, is a walk from x to the same unused task, for the reason given in that proof. Its first i edges are unchanged. If $j < k$, each later edge is an edge of the old walk whose position number has dropped by $j - i$: for example, the new $(i+1)$st edge $v_i v_{j+1}$ is the old $(j+1)$st edge $v_j v_{j+1}$. Since $j - i$ is even, each of these edges keeps the parity of its position, so it is still unmatched exactly when its position is odd. Therefore the shorter walk is still alternating. It has length $k - (j - i) < k$, which contradicts the choice of a shortest walk. Hence a shortest walk repeats no vertex. It is an alternating path with at least one step whose ends are both unmatched, that is, an augmenting path.

The search of Section 8.4 produces an augmenting path directly: following its predecessor records backward never repeats a vertex, because each vertex was reached later than its predecessor.

Solution to Exercise 8.5

The fact that a quantity increases by one at every step does not by itself rule out infinitely many steps. The counter $0, 1, 2, 3, \ldots$ increases by one at every step and never stops. To conclude that a procedure stops, we also need a ceiling that the quantity can never pass.

Finiteness of L provides this ceiling. Each edge of a matching contains exactly one student, and no two edges of a matching contain the same student. So sending each edge to its student is an injective map from the matching to L, and a matching has at most $|L|$ edges. Suppose the procedure starts from a matching with k edges. After m steps the matching has $k + m$ edges, so $k + m \leq |L|$, that is, $m \leq |L| - k$. Hence at most $|L| - k$ steps can occur, and at most $|L|$ when the procedure starts from the empty matching ($k = 0$). In the language of Habit 5, the nonnegative integer $|L| - |M|$ decreases by one at every step, so it cannot keep decreasing forever.

Solution to Exercise 8.6

(a) The search reaches students and tasks by two moves: from a reached student s along an unmatched edge s–t to the task t, and from a reached used task to its owner. A task is reached only by the first move, and a student other than x only by the second.

$B \subseteq N(A)$. Every task t in B was reached by the first move, from some student s in A along an edge s–t. So s accepts t, and $t \in N(A)$.

$N(A) \subseteq B$. Let $t \in N(A)$, and choose a student $s \in A$ that accepts t.

- *The edge s–t is unmatched.* Because the search is complete, every reached student has been inspected, and inspecting s follows every unmatched edge at s. So t has been reached, and $t \in B$.

- *The edge s–t is matched.* Then s is matched, so $s \neq x$, since x is unmatched. A student other than x is reached only by a move from the task it holds, and the task s holds is t, its unique partner. So t was reached before s, and $t \in B$.

In both cases $t \in B$. The two containments together give $N(A) = B$.

Completeness is used only in the unmatched case.

(b) The containment $B \subseteq N(A)$ must still hold, with the same proof: every reached task was reached from a reached student along an edge. The matched case in the proof of $N(A) \subseteq B$ also survives, because it uses only the way students are reached. Only the unmatched case can fail, because a student that has not been completely inspected may have an unmatched edge that the search never followed. So $N(A) \subseteq B$ need not hold.

Here is an example. Let Ada accept tasks 1 and 2, let Ben accept only task 1, and let $M = \{\text{Ben–1}\}$. Search from Ada, and stop right after reaching task 1 and its owner Ben, before following the edge Ada–2. Then $A = \{\text{Ada}, \text{Ben}\}$ and $B = \{1\}$, but $N(A) = \{1, 2\}$, so $N(A) \not\subseteq B$.

Now take any early-stopped search in which $N(A) \subseteq B$ fails. Step 1 of the proof of Proposition 8.6 still works. Every reached task is used, because the search has reached no unused task, and its owner has been reached, because the owner of a used task is marked as reached at the moment the task is. Every reached student other than x was reached from the task it holds. So the argument of Step 1 gives $|B| = |A| - 1$. Since $B \subseteq N(A)$ and $N(A)$ contains at least one task outside B, we get $|N(A)| \geq |B| + 1 = |A|$. Hence A is not a shortage set. In the example, $|N(A)| = 2 = |A|$, and indeed $\{\text{Ada–2}, \text{Ben–1}\}$ is a full assignment.

B.3 Week 9

Solution to Exercise 9.1

Let

$$N(a) = N(b) = N(c) = \{1, 2\}, \qquad N(d) = \{3, 4\}.$$

Every student accepts two tasks, and $N(L) = \{1, 2, 3, 4\}$, so $|N(L)| = 4 = |L|$. The group $S = \{a, b, c\}$ has $N(S) = \{1, 2\}$, so $|N(S)| = 2 < 3 = |S|$. A full assignment would give a, b and c three different tasks, each acceptable to one of them and therefore lying in $N(S)$. But $N(S)$ has only two members, so no matching saturates L.

The tests for single students read $|N(x)| = 2 \geq 1$, and the test for the whole class reads $4 \geq 4$. All of them pass. The shortage lies in the group of three students who compete for the same two tasks, and only the test for that group detects it. In fact, checking all sixteen groups shows that $\{a, b, c\}$ is the only group that fails.

Solution to Exercise 9.2

The complete check is

S	$N(S)$	$\lvert N(S)\rvert \geq \lvert S\rvert$
$\varnothing$	$\varnothing$	$0 \geq 0$
$\{a\}$	$\{1\}$	$1 \geq 1$
$\{b\}$	$\{1, 2\}$	$2 \geq 1$
$\{c\}$	$\{2, 3\}$	$2 \geq 1$
$\{a, b\}$	$\{1, 2\}$	$2 \geq 2$
$\{a, c\}$	$\{1, 2, 3\}$	$3 \geq 2$
$\{b, c\}$	$\{1, 2, 3\}$	$3 \geq 2$
$\{a, b, c\}$	$\{1, 2, 3\}$	$3 \geq 3$

Every inequality holds, so the graph satisfies Hall's condition. A group is tight when the two sides of its inequality are equal, so the tight groups are $\varnothing$, $\{a\}$, $\{a, b\}$ and $\{a, b, c\}$. The empty group is excluded because it is empty, and the whole class $\{a, b, c\}$ because it is not proper. So the nonempty proper tight subsets are exactly $\{a\}$ and $\{a, b\}$.

A matching that saturates L is a–1, b–2, c–3. Each pair is an edge, because $1 \in N(a)$, $2 \in N(b)$ and $3 \in N(c)$, and the three tasks are different. It is the only such matching: a must take task 1, so b must take task 2, and then c must take task 3. Once this matching is known, the necessary direction of Hall's theorem (Section 9.2) guarantees all eight inequalities at once, without the table.

Solution to Exercise 9.3

Write $X = N(S)$ and $Y = N(T) \setminus N(S)$. We show that $N(S) \cup N(T) = X \cup Y$ and that X and Y are disjoint.

First, $X \cup Y \subseteq N(S) \cup N(T)$, because $X = N(S)$ and $Y \subseteq N(T)$. Conversely, let $t \in N(S) \cup N(T)$. If $t \in N(S)$, then $t \in X$. If $t \notin N(S)$, then t must lie in $N(T)$, because it lies in the union, and so $t \in N(T) \setminus N(S) = Y$. In both cases $t \in X \cup Y$. Hence $N(S) \cup N(T) = X \cup Y$.

Next, X and Y are disjoint, because by the definition of a set difference every member of Y lies outside $N(S) = X$.

For disjoint finite sets, the size of the union is the sum of the sizes (Section 3.1). Therefore

$$\lvert N(S) \cup N(T)\rvert = \lvert X\rvert + \lvert Y\rvert = \lvert N(S)\rvert + \lvert N(T) \setminus N(S)\rvert.$$

Nowhere did we assume that $N(S)$ and $N(T)$ are disjoint. If they overlap, each common task is counted once, as a member of $N(S)$, and the set difference $N(T) \setminus N(S)$ leaves it out. In the example of Section 9.4, $N(S) = \{1, 2\}$ and $N(\{c\}) = \{2, 3\}$. Their union $\{1, 2, 3\}$ has $3 = 2 + \lvert\{3\}\rvert$ members, whereas $\lvert N(S)\rvert + \lvert N(\{c\})\rvert = 4$ would count task 2 twice.

Solution to Exercise 9.4

The sizes $\lvert N(S)\rvert$ and $\lvert S\rvert$ are natural numbers, so their difference $k = \lvert N(S)\rvert - \lvert S\rvert$ is an integer. Moreover, $k > 0$ because $\lvert N(S)\rvert > \lvert S\rvert$. There is no integer strictly between 0 and 1, so $k \geq 1$, that is, $\lvert N(S)\rvert \geq \lvert S\rvert + 1$.

For arbitrary real numbers this step is not valid, because a positive real number can be smaller than 1. For example, $1.2 > 1$, but $1.2 \geq 1 + 1 = 2$ is false. The extra unit in Case 2 comes from the fact that the sizes of finite sets are integers.

Solution to Exercise 9.5

By Hall's theorem, it is enough to show that $|N(S)| \geq |S|$ for every set S of students. Fix such a set S, and let E_S be the set of edges whose student endpoint lies in S. We count E_S in two ways.

By students. Each student of S accepts exactly d tasks, so it lies on exactly d edges. Each edge has only one student endpoint, so every edge of E_S is counted exactly once. Hence $|E_S| = d|S|$.

By tasks. The task endpoint of each edge in E_S is acceptable to a member of S, so it lies in $N(S)$. Group the edges of E_S according to their task endpoint, one group for each of the $|N(S)|$ tasks of $N(S)$. A task of $N(S)$ is acceptable to at most d students in the whole graph, so its group has at most d edges. Adding up the sizes of the groups counts every edge of E_S exactly once. Hence $|E_S| \leq d|N(S)|$.

Comparing the two counts,

$$d|S| \leq d|N(S)|.$$

Since $d > 0$, we may divide both sides by d; dividing by a positive number keeps the direction of an inequality. This gives $|S| \leq |N(S)|$. The argument works for every $S \subseteq L$, including $S = \varnothing$, where both counts are 0. So the graph satisfies Hall's condition, and Hall's theorem gives a matching that saturates L, which is a full assignment. For instance, with $d = 2$, the students a, b, c with $N(a) = \{1, 2\}$, $N(b) = \{2, 3\}$ and $N(c) = \{1, 3\}$ satisfy the hypotheses, and a–1, b–2, c–3 is a full assignment.

The proof uses $d > 0$ at exactly one point, the division. If $d = 0$, the inequality $d|S| \leq d|N(S)|$ reads $0 \leq 0$ and says nothing about $|S|$ and $|N(S)|$. The conclusion fails too: when $d = 0$, no student accepts any task, so there is no full assignment unless there are no students at all.

Solution to Exercise 9.6

Here is a model portfolio.

(a) An assignment certificate. For the graph of Exercise 9.2, with $N(a) = \{1\}$, $N(b) = \{1, 2\}$ and $N(c) = \{2, 3\}$, the matching a–1, b–2, c–3 saturates L. A reader checks it by confirming that each pair is an edge ($1 \in N(a)$, $2 \in N(b)$, $3 \in N(c)$), that each student appears in exactly one pair, and that the three tasks are different.

(b) A shortage certificate. For the graph of Exercise 9.1, with $N(a) = N(b) = N(c) = \{1, 2\}$ and $N(d) = \{3, 4\}$, take $S = \{a, b, c\}$. A reader checks it by listing the tasks that the members of S accept and collecting them: $N(S) = \{1, 2\}$, so $|N(S)| = 2 < 3 = |S|$. This proves that no full assignment exists. In a full assignment the three students of S would receive three different tasks, each acceptable to a member of S and therefore lying in $N(S)$, but $N(S)$ has only two members. This is the necessary direction of Hall's theorem (Section 9.2).

(c) A proof of sufficiency. *Claim: let G be a finite bipartite graph with parts L and R such that $|N(S)| \geq |S|$ for every $S \subseteq L$. Then G has a matching that saturates L.*

The proof below follows Section 9.3. It quotes Lemma 8.5 and the basic facts about the search from Section 8.4, and it repeats the proof of Proposition 8.6, because that is the step where Hall's condition is used.

Proof. Start with the empty matching M. As long as some student u is unmatched, carry out a round, as follows. Run the complete search of Section 8.4 from u. The search first reaches u. From each reached student it follows every edge not in M and reaches the task at the other end; when it reaches a task held by some student, it also reaches that task's owner. Each vertex is reached at most once (Section 8.4), and for each reached vertex other than u the search remembers its predecessor, the vertex from which it was reached. The search stops as soon as it reaches an unused task. Otherwise it continues until every reached student has been inspected. The graph is finite, so the search ends.

If the search reaches an unused task t, follow the predecessors back from t. Each vertex was reached later than its predecessor, so no vertex is met twice. There are only finitely many vertices, so the backward walk cannot go on forever. It can stop only at a reached vertex without a predecessor, and the only such vertex is u. So the backward walk ends at u, and read forward its vertices form a path from u to t. Each step into a task uses an edge not in M, and each step into a student goes from a task to its owner along an edge of M. Hence the edges of the path alternate between edges not in M and edges of M, and the path runs from the unmatched student u to the unused task t. It has at least one step, because u is a student and t is a task. It is therefore an augmenting path, and by Lemma 8.5, flipping M along it gives a matching with one more edge.

Suppose instead that the search ends without reaching an unused task. Let A be the set of reached students, including u, and let B be the set of reached tasks. Every task in B is held by some student, because no unused task was reached, and the search reached that owner, so the owner lies in A. The owner is not u, because u is unmatched. Conversely, every student in A other than u was reached as the owner of a task in B. A matching gives each task at most one owner and each student at most one task, so sending each task of B to its owner is a bijection from B to $A \setminus \{u\}$. Hence $|B| = |A| - 1$.

Moreover, $N(A) = B$ (compare Exercise 8.6). Every reached task was reached along an edge from a reached student, so $B \subseteq N(A)$. Conversely, let $s \in A$ and let t be a task that s accepts. If the edge s–t is not in M, then t was reached: a search that fails has inspected every reached student, including s, and inspecting s reaches t along this edge unless t has been reached already. So $t \in B$. If s–t is in M, then s is matched, so $s \neq u$, and s was reached as the owner of its own task, which is t; again $t \in B$. Therefore

$$|N(A)| = |B| = |A| - 1 < |A|,$$

which contradicts Hall's condition for the set A. So this outcome never occurs, and every round adds one edge to M.

Each edge of a matching contains exactly one student, and different edges contain different students, so M never has more than $|L|$ edges. Since M starts with no edges and gains one in every round, there are at most $|L|$ rounds. The rounds stop only when every student is matched, so the final matching saturates L. If $L = \varnothing$, there are no rounds, and the empty matching saturates L.

This completes the proof.

(d) Maximal and maximum matchings. A matching is *maximal* if no edge of the graph can be added to it without creating a shared endpoint, and *maximum* if no matching in the graph has more edges. Every maximum matching is maximal, because an edge that could be added would produce a larger matching. The converse fails. Suppose Ada accepts tasks 1 and 2 and Ben accepts only task 1. The matching {Ada–1} is maximal: adding Ada–2 would give Ada two tasks, and adding Ben–1 would give task 1 two owners. But it is not maximum, because {Ada–2, Ben–1} has two edges (Exercise 8.1). A greedy procedure hands out tasks one at a time and never takes a task back. When it has finished, its matching is maximal, because each student left without a task found every acceptable task already taken. But that matching need not be maximum. In the example, a greedy procedure that starts by giving Ada task 1 finishes with {Ada–1} and leaves Ben without a task, although a full assignment exists. Getting stuck therefore proves nothing about impossibility. The augmenting-path repair revises earlier choices: here it follows the path Ben–1–Ada–2, moves Ada to task 2, and frees task 1 for Ben. Only a shortage certificate, as in part (b), proves that no full assignment exists.

B.4 Week 10

Solution to Exercise 10.1

The row rule gives

$$\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 2 \cdot 1 + 1 \cdot (-2) \\ 0 \cdot 1 + 3 \cdot (-2) \end{pmatrix} = \begin{pmatrix} 0 \\ -6 \end{pmatrix}.$$

Now use the columns. The columns of A are $Ae_1 = (2, 0)$ and $Ae_2 = (1, 3)$, and $x = (1, -2) = 1e_1 - 2e_2$. By linearity,

$$Ax = Ae_1 - 2Ae_2 = (2, 0) + (-2, -6) = (0, -6),$$

the same answer. The coordinates of x say how much of each column to take: one copy of the first column and minus two copies of the second.

Solution to Exercise 10.2

The kernel consists of the pairs with $x + y = 0$, that is, with $y = -x$. Writing t for the first coordinate, these are the pairs $(t, -t)$, so

$$\ker T = \{(t, -t) : t \in \mathbb{R}\}.$$

Similarly, $x + y = 5$ exactly when $y = 5 - x$, so the solutions of $T(x, y) = 5$ are the pairs $(t, 5 - t)$ with $t \in \mathbb{R}$. This matches the description in Section 10.3: $(0, 5)$ is one solution, and every solution has the form

$$(t, 5 - t) = (0, 5) + (t, -t),$$

one particular solution plus a vector in the kernel. The map remembers the sum $x + y$ but forgets how that sum is split between the two coordinates. Adding a kernel vector $(t, -t)$ moves an amount

t from the second coordinate to the first, which changes the split but not the sum; these are exactly the changes that T cannot see. For instance, $(0, 5)$ and $(1, 4)$ both have output 5, so T is not injective.

Solution to Exercise 10.3

Write $U(s,t) = \big((s+t)/2, (s-t)/2\big)$. By the definition of an inverse in Section 3.2, we must check that $U(T(x,y)) = (x,y)$ for every $(x,y) \in \mathbb{R}^2$ and that $T(U(s,t)) = (s,t)$ for every $(s,t) \in \mathbb{R}^2$.

For the first, $T(x,y) = (x+y, x-y)$, so we apply U with $s = x+y$ and $t = x-y$:

$$U(T(x,y)) = \left(\frac{(x+y)+(x-y)}{2},\ \frac{(x+y)-(x-y)}{2}\right) = \left(\frac{2x}{2},\ \frac{2y}{2}\right) = (x,y).$$

For the second, T adds and subtracts the two coordinates of $U(s,t)$:

$$T(U(s,t)) = \left(\frac{s+t}{2} + \frac{s-t}{2},\ \frac{s+t}{2} - \frac{s-t}{2}\right) = \left(\frac{2s}{2},\ \frac{2t}{2}\right) = (s,t).$$

Both compositions are identity maps, so U is the inverse of T, in agreement with the inverse $b \mapsto \frac{1}{2}Ab$ found in Section 10.3. Both checks are needed, and each must hold at every input, not only at a few chosen ones.

Solution to Exercise 10.4

The dot product is $u \cdot v = 1 \cdot 2 + 2 \cdot (-1) = 0$. The squared lengths are $1^2 + 2^2 = 5$ and $2^2 + (-1)^2 = 5$, so both lengths equal $\sqrt{5}$. The Cauchy–Schwarz inequality reads $|0| \le \sqrt{5} \cdot \sqrt{5} = 5$, which is true with a large gap. The inequality is strict, as the equality case of Section 10.5 predicts. Since $v \neq 0$, equality would require u to be a multiple of v, and it is not: $(1,2) = c(2,-1)$ would need both $2c = 1$ and $-c = 2$, which is impossible. Notice also that the two vectors are orthogonal although neither of them is zero.

Solution to Exercise 10.5

Both sides of the equation $\|u+v\|_2 = \|u\|_2 + \|v\|_2$ are nonnegative, and nonnegative numbers are equal exactly when their squares are equal (Section 10.5). So we must decide when $\|u+v\|_2^2 = (\|u\|_2 + \|v\|_2)^2$. The proof in Section 10.5 gives the chain

$$\begin{aligned} \|u+v\|_2^2 &= \|u\|_2^2 + 2(u \cdot v) + \|v\|_2^2 \\ &\le \|u\|_2^2 + 2|u \cdot v| + \|v\|_2^2 \\ &\le \|u\|_2^2 + 2\,\|u\|_2\,\|v\|_2 + \|v\|_2^2 = (\|u\|_2 + \|v\|_2)^2. \end{aligned}$$

If either inequality were strict, the first number would be strictly smaller than the last. So the two ends are equal exactly when both inequalities are equalities.

The first inequality is an equality exactly when $u \cdot v = |u \cdot v|$, that is, when $u \cdot v \ge 0$; if $u \cdot v < 0$, then $|u \cdot v| > 0 > u \cdot v$. The second is an equality exactly when $|u \cdot v| = \|u\|_2\,\|v\|_2$. By the equality case of the Cauchy–Schwarz inequality, this happens exactly when $v = 0$ or u is a multiple of v.

Now combine the two conditions. If $v = 0$, both hold, since then $u \cdot v = 0$. If $v \neq 0$ and $u = sv$, then $u \cdot v = s(v \cdot v)$, and since $v \cdot v > 0$ this is nonnegative exactly when $s \geq 0$. So equality holds exactly when $v = 0$ or $u = sv$ with $s \geq 0$. In the plane, this means that the two arrows point the same way, or that one of them is the zero vector.

For a strict example, take $v = (1, 1)$ and $u = -v = (-1, -1)$. Then $u + v = 0$, so $\|u + v\|_2 = 0$, while $\|u\|_2 + \|v\|_2 = \sqrt{2} + \sqrt{2} = 2\sqrt{2}$. Here u is a multiple of v, but with the negative factor $s = -1$.

Solution to Exercise 10.6

Assume that $a_1u_1 + \cdots + a_ku_k = 0$, and pick any index j between 1 and k. Take the dot product of both sides with u_j. By linearity in the first argument, applied one term at a time as in Section 10.2, the left side becomes

$$a_1(u_1 \cdot u_j) + \cdots + a_k(u_k \cdot u_j),$$

and the right side becomes $0 \cdot u_j = 0$, the dot product of the zero vector with u_j. For every $i \neq j$, orthogonality gives $u_i \cdot u_j = 0$, so all of those terms disappear. Only the term with $i = j$ remains:

$$a_j(u_j \cdot u_j) = a_j \left\|u_j\right\|_2^2 = 0.$$

The vector u_j is nonzero, so its squared length is positive. Dividing by it gives $a_j = 0$. Because j was an arbitrary index, every coefficient is zero, which is exactly the definition of linear independence in Section 10.1.

The word "nonzero" cannot be left out. Take $u_1 = (1, 0)$ and $u_2 = (0, 0)$. They are orthogonal, since $u_1 \cdot u_2 = 0$. Yet $0u_1 + 1u_2 = (0, 0)$ is a combination equal to zero whose second coefficient is 1, so the list is not linearly independent. In the proof above, this is exactly where the division by $\|u_2\|_2^2 = 0$ would be impossible.

B.5 Week 11

Solution to Exercise 11.1

Name the input errors $\Delta x = x - a$ and $\Delta y = y - b$; the symbol Δ (the Greek capital letter delta) is simply a label for a difference. Subtracting and regrouping gives

$$(2x - y) - (2a - b) = 2(x - a) - (y - b) = 2\Delta x - \Delta y.$$

Now use the triangle inequality, then the product rule $|cx| = |c|\,|x|$ from Section 2.6:

$$\begin{aligned} |2\Delta x - \Delta y| &\leq |2\Delta x| + |-\Delta y| \\ &= 2|\Delta x| + |\Delta y| \\ &\leq 2(0.02) + 0.03 = 0.07. \end{aligned}$$

So $|(2x - y) - (2a - b)| \leq 0.07$ for all numbers satisfying the hypotheses.

The bound is attained. Take $a = b = 0$, $x = 0.02$ and $y = -0.03$. Both hypotheses hold, and $(2x - y) - (2a - b) = 0.04 + 0.03 = 0.07$. Because the second error is subtracted, the worst case

comes from errors of opposite signs. Errors of the same sign partly cancel: $x = 0.02$ and $y = 0.03$, with $a = b = 0$, give only $0.04 - 0.03 = 0.01$. Since 0.07 is attained, no smaller bound holds in every case.

Solution to Exercise 11.2

Let $\varepsilon > 0$ be given. By the Archimedean property, choose an integer $N > 3/\varepsilon$; it is positive because $3/\varepsilon > 0$. Let $n \geq N$. Then $n + 1 > N > 0$, so taking reciprocals gives $1/(n+1) < 1/N$, and multiplying by 3 gives $3/(n+1) < 3/N$. Multiplying $N > 3/\varepsilon$ by the positive number ε/N gives $\varepsilon > 3/N$. Together,

$$|a_n - 0| = \frac{3}{n+1} < \frac{3}{N} < \varepsilon \qquad \text{for every } n \geq N.$$

So the threshold is any integer $N > 3/\varepsilon$, and $a_n \to 0$. For example, $\varepsilon = 0.01$ calls for $N > 300$, so $N = 301$ works; indeed $a_{301} = 3/302 \approx 0.00993$. (The smallest threshold for this tolerance is $N = 300$, since $a_{299} = 3/300 = 0.01$ is not less than 0.01; any threshold that works is acceptable.)

Solution to Exercise 11.3

Recall that $d_\infty(x, y) = \max_{1 \leq j \leq n} |x_j - y_j|$ is the largest of the n numbers $|x_1 - y_1|, \ldots, |x_n - y_n|$.

Nonnegativity. Each of these numbers is an absolute value and so is at least 0. Their largest is one of them, so $d_\infty(x, y) \geq 0$.

Zero exactly for equal vectors. If $x = y$, every coordinate difference is 0, so all n numbers are 0 and their largest is 0. Conversely, suppose $d_\infty(x, y) = 0$. Every number $|x_j - y_j|$ is at most the largest one, which is 0, and at least 0, so $|x_j - y_j| = 0$ and hence $x_j = y_j$ for every j. Two vectors with the same coordinates are equal, so $x = y$.

Symmetry. For each j, $|x_j - y_j| = |y_j - x_j|$, because $y_j - x_j = -(x_j - y_j)$ and changing the sign does not change the absolute value. So $d_\infty(x, y)$ and $d_\infty(y, x)$ are the largest of the same list of numbers, and they are equal.

Together with the triangle inequality proved in Section 11.2, these facts show that d_∞ satisfies all three requirements of Definition 11.1, so it is a metric on $\mathbb{R}^n$.

Solution to Exercise 11.4

The hypothesis of Proposition 11.9 holds with $D = 2$ and $q = 1/3$, so for all $m > n$

$$d(x_m, x_n) \leq \frac{2(1/3)^n}{1 - 1/3} = 3\Big(\frac{1}{3}\Big)^n = 3^{1-n},$$

a bound that does not depend on m.

The proposition also tells us that the sequence is Cauchy. To find an explicit threshold, we need to know when 3^{1-n} becomes small. With $q = 1/3$ we have $c = 1/q - 1 = 2$, and Bernoulli's inequality gives $3^n \geq 1 + 2n$, so $3^{1-n} \leq 3/(1+2n)$. Let $\varepsilon > 0$, and choose an integer $N > 3/(2\varepsilon)$.

For $n \geq N$ we get $1+2n > 2n \geq 2N > 3/\varepsilon$, hence $3^{1-n} \leq 3/(1+2n) < \varepsilon$. Now take any $m, n \geq N$. If $m = n$, their distance is $0 < \varepsilon$. Otherwise name them so that $m > n$, which is allowed because d is symmetric; then $d(x_m, x_n) \leq 3^{1-n} < \varepsilon$. So the sequence is Cauchy, with threshold any integer $N > 3/(2\varepsilon)$.

This shows only that the sequence is Cauchy. To conclude that it converges to a point of the space, we would also need the space to be complete.

Solution to Exercise 11.5

Take $x_n = 2 + 2^{-(n+1)}$ for $n \in \mathbb{N}$, that is, $2.5,\ 2.25,\ 2.125, \ldots$. Since $0 < 2^{-(n+1)} \leq 1/2$, every term satisfies $2 < x_n \leq 2.5$, so the sequence lies in $(2, 3)$.

The sequence is Cauchy. On the real line it converges to 2. Indeed, $|x_n - 2| = 2^{-(n+1)}$, and $2^{n+1} \geq n + 2$ (Section 11.3), so $|x_n - 2| \leq 1/(n+2) < 1/(n+1)$. Given $\varepsilon > 0$, choose an integer $N > 1/\varepsilon$; then every $n \geq N$ satisfies $|x_n - 2| < 1/(n+1) < 1/N < \varepsilon$. By Proposition 11.5, the sequence is Cauchy on the real line. The Cauchy condition involves only distances between terms, and these are the same in $(2, 3)$ as in $\mathbb{R}$, so the sequence is Cauchy in $(2, 3)$.

The sequence has no limit in $(2, 3)$. Suppose it converged to a point L of $(2, 3)$. Distances in $(2, 3)$ are distances in $\mathbb{R}$, so it would also converge to L on the real line. Since it converges to 2 there, Proposition 11.3 forces $L = 2$. But 2 is not in $(2, 3)$, a contradiction. So this is a Cauchy sequence in $(2, 3)$ with no limit in $(2, 3)$, and in particular $(2, 3)$ is not complete.

Solution to Exercise 11.6

1. The supremum. Every $x \in (0, 1)$ satisfies $x < 1$, so 1 is an upper bound for $(0, 1)$. Now let b be any upper bound; we show $b \geq 1$. Suppose instead that $b < 1$. Since $1/2 \in (0, 1)$, we have $1/2 \leq b$, so $b > 0$. Consider $t = (b + 1)/2$, the midpoint of b and 1. Because $b < 1$, we have $b < t < 1$, and $t > b > 0$, so $t \in (0, 1)$. But $t > b$, so b is not an upper bound, a contradiction. Hence every upper bound b satisfies $b \geq 1$, and 1 is a supremum of $(0, 1)$.

2. No maximum. A maximum would be a member x of $(0, 1)$ that is also an upper bound. Take any $x \in (0, 1)$ and put $t = (x + 1)/2$. Because $0 < x < 1$, we have $x < t < 1$, so t is a member of $(0, 1)$ that is larger than x. Thus x is not an upper bound, and no member of $(0, 1)$ is a maximum.

3. Supremum versus maximum. The interval $(0, 1)$ is nonempty, and 1 is an upper bound for it. By part 2 it has no maximum. So the statement "every nonempty set of real numbers that has an upper bound has a maximum" is false, with $(0, 1)$ as a counterexample. The difference is that a maximum must belong to the set, while a supremum need not. The supremum 1 of $(0, 1)$ is exactly the point missing from the interval, just as the limit 0 was missing from $(0, 1)$ in Section 11.6.

B.6 Week 12

Solution to Exercise 12.1

Contraction factor. For all real x and y,

$$T(x) - T(y) = \frac{3x+1}{4} - \frac{3y+1}{4} = \frac{3(x-y)}{4}, \qquad \text{so} \qquad |T(x) - T(y)| = \frac{3}{4}|x-y|.$$

Hence T is a contraction of $\mathbb{R}$ with factor $3/4$. No smaller factor works: if $q < 3/4$, the pair $x = 1$, $y = 0$ gives $|T(1) - T(0)| = 3/4 > q = q\,|1-0|$. So the smallest contraction factor is $3/4$.

Fixed point. The equation $(3x+1)/4 = x$ becomes $3x+1 = 4x$, so $x = 1$. Indeed $T(1) = 4/4 = 1$.

Iterates. From $x_0 = 0$ we get $x_1 = 1/4$, $x_2 = 7/16$ and $x_3 = 37/64$. The gaps to the fixed point are $1 - x_n = 1,\ 3/4,\ 9/16,\ 27/64$, each three quarters of the one before, which suggests $x_n = 1 - (3/4)^n$. We prove this by induction on n. For $n = 0$, the formula gives $1 - 1 = 0 = x_0$. Suppose $x_n = 1 - (3/4)^n$ for some n. Then

$$x_{n+1} = \frac{3x_n + 1}{4} = \frac{3 - 3(3/4)^n + 1}{4} = \frac{4 - 3(3/4)^n}{4} = 1 - \frac{3}{4}\left(\frac{3}{4}\right)^n = 1 - \left(\frac{3}{4}\right)^{n+1},$$

which is the formula for $n+1$. So $x_n = 1 - (3/4)^n$ for every $n \in \mathbb{N}$.

For comparison, the a priori estimate (12.1) with $q = 3/4$ and $D = 1/4$ gives $|x_n - 1| \le (3/4)^n \cdot 4 \cdot \frac{1}{4} = (3/4)^n$. By the formula, this is exactly the true error, so here the a priori estimate is attained for every n.

Solution to Exercise 12.2

By the residual estimate (12.2),

$$d(z, x_*) \le \frac{d(z, Tz)}{1-q} \le \frac{0.002}{1 - 3/4} = 4 \cdot 0.002 = 0.008.$$

So the theorem guarantees $d(z, x_*) \le 0.008$.

The residual bound 0.002 is not itself an upper bound on the error in every case. For example, $T(x) = 3x/4$ is a contraction of the nonempty complete space $\mathbb{R}$ with factor $3/4$, and its fixed point is 0. The point $z = 0.008$ has $Tz = 0.006$, so its residual is exactly 0.002, while its distance to the fixed point is 0.008, four times larger. This example also shows that the bound 0.008 cannot be improved using only the information given.

Solution to Exercise 12.3

Self-map. Let $1/3 \le x \le 2/3$. Squaring nonnegative numbers preserves their order, so $1/9 \le x^2 \le 4/9$. Adding 1 gives $10/9 \le x^2 + 1 \le 13/9$, and dividing by 3 gives $10/27 \le T(x) \le 13/27$. Since $1/3 = 9/27 \le 10/27$ and $13/27 \le 18/27 = 2/3$, the output $T(x)$ lies in $[1/3, 2/3]$.

Contraction factor. For $x, y \in [1/3, 2/3]$ we have $0 \le x + y \le 4/3$, so

$$|T(x) - T(y)| = \frac{|x-y|\,|x+y|}{3} \le \frac{4/3}{3}\,|x-y| = \frac{4}{9}|x-y|.$$

So $4/9$ is a contraction factor on this interval, smaller than the factor $2/3$ used on $[0, 1]$.

Where the iterates lie. The first iterate is $x_1 = T(0) = 1/3$, which lies in $[1/3, 2/3]$. If some x_n lies in $[1/3, 2/3]$, then so does $x_{n+1} = T(x_n)$, by the self-map property just proved. By induction, every iterate from x_1 on lies in $[1/3, 2/3]$. The starting point $x_0 = 0$ does not, which is why the smaller factor becomes available only after the first step.

The improved certificate. The interval $[1/3, 2/3]$ is nonempty, and it is complete, because Section 11.6 showed that every closed interval $[a, b]$ is complete. So Theorem 12.2 applies to T on $[1/3, 2/3]$ with $q = 4/9$. It gives a fixed point in $[1/3, 2/3]$. Since $[1/3, 2/3]$ is part of $[0, 1]$ and T has only one fixed point in $[0, 1]$, this is the same point x_* as in Section 12.5. The point $z = x_2 = 10/27$ lies in $[1/3, 2/3]$, and its residual is $19/2187$, as computed in Section 12.5. Since $1/(1 - 4/9) = 9/5$, the residual estimate gives

$$d(x_2, x_*) \le \frac{9}{5} \cdot \frac{19}{2187} = \frac{171}{10935} = \frac{19}{1215} = 0.01563\ldots,$$

which improves the earlier certificate $19/729 = 0.02606\ldots$. (The true error, found in Section 12.5, is about 0.0116.)

The improvement is justified because the residual estimate was applied in the space $[1/3, 2/3]$, where everything it needs holds: T maps $[1/3, 2/3]$ into itself, the factor $4/9$ works for all pairs of points of that interval, and both x_2 and x_* lie in it. Using the factor $4/9$ for a point outside the interval, such as $x_0 = 0$, would not be justified.

Solution to Exercise 12.4

The map $x \mapsto x/2$ on $(0, 1)$: completeness fails. The space is nonempty, since it contains $1/2$. It is a self-map, because $0 < x < 1$ gives $0 < x/2 < 1/2 < 1$. The contraction inequality holds with factor $1/2$, because $|x/2 - y/2| = |x - y|/2$. Completeness fails: as Section 11.6 showed, the sequence $2^{-(n+1)}$ is a Cauchy sequence in $(0, 1)$ whose only possible limit, 0, does not belong to $(0, 1)$.

The maps $x \mapsto x$ and $x \mapsto x + 1$ on $\mathbb{R}$: no factor below one. The space $\mathbb{R}$ is nonempty and complete (Section 11.6). Both maps send real numbers to real numbers, so both are self-maps of $\mathbb{R}$. For both, $|T(x) - T(y)| = |x - y|$ for all x and y. So the inequality holds with $q = 1$, but with no $q < 1$: for $x = 1$ and $y = 0$ we would need $1 \le q$. The failed hypothesis is the requirement $q < 1$.

The map $x \mapsto x + 1/x$ on $[1, \infty)$: no factor below one. The space is nonempty, since it contains 1. It is a self-map, because $x \ge 1$ gives $x + 1/x > x \ge 1$. Section 12.6 showed that every pair of different points moves strictly closer, so the inequality holds with $q = 1$, but that no single factor $q < 1$ works. It remains to prove that $[1, \infty)$ is complete. We show that it is closed in $\mathbb{R}$, exactly as Section 11.6 did for $[0, 1]$. Suppose that numbers $x_n \ge 1$ converge in $\mathbb{R}$ to a number L, and suppose $L < 1$. The tolerance $(1 - L)/2$ is positive, so for a late enough index $|x_n - L| < (1 - L)/2$, and then

$$x_n < L + \frac{1 - L}{2} = \frac{1 + L}{2} < 1,$$

which contradicts $x_n \ge 1$. So $L \ge 1$, that is, $L \in [1, \infty)$. Thus $[1, \infty)$ is closed in the complete space $\mathbb{R}$, and by Proposition 11.8 it is complete.

The map $x \mapsto x/2 + 1$ on $[0, 1]$: the self-map condition fails. The interval $[0, 1]$ is nonempty and complete (Section 11.6). For all $x, y \in [0, 1]$, $|T(x) - T(y)| = |x - y|/2$, so the contraction inequality holds with factor $1/2$. But $T(1) = 3/2$ does not lie in $[0, 1]$, so T is not a map from $[0, 1]$ to itself.

Solution to Exercise 12.5

Fixed point. The equation $x = ax + b$ becomes $(1 - a)x = b$. Since $|a| < 1$, we have $a \neq 1$, so $1 - a \neq 0$ and we may divide: $x_* = b/(1 - a)$. (Also, $|T(x) - T(y)| = |a|\,|x - y|$, so T is a contraction of $\mathbb{R}$ with factor $|a|$, and Theorem 12.2 confirms that this is the only fixed point.)

The error formula. Subtract the equation $x_* = ax_* + b$ from the recurrence $x_{n+1} = ax_n + b$. The constants b cancel, leaving

$$x_{n+1} - x_* = a(x_n - x_*).$$

Now induct on n. For $n = 0$ the formula reads $x_0 - x_* = a^0(x_0 - x_*)$, which is true because $a^0 = 1$ (when $a = 0$, read a^0 as 1, as we agreed after Theorem 12.2). If $x_n - x_* = a^n(x_0 - x_*)$ for some n, then

$$x_{n+1} - x_* = a(x_n - x_*) = a \cdot a^n(x_0 - x_*) = a^{n+1}(x_0 - x_*),$$

which is the formula for $n + 1$.

How the iterates move. In every case $|x_n - x_*| = |a|^n\,|x_0 - x_*|$, so the distance to the fixed point shrinks by the factor $|a|$ at each step. If $x_0 = x_*$, every iterate equals x_*, and if $a = 0$, every iterate from x_1 on equals x_*. Otherwise the sign of a^n decides on which side of x_* the iterate x_n lies. If $0 < a < 1$, every power a^n is positive, so all iterates lie on the same side of x_* as x_0 and approach it from that side. If $a < 0$, then a^n is positive for even n and negative for odd n, so the iterates jump from one side of x_* to the other at every step, while the jumps get shorter. For example, $a = -1/2$ and $b = 3/2$ give the fixed point 1, and the iterates from $x_0 = 0$ are $0,\ 3/2,\ 3/4,\ 9/8,\ 15/16, \ldots$, alternately below and above 1.

Solution to Exercise 12.6

By Theorem 12.2, applied to each map separately, T and S have fixed points x_T and x_S, so $T(x_T) = x_T$ and $S(x_S) = x_S$.

We must compare points that come from two different maps, while the contraction inequality compares two outputs of the *same* map. So insert the intermediate point $T(x_S)$, which separates the change of input from the change of map:

$$\begin{aligned} d(x_T, x_S) &= d(T(x_T), S(x_S)) \\ &\leq d(T(x_T), T(x_S)) + d(T(x_S), S(x_S)) \\ &\leq q\, d(x_T, x_S) + \eta. \end{aligned}$$

The first term is bounded by the contraction inequality for T. The second is bounded by the hypothesis $d(Tx, Sx) \leq \eta$ with $x = x_S$. Write $r = d(x_T, x_S)$. Then $r \leq qr + \eta$. Subtracting

qr gives $(1-q)r \le \eta$, and dividing by the positive number $1-q$ gives $r \le \eta/(1-q)$. Thus $d(x_T, x_S) \le \eta/(1-q)$.

The residual estimate gives the same result in one line. Apply (12.2) to the map T and the point $z = x_S$. Since $x_S = S(x_S)$, the residual of x_S for T is $d(x_S, T(x_S)) = d(S(x_S), T(x_S)) \le \eta$, so $d(x_S, x_T) \le \eta/(1-q)$. In words, the fixed point of S is an approximate fixed point of T.

Notice that the contraction property of S is used only to guarantee that x_S exists. The result says that if two contractions with the same factor differ by at most η at every point, then their fixed points differ by at most $\eta/(1-q)$: a small change in the map causes only a small change in the fixed point, as long as q is not close to 1.

APPENDIX C

Solutions for Chapters 13–18

This appendix contains the solutions for Chapters 13–18: finite probability, sign sequences, the two chapters on Lean, reading an unfamiliar proof, and the capstone investigation. Several solutions for Weeks 15 and 16 are written in Lean; Section D.4 explains how to have Lean check them on your own computer. The independent capstone of Section 18.7 is your own project, so it has no model solution here; Section E.6 suggests how it can be assessed.

C.1 Week 13

Solution to Exercise 13.1

First check that the probabilities add up to 1: $\frac{1}{2} + \frac{1}{3} + \frac{1}{6} = \frac{3}{6} + \frac{2}{6} + \frac{1}{6} = 1$. Using the formula $\mathbb{E}\,X = \sum_i x_i\,\mathbb{P}(X = x_i)$ from Section 13.2,

$$\mathbb{E}\,X = 0 \cdot \tfrac{1}{2} + 2 \cdot \tfrac{1}{3} + 5 \cdot \tfrac{1}{6} = \tfrac{4}{6} + \tfrac{5}{6} = \tfrac{9}{6} = \tfrac{3}{2}.$$

Both of the values 2 and 5 are at least $3/2$. Proposition 13.5 guarantees in advance that the variable takes some value at least $3/2$ with positive probability. The expectation $3/2$ is not itself a value of the variable: it is a weighted average of the values, not a possible result.

Solution to Exercise 13.2

A labeling gives each of the three vertices one of two labels, so there are $2^3 = 8$ labelings. Write a labeling by listing the labels of the three vertices in a fixed order. In two of the labelings, 000 and 111, all three vertices get the same label, so all three are on the same side and the cut has size 0. In each of the other six, two vertices share a label and the third vertex has the other label. The two edges at the third vertex cross, and the edge joining the two vertices with the same label does not, so the cut has size 2. The possible cut sizes are therefore 0 and 2; sizes 1 and 3 never occur. The average over all eight labelings is

$$\frac{2 \cdot 0 + 6 \cdot 2}{8} = \frac{12}{8} = \frac{3}{2},$$

which is half of the three edges, as in the proof of Theorem 13.8. The average $3/2$ is not a possible cut size, but the labelings with cut size 2 do at least as well as the average.

Solution to Exercise 13.3

There are $m = 4$ edges, so at the start $C = 0$, $U = 4$, and $F = 2$.

- Vertex 1 has no placed neighbors, so $\ell = r = 0$. The tie is broken by placing 1 on the left. No edge is settled yet, so $C = 0$, $U = 4$, and $F = 2$.
- Vertex 2 has neighbors 1 (on the left) and 3 (not yet placed), so $\ell = 1$, $r = 0$. Place 2 on the right. Edge 12 is settled and crosses, so $C = 1$, $U = 3$, and $F = 1 + 3/2 = 5/2$.
- Vertex 3 has neighbors 2 (on the right) and 4 (not yet placed), so $\ell = 0$, $r = 1$. Place 3 on the left. Edge 23 crosses, so $C = 2$, $U = 2$, and $F = 3$.
- Vertex 4 has neighbors 3 and 1, both on the left, so $\ell = 2$, $r = 0$. Place 4 on the right. Edges 34 and 41 cross, so $C = 4$, $U = 0$, and $F = 4$.

Each increase of F equals $|\ell - r|/2$, namely 0, 1/2, 1/2, and 1. The final cut is $L = \{1, 3\}$, $R = \{2, 4\}$, and all four edges cross, so its size is 4. Theorem 13.8 guarantees only $m/2 = 2$. Here the construction does better than its guarantee. In fact it finds a largest possible cut, because no cut can have more than $m = 4$ crossing edges. The construction does not always do this well, as the path example in Section 13.5 shows.

Solution to Exercise 13.4

Let A_1, A_2, A_3, A_4 be the bad events. Their union is the event that at least one bad event occurs. The union bound (Proposition 13.6) gives

$$\mathbb{P}(A_1 \cup A_2 \cup A_3 \cup A_4) \le \sum_{j=1}^{4} \mathbb{P}(A_j) \le 4 \cdot 0.2 = 0.8.$$

So the complement of the union, which is the event that all four bad events are avoided, has probability at least $1 - 0.8 = 0.2$. This is positive, so the complement contains an outcome of positive probability (Section 13.1). That outcome lies outside the union, so it belongs to none of the four bad events. The argument proves that such an outcome exists; it does not claim that every outcome avoids the bad events.

With the bound 0.25, the same calculation gives only $\mathbb{P}(A_1 \cup \cdots \cup A_4) \le 4 \cdot 0.25 = 1$, and every event has probability at most 1. So the argument proves nothing, and in fact the conclusion can fail. Take a uniform space with four outcomes $\omega_1, \omega_2, \omega_3, \omega_4$, and let $A_j = \{\omega_j\}$ for each j. Each bad event has probability $1/4 = 0.25$, but their union is the whole sample space, so no outcome avoids them all. The conclusion can also hold: in the same space, if $A_1 = A_2 = A_3 = A_4 = \{\omega_1\}$, then each bad event again has probability $1/4$, and ω_2 avoids them all. So with the bound 0.25, the information given does not decide the question.

Solution to Exercise 13.5

Toss one fair coin, and let X be 0 if it shows 0 and 10 if it shows 1. Then $\mathbb{E}\,X = \frac{1}{2} \cdot 0 + \frac{1}{2} \cdot 10 = 5$, so the expectation is at least 5, but X takes the value 0 at an outcome of probability $1/2$.

What the expectation does guarantee is stated in Proposition 13.5: in a finite probability space, $X(\omega) \ge \mathbb{E}\,X$ for at least one outcome ω of positive probability, and $X(\omega) \le \mathbb{E}\,X$ for at least one

outcome ω of positive probability. Here the outcome where the coin shows 1, with $X = 10 \geq 5$, is the first kind, and the outcome where it shows 0 is the second.

Solution to Exercise 13.6

Markov's inequality (Proposition 13.10) with $a = 12$ gives

$$\mathbb{P}(X \geq 12) \leq \frac{\mathbb{E}\,X}{12} = \frac{3}{12} = \frac{1}{4}.$$

For an example where equality holds, take a uniform space with four outcomes $\omega_1, \omega_2, \omega_3, \omega_4$, and let $X(\omega_1) = 12$ and $X(\omega_2) = X(\omega_3) = X(\omega_4) = 0$. Then X is nonnegative, $\mathbb{E}\,X = 12/4 = 3$, and $\mathbb{P}(X \geq 12) = \mathbb{P}(\{\omega_1\}) = 1/4$, exactly the bound. So Markov's inequality cannot be improved in general.

The example can be found by asking when the two estimates in the proof of Markov's inequality are equalities. The first sum equals $a\,\mathbb{P}(X \geq a)$ only if X is exactly a at every outcome of positive probability where $X \geq a$, and the second sum equals 0 only if X is 0 at every outcome of positive probability where $X < a$. So, at outcomes of positive probability, X should take only the values 0 and 12, and then $\mathbb{E}\,X = 12\ \mathbb{P}(X = 12) = 3$ forces $\mathbb{P}(X = 12) = 1/4$.

C.2 Week 14

Solution to Exercise 14.1

Write $a = (a_0, a_1, a_2) = (1, 1, -1)$, so $n = 3$. The aperiodic correlations are

$$A_1 = a_0a_1 + a_1a_2 = 1 \cdot 1 + 1 \cdot (-1) = 0, \qquad A_2 = a_0a_2 = 1 \cdot (-1) = -1.$$

For the periodic correlations, the positions wrap around modulo 3. At shift one the last term is $a_2a_{3 \bmod 3} = a_2a_0$, and at shift two the last two terms are $a_1a_{3 \bmod 3} = a_1a_0$ and $a_2a_{4 \bmod 3} = a_2a_1$. So

$$C_1 = a_0a_1 + a_1a_2 + a_2a_0 = 1 \cdot 1 + 1 \cdot (-1) + (-1) \cdot 1 = -1,$$
$$C_2 = a_0a_2 + a_1a_0 + a_2a_1 = 1 \cdot (-1) + 1 \cdot 1 + (-1) \cdot 1 = -1.$$

Equation (14.1) predicts $C_1 = A_1 + A_2 = 0 + (-1) = -1$ and $C_2 = A_2 + A_1 = -1$, which agrees. Since $|A_1| = 0$ and $|A_2| = 1$ are both at most one, $(1, 1, -1)$ is a Barker sequence. Its periodic correlations at nonzero shifts are not zero, as expected: for an odd length, no periodic correlation can be zero (see the pause at the end of Section 14.2).

Solution to Exercise 14.2

Every term of a correlation is a product of two entries of the sequence, and $(-x)(-y) = xy$. For $1 \leq s \leq n - 1$,

$$A_s(-a) = \sum_{j=0}^{n-1-s} (-a_j)(-a_{j+s}) = \sum_{j=0}^{n-1-s} a_ja_{j+s} = A_s(a).$$

For $0 \le s \le n-1$, in the same way,

$$C_s(-a) = \sum_{j=0}^{n-1} (-a_j)(-a_{(j+s) \bmod n}) = \sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n} = C_s(a).$$

Consequently, a is a Barker sequence exactly when $-a$ is, and a has $C_s(a) = 0$ for all s with $1 \le s \le n-1$ exactly when $-a$ has this property. Since $-a$ is never equal to a (their first entries differ), the sequences counted in the table of Section 14.6 come in pairs $\{a, -a\}$, and this is why every count there is even.

Solution to Exercise 14.3

The rows of H are

$$\text{row } 0 = (1,1,1,-1), \quad \text{row } 1 = (-1,1,1,1), \quad \text{row } 2 = (1,-1,1,1), \quad \text{row } 3 = (1,1,-1,1).$$

The entry in position (i, ℓ) of HH^{T} is the dot product of row i with row ℓ. So

$$\begin{aligned}
(HH^{\mathsf{T}})_{02} &= 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot 1 + (-1) \cdot 1 = 1 - 1 + 1 - 1 = 0, \\
(HH^{\mathsf{T}})_{13} &= (-1) \cdot 1 + 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot 1 = -1 + 1 - 1 + 1 = 0, \\
(HH^{\mathsf{T}})_{22} &= 1 \cdot 1 + (-1) \cdot (-1) + 1 \cdot 1 + 1 \cdot 1 = 4.
\end{aligned}$$

These agree with $HH^{\mathsf{T}} = 4I$, which has 0 off the diagonal and 4 on it. By Proposition 14.5(a), the entry in position $(0,2)$ equals $C_{(0-2) \bmod 4} = C_2$, and the entry in position $(1,3)$ equals $C_{(1-3) \bmod 4} = C_2$ as well. Both agree with the value $C_2 = 0$ computed in Section 14.2.

Solution to Exercise 14.4

The definition of C_s requires $0 \le s \le n-1$, and the definition of C_{n-s} requires $0 \le n-s \le n-1$, that is, $1 \le s \le n$. Both sides are therefore defined exactly when $1 \le s \le n-1$. When $n = 1$ there is no such s, and there is nothing to prove.

Let $1 \le s \le n-1$. By definition,

$$C_{n-s} = \sum_{j=0}^{n-1} a_j a_{(j+n-s) \bmod n}.$$

The integers $j + n - s$ and $j - s$ differ by n, so they have the same remainder on division by n, and $(j+n-s) \bmod n = (j-s) \bmod n$. Hence

$$C_{n-s} = \sum_{j=0}^{n-1} a_j a_{(j-s) \bmod n}.$$

Now set $k = (j-s) \bmod n$. As j runs through $0, 1, \ldots, n-1$, so does k, each value exactly once, because $j \mapsto (j-s) \bmod n$ is a bijection of $\{0, 1, \ldots, n-1\}$ (Section 14.3). Moreover $j \equiv k + s$

$(\text{mod } n)$, and j lies between 0 and $n-1$, so $j = (k+s) \bmod n$. Replacing both occurrences of the old index,

$$\begin{aligned} C_{n-s} &= \sum_{k=0}^{n-1} a_{(k+s) \bmod n}\, a_k \\ &= \sum_{k=0}^{n-1} a_k\, a_{(k+s) \bmod n} \\ &= C_s. \end{aligned}$$

The first equality lists the same terms in a different order, the second swaps the two factors in each product, and the last is the definition of C_s.

Equation (14.1) gives a second proof. It says $C_s = A_s + A_{n-s}$, and applied to the shift $n-s$ it says $C_{n-s} = A_{n-s} + A_{n-(n-s)} = A_{n-s} + A_s$. The two right sides are equal. For example, the sequence $(1, 1, -1)$ of Exercise 14.1 has $C_1 = C_2 = -1$.

Solution to Exercise 14.5

By Section 14.4, the order must be a perfect square. By Lemma 14.6, an order greater than 2 must be divisible by four.

- 8 is not a perfect square: for an integer r, if $|r| \le 2$ then $r^2 = |r|^2 \le 4$, and if $|r| \ge 3$ then $r^2 = |r|^2 \ge 9$. So 8 is ruled out.
- $9 = 3^2$ and $25 = 5^2$ are perfect squares, but neither is divisible by four. Both are ruled out. (Both are also odd, so the parity argument of Section 14.2 rules them out as well.)
- $16 = 4^2 = 4 \cdot 2^2$ and $36 = 6^2 = 4 \cdot 3^2$ satisfy both conditions. They remain possible orders, of the form $4u^2$ with $u = 2$ and $u = 3$.

The answer does not show that a circulant Hadamard matrix of order 16 or 36 exists. Necessary conditions can only rule orders out; passing both tests means only that these two tests cannot exclude 16 and 36. In fact, both orders were ruled out long ago by the stronger restrictions mentioned in Section 14.7. You can even settle order 16 yourself: extending the search of Section 14.6 to length 16 means examining $2^{16} = 65536$ sequences, and none of them has $C_s = 0$ for every shift s with $1 \le s \le 15$. As Exercise 14.6 explains, a complete search of this kind rules out that one order and no other.

Notice also that only the divisibility condition applies to all Hadamard matrices; the square condition needs the circulant pattern. For example, Hadamard matrices of order 8 do exist. Write h_0, h_1, h_2, h_3 for the rows of the circulant Hadamard matrix H of order four from Section 14.3, and form the 8-by-8 matrix with the rows

$$(h_0, h_0),\ (h_1, h_1),\ (h_2, h_2),\ (h_3, h_3),\ (h_0, -h_0),\ (h_1, -h_1),\ (h_2, -h_2),\ (h_3, -h_3).$$

Here (h_i, h_i) means row h_i followed by a second copy of it, and $(h_i, -h_i)$ means row h_i followed by its negative. All the entries are ± 1. The dot product of two of these rows is the dot product

of their first halves plus the dot product of their second halves. Since different rows of H are orthogonal, this gives, for $i \neq \ell$,

$$(h_i, h_i) \cdot (h_\ell, h_\ell) = 2\, h_i \cdot h_\ell = 0 \qquad \text{and} \qquad (h_i, -h_i) \cdot (h_\ell, -h_\ell) = 2\, h_i \cdot h_\ell = 0,$$

and for all i and ℓ, including $i = \ell$,

$$(h_i, h_i) \cdot (h_\ell, -h_\ell) = h_i \cdot h_\ell - h_i \cdot h_\ell = 0.$$

Thus any two different rows are orthogonal, and by Section 14.3 the matrix is a Hadamard matrix of order 8. No Hadamard matrix of order 8 is circulant, because 8 is not a perfect square.

Solution to Exercise 14.6

Existence. Suppose the search finds a sequence a of length n with $C_s(a) = 0$ for every s with $1 \leq s \leq n - 1$. By Proposition 14.5(b), the circulant matrix of a is a circulant Hadamard matrix of order n. This conclusion does not depend on trusting the search: anyone can recompute the $n - 1$ correlations of this one sequence, by hand or with an independent program. The sequence is a *certificate* of existence that can be checked on its own.

Nonexistence at one length. Suppose instead that the search examines every one of the 2^n sequences correctly and finds none that qualifies. The first row of a circulant Hadamard matrix of order n would be one of the sequences examined, so no such matrix exists. This conclusion has no short certificate. To accept it, we must trust that the program examined every case and computed every correlation correctly, or run an independent search ourselves.

Only that length. The search examined sequences of length n and no others, so it says nothing about any other length. The table of Section 14.6, for instance, says nothing about length 16.

All orders greater than 4. There are infinitely many such orders, so no finite search can cover them. What is needed is a proof that works for an arbitrary order n. The proof of Proposition 14.7 is an argument of this kind, since it applies to every n at once, but it is not strong enough: it still leaves the orders $4u^2$ with $u \geq 2$ open. Alternatively, one could prove a reduction showing that any example, if one existed, would have to appear among finitely many cases, and then search all of those cases.

C.3 Week 15

The Lean solutions for Weeks 15 and 16 use the definitions from Chapters 15 and 16, such as `Inj`, `preimage`, and `Second`. To run them, put them in the same file as those definitions, after the definitions and inside `namespace MathCourse`, as described in Section D.4. Like the examples in the chapters, they are written for Lean 4.19.0. Read each one alongside its ordinary mathematical argument.

Solution to Exercise 15.1

Here `P` and `Q` are propositions, and `P -> Q` is the proposition "if P, then Q." The name `h` denotes a proof of that implication. Such a proof works as a function: it turns any proof of P into a proof

of Q. So when `hp : P`, the application `h hp` is a proof of Q, that is, `h hp : Q`. In words, it carries out the inference "P holds, and P implies Q; therefore Q holds."

Solution to Exercise 15.2

A proof of a conjunction is a pair of proofs, so we take the first component:

```
theorem and_left (P Q : Prop) : And P Q -> P := by
  intro h
  exact h.left
```

After `intro h`, the hypothesis is `h : And P Q` and the goal is `P`. The term `h.left` is the first component of the pair, a proof of P, and `exact` closes the goal with it. That is the only inference: if P and Q both hold, then in particular P holds. The theorem does not claim that P is true on its own; it claims that P is true whenever $P \wedge Q$ is.

Solution to Exercise 15.3

Offer the witness 5 and check its property:

```
theorem witness_five : Exists (fun n : Nat => n + 2 = 7) := by
  exact Exists.intro 5 rfl
```

The statement is the Lean form of $\exists n \in \mathbb{N} : \; n + 2 = 7$. The first input of `Exists.intro` is the witness 5. The second must be a proof of `5 + 2 = 7`, and the proof object `rfl` supplies it, because `5 + 2` reduces to `7`. Since `n` is declared to be a `Nat`, the witness must be a natural number; in fact 5 is the only one that works.

Solution to Exercise 15.4

Applying a function to equal inputs gives equal outputs, but equal outputs need not come from equal inputs. The term `congrArg g h` applies g once more to both sides of `h : g x = g y`. This makes sense only when g can be applied to its own outputs, that is, when its codomain is also its domain, as for `g : Nat -> Nat`. Then the result is a proof of `g (g x) = g (g y)`. In every case `congrArg` moves forward through a function, never backward, so it cannot remove g from both sides. The missing hypothesis is injectivity of g, which says precisely that equal outputs come from equal inputs: with `hg : Inj g`, the term `hg x y h` is a proof of `x = y`. Without that hypothesis the conclusion can be false. The constant function $g : \mathbb{N} \to \mathbb{N}$ with $g(n) = 0$ for every n satisfies $g(1) = g(2)$, yet $1 \neq 2$.

Solution to Exercise 15.5

Given $f(x) = f(y)$, apply g to both sides and then use injectivity of the composition:

```
theorem inj_of_comp {A B C : Type}
    (f : A -> B) (g : B -> C)
    (hcomp : Inj (fun x => g (f x))) : Inj f := by
  intro x y hxy
  exact hcomp x y (congrArg g hxy)
```

After `intro x y hxy`, we have `hxy : f x = f y` and the goal `x = y`. The term `congrArg g hxy` proves `g (f x) = g (f y)`. Lean accepts it as evidence that the composition takes equal values at `x` and `y`, because `(fun x => g (f x)) x` beta-reduces to `g (f x)` (Section 15.5). Then `hcomp x y` turns that evidence into a proof of `x = y`. This is the first part of the proof of Proposition 3.3, written in Lean.

The only hypothesis is about the composition; the proof never uses injectivity of g. That is as it should be, because the second part of Proposition 3.3 shows that g need not be injective. In the example given there, the maps f and $g \circ f$ are injective, but g sends 1 and 2 to the same point; the point 2 lies outside the image of f, so the composition never detects the collision.

Solution to Exercise 15.6

Natural-number subtraction is truncated at zero: when the usual answer would be negative, the result is 0. So `(7 - 10 : Nat)` computes to 0. The integers contain negative numbers, so `(7 - 10 : Int)` computes to -3. The same symbols therefore name two different operations, and the type decides which one is meant. The lesson is that changing the domain of a theorem can change the meaning of its operations, not only the set of values its variables may take. Before translating a statement about integers into one about natural numbers, check every subtraction. For instance, the law $(a - b) + b = a$ holds for all integers, but in `Nat` it fails whenever $a < b$: `(7 - 10) + 10` is 10, not 7.

C.4 Week 16

Solution to Exercise 16.1

The fact `hb : g b = c` says that b is a preimage of the target c under g; it is obtained by unpacking `hg c`. The fact `ha : f a = b` says that a is a preimage of b under f; it is obtained by unpacking `hf b`.

The witness must lie in A because of what the goal asks for. The theorem claims `Surj (fun x => g (f x))`, and the composition `fun x => g (f x)` takes its inputs from `A`, the domain of f. So after `intro c`, the goal asks for some `x` in `A` with $g(f(x)) = c$. The proposed witness a has the right type because it was obtained from `hf b`, which says that some element of `A` is sent to b by f. The element b would not do: it lies in B, and proposing it would be a type error.

Solution to Exercise 16.2

An element lies in the union $S \cup T$ when it lies in S or in T, so union is represented by the disjunction of the two membership predicates:

```
def join {A : Type} (s t : A -> Prop) : A -> Prop :=
  fun x => Or (s x) (t x)

theorem preimage_join {A B : Type} (f : A -> B)
    (s t : B -> Prop) (x : A) :
    Iff (preimage f (join s t) x)
      (join (preimage f s) (preimage f t) x) := by
  rfl
```

As for `preimage_meet`, both sides unfold to the same proposition. The left side `preimage f (join s t) x` becomes `join s t (f x)`, and then `Or (s (f x)) (t (f x))`. The right side `join (preimage f s) (preimage f t) x` becomes `Or (preimage f s x) (preimage f t x)`, and then the same `Or (s (f x)) (t (f x))`. So `rfl` proves the equivalence. In ordinary notation, the theorem is the identity $f^{-1}(S \cup T) = f^{-1}(S) \cup f^{-1}(T)$ of Exercise 3.4, whose proof was the same chain of definitions.

Solution to Exercise 16.3

Write the definition of `Second r v w` as `And P (And Q R)`, where P is $w \neq v$, Q says that there is no edge $v \to w$, and R says that $v \to u \to w$ for some u. The first claim is P, the first component of `h`. The second claim is R, the second component of the second component:

```
theorem second_not_self {V : Type}
    (r : V -> V -> Prop) (v w : V)
    (h : Second r v w) : Not (w = v) := by
  exact h.left

theorem second_route {V : Type}
    (r : V -> V -> Prop) (v w : V)
    (h : Second r v w) :
    Exists (fun u : V => And (r v u) (r u w)) := by
  exact h.right.right
```

The projections follow the grouping in the definition. If `Second` had been defined with the shape `And (And P Q) R` instead, the same two facts would be `h.left.left` and `h.right`. The mathematical meaning would be unchanged, but the projections would differ.

Solution to Exercise 16.4

The original theorem had a list of hypotheses and a conclusion. The new theorem has the same conclusion and one more hypothesis, `h : False`. Since `False` has no proof, nobody can ever supply this hypothesis, so the new theorem can never be applied. It says only that the conclusion

would follow *if* a proof of `False` were available, and that is true of every conclusion whatsoever, by the rule `False.elim`. The original question was whether the conclusion follows from the original hypotheses, and the new theorem says nothing about that. Lean's acceptance does not reliably reveal the change. The new proof needs no `sorry`, and `#print axioms` may report no axioms at all, as it does for `impossible_context`. If the new proof never uses the original hypotheses, Lean may warn that they are unused, as it does for `surj_comp_assumed`. But such a warning is easy to overlook, and there is nothing to warn about when the original theorem had no hypotheses: `impossible_context`, which has exactly that shape, produces no warning at all. Only a comparison of the new statement with the original one, hypothesis by hypothesis, is sure to reveal the change.

Solution to Exercise 16.5

Step 1: compare the statement with the intended one. Examine the theorem's declaration, that is, its parameters and its conclusion, and fill in the two-column table of Section 16.1, including the type of every variable. A changed domain, such as `Nat` where integers were meant, shows up here as a mismatch of types.

Step 2: read the definitions. Look up each definition that the statement uses and read its body, the part after `:=`. Compare its conditions with the mathematical definition. A missing exclusion, such as a version of `Second` without the condition `Not (r v w)`, shows up here and nowhere else, because the name of the definition stays the same.

Step 3: check the proof. Run Lean on the whole file, wait until it has finished, read every warning, and run `#print axioms` on the theorem. An unfinished proof shows up as the warning `declaration uses 'sorry'` at the declaration that contains the gap, and as `sorryAx` in the axiom list of every theorem that depends on it.

All three steps are needed, because each problem can slip past the other two. A complete proof about `Nat` where integers were meant, with correct definitions, passes Steps 2 and 3. A complete proof without any `sorry`, but about a definition with a missing exclusion, passes Steps 1 and 3, since the declaration itself looks right. A correct statement whose proof is unfinished passes Steps 1 and 2.

Solution to Exercise 16.6

The three kinds of check differ in what is checked and in how many cases are covered.

- *A finite test in Python.* At length 5, the program of Section D.3 runs through all $2^5 = 32$ sign sequences and finds exactly 4 Barker sequences (Section 14.6). This covers 32 cases and no others. Its answer also depends on the program being a correct translation of the definitions, and no checker compares the code with the definitions; only a careful reading can do that.
- *A Lean proof about specific objects.* The periodic correlation at shift 1 of the sequence $(1, 1, 1, -1)$ is $a_0a_1 + a_1a_2 + a_2a_3 + a_3a_0 = 0$:

```
theorem corr_shift_one :
    (1 : Int) * 1 + 1 * 1 + 1 * (-1) + (-1) * 1 = 0 := by
  rfl
```

Lean checks this proof completely, but it covers one case: one sequence and one shift. Even here, the encoding must match the definition. The last product, $a_3 a_0 = (-1) \cdot 1$, is the wrap-around term of the periodic correlation.

- *A universal theorem in Lean.* The theorem `preimage_meet` holds for all types A and B, every function f, all predicates `s` and `t`, and every element `x`. One proof covers infinitely many cases. A far more ambitious example is the formalization described in Section 14.7: according to the repository, it contains a Lean proof of a statement about circulant Hadamard matrices of every order.

These scopes stay different even when all three checks finish without an error. A proposition about specific objects does not become universal because it was proved in Lean: `corr_shift_one` says nothing about other sequences or other lengths. And in every case the encoded conditions must match the intended definition, for instance the aperiodic correlation in the Barker test and the periodic one in `corr_shift_one`.

C.5 Week 17

Solution to Exercise 17.1

Take any point c of the space and the constant sequence $x_n = c$. Every ball $B(c, r)$ contains x_n for every index n, so it contains terms at infinitely many indices, yet the sequence takes only one value, c. The ball itself may contain many other points of the space, as $B(c, r)$ does on the real line, but only one of them is a value of the sequence.

The proof does not need different values. Lemma 17.3 counts indices, not values, and Step 2 needs an infinite set of indices only so that it can always choose a larger index. Repeated values do no harm. For a constant sequence, every subsequence is constant, and a constant sequence converges at once to its value, since every term is at distance 0 from it.

Solution to Exercise 17.2

Without the requirement $n_k > n_{k-1}$, the chosen indices need not increase, so the terms $x_{n_1}, x_{n_2}, \ldots$ need not form a subsequence. This really can happen. For the constant sequence $x_n = c$, every ball that contains c contains every term, and every other ball contains none. The chosen ball must give an infinite set S_k, so it contains c, and $S_k = \mathbb{N}$ for every k. The rule "take the smallest member of S_k" then gives $n_k = 0$ for every k, and the list $x_0, x_0, x_0, \ldots$ uses one index again and again. It is not a subsequence. The distance estimates of Steps 3 and 4 still hold for the chosen terms, because they use only $n_k \in S_k$; what fails is the claim that these terms form a subsequence, which is what the theorem promises.

The repair: after choosing n_{k-1}, choose n_k from the set $S_k \setminus \{0, 1, \ldots, n_{k-1}\}$. Any member of this set lies in S_k and is larger than n_{k-1}, as required. The set is never empty. If it were, every member of S_k would lie in the finite set $\{0, 1, \ldots, n_{k-1}\}$, so S_k would be finite; but S_k is infinite by Step 1.

Solution to Exercise 17.3

(a) Let $x_0, x_1, \ldots$ be a Cauchy sequence in X. The Cauchy condition with tolerance $1/2$ supplies a threshold N such that $d(x_m, x_n) < 1/2$ for all $m, n \geq N$. Every distance in X is 0 or 1, so these distances are all 0, and $x_n = x_N$ for every $n \geq N$. The sequence converges to the point x_N of X: for every $\varepsilon > 0$ and every $n \geq N$, $d(x_n, x_N) = 0 < \varepsilon$. So every Cauchy sequence in X converges in X, and X is complete.

(b) Suppose X is finite, with members $c_1, \ldots, c_k$ (if X is empty, $k = 0$ and the empty list covers it). Let $r > 0$. Every point x of X is one of the c_i, and $d(x, x) = 0 < r$, so the k balls $B(c_1, r), \ldots, B(c_k, r)$ cover X. Hence X is totally bounded. Conversely, suppose X is totally bounded, and take $r = 1/2$. Finitely many balls $B(c_1, 1/2), \ldots, B(c_k, 1/2)$ cover X. Each of them contains only its center, because every other point is at distance 1 from it. So every point of X is one of $c_1, \ldots, c_k$, and X is finite.

(c) Suppose first that there is a point c with $x_n = c$ for infinitely many indices n, and let S be the set of these indices. As in Step 2 of the proof of Theorem 17.5, choose any $n_1 \in S$ and then, for each $k \geq 2$, an index $n_k \in S$ with $n_k > n_{k-1}$. This is always possible, because S is infinite and an infinite set of natural numbers contains a number larger than any given one. The subsequence $x_{n_1}, x_{n_2}, \ldots$ is the constant sequence $c, c, c, \ldots$, which converges to c, since every term is at distance $0 < \varepsilon$ from c for every $\varepsilon > 0$.

Conversely, suppose that a subsequence $x_{n_1}, x_{n_2}, \ldots$ converges to a point c of X. For the tolerance $1/2$ there is a threshold K such that $d(x_{n_k}, c) < 1/2$ for every $k \geq K$. Every distance in X is 0 or 1, so $x_{n_k} = c$ for every $k \geq K$. The indices $n_K < n_{K+1} < n_{K+2} < \cdots$ are infinitely many different natural numbers, and the sequence takes the value c at each of them.

Now take $X = \mathbb{N}$. By (a) and (b), it is complete but not totally bounded. The sequence $x_n = n$ takes each value at only one index, so by what we just proved, no subsequence of it converges, although X is complete. Completeness promises a limit only for a sequence that is already a Cauchy sequence; it says nothing about whether an arbitrary sequence has a subsequence of that kind. Producing such a subsequence is the job that total boundedness does in Theorem 17.5.

Solution to Exercise 17.4

Fix $r > 0$. Since $b - a > 0$, the Archimedean property gives a positive integer N with $N > (b-a)/r$. Multiplying by r/N shows that the spacing $h = (b - a)/N$ satisfies $0 < h < r$. Use the $N + 1$ centers

$$c_k = a + kh, \qquad k = 0, 1, \ldots, N.$$

The first center is $c_0 = a$, the last is $c_N = a + (b - a) = b$, and the others lie between them, so all of them are points of $[a, b]$, as the definition requires.

Let $x \in [a, b]$. If $x = b$, then x is the center c_N, and $d(x, c_N) = 0 < r$. Otherwise $a \le x < b$. Let k be the largest index with $c_k \le x$; there is one, because $c_0 = a \le x$. Then $k < N$, because $c_N = b > x$, and $x < c_{k+1}$. Hence

$$d(x, c_k) = x - c_k < c_{k+1} - c_k = h < r,$$

so x lies in the ball of radius r about c_k. Thus the $N + 1$ balls of radius r cover $[a, b]$.

The number $N + 1$ of balls, and the positions of the centers, depend on the radius r and on the interval, through its endpoints a and b. They do not depend on the point x: the list of centers was chosen as soon as r was given, before any point x was considered, and that one list covers every $x \in [a, b]$.

Solution to Exercise 17.5

The out-neighbors of v are u and w, so $N_1^+(v) = \{u, w\}$. The only route of length two from v is $v \to u \to w$, because w has no out-neighbors. So $R_2(v) = \{w\}$. But w is already a first out-neighbor of v, because of the edge $v \to w$, so it is excluded from the second out-neighborhood, and $N_2^+(v) = \varnothing$. The three sets have 2, 0 and 1 members.

In every directed graph, $N_2^+(v)$ is obtained from $R_2(v)$ by removing v and the first out-neighbors of v. So $N_2^+(v) \subseteq R_2(v)$, and $|R_2(v)| \ge |N_2^+(v)|$. A version of the conjecture that asked for a vertex with $|R_2(v)| \ge |N_1^+(v)|$ would therefore be easier to satisfy: every vertex that satisfies Seymour's inequality also satisfies the new one, but not conversely. To see that the truth of the inequality can really change, add a fourth vertex z and the edge $w \to z$. The graph is still oriented. Now $N_1^+(v) = \{u, w\}$, $R_2(v) = \{w, z\}$ and $N_2^+(v) = \{z\}$. At v the inequality $|R_2(v)| \ge |N_1^+(v)|$ holds, since $2 \ge 2$, while Seymour's inequality $|N_2^+(v)| \ge |N_1^+(v)|$ fails, since $1 < 2$.

So a formal statement that used $R_2(v)$ would state a weaker claim than Seymour's conjecture, and a proof of it would not prove the conjecture. In fact the weaker claim has a short proof, by an extremal choice in the style of Chapter 7. In a nonempty finite directed graph, choose a vertex v whose outdegree $\delta = |N_1^+(v)|$, the number of its out-neighbors, is as small as possible; such a vertex exists because the outdegrees form a nonempty finite set of natural numbers. If $\delta = 0$, then v has no out-neighbors and no routes of length two, so $|R_2(v)| = 0 \ge 0 = |N_1^+(v)|$. If $\delta \ge 1$, pick an out-neighbor u of v. Every out-neighbor z of u is reached by the route $v \to u \to z$, so $N_1^+(u) \subseteq R_2(v)$. The outdegree of u is at least the smallest outdegree δ, and therefore

$$|R_2(v)| \ge |N_1^+(u)| \ge \delta = |N_1^+(v)|.$$

A few lines settle the altered statement, while Seymour's conjecture stayed open for more than thirty years. In an oriented graph v itself never lies in $R_2(v)$, so all of the difficulty lies in excluding the first out-neighbors, which is what turns $R_2(v)$ into $N_2^+(v)$. The replacement changes what the statement means; it is not a harmless simplification of the notation.

Solution to Exercise 17.6

Each request should say what the reader knows, name one particular proof, and ask only for what that proof needs. For the contraction theorem, written by a reader who has met metric spaces but

not yet the rest of Chapter 11, a suitable request is: “I know metric spaces, the triangle inequality and proof by induction. I want to read the proof of the contraction theorem that starts from any point, applies the map again and again, shows that the iterates form a Cauchy sequence, and takes their limit. Teach me convergence of a sequence, Cauchy sequences, completeness and the bound for a sum of geometrically shrinking steps, only as far as this proof needs them. After each concept, show me the step of the proof that uses it.” For Theorem 17.5, written by a reader who knows all of Chapter 11, a suitable request is: “I know metric spaces, Cauchy sequences and completeness. I want to read the proof that every sequence in a complete, totally bounded metric space has a convergent subsequence by keeping infinitely many terms inside smaller and smaller balls. Teach me finite covers by small balls, the infinite pigeonhole principle and subsequences, only as far as this proof needs them. After each concept, show me the step of the proof that uses it.”

Both proofs use convergence, Cauchy sequences and completeness. Only the contraction proof needs fixed points, the contraction factor q and the geometric-sum bound of Proposition 11.9. Only the subsequence proof needs balls and total boundedness, the infinite pigeonhole principle, and subsequences. So each request asks for something that the other proof does not need.

If you have not run the formal check yourself, a reading record may report, on your own authority, the statements and definitions you have read and decoded, the names of the files and theorems you inspected, the roles of the dependencies you identified, and any proofs you reconstructed on your own. It may also report what the repository says about its formal check, attributed to the repository: “the repository reports that the check passes,” not “the check passes.” It may not claim that you compiled the project, that the check passed when you ran it, or that you have verified the imported proofs. Each of those is a separate claim that needs its own evidence (Section 17.8).

C.6 Week 18

Solution to Exercise 18.1

Apply the one-step estimate with $n = 0, 1, 2, 3$ in turn. Each time, substitute the bound already found, multiplied by q; since $q > 0$, multiplying keeps the direction of the inequality:

$$
\begin{aligned}
e_1 &\le q e_0 + \varepsilon_0, \\
e_2 &\le q e_1 + \varepsilon_1 \le q(q e_0 + \varepsilon_0) + \varepsilon_1 = q^2 e_0 + q\varepsilon_0 + \varepsilon_1, \\
e_3 &\le q e_2 + \varepsilon_2 \le q(q^2 e_0 + q\varepsilon_0 + \varepsilon_1) + \varepsilon_2 = q^3 e_0 + q^2\varepsilon_0 + q\varepsilon_1 + \varepsilon_2, \\
e_4 &\le q e_3 + \varepsilon_3 \le q(q^3 e_0 + q^2\varepsilon_0 + q\varepsilon_1 + \varepsilon_2) + \varepsilon_3 \\
&= q^4 e_0 + q^3\varepsilon_0 + q^2\varepsilon_1 + q\varepsilon_2 + \varepsilon_3.
\end{aligned}
$$

Part 1 of Theorem 18.1 with $n = 4$ reads $e_4 \le q^4 e_0 + \sum_{j=0}^{3} q^{3-j}\varepsilon_j$. Writing out the sum for $j = 0, 1, 2, 3$ gives $q^3\varepsilon_0 + q^2\varepsilon_1 + q\varepsilon_2 + q^0\varepsilon_3$, so the two bounds are exactly the same.

In the bound for e_{10}, the update error ε_6 carries the weight $q^{10-1-6} = q^3$. To see why, count updates: ε_6 is made in the update that produces y_7, and after it come the three updates that produce y_8, y_9 and y_{10}, each of which multiplies its effect by q.

Solution to Exercise 18.2

The formula. At $n = 0$ the formula gives $q^0 a + \eta(1 - q^0)/(1 - q) = a = y_0$. Suppose that $y_n = q^n a + \eta(1 - q^n)/(1 - q)$ for some n. Then

$$y_{n+1} = q\left(q^n a + \eta\,\frac{1 - q^n}{1 - q}\right) + \eta = q^{n+1} a + \eta\,\frac{q - q^{n+1} + 1 - q}{1 - q} = q^{n+1} a + \eta\,\frac{1 - q^{n+1}}{1 - q},$$

which is the formula for $n + 1$. By induction it holds for every $n \in \mathbb{N}$.

Equality in part 2. The map $T(x) = qx$ is a contraction of $\mathbb{R}$ with factor q, its fixed point is 0, and every update error is $|y_{n+1} - q y_n| = \eta$. So the example satisfies the hypotheses of part 2 of Theorem 18.1 with this η. Its starting error is $e_0 = |a - 0| = a$. Since $a \geq 0$, $\eta \geq 0$ and $0 < q^n \leq 1$, every y_n is nonnegative, so

$$e_n = y_n = q^n e_0 + \eta\,\frac{1 - q^n}{1 - q},$$

which is exactly the bound of part 2, for every $n \geq 1$.

Sharpness. Now let $a = 0$, so that $e_0 = 0$, let $\eta > 0$, and let K be any number smaller than $1/(1 - q)$. If the factor $1/(1 - q)$ in part 2 could be replaced by K, this example would satisfy $e_n \leq q^n e_0 + K\eta = K\eta$ for every $n \geq 1$, that is,

$$\eta\,\frac{1 - q^n}{1 - q} \leq K\eta.$$

Dividing by $\eta > 0$ and multiplying by $1 - q > 0$ turns this into $1 - q^n \leq K(1 - q)$, or $q^n \geq 1 - K(1 - q)$. Since $K(1 - q) < 1$, the number $c = 1 - K(1 - q)$ is positive. Because $q^n \to 0$, there is a threshold beyond which $q^n < c$, and for every such n the required inequality fails. So the smaller factor K fails for this example, at every step from some point on. No factor smaller than $1/(1 - q)$ works for every iteration that satisfies the hypotheses, which is what it means for the factor $1/(1 - q)$ to be sharp.

Solution to Exercise 18.3

The hypotheses. The real line with the distance $|x - y|$ is nonempty, and it is complete; the completeness of the real numbers is one of the starting facts of Chapter 11. The map $T(x) = 0$ sends every real number to the real number 0, so it maps $\mathbb{R}$ into itself. For every q with $0 < q < 1$ and all real x and y,

$$|T(x) - T(y)| = |0 - 0| = 0 \leq q|x - y|,$$

so T is a contraction with factor q. Its fixed point is 0, because $T(0) = 0$. Every y_n is a real number, so it lies in $X = \mathbb{R}$. Finally,

$$|y_{n+1} - T(y_n)| = |(-1)^{n+1}\eta - 0| = \eta,$$

so the update errors are $\varepsilon_n = \eta \geq 0$, and the hypothesis of part 2 holds with this same number η.

The bound of part 2. Every error is $e_n = |(-1)^n \eta - 0| = \eta$. For $n \geq 1$, the bound of part 2 is

$$q^n \eta + \eta\,\frac{1 - q^n}{1 - q} = q^n \eta + \eta\left(1 + q + \cdots + q^{n-1}\right).$$

The sum in parentheses contains the term 1, and all its other terms and the term $q^n\eta$ are nonnegative, so the bound is at least $\eta = e_n$. The bound holds, as the theorem promises.

No convergence. Consecutive terms are always 2η apart:

$$|y_{n+1} - y_n| = |(-1)^{n+1}\eta - (-1)^n\eta| = |-2(-1)^n\eta| = 2\eta.$$

Take the tolerance η, which is positive. For every proposed threshold N, the indices N and $N+1$ are both at least N, and $|y_{N+1} - y_N| = 2\eta \geq \eta$. So no threshold works, and the sequence is not a Cauchy sequence. By Proposition 11.5, every convergent sequence is a Cauchy sequence, so this sequence does not converge.

Why the bound does not settle convergence. The error bound controls one number at each step, the distance from y_n to x_*. Here that distance is η at every step, so the bound holds, and the distances even form a constant sequence. But two different points can lie at the same distance from x_*, here η and $-\eta$, and the points may keep jumping between them. Knowing the distances to x_* forces convergence only when those distances tend to zero, and with update errors that stay at η, the bound of part 2 does not tend to zero. That is why part 3 of the theorem needs the stronger hypothesis $\varepsilon_n \to 0$, which fails here.

Solution to Exercise 18.4

The bound. Every weight q^{n-1-j} in part 1 of Theorem 18.1 is positive, so we may replace each ε_j by the number aq^j, which is at least as large:

$$\begin{aligned}\sum_{j=0}^{n-1} q^{n-1-j}\varepsilon_j &\leq \sum_{j=0}^{n-1} aq^{n-1-j}q^j \\ &= \sum_{j=0}^{n-1} aq^{n-1} \\ &= anq^{n-1}.\end{aligned}$$

The exponent is $n-1-j+j = n-1$ in every term, and there are n terms, one for each j from 0 to $n-1$. Part 1 therefore gives $e_n \leq q^n e_0 + anq^{n-1}$ for every $n \geq 1$.

The factor n. We still have to show that nq^{n-1} tends to zero. It is not enough that $q^{n-1} \to 0$, because q^{n-1} is multiplied by the growing number n. Put $r = (1+q)/2$. Since $q < 1$, we have $q = \frac{q+q}{2} < \frac{1+q}{2} = r < \frac{1+1}{2} = 1$, so $q < r < 1$. Put $t = q/r$; then $0 < t < 1$, because $0 < q < r$. Since $0 < t < 1$, the powers $1, t, \ldots, t^{n-1}$ decrease, so each of these n numbers is at least the last one, t^{n-1}. Adding them up gives

$$nt^{n-1} \leq 1 + t + \cdots + t^{n-1} = \frac{1-t^n}{1-t} \leq \frac{1}{1-t}.$$

Multiply by the positive number r^{n-1} and use $q = rt$, so that $q^{n-1} = r^{n-1}t^{n-1}$:

$$0 \leq nq^{n-1} = r^{n-1}\,nt^{n-1} \leq \frac{r^{n-1}}{1-t}.$$

The number $1 - t$ is fixed and positive, and r^{n-1} becomes as small as we like, because $0 < r < 1$.

The whole bound tends to zero. Write $b_n = q^n e_0 + anq^{n-1}$ for the upper bound, and let $\delta > 0$. We find a threshold beyond which each of the two terms of b_n is less than $\delta/2$.

- If $e_0 = 0$, the first term is zero; put $N_1 = 1$. Otherwise, $q^n \to 0$ (Section 11.7) gives a threshold N_1 with $q^n < \delta/(2e_0)$, and so $q^n e_0 < \delta/2$, for every $n \geq N_1$.
- If $a = 0$, the second term is zero; put $N_2 = 1$. Otherwise, $r^m \to 0$ gives a threshold M with $r^m < (1-t)\delta/(2a)$ for every $m \geq M$. Put $N_2 = M + 1$. For every $n \geq N_2$ we have $n - 1 \geq M$, and so $anq^{n-1} \leq ar^{n-1}/(1-t) < \delta/2$.

For every n that is at least both N_1 and N_2, we get $0 \leq b_n < \delta/2 + \delta/2 = \delta$. So $b_n \to 0$.

Since $0 \leq e_n \leq b_n$, the same threshold works for the errors: $e_n < \delta$ for every n that is at least both N_1 and N_2. So $e_n \to 0$, and since $e_n = d(y_n, x_*)$, this says exactly that $y_n \to x_*$. This conclusion also follows from part 3 of the theorem, because $aq^j \to 0$. What the bound b_n adds is information about speed: it says how small the error is after a given number of steps.

Solution to Exercise 18.5

Several students may now receive the same task, as long as its capacity is not exceeded, so an assignment with capacities need not be a matching in the original graph. We translate the problem into one about ordinary matchings by giving every unit of capacity its own vertex, called a *slot*.

The condition is necessary. Suppose an assignment with capacities exists, and let S be any set of students. Every student in S receives a task that they accept, and that task belongs to $N(S)$. For each task $r \in N(S)$, let m_r be the number of students in S who receive r; then $m_r \leq c_r$. Every student in S is counted in exactly one of the numbers m_r, so

$$|S| = \sum_{r \in N(S)} m_r \leq \sum_{r \in N(S)} c_r.$$

The condition is sufficient: a new graph. Now suppose the inequality holds for every $S \subseteq L$. Build a new bipartite graph. Its students are those of L. In place of each task r it has c_r different slots $(r, 1), (r, 2), \ldots, (r, c_r)$; a task of capacity zero gets no slots at all. A student s is joined to the slot (r, i) exactly when s accepts the task r. For example, if task 1 has capacity 2 and task 2 has capacity 1, the slots are $(1, 1)$, $(1, 2)$ and $(2, 1)$, and a student who accepts task 1 is joined to both $(1, 1)$ and $(1, 2)$. The new graph is finite, because L and R are finite and each capacity is a nonnegative integer.

Hall's condition in the new graph. Let $S \subseteq L$. A slot (r, i) is joined to some student of S exactly when some student of S accepts r, that is, exactly when $r \in N(S)$. So the neighbors of S in the new graph are all the slots of all the tasks in $N(S)$. Slots of different tasks are different, because their first coordinates differ, and task r has exactly c_r slots. Hence S has exactly $\sum_{r \in N(S)} c_r$ neighbors in the new graph, and by assumption this number is at least $|S|$. This is Hall's condition (H) for the new graph.

Back to tasks. By Hall's theorem (Theorem 9.1), the new graph has a matching that saturates L: every student receives a slot they are joined to, and no slot goes to two students. Give each student

the task r of the slot (r, i) they received. The student accepts r, because they are joined to that slot. Task r is given to at most c_r students, because different students received different slots and r has only c_r slots. So this is an assignment with capacities.

When every capacity equals 1, each task has exactly one slot, the new graph is just the original graph, and the condition reads $|S| \le |N(S)|$. This is Hall's theorem again.

Solution to Exercise 18.6

Here is one possible record.

Question. The map $T(x) = (3x + 1)/4$ is a contraction of $\mathbb{R}$ with factor $q = 3/4$, and its fixed point is 1. If every update is computed with an error of at most $1/100$, do the points still converge to 1? If not, how far from 1 can they be?

Background. Arithmetic with fractions; induction (Chapter 5); distance, the triangle inequality, convergence, and the finite geometric sum (Chapter 11); the contraction theorem (Chapter 12).

Outline of the proof. (1) Insert the exact update $T(y_n)$ between y_{n+1} and the fixed point, and use the triangle inequality, the equation $T(1) = 1$ and the contraction inequality to get $e_{n+1} \le q e_n + \varepsilon_n$. (2) Unfold this by induction to $e_n \le q^n e_0 + \sum_{j=0}^{n-1} q^{n-1-j} \varepsilon_j$. (3) Bound every update error by η and add up the geometric weights, which gives $e_n \le q^n e_0 + \eta(1 - q^n)/(1 - q)$. Here this reads $|y_n - 1| \le (3/4)^n + \frac{1}{25}\bigl(1 - (3/4)^n\bigr)$.

Counterexample to the first guess. With the constant bias $1/100$, the points are $y_n = \frac{26}{25}\bigl(1 - (3/4)^n\bigr)$, a formula proved by induction in Section 18.5. They converge to $26/25$, not to the fixed point 1. So small update errors do not guarantee convergence to the fixed point.

An exact check. At $n = 1$ the iterate is $y_1 = \frac{3 \cdot 0 + 1}{4} + \frac{1}{100} = \frac{26}{100} = \frac{13}{50}$, in agreement with the closed form $\frac{26}{25}\bigl(1 - \frac{3}{4}\bigr) = \frac{26}{100}$. The true error is $\bigl|\frac{13}{50} - 1\bigr| = \frac{37}{50}$, and the certificate is $\frac{3}{4} + \frac{1}{25} \cdot \frac{1}{4} = \frac{3}{4} + \frac{1}{100} = \frac{19}{25} = \frac{38}{50}$. So $\frac{37}{50} \le \frac{38}{50}$, computed exactly. This check confirms the formulas at one index only. The closed form and the certificate hold for every n because they were proved.

What is new. The project reconstructs and studies a standard bound for contractions whose updates are computed with errors, together with exact examples. It does not establish a new theorem compared with the published literature.

A further question. Let $T(x) = qx + b$ on $\mathbb{R}$, with $0 < q < 1$, and suppose the update errors alternate in sign, so that $y_{n+1} - T(y_n)$ equals $\eta, -\eta, \eta, -\eta, \ldots$ in turn. After many steps, how far from the fixed point do the points end up? Is this distance a smaller multiple of η than the $\eta/(1 - q)$ allowed by part 2 of Theorem 18.1? For $q = 3/4$ and $\eta = 1/100$, the computation in Section 18.5 suggests that the points settle only $1/175$ away from 1, far less than the $1/25$ that the certificate allows. Since $1/175 = \eta/(1 + q)$ in that case, a natural guess is that the answer is $\eta/(1 + q)$ for every such map. The restriction to maps of the form $qx + b$ with $q > 0$ matters: Section 18.5 also shows that for $T(x) = -qx$ alternating errors do not cancel at all. The proofs in this chapter cannot answer the question, because the triangle inequality ignores the signs of the update errors and so cannot take advantage of their cancellation.

APPENDIX D

Running the examples on a computer

None of the proofs in this book needs a computer. Still, a computer can help in two ways: it can carry out a finite experiment exactly, and it can check a formal proof. This appendix shows how to do both for the examples in the book. Sections D.1 and D.2 explain the small part of the programming language Python that the experiments use, including exact arithmetic with fractions. Section D.3 then gives three complete programs: the count of sign sequences from Chapter 14, the assignment search of Chapter 8, and the biased iteration of Chapter 18. The last section explains how to check the Lean examples of Chapters 15 and 16.

D.1 The smallest amount of Python needed

The programs in this appendix are written in Python, a free programming language that runs on all common operating systems; any version from Python 3.10 on will do. The programs use only Python's *standard library*, the collection of tools that comes with every Python installation, so nothing else has to be installed. This section explains just enough of the language to read the programs and run them.

Running a program

A Python program is a plain text file whose name ends in `.py`, for example `signs.py`. Write it in a plain text editor; a word processor adds formatting that Python cannot read. To run the program, open a *terminal*, a window for typing commands to the computer. Move to the folder that contains the file by typing `cd` followed by the location of that folder; for example, `cd Documents/math` moves into the folder `math` inside the folder `Documents`. Then type

```
python3 signs.py
```

On some systems the command is `python` instead of `python3`. The command `python3 --version` shows which version of Python is installed.

Short calculations can also be typed directly into Python. Type `python3` in the terminal with no file name. Python then displays its *prompt*, usually >>>, the short sign that a program shows when it is waiting for you to type. Python carries out each line as soon as you press Enter; typing `2 + 2` there displays `4`. Type `exit()` to return to the terminal. The terminal has a prompt of its own, and the two must be kept apart. The terminal's prompt expects commands that start programs, such as `python3 signs.py`, while Python's prompt expects lines of Python. A line typed at the wrong prompt produces an error message that has nothing to do with the mathematics you wanted to test. The characters >>> belong to the prompt, so never type them yourself or copy them into a program file.

Names, numbers and lists

The line `x = 5` stores the value 5 under the name `x`. Later lines can use `x`, and a later line `x = x + 1` replaces the stored value by 6. So the sign = in Python is an instruction to store a value. It is not a claim that two things are equal; read as an equation, $x = x + 1$ would be false for every number. To *test* whether two values are equal, Python uses a double sign ==. The test `2 + 2 == 4` has the value `True`, and `2 + 2 == 5` has the value `False`. The other comparisons are written <, <=, >, >=, and `!=` for "is not equal to."

Arithmetic uses +, -, * for multiplication, and / for division. Two stars denote a power, so `2 ** 10` is $2^{10} = 1024$. The function `abs` gives the absolute value: `abs(-3)` is 3. The sign `%` gives the remainder of Section 5.5: `17 % 5` is 17 mod 5 = 2, and `-7 % 5` is 3, because $-7 = (-2) \cdot 5 + 3$. As in that section, the remainder on division by a positive number is never negative. The command `print` displays values on the screen, separated by spaces: `print(x, x + 1)` displays `6 7` when `x` holds 6.

A *list* is an ordered collection of values, written in square brackets. In `a = [1, 1, 1, -1]`, the first entry is `a[0]`, not `a[1]`: Python numbers the positions of a list from zero, just as Chapter 14 numbers the entries of a sign sequence. So `a[3]` is the last entry, −1. The *length* `len(a)`, the number of entries, is 4. The programs below use two operations that change a list. The line `a.append(9)` adds the entry 9 at the end of `a`, and `a.pop()` removes the last entry of `a` and hands it back as its value.

Loops

A `for` loop repeats a block of lines once for each value in a range. The block consists of the indented lines below the line that begins with `for`; Python uses this indentation, not brackets, to tell where the block ends. For example:

```
a = [1, 1, 1, -1]
value = 0
for j in range(3):
    value = value + a[j] * a[j + 1]
print(value)
```

Here `range(3)` produces the values 0, 1, 2: it starts at zero and stops just before 3. So the indented line runs three times, with $j = 0$, $j = 1$ and $j = 2$, and each time it adds one product to the running total stored in `value`. The products are $a_0a_1 = 1$, $a_1a_2 = 1$ and $a_2a_3 = -1$, so `value` goes from 0 to 1, then 2, then 1, and the program prints 1. This is the aperiodic correlation A_1 of $(1, 1, 1, -1)$ computed in Section 14.1.

With two numbers, `range(1, 14)` produces $1, 2, \ldots, 13$: it starts at the first number and again stops just before the second. The values of a loop can also come from a list: a block that begins with `for x in [2, 3, 5]:` is carried out three times, with `x` equal to 2, then 3, then 5.

Functions

A *function* packages a calculation so that it can be used again with different inputs.

```
def correlation(a, shift):
    value = 0
    for j in range(len(a) - shift):
        value = value + a[j] * a[j + shift]
    return value
```

The line beginning with `def` gives the function a name, `correlation`, and names its two inputs, `a` and `shift`. The indented lines below it carry out the calculation, and the line beginning with `return` gives the result. For a list `a` of length n and a shift s, the loop runs over `range(len(a) - shift)`, which here is `range(n - s)` and stops just before $n - s$. So j takes the values $0, 1, \ldots, n - 1 - s$, exactly the summation range in the definition of A_s (Definition 14.1). For example, `correlation([1, 1, 1, -1], 1)` is 1 and `correlation([1, 1, 1, -1], 3)` is -1, the values of A_1 and A_3 found in Section 14.1. The result is the correlation A_s only when `a` is a sign sequence and $1 \le s \le n - 1$, the shifts allowed in the definition. The function does not check this. Given a list of n signs and the shift 0, for instance, it simply returns n, although the definition leaves shift zero out. Section D.3 uses the same calculation, written more compactly, under the name `aperiodic`, next to a separate function `periodic` for the periodic correlation of Chapter 14.

Decisions and other loops

A line `if` *condition*`:` is followed by an indented block, which is carried out only when the condition is true. Conditions can be combined with the words `and`, `or` and `not`, which have the meanings of Section 2.2. A `while` loop repeats its block as long as its condition is true. After the following lines, for example, `n` holds 8, the smallest natural number whose square is at least 50:

```
n = 0
while n * n < 50:
    n = n + 1
```

Inside a loop, `break` leaves the innermost loop containing it at once, and `continue` skips the rest of the block and goes on with the next pass. The special value `None` means "no value here." The test `x is None` asks whether `x` holds this special value, and `x is not None` asks the opposite.

Tuples, sets and dictionaries

A *tuple* is like a list but is written with round brackets, as in `(1, -1, 1)`, and cannot be changed after it has been made. Its entries are numbered from zero, as in a list, so a tuple is a natural way to store a sign sequence $(a_0, a_1, \ldots, a_{n-1})$. A function that has to give back several values at once can return them as a tuple. A piece of text is written between quotation marks, as in `"shortage"`. When such text is displayed as an entry of a tuple, Python puts it between single quotes, as in `('shortage', 1)`.

A *set* records membership only, as in Section 3.1: order and repetition do not matter. The set containing 1 and 2 is written `{1, 2}`, and `{1, 2, 2}` is the same set. The test `2 in s` is `True` when 2 is a member of the set `s`, and `not in` tests the opposite. The union $s \cup t$ is written `s | t`, so `{1, 2} | {2, 3}` is `{1, 2, 3}`. The empty set is written `set()`, because empty braces `{}` mean something else in Python: an empty dictionary, described below. A loop `for x in s:` runs through the members of a set `s`, but since a set has no order, Python does not promise any particular order of visits. When the order matters, loop over `sorted(s)` instead. This is a list of the members of `s` in increasing order; for example, `sorted({3, 1, 2})` is `[1, 2, 3]`.

A *dictionary* is a table that pairs each *key* with a value. After `owner = {}` creates an empty table, the line `owner[3] = 0` records the pair "key 3, value 0," and afterwards `owner[3]` looks the value up. The test `3 in owner` asks whether the key 3 has an entry. Asking for `owner[5]` when the key 5 has no entry stops the program with an error, whereas `owner.get(5)` simply gives `None`. Python prints the table as `{3: 0}`, listing each key, a colon, and its value. The official Python tutorial explains lists, tuples, sets and dictionaries in more detail [16].

Adding up and checking many cases at once

Python can write a sum much as mathematics does. The `for` loop that computed A_1 for `a = [1, 1, 1, -1]` can be replaced by the single expression

```
sum(a[j] * a[j + 1] for j in range(3))
```

which reads like $\sum_{j=0}^{2} a_j a_{j+1}$ and has the same value, 1. In the same way, `all(`*test* `for s in range(1, n))` is `True` when the test holds for every s with $1 \le s \le n - 1$, and `False` as soon as it fails for one of them. For example, with the function `correlation` above, `all(abs(correlation(a, s)) <= 1 for s in range(1, 4))` is `True` for `a = [1, 1, 1, -1]`, and `False` for `a = [1, 1, -1, -1]`, whose correlation at shift two is $A_2 = -2$ (Section 14.1). When the range contains no values at all, as `range(1, 1)` does, the result is `True`: a statement about every member of an empty collection is vacuously true (Section 2.5). Finally, `sum(1 for x in xs if` *test*`)` adds 1 for each member `x` of the list `xs` that passes the test, so it counts those members. For instance, `sum(1 for x in [3, 8, 5, 10] if x > 4)` is 3, because exactly three of the four numbers exceed 4.

Two more conventions appear in the programs below. Python ignores everything on a line after the sign `#`, so this sign introduces a comment for human readers. And a line `from fractions import Fraction` makes one tool from the standard library available to the program, here the tool `Fraction` for exact rational arithmetic, which Section D.2 describes.

D.2 An exact fraction is not a rounded decimal

When a program contains a number such as `0.1`, Python does not store one tenth exactly. It stores a *floating-point number*: a binary approximation that is correct to about sixteen significant digits. The approximation is very good, but it is not exact, and a tiny error can change the answer to a yes-or-no question. The two lines

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

display `0.30000000000000004` and `False`. In an experiment that tests an equation or an inequality, such a rounding error could make a true statement look false, or a false one look true.

When the numbers involved are rational, the standard library offers exact arithmetic instead [17]. A `Fraction` stores an integer numerator and a positive integer denominator, reduced to lowest terms, and it adds, subtracts, multiplies and divides by the usual rules for fractions, with no rounding at any stage. For example, `Fraction(1, 3)` is the fraction $1/3$. A fraction with denominator 1 can be given by its numerator alone, so `Fraction(0)` is $0/1$, that is, zero. Python's integers are exact as well, however large they become.

```
from fractions import Fraction
print(Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10))
x = Fraction(1, 3)
y = (x * x + 1) / 3
print(y)
```

This program displays `True` and then `10/27`. The last three lines use the map $T(x) = (x^2 + 1)/3$ of Section 12.5: the name `x` holds the iterate $x_1 = 1/3$, so `y` is $T(x_1) = x_2$, and Python finds exactly the value $10/27$ computed by hand in that section. (Typed at Python's prompt without `print`, the same value is shown in the form `Fraction(10, 27)`.)

Two details deserve attention. First, dividing one ordinary integer by another with `/` gives a floating-point number, so `1 / 3` is `0.3333333333333333`. When a `Fraction` is combined with an integer or with another `Fraction`, however, the result is again a `Fraction`. That is why the line `y = (x * x + 1) / 3` can use the plain integers 1 and 3 and still be exact. Combining a `Fraction` with a floating-point number such as `0.5`, on the other hand, gives a floating-point number, so the two kinds of number should not be mixed. Second, the exact rational number one tenth must be written `Fraction(1, 10)`. Writing `Fraction(0.1)` instead converts the floating-point number `0.1` into a fraction exactly as it is stored, and `print(Fraction(0.1))` displays

```
3602879701896397/36028797018963968
```

The denominator is 2^{55}, and this fraction is close to one tenth but not equal to it.

The experiments of Section D.3 keep every comparison exact. The sign-sequence counts use only integers, and the iteration uses fractions; decimals appear only when a result is displayed. Exact arithmetic removes one source of doubt, but not every source. It cannot tell you whether a loop runs over exactly the objects you intended, or whether the program computes the quantity that the text defines. To answer those questions, read the code against the definitions, as Section D.3 does for each program.

D.3 Three experiments

Each of the following three programs is complete: save it in a file of its own and run it as described in Section D.1.

Counting sign sequences

The following program produces the table of Section 14.6.

```
from itertools import product

def aperiodic(a, s):
    n = len(a)
    return sum(a[j] * a[j + s] for j in range(n - s))

def periodic(a, s):
    n = len(a)
    return sum(a[j] * a[(j + s) % n] for j in range(n))

def is_barker(a):
    return all(abs(aperiodic(a, s)) <= 1 for s in range(1, len(a)))

def is_circulant_row(a):
    return all(periodic(a, s) == 0 for s in range(1, len(a)))

for n in range(1, 14):
    seqs = list(product([1, -1], repeat=n))
    barker = sum(1 for a in seqs if is_barker(a))
    circ = sum(1 for a in seqs if is_circulant_row(a))
    print(n, barker, circ)
```

The first line makes the tool `product`, from the part of the standard library called `itertools`, available to the program. Called as `product([1, -1], repeat=n)`, it fills n positions, each with an entry of the list `[1, -1]`, in every possible way; the input `repeat=n` says how many positions there are. So it produces all 2^n sign sequences of length n, one at a time, each as a tuple; for $n = 2$ these are `(1, 1)`, `(1, -1)`, `(-1, 1)` and `(-1, -1)`. The line `seqs = list(...)` collects them into one list.

Before trusting the output, read each function against the definition it is meant to carry out.

- The function `aperiodic` is the function `correlation` of Section D.1, written more compactly with `sum`. As explained there, `range(n - s)` runs j through $0, 1, \ldots, n-1-s$, the summation range of A_s in Definition 14.1.
- In `periodic`, the index j runs through all of $0, 1, \ldots, n-1$, and `(j + s) % n` is the position $(j + s)$ mod n of Definition 14.3. This is where the wrap-around happens.
- The test `is_barker(a)` is `True` exactly when $|A_s| \le 1$ for every shift s with $1 \le s \le n-1$,

because `range(1, len(a))` produces exactly these shifts. This is Definition 14.2. For $n = 1$ there are no such shifts, so `all(...)` is vacuously `True`, and both sequences of length one count as Barker sequences, just as in Chapter 14.

- In the same way, `is_circulant_row(a)` is `True` exactly when $C_s = 0$ for every shift s with $1 \le s \le n - 1$. By Section 14.3, these are exactly the sequences that form the first row of a circulant Hadamard matrix, which explains the name of the function.

For each length n from 1 to 13, the main loop counts the Barker sequences of length n, storing the count in `barker`, and the sequences with $C_s = 0$ for $1 \le s \le n - 1$, storing that count in `circ`. It then prints one line with the three numbers n, `barker` and `circ`. The thirteen lines agree with the thirteen rows of the table. The first line is `1 2 2`, because both tests are vacuously true at length one. The fourth line is `4 8 8`: of the 16 sign sequences of length four, eight are Barker sequences, and eight have all three periodic correlations C_1, C_2, C_3 equal to zero.

To go further, raise the upper limit `14` in the line `for n in range(1, 14):`. Each additional length doubles the number of sequences, and each sequence takes a little longer to check, so the running time slightly more than doubles with every length added. The memory needed doubles as well, because the list `seqs` holds all 2^n sequences of one length at the same time. With the limit `21`, the program covers all lengths up to 20, where there are 2^{20} sequences, slightly more than a million, and it finishes well within a minute on an ordinary computer. At length 25 the list alone has more than 33 million entries and needs several gigabytes of memory.

An assignment search that returns a certificate

The next function carries out the procedure of Section 8.4. Students are numbered $0, 1, 2, \ldots$, and `options` is a list whose entry `options[s]` is the set $N(s)$ of tasks acceptable to student `s`.

```
def assign(options):
    # options[s] is the set of tasks acceptable to student s.
    # Returns ("assignment", task_of) or ("shortage", A, N(A)).
    owner = {}     # task -> student who currently holds it
    task_of = {}   # student -> task that student currently holds
    for root in range(len(options)):
        came_from = {}        # task -> student from which it was reached
        reached = [root]      # students reached by the search
        to_explore = [root]   # reached students not yet inspected
        free_task = None
        while to_explore and free_task is None:
            student = to_explore.pop()
            for task in sorted(options[student]):
                if task in came_from or task_of.get(student) == task:
                    continue
                came_from[task] = student
                if task not in owner:
                    free_task = task
                    break
                reached.append(owner[task])
```

```
                to_explore.append(owner[task])
        if free_task is None:     # the search failed
            needed = set()        # will become N(A)
            for s in reached:
                needed = needed | options[s]
            return ("shortage", reached, needed)
        task = free_task          # flip the augmenting path back to the root
        while task is not None:
            student = came_from[task]
            old_task = task_of.get(student)
            owner[task] = student
            task_of[student] = task
            task = old_task
    return ("assignment", task_of)

print(assign([{1, 2}, {1}, {2, 3}]))
print(assign([{1, 2}, {1}, {2}]))
```

Here is how the function matches the procedure of Chapter 8.

The matching. Two dictionaries describe the current matching M: `owner[t]` is the student who holds task `t`, and `task_of[s]` is the task that student `s` holds. A task without an entry in `owner` is unused, and a student without an entry in `task_of` is unmatched. Both dictionaries start empty, so M starts as the empty matching. The students then take turns as the root of a search, in the order $0, 1, 2, \ldots$. When a student's turn comes, it is still unmatched. Indeed, a flip changes the task of students who already hold one, and the only student it gives a first task is the root of the search. So a student receives its first task on its own turn, and not before.

The search. The dictionary `came_from` records each reached task together with its predecessor, the student from which it was reached. The list `reached` holds the reached students, and `to_explore` holds those that have been reached but not yet inspected. A list used as a condition counts as true when it is not empty, so the `while` loop runs as long as some reached student is waiting to be inspected and no unused task has been found. Each pass takes a student from `to_explore` and inspects it, as in Section 8.4. Thanks to `sorted`, the student's acceptable tasks are taken in increasing order, as in the trace of Section 8.6. The `continue` line skips a task that has already been reached, and also the task that the student holds, since the edge to that task is matched.

Every task that is not skipped is marked as reached, with the current student as its predecessor. If the task is unused, the search has succeeded: `break` ends the inspection, and the `while` loop stops too, because `free_task` is no longer `None`. If the task is used, its owner is marked as reached and waits to be inspected. By the argument of Section 8.4, this owner has not been reached before, so no student is reached twice. The program does not need to record the predecessor of such a student separately: it is the task the student holds, which `task_of` already records. Because `pop()` always takes the most recently reached of the students still waiting, the students are not inspected in the order of breadth-first search. As Section 8.4 points out, the order does not matter, as long as the search continues until it succeeds or every reached student has been inspected.

The flip. If an unused task was found, the last loop flips M along the augmenting path, walking

backward from the unused task to the root. At each step, `student` is the predecessor of the current task. This student gives up its old task, saved as `old_task`, and takes the current task. The walk then continues from `old_task`, which is the task from which this student was reached. In the next pass, `owner` records the new holder of that task, namely its predecessor, so no out-of-date entry is left behind. When the walk arrives at the root, `old_task` is `None`, because the root held no task, and the loop stops. The two dictionaries now describe the matching obtained by flipping M along the augmenting path, which by Lemma 8.5 has one more edge than M.

Here is the flip in the first test at the end of the program. When student 1 becomes the root, student 0 holds task 1. The search ends with `free_task` equal to 2 and with `came_from` equal to `{1: 1, 2: 0}`: task 1 was reached from student 1, and task 2 from student 0. The first pass of the flip gives task 2 to its predecessor, student 0, whose old task is 1. The second pass gives task 1 to its predecessor, student 1, whose old task is `None`, so the walk stops. Now student 0 holds task 2 and student 1 holds task 1.

A failed search. The `while` loop stops only when an unused task has been found or when `to_explore` is empty, that is, when every reached student has been inspected. So the search is complete in the sense of Section 8.4. Suppose that it ends without finding an unused task, and let A be the set of reached students. The three lines that build `needed` collect every task accepted by some student of A, so `needed` is the set $N(A)$. By Proposition 8.6, $N(A)$ has exactly one member fewer than A, so A is a shortage set. The function returns A and $N(A)$ at once, without trying the remaining students, because a shortage set already proves that no matching gives every student a task (Section 8.5).

The two test lines use the examples of Section 8.6, with the students a, b, c numbered $0, 1, 2$ and the tasks keeping their names $1, 2, 3$. The word in quotation marks at the start of each answer says which kind of answer it is. The program displays

```
('assignment', {0: 2, 1: 1, 2: 3})
('shortage', [2, 0, 1], {1, 2})
```

In the first line, the dictionary `{0: 2, 1: 1, 2: 3}` pairs student 0, that is a, with task 2, then b with task 1, and c with task 3. This is the same assignment as in Section 8.6, reached by a different route, because the program takes the roots in the order a, b, c rather than a, c, b. The search from a finds the unused task 1 at once, and a takes it. The search from b reaches task 1, its owner a, and then the unused task 2; the flip moves a to task 2 and gives task 1 to b. The search from c first reaches task 2, now held by a, and then the unused task 3, so the flip simply gives task 3 to c.

In the second test, task 3 has been removed from c's options. The searches from a and b run exactly as before and give the matching $\{a\text{–}2,\ b\text{–}1\}$. The search from c reaches task 2 and its owner a, then a's other task 1 and its owner b. Student b accepts no further task, so the search fails. The list `[2, 0, 1]` contains the reached students c, a, b, in the order in which the search reached them, and `{1, 2}` is the set of tasks they accept: three students, but only two tasks. This is the shortage set found in Section 8.6, although the program reached it from a different matching.

Neither kind of answer has to be taken on trust, because each is a certificate in the sense of Section 8.5. To check an assignment, make sure that every student has a task, that each student's

task is one the student accepts, and that no task appears twice. To check a shortage, list the tasks that the reported students accept, and count them: there must be fewer tasks than students.

The biased iteration with exact fractions

The last program checks the numerical experiment of Section 18.5. It computes the iterates of the biased update

$$y_{n+1} = \frac{3y_n + 1}{4} + \frac{1}{100}, \qquad y_0 = 0,$$

with exact fractions. For each index n from 0 to 20, it checks the closed form $y_n = \frac{26}{25}\bigl(1 - (\frac{3}{4})^n\bigr)$ and the error certificate of Section 18.5, the bound of part 2 of Theorem 18.1, which here reads

$$|y_n - 1| \le \left(\frac{3}{4}\right)^n e_0 + \eta\,\frac{1 - (3/4)^n}{1 - 3/4}, \qquad e_0 = |y_0 - 1| = 1, \quad \eta = \frac{1}{100}.$$

```
from fractions import Fraction

q = Fraction(3, 4)        # contraction factor of T(x) = (3x + 1)/4
eta = Fraction(1, 100)    # the bias added at every update
y = Fraction(0)           # the starting point y_0
e0 = abs(y - 1)           # initial error; the fixed point of T is 1
for n in range(21):
    closed = Fraction(26, 25) * (1 - q**n)
    assert y == closed                  # closed form, checked exactly
    error = abs(y - 1)
    bound = q**n * e0 + eta * (1 - q**n) / (1 - q)
    assert error <= bound               # bound, checked exactly
    print(n, round(float(error), 4), round(float(bound), 4))
    y = (3 * y + 1) / 4 + eta           # the biased update gives y_{n+1}
```

A line `assert` *condition* does nothing when the condition is true, and stops the program with an error message when it is false. So if the program runs to the end and displays all 21 lines, the closed form and the bound have both been confirmed, in exact arithmetic, for $n = 0, 1, \ldots, 20$. At $n = 0$ the inequality reads $1 \le 1$, which needs no theorem; part 2 of Theorem 18.1 covers every $n \ge 1$.

The function `float` converts a fraction to the nearest floating-point number, and `round(...,4)` rounds the result to four decimal places. Both happen only inside the `print` line, after the exact checks, so rounding affects what is displayed and nothing else. The displayed bounds are therefore approximations, and some of them are rounded down. A bound that is to be used as a guarantee must be rounded up instead, as Section 12.5 explained. Here is part of the output:

```
0 1.0 1.0
1 0.74 0.76
2 0.545 0.58
...
10 0.0186 0.0941
11 0.0039 0.0805
12 0.0071 0.0704
...
20 0.0367 0.043
```

Python leaves out trailing zeros when it displays a number, so `0.58` stands for 0.5800 and `1.0` for 1.0000. The row for $n = 1$ shows the decimal forms of the exact values $37/50$ and $19/25$ found by hand in the solution to Exercise 18.6. The middle column shows what Chapter 18 predicts: the true error falls until $n = 11$ and then rises again toward $1/25 = 0.04$, while the bound in the last column decreases at every step toward the same limit $1/25$. Remember that a finite run confirms the formulas only at the indices it reaches; the proofs in Chapter 18 cover every index.

D.4 Checking the Lean examples

Install Lean by following the official instructions [9]. As Section 15.1 explains, they set up the editor VS Code together with its Lean extension.

The examples are written for Lean version 4.19.0. Typing `lean --version` in a terminal shows which version you have. Lean changes from one version to the next, so if an example that you copied exactly is rejected, check the version first. The installation includes a small program called *elan*, which manages the versions of Lean and lets each folder choose its own. To use version 4.19.0, choose a folder for your Lean work and create in it a plain text file named `lean-toolchain`, whose only line is

```
leanprover/lean4:v4.19.0
```

From then on, whenever Lean is started in that folder, elan downloads version 4.19.0 if it is not yet installed and uses it.

Then create, in the same folder, a file named `proofs.lean` with the same shape as in Section 15.1:

```
import Std

namespace MathCourse

-- the definitions and theorems go here

end MathCourse
```

Everything from `--` to the end of a line is a comment, which Lean ignores. Between the two lines that contain `MathCourse`, copy the Lean *declarations*, that is, the displayed blocks of Lean code that introduce a definition with `def` or a theorem with `theorem`, from Chapters 15 and 16

and from the solutions for Weeks 15 and 16 in Appendix C. Take them in the order in which they appear: first those of Chapter 15, then those of Chapter 16, and finally those of the solutions. The order matters because Lean reads the file from top to bottom and accepts a name only after it has been defined. For example, the definition of `Inj` in Section 15.5 must come before `inj_comp` and every later theorem that uses `Inj`. Following the order of the book guarantees this.

Leave out the other displays. Some of them show what Lean prints, such as a proof state or an error message, and a few repeat some lines of a longer proof. The `#print axioms` commands of Section 16.6 are discussed below. The one-line example `theorem bad` in the text of that section does not belong in the file either, since it uses `sorry` on purpose.

Running the check

Open the file in VS Code. Lean checks it while you type, and when the cursor is inside a proof, the infoview beside the file shows the proof state at that point. To check the whole file at once from a terminal, move to the folder that contains it, as in Section D.1, and type

```
lean proofs.lean
```

If the command prints nothing at all, every declaration was accepted, with no error and no warning. An *error* means that the declaration containing it was rejected. A *warning* means that the declaration was accepted but something in it deserves a second look. With all the declarations in the file, the output is not quite empty: every declaration is accepted, but Lean prints two warnings, each followed by a note on how such warnings can be switched off. The first of them looks like this:

```
proofs.lean:98:5: warning: unused variable `hf`
note: this linter can be disabled with `set_option linter.unusedVariables false`
```

Every message begins with the name of the file, the line and the column where the problem is, and then says whether it is an error or a warning. Lean counts the columns of a line from 0, so column 5 is the sixth character of the line. Here that is where the name `hf` begins, after four spaces of indentation and an opening bracket. The line number in your file may differ from 98, depending on how many blank lines you leave between declarations. The second warning says the same about `hg`. Both concern the theorem `surj_comp_assumed`, and Section 16.5 explains why that theorem never needs these two hypotheses.

Asking which axioms a proof uses

To list the axioms on which a theorem depends (Section 16.6), add a line such as

```
#print axioms MathCourse.surj_comp
```

after the line `end MathCourse`. The command uses the full name, with the prefix `MathCourse`, because it stands outside the namespace. Lean answers

```
'MathCourse.surj_comp' does not depend on any axioms
```

and from then on the command `lean proofs.lean` prints this line as well. The theorems `inj_comp` and `preimage_meet` give the same answer. Not every theorem in the book does: `not_surj_succ` and `succ_surj_int` from Section 16.7 depend on the standard axiom `propext`, through the library lemmas they use, and as that section explains, this is no cause for concern.

The command `#check`, which Section 16.7 uses to display the exact statement of a library lemma, can be added to the file in the same way, on a line of its own outside every proof. For example, the line `#check Nat.succ_ne_zero` makes Lean print the statement of that lemma, and `lean proofs.lean` prints it too.

What an unfinished proof looks like

To see how Lean reacts to a gap, temporarily replace the last line of the proof of `surj_comp`, which is `rw [ha, hb]`, by the single word `sorry`, keeping the same indentation. Lean still accepts the file, but it prints the warning `declaration uses 'sorry'`, pointing at the name of the theorem, and the `#print axioms` line above now reports

```
'MathCourse.surj_comp' depends on axioms: [sorryAx]
```

Put the original line back before you rely on the file again: a file that contains `sorry` has not proved what it states.

Where to go next

For further practice, *Mathematics in Lean* is an interactive textbook with many exercises [15]. It is built on Mathlib, the large mathematical library that the examples of this book do without, and its own instructions explain how to set it up. It is a natural next step once the statements, goals and proof states of Chapters 15 and 16 feel familiar.

APPENDIX E

A compact guide to notation and mathematical judgment

This appendix gathers in one place two things that the book develops gradually: its notation, and the mathematical judgment that guides how its proofs are read and found. Section E.1 is a table of the main symbols, each with the place where it is explained. Sections E.2–E.5 collect the ways of reading and reasoning that run through the chapters: what the main structures of the book allow and what its main proofs actually use, how to choose the next move in a proof, how to turn a reading difficulty into a precise question, and how to keep a record of a proof you are studying. Section E.6 suggests a rubric for the capstone investigation, and Section E.7 describes what the course aims to leave you able to do.

E.1 A notation map, with places to return to

The table below lists the main symbols of the book, roughly in the order in which they appear, together with the place where each one is explained. A one-line summary can remind you what a symbol means, but to use the symbol with confidence again, reread the section where it was introduced, together with its examples.

A few symbols, and many letters, do more than one job. The surrounding sentence, and the kind of object involved, decide which job is meant.

- Vertical bars: for a real number x, $|x|$ is its absolute value; for a finite set S, $|S|$ is the number of its members. Double bars, as in $\|x\|_2$, give the length of a vector.
- Round brackets: $(1, 3)$ can be the interval of real numbers strictly between 1 and 3, or the ordered pair with first entry 1 and second entry 3. Square brackets also have two uses: $[1, 3]$ is an interval, while $[x]$ is the equivalence class of x.
- The arrow $\to$: in $f : A \to B$ it names the input and output sets of a function; in $u \to v$ it is a directed edge of a graph; in $x_n \to x_*$ it says that a sequence converges. The arrow with a bar, as in $x \mapsto x^2$, names the rule of a function. The double arrow $\Rightarrow$ is different again: it joins two statements.
- The symbol f^{-1} has two uses. When f has an inverse, f^{-1} names that inverse function, and then $f^{-1}(y)$ is a single input (Section 3.2); it is not the reciprocal $1/f(y)$. Applied to a set T of outputs, $f^{-1}(T)$ is the preimage of T, a set of inputs, and it makes sense for every function f, whether or not f has an inverse. When f does have an inverse, the two readings agree: the preimage of T is the image of T under the inverse function (Section 3.4).
- Letters are reused from chapter to chapter. For example, q is a quotient in Chapter 5 and a

contraction factor in Chapters 12 and 18. The capital A is a set in Chapter 3, a matrix in Chapter 10 and an event in Chapter 13, while A_s is an autocorrelation in Chapter 14. Similarly, I is an identity matrix in Chapter 10, but I_A is the indicator of an event in Chapter 13; and e_j is a standard basis vector in Chapter 10, but e_n is the error at step n in Chapter 18.

Symbol	Meaning, and where it is explained
$\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$	The natural numbers $0, 1, 2, \ldots$ (zero included), the integers, the rational numbers, and the real numbers (Section 1.8).
$\sum_{j=a}^{b} x_j$	The sum $x_a + x_{a+1} + \cdots + x_b$, with one term for each integer j from a to b (Section 1.8). When $b < a$ there are no terms, and this *empty sum* is 0 (Section 5.1). A sum such as $\sum_{v \in V} \deg(v)$ has one term for each member v of the finite set V, added in any order (Section 7.2).
$x < y, x \le y$	x is strictly smaller than y; x is smaller than or equal to y. The two differ only when $x = y$ (Section 1.8).
$\sqrt{t}$	For a real number $t \ge 0$, the nonnegative number whose square is t. So $\sqrt{9} = 3$, not -3 (Section 1.8).
$P \Rightarrow Q$, $P \Leftrightarrow Q$	"If P, then Q"; "P if and only if Q," which means both $P \Rightarrow Q$ and $Q \Rightarrow P$. Writing either one states a claim; it does not say that the claim has been proved (Section 2.1).
$\neg, \wedge, \vee$	Not; and; or. The "or" is inclusive: $P \vee Q$ is true when at least one of P and Q is true, including when both are (Section 2.2).
$\forall, \exists$	For every; there exists (at least one). Quantifiers are read from left to right, and a value chosen at an $\exists$ may depend on every value chosen before it, but not on anything chosen after it. So changing the order can change the meaning (Section 2.3).
ε, δ	Traditional names for tolerances, that is, positive amounts of error we are prepared to allow. They are names, not fixed numbers: "for every $\varepsilon > 0$" covers $\varepsilon = 1/10$, $\varepsilon = 1/1000$, and every other positive value (Sections 2.3 and 11.1). In Chapter 18, ε_n is instead an update error; see the row for y_n at the end of the table.
$\lvert x \rvert$	The absolute value of a real number x: its distance from 0 on the number line (Section 2.6).
$[a, b]$, (a, b), $[0, \infty)$	Intervals of real numbers. A square bracket includes its endpoint and a round bracket excludes it; $[0, \infty)$ is the set of all real x with $x \ge 0$ (Section 2.6).
$x \in A, x \notin A$	x is a member of the set A; x is not a member of A (Section 3.1).
$A \subseteq B, \varnothing$	Every member of A is also a member of B; the empty set, which has no members (Section 3.1).
$\{x \in A : P(x)\}$	The set of members of A that have the property P. Read the colon as "such that" (Section 3.1).

Symbol	Meaning, and where it is explained
$A \cup B$, $A \cap B$	Union: the objects in at least one of A and B. Intersection: the objects in both (Section 3.1).
$A \setminus B$	The members of A that are not in B. The result is a set, not a number: it removes members rather than subtracting values (Section 3.1).
$A \times B$	The set of ordered pairs (a, b) with $a \in A$ and $b \in B$ (Section 3.1).
$\lvert S \rvert$	The number of members of a finite set S, each counted once; $\lvert\varnothing\rvert = 0$ (Section 3.1).
$f : A \to B$, $x \mapsto x^2$	A function with domain A and codomain B; the function that sends each input x to x^2 (Section 3.2).
$g \circ f$, id_A, f^{-1}	"g after f": apply f first, then g. The identity function on A, which sends every element to itself. The inverse of $f : A \to B$, when it exists: the function $f^{-1} : B \to A$ with $f^{-1}(f(x)) = x$ for every $x \in A$ and $f(f^{-1}(y)) = y$ for every $y \in B$ (Section 3.2).
$f(S)$, $f^{-1}(T)$	The image of a set S of inputs: all outputs $f(x)$ with $x \in S$. The preimage of a set T of outputs: all inputs x with $f(x) \in T$ (Section 3.4).
$d \mid a$, $\gcd(a, b)$	The positive integer d divides the integer a, that is, $a = dk$ for some integer k. The greatest common divisor of two nonnegative integers a and b that are not both zero: the largest positive integer that divides both (Section 5.5).
$a \bmod m$	The remainder when the integer a is divided by the positive integer m; it is always one of the numbers $0, 1, \ldots, m-1$ (Section 5.5).
$x \sim y$, $[x]$	x is related to y; the equivalence class of x, the set of all elements related to x (Section 6.1).
$a \equiv b \pmod m$	a is congruent to b modulo m: the positive integer m divides $a - b$ (Section 6.2).
$[a]_m$, $\mathbb{Z}/m\mathbb{Z}$	The residue class of a: the whole set of integers congruent to a modulo m, not just the one integer a. The set of all m residue classes (Section 6.2).
$x \preceq y$	x is below y in a partial order, a relation that is reflexive, antisymmetric and transitive (Section 6.6).
$\deg(v)$	The degree of a vertex v: the number of its neighbors (Section 7.1).
$u \to v$	A directed edge from u to v; v is then an out-neighbor of u (Section 7.5).
$N_1^+(v)$, $N_2^+(v)$	The first out-neighborhood of v: the vertices at directed distance exactly one from v. The second out-neighborhood: the vertices at directed distance exactly two (Section 7.7).
$N(s)$, $N(S)$	The tasks accepted by the student s; the tasks accepted by at least one student of the set S (Sections 8.1 and 8.5).
$S \subsetneq L$	S is a proper subset of L: $S \subseteq L$ and $S \neq L$ (Section 9.4).

Symbol	Meaning, and where it is explained
$\mathbb{R}^n$	The set of vectors $(x_1, \ldots, x_n)$ with n real coordinates (Section 10.1).
A_{ij}, Ax	The entry of the matrix A in row i and column j; the vector obtained by applying A to x (Section 10.2).
$\ker T$	The kernel of a linear map T: the inputs that T sends to the zero vector (Section 10.3).
$u \cdot v$	The dot product: multiply corresponding coordinates and add. The result is a number, not a vector (Section 10.4).
$\|x\|_2$	The Euclidean length $\sqrt{x \cdot x}$ of a vector. The subscript 2 is part of the name, not a coordinate (Section 10.4).
A^T, I	The transpose of A, obtained by exchanging rows and columns. The identity matrix, with ones on the diagonal and zeros elsewhere (Section 10.6).
(X, d), $d(x, y)$	A metric space: a set X together with a chosen metric d, a rule that gives every pair of points a distance. The number $d(x, y)$ is the distance from x to y. For all points x, y, z, a metric must satisfy three requirements: $d(x, y) \geq 0$, with $d(x, y) = 0$ exactly when $x = y$; $d(x, y) = d(y, x)$; and the triangle inequality $d(x, z) \leq d(x, y) + d(y, z)$. The choice of metric is part of the problem (Section 11.2).
d_2, d_∞	Two metrics on $\mathbb{R}^n$: the Euclidean distance $\|x - y\|_2$, and the largest of the coordinate differences $\|x_j - y_j\|$ (Section 11.2).
$x_n \to x_*$	The sequence converges to x_*: for every tolerance $\varepsilon > 0$, all terms from some index on are at distance less than ε from x_* (Section 11.3).
x_*	A name for a special point: a limit in Chapter 11, a fixed point in Chapters 12 and 18. The star is part of the name (Section 11.3).
Tx	Short for $T(x)$ (Section 12.1).
Ω, $p(\omega)$	The sample space, a finite set of outcomes; the probability of the outcome ω. The numbers $p(\omega)$ are nonnegative and add up to 1 (Section 13.1).
$\mathbb{P}(A)$	The probability of an event $A \subseteq \Omega$: the sum of $p(\omega)$ over the outcomes $\omega \in A$ (Section 13.1).
$\mathbb{E}\,X$	The expectation of a random variable X, that is, of a function $X : \Omega \to \mathbb{R}$: the weighted average $\sum_{\omega \in \Omega} p(\omega) X(\omega)$ (Section 13.2).
I_A	The indicator of the event A: it equals 1 at outcomes in A and 0 at all other outcomes (Section 13.3).

Symbol	Meaning, and where it is explained
A_s, C_s	For a sign sequence $(a_0, \ldots, a_{n-1})$, whose entries are all 1 or -1: the aperiodic autocorrelation $A_s = \sum_{j=0}^{n-1-s} a_j a_{j+s}$ at shift s, defined for $1 \le s \le n-1$, in which nothing wraps around; and the periodic autocorrelation $C_s = \sum_{j=0}^{n-1} a_j a_{(j+s) \bmod n}$ at shift s, defined for $0 \le s \le n-1$, in which positions wrap around modulo n (Sections 14.1 and 14.2).
`a : T`, `f x`, `:=`	In Lean: `a` has type `T` (for a proposition `P`, `h : P` says that `h` is a proof of `P`); the function `f` applied to `x`; "is defined as" or "is proved by" (Section 15.2).
`sorry`	In Lean, a placeholder for a missing proof. The command `#print axioms` shows whether a theorem depends on one: if it does, the list includes the axiom `sorryAx` (Section 16.6).
$B(c, r)$	The ball with center c and radius $r > 0$ in a metric space X: the points of X whose distance from c is strictly less than r (Section 17.2).
y_n, ε_n, e_n	In Chapter 18: the points of an iteration in which each update may contain an error, so that y_{n+1} is only close to $T(y_n)$; the update error ε_n, a bound $d(y_{n+1}, T(y_n)) \le \varepsilon_n$ on the error made in the update that produces y_{n+1}; and the error $e_n = d(y_n, x_*)$, the distance from y_n to the fixed point (Sections 18.1 and 18.3).

E.2 Structures, what proofs use, and changes of description

Almost every object in this book is a set with something extra attached, which Section 6.4 calls a *structure*. The set says which objects a question is about (Chapter 3). A *relation* says which pairs of objects are linked or compared (Section 6.1); "a is congruent to b modulo m" and "u is adjacent to v" are examples. An *operation*, such as addition, combines two elements of a set into one element of the same set (Section 6.4). A function, also called a map, sends each object of one set to an object of another (Section 3.2), and so lets us carry a question from one description to another.

The extra structure decides which questions can even be asked.

- Addition and scaling let us form linear combinations of vectors (Chapter 10).
- The order of the natural numbers lets us pick out a smallest or a largest object. Every nonempty set of natural numbers has a smallest member (the well-ordering property of Sections 4.2 and 5.2), and every nonempty finite set of natural numbers also has a largest member. Chapter 7 uses these facts to choose a shortest walk between two vertices that are joined by some walk, and a longest directed path in a nonempty finite directed graph.
- A distance lets us say that one point approximates another, and so lets us speak of limits (Chapter 11).
- A probability on a finite set of outcomes lets us take weighted averages (Chapter 13).

In a set with no distance, for instance, the phrase "x_n gets close to x_*" has no meaning.

Habit 3 (Section 3.4) asks which information a proof actually uses. For the main theorems of the book the answers are short.

- The proofs of Hall's theorem (Chapter 9) use only which students accept which tasks, and how many members various finite sets have. They never use what the students and the tasks actually are. That is why the theorem applies to every finite bipartite graph, whatever its vertices stand for.
- The proof of the contraction theorem (Theorem 12.2) works entirely with distances. Besides the facts that the space is nonempty and that T maps it into itself, it uses the contraction inequality, completeness, and the three requirements of a metric, above all the triangle inequality. It never adds two points or multiplies a point by a number. That is why it works in metric spaces that have no addition at all.
- The proof of the Cauchy–Schwarz inequality (Theorem 10.2) uses only three properties of the dot product: linearity in each argument, symmetry, and positivity. It never looks at individual coordinates. So the same inequality, with the same proof, holds for any other way of multiplying two vectors to get a number, as long as this product has the same three properties and the length of a vector u is defined in the same way, as the square root of the product of u with itself. For example, on $\mathbb{R}^2$ the product that sends u and v to $2u_1v_1 + u_2v_2$ is symmetric and linear in each argument. It is also positive: the product of u with itself is $2u_1^2 + u_2^2$, which is never negative and is zero only when $u = 0$. So $|2u_1v_1 + u_2v_2| \le \sqrt{2u_1^2 + u_2^2}\sqrt{2v_1^2 + v_2^2}$ for all $u, v \in \mathbb{R}^2$, with no new work.
- Chapter 14 first translates a condition on a sign sequence, that all its periodic correlations at nonzero shifts are zero, into a condition on a matrix: different rows of its circulant matrix are orthogonal (Proposition 14.5). Two arguments then restrict the order of such a matrix, that is, its number of rows. Adding up all the correlations shows that the order of the matrix is a perfect square (Section 14.4), and this argument relies on the circulant pattern. Comparing three rows shows that the order is divisible by four when it is greater than 2 (Lemma 14.6). This second argument uses only two facts, that the entries are ± 1 and that different rows are orthogonal, and these two facts are exactly what makes a square matrix a Hadamard matrix (Section 14.3). So it holds for every Hadamard matrix, circulant or not.

Noticing what a proof does *not* use tells you where its conclusion might reach further. It is a guide for guessing, not a proof: every proposed extension still has to be checked against the argument, line by line.

Many steps in the book change the description of a problem without changing the problem itself, and Habit 6 (Section 6.6) asks what such a change preserves. A new description helps only when we know exactly what it keeps and what it throws away. Here are four changes of description from the book.

- *An inverse loses nothing.* If f has an inverse h, then every input can be recovered from its output, as $x = h(f(x))$. In particular, $f(x) = f(y)$ gives $x = h(f(x)) = h(f(y)) = y$, so different inputs never share an output, and a question about inputs can be answered by looking

at outputs (Section 3.3).

- *A quotient forgets on purpose.* Passing from an integer a to its residue class $[a]_m$ forgets everything about a except its remainder on division by m. This is safe only for rules that give the same answer for every representative of a class, which is why well-definedness has to be checked (Section 6.3).
- *A matrix turns a linear map into a table of numbers.* Column j of the matrix records where the map sends the standard basis vector e_j (Section 10.2), and composing two linear maps corresponds to multiplying their matrices (Section 10.6).
- *An indicator turns an event into a quantity.* The indicator of an event is the random variable that equals 1 when the event occurs and 0 when it does not. Counting how many events occur then becomes adding numbers, and linearity of expectation applies (Section 13.3).

These constructions are quite different, but each one gives a language in which the step we need is easier to state and to justify.

E.3 Choosing the next move

Mathematics can look like a long list of tricks, each of which has to be memorized. In practice it helps far more to recognize the kind of situation you are in. Four situations come up again and again in this book, and each one suggests a natural next move.

When a general claim looks doubtful

Look for a counterexample before you try to prove the claim. Start with small cases and boundary cases: zero, the empty set, the smallest allowed input, two objects that happen to be equal. A useful counterexample is rarely found by trying cases at random. It is built to break the step that the claim seems to need. Two examples from the book show how.

- Proposition 3.3 shows that $g \circ f$ can be injective although g is not. In the counterexample, f maps $\{0, 1\}$ into $\{0, 1, 2\}$ and never reaches 2, while g sends 2 to the same output as 1. The composition only ever applies g to outputs of f, so it never meets this collision.
- In Section 18.2, the guess that small update errors still give convergence to the fixed point fails for a constant bias. The map $x \mapsto qx$ on $\mathbb{R}$, with $0 < q < 1$, has the fixed point 0. Adding the same small amount $\eta > 0$ at every update moves the limit from 0 to $\eta/(1 - q)$, however small η is.

In both cases the counterexample did more than refute a claim. It showed which feature a correct statement has to take into account: the outputs that f actually reaches, and errors that never die out.

When you need to show that something exists

A direct construction is often out of reach. Ask instead what a failure would force. The book turns that question into a proof in three different ways.

- *Search and repair.* The search of Chapter 8 looks for an augmenting path. If it finds one, the matching can be enlarged. If it fails, the failed search produces a group of students who together accept too few tasks (Proposition 8.6), and then no matching can give every student a task. Either outcome tells us something, which is the point of Habit 7 (Section 8.5).
- *An extremal choice.* Choose an object that is as large or as small as possible, and show that it must already have the property you want. A longest directed path in a nonempty finite acyclic oriented graph ends at a sink, because an arrow leaving its last vertex would give either a longer path or a directed cycle (Theorem 7.8). The same idea also proves statements about every natural number. Suppose that such a statement fails for some number, and choose the smallest number at which it fails. Being smallest gives it an extra property, that the statement holds at every smaller number, and this often leads to a contradiction (Section 5.2).
- *An average.* A random variable X on a finite probability space cannot be smaller than its expectation $\mathbb{E}\,X$ at every outcome of positive probability. Outcomes of probability zero contribute nothing to $\mathbb{E}\,X = \sum_{\omega\in\Omega} p(\omega)X(\omega)$, and a weighted average of numbers that are all smaller than $\mathbb{E}\,X$ would itself be smaller than $\mathbb{E}\,X$. So some outcome ω of positive probability has $X(\omega) \geq \mathbb{E}\,X$ (Proposition 13.5). Theorem 13.8 uses this to show that every graph with m edges has a cut of size at least $m/2$. Labeling the vertices at random gives a cut whose expected size is $m/2$, so some labeling gives a cut at least that large, although the proof never says which one. Habit 8 (Section 13.6) describes this approach.

Which approach fits depends on the problem. Can a partial solution be repaired one step at a time? Is there a natural quantity to make as large or as small as possible? Is there an average that you can compute exactly?

When the full answer is out of reach

Ask what you really need to know. Often it is much less than the full answer.

- To know how close a point z is to the fixed point of a contraction, you do not need to find the fixed point. The residual estimate of Theorem 12.2 bounds the error by $d(z, Tz)/(1-q)$, and Tz is usually easy to compute. In Section 12.5, this certified that $x_2 = 10/27$ lies within $19/729$ of the fixed point of $T(x) = (x^2+1)/3$ on $[0,1]$ before the fixed point had been found.
- To show that no circulant Hadamard matrix of order 12 exists, you do not need the complete answer claimed by the research described in Section 14.7. The necessary condition of Proposition 14.7 already rules it out: an order greater than 1 must have the form $4u^2$ for a positive integer u. The numbers of this form are $4, 16, 36, \ldots$, growing with u, and 12 lies strictly between the first two, $4 \cdot 1^2 = 4$ and $4 \cdot 2^2 = 16$. So 12 is not of this form.

Settling for less than the full answer has a risk: you may end up answering an easier question than the one you were asked. If you replace the original question by an easier one, say so and keep a record of the change, as Habit 2 (Section 2.6) asks. For example, Proposition 14.7 settles order 12 completely. As an answer to the larger question "which orders are possible?", however, it is only partial. Besides 1 and 4, which do occur, it still allows the orders $16, 36, 64, \ldots$, and it says nothing about whether any of them actually occurs.

When a bound appears

Ask whether the constant in the bound could be lowered and, if not, which example prevents any improvement. Such an example usually attains the bound, or comes as close to it as we like, so it also pays to ask when the bound is attained (Section 9.6).

- The Cauchy–Schwarz inequality $|u \cdot v| \le \|u\|_2 \, \|v\|_2$ holds with equality exactly when $v = 0$ or u is a multiple of v (Section 10.5). Because equality occurs for nonzero vectors, for example when $u = v \neq 0$, the constant factor 1 on the right cannot be replaced by anything smaller.
- In the constant-bias example of Section 18.4, the error bound of Theorem 18.1, part 2, holds with equality at every step. With a bias $\eta > 0$, these exact errors $\eta(1 - q^n)/(1 - q)$ approach $\eta/(1 - q)$ as n grows. So the factor $1/(1 - q)$ is sharp. Any smaller factor would give a bound below the true error once n is large enough, so no cleverer proof can lower it.

E.4 Turning a reading difficulty into a precise question

Suppose you understand what a theorem says but cannot follow one line of its proof. Asking an assistant, or a classmate, to explain the whole proof again rarely helps: the new explanation may skip the same step, or replace it by a different argument that you then have to check from the beginning. It works better to turn the difficulty into a precise question, as Habit 4 (Section 4.5) suggests. Write down three things:

- the exact line that you do not follow;
- what is known at that point of the proof: the hypotheses, and the facts already established;
- the last step that you do understand.

Then ask which definition or earlier fact leads from what is known to the line. A good answer is usually short: one calculation written out in more detail, or one small example. It is rarely a new technique.

Here are two questions of this kind, one from each half of the course.

- In Case 1 of the inductive proof of Hall's theorem (Section 9.4), S is a nonempty proper group of students with $|N(S)| = |S|$, and T is any group of the remaining students. A precise question about one line of that case is: "I know that the task sets $N(S)$ and $N(T)$ can overlap. Why is $|N(S) \cup N(T)| = |N(S)| + |N(T) \setminus N(S)|$?" The answer is a splitting into two sets with no common member. Every task in the union lies either in $N(S)$ or in $N(T) \setminus N(S)$, never in both, and the sizes of two disjoint finite sets add (Section 3.1). Nothing more about graphs is needed. Exercise 9.3 asks you to write this out in full.
- From the proof of Theorem 18.1 (Section 18.3): "I see that each update error is later multiplied by q. Why is the weight of ε_j in the bound for e_n exactly q^{n-1-j}?" The answer is a count. The error ε_j is made in the update that produces y_{j+1}. After it come the updates that produce $y_{j+2}, \ldots, y_n$, and there are $n - (j + 1) = n - 1 - j$ of them, each contributing one factor q.

Once you have the answer, go back to the proof and write the step in your own words. Then

change one small feature and work out what changes.

In the Hall example, first recall how the proof reaches its conclusion. It takes three moves. First, the students of $S \cup T$ together accept exactly the tasks in $N(S) \cup N(T)$, so Hall's condition for this group gives $|N(S) \cup N(T)| \geq |S \cup T|$, and $|S \cup T| = |S| + |T|$ because the two groups have no student in common. Second, the identity from the question, together with $|N(S)| = |S|$, turns the left side into $|S| + |N(T) \setminus N(S)|$. Third, subtracting $|S|$ from both sides leaves $|N(T) \setminus N(S)| \geq |T|$. Now suppose that T were allowed to share students with S. The identity still holds, but the equation $|S \cup T| = |S| + |T|$ does not, because a student who belongs to both groups is counted once on the left and twice on the right. So the conclusion no longer follows, and it can even be false: for $T = S$ it would read $0 \geq |S|$, which fails because S is nonempty.

In the second example, what if the update errors were numbered from 1 instead of 0, so that ε_j bounds the error in the update that produces y_j? Then $n - j$ updates follow it, and its weight becomes q^{n-j}. And what if $q = 1$, so that the map never increases distances but need not shrink them? If the map still has a fixed point x_*, the one-step estimate and the counting work as before, but every weight is now 1, so the bound becomes $e_0 + \varepsilon_0 + \cdots + \varepsilon_{n-1}$. When every update error equals the same number $\eta > 0$, this is $e_0 + n\eta$, which grows without limit. The error really can grow this fast. Every point is a fixed point of the identity map $T(x) = x$ on $\mathbb{R}$; take $x_* = 0$, start at $y_0 = 0$, and add η at every update, so that $y_{n+1} = y_n + \eta$. Then $e_n = y_n = n\eta$, exactly the bound. If you can answer variations like these, the explanation has become your own: you can reason with it, and not only repeat the words in which it was given.

The same approach works for a definition that you do not understand. Ask, in this order: What kind of object does the definition apply to? What exactly must such an object satisfy? What is one example, and what is a *near miss*, an object that fails only one of the requirements? For the definition of an odd integer (Definition 1.1), the objects are integers, and an integer a is odd when $a = 2k + 1$ for some integer k. The number $7 = 2 \cdot 3 + 1$ is an example. The number 8 is a near miss: $8 = 2 \cdot 3.5 + 1$ has exactly the right shape, but 3.5, the only number k with $2k + 1 = 8$, is not an integer. So the requirement that fails is precisely "k is an integer". Finally, when the definition appears in a proof, decide in which of the two directions of Section 1.4 it is used. Does the proof *unpack* the definition for an object it is given, or must it *recognize* that a new object satisfies the definition? The first proof of the book, Proposition 1.2 in Section 1.6, does both: it unpacks the fact that a and b are odd, and it recognizes that $a + b$ meets the definition of even.

E.5 What a finished reading record contains

Section 17.8 introduced the *reading record*: a written account of a proof you are studying, saying what you have read, what you have understood, and what remains open. While you work, the record grows piece by piece. A finished record contains at least the following.

- The exact statement of the result, copied from a named source. Note which version you read, by its date or version number, because a preprint or a repository can change.
- The same statement in your own words, in plain language.
- The objects that the statement is about, and the hypotheses on them.

- A map of the proof's dependencies. It can be a table like the prerequisite map of Section 17.7, a diagram, or a paragraph.
- One delicate step of the proof, written out in full detail.

Some entries need a note beside them:

- for a result you use without having read its proof: its hypotheses, and why they hold in your situation;
- for a computer calculation: the range of cases it covered, and whether it used exact arithmetic or rounded decimals (Section D.2);
- for a result checked by Lean: exactly which statement Lean checked, which axioms its proof depends on (Section 16.6), and whether you ran the check yourself or are reporting what a source says about it (Section 17.8).

In addition, give every entry two labels, as in the section "Keeping track of evidence" of *To the reader*. The first says how well you understand the entry: *understood*, *can follow*, *borrowed*, or still *open*. Section 17.7 explains these labels, and Section 17.8 uses them for a research proof. The second label says what supports the entry: proved, cited from another source, checked by computer for finitely many cases, or not established. An entry that rests on one of the four starting facts listed in *To the reader* can say so, since those facts are recorded as assumptions.

The record ends with your own explanation of the proof and with one variation that you have worked out yourself, such as a hypothesis removed or a conclusion strengthened, together with your reasons for what happens in the new case. A saved conversation with an assistant cannot take the place of this record. A conversation mixes guesses, corrections and results, and it does not mark which statements survived checking. The record should let another reader rebuild the final argument without having to read the whole conversation.

E.6 A suggested capstone rubric

The table below suggests one way to assess the independent capstone of Section 18.7. Each row names one quality of a finished project, gives its share of the grade, and describes what a strong project shows. The row "Understanding" assesses the short oral discussion that ends the investigation. The other rows follow that section's list of what the write-up should contain.

Quality	Weight	What a strong project shows
Claim and scope	20%	The exact objects, hypotheses and conclusion of each result. If the question changed during the project, both the original and the revised question are stated, and the difference between them is explained. Each claim is marked with the evidence for it: proved, cited from another source, checked by computer for finitely many cases, or not established.
Argument	35%	A complete proof or a counterexample, with a map of the background that the argument needs. The delicate steps are written out in full, and each step names the results it depends on.
Understanding	20%	In the oral discussion, the student explains a delicate step without help, and, when an assumption or an example is changed, correctly predicts what still holds and what fails.
Reproducibility	15%	Computer checks where they are relevant, with the program, the range of cases it covers and the software used, so that anyone can repeat them. If no computation is relevant, the write-up explains how a reader can check the claims without a computer, for example by working small cases by hand.
Context	10%	Accurate credit to every source used; a modest statement of what is new compared with the published literature and what is new only to the student; an open account of how an AI assistant was used and of what remains unsettled.

Three faults cannot be outweighed by good presentation: a statement that has been changed without saying so, a missing step on which the argument depends, and a claim of novelty without support. A project with any of these faults needs to be corrected before it can pass. The rubric rewards careful work on a small investigation of the student's own; nobody is expected to solve a famous problem.

E.7 What you can do at the end of the course

You do not need to remember every displayed formula in this book. What matters is what you can do. Faced with a statement, you can say exactly what it requires. Faced with a proof you do not yet follow, you can find the place where your understanding stops, and ask for the one concept or step that would carry you past it. When an explanation arrives, from a book, a classmate or an assistant, you can test it on an example, see what role it plays in the whole proof, and say clearly what has not yet been established.

The next unfamiliar theorem you meet will still contain mathematics you have not seen. The difference is that you now have a way in: identify the structures involved, lay out which steps depend on which, learn the pieces you are missing, and return to the argument. Working with an AI assistant is worthwhile when what it produces helps you understand and gives you something you can check. It is not worthwhile when it merely fills an empty page with text that looks like

mathematics.

References and further reading

[1] E. Lehman, F. T. Leighton, and A. R. Meyer, *Mathematics for Computer Science*. The textbook for MIT course 6.1200J, available as a free PDF from MIT OpenCourseWare. A good further source on logic, induction, graphs, matchings, and probability. `https://ocw.mit.edu/courses/6-1200j-mathematics-for-computer-science-spring-2024/pages/readings/`

[2] OpenAI, *math*: a public repository of mathematical preprints, with Lean formalizations and descriptions of what each formalization covers. `https://github.com/openai/math`

[3] OpenAI, "Seymour's second-neighborhood conjecture." Description of the scope of the Lean formalization, with a link to the accompanying preprint, *A proof of Seymour's second-neighborhood conjecture* (September 2026). `https://raw.githubusercontent.com/openai/math/main/lean/docs/173.md`

[4] OpenAI, "The circulant Hadamard and Barker-sequence conjectures." Description of the scope of the Lean formalization, with a link to the accompanying preprint, *The circulant Hadamard conjecture* (September 2026). `https://raw.githubusercontent.com/openai/math/main/lean/docs/179.md`

[5] OpenAI, `Main.lean`: the Lean file containing the proof of the second-neighborhood theorem `exists_goodVertex`. `https://raw.githubusercontent.com/openai/math/main/lean/OAI/Combinatorics/SecondNeighborhood/Main.lean`

[6] OpenAI, `SeymourSecondNeighborhood.lean`: the Lean file containing the official statement of the second-neighborhood theorem, against which the proof is checked. `https://raw.githubusercontent.com/openai/math/main/lean/ComparatorChallenges/SeymourSecondNeighborhood.lean`

[7] OpenAI, `Reduction.lean`: the Lean file containing the reduction step of the second-neighborhood proof. `https://raw.githubusercontent.com/openai/math/main/lean/OAI/Combinatorics/SecondNeighborhood/Reduction.lean`

[8] OpenAI, configuration file for the formal check of the second-neighborhood theorem, listing the statement file, the proof file, the theorem name, and the permitted axioms. `https://raw.githubusercontent.com/openai/math/main/lean/ComparatorChallenges/SeymourSecondNeighborhood.json`

[9] Lean, official installation instructions. `https://lean-lang.org/install/`

[10] J. Avigad, L. de Moura, S. Kong, and S. Ullrich, *Theorem Proving in Lean 4*, chapter "Propositions and Proofs." `https://lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/`

[11] J. Avigad, L. de Moura, S. Kong, and S. Ullrich, *Theorem Proving in Lean 4*, chapter "Quantifiers and Equality." https://lean-lang.org/theorem_proving_in_lean4/Quantifiers-and-Equality/

[12] J. Avigad, L. de Moura, S. Kong, and S. Ullrich, *Theorem Proving in Lean 4*, chapter "Tactics." https://lean-lang.org/theorem_proving_in_lean4/Tactics/

[13] *The Lean Language Reference*, section "Axioms." https://lean-lang.org/doc/reference/latest/Axioms/

[14] *The Lean Language Reference*, section "Validating a Lean Proof." https://lean-lang.org/doc/reference/latest/ValidatingProofs/

[15] J. Avigad and P. Massot, with contributors, *Mathematics in Lean*. An interactive textbook with exercises, built on the Mathlib library. https://leanprover-community.github.io/mathematics_in_lean/

[16] Python Software Foundation, *The Python Tutorial*, section "Data Structures." https://docs.python.org/3/tutorial/datastructures.html

[17] Python Software Foundation, *The Python Standard Library*, section "fractions—Rational numbers." https://docs.python.org/3/library/fractions.html

All web addresses were accessed in October 2026. The pages and repository files they point to may change after that date.